

AEGIS

The Aegis Architect and Operator Handbook

You have been given a deployed Aegis and the SDK that drives it. This book is how you work with both.

Aegis is Integrum's commercial implementation of four open standards — CARE, PACT, EATP and CO — published by the Terrene Foundation under CC BY 4.0. Aegis implements those standards; it does not own them.

Generated 2026-10-08

Contents

The Aegis Architect and Operator Handbook

Part 01 — Orientation

- 01.1 — What you were given
- 01.2 — The mental model
- 01.3 — Your first working session

Part 02 — Working through the harness

- 02.1 — The two surfaces
- 02.2 — Standing up an organisation
- 02.3 — Envelopes, clearance and knowledge
- 02.4 — Trust chains and postures
- 02.5 — Objectives and the work loop
- 02.6 — Approvals, holds and evidence
- 02.7 — Role agents, and surviving a re-org

Part 03 — Extending the platform

- 03.1 — Writing a tool an agent can call
- 03.2 — Agents, skills and pipelines
- 03.3 — The tool-agent lifecycle, definition to retirement
- 03.4 — Who may call your tool: applications, grants and policy
- 03.5 — Testing an extension before it touches a real organisation
- 03.6 — Choosing between a tool, a skill, an agent and a pipeline

Part 04 — The API surface

- 04.1 — Calling the API
- 04.2 — Every failure this client can raise
- 04.3 — Credentials, and what a key is not
- 04.4 — Lists, filters and pagination
- 04.5 — Concurrency, timeouts and streams
- 04.6 — Reading a response honestly

Part 05 — The web console

- 05.1 — First login
- 05.2 — Submitting an objective
- 05.3 — Your inbox
- 05.4 — When something is held
- 05.5 — What you can see, and why
- 05.6 — The rest of your workspace

Part 06 — Architecture

- 06.1 — What core and the SDK are
- 06.2 — The request path
- 06.3 — The governed organisation
- 06.4 — Trust and the audit spine
- 06.5 — The execution model
- 06.6 — Designing against the platform

Part 07 — Deployment architecture

- 07.1 — What a deployment is
- 07.2 — The cluster and its namespaces
- 07.3 — Images and the registry
- 07.4 — The network boundary
- 07.5 — Stateful services
- 07.6 — Identity and secrets
- 07.7 — Operating the deployment

Part 08 — The capability catalogue

- 08.1 — How to read this catalogue
- 08.2 — Organisation and identity
- 08.3 — Agents and execution
- 08.4 — Work and objectives
- 08.5 — Governance and trust
- 08.6 — Knowledge and data
- 08.7 — Evidence and operations
- 08.8 — Commercial and extensibility

Part 09 — The governance architecture

- 09.1 — The two planes
- 09.2 — Envelopes and constraints
- 09.3 — Clearance and classification
- 09.4 — The decision gradient
- 09.5 — Trust chains and postures
- 09.6 — The audit spine and evidence
- 09.7 — Containment and explainable refusal

Part 10 — The organisation structure framework

- 10.1 — The organisation is the system
- 10.2 — The D/T/R grammar
- 10.3 — Addressing
- 10.4 — Roles, authority and intent

- 10.5 — Trust chains and delegation
- 10.6 — Role agents
- 10.7 — Re-orgs, bridges and workspaces

Part 11 — The deployment pack

- 11.1 — What is in the pack
- 11.2 — Provisioning the substrate
- 11.3 — Configuration reference
- 11.4 — The runtime and its nodes
- 11.5 — Data stores and migrations
- 11.6 — Releasing and rolling back
- 11.7 — Operating day to day
- 11.8 — Verifying the deployment

The Aegis Architect and Operator Handbook

You have been given a deployed Aegis and the SDK that drives it. This book is how you work with both.

Aegis is Integrum's commercial implementation of four open standards — **CARE**, **PACT**, **EATP** and **CO** — published by the **Terrene Foundation** under CC BY 4.0. Aegis implements those standards; it does not own them.

Who this is for, and what it assumes

You are an **architect**, an **operator**, or an **engineer designing against the platform**. The first two design the governed organisation, stand it up, give agents their bounds, and keep the result running and evidenced. The third is building something that talks to Aegis — an integration, a service, a product surface — and needs to know how the platform is put together before choosing where to attach. Parts 01 to 05 are written for the first two; part 06 is written for the third, and every part is readable by all three. None of you is writing the platform's own source, and you were not given it — nothing here asks you to open a file you do not have.

It assumes you can read Python and are comfortable at a shell. It does not assume you have used Aegis, or that you know what any of the four standards say.

The one thing to understand before you start

The harness is the primary working surface. The web console is the second one.

Almost everything an architect or operator does — creating an organisation, defining units and roles, writing an operating envelope, establishing a trust chain, moving an agent's posture, submitting work, pulling evidence — is done by driving the SDK from your own code, against a deployment you point it at. That path is scriptable, reviewable, diffable, and repeatable across environments.

The console is real and it is not a toy: it is where humans answer the questions Aegis stops to ask, and it is where you look when you want to see the state rather than change it. But an organisation built by clicking is an organisation nobody can rebuild, review or promote between environments. Build it in code.

Every chapter in parts 01 to 04 shows the harness path. Where the console does the same thing, the chapter names the screen so you can find it. Part 05 covers the console on its own terms. Part 06 is neither path: it is the system-level picture both of them drive.

The parts

#	Part	Read it when
01	Orientation	First. What you were given, how governance actually works here, and your first working session
02	Working through the harness	The spine. Standing up an organisation, bounding it, running work through it, and evidencing it
03	Extending the platform	You are adding a capability — a tool, an agent, a skill, a pipeline
04	The API surface	You are calling Aegis over HTTP directly, debugging an authorization failure, or asking what the client does with errors, credentials, pages, load and streams
05	The web console	You want to know what a screen shows, or you are the human answering a held decision
06	Architecture	The system-level picture. The plane split, the governance objects, where enforcement happens, and where your integration attaches
07	Deployment architecture	You are deploying the platform, sizing it, or asking where its boundaries actually fall
08	The capability catalogue	You are asking what Aegis can do — the complete capability surface, by area, with the operation that reaches each one
09	The governance architecture	You are satisfying an auditor. The two planes, envelopes, clearance, the decision gradient, trust chains, the audit spine, and containment
10	The organisation structure framework	You are designing the organisation your agents will live in — the D/T/R grammar, addressing, roles, trust chains and re-orgs
11	The deployment pack	You are standing a deployment up and running it — provisioning, the configuration reference, the runtime's node catalogue, releasing, operating, verifying

Read **01** in order. After that, **02** is the working reference you will return to; **03** through **11** are consulted rather than read through.

If you were handed this book to answer one question, three parts answer the three most common ones on their own: **08** for *what can it do*, **09** for *how is it governed*, and **10** for *how do we model our organisation in it*.

The tree below says which part you reach for, not which part to read next:

```
the book
|
├─ 01 Orientation                read in order – start here
├─ 02 Working through the harness the spine; you return to it
|   └─ 03 Extending the platform
|       └─ 04 The API surface
|           └─ 05 The web console
|
├─ 06 Architecture                consulted from any part above
|   └─ the system-level picture: how the platform is put together,
|       where its boundaries are, and where your integration attaches
|
├─ 07 Deployment architecture     consulted from any part above
|   └─ the reference deployment: the cluster, the network boundary,
|       the stateful services, and how identity and secrets reach them
|
├─ 08 The capability catalogue     the reference. What the platform does,
|   └─ organised by capability area, each with the operation that reaches it
|
├─ 09 The governance architecture the auditor's part
|   └─ two planes, envelopes, clearance, the decision gradient, trust
|       chains and postures, the audit spine, containment and refusal
|
├─ 10 The organisation structure  the designer's part
|   └─ D/T/R, addressing, roles and intent, trust chains, role agents,
|       re-orgs, bridges and workspaces
|
└─ 11 The deployment pack         the operator's working set
    └─ provisioning, the configuration reference, the runtime's node
        catalogue, releasing and rolling back, operating, verifying
```

How claims in this handbook are kept true

Every load-bearing claim names something **you can reach from what you were given**: an HTTP operation this client performs, or a symbol this package exports. Both are checkable, by you, on your own machine:

```
python -m aegis_sdk.handbook.check
```

That command re-resolves every anchor against your installed build and fails naming any that no longer resolves. It also fails on a broken cross-reference between chapters, so a link in this book either goes somewhere or the check says so. Because every anchor names something you hold, the check runs on your own machine rather than ours — the book's claims are verifiable by its reader, not only by its author.

What this book covers

This book is written for the surface you were given: the deployment, the client that drives it, and the console that shows it. Every chapter is actionable from there — you are never asked to open a file you do not have.

Part 06 covers the platform's architecture, and that is not a contradiction, because *architecture* is a word that usually means the inside. Part 06 describes the split between the Trust Plane and the Execution Plane, the governance objects and how they relate, and where enforcement happens — and every claim it makes rests on an operation you can call or a symbol you can import. It gives you the shape of the system you are designing against, in terms you can verify yourself.

Who is accountable

The organisation deploying an agent keeps the accountability for what that agent does. It cannot be delegated, and no amount of autonomy transfers it. What this platform provides is the **proof** — the bounded mandate, the tamper-evident audit trail, the fail-closed gate, the attestable lineage — that lets that organisation show an auditor how it discharged the accountability it already had.

You keep the accountability. This gives you the evidence.

Part 01 — Orientation

Three chapters, read in order. They are the only part of this book that is meant to be read straight through, and skipping them is the most expensive shortcut available: chapter 01.2 in particular describes a model people routinely guess wrong, and every design built on the wrong guess has to be rebuilt.

#	Chapter	What you get
01.1	What you were given	The shape of the thing in your hands, and what each piece is for
01.2	The mental model	Where governance actually happens. The load-bearing chapter of the whole book
01.3	Your first working session	Install, point at a deployment, authenticate, and make a real call — end to end

Why 01.2 is the one to read even in a hurry

The single most expensive misunderstanding on this platform is about *where* governance happens. People assume Aegis sits between an agent and the outside world, receiving the agent's requests and forwarding the approved ones. It does not. Nothing here does that, and building against the wrong model produces designs that cannot work — usually discovered late, when someone asks where the forwarding code is and the answer is that there isn't any.

That chapter is short and it decides how you design everything afterwards.

What this part deliberately does not do

It does not teach the four standards. Aegis implements CARE, PACT, EATP and CO; you can operate a deployment competently without reading any of them, and this book uses the Foundation's vocabulary where it uses it rather than inventing a parallel one. Where a term is load-bearing — *envelope*, *posture*, *clearance*, *verification gradient* — the chapter that first needs it defines it in place.

Next: 01.1 — What you were given

01.1 — What you were given

Three things, and it is worth being precise about each, because the boundaries between them decide which one you reach for.

1. A deployed Aegis

A running instance, reachable over HTTPS at a base URL someone gave you. It holds your organisation's state: units, roles, agents, envelopes, trust chains, knowledge, objectives, and the audit trail of everything that has happened to them.

You do not administer the process. You administer what is *inside* it.

2. This SDK — `aegis_sdk`

A Python client for that deployment. It is not a thin HTTP wrapper: it is organised into typed modules that mirror the governed nouns, so the code you write reads like the organisation you are describing.

```
client.organizations  # the tenant root
client.units         # organisational units — the knowledge boundaries
client.roles         # roles within units, and their authority level
client.teams         # working groups
client.role_envelopes # what a role's delegate may do
client.knowledge     # governing documents, with a review lifecycle
client.ontology      # the vocabulary a vertical speaks
client.agents        # the agents themselves
client.trust.chains   # who delegated authority to whom
client.trust.postures # how much autonomy an agent currently holds
client.trust.audit    # the trust-decision record
client.objectives     # work you ask for
client.requests       # work items that need a human
client.sessions       # an agent's actual run
client.approvals      # the queue of things waiting on a person
client.compliance     # evidence export and audit verification
```

The full client exposes considerably more than this — revenue and licensing, analytics, connectors, webhooks, notifications, pools, applications, specialist configuration, LLM provider settings. The list above is the set an architect or operator touches on a normal day.

Two facts about the client that shape how you use it:

- **It is asynchronous.** Every call is `await` ed. If you are writing a provisioning script, it runs inside `asyncio.run(...)`.
- **Its return shapes are not uniform, and this catches people.** Some modules return typed models with attribute access (`agent.id`); the vertical-standup modules return raw dictionaries with subscript access (`unit["id"]`). Mixing them up produces a `TypeError` or an `AttributeError` at the point of use rather than at the call. When in doubt, `print(type(result))` once.

3. This handbook, and the working artifacts beside it

The handbook ships *inside* the package, at `aegis_sdk/handbook/`, so the verification command in the introduction runs on your own machine against your own copy.

Beside it, at `aegis_sdk/coc/`, is a set of **working artifacts** — the same working mode this book describes, in a form your tooling can load rather than a form you read. Every path below is a link, and all of them resolve in your checkout.

Two agent briefs — a specialist's responsibilities, written so a question can be answered without inventing server behaviour:

brief	for
<code>coc/agents/aegis-sdk-specialist.md</code>	building against the client — what it exposes, what it refuses to assume
<code>coc/agents/aegis-operator.md</code>	running an organisation day to day, across both surfaces

Six skills — the order of operations for a task, not an explanation of it:

skill	when
<code>coc/skills/working-against-a-deployment.md</code>	connecting, establishing what you hold, measuring reachability
<code>coc/skills/running-an-objective.md</code>	submitting work and following it to a result
<code>coc/skills/registering-a-tool-or-agent.md</code>	extending the platform with something of your own
<code>coc/skills/reading-trust-and-governance.md</code>	answering "why was this allowed" or "why was it not"
<code>coc/skills/diagnosing-a-refusal.md</code>	a call came back denied and you need to know which kind
<code>coc/skills/day-two-operations.md</code>	the work that starts after the organisation exists

Six guardrails — obligations carried with their reasons, so they can be argued with rather than only obeyed:

guardrail	the obligation
<code>coc/guardrails/credential-reachability.md</code>	establish what your credential can reach before concluding anything from a refusal
<code>coc/guardrails/error-taxonomy.md</code>	tell a refusal from a fault from an unauthenticated call; they are three things
<code>coc/guardrails/reading-a-measurement.md</code>	name what would have proved you wrong before citing what you measured
<code>coc/guardrails/client-model-fidelity.md</code>	the client will not invent a contract the platform does not make
<code>coc/guardrails/sentinels-and-defaults.md</code>	a default is a claim; an absent value and a zero are not the same

guardrail	the obligation
<code>coc/guardrails/billing-integrity.md</code>	what a spend control does and does not establish

And one runnable check — `coc/probe.py`, invoked as `python -m aegis_sdk.coc.probe`: a reachability and transport-honesty probe you run rather than read. `coc/README.md` is the door into all of it.

△ "Agent brief", not "agent". This platform's own word *agent* means the thing under governance — the delegate that does work inside an envelope. An agent brief is a document describing a specialist's responsibilities. The two are unrelated and readers conflate them instantly, so this book keeps bare "agent" for the platform's sense throughout and says "agent brief" for the other.

Those artifacts are the **primary** working mode; this book is the explanation behind them. They take precedence, and the probe's output is the reading to trust about your own deployment.

What this book cites, and why

You build against a running Aegis over HTTP. This book therefore cites only **surface you can reach** — an HTTP operation this client performs, written `api:POST /api/v1/trust/establish`, or a symbol this package exports, written `sdk:aegis_sdk.TrustChain`. It cites no file paths and no line numbers into the platform's own source, because a coordinate into a tree you do not hold is a coordinate you cannot follow.

Both forms resolve on your machine under `python -m aegis_sdk.handbook.check`, so every operation and every symbol this book names can be confirmed against the build you actually have.

The two surfaces, stated once so the rest of the book can be terse

	The harness	The console
what it is	this SDK, driven from your own code	the web application at your deployment's base URL
who uses it	architects, operators	anyone with a login, including you
good at	building, bounding, provisioning, promoting between environments, evidencing	answering a held decision, inspecting current state, day-to-day work
bad at	being read by someone who does not code	being reviewed, diffed, repeated, or promoted
leaves behind	code someone can review before it runs	an audit entry, and no reviewable intent

Both surfaces hit the same API and are governed identically. Choosing the console does not get you a lighter check, and choosing the harness does not get you a heavier one. The enforcement sits behind the API rather than in either client, which is why the same call from either side lands on the same verdict.

Next: 01.2 — The mental model

01.2 — The mental model

Aegis is not a proxy

When a governed agent writes to an external system — creates a Jira ticket, pushes a commit, sends a message — the request does **not** travel to Aegis for Aegis to forward. Aegis never takes possession of the call. There is no generic egress gateway, no outbound request broker, no forwarding layer.

This is the thing people get wrong, and they get it wrong in a specific way: they assume that because the platform can *stop* an action, it must be *carrying* the action. It is not. It is standing beside it.

Here is the actual shape on the `claude_agent_sdk` runtime, which is the one this model describes. Runtimes differ in what sits on the call path, so establish which one your deployment runs before designing against these steps:

1. The agent decides to call a tool.
2. Before that call executes, a `PreToolUse` hook fires on the agent's own call path, inside the agent's own process.
3. The hook consults Aegis and gets back a verdict.
4. On a permitting verdict, the **original call proceeds and the tool itself does the work**. Aegis is no longer involved.
5. Afterwards a `PostToolUse` hook records what happened.

The verdict happens *in line*, not *in between*.

Two wrong models, and why each fails

Wrong model 1: "ask permission, then act." The agent asks Aegis whether it may do something, receives a yes, and then does it. This is wrong because it is *advisory* — an agent that skips the asking step proceeds unimpeded. Governance that depends on the governed party choosing to participate is not governance. Hooking the call path is what makes the check something that happens *to* the agent rather than something the agent invokes.

 **The enforcement point is a property of the runtime, so establish which runtime your deployment runs before you design against a hook.**

The hook model described here is the `claude_agent_sdk` runtime. The **default** runtime is `kaizen_native`, which places no pre-execution hook on the call path. The platform declares this rather than leaving you to infer it: it emits a `default_has_no_pre_execution_gate` notice at boot naming the runtime and the enforcement point, and it will *refuse to start* — not warn — when the deployment has set the environment variable requiring a pre-execution gate. A deployment that must have one says so in configuration and the platform holds it to that, fail-closed.

The property that holds on every runtime is custody, not interception: the agent does not hold the keys. Credentials for the systems your tools write to are not the agent's to present. That is true whatever runtime is configured, and it is the one to reason from.

Wrong model 2: "intercept and forward." The agent's request goes to Aegis, Aegis evaluates it and then performs the write on the agent's behalf. This is wrong because there is nothing here that does it. There is no generic forwarding component — no egress proxy, no outbound gateway, no request broker. You can confirm this from your own side: no operation in this client accepts an outbound request and returns a foreign system's response, and none names a forwarding endpoint. If your design has Aegis making the outbound call, you are designing a component that does not exist and will have to be built.

The distinction is not academic. Under the correct model, Aegis needs no credentials for the external system, never sees the response body, and cannot become a bottleneck or a single point of failure for the agent's actual work. All three of those follow directly from *not* being in the data path.

The reference implementation, and what its shape tells you

The reference implementation is the Claude Code wrapper. It registers **four hooks** with the agent runtime's hook system — two before a tool call and two after:

when	hook	what it does
PreToolUse	trust verification	asks the trust plane for a verdict
PreToolUse	budget enforcement	refuses if the spend envelope is exhausted
PostToolUse	audit logging	records what happened
PostToolUse	cost recording	attributes the spend

Note the fallback structure, because it tells you the posture of the design. The wrapper prefers to register with the runtime's own hook manager; if that registration raises, it falls back to an internal registry rather than proceeding ungoverned. Failing to register is not treated as permission to run unhooked.

The pre-hook's shape tells you everything about the model:

```
async def pre_tool_hook(
    tool_name: str,
    tool_input: dict[str, Any],
) -> dict[str, Any]:
    ...
```

It receives the tool's **name** and its **input**, and it returns the tool input, possibly modified. It does not return a response, because it never made a request. On failure it raises — a trust-verification error, or a constraint-violation error from the constraint enforcer — and the tool call never happens. On success the call continues with the input the hook handed back.

That return type — `dict` in, `dict` out — is the proof that this is interception rather than proxying. A forwarding layer would return the external system's response. This returns the arguments.

Trust verification itself is a call *out* to the trust plane, keyed on the agent id and an action string built as `f"tool:{tool_name}"`, with the organisation and user carried in the context — **never taken from anything the tool says about itself**.

The verdicts

The evaluation resolves into CARE's four-zone verification gradient. Four values, and **exactly four** — you will see one of them on the wire as the verdict's `level`:

```
auto_approved / flagged / held / blocked
```

- **auto_approved** — proceed, recorded.
- **flagged** — proceed, but surfaced for attention.
- **held** — stopped pending a human decision.
- **blocked** — refused.

Two behaviours worth knowing because they surprise people. Thoroughness can *upgrade* a verdict: a `full` thoroughness setting promotes `auto_approved` to `flagged`, so a stricter setting produces more **signal** rather than merely more refusals — if you set thoroughness high and see your flagged count rise, that is the feature, not a regression. And **held is not a soft blocked**. Under `delegated` posture the enforced-execution path *returns* on a constraint violation rather than parking the work, which means a held action **ends the agent's run** instead of queuing politely. Plan for a held action to terminate the run, not to pause it. Chapter 03.1 has the incident where that mattered.

The permission set is materialized, not asked for

The clearest evidence that governance rides the agent's own call path is that Aegis *writes the agent's permission file*. Trust posture is materialised into a concrete allow/deny list before the agent runs — you can read the posture an agent currently holds at `api:GET /api/v1/agents/{id}/trust-posture`:

Posture	allow	deny
<code>pseudo</code>	—	<code>Bash</code> , <code>Edit</code> , <code>Write</code> , <code>NotebookEdit</code>
<code>supervised</code>	<code>Read</code> , <code>Glob</code> , <code>Grep</code>	<code>Bash</code> , <code>Edit</code> , <code>Write</code> , <code>NotebookEdit</code>
<code>shared_planning</code>	+ <code>Edit</code> , <code>Write</code>	—
<code>continuous_insight</code>	+ <code>Bash</code>	—
<code>delegated</code>	+ <code>NotebookEdit</code>	—

This is written into the agent's own `settings.json` — its native permission mechanism — and the selection **falls back to supervised for an unrecognised posture, not to the permissive end**. That default is the whole design in one line: an unknown posture is treated as a constrained one. A proxy

would not need this file to exist. An in-line interceptor does, because the enforcement point is inside the agent's runtime, and the runtime has to be told what it may do.

The one egress control, and what it covers

Aegis is not a proxy for *tool calls*. One thing is gated outbound, and it is worth naming precisely: **LLM content**.

A fail-closed classification guard governs outbound **LLM content**. Its terms:

- It ships the **mechanism** only. Which environments are guarded is configuration the deployment sets, and the classification **policy** is supplied by the deployment, through a classifier it installs. The canonical default is no guarded environment and no classifier — an inert pass-through with zero overhead. This is deliberate: canonical Aegis stays generic and client deployments adapt.
- Inside a guarded environment it is **fail-closed** — an *absent* classifier and a classifier that *raises* both produce DENY. A guarded environment with undefined policy does not let content out.
- It covers the LLM calls your own process makes, and only those.

A runtime that spawns a subprocess leaves that coverage the moment it spawns. Four do —

`claude_agent_sdk`, `claude_code`, `openai_codex` and `gemini_cli` — and each launches an external CLI that performs its own egress, outside any in-process content guard, whatever the classifier would have said. Those four carry a **structural** control instead of a classifying one: a deployment that must guarantee no external-LLM egress disables them, and constructing one then raises rather than succeeding quietly. Structural inability to construct it is the control, and it is stronger than a policy that inspects what is sent. If that guarantee is one you have to make, establish which of the four your deployment has enabled before you promise it.

So: content leaving *your process* toward an LLM can be classified. A tool call an agent makes is governed at the hook, not forwarded. Neither of those is a general-purpose egress proxy, and no such thing exists here.

What follows from all this

If you remember one sentence: **where there is a hook, the hook decides and the tool acts** — and the enforcement point is a property of the runtime, so establish which runtime your deployment runs before you design against it.

Practical consequences for anything you build:

- Aegis holds no credentials for the systems your tools write to. Your tool does.
- A verdict is about an *intent* — a tool name and its arguments — never about a response body, because no response exists yet when the verdict is made.
- If governance must cover a new execution path, the question is not "how do we route it through Aegis" but "where on that path does the hook fire". A path with no hook is ungoverned no matter how much policy exists elsewhere.

- Anything that spawns a subprocess leaves the reach of every in-process control the moment it spawns. Govern it before the spawn, or disable it.
-

Next: 01.3 — Your first working session

01.3 — Your first working session

End to end, from nothing to a call that proves your credentials, your network path and your permissions all work. Fifteen minutes, and it is worth doing before you write anything real: almost every "the SDK is broken" report resolves to one of the three things this chapter separates.

Install

```
pip install -e .
```

⚠ **Install from source, not from the public index.** The name `aegis-sdk` on PyPI belongs to an unrelated third-party package. `pip install aegis-sdk` will succeed, install something else entirely, and then fail in ways that look like your code is wrong.

Confirm you have the right thing:

```
import aegis_sdk
print(aegis_sdk.__version__)
```

Point it at your deployment

There is **no default base URL, deliberately, and it will refuse to start without one.**

```
export AGENTIC_OS_BASE_URL="https://aegis.your-company.example" # your deployment
export AGENTIC_OS_API_KEY="sk_live..." # your key
```

If you omit the base URL, `sdk:aegis_sdk.ClientConfig` raises `sdk:aegis_sdk.ConfigurationError` with a message naming the variable. That refusal is the design working: a client that fell back to a built-in host would send production traffic to a placeholder endpoint and report success, so the failure is made loud and early instead.

Other variables the client reads, all optional: `AGENTIC_OS_TIMEOUT` (default 30.0 seconds), `AGENTIC_OS_MAX_RETRIES` (3), `AGENTIC_OS_VERIFY_SSL` (true), `AGENTIC_OS_DEBUG` (false). Set `AGENTIC_OS_DEBUG=true` while you are getting started; it is the difference between "something went wrong" and a request you can read.

Before the first call: two checks that need no credentials

Both run offline and both answer a question you would otherwise guess at:

```
python -m aegis_sdk.handbook.check # does this build still have what this book names?
python -m aegis_sdk.coc.probe --transports-only # which client paths are real, before any network
```

The first is this handbook's own verifier. The second reports which transports the client genuinely has, without contacting anything — so a later failure can be attributed to the deployment rather than to the package.

Then point the probe at your deployment once you have the variables below:

```
python -m aegis_sdk.coc.probe --base-url "$AGENTIC_OS_BASE_URL" --api-key "$KEY"
```

That measures **reachability** — what your credential can actually get to. Run it before you conclude anything about permissions; 04.1 explains why reachability and permission are different questions that produce the same 403.

The first call

```
import asyncio
from aegis_sdk import AgenticOSClient

async def main() -> None:
    client = AgenticOSClient.from_env()
    me = await client.auth.get_current_user()
    print(me)

asyncio.run(main())
```

`sdk:aegis_sdk.AgenticOSClient.from_env` reads the environment above.
`client.auth.get_current_user()` calls `api:GET /api/v1/auth/me` and returns a `sdk:aegis_sdk.User`.

This one call is a better smoke test than a health check, because it exercises the whole chain rather than only reachability. Read the failure by *kind*:

what you see	what it means	where to go
<code>ConfigurationError</code>	the client never made a request — no base URL	set <code>AGENTIC_OS_BASE_URL</code>
<code>ConnectionError</code> / <code>NetworkError</code>	DNS, TLS or routing — the deployment was not reached	your network, VPN, or the URL itself
<code>AuthenticationError</code>	reached it; the key is absent, malformed or revoked	reissue the key
<code>AuthorizationError</code>	reached it, key is valid, this principal may not do this	04.1 — and read the warning below first
a <code>ValidationError</code> on <code>role</code>	the call succeeded ; the client could not parse the reply	see immediately below
a <code>User</code> printed	everything works	continue

⚠ This diagnostic can fail for the very credential it exists to diagnose, and the failure does not look like an auth problem. `sdk:aegis_sdk.User` requires six fields — `id`, `email`, `name`, `organization_id`, `organization_name` and `role` — and a principal thin on any one of them does not fit the model. `role` is the one this bites on, because an API-key principal carries no role:

what the server sends	what the client does
<code>role</code> omitted	raises <code>ValidationError</code> — <code>role: missing</code>
<code>role: null</code>	raises <code>ValidationError</code> — <code>role: string_type</code>
<code>role: ""</code>	parses

If you hit it, do not conclude your key is broken. A `ValidationError` here means the request was authenticated and answered — the reply simply did not fit `sdk:aegis_sdk.User`. Drop to the raw call to see what came back, and continue: nothing else in this book depends on `get_current_user()` succeeding.

If you are using a token rather than an API key, this call is the straightforward one and the caveat above is unlikely to bite. It is worth doing both if you hold both, because the two credential types do not reach the same surface — see 04.1.

🚫 Do not spend a day tuning scopes on an `AuthorizationError` before reading chapter 04.1. An API key and a token do not reach the same surface, and a reachability denial and a scope denial arrive as the same generic 403 — the response itself cannot tell you which one you are holding. 04.1 gives you a one-minute test that settles it. If your scopes look right, they probably are; widened API-key route coverage is [immediate roadmap].

Now do something that changes state

Creating an organisation is the smallest end-to-end write, and it is the root every other noun hangs off.

```
org = await client.organizations.create(
    name="Northwind Manufacturing",
    slug="northwind",
    plan_tier="professional",
)
print(org["id"])          # note: a dict, not a model – see 01.1
```

That is `api:POST /api/v1/organizations`. Read it back with `api:GET /api/v1/organizations/{id}`, and list what you can see with `api:GET /api/v1/auth/me/organizations`.

In the console, the same object is what you land in after signing in; there is no separate "create organisation" screen for most deployments, because this is provisioning rather than daily work. That asymmetry is the pattern for the whole book: the harness does the building, the console does the working.

Where to go next

You now have a client that authenticates and a tenant root to hang things off. [Part 02](#) is the working spine — units, roles, envelopes, trust, work, evidence — and it assumes exactly the state this chapter left you in.

Next: [Part 02 — Working through the harness](#)

Part 02 — Working through the harness

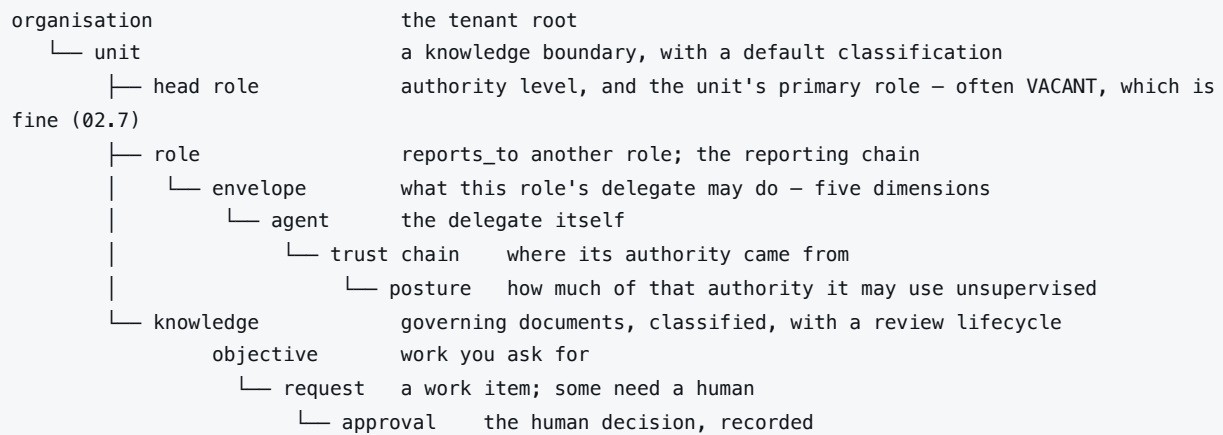
This is the spine of the book. Everything an architect designs and an operator runs is here, in the order you actually do it: describe the organisation, bound it, wire trust through it, put work into it, and pull evidence out of it.

Each chapter shows the **harness path first** — the SDK call you write, run and review — and then names the **console equivalent** where one exists, so you can find the same thing on a screen when a human needs to look at it.

#	Chapter	You leave able to
02.1	The two surfaces	Choose the right surface for a task, and know why the choice matters
02.2	Standing up an organisation	Provision a tenant, units, roles and teams from code
02.3	Envelopes, clearance and knowledge	Bound what a role's delegate may do, and control what it can see
02.4	Trust chains and postures	Establish delegated authority, move an agent's autonomy, and revoke safely
02.5	Objectives and the work loop	Put work in, watch it, and understand the states it passes through
02.6	Approvals, holds and evidence	Answer what Aegis stops to ask, and export the record afterwards
02.7	Role agents, and surviving a re-org	Stand up a delegate on a seat nobody occupies, and move the org chart without losing it

The shape of the whole thing, in one sequence

Every chapter below is a slice of this. It is worth reading once as a whole before reading any part of it in detail, because each step constrains the next and the constraint direction is not obvious from any single call.



Two properties of that tree decide most design questions, and both are easy to get backwards:

1. **A unit is a knowledge boundary, not just a box on a chart.** Being a colleague of someone does not grant sight of their work. Visibility is a property of the unit and the clearance, not of the org chart's shape.
2. **Envelopes compose by intersection, never by union.** A delegate cannot be granted more than the role that defines its envelope holds. Adding a second envelope narrows; it does not widen. Chapter 02.3 is explicit about this because designs that assume otherwise fail late, at the point where someone expects an escalation path that cannot exist.

Before you start: build it twice

The reason this part is harness-first rather than console-first is not aesthetic. An organisation you can re-create from a script is one you can review before it exists, promote from staging to production unchanged, diff when it drifts, and rebuild after an incident. An organisation assembled by clicking is none of those, and the difference is invisible until the day you need it.

Write your provisioning as an idempotent script from the beginning, even for a pilot. Chapter 02.2 shows the shape.

Next: 02.1 — The two surfaces

02.1 — The two surfaces

Aegis has one API and two clients over it: **this SDK**, which you drive from your own code, and **the web console**, which people use in a browser. This chapter is about choosing between them, because the choice is not a matter of taste and getting it wrong is expensive in a way that shows up months later.

They are the same system

Both surfaces speak to the same HTTP API and are governed by the same checks. The console has no privileged back channel, and the SDK has no bypass. A call that is refused for you in one is refused in the other, for the same reason, and produces the same audit entry.

The client you drive is `sdk:aegis_sdk.Agentic0SClient`, and the enforcement lives behind the API rather than in either client, which is why parity holds without anyone maintaining it. Act on that: a control you verified through the SDK does not need re-verifying through the console.

Why the harness is the primary surface

Not because it is more powerful. Because of what it leaves behind.

	harness	console
reviewable before it runs	yes — it is code, in a branch, in a diff	no — the action <i>is</i> the record
repeatable	yes — run it against staging, then production	no — someone repeats it by hand, differently
diffable when it drifts	yes — re-run and compare	only by reading screens
rebuildable after an incident	yes	from memory
legible to a non-programmer	no	yes
good for a decision that needs a person	no	yes

The asymmetry is the point. An organisation's *structure* — who reports to whom, what each role's delegate may spend, which units can see which knowledge — is configuration that outlives the person who set it and must be defensible to an auditor. Configuration like that belongs in code.

An organisation's *operation* — approving a held payment, reading why an objective stopped, checking an inbox — is human judgment happening now. That belongs on a screen.

A rule of thumb that holds up: if the action would be identical next quarter in a fresh environment, write it in the harness. If the action is a person deciding something about a specific case, do it in the console.

What only the console can do

Worth being explicit, because "harness-first" is sometimes read as "harness-only", and that produces operators who cannot do their jobs.

- **Answer a held decision as yourself.** Approvals are attributable to a human, and the console is where a human is authenticated as themselves rather than as a provisioning key.
- **Read state you have not scripted a view for.** Dashboards, activity feeds, drift alerts and the inbox are assembled views. You can reach the same data through the SDK, but you would be rebuilding a screen that already exists.
- **Be used by people who do not write Python.** Most of an organisation's users are only ever going to see the console. Part 05 is written for them, and it is worth reading if you are designing what they will meet.

What only the harness can do

- **Provision.** Creating an organisation, its units, its roles and their envelopes as one reviewable transaction.
- **Promote.** The same script, a different `AGENTIC_OS_BASE_URL`.
- **Assert.** You can write checks about your own organisation — "no envelope permits an external wire transfer", "every unit has a head role" — and run them on a schedule. Nothing in the console does this, and nothing in Aegis does it for you.
- **Bulk anything.** Two hundred roles is a loop. It is not two hundred forms.

The pattern every chapter in this part follows

```
# HARNESS — what you write
unit = await client.units.create(name="Treasury", unit_type="department")
```

In the console: *Organisation* → *Units* → *New unit*. Same result, same audit entry, no reviewable artifact.

When a chapter names a console screen, it is telling you where a human will meet the thing you just built — not suggesting you build it there.

One caution about mixing them

If you provision with a script and then let people edit the result in the console, your script stops describing reality and starts describing an intention. That is fine as long as everyone knows which one is authoritative. Decide early, write it down, and prefer making the script re-runnable (create-or-update) over forbidding console edits, which nobody will honour under pressure.

Next: 02.2 — Standing up an organisation

02.2 — Standing up an organisation

The provisioning path, in the order the platform requires it, with the constraints that are not obvious from any single call.

The order is not arbitrary

```
organisation -> unit -> head role -> further roles -> teams
```

Each step needs the identifier the previous one returned. A unit cannot exist without an organisation to hold it; a role cannot exist without a unit; the reporting chain is expressed by pointing one role at another, so the manager must exist before the report.

You cannot usefully parallelise this. You can parallelise *sibling* roles under a unit that already exists, and for a large org chart that is where the time goes.

1. The organisation

```
org = await client.organizations.create(  
    name="Northwind Manufacturing",  
    slug="northwind",  
    plan_tier="professional",  
)  
org_id = org["id"]
```

`api:POST /api/v1/organizations` . This is the tenant root; everything below is scoped to it. Read it back with `api:GET /api/v1/organizations/{id}` , and list the organisations your principal can see with `api:GET /api/v1/auth/me/organizations` .

2. Units — and the classification decision you are making

```
unit = await client.units.create(  
    name="Treasury",  
    unit_type="department",  
    description="Cash and liquidity management",  
    default_classification="confidential",  
)  
unit_id = unit["id"]
```

`api:POST /api/v1/organization-units` .

`unit_type` is `department` or `team` . Nest with `parent_unit_id` ; omit it for a root unit.

`default_classification` is the most consequential argument on this call and it defaults to the most permissive value. The five levels, in order:


```
public < restricted < confidential < secret < top_secret
```

The default is `public`. Knowledge created in a unit inherits that level unless overridden, and the level is what clearance is checked against later. A unit created with the default and populated with real material is a unit whose contents are readable more widely than anyone intended, and nothing will tell you — it is not an error state, it is a choice you made by omission.

Set it explicitly on every unit, every time. If you take one habit from this chapter, take that one.

Other arguments worth knowing: `budget_allocation` and `budget_currency` (the unit's financial envelope), `headcount_limit`, `responsibilities`, and `constraint_overrides` for unit-level constraint defaults.

3. Roles — the reporting chain, and the head-role rule

Every unit needs a head. Create it first, flagged:

```
head_role = await client.roles.create(
    organization_unit_id=unit_id,
    title="Head of Treasury",
    authority_level=4,
    is_primary_for_unit=True,
)
head_role_id = head_role["id"]
```

Then its reports, pointing at it:

```
analyst = await client.roles.create(
    organization_unit_id=unit_id,
    title="Treasury Analyst",
    authority_level=1,
    reports_to_role_id=head_role_id,
)
```

`api:POST /api/v1/organization-roles`.

Three things about roles that are routinely misread:

- **`authority_level` is informational and does NOT gate access.** It runs 1 (staff) to 5 (C-suite) and describes seniority. What gates access is *clearance*, which is a separate axis and is chapter 02.3. A level-5 role with no clearance sees less than a level-1 role with clearance. Designing a permission model around `authority_level` produces something that looks like it works and does not.
- **`reports_to_role_id` is the delegation path.** Trust chains follow the reporting chain, not the unit tree — a role with no manager is a chain root. Chapter 02.4 depends on this being right, so get the reporting lines correct at creation rather than fixing them later.
- **`auto_generate_agent` defaults to `True`.** Creating a role creates an agent for it unless you say otherwise. That is usually what you want, and it is occasionally a surprise when a provisioning run produces more agents than roles you meant to staff. Pass `auto_generate_agent=False` when you

are building structure ahead of staffing it.

- **Creating a UNIT auto-creates its primary role, and that role arrives UNPARENTED** — `reports_to_role_id` is null, so by the bullet above it anchors its own trust chain instead of inheriting one. Nothing warns you. Parent it explicitly after creating the unit.

Building structure ahead of staffing it is the normal case, not a degenerate one — a lead seat with nobody in it is valid, and a delegate agent can be stood up, trusted and activated on it. Chapter 02.7 has that sequence, along with what happens to the agent when somebody is eventually appointed, when they leave, and when the role moves.

Read roles back with `api:GET /api/v1/organization-roles` and `api:GET /api/v1/organization-roles/{id}`; the assembled hierarchy is at `api:GET /api/v1/admin/roles/hierarchy`. Assign a human to a role with `api:POST /api/v1/organization-roles/{id}/assign-user`.

4. Teams

```
team = await client.teams.create(name="Cash Ops", description="Daily cash desk")
await client.org_standup.add_team_member(team["id"], user_id="user...")
```

`api:POST /api/v1/teams`, members at `api:POST /api/v1/teams/{id}/members`. Teams are working groups; they do not carry the knowledge-boundary semantics a unit does. If you want a boundary, you want a unit.

Note where `add_team_member` lives — on `client.org_standup`, not on `client.teams`. That is not a typo, and the next section is about why.

5. Vocabulary — make the vertical speak its own language

```
await client.ontology.apply_preset(org_id, preset_id="finance")
```

`api:POST /api/v1/ontology/preset/{id}`. Available presets are at `api:GET /api/v1/ontology/presets`, and the resolved configuration for an organisation at `api:GET /api/v1/ontology/config/{id}`.

This changes the nouns the product uses in that tenant. It is cheap to do at provisioning time and disruptive to change once people have learned the old words, so decide it early.

⚠ Writing and reading are different modules, with different return types

This is the single thing about the SDK most likely to cost you an afternoon, so it gets its own section.

The typed standup modules — `client.organizations`, `client.units`, `client.roles`, `client.teams`, `client.role_envelopes`, `client.knowledge` — expose `create` and `get` and **nothing else**. There is no `client.units.list()`. Reaching for one is the obvious move and it raises `AttributeError`.

The list operations live on `client.org_standup` :

you want	you call	you get back
create a unit	<code>client.units.create(...)</code>	a dict — <code>unit["id"]</code>
get one unit	<code>client.units.get(unit_id)</code>	a dict
list units	<code>client.org_standup.list_units(.. .)</code>	an <code>OrganizationUnitList</code> model — <code>.records</code> , <code>.total</code>
list roles	<code>client.org_standup.list_roles(.. .)</code>	a model with <code>.records</code>
list teams	<code>client.org_standup.list_teams(.. .)</code>	a model with <code>.records</code>
list organisations	<code>client.org_standup.list_organiza tions(...)</code>	a model with <code>.records</code>

So the write returns a dictionary and the read returns a Pydantic model, and the two are accessed differently: `unit["id"]` on the way in, `unit.id` on the way out. Mixing them is the most common first-run failure in a provisioning script, and it fails at the point of *use* rather than at the call, which is what makes it annoying to locate.

`list_units` also takes `parent_unit_id` , `unit_type` , `include_archived` , and `limit` / `offset` — and **limit defaults to 50**. A tenant with more than 50 units silently gives you the first 50. Page it, or you will write a create-if-missing check that re-creates everything past the first page.

Writing this as a script you can run twice

The single highest-value habit in this chapter. A provisioning script that fails halfway leaves an organisation in a state nobody designed, and the instinct — to delete it and start over — stops being available the moment there is real data in it.

```

import asyncio
from aegis_sdk import AgenticOSClient

async def all_units(client):
    """Every unit, paged – list_units caps at 50 per call."""
    out, offset = [], 0
    while True:
        page = await client.org_standup.list_units(limit=200, offset=offset)
        out.extend(page.records)          # models: attribute access
        if len(out) >= page.total or not page.records:
            return out
        offset += len(page.records)

async def ensure_unit(client, *, name, unit_type, classification):
    """Return the existing unit with this name, or create it."""
    for u in await all_units(client):
        if u.name == name:                # model -> u.name
            return u.id
    created = await client.units.create(  # dict -> created["id"]
        name=name, unit_type=unit_type, default_classification=classification
    )
    return created["id"]

async def provision() -> None:
    client = AgenticOSClient.from_env()
    treasury_id = await ensure_unit(
        client, name="Treasury", unit_type="department", classification="confidential"
    )
    print(treasury_id)

asyncio.run(provision())

```

Match on something stable. Names get edited, so if your units carry a `code` or a tag, key on that instead of `name` — otherwise the day someone renames "Treasury" to "Group Treasury" your idempotent script creates a second one.

In the console: the organisation builder covers this ground — *Organisation* → *Builder*, with `api:POST /api/v1/organization-builder/validate` and `api:POST /api/v1/organization-builder/deploy` behind it, and progress at `api:GET /api/v1/organization-builder/deployment-status`. It is genuinely useful for *designing* a structure interactively and for showing a stakeholder what you propose. Export the result into a script before you rely on it.

Verifying what you built

Before moving on, assert the structure rather than eyeballing it. This is the kind of check nothing runs for you:

```
for u in await all_units(client):  
    assert u.default_classification != "public", f"{u.name} is public"
```

`default_classification` is a field on the returned unit model, so this reads the committed state rather than the value you believe you sent. That one assertion catches the failure mode this chapter opened with, and it costs a line.

Next: 02.3 — Envelopes, clearance and knowledge

02.3 — Envelopes, clearance and knowledge

Three separate controls that people routinely collapse into one. They answer different questions, they are checked at different moments, and a design that confuses them fails in a way that is hard to diagnose because each individual piece looks correct.

control	answers	attached to
envelope	what may this delegate <i>do</i> ?	a role, defined by its supervising role
clearance	what may this role <i>see</i> ?	a role
classification	how sensitive is this <i>thing</i> ?	a unit (as a default) and each knowledge item

Access is decided by clearance against classification. **Authority level does not enter into it** — a C-suite role with no clearance sees less than a staff role with clearance. This is the single most common modelling error on the platform.

Envelopes — the five constraint dimensions

An envelope is created by a **defining role** (the supervisor) and applies to a **target role** (the delegate):

```
envelope = await client.role_envelopes.create(
    defining_role_id=head_role_id,
    target_role_id=analyst_role_id,
    constraint_config={
        "financial": {"max_amount": 100_000, "currency": "USD"},
        "operational": {"blocked_actions": ["wire_transfer_external"]},
    },
    status="draft",
)
```

api:POST /api/v1/role-envelopes .

⚠ This write needs the exact scope `roles:write` on an API-key caller. The router admits any key holding SOME `roles:*` scope, but the route itself carries `require_scope("roles:write")` -- so `roles:read` does not stand in for it, and the refusal arrives on the call AFTER the role (and any agent auto-generated for it) already exists. If you are provisioning with a key, mint it with the set `examples/stand_up_a_vertical.py::REQUIRED_SCOPES` names before you start.

The five dimensions come from CARE and are the vocabulary to design in:

dimension	bounds
financial	spend — amounts, currency, budget
operational	which actions are permitted or blocked
temporal	when, and for how long
data access	which data may be reached

dimension	bounds
communication	who or what may be contacted

`status` defaults to `draft`, and `draft` does not enforce

`create` takes `status` and its default is `"draft"`. A draft envelope is a description of an intention. It is not in force.

```
await client.trust_posture.activate_role_envelope(envelope["id"]) # now it enforces
```

`api:POST /api/v1/role-envelopes/{id}/activate`, and `api:POST /api/v1/role-envelopes/{id}/suspend` to take it back out of force.

This is the failure mode to design against: a provisioning script that creates envelopes and never activates them produces an organisation that looks fully bounded in every listing and enforces nothing. Nothing errors. The `status` field is the only thing that distinguishes the two states, so assert on it:

```
envelopes = await client.trust_posture.list_role_envelopes(head_role_id)
for e in envelopes.records:
    assert e.status == "active", f"envelope {e.id} is {e.status}, not enforcing"
```

⚠ **defining_role_id is required — there is no org-wide listing.** The backend exposes list-by-supervisor and nothing else, so you enumerate the envelopes of the defining roles you provisioned, not every envelope in the organisation. A completeness check written as "list them all and assert" is therefore only as wide as the roles you named: pass each supervisor you created, and treat a role you forgot to pass as invisible rather than empty.

You can pass `status="active"` at creation instead. Prefer that when the envelope is fully specified, and use `draft` deliberately when a human is going to review it first — not by accident.

Envelopes tighten; they never widen

An active envelope is validated against the supervisor's own envelope, and the composition is **monotonic tightening** — an intersection, never a union. A delegate cannot be granted authority the defining role does not itself hold.

Two consequences worth designing around:

- **You cannot build an escalation path by adding an envelope.** Widening is not expressible. If a delegate needs to exceed its bounds, the answer is a human decision (02.6), not a second envelope.
- **Tightening a supervisor tightens everything beneath it**, immediately and without touching the children. That is a useful lever in an incident and a surprising one if you did not expect it.

`clearance_ceiling` on the envelope caps how far the target's clearance can carry inside this delegation, and `verification_defaults` sets how thoroughly its actions are checked.

Read envelopes back with `api:GET /api/v1/role-envelopes` and `api:GET /api/v1/role-envelopes/{id}`. For an agent rather than a role, the resolved picture is at `api:GET /api/v1/constraints/agents/{id}/effective` — and that is the one to trust when you are debugging, because it is the composition rather than any single input. `api:GET /api/v1/constraints/agents/{id}/inherited` shows what came down from above, and `api:POST /api/v1/constraints/agents/{id}/validate` checks a proposed change before you make it.

Clearance — what a role may see

Clearance is a separate axis, assigned per role:

```
await client.trust_posture.create_role_clearance(
    role_id=analyst_role_id,
    max_clearance="confidential",
    justification="Handles cash-position reporting for the Treasury desk.",
)
```

`api:POST /api/v1/role-clearances`. Note it lives on `client.trust_posture`, not on a `clearances` module.

Four rules the SDK and server enforce, worth knowing before you write the call:

1. **The levels are fixed** — `public`, `restricted`, `confidential`, `secret`, `top_secret`. Anything else raises `ValueError` client-side, before a request is made.
2. **justification is required by the server at `confidential` and above**. Omitting it fails server-side, which is a different error from the one above and reads differently.
3. **`secret` and `top_secret` require `compartments`**. Clearance at those levels is compartmented; a level alone does not grant sight.
4. **⚠ A new clearance lands in `vetting_status="pending"` and is not yet in force**. Approval and rejection are separate governance endpoints — `api:POST /api/v1/role-clearances/{id}/approve` and `api:POST /api/v1/role-clearances/{id}/reject`. This is the same shape as the draft envelope above: creating it is not granting it. A provisioning script that creates clearances and walks away has staffed an organisation whose roles cannot see their own material, and the symptom is an empty screen rather than an error.

The return value of `create_role_clearance` is the **raw persisted row**, in which `compartments_json` is a JSON *string* rather than a deserialised list. Parse it if you need it; do not index it as a list.

Knowledge — the governed material itself

```
policy = await client.knowledge.create(
    title="Treasury Approval Policy",
    content_markdown="# Approval Policy\nAll wires above $50k require sign-off.",
    knowledge_type="policy",
    path="/treasury/policies/approvals",
    classification="confidential",
    owner_unit_id=unit_id,
)
await client.knowledge.publish(policy["id"])
```

`api:POST /api/v1/knowledge`, then `api:POST /api/v1/knowledge/{id}/publish`.

- `knowledge_type` is one of `policy`, `procedure`, `reference`, `faq`.
- `classification` uses the same five levels; `secret` and `top_secret` require a `compartment`, mirroring clearance.
- `path` is a containment path and is how material is organised and addressed.

Creating knowledge does not publish it — a third instance of the same shape. The lifecycle has more stages than create-and-publish, and for anything that goes through review you will want them:

`api:POST /api/v1/knowledge/{id}/submit`, `api:POST /api/v1/knowledge/{id}/assign-reviewer`, `api:POST /api/v1/knowledge/{id}/approve`, `api:POST /api/v1/knowledge/{id}/request-changes`, `api:POST /api/v1/knowledge/{id}/reject`, and `api:POST /api/v1/knowledge/{id}/archive`.

Versioning is `api:POST /api/v1/knowledge/{id}/version` and `api:POST /api/v1/knowledge/{id}/create-revision`.

Sharing across a unit boundary is explicit and non-transitive

Units are knowledge boundaries. Material does not flow between them because two units are adjacent on the chart — it flows because someone wrote a share policy: `api:POST /api/v1/knowledge-share-policies`, listed at `api:GET /api/v1/knowledge-share-policies`, and suspended or reactivated at `api:POST /api/v1/knowledge-share-policies/{id}/suspend` / `api:POST /api/v1/knowledge-share-policies/{id}/reactivate`.

Sharing is not transitive. If A shares with B and B shares with C, C does not see A's material. Every hop is its own policy and its own decision. Designs that assume transitivity produce an access model that is more permissive on paper than in practice — which is the safe direction, but it means the capability you promised a stakeholder does not exist.

`api:GET /api/v1/knowledge-share-policies/due-for-review` is worth putting on a schedule; share policies are exactly the thing that gets created for a project and outlives it.

The three-way check, once

When a request to read something is evaluated, three things have to line up: the **role's clearance** (approved, not pending), the **item's classification** (plus compartment, at the top two levels), and any **share policy** if the reader is outside the owning unit. A failure in any one of them presents as "you

cannot see this", and the three are not distinguished in the message.

When you are debugging an unexpected denial, check them in that order — clearance first, because `vetting_status="pending"` is the most common cause and the cheapest to confirm.

In the console: clearances and envelopes appear under the governance and admin areas, and a held or refused access shows up in the approvals queue (02.6). The console is where the *reviewer* approves a pending clearance; the harness is where you create the fifty that need reviewing.

Next: 02.4 — Trust chains and postures

02.4 — Trust chains and postures

Two more things people collapse into one, and again they answer different questions:

- A **trust chain** answers *where did this agent's authority come from?* It is lineage — a genesis rooted in a human, and delegations down from there.
- A **posture** answers *how much of that authority may it use without a human in the loop right now?* It is a level, and it moves.

An agent can hold a valid chain and a restrictive posture. That is the normal state of a new agent and it is the state to design for.

Establishing a chain

```
chain = await client.trust.chains.establish(  
    agent_id=agent.id,  
    authority_id=head_role_id,  
    capabilities=["cash_forecasting", "reporting"],  
    expires_in_days=365,  
)
```

`api:POST /api/v1/trust/establish`, returning `sdk:aegis_sdk.EstablishedTrustChain`.

Four things about this call:

- **authority_id** is required and has no default. It names the organisational authority the delegation comes from.
- **Human origin is derived server-side from the authenticated caller and cannot be supplied by the client.** The `human_origin_data` parameter is accepted and ignored. This is not an oversight — *design intent, not observable*: an identity a client can assert is an identity a client can forge, so the one thing the audit trail must not be able to lie about is taken from the session instead of the body. Practically: **the human on the record is whoever's credentials ran the script.** Provision under a real, attributable identity, not a shared machine key, or your lineage records a service account as the origin of every delegation in the tenant.
- **constraints** is a list of strings, not a dictionary. It carries constraint *labels*; the actual bounds live in envelopes (02.3).
- **expires_in_days** defaults to 365. Chains expire. Put the renewal on a calendar at provisioning time, because the failure mode a year out is agents quietly losing authority.

Read chains with `api:GET /api/v1/trust/chains` and `api:GET /api/v1/trust/chains/{id}`; the delegation path for one is at `api:GET /api/v1/trust/chains/{id}/delegation-path`, and an agent's assembled trust context at `api:GET /api/v1/trust/agents/{id}/trust-context`.

Chains follow the reporting chain, not the unit tree. A role with no `reports_to_role_id` is a chain root. If your reporting lines are wrong (02.2), your lineage is wrong, and it will be wrong in the audit export rather than in anything that fails loudly.

Verifying

```
result = await client.trust.chains.verify(...)
```

`api:POST /api/v1/trust/verify` . This is the operation the enforcement path calls on the agent's behalf; you will mostly use it to answer "would this be allowed?" without doing the thing.

Postures — the five levels

Aegis uses the CARE-aligned vocabulary, lowercase:

level	posture	what it means
1	<code>pseudo</code>	no autonomous action; a human does everything
2	<code>supervised</code>	may read; every write is a human's
3	<code>shared_planning</code>	plans with a human, acts within the plan
4	<code>continuous_insight</code>	acts, with continuous human visibility
5	<code>delegated</code>	acts within its envelope without per-action review

An agent's effective autonomy is `min(agent posture, ceiling, selected)` — the lowest of what it holds, what its unit permits, and what was chosen for this piece of work. Raising one of the three does not raise the result; you have to find the binding one. `max_trust_posture` on a unit is the ceiling, and it cascades: a child cannot exceed its parent.

Read the current posture:

```
posture = await client.trust.postures.get(agent_id)
```

`api:GET /api/v1/agents/{id}/trust-posture` .

Moving a posture — three different operations, and they are not interchangeable

This is where the harness earns its place, because the three paths have different governance and the console blurs them into one button.

1. Request a progression — the governed path. The agent (or you on its behalf) asks to move up; a human approves.

```
result = await client.trust.postures.request_progression(...)
```

🚫 There is **no dedicated /request-upgrade endpoint** — this and `postures.override()` below both go through the SAME `api:PUT /api/v1/agents/{id}/trust-posture`, which decides internally whether the change applies at once or is held for a human. A resolution comes back through `api:POST /api/v1/agents/{id}/trust-posture/approve` or `.../reject`. Pending requests across the tenant are at `api:GET /api/v1/posture/pending-approvals`; one agent's pending request is at `api:GET /api/v1/agents/{id}/trust-posture/pending`.

The change is not always applied when this returns. Raising an agent above `supervised` is held for approval — branch on `result.approval_pending` before reporting the agent progressed; while it is `True`, `result.posture` is the posture the agent is **still running at**, not the one you asked for. This is the confirmed root cause of a partner-reported symptom: code that assumed the return always carried the new posture could not parse the approval-pending reply and raised *after the server had already accepted the request* — and a caller that then retries files a **second** pending request against the same agent, because the endpoint has no idempotency key. **Before retrying after an ambiguous failure** (a timeout, a dropped connection), call `get_pending_approval(agent_id)` to find out whether the first attempt already landed — `None` means it did not.

Approving is addressed by AGENT, and optionally by request.

`postures.approve_transition(agent_id, notes=..., approval_id=...)` sends that identifier to the endpoint as `request_id`.

⚠️ **This section previously said the endpoint had no such field. That was wrong, and it was wrong for a reason worth recording.** Earlier releases of this SDK wrote the identifier under the key `approvalId`. The endpoint declares `request_id` and reads nothing else, so the key was dropped on arrival and the call behaved exactly as if no identifier had been sent — including refusing as ambiguous when the agent had several requests pending. The symptom was real; the diagnosis in this book was not. If you tried to disambiguate before and concluded it does nothing, that was the client, not the endpoint. **The endpoint does address one request when you name it.**

With more than one request pending, name one. Omit the identifier while the agent has exactly one pending request and the call still works; omit it once it has more than one and the endpoint refuses rather than picking, which is the behaviour you want — a decision must not land on a request nobody selected.

Checking `get_pending_approval(agent_id)` before every `request_progression()` call and treating a non-`None` result as "already asked, do not ask again" is still good advice, and still the cheapest path: never letting a second request become pending means never having to disambiguate at all. It is no longer the *only* safe path, and it is not a workaround for a gap in the endpoint.

2. Override — set it directly.

```
await client.trust.postures.override(...)
```

`api:PUT /api/v1/agents/{id}/trust-posture` — the same endpoint as `request_progression()` above, and it carries the same two-outcome shape: a downward move (an emergency restriction) applies immediately, but an upward override above `supervised` is held for approval exactly like a normal progression request. Branch on `result.approval_pending` here too; do not assume an

override always takes effect at once. This is the one to be careful with regardless — it is legitimate (an incident, a migration, a deliberate decision) and it is also the operation that turns a governed progression into an assertion. Use it knowingly and record why.

3. Let the evidence decide. Aegis accumulates evidence about whether an agent has earned more autonomy, and will tell you:

- `api:GET /api/v1/agents/{id}/trust-posture/upgrade-eligibility` — is it eligible?
- `api:GET /api/v1/agents/{id}/trust-posture/evidence` — on what basis?
- `api:GET /api/v1/agents/{id}/trust-posture/metrics` — the underlying numbers
- `api:GET /api/v1/agents/{id}/trust-posture/evaluate` — evaluate progression now
- `api:GET /api/v1/agents/{id}/trust-posture/history` — how it got here

This is the intended workflow and it is the one most deployments never turn on, because overriding is quicker. The history endpoint is what an auditor will ask for; if every entry in it is an override, the gradient was decorative.

Design for the ratchet to move slowly. Start agents at `supervised`, let evidence accumulate, promote deliberately. An agent provisioned at `delegated` because it was easier has never been observed doing anything under supervision, so there is no evidence behind its autonomy — which is precisely the thing you would need to show.

Revoking — and the two shapes that are easy to confuse

```
result = await client.trust.chains.revoke(agent_id, reason="Security incident")
print(result.total_revoked)
```

`api:POST /api/v1/trust/revoke/{id}/cascade` .

⚠ **Revocation ALWAYS cascades. There is no non-cascading revoke.** The `cascade=` parameter is deprecated and ignored, and passing it raises a `DeprecationWarning` rather than changing anything. Revoking one agent revokes everything that derived authority from it.

Look before you cut. The impact is knowable in advance:

```
impact = await client.trust.chains.analyze_revocation_impact(agent_id)
```

`api:GET /api/v1/trust/revoke/{id}/impact` . Run this first, every time. It is the difference between removing one agent and silently removing a department's.

`api:POST /api/v1/trust/revoke/by-human/{id}` revokes everything originating from a particular person — the operation you want when someone leaves — and `api:POST /api/v1/trust/revoke-delegation` removes a single delegation.

There is no reversible pair — only revoke, which is not reversible

🛑 `client.trust.chains.suspend(agent_id, reason)` and `.reinstate(agent_id)` do not do this. Both are deprecation shims that **always raise `UnsupportedOperationError`** — there is no server route for suspending or reinstating a trust chain directly. `SUSPENDED` exists only as a lifecycle state a chain reaches automatically during cascade revocation of a bridge-sourced chain (see the callout below); it is not something you can put an agent into or take it out of on demand. If you need to stop an agent acting without permanently ending its chain, the honest tool today is `postures.override(agent_id, new_posture="pseudo", reason=...)` (the posture section above) — reversible, and a downward move applies immediately rather than waiting on approval. `revoke()` remains the only permanent, cascading removal, and it is genuinely not reversible.

⚠️ **A revoked chain that originated from a bridge can read back as `suspended`**. If you are writing a check that asks "is this revoked?" by comparing a status string, that check can miss a genuinely revoked chain. Do not rely on a status-only comparison for a security decision; use the revocation impact and audit surfaces, which describe what happened rather than what state a row is in. **UNVERIFIED** in this edition — the behaviour is reported and the distinction is real, but it could not be confirmed against a live deployment from here. Treat it as a lead: if you have a revoked bridge-origin chain to hand, check what its status field actually says before writing a check that depends on the answer.

The audit surface for trust decisions

```
await client.trust.audit.query(...)
```

`api:GET /api/v1/trust/audit`, filtered by originating human at `api:GET /api/v1/trust/audit/by-human/{id}`, and written to with `api:POST /api/v1/trust/audit`.

This is a different record from the general audit log (02.6). It is specifically the trust-decision trail — who delegated what to whom, and on what basis — and it is the one that answers a governance question rather than an operational one.

In the console: an agent's posture, its evidence and its history are visible on the agent's own page, and posture approvals join the same queue as other held decisions. Revocation is deliberately not a one-click action.

Next: 02.5 — Objectives and the work loop

02.5 — Objectives and the work loop

An **objective** is work you ask for. What comes back is not a result but a process: the objective decomposes into **requests**, some of which need a person, and agents run **sessions** against them.

```
objective -> requests -> sessions      and, wherever a human is needed,  
                                         a decision (02.6)
```

Creating one

```
objective = await client.objectives.create(  
    title="Reconcile the September cash position",  
    description="Pull balances from all accounts, reconcile against ledger, "  
        "flag discrepancies over $1,000.",  
    priority=7,  
)
```

`api:POST /api/v1/objectives`, returning `sdk:aegis_sdk.Objective`.

Points that are not obvious from the signature:

- **agent_id** is optional, and omitting it is meaningful. The server assigns the caller's delegate agent. That is usually right for a person submitting their own work and usually wrong for a provisioning script, which has no business delegate — name the agent explicitly when the caller is not the intended worker.
- **priority** is an int 0–10 here and a string tier on the wire. The SDK maps it to the server's low/medium/high/urgent tiers. So priority is coarser than it looks: 6 and 7 may be the same tier. Do not build scheduling logic on the integer.
- **There is no metadata bag.** Objectives have no persisted metadata field. An earlier client sent one and the server dropped it silently. If you need to correlate an objective with something in your own systems, put the correlation key in the title or description, where you can search for it.
- **selected_posture** lets the originator choose a posture for this work, bounded by `min(agent, ceiling, selected)` from 02.4. Choosing a *lower* posture than the agent holds is the useful direction — it is how you ask for a careful run of something ordinarily routine.

The lifecycle, and the states you will actually see

Objectives move through `draft` → `pending` → `in_progress` → `completed`, with `cancelled` and `failed` as terminal alternatives (`sdk:aegis_sdk.ObjectiveStatus`).

A created objective may need submitting before it starts: `api:POST /api/v1/objectives/{id}/submit`. Check the status rather than assuming creation started the work — this is the same create-is-not-activate shape as draft envelopes and pending clearances (02.3), and it is the third place in this part where the platform separates "described" from "in force".

Requests move through `pending` → `claimed` → `in_progress` → `completed`, with `escalated` and `cancelled` (`sdk:aegis_sdk.RequestStatus`). **escalated** is the one to watch: it means the work reached something it could not decide.

Watching work

```
progress = await client.objectives.get_progress(objective_id)
requests = await client.objectives.get_requests(objective_id)
decisions = await client.objectives.get_decisions(objective_id)
artifacts = await client.objectives.get_artifacts(objective_id)
```

`api:GET /api/v1/objectives/{id}/progress`, `api:GET /api/v1/objectives/{id}/requests`, `api:GET /api/v1/objectives/{id}/decisions`, `api:GET /api/v1/objectives/{id}/artifacts`, and the decomposition itself at `api:GET /api/v1/objectives/{id}/task-graph`.

Prefer the stream over a polling loop. `api:GET /api/v1/objectives/{id}/progress/stream` gives you progress as it happens. A poll on `get_progress` is the reflex and it costs you both latency and request budget for a worse signal.

Those four reads answer genuinely different questions, and knowing which one you want saves time:

you want to know	read
how far along is it	<code>progress</code>
what is it broken into, and what is stuck	<code>requests</code>
what judgments were made, and by whom	<code>decisions</code>
what did it produce	<code>artifacts</code>

`decisions` is the one architects under-use. It is the objective's own account of what was decided along the way, and it is where a surprising outcome is usually explained.

When work needs a person

Requests that need human action surface through the request surface:

```
await client.requests.claim(request_id)
await client.requests.complete(request_id, ...)
await client.requests.escalate(request_id, ...)
```

`api:POST /api/v1/requests/{id}/claim`, `api:POST /api/v1/requests/{id}/complete`, `api:POST /api/v1/requests/{id}/escalate`, `api:POST /api/v1/requests/{id}/release` to put it back, and `api:POST /api/v1/requests/{id}/findings` to attach what you found.

Claim before you work. A claimed request is one nobody else picks up; an unclaimed one being worked by two people is the ordinary way duplicate effort happens. And **release rather than abandoning** — a request claimed by someone who went on holiday is invisible to everyone else and

looks like it is being handled.

Sessions — what an agent actually did

```
session = await client.sessions.get(session_id)
messages = await client.sessions.get_messages(session_id)
```

`api:GET /api/v1/sessions/{id}`, `api:GET /api/v1/sessions/{id}/messages`, `api:GET /api/v1/sessions/{id}/artifacts`, `api:GET /api/v1/sessions/{id}/context`, `api:GET /api/v1/sessions/{id}/subagents`, and live at `api:GET /api/v1/sessions/{id}/stream`. Your own are at `api:GET /api/v1/sessions/my`.

Sessions can be paused and resumed — `api:POST /api/v1/sessions/{id}/pause`, `api:POST /api/v1/sessions/{id}/resume`, `api:POST /api/v1/sessions/{id}/end` — which is the humane way to stop something that is going wrong without revoking the agent's trust (02.4). Reach for pause first; revocation cascades and is not reversible.

⚠ A held action ends the run

Carried forward from 01.2 because it changes how you design work: under `delegated` posture a constraint violation **returns** rather than parking, so a held action terminates the agent's run instead of queuing it politely.

The practical consequence is that **long autonomous runs and tight envelopes compose badly**. If a run is going to touch the edge of its envelope, it will stop there, and everything after that point in the run does not happen. Either widen the envelope deliberately for that class of work, or decompose the objective so the boundary-crossing part is its own small request that can be approved and re-run without discarding an hour of progress.

Stopping work

`api:POST /api/v1/objectives/{id}/stop` and `client.objectives.cancel(...)`. Stop the objective rather than the agent when the problem is this piece of work; the agent is presumably fine and other work depends on it.

In the console: submitting an objective is chapter 05.2, watching it is the inbox in 05.3, and a stopped-and-waiting objective is 05.4. Those chapters are written for the person doing the work rather than the person who designed the system, and they are worth reading if you are deciding what that person will meet.

Next: 02.6 — Approvals, holds and evidence

02.6 — Approvals, holds and evidence

This is the chapter that closes the loop the whole product exists for. Aegis stops and asks a human; a human answers; the question, the answer and the reasoning become evidence someone else can inspect later.

The approvals queue

```
pending = await client.approvals.list_pending(agent_id=agent.id, limit=25)
for record in pending.get("records", []):
    await client.approvals.approve(
        record["id"],
        reviewed_by="user_head_treasury",
        reason="Within envelope; policy satisfied.",
    )
```

`api:GET /api/v1/approvals/pending`, then `api:POST /api/v1/approvals/{id}/approve` or `api:POST /api/v1/approvals/{id}/reject`. There is also `client.approvals.modify(...)`, which approves a changed version of what was asked — the middle answer between yes and no, and the one reviewers reach for most once they know it exists.

reason is not decoration. It is the part of the record that answers *why*, and it is the only part a future auditor cannot reconstruct from anything else. "Approved" tells them what happened; it does not tell them whether the decision was sound. Write the reason as though the reader is a stranger, because they are.

⚠ There is more than one queue, and this catches people

An approval is not the only thing that can be waiting on a person, and the different kinds surface in different places:

waiting on a person	where it is
an action held by the verification gradient	<code>api:GET /api/v1/approvals/pending</code>
a posture progression request	<code>api:GET /api/v1/posture/pending-approvals</code>
a pending role clearance	the clearance surface (02.3)
an escalated request	<code>api:GET /api/v1/agent-escalations/pending</code>
a pseudo-agent request needing a human to act	<code>api:GET /api/v1/pseudo-requests</code>
a change request against the organisation	<code>api:GET /api/v1/change-requests</code>

So **an empty approvals queue does not mean nothing is waiting**. This is a real source of confusion for operators, who reasonably read one empty queue as "all clear" and leave a posture upgrade or a pending clearance sitting for weeks.

If you build one operational check, build the one that sweeps all six. Nothing in the product does it for you.

Escalations have their own live channel at `api:GET /api/v1/agent-escalations/stream`, and are resolved with `api:POST /api/v1/agent-escalations/{id}/resolve` or cancelled with `api:POST /api/v1/agent-escalations/{id}/cancel`.

Change requests — governed change to the organisation itself

Structural change can go through review rather than being applied directly:

```
create    -> api:POST /api/v1/change-requests
submit    -> api:PATCH /api/v1/change-requests/{id}/submit
approve   -> api:PATCH /api/v1/change-requests/{id}/approve
deny      -> api:PATCH /api/v1/change-requests/{id}/deny
apply     -> api:PATCH /api/v1/change-requests/{id}/apply
```

There is also a counter-proposal path — `api:PATCH /api/v1/change-requests/{id}/counter-propose` and `api:PATCH /api/v1/change-requests/{id}/accept-counter` — which is how a reviewer says "not that, but this" without rejecting outright.

Note that approve and apply are separate steps. An approved change request has not changed anything yet. Same shape as draft envelopes, pending clearances and unsubmitted objectives — the fourth instance in this part, and by now it should read as the platform's consistent habit rather than a quirk: *describing* and *effecting* are always two operations.

For an architect this is the mechanism that lets org changes be reviewed the way code is. It is worth using for anything that touches envelopes or clearances, which are precisely the changes nobody notices until they matter.

The audit trail

Two surfaces, and they answer different questions.

The general audit log — operational, everything that happened:

```
logs = await client.observe_audit.list_logs(...)
history = await client.observe_audit.resource_history("agent", agent_id)
activity = await client.observe_audit.user_activity(user_id)
```

`api:GET /api/v1/audit/logs`, `api:GET /api/v1/audit/logs/{id}`, `api:GET /api/v1/audit/resources/{type}/{id}`, `api:GET /api/v1/audit/users/{id}`, exported at `api:GET /api/v1/audit/export`.

The trust audit — governance, who delegated what and why (02.4): `api:GET /api/v1/trust/audit`.

When someone asks "what did this agent do", the first is the answer. When they ask "on whose authority", the second is.

Evidence you can hand to a third party

This is the product's actual value proposition, so it is worth knowing the operations by name.

```
verification = await client.compliance.verify_audit(...)
export = await client.compliance.export_audit(...)
```

`api:GET /api/v1/compliance/audit/verify` — **verify the audit chain's integrity**, and `api:POST /api/v1/compliance/audit/export` to export it, with entries at `api:GET /api/v1/compliance/audit/entries` .

The verification operation is the one that matters and the one most operators never run. A tamper-evident trail is only evidence if someone has checked that it has not been tampered with; an unverified trail is a claim about itself. **Run the verification on a schedule and keep the result** — a verification you ran after an incident is worth much less than a series of them from before it.

The compliance module also carries SOC 2 evidence generation and export, HIPAA enablement and settings, retention policies (list, create, update, execute), and a dashboard at

```
client.compliance.get_dashboard() .
```

Retention deserves a deliberate decision

Retention policies decide how long the record survives. Set them knowingly: evidence you discarded to save storage is evidence you cannot produce, and the question is usually asked long after the decision. Conversely, a policy that keeps everything forever has its own regulatory cost. This is a decision for the organisation, not a default to inherit — but it is exactly the kind of default that gets inherited.

What the accountability actually rests on

The organisation deploying an agent keeps the accountability for what that agent does. It cannot be delegated, and no amount of autonomy transfers it.

What this chapter's surfaces give you is the **proof** that the accountability was discharged: the bounded mandate (02.3), the attestable lineage (02.4), the human decision with its reasoning (this chapter), and a trail whose integrity can be checked rather than asserted.

That proof is only as good as the habits behind it. Three that do most of the work, and none of which the platform will do for you:

1. **Write real reasons on decisions.** A queue of "approved" with no reasoning is a record that a person clicked, not that a person decided.
2. **Verify the chain on a schedule, and keep the results.**
3. **Sweep all six waiting-queues, not just the approvals one.**

You keep the accountability. This gives you the evidence.

In the console: the approvals queue is where a named human answers, and chapter 05.4 explains the same thing from that person's side — including why the approvals queue can be empty while something is genuinely held, which is the console-side face of the multiple-queues problem above.

This is the last chapter of part 02. Next: Part 03 — Extending the platform, or Part 04 — The API surface if you are debugging a call.

02.7 — Role agents, and surviving a re-org

The sequences that work, in the order the platform requires them, with the traps named where they bite. Everything here was derived by working a real deployment and then checking each finding against the platform source, because several things that *appeared* to be defects turned out to be old builds, and several things that appeared to work were doing something other than advertised.

Read 02.2 first for the org-provisioning order and 02.4 for what a trust chain is. This chapter is the step between them: the delegate agent that stands in for a human in a role, and what happens to it when the org chart moves.

The one idea that makes the rest make sense

A delegate agent belongs to the ROLE, not to the person sitting in it.

Once that is in view, most of the surprising behaviour stops being surprising. The agent survives an occupant leaving. It survives a new occupant arriving — its trust lineage is re-minted for the new seat. It does *not* survive the role being deleted, because the role is the thing it belonged to.

And the corollary, which is the single most expensive misconception here:

A role with no human in it is the ordinary case, not an error state.

Seats are routinely created before anybody is appointed. A deployment can legitimately run with every lead role vacant. If your build refuses to create, trust or activate an agent on a vacant role, see § "If your deployment refuses a vacant role" at the end — that is an old build, not a rule you must design around.

1. A team, and the role you did not ask for

```
unit = await client.units.create(
    name="Treasury",
    unit_type="department",
    default_classification="confidential",
)
```

Two traps, both silent.

default_classification must be one of five exact values — `public`, `restricted`, `confidential`, `secret`, `top_secret`. Plausible synonyms are rejected with a 422; `internal` is the one people reach for and it is not a value.

Creating a unit auto-creates its primary role, and that role arrives UNPARENTED —

`reports_to_role_id` is null, so it is a root of the reporting tree rather than a child of anything. Nothing warns you. Parent it explicitly:

```
await client.role_admin.update(primary_role_id, reports_to_role_id=parent_role_id)
```

An unparented lead role is not cosmetic. Trust chains follow the reporting structure (02.4), so a role that reports to nothing anchors its own chain instead of inheriting one.

2. A role, and its occupant — two calls, never one

`roles.create()` takes no `assigned_user_id`. Creating the seat and seating somebody in it are separate operations, in that order:

```
role = await client.roles.create(
    organization_unit_id=unit_id,
    title="Head of Treasury",
    authority_level=3,
)
await client.role_admin.assign_user(role["id"], user_id=user_id)
```

⚠ **Two role surfaces, and this chapter uses both.** `client.roles` carries `create`, `get` and `list` on `/api/v1/organization-roles` and returns untyped dicts — which is why `role["id"]` works above and would not if you reached for the typed client instead. Everything that *administers* a role that already exists — updating it, seating or vacating its occupant, linking or unlinking its agent — is on `client.role_admin`, the full `/api/v1/roles` surface, and returns typed models. A call written against the wrong one fails with `AttributeError` before it reaches the server.

You may legitimately stop after the first call. A seat with nobody in it is a valid, workable state — see § "Standing up a lead seat nobody occupies yet".

3. A role agent — one call, and be explicit about the model

```
result = await client.agents.create_delegate_agent(
    role_id=role_id,
    provider=os.environ["AEGIS_LLM_PROVIDER"],
    model_id=os.environ["AEGIS_LLM_MODEL"],
)
agent = result.agent
```

`api:POST /api/v1/delegate-agents`. **Prefer this over hand-rolling an agent with `is_shadow_agent=True`** (03.2 shows the hand-rolled form and why not): this call creates the agent, establishes the trust chain, and links the role in one operation. The hand-rolled version leaves you to remember the chain, and an agent without one cannot be activated.

Pass `provider` and `model_id` explicitly. They are optional in the signature and the server-side defaults are not guaranteed to be a runnable pair — on some deployments the default combination fails at provisioning time rather than at call time, which surfaces much later and much less clearly. Read both from your own configuration; never hardcode a model string.

Check whether the role already has one. `roles.create()` defaults to `auto_generate_agent=True`, so the seat may already be occupied by an agent. A role carries at most one delegate agent, and this call will not stop you from trying.

4. Promoting an agent — two principals, and not the endpoint you would guess

```
await client.trust_posture.update_trust_posture(
    agent_id, posture="supervised", reason="routine upgrade"
)
# ... then, as a DIFFERENT principal:
await client.trust_posture.approve_posture_transition(agent_id)
```

`api:POST /api/v1/agents/{agent_id}/trust-posture/approve`. Note that `update_trust_posture` returns a **union** — a completed transition when no approval was required, or an approval-pending envelope when it was. Branch on which you got; do not assume the write landed.

🚫 **Keep `reason` short and plain — this field is a live trap.** It is required, it is free text, and a descriptive sentence in it can be rejected by a web application firewall sitting in front of the deployment. What comes back is **raw HTML with a 403 and no JSON body**, which is indistinguishable at a glance from an authorization denial — so the natural conclusion is that you lack permission to promote, when in fact the gateway never forwarded the request. `"routine upgrade"` passes. **Before believing any 403 on this call, check whether the body is HTML.**

🚫 **Do not reach for `request-upgrade`.** It is gated on execution evidence, and a newly created agent has none — it cannot produce evidence until it is promoted, and cannot be promoted without evidence. For a new agent that gate is circular. The update-then-approve path is the one that works: it records the request, routes it for human approval, and returns a `202` on the deferred branch.

The approval must come from a second principal. The server derives both identities itself and refuses a self-approval, fail-closed, with a `400` naming the collision. This is a correct control — do not try to route around it by re-authenticating as the same person.

What "promoted" costs you, stated once: a posture change resets the agent's accumulated execution evidence. Two agents with identical configuration can sit in different evidence states purely because one was promoted more recently.

5. Moving a person between teams — move the ROLE

🚫 **Do not un-assign the person and re-assign them elsewhere.** Re-point the role, or move the role between units. `reports_to_role_id` is the safest field in this whole surface to change:

```
await client.role_admin.update(role_id, reports_to_role_id=new_manager_role_id)
```

The write is confined to the field you named. It does legitimately propagate — direct-report counts on both the old and new manager, access-cache invalidation, and a rewiring of the trust delegations between manager and subordinate agents. That propagation is correct and required; the reporting column and the trust edge have to agree.

⚠ **The trust rewiring is fire-and-forget.** If it fails, the call still returns `200` and the only signal is an `X-Warning` response header saying trust chains may need manual regeneration. **Read that header.** It is the one place this operation can half-succeed.

The safe recipe: UNLINK THE AGENT FIRST, THEN UNASSIGN THE USER

If you must separate a person from a seat, do it in this order:

```
await client.role_admin.unlink_agent(role_id, agent_id)    # 1. break the role -> agent pointer
await client.role_admin.unassign_user(role_id, user_id)    # 2. now vacate the seat
```

Unlinking first leaves the vacate step nothing to act on. Every cascade in this area is reached through the role's `shadow_agent_id` pointer; clear that pointer and the cascade has no agent to find.

Measured on a live deployment: two re-orgs done in the other order each needed a full repair sequence afterwards; the same move done unlink-first needed **zero**. It is the single most useful ordering in this chapter.

⚠ **This matters most on a build that predates the vacancy retraction**, where vacating actually did suspend the agent and its chain. On a current build the vacate step is far gentler (below) and the ordering costs you nothing. **Do it this way regardless** — it is correct on both, and you frequently do not know which build you are talking to.

🚫 **The one-way door this avoids, on a pre-retraction build.** Once a role has been vacated there, the two halves of the repair are **not** equally available:

repair	on a pre-retraction build
restore the agent's <code>status</code>	succeeds (200)
re-establish the trust chain	refused (422) — the role is vacant

So the agent comes back and its authority does not, and the only way to restore the chain is to seat a human first. That asymmetry is what makes an un-repaired cascade on a vacated role irreversible in practice, and it is why the ordering above is worth more than the repair sequence below.

If you must un-assign anyway

Un-assigning clears the occupant from the role and unbinds the agent from the departed person — and that is *all* it does on a current build. The agent stays active. Its trust chain stays live. The role keeps pointing at the agent, which is correct: the agent belonged to the seat, not to the leaver.

Assigning the next person then re-binds automatically: the stale chain is suspended and a fresh one minted for the new seat.

⚠ **That re-bind is best-effort and never raises.** A transient failure inside it is logged and swallowed, and the assignment still returns `200` — leaving the agent bound to the person who left. **Verify after every assignment** (§ 7). If the binding did not move, repeat the assignment; it is idempotent in the direction you want.

6. Un-linking an agent — know what it leaves behind

```
await client.role_admin.unlink_agent(role_id, agent_id)
```

This clears the role's pointer to the agent. **It does not touch the agent** — the agent remains `active`, and **its trust chain remains live while belonging to no role at all**.

That asymmetry is worth stating plainly, because the neighbouring operation disagrees with it: deleting a role *refuses* with a `409` rather than leave a live trust chain orphaned. Un-linking reaches the same state with no refusal and no warning. If you un-link deliberately, revoke the chain yourself or re-link the agent promptly.

To clear a field on an agent, send an empty string, not null. `PATCH` on an agent silently drops `null` values — the request succeeds with "No fields to update" and nothing changes.

`shadow_for_user_id=""` clears; `None` is a no-op that looks like a success.

7. Verifying anything — the list endpoint, and both sides

🚫 **The agent detail endpoint omits the binding fields.** `is_shadow_agent`, `shadow_for_user_id` and `human_role_id` read as `null` from `api:GET /api/v1/delegate-agents/{agent_id}` **whether or not they are set**. It is a serializer gap, not a data state, and it bites hardest during verification — precisely when you are trying to confirm a link you just made.

Use the list endpoint. It is the reliable read.

```
listing = await client.agents.list_delegate_agents(status="active")
```

`api:GET /api/v1/delegate-agents` .

After every role operation, assert all three of these — not one:

assert	why
<code>agent.human_role_id</code> points at the role	the agent's side of the link
<code>role.shadow_agent_id</code> points at the agent	the role's side — maintained by <i>different</i> code
a trust chain exists and is not suspended	a third store, with its own status

They are three separate records maintained by disjoint code paths with no reconciliation between them. Checking one and inferring the others is the single most reliable way to be wrong here. Read the chain with `api:GET /api/v1/delegate-agents/{agent_id}/trust-chain` .

Ordering rule, because it is counter-intuitive and enforced: **trust before status**. `api:POST /api/v1/delegate-agents/{agent_id}/activate` refuses with a `422` unless a chain already exists. Any sequence that activates first and establishes trust second fails on its first call.

8. What is irreversible — read this before deleting anything

Roles are recoverable. Agents are not.

Deleting a role is a *soft* delete by default: the role still resolves by id, still carries its occupant, and can be restored by setting its status back to `active`. A genuinely orphaned role id returns `404`, so "it still resolves" is a real signal, not a caching artifact.

🛑 **But deleting a role archives its delegate agent, and `archived` is TERMINAL.** The server enforces it — there is no transition out, so activation returns `422` forever. Worse, the soft-deleted role keeps its pointer to the agent, and the uniqueness check that guards linking counts archived roles too. So the agent ends up **both permanently dead and permanently reserved** by the tombstone role. There is no `unlink-from-archived-role` path.

Recovering the capability means provisioning a **new** agent against a **new** role. Plan role deletion accordingly.

⚠️ **Two further traps in the same family:**

- **`deactivate` also archives.** Despite the name, and despite platform-side documentation that has directed operators to it for "temporarily suspending" an agent, it reaches the same terminal state as delete. **There is no supported way to temporarily suspend a delegate agent** — no API route in the platform puts one into `suspended`.
- **Re-pointing `human_role_id` does not detach the agent from its old role.** The role's own pointer is untouched, so deleting that old role still archives the agent you thought you had moved. Un-link explicitly (§ 6) before deleting.

Standing up a lead seat nobody occupies yet

The common case, written out, because it is the one people assume is unsupported:

```
role = await client.roles.create(organization_unit_id=unit_id, title="Head of Treasury",
authority_level=3)
res = await client.agents.create_delegate_agent(role_id=role["id"], provider=prov, model_id=model)
await client.agents.activate_delegate_agent(res.agent["id"])
```

No occupant, at any point. The trust chain's human origin is **you** — the authenticated principal establishing it — not the role's occupant, which is why an empty seat removes nothing the platform needs.

When somebody is eventually appointed, `assign_user` re-binds the existing agent to them. You do not rebuild it.

If your deployment refuses a vacant role

A build that answers `422 "linked role is vacant"` on activation or on trust establishment predates the retraction of that rule. On such a build the refusal is a **one-way door**: an agent can be created on a vacant seat and never completed, sitting `draft` with no chain, indefinitely.

Do not work around it by seating a placeholder human — that writes a fictitious person into the audit trail, which is a worse problem than the one it solves, and one you will have to unpick later. Upgrade the deployment.

What this chapter does not tell you

- **Whether any of it is enforced the way it is described.** These are the sequences that work and the traps that bite, derived from a live deployment and checked against source. A route existing is not a control enforcing — 04.6 is the chapter about reading a response honestly, and it applies to every `200` above.
- **What your build actually does.** Several behaviours documented here changed recently, and the refusals you meet are evidence about your deployment's revision as much as about the platform. When this chapter and your deployment disagree, establish which revision you are on before concluding either is wrong.

Part 03 — Extending the platform

Audience: you are adding a capability. Part 02 was about describing and running a governed organisation with what is already there. This part is about giving it something new to do — a tool an agent can call, an agent configured for a job, a skill, a pipeline — and about everything that comes after "it works": who may call it, what happens when it fails, and how you take it out of service.

This is still harness work. There is no console path for authoring a tool.

#	Chapter	Read it when
03.1	Writing a tool an agent can call	You are adding a capability an agent will use
03.2	Agents, skills and pipelines	You are configuring who does the work
03.3	The tool-agent lifecycle	You are registering a shared capability
03.4	Who may call your tool	You are deciding which consumers may invoke it
03.5	Testing an extension	Before it touches a real organisation
03.6	Choosing between a tool, a skill, an agent and a pipeline	You are deciding what to build in the first place

Read 03.6 first if you have not decided what you are building. It is the shortest path from "I need to add X" to knowing which of six shapes X is, and the wrong choice is cheap to make and expensive to unwind.

The one distinction that runs through this whole part

"Tool" names two different objects here.

An **in-process tool** is a function inside the agent's runtime. It has no id, no status and no record of its own. Governance reaches it at the moment of the call, through a hook. Nothing needs to be registered, and that is the whole design.

A **registered tool agent** is a first-class platform object with an id, a lifecycle status, an accountable role, components, consumers, a derived envelope and an invocation history. Every step of its life is a governance event.

Choosing the first while describing the second to your stakeholders is the most common expensive mistake in this part: an in-process tool has no consumers list, no impact analysis and no invocation history, so none of the questions an auditor asks about it have mechanical answers. 03.1 opens with the full comparison.

The working artifacts this part hands off to

The chapters here are the explanation. The **working** half ships beside them, under `aegis_sdk/coc/`, and three of those artifacts are this part's subject:

artifact	use it when
<code>coc/skills/registering-a-tool-or-agent.md</code>	you are actually registering something — the order of operations, condensed
<code>coc/agents/aegis-sdk-specialist.md</code>	you are building against the client and want its brief rather than its prose
<code>coc/skills/diagnosing-a-refusal.md</code>	your extension was refused and you need to know which kind of refusal it was

Have the skill open while you work and this part open when you want to know *why*. Chapter 01.1 carries the full linked corpus, including the guardrails 03.4, 03.5 and 03.6 point at.

A chapter here and its companion skill are the same claim at two resolutions. If they ever disagree, report it rather than choosing between them.

Read 01.2 first if you have not

The most expensive misunderstanding on this platform is about *where* governance happens, and it decides how you write a tool. People assume Aegis sits between an agent and the outside world, receiving the agent's requests and forwarding the approved ones. It does not.

The practical consequence for this part: **your tool holds the credentials and does the work.** Aegis is not in the data path. If your design has Aegis making the outbound call, you are designing a component that does not exist.

Chapter 01.2 is short and it is the load-bearing one.

Two things about authorization to settle before you register anything

Both are covered in full in chapter 04.1. Named here because registration is a *write*, and the write surface is where they matter most.

- **Personas belong to sessions; scopes belong to keys.** A route gated on personas wants a user session, and a wider scope will not substitute for one. If you are debugging API-key authorization and your scopes look right, they probably are — 04.1 gives you a one-minute test that tells you which model a route follows.

- **A permission your role holds can still be refused.** Denials reach you as HTTP refusals on the routes you call, and the response names the permission and not what evaluated it — so re-granting the role is not the diagnosis and will not clear it. 04.1 gives you the order to work through instead.

Three defaults in this part that are not what you would guess

Each is documented where it lives; collected here because a default nobody chose is still a default you own.

- **A task agent's `posture_ceiling` defaults to `delegated`** — the *most* autonomous posture. Nearly every other default in this book is restrictive. 03.6.
- **An external agent's four budget and rate limits default to `-1`, meaning unlimited** — and `-1` compares as smaller than every threshold, so a naive numeric check fires loudest on the agent that has no limit at all. 03.6.
- **An application's `default_posture_ceiling` defaults to `pseudo`** — the most restrictive, which is right, and which surprises people whose grants then cannot exceed it. 03.4.

How this part is anchored

Claims name **surfaces you can reach** — an HTTP operation this client performs, a symbol this package exports — rather than coordinates in a source tree you were not given. Verify them against your own installed package:

```
python -m aegis_sdk.handbook.check
```

A resolving anchor tells you the operation exists in the surface you were given. Where a claim is about what the server **enforces**, this part says so explicitly — and chapter 03.2 shows you how to confirm a route end to end rather than inferring it from the client alone.

What this part is careful about

Aegis is Integrum's commercial implementation of four open standards — CARE, PACT, EATP and CO — published by Terrene Foundation under CC BY 4.0. Aegis implements them; it does not own them. Where this part uses standard terminology, it is using the Foundation's vocabulary deliberately.

A note on liability, because the distinction matters to how you build: the organisation deploying an agent keeps the accountability for what that agent does. It cannot be delegated, and no amount of autonomy transfers it. What this platform provides is the *proof* — the bounded mandate, the tamper-evident audit trail, the fail-closed gate, the attestable lineage — that lets that organisation show an auditor how it discharged the accountability it already had. Build accordingly: the artifacts you produce are evidence, and evidence that cannot be shown to a third party is not doing its job.

Next: 03.1 — Writing a tool an agent can call

03.1 — Writing a tool an agent can call

"Tool" names two different objects here, and the difference decides your design

Settle this before you write anything, because the two have almost nothing in common and choosing wrong is not a refactor — it is a rewrite.

An **in-process tool** is a function inside the agent's own runtime. The agent calls it directly. It has no id, no status, no record of its own, and it never appears in any listing. Governance reaches it at the moment of the call, through the hook described in [chapter 01.2](#). This chapter is about that one.

A **registered tool agent** is a first-class object on the platform —

`sdk:aegis_sdk.modules.tool_agents.ToolAgent`. It has an id, a lifecycle status, an accountable role, components, consumers, a derived envelope and an invocation history. It is created, granted, invoked and retired over the API, and every one of those steps is a governance event. [Chapter 03.3](#) is its lifecycle end to end.

	in-process tool	registered tool agent
is	a function in the agent's runtime	a platform object with an id
governed at	the tool call, by a hook	the invoke call, by the platform
you declare	nothing	name, accountable role, capabilities
it returns	whatever its caller expects	<code>sdk:aegis_sdk.modules.tool_agents.ToolAgentInvocationResult</code>
failure is	an exception in the agent's process	a status on a recorded invocation
retired by	deleting the code	<code>api:PATCH /api/v1/tool-agents/{id}/status</code>
callable by	whoever runs the agent	an application holding a grant (03.4)

The mistake to avoid is building the first while describing the second to your stakeholders. An in-process tool has no consumers list, no impact analysis and no invocation history, so none of the questions an auditor asks about it have mechanical answers. If the capability is shared, is called by more than one thing, or has to be evidenced, it wants to be a registered tool agent. [Chapter 03.6](#) is the fuller decision.

The short version, for an in-process tool

It needs **no Aegis-specific contract**. You write the tool. That is the whole obligation.

This surprises people who expect a governance platform to demand a policy declaration, an identity assertion, a capability manifest and a registration ceremony before it will let a tool run. Aegis asks for none of that, and the reason is structural rather than generous: the two things the gate needs, it already has without asking you.

Why the gate does not need your help

The tool's name comes from the call. The pre-hook receives the tool name and the tool input as arguments, from the invocation itself. Nothing needs to be declared anywhere for the gate to know which tool is being invoked — the invocation carries it.

That is worth pausing on, because it is also the security property: **the gate does not ask your tool what it is.** A tool cannot misreport its own name to obtain a softer verdict, because it is never consulted.

The identity is server-derived. It comes from the authenticated session — the organisation and user are read from the wrapper's own configuration and passed into the verification context, **never from anything the tool says about itself.** A tool cannot assert a different identity to widen its own envelope, because its assertion is not an input.

That second point is a security property, not an implementation convenience. If a tool could declare who it was acting as, the declaration would be forgeable by whatever is running the tool — which is precisely the party being governed. A managed party that self-reports its own identity has not been identified. So the platform does not accept identity from tools, and consequently there is no field for you to fill in.

The same reasoning is enforced on the HTTP side, where the identity of an approver is derived server-side from the session and never taken from a request body. Chapter 04.1 covers that surface.

What your tool receives

Two things, and only two: **the tool's name** and **the tool's input**.

That is the entire evidence base the verdict is made on, and it has three consequences worth designing around rather than discovering.

The verdict is about an intent, not an outcome. No response exists when the decision is made, because the call has not happened. A rule of the form "block this if the result contains customer data" is not expressible at this point in the path — there is nothing yet to inspect. Rules here are about *what you are about to do*, phrased over the tool name and the arguments.

Anything the gate can reason about must be visible in the arguments. A tool whose input is an opaque handle — a job id, a session token, a pre-built request object — hands the gate an argument list it cannot read. It is still governed by name, but no argument-level rule can apply to it, because the distinguishing detail sits on the other side of the handle. If you want the amount in a payment tool to be governable, the amount has to be an argument.

Whatever is not an argument is invisible. Configuration read from the environment at call time, a value cached from a previous call, a default buried in the tool's own module — none of it reaches the gate. Two calls that will do wildly different things can present identically.

```
# DO – the governable facts are arguments, so a rule can see them
async def transfer_funds(amount_usd: float, destination_account: str) -> dict:
    ...

# DO NOT – the amount is inside an opaque handle the gate cannot read
async def transfer_funds(request_handle: str) -> dict:
    ...
```

Design intent, not observable: that the argument-visibility property is deliberate rather than incidental. What is observable from your side is the hook's shape — name and input in, input out — and the consequence follows from it.

What your tool must return

For an in-process tool, Aegis imposes no return shape at all. Your tool returns whatever its own caller — the agent runtime — expects. There is no Aegis envelope to wrap, no status field to populate, no receipt to emit. If you find yourself designing a return type "so Aegis can understand it", stop: nothing here reads it.

For a registered tool agent the return shape is fixed, and it is worth studying even if you are building the in-process kind, because it is the clearest statement the platform makes about what it considers an invocation to consist of. `sdk:aegis_sdk.modules.tool_agents.ToolAgentInvocationResult` carries nine fields:

field	what it is
<code>content</code>	the output
<code>model</code>	which model produced it
<code>dispatch_path</code>	<code>llm</code> , or <code>builtin_tool</code> when no model was used
<code>usage</code>	token accounting
<code>cost</code>	what it cost
<code>trust_chain_id</code>	the chain the invocation ran under
<code>constraint_status</code>	how the constraints resolved
<code>verification_zone</code>	<code>auto_approved</code> , <code>flagged</code> , <code>held</code> or <code>blocked</code>
<code>audit_anchor_id</code>	the anchor the record is bound to

Note the ratio. **One field is the answer; eight are the provenance of the answer.** That is the platform's shape in miniature — the result is not the product, the evidenced result is. `verification_zone` is CARE's four-zone gradient, the same four values chapter 01.2 describes, arriving on a return type you can read from your own code.

When your tool fails

The in-process case and the registered case fail in genuinely different places, and conflating them produces a monitoring plan pointed at the wrong surface.

An in-process tool fails inside the agent's process. It raises; the agent's runtime sees the exception. Aegis is not in that path and does not learn what happened from the failure itself. Your tool's own logging and your own error handling are the primary record. Do not assume a failed tool call appears in a platform listing — nothing reachable from here promises that.

A refusal is not a failure, and the difference matters. If the pre-hook verdict denies the call, the tool never runs. Your tool sees nothing at all — not an error, not an invocation. From inside the tool, a refused call and a call that was never attempted are identical. So a tool that counts its own invocations is counting *permitted* invocations, and the gap between that number and what the agent attempted is not visible from where you are standing.

A registered tool agent fails against the API, where the failure is typed. Invoking one can raise:

- `sdk:aegis_sdk.GovernanceViolationError` — governance denied it. A budget was exhausted, an approval was required, a constraint refused. Retrying is wrong.
- `sdk:aegis_sdk.ServiceError` — including the fail-closed case where the governance service itself is unreachable. **The platform refuses rather than proceeding ungoverned**, which means an outage in the governance path presents as your tool being unavailable rather than as your tool running unchecked. That is the correct trade and it will still page someone at 3am.
- `sdk:aegis_sdk.NotFoundError`, `sdk:aegis_sdk.ValidationError` and `sdk:aegis_sdk.AuthorizationError` — the ordinary shapes.

Every one of these carries the HTTP status in `details["status_code"]`, and the status is the discriminator rather than the class — two statuses in common use map to no subclass at all and land on the base `sdk:aegis_sdk.AgenticOSError`. `coc/guardrails/error-taxonomy.md` has the full table; read it before you write an `except` clause.

What the platform does with a failing tool agent: it records the invocation and carries on. There is no automatic circuit breaker, no automatic deactivation, and nothing that quarantines a tool that has failed *n* times. If you want a failing tool taken out of service, that is your operational decision, expressed as `api:PATCH /api/v1/tool-agents/{id}/status` moving it to `suspended`. Chapter 03.3 covers the status machine and the one-way edges in it.

Versioning the contract, not just the code

An in-process tool's version is whatever you deployed. There is no platform registry to disagree with you, which sounds simple and is exactly why the hazard is easy to miss: **the governance around a tool is keyed on its name**.

So renaming a tool silently detaches it from every rule written about it. Nothing errors. Nothing warns. The rules still exist, still parse, and now describe a tool that no longer exists — while the renamed tool is governed by whatever the default for an unrecognised name happens to be.

```
# DO      keep the name stable and version the behaviour behind it, OR
#         land the new name and the rules referencing it in one change
# DO NOT  rename a governed tool and treat the rules as someone else's problem
```

The same reasoning applies with more force to a widened *signature*. Adding an argument that expands what the tool can do — a `force` flag, a destination that was previously fixed — changes the tool's reach without changing its name, so every rule keyed on that name now covers strictly more than it was written for. Treat a widening signature change as a governance change and re-read the rules.

For a registered tool agent the versioning surface is explicit and is covered in 03.3; for an agent's own configuration it is `api:POST /api/v1/agents/{id}/versions`, covered in 03.2.

When you *do* need to declare something

Finer-grained governance — rules about *categories* of action rather than about named tools — needs the platform to know what kind of effect a tool has. Reading a file and deleting a database row are both "a tool call"; a policy that treats them identically is not much of a policy.

Two vocabularies exist for that purpose, and they are the subject of the cautionary tale below:

- **The tool registry** — keyed by tool id, and its ids are **snake_case**: `read_file`. This is the catalogue of tools the platform knows about.
- **The posture permission set** — which names tools the way the agent runtime does, in **PascalCase**: `Read`, `Write`, `Edit`, `Glob`, `Grep`, `Bash`, `NotebookEdit`. This is what actually appears in a materialised allow/deny list.

The two are not the same vocabulary, and nothing translates between them for you. A tool id that looks correct in one is simply absent from the other.

If you are adding a tool that needs category-level governance, work out which vocabulary the consuming gate speaks *before* you write the declaration, and check that a producer actually emits in that vocabulary. That sentence is doing a lot of work, and the next section is why.

The cautionary tale: a vocabulary mismatch that disabled a gate

Status: fixed. What follows is a historical incident, not current behaviour. It is here because it is the most instructive failure this subsystem has produced, and because the shape of it is easy to recreate in your own code.

A fail-closed action-declaration gate resolved each tool's declared effect from the **snake_case** tool registry. The runtime, meanwhile, emits **PascalCase** names verbatim — the adapter passes the block's name straight through, and it arrives at the gate unchanged as `tool_name`.

There was no bridge between the two vocabularies. Measured at the time across the adapter layer:

```
snake_case tool id ('read_file')    -> 0 occurrences
the runtime's own name field        -> 2 occurrences    <- positive control
alias / normalise tables anywhere   -> 0 occurrences
```

Note the positive control in the middle line. Without it, the zero on the first line would be indistinguishable from a search that was simply looking in the wrong place — a check that cannot produce a different answer when the proposition is false is not evidence. That is a habit worth copying into your own checks, and this book uses it throughout. The measured effect of merging the gate unchanged:

```
Read -> HELD    Write -> HELD    Edit -> HELD
NotebookEdit / WebSearch / WebFetch / Task / TodoWrite -> HELD
Glob -> READ    Grep -> READ    Bash -> WRITE
```

`Read` is the most common call in the product, and under `delegated` posture the enforced-execution path **returns** on a constraint violation rather than parking the work — so an auto-claimed agent would die on its first file read.

Three things about this are worth carrying with you:

The gate was correct and the effect was catastrophic. The mechanism did exactly what a fail-closed gate should do. It held actions it could not resolve. Everything that went wrong went wrong upstream of it, in the fact that nothing produced declarations in the vocabulary the gate consumed.

A spot-check would have passed. Glob, Grep and Bash resolved correctly — but only because their lowercased spellings coincidentally equal a registry id. Three tools working is exactly the sample that makes a developer stop checking.

Fail-closed converted a benign bug into a total outage. The mismatch had existed all along and had been harmless, because everything fell through to an `else "read"` default. Tightening the gate turned a latent vocabulary disagreement into a shutdown. This is the general hazard of fail-closed design: it does not create the disagreement, it *reveals* it, all at once, in production.

The remedy was deliberately not improvised alongside the fix. Which vocabulary is canonical is a design decision on a path that fails closed, and it was settled separately rather than guessed at under time pressure. Declining to invent an answer under those conditions was the right call.

Practical guidance

For an ordinary in-process tool. Write it. Do not add Aegis-specific plumbing, do not accept an identity argument, do not try to pre-declare permissions. The hook will see the call.

Make the governable facts arguments. Anything a rule might need to reason about has to be visible in the input, because that is the whole of what the gate receives.

If your tool spawns a subprocess. Understand that it leaves the reach of every in-process control at the moment it spawns — see chapter 01.2 on the egress guard's stated coverage boundary. The pre-hook still fires on the tool call itself, so the *decision to spawn* is governed; anything the child process

subsequently does on its own is not.

If your tool holds credentials, it holds them alone. Aegis has no credential for the system you are writing to and never sees the response. Rotation, scoping and least privilege on that credential are yours, and they are not covered by anything on this platform.

If you need category-level governance. Establish, with a command whose output you can quote, that a producer emits declarations in the vocabulary your consumer reads. "The registry has an entry for it" is **not** that check — in the incident above the registry was fully populated and correct throughout. The registry was never the broken part; the join to it was.

If you are tightening a gate. Enumerate what the change will now hold, and measure it, before merging. In the incident above that table existed, which is why the outage was caught in review rather than in production — the enumeration is what caught it, not the intent behind the change.

Before you hand a tool to an agent

1. **Which kind is it?** In-process, or a registered tool agent? If it is shared, invoked by more than one consumer, or has to be evidenced, it is the second and 03.3 is your chapter.
2. **Are the governable facts arguments?** If the amount, the destination or the scope is inside an opaque handle, no argument-level rule can reach it.
3. **Does it spawn?** If so, govern the decision to spawn, because nothing here governs what happens afterwards.
4. **What does it do on failure?** Whose log receives the exception, and who reads that log?
5. **Is the name stable?** Every rule about this tool is keyed on the name you are about to freeze.
6. **Has it been exercised against something other than your production organisation?** [Chapter 03.5](#) covers how, and what each testing surface does and does not prove.

What is not settled

The posture→permission mapping and the action-category vocabulary are two different things and this chapter has been careful not to merge them. The posture permission set is a materialized allow/deny list written into an agent's own settings file (chapter 01.2); the tool registry is a catalogue of tool definitions with declared effects. They overlap in the tools they name and they disagree in how they spell them.

UNVERIFIED: whether a single canonical vocabulary has since been chosen and a bridge landed between the two. The PascalCase posture permission set is present and unchanged, and the fix — whatever form it took — is not annotated in a way this edition could trace. Do not infer from this chapter that the two vocabularies have been reconciled. Check before you rely on either.

UNVERIFIED: whether a *refused* in-process tool call is recorded anywhere an architect can read. The pre-hook has the information; whether it is durably written and retrievable is not settled from what is reachable here. If your compliance story depends on evidencing attempts that were blocked, establish this against your own deployment before you promise it.

Next: 03.2 — Agents, skills and pipelines

03.2 — Agents, skills and pipelines

Part 02 built the organisation and bounded it. This chapter is about what actually does the work inside those bounds — the agents themselves, the skills they carry, and the pipelines that sequence them.

Chapter 03.6 is the decision between these shapes. This one is the mechanics of each.

Agents

```
agent = await client.agents.create(
    name="Treasury Assistant",
    agent_type="chat",
    workspace_id=org_id,
    model_id=os.environ["AEGIS_MODEL"],
    system_prompt="You assist the Head of Treasury within policy.",
    capabilities=["cash_forecasting", "reporting"],
)
```

`api:POST /api/v1/agents`, returning `sdk:aegis_sdk.Agent`. Note this returns a typed **model** — `agent.id`, not `agent["id"]` — unlike the `standup` modules from 02.2. Listing is `api:GET /api/v1/agents`; the ones you can reach are at `api:GET /api/v1/agents/accessible`, returning `sdk:aegis_sdk.core.agents.AccessibleAgent` records.

Never hardcode a model name. Read it from configuration, as above. A model string baked into a provisioning script outlives the model.

agent_type is a narrower field than it looks

`sdk:aegis_sdk.AgentType` accepts nine values, and **only four of them are creatable**: `chat`, `task`, `pipeline`, `custom`. The other five — `shadow`, `pseudo`, `tool`, `esa`, and the forward-compatibility sentinel `unknown` — are minted server-side by internal generators and appear on *read*, never on *write*. The creatable subset is exposed as `sdk:aegis_sdk.types.CREATABLE_AGENT_TYPES`, and the SDK rejects a non-creatable value client-side before a request is made rather than round-tripping a doomed one.

The asymmetry is deliberate: a read model has to accept everything the server can persist, or `agents.list()` raises on any organisation that has a shadow agent in it. If you are writing code that branches on `agent_type`, handle the read values — you will meet them.

unit_type is a different domain from **agent_type** and they are easy to conflate.

`sdk:aegis_sdk.UnitType` is `atomic` or `composite`, the work-unit classification; `agent_type` is behavioural. Sending `agent_type="atomic"` is a mistake the type system will not catch for you, because both are strings on the wire.

`sdk:aegis_sdk.AgentStatus` carries six values — `draft`, `active`, `archived`, `deprecated`, `suspended`, `revoked` — and all six are reachable. An agent your listing has never shown you in `suspended` is one your organisation has not needed to suspend yet.

Delegate agents — the shape that matters for governance

A **delegate agent** stands in for a human in a role. This is the configuration that connects the agent to the org structure, and getting it right at creation is what makes trust, envelopes and audit line up later:

```
agent = await client.agents.create(
    name="Treasury Assistant",
    agent_type="chat",
    workspace_id=org_id,
    model_id=model_id,
    system_prompt="You assist the Head of Treasury within policy.",
    is_shadow_agent=True,
    shadow_for_user_id="user_head_treasury",
    human_role_id=head_role_id,
    organization_unit_id=unit_id,
)
```

The four fields at the bottom are the ones that matter. Without them you have an agent that exists but is not anybody's delegate: it sits outside the reporting chain, so trust chains do not flow to it (02.4), envelopes defined on its role do not reach it (02.3), and its actions attribute to nothing in particular in the audit trail.

Prefer the dedicated path, which takes the role and derives the rest:

```
result = await client.agents.create_delegate_agent(role_id=head_role_id)
```

api: `POST /api/v1/delegate-agents`, returning

`sdk:aegis_sdk.core.agents.DelegateAgentCreateResult` — which is `{agent, role, trust_chain}`.

It establishes the trust chain as part of the call, which is the real reason to prefer it: the hand-rolled version above creates an agent and leaves you to remember 02.4.

Two server-side constraints will refuse it, and each is a governance rule rather than a validation quirk:

- **The role must RESOLVE, and must belong to your tenant.** A `role_id` that names nothing, or names a role in another organisation, raises `sdk:aegis_sdk.ValidationError`. Note what this checks: that the role exists and is yours — not that anybody sits in it.
- **External and board roles receive governance-only agents**, not operational ones. A director's delegate can observe, approve and audit; it does not act.

A vacant role is ACCEPTED, and a previous revision of this page said otherwise

A role with no human occupant is the ordinary case, not an error state. Team lead seats are routinely created before anyone is appointed to them, and a deployment can legitimately run with every lead role vacant.

Earlier revisions of this page listed "the role must not be vacant" as a third refusal. **That rule is retracted** and the server no longer enforces it — neither on creation, nor on `POST /api/v1/trust/establish`, nor on activation. What survives is the resolvability check above, which is a different question.

The retraction matters because the old rule was a **one-way door**. Vacating a role suspended its delegate agent, and the gate then refused to bring it back until a human was seated — so a role whose occupant left could not be re-enabled, and a seat that was never filled could not be enabled at all. An agent could be created and never completed: permanently `draft`, with no trust chain.

If you are running against a deployment that still refuses a vacant role, you are on a build that predates the retraction. **Do not design around it** — the workarounds all involve seating a placeholder human, which puts a fictitious person in your audit trail. Upgrade the deployment instead.

Do not pre-check occupancy client-side. An SDK caller that refuses to call because `is_vacant` is true is blocking a request the API now accepts.

Activate and deactivate with `api:POST /api/v1/delegate-agents/{id}/activate` and `api:POST /api/v1/delegate-agents/{id}/deactivate`, and see them all at `api:GET /api/v1/delegate-agents`.

Recall from 02.2 that `roles.create(auto_generate_agent=True)` is the default, so a role may already have its delegate. Check before creating a second one — the call will not stop you.

Versions — cut one before you change a production agent

```
await client.agents.versions.create(agent.id, changelog="Tightened the refusal wording.")
```

`api:GET /api/v1/agents/{id}/versions` to list, `api:GET /api/v1/agents/{id}/versions/{v}` to read one, `api:POST /api/v1/agents/{id}/versions` to cut one, and `api:POST /api/v1/agents/{id}/versions/{v}/rollback` to go back — which returns the `sdk:aegis_sdk.Agent` at that version.

Cut a version before you change a production agent's prompt or configuration. Rollback is the cheap way out of a bad prompt, and it only exists if you made the version. The `changelog` argument is optional and you should always pass it: the version number tells you *that* something changed, and only the changelog tells you what you were trying to do.

This surface is one of the real differences between an agent and a tool agent (03.3), which has no versioning at all.

Contexts and tools

Contexts carry named configuration an agent works within: `api:GET /api/v1/agents/{id}/contexts` and `api:POST /api/v1/agents/{id}/contexts`, which takes a `name`, a `content_type` (`text`, `file` or `url`) and the `content` itself; `is_active` defaults to true.

Tools attached to an agent are at `api:GET /api/v1/agents/{id}/tools` and `api:POST /api/v1/agents/{id}/tools`, on `sdk:aegis_sdk.core.agents.AgentToolsModule`. The attach call takes a `tool_type`, a `name` and a `description`, plus an optional `config`:

```
await client.agents.tools.add(
    agent.id, tool_type="mcp", name="search", description="Search the docs.", config={}
)
```

`tool_type` is the shape of the tool — `mcp`, `function`, `api`. Attachments are updatable at `api:PUT /api/v1/agents/{id}/tools/{tool_id}` and removable at `api:DELETE /api/v1/agents/{id}/tools/{tool_id}`.

There is a second route to the same place: `tools_json` on the create payload, which carries a tool configuration array at creation time. Prefer the attachment calls for anything you will change later — an attachment has an id, and a JSON blob does not.

Running one

`api:POST /api/v1/agents/{id}/execute`, or `api:POST /api/v1/agents/{id}/execute/stream` for streamed output.

```
result = await client.agents.execute(agent.id, message="Summarise the position.")
```

This is a synchronous chat-completion call, and three properties follow from that. The body key is `message`, not `objective`. There is no wait-or-poll concept: the call either completes or raises. And `context` here is a dict injected into the agent's system prompt for this call — not the persisted `contexts` resource above, which is a different thing with a confusingly similar name.

`thread_id` is yours to supply for correlation; the server generates one if you do not. The streaming variant emits Server-Sent Events, each carrying its kind under the key `type`.



Correction to an earlier edition. This chapter previously stated that executions are listed at `api:GET /api/v1/agents/{id}/executions` and cancellable at `api:POST /api/v1/agents/{id}/executions/{eid}/cancel`. **Those calls return 404, and so does `api:GET /api/v1/agents/{id}/executions/{eid}`**. The SDK carries an explicit warning on each of the three. Agent execution is synchronous and is **not persisted as an individually-addressable record**, so there is nothing to poll, list, or cancel — the result comes back from `execute` and that is the only place it exists.

The methods remain on the client, which is why the earlier claim looked well-grounded: **a method existing is not the same as an operation being served**. That is the exact limit the handbook's own anchor check names — a resolving `api:` anchor proves this client declares the operation, never that the server answers it. Three anchors in one paragraph, all resolving, all 404.

For most work you do not call `execute` directly — you submit an objective (02.5) and let the platform dispatch. Direct execution is for testing and for cases where you are the orchestrator.

Test before you ship: `api:POST /api/v1/agents/{id}/test-draft` exercises a draft without putting it in front of anyone, and without persisting the draft. Chapter 03.5 covers it and what it does and does not prove.

`api:POST /api/v1/agents/{id}/duplicate` forks an agent — the right move for trying a variant without touching the original.

Skills

A skill is a reusable capability you attach to agents rather than re-describing per agent. It is **content**, not execution: markdown instructions plus a declared list of required tools.

```
skill = await client.skills.create(
    name="cash-forecasting",
    category="reasoning",
    description="How this organisation forecasts overnight cash.",
    content_markdown="# Cash forecasting\n1. Read the overnight position...",
    tools_required=["treasury_feed"],
)
await client.skills.assign_agent_skill(agent.id, skill.id, priority=1)
```

`api:POST /api/v1/skills` ; `api:POST /api/v1/skills/agents/{id}/skills` assigns and `api:GET /api/v1/skills/agents/{id}/skills` lists what an agent has.

category is required, and it is validated against a fixed set the SDK exposes as `sdk:aegis_sdk.core.skills.VALID_SKILL_CATEGORIES` — `coding` , `data` , `web` , `file` , `reasoning` , `custom` . The SDK checks it client-side and raises `sdk:aegis_sdk.ValidationError` with an actionable message rather than letting the server return an opaque 422. There is no `config` field on this surface, and the older `skill_type` vocabulary is not what the endpoint reads.

`priority` on the assignment is an ordering, `0` being highest, and must be `>= 0` . Assigning the same skill twice raises rather than silently re-ordering.

The skills module is the fuller CRUD surface in this part — `list` , `create` , `get` , `update` , `delete` , and `get_by_name` . `list` filters on `category` , `is_public` and a `search` substring.

`get_by_name` returns `None` rather than raising when nothing matches, which makes it the obvious primitive for an idempotent provisioning script:

```
skill = await client.skills.get_by_name("cash-forecasting")
if skill is None:
    skill = await client.skills.create(name="cash-forecasting", category="reasoning", ...)
```

That is the create-if-missing pattern 02.2 had to hand-roll for units. ⚠ **But it is a client-side scan over a bounded listing, not a server-side lookup**, so in an organisation with more skills than that window an existing skill can sit outside it, `get_by_name` returns `None` , and the script above creates a duplicate. Nothing errors. 03.6 has the full treatment and the mitigation — keep your own name→id registry once you are past a handful.

`duplicate` is the honest way to fork an **agent** for a variant (`api:POST /api/v1/agents/{id}/duplicate`) rather than editing the shared one and surprising everyone it was assigned to.

⚠ **Skills have no fork operation.** There is no skills duplicate route, so `skills.duplicate()` raises `sdk:aegis_sdk.UnsupportedOperationError` rather than answering a 404. To fork a skill, `get` it and `create` a new one from the same fields.

Pipelines

Pipelines compose multi-step work: `client.pipelines`, with `sdk:aegis_sdk.Pipeline`, `sdk:aegis_sdk.PipelineNode` and `sdk:aegis_sdk.PipelineConnection` describing the graph, and `sdk:aegis_sdk.PipelinePattern` naming the composition shapes available:

```
sequential · parallel · conditional · iterative · fallback  
routing · ensemble · saga · event_driven
```

`api:POST /api/v1/pipelines` creates one, `api:GET /api/v1/pipelines` lists them.

⚠ The create path and the graph-save path take different node shapes

This is the one thing in this section most likely to cost you an afternoon.

`pipelines.create(nodes=[...])` builds `sdk:aegis_sdk.PipelineNode`, which requires `id`, `name` and `node_type`. `api:PUT /api/v1/pipelines/{id}/graph` — the editor's save path — takes nodes requiring `node_type` and `label`, with `id` optional, plus optional `agent_id`, `position_x`, `position_y` and `config`.

```
# create() — needs id + name + node_type  
{ "id": "n1", "name": "Research", "node_type": "agent", "agent_id": "agent_1" }  
  
# save_graph() — needs node_type + label  
{ "id": "n1", "label": "Research", "node_type": "agent", "agent_id": "agent_1" }
```

A node dict that works in one is rejected by the other, and the difference is a single field name. Build the graph one way and stay with it.

`save_graph` **replaces the whole graph wholesale** rather than merging, so read before you write if anything else might have edited it. A cross-pipeline id collision surfaces as a 409, which the SDK raises as `sdk:aegis_sdk.ValidationError` rather than an opaque 500 — a useful, specific error, and worth recognising.

Connections take `source_node_id` and `target_node_id`, with optional `source_handle`, `target_handle` and a `condition`.

Validating and running

```
report = await client.pipelines.validate(pipeline.id)  
result = await client.pipelines.execute(pipeline.id, inputs={"topic": "cash"})
```

`api:POST /api/v1/pipelines/{id}/validate` returns a raw dict — `{"valid": bool, "errors": [...], "warnings": [...]}` — and checks node connectivity, missing configuration, circular dependencies and agent availability. **Run it before every execution of a graph you have edited**; it is the cheapest check in this part.

🚫 `report["valid"]` being `True` does not mean the graph is free of every real risk. By deliberate severity contract, several defect classes are reported as `warnings` rather than `errors`, and a warning never flips `valid` to `False`:

- a node bound to an agent id with **no trust posture configured at all** (fail-closes to `pseudo` at execution — refused, but for a different reason than a posture deliberately set low)
- a node bound to an agent **already at `pseudo`** (refused at execution; the graph did not change, the agent's posture did)
- an approval gate with **no outbound edges at all** (indistinguishable from an unfinished draft, so it is not condemned as broken)
- an approval gate with **no timeout configured** (an unanswered approval holds the run open indefinitely)
- an **agent node with no outbound edge** (a legitimate terminal step, but also indistinguishable from a graph nobody finished wiring — a run ending there records no outcome)

Two related classes ARE hard errors and do flip `valid` to `False`: a node type with **no execution handler that the run can actually reach**, and an agent binding that **resolves to no agent in the organization** — the second of these previously read back as a silent `valid: true` and is the confirmed root cause of the partner-reported symptom *"validate returns `valid: true` ... over graphs carrying ... an agent id that resolves on no agent route"*.

If your acceptance criterion is "nothing could possibly go wrong", check `report["warnings"]` too, not only `report["valid"]`. A pipeline that validates clean can still refuse at execution time on a posture that changed between the two calls — posture is read from a mutable store, not from the graph — which is why that class is a warning rather than an error: the graph itself did nothing wrong.

`execute` starts the run at `api:POST /api/v1/executions/start`, which answers only once the run has finished, then reads the run record back from `api:GET /api/v1/pipelines/{id}/executions/{eid}`. It returns `sdk:aegis_sdk.PipelineExecution`, carrying `status` (a `sdk:aegis_sdk.ExecutionStatus`), `outputs`, `error`, `node_results`, `duration_ms`. `node_results` is the field to read when a pipeline half-worked: it is per-node, so it tells you *where* it stopped rather than only that it did.

Unlike agent executions, **pipeline executions are individually addressable**: `api:GET /api/v1/pipelines/{id}/executions` lists them and `api:POST /api/v1/pipelines/{id}/executions/{eid}/cancel` cancels one. The listing's `status` filter is applied by the SDK across every page, because the route itself does not filter. `execute`'s `wait` argument is deprecated: the server has no asynchronous mode, so it changes nothing, and passing it warns.

`api:POST /api/v1/pipelines/{id}/duplicate` forks a pipeline, which is how to try a change to a graph other work depends on.

Reach for a pipeline when the *shape* of the work is fixed and known in advance. Reach for an objective (02.5) when it is not — objectives decompose themselves, and a pipeline that encodes a decomposition the platform would have derived is a maintenance burden you chose.

Tool agents and task agents

Two further kinds, each with its own surfaces and its own governance. Both get full treatment elsewhere in this part; the summary here is so you recognise which is which.

- **Tool agents** — `api:GET /api/v1/tool-agents`, `api:POST /api/v1/tool-agents`, composed of components (`api:POST /api/v1/tool-agents/{id}/components`) and invoked at `api:POST /api/v1/tool-agents/{id}/invoke`. Two reads matter before you change one: `api:GET /api/v1/tool-agents/{id}/consumers` tells you who depends on it, and `api:GET /api/v1/tool-agents/{id}/impact` tells you what breaks if you change it — including transitive composites. Its bounds are summarised at `api:GET /api/v1/tool-agents/{id}/envelope-summary`. Full lifecycle: 03.3. Who may call it: 03.4.
- **Task agents** — `api:GET /api/v1/task-agents`, `api:POST /api/v1/task-agents`, with `api:POST /api/v1/task-agents/{id}/test`. A delegation target another agent hands work to. **Its posture_ceiling defaults to delegated** — the most autonomous posture, and the one permissive default in this part. Set it down deliberately; 03.6 has the detail.

Check consumers and impact before editing a shared tool agent. It is the same discipline as revocation impact in 02.4: the operation that tells you the blast radius exists, it is one call, and the alternative is finding out afterwards.

External agents — the boundary case

`api:GET /api/v1/external-agents`, `api:POST /api/v1/external-agents`, invoked at `api:POST /api/v1/external-agents/{id}/invoke`, with an invocation history at `api:GET /api/v1/external-agents/{id}/invocations`.

These live on `client.integrations` (`sdk:aegis_sdk.IntegrationsModule`), not on a module of their own — worth knowing, because looking for `client.external_agents` finds nothing.

They reach something outside your deployment, so they carry credentials and budget of their own. Treat registering one as a security change rather than a configuration change: it is the point where your governed organisation acquires an edge to a system it does not govern. **Their four limit fields default to -1, meaning unlimited** — 03.6 covers the sentinel and why you must never compare it numerically.

Drift

Agents drift — behaviour moves away from what was configured and approved. `api:GET /api/v1/agentic/agents/{id}/drift` for one agent, `api:GET /api/v1/agentic/drift/alerts` for the tenant, with `api:POST /api/v1/agentic/agents/{id}/drift/escalate` and `api:POST /api/v1/agentic/agents/{id}/drift/recover`.

This is a monitoring surface most deployments never wire up, and it is the one that answers "is this agent still doing what we approved?" — a question the trust posture (02.4) assumes someone is asking, and which no test suite can answer for you (03.5).

Next: 03.3 — The tool-agent lifecycle

03.3 — The tool-agent lifecycle, definition to retirement

Chapter 03.1 drew the line between an in-process tool and a **registered tool agent**. This chapter is the second one, end to end: what you declare, what the server fills in, how it becomes callable, what an invocation gives you back, and how you take it out of service without breaking whatever was depending on it.

Everything here is `client.tool_agents`, whose module type is `sdk:aegis_sdk.ToolAgentsModule`. Thirteen operations, and the order you use them in matters more than any individual one.

The working checklist is `coc/skills/registering-a-tool-or-agent.md`. That is the artifact to have open while you register something — the order of operations, condensed, with the places the order matters. This chapter is the explanation behind it: what each object is, what the server decides for you, and what each step's evidence does and does not establish. If the two ever disagree, one of them is wrong — say so rather than picking.

The shape of the object

A tool agent is `sdk:aegis_sdk.modules.tool_agents.ToolAgent` — a typed model, so `agent.id`, not `agent["id"]`. The fields that decide its behaviour:

field	why it matters
<code>id</code>	everything downstream keys on it — grants, policies, invocations
<code>status</code>	which lifecycle state it is in; gates whether it can be invoked
<code>registered_to_role_id</code>	the accountable role ; see below
<code>capabilities_json</code>	how a grant flow matches it to a request
<code>composition_role</code>	<code>standalone</code> , or a role within a composite
<code>parent_composite_id</code>	set when this agent is a component of another
<code>consumer_count</code>	how many applications currently depend on it
<code>model_id</code> , <code>temperature</code> , <code>max_tokens</code> , <code>system_prompt</code>	its own inference configuration

Two of those carry more weight than their names suggest, and they are where registrations go wrong.

Step 1 — Create, and understand what the two required arguments are for

```
agent = await client.tool_agents.create(
    name="Cash Position Summariser",
    registered_to_role_id=head_of_treasury_role_id,
    capabilities=["cash_forecasting", "summarise"],
    description="Summarises the overnight cash position from the treasury feed.",
    model_id=os.environ["AEGIS_MODEL"],
    system_prompt="Summarise a cash position. Never advise on a transaction.",
)
```

`api:POST /api/v1/tool-agents`, returning `sdk:aegis_sdk.modules.tool_agents.ToolAgent`.

registered_to_role_id is required, and it is the accountability anchor — not a piece of metadata. It names the role that owns this capability. The grant flow (03.4) matches approvals against it, and the audit trail attributes the agent's actions through it. An agent registered to the wrong role is not mislabelled; it is accountable to the wrong person, and everything downstream — who approves a grant, whose envelope bounds it, whose name is on the record — follows the wrong chain. Pick it deliberately.

capabilities must be non-empty, and the server refuses an empty array. The reason is worth internalising: a zero-capability agent cannot be matched by the grant-approval flow, so it is an agent nothing can ever be authorised to call. The refusal is the platform declining to create an object that could not work. The SDK accepts either a `list[str]` or a pre-serialised JSON array string and serialises the list for you.

Read the returned object rather than assuming it echoes your input. The server fills defaults — `temperature`, `max_tokens`, `composition_role`, and the initial `status`. A default you did not choose is still a default you now own.

It is created in **draft**. Nothing can invoke it yet. That is deliberate and it gives you the next two steps.

Step 2 — Components, and why they go before the status move

A tool agent can be **composite**: built from other tool agents.

```
await client.tool_agents.add_component(
    parent.id,
    component_agent_id=child.id,
    role="summariser",
    version_constraint=">=1.2",
)
components = await client.tool_agents.list_components(parent.id) # confirm; do not assume
```

`api:POST /api/v1/tool-agents/{id}/components`, `api:GET /api/v1/tool-agents/{id}/components`, and `api:DELETE /api/v1/tool-agents/{id}/components/{dep}` to remove one. **Cycle detection is enforced server-side**, so a composition that would loop is refused rather than accepted and discovered at invocation time.

Wire components before you move the status, not after. A status transition is a governance event: it is recorded, and the record asserts something about the agent as it stood at that moment. Moving an incomplete agent to `active` and then completing it means the transition was recorded against a shape that did not yet exist. The record is not wrong about the transition; it is wrong about what was transitioned. Land the shape, then move the status.

`version_constraint` is worth using even when there is only one version. It is the declaration that makes a later component upgrade a decision rather than a surprise.

Step 3 — Read the envelope, and read it as a bound rather than a description

```
env = await client.tool_agents.get_envelope_summary(agent.id)
```

`api:GET /api/v1/tool-agents/{id}/envelope-summary`, returning
`sdk:aegis_sdk.modules.tool_agents.ToolAgentEnvelopeSummary`:

field	what it tells you
<code>trust_posture</code> / <code>posture_source</code>	the posture in force, and where it came from
<code>unit_ceiling</code> / <code>unit_ceiling_source</code>	the ceiling the containing unit imposes, and its origin
<code>budget_allocated_usd</code> / <code>budget_used_usd</code>	the spend envelope and what is left of it
<code>active_capabilities</code> / <code>total_capabilities</code>	how many of its declared capabilities are live
<code>must_rules</code> / <code>must_not_rules</code> / <code>should_rules</code>	the constraint counts in force
<code>constraint_sources</code>	a list of <code>sdk:aegis_sdk.modules.tool_agents.ConstraintSource</code> , each naming a level and its contribution

The `_source` fields are the most useful thing on this object and the easiest to skim past. They answer "why is this the posture?" — which is the question you actually have when a posture is not what you set. `constraint_sources` goes further: it decomposes the envelope by the inheritance level each constraint came from, so you can see that a bound you did not set came from the unit above.

The envelope is derived, not declared. It composes from what contains the agent, so it can be tighter than anything you set directly, and it can change without anyone touching this agent — a supervisor tightening their own envelope tightens this one immediately (02.3). Two consequences:

```
# DO      re-read the envelope after ANY change to what contains the agent
# DO NOT  cache it at registration and treat it as a property of the agent
```

A cached envelope is a claim about the past. On this platform, that specific claim goes stale by design.

Step 4 — Move the status deliberately, because some edges are one-way

```
await client.tool_agents.change_status(agent.id, "active")
```

`api:PATCH /api/v1/tool-agents/{id}/status`. The SDK mirrors the server's allowlist client-side as `sdk:aegis_sdk.modules.tool_agents.ALLOWED_TOOL_AGENT_STATUS_TARGETS`, so an invalid target raises before a request is made:

```
active · suspended · deprecated · revoked · archived
```

draft is not in that set, and its absence is the design. **draft** is initial-only: it is where an agent starts and it is never a valid transition *target*. You cannot put an agent back into draft to "unpublish" it and rework it quietly. There is no undo edge.

The practical reading, for the five targets that do exist:

- **active** — invocable, subject to grants and governance.
- **suspended** — temporarily out of service. This is the reach for a misbehaving tool; it stops invocations without asserting the agent is finished.
- **deprecated** — still there, signalling that consumers should move off.
- **revoked** — withdrawn. Reach for this when the agent should not have been callable.
- **archived** — retired.

The server enforces the transition machine, so an invalid move raises rather than silently landing. Plan the path before you take the first step.

Step 5 — Invoke it, and be exact about what a success proves

```
result = await client.tool_agents.invoke(  
    agent.id,  
    message="Summarise last night's cash position.",  
    application_id=app.id,  
)
```

`api:POST /api/v1/tool-agents/{id}/invoke`, returning `sdk:aegis_sdk.modules.tool_agents.ToolAgentInvocationResult` — the nine-field shape 03.1 tabulates, of which `verification_zone`, `constraint_status`, `trust_chain_id`, `cost` and `audit_anchor_id` are the governance half.

Passing `application_id` changes what the call means. With it, the invocation is made *on behalf of an application*, and the platform requires that application to be active and to hold an active grant for this agent; the application's budget is what gets consumed. Without it, you are invoking as yourself. Chapter 03.4 is the grant model, and it is the difference between a tool you can call and a tool your organisation can call.

Two independent 30-second clocks bound how long this call can take, and they are configured in different places. The server's own tool-execution default is 30s; the SDK client's own request timeout also defaults to 30s and applies to connect, write, read and pool-wait alike (see 04.5 before raising it for a slow tool — a single higher number lengthens all four, including how long an unreachable deployment takes to say so). A slow tool can exhaust either clock, and the two failures look different: the server's own timeout comes back as an ordinary response your typed result models, while the client's own timeout raises `sdk:agis_sdk.TimeoutError` before any response is parsed at all. If invocations of a tool that should be fast are timing out, raise the **client's** timeout first (on a second, dedicated client — 04.5's pattern) and re-measure — a partner report of a 180-second stall against a nominally-trivial built-in tool was traced this far and no further: the platform's own tool-execution path resolves the same 30s default, not 180s, and whether the SDK's own bundled tool set even contains the tool in question was not established from here. Treat a timeout on a "trivial" tool as a measurement to keep pursuing, not a known bug with a known fix.

A successful invoke proves the path is live. It does not prove the agent is correct, and it does not prove anything was governed. Confirm the record separately:

```
history = await client.tool_agents.list_invocations(agent.id)
```

`api:GET /api/v1/tool-agents/{id}/invocations` . **An empty invocation list after a successful invoke is a finding, not a formality** — it means the call happened and the record did not, and the record is what an auditor will ask for.

Read `verification_zone` on every result you care about. A `flagged` invocation succeeded *and* was surfaced for attention; treating it as a plain success discards the signal the gradient exists to give you.

Step 6 — Before you change or retire it, ask who is depending on it

Two reads, one call each, and they answer different questions.

```
consumers = await client.tool_agents.list_consumers(agent.id)      # who holds a grant
impact = await client.tool_agents.get_impact(agent.id)            # what breaks
```

`api:GET /api/v1/tool-agents/{id}/consumers` lists the applications granted access, filterable by `active`, `revoked` or `all`. `api:GET /api/v1/tool-agents/{id}/impact` is the stronger one: it reports direct composites, **transitive** composites, and affected applications — the blast radius including the composites you did not know this agent was inside.

Run the impact read before deprecating, revoking, or changing a shared tool agent. It is the same discipline as revocation impact in 02.4: the operation that tells you the blast radius exists, it is one call, and the alternative is finding out afterwards from someone whose workflow stopped.

`consumer_count` on the agent record is a cheap pre-filter — a zero there means nothing currently holds a grant — but it counts consumers, not composites, so it is not a substitute for the impact read.

Updating a live tool agent

```
await client.tool_agents.update(agent.id, system_prompt="...", max_tokens=8192)
```

`api:PUT /api/v1/tool-agents/{id}` takes any of `name`, `description`, `composition_role`, `capabilities_json`, `tools_json`, `model_id`, `system_prompt`, `temperature` and `max_tokens`.

Two of those are governance changes wearing the clothes of configuration:

Changing `capabilities_json` changes what grants can match. Capabilities are how the grant-approval flow finds this agent. Narrowing them can silently strip an agent out of a matching set; widening them can make it eligible for authorisations nobody intended. Re-read the consumers after either.

Changing `system_prompt` changes behaviour with no version to roll back to. Unlike an ordinary agent (03.2), a tool agent has no versions endpoint — there is no `versions` list and no rollback for this object type. If you need the ability to return to a known prompt, keep it yourself, in the code that provisions the agent, under source control. This is the strongest argument in this part for provisioning tool agents from a script rather than by hand.

Retiring one without breaking its consumers

The order that does not surprise anyone:

1. `get_impact` — establish the blast radius, including transitive composites.
2. `change_status(..., "deprecated")` — signal, do not sever. Existing consumers keep working while they migrate.
3. **Migrate the consumers** — revoke their grants once each has moved (03.4).
4. `change_status(..., "revoked")` or `"archived"` — sever.

Skipping straight to `revoked` on an agent with live consumers works exactly as designed and breaks exactly what the impact read would have named.

A checklist before you call a registration done

- **Re-read the agent** and compare it to what you intended, field by field, not by eyeballing the shape. The server filled defaults you did not choose.
- **Confirm the envelope is the bound you meant**, and that you understand where each `_source` points.
- **Confirm one invocation is recorded**, not merely that one succeeded.
- **Confirm the accountable role is right**. This is the hardest one to change later and the easiest to get wrong now.
- **Say which credential did the registration**. An agent registered by a key and one registered by a session are the same object with different provenance, and the provenance is what the audit trail carries. Check yours with `api:GET /api/v1/auth/me` — an API key has no role and no personas,

ever, which also means a large part of the mutating surface may be closed to it (04.1).

What is not settled

UNVERIFIED: whether a tool agent's `status` is re-checked at invocation time or only at grant time. Moving an agent to `suspended` is the documented way to take it out of service, and this edition could not establish from the client surface whether an in-flight or immediately-subsequent invocation is refused. If you are relying on `suspended` as an incident control, test the latency of it against your own deployment before you need it.

UNVERIFIED: whether a `revoked` or `archived` agent's invocation history remains readable. The retention question matters for an audit that outlives the agent, and the client surface does not answer it.

Next: 03.4 — Who may call your tool

03.4 — Who may call your tool: applications, grants and policy

Chapter 03.3 got a tool agent to `active`. That makes it *callable*; it does not make it *available to anything*. This chapter is the other half — the model that decides which consumers may invoke it, under what posture, against whose budget, and with what data in scope.

The whole model is `client.applications` (`sdk:aegis_sdk.ApplicationsModule`), and it is four objects that compose:

object	answers
application	who is asking, and what are <i>its</i> outer bounds?
grant	may this application invoke this tool agent at all?
invocation policy	for this application/agent pair, what are the tighter bounds?
delegation matrix	which of those bounds may an application operator change?

Read that table as a chain of narrowing. Nothing further down widens anything further up — the same monotonic-tightening property envelopes have (02.3), applied to consumers rather than to roles.

The distinction that makes the model make sense

Applications hold authority to invoke shared tool agents. They do not own tool agents. A tool agent is registered once, to an accountable role (03.3); any number of applications may then be granted the right to call it. That separation is what makes a tool agent *shared* rather than copied — one implementation, one accountable owner, many authorised consumers, each with its own bounds and its own budget.

If you find yourself creating a second tool agent because a second consumer needs slightly different limits, stop: that is what an invocation policy is for.

Step 1 — Create the application

```
app = await client.applications.create(
    name="Treasury Console",
    owner_role_id=head_of_treasury_role_id,
    description="Internal console used by the treasury desk.",
    default_posture_ceiling="supervised",
    budget_monthly="2500.00",
)
```

`api:POST /api/v1/applications`, returning `sdk:aegis_sdk.modules.applications.Application`.

Four things about the arguments:

`owner_role_id` is required and is an accountability anchor, exactly as `registered_to_role_id` is on a tool agent. The application answers to a role.

`default_posture_ceiling` is the application's own ceiling and it bounds every grant beneath it. It defaults to `pseudo` — the most restrictive posture, which is the right default and an easy one to be surprised by. A grant cannot exceed it, and the server enforces that rather than trusting the caller.

Money is a string. `budget_monthly` is `"2500.00"`, not `2500.00`. So are `budget_allocation` on a grant and `budget_per_invocation` on a policy. Passing a float where a decimal string is expected is the kind of mistake that works in testing and rounds wrongly in production; the string type is deliberate.

`data_scope_json` and `allowed_models_json` are JSON strings, defaulting to `"[]"`. An empty scope is not "everything" — see the caution below.

It is created in a pending state, not an active one. An application has to be approved before it can consume anything, which is the same create-is-not-grant shape envelopes, clearances and knowledge all have (02.3). A provisioning script that creates applications and walks away has built a console nobody can use, and the symptom is a refusal rather than an error.

Lifecycle moves go through `api:PATCH /api/v1/applications/{id}/status`, whose targets are `active`, `suspended`, `archived` and `rejected`.

Step 2 — Grant the tool agent

```
grant = await client.applications.grant_tool_agent(
    app.id,
    agent_id=agent.id,
    posture_ceiling="supervised",
    rationale="Treasury desk needs overnight cash summaries in the console.",
    budget_allocation="500.00",
)
```

`api:POST /api/v1/applications/{id}/grants`, returning `sdk:aegis_sdk.modules.applications.ToolAgentGrant`. Read them back with `api:GET /api/v1/applications/{id}/grants` and remove one with `api:DELETE /api/v1/applications/{id}/grants/{grant_id}`.

rationale is mandatory, and this is a CARE requirement rather than a form field. A constraint without a stated reason is a constraint nobody can review: six months later the question is not "what is the ceiling?" — that is on the record — but "why is it this?", and only the rationale answers it. Write it for the person who will read it during an audit, not for the validator.

posture_ceiling cannot exceed the application's own `default_posture_ceiling`, and the server enforces it. You cannot grant an application more posture than the application has. This is the monotonic tightening again, one level down.

`budget_allocation` carves out part of the application's budget for this specific agent. `constraint_summary_json` carries the constraint summary that was in force when the grant was made — a snapshot, for the record.

The grant record carries `status`, `granted_by`, `granted_at` and `revoked_at`. A revoked grant is not deleted; it is a revoked row, and it stays readable — pass `status="revoked"` or `status="all"` to `list_consumers` on the agent side (03.3) to see them.

Step 3 — Tighten with an invocation policy

A grant says *whether*. A policy says *how far*, for one application/agent pair.

```
await client.applications.update_policy(  
    app.id,  
    agent.id,  
    posture_ceiling="supervised",  
    budget_per_invocation="0.75",  
    data_scope_json='["treasury/cash-positions"]',  
    allowed_models_json='["<model-id-from-your-config>"]',  
    prompt_prefix="You are answering inside the Treasury Console. Do not advise.",  
    knowledge_attachment_ids_json='["kn_abc123"]',  
)
```

`api:PUT /api/v1/applications/{id}/policy/{agent_id}`, read at `api:GET /api/v1/applications/{id}/policy/{agent_id}`, returning `sdk:aegis_sdk.modules.applications.InvocationPolicy`.

The six knobs, and what each is really for:

field	bounds
<code>posture_ceiling</code>	how autonomous this pairing may be
<code>budget_per_invocation</code>	per-call spend cap (a decimal string)
<code>data_scope_json</code>	which data this pairing may reach
<code>allowed_models_json</code>	which models it may use
<code>prompt_prefix</code>	context prepended to every invocation from this application
<code>knowledge_attachment_ids_json</code>	which knowledge items are attached

`prompt_prefix` is the one to be careful with, because it is the only knob here that shapes behaviour rather than bounding it. It is a governance surface with a usability feel: it changes what the agent is told on every call from this application, and it is invisible from the agent's own configuration. If an agent behaves differently for one consumer than another and its own configuration is identical, this is the first place to look.

△ An empty `data_scope_json` is ambiguous from the client, and you should settle it against your own deployment before relying on it. `[]` could mean "no data in scope" (the restrictive reading) or "no restriction declared" (the permissive one), and the two are opposite. The client cannot tell you

which; test it. This is the sentinel hazard `coc/guardrails/sentinels-and-defaults.md` treats at length — a value in the ordinary range that means something outside it, where the ambiguity resolves permissively by default and nothing announces it.

Design intent, not observable: that the policy is intended to be strictly narrowing relative to the grant. The narrowing relationship between the application ceiling and the grant ceiling **is** server-enforced and stated as such by the SDK; whether every policy field is validated against its grant in the same way is not established from the client surface. Do not assume a policy cannot widen — check.

Step 4 — Decide what an operator may change, with the delegation matrix

The delegation matrix is the answer to "who may adjust these bounds without coming back to you?". It is per-parameter, and it is where an architect encodes the difference between a knob a desk manager may turn and one only governance may.

```
await client.applications.update_delegation_matrix(app.id, entries=[
    {"parameter_name": "prompt_prefix",
     "permission_level": "edit",
     "rationale": "Desk owns its own phrasing."},
    {"parameter_name": "posture_ceiling",
     "permission_level": "locked",
     "rationale": "Autonomy level is a governance decision, not a desk one."},
    {"parameter_name": "budget_per_invocation",
     "permission_level": "view",
     "rationale": "Desk should see the cap; only finance changes it."},
])
```

`api:PUT /api/v1/applications/{id}/delegation-matrix`, read at `api:GET /api/v1/applications/{id}/delegation-matrix`, returning `sdk:aegis_sdk.modules.applications.DelegationMatrixEntry` records. Four permission levels:

level	meaning
<code>edit</code>	the operator may change the value
<code>toggle</code>	the operator may switch it, within bounds
<code>view</code>	visible, not changeable
<code>locked</code>	not changeable, and not an operator's concern

Three constraints worth knowing before you write one: **every entry requires a `rationale`** — the same CARE requirement as on a grant; **the bulk set is capped at 50 entries**; and **changing the matrix itself requires application-admin authority**, which is what stops an operator from widening their own permissions.

The matrix is enforced on the policy write path. `update_policy` is gated through it server-side: a parameter at `locked` or `view` cannot be updated, and the attempt returns a 403 that the SDK raises as `sdk:aegis_sdk.AuthorizationError`. So a 403 on a policy update that your role otherwise permits

is usually the matrix, not your permissions — a genuinely useful thing to know, because the two failures look identical from the outside.

Step 5 — Operators, and what assignment actually does

```
await client.applications.add_operator(app.id, user_id=user_id, role="app_admin")
```

`api:POST /api/v1/applications/{id}/operators` , listed at `api:GET /api/v1/applications/{id}/operators` , removed at `api:DELETE /api/v1/applications/{id}/operators/{user_id}` . Two roles: `app_operator` and `app_admin` . Adding or removing one requires application-admin authority.

Operator assignment also decides visibility, not just capability. Org admins see every application; scoped roles see only the applications they are explicitly assigned to as operators, filtered server-side. So an application a colleague cannot find is frequently an assignment they do not have rather than a bug — and `api:GET /api/v1/applications` returning fewer rows for them than for you is the system working.

Watching what the application actually does

Four read surfaces, and they answer different questions:

- `api:GET /api/v1/applications/{id}/invocations` — this application's tool-agent invocation history, returning `sdk:aegis_sdk.modules.applications.InvocationListResult` . The consumer-side view of what 03.3 shows agent-side.
- `api:GET /api/v1/applications/{id}/audit-events` — **the one to reach for**. It aggregates the application's own events with events on *every tool agent it has been granted*, newest first. That composition is exactly the question an auditor asks and exactly the join you would otherwise write by hand. Note it returns a **flat list**, not an enveloped one.
- `api:GET /api/v1/applications/{id}/notifications` — pending constraint-change notifications for you on this application. ⚠ **Only notifications inside a 24-hour grace window are returned**, so this is a surface to poll rather than to check occasionally; a constraint change you did not see is a constraint change you are now operating under.
- `api:GET /api/v1/applications/{id}/summary` — a lightweight `sdk:aegis_sdk.modules.applications.ApplicationSummary` for a listing.

Content freeze — the control most deployments never find

An application can be put under a **content freeze**: a declared window during which its content is not to change, with a staleness threshold.

```
status = await client.applications.get_freeze_status(app.id)
```

```
api:GET /api/v1/applications/{id}/freeze-status , returning
sdk:aegis_sdk.modules.applications.FreezeStatus — is_frozen , reason , freeze_start ,
freeze_end , staleness_days and a warning . The freeze is configured through the ordinary update
path ( api:PUT /api/v1/applications/{id} ) via content_freeze_enabled , content_freeze_start ,
content_freeze_end and content_freeze_staleness_days .
```

While a freeze is active, an update touching the freeze fields can be refused with a 403 requiring application-admin authority. That is the verification gradient's `held` zone arriving on a configuration surface: the change is not rejected as invalid, it is held pending someone with the authority to make it. If you meet an inexplicable 403 on an application update, check the freeze status before you check your permissions.

Reach for a freeze during a period where a consumer-facing surface must be stable — a reporting close, an audit window, a regulated quiet period. It is the mechanism that makes "nothing changed during the window" an assertion you can evidence rather than a claim you make.

Archiving, and the cascade you should expect

```
await client.applications.archive(app.id)
```

`api:DELETE /api/v1/applications/{id}` — and read what it does before you call it:

1. it **revokes every active grant**,
2. it **invalidates every active invocation policy**,
3. then it transitions the application to `archived` .

It requires org-admin authority, and it returns the archived application record rather than an empty response.

That cascade is the correct behaviour and it is wider than the word "archive" suggests. Every consumer relationship this application had is severed in one call. Before you make it, run the impact read from the *agent* side (03.3) for each agent this application was the significant consumer of — the cascade tells you what the application loses; the impact read tells you what everything else loses.

Diagnosing a refused invocation

An invoke that fails through this model has five plausible causes, and they present similarly. Check them in this order, because it is cheapest-first:

1. **Is the application active?** A pending or suspended application consumes nothing. `api:GET /api/v1/applications/{id}` .
2. **Is there an active grant for this agent?** `api:GET /api/v1/applications/{id}/grants` — and check `status` , since a revoked grant is still a row.
3. **Is the agent itself `active` ?** A `suspended` or `revoked` tool agent is not callable regardless of the grant (03.3).

4. **Is a budget exhausted?** Application budget, grant allocation, and per-invocation cap are three separate limits and any one of them refuses. `budget_consumed` against `budget_monthly` on the application is the first read.
5. **Is the posture ceiling too low for what was asked?** The effective ceiling is the tightest of the application's default, the grant's, and the policy's.

A `sdk:aegis_sdk.GovernanceViolationError` is the shape most of these arrive in. Retrying is wrong for every one of them — each is a decision, not a transient failure. The general procedure for telling one kind of refusal from another is `coc/skills/diagnosing-a-refusal.md` ; the list above is its tool-agent-specific case.

What is not settled

UNVERIFIED: whether an invocation policy is validated against its grant the way a grant is validated against the application ceiling. The grant-versus-application check is stated as server-enforced; the policy-versus-grant relationship is not established from the client surface. Until you have tested it on your own deployment, do not treat a policy as structurally incapable of widening.

UNVERIFIED: the semantics of an empty `data_scope_json` . Restrictive and permissive readings are both consistent with what is reachable from here, and they are opposite. Settle it before you rely on the empty default as a bound.

UNVERIFIED: whether revoking a grant terminates an in-flight invocation or only prevents new ones. If you need a revocation to be an incident control rather than a housekeeping action, measure it before you need it.

Next: 03.5 — Testing an extension

03.5 — Testing an extension before it touches a real organisation

The extension you have built will run inside a governed organisation, with real roles, real budgets, a real audit trail and real people whose work depends on it. This chapter is about what you can exercise before that, what each surface actually proves, and — the part most testing plans get wrong — **what each one does not prove**.

The organising question, and it is worth asking of every check in this chapter: *would this have produced a different result if my extension were broken?* A check that answers the same way either way is not a test, whatever it printed.

The four testing surfaces, and what each one covers

surface	what runs	governed?	writes?
draft test (<code>agents.test_draft</code>)	a real model call with unsaved config	no	nothing
task-agent test (<code>task_agents.test</code>)	a direct model call	no — stated plainly	nothing
pipeline validate (<code>pipelines.validate</code>)	a structural check	n/a	nothing
the real invoke (<code>tool_agents.invoke</code>)	the production path	yes	everything

Read the "governed?" column carefully. **Three of the four testing surfaces bypass governance**. That is what makes them fast and safe; it is also what makes them silent about the half of your extension that governance decides. Nothing on this platform lets you dry-run a governed invocation.

Testing a draft agent configuration

The most useful of the four, because it separates two things people usually change together: what the agent is configured to do, and whether that configuration is saved.

```
result = await client.agents.test_draft(
    agent.id,
    message="Summarise last night's cash position.",
    draft_system_prompt="You summarise cash positions. Never advise on a transaction.",
)
print(result.content, result.finish_reason)
```

api:POST /api/v1/agents/{id}/test-draft , returning
sdk:aegis_sdk.core.agents.TestDraftResult — content , model , usage , finish_reason ,
thread_id , timestamp , and is_test , which is always True .

Three properties make it the right tool for prompt iteration:

- **Nothing is written.** The draft prompt and draft instructions are used for this call and discarded. The saved agent is untouched, so a colleague using it in parallel sees no change.
- **It runs against a separate per-request test budget, capped server-side.** You are not spending the agent's production allowance to iterate on wording.
- **draft_instructions_json** and **draft_system_prompt** substitute for the saved ones, so you can test a change before committing to it.

Two constraints that bite: `conversation_history` entries must be `user` or `assistant` — a `system` role is rejected server-side, because the system prompt is what you are testing rather than something you supply. And **an archived agent cannot be tested**, raising `sdk:aegis_sdk.ValidationError`; if a draft test fails on an agent you have not touched, check its status before you check your payload.

What it does not prove: anything about governance. The verdict, the envelope, the budget, the audit record — none of them are exercised. A draft test tells you the model does the right thing when asked. It says nothing about whether the agent will be *allowed* to ask.

Testing a task agent, and the caveat that comes with it

```
result = await client.task_agents.test(agent.id, prompt="Extract the totals.")
```

api:POST `/api/v1/task-agents/{id}/test`, returning

`sdk:aegis_sdk.modules.task_agents.TaskAgentTestResult` — `agent_id`, `prompt`, `result`.

This is a direct model call and it is explicitly not governance-gated. The SDK says so in its own documentation of the method, which is the honest thing for it to do and the thing to carry away from this section. A green task-agent test proves the prompt produces sensible output. It is evidence about the model, not about the platform.

`max_turns` is accepted and is reserved for future multi-turn testing; do not build a multi-turn test plan around it today.

Validating a pipeline before you run it

```
report = await client.pipelines.validate(pipeline.id)
```

api:POST `/api/v1/pipelines/{id}/validate`. It checks node connectivity, missing required configuration, circular dependencies, and agent availability — the four structural ways a pipeline is wrong before it has done anything.

This is the strongest pre-execution check in this part, because a pipeline is a declared graph and a graph can be checked. It is also the narrowest: it establishes that the pipeline *can* run, never that running it produces the right answer.

Note a genuine hazard in the pipelines module: **the create path and the graph-save path do not take the same node shape.** `pipelines.create(nodes=[...])` builds `sdk:aegis_sdk.PipelineNode`, which requires `id`, `name` and `node_type`; `api:PUT /api/v1/pipelines/{id}/graph` takes nodes

requiring `node_type` and `label`, with `id` optional. A node dict that works in one call is rejected by the other, and the difference is a single field name. Build the graph one way and stay with it.

Provisioning a place to test in

Nothing here sandboxes for you. The isolation you get is the isolation you build, and there are three levels of it.

Duplicate the object. The configurable objects in this part have fork operations: `api:POST /api/v1/agents/{id}/duplicate`, `api:POST /api/v1/pipelines/{id}/duplicate`, and `api:POST /api/v1/specialist-system/specialists/{id}/clone`. Fork, change the fork, test the fork. This is the cheapest isolation available and it is the honest way to try a variant — editing the shared one and surprising every agent that had it is the alternative.

⚠ **Skills are the exception, and the omission is not an oversight.** There is no skills duplicate route, so `skills duplicate()` raises `sdk:aegis_sdk.UnsupportedOperationError` rather than answering a 404 — it is kept as a named stub so the gap names itself instead of looking like a server fault. To fork a skill, `get` it and `create` a new one from the same fields.

Use a separate application. Create a test application with its own tiny `budget_monthly`, grant it the agent under a deliberately low `posture_ceiling`, and invoke through it (03.4). This is the only way to exercise the *governed* path without touching a production consumer's budget or bounds — and because the posture ceiling is yours to set, you can watch the agent be refused on purpose, which is the most informative test in this chapter.

Use a separate organisation. The strongest isolation, and the most work. Stand up a whole test organisation as 02.2 describes and provision your extension into it from the same script that provisions production. If the script is the artifact, both environments are the same artifact with different inputs, which is the entire argument for building in code rather than by clicking.

Testing the governed path — the only way there is

There is no dry-run for a governed invocation. To exercise governance, you invoke for real against a scope you control:

```
result = await client.tool_agents.invoke(
    agent.id, message="...", application_id=test_app.id,
)
assert result.verification_zone in ("auto_approved", "flagged")
assert result.audit_anchor_id
history = await client.tool_agents.list_invocations(agent.id)
assert history.get("records"), "the call succeeded and the record did not land"
```

The history call returns the raw payload rather than a typed model, so read it defensively — the assertion above is about the record *existing*, not about a shape this edition can pin.

Three assertions, and each one is doing distinct work:

- `verification_zone` — the actual verdict, not merely that the call returned. A `held` or `blocked` zone on a call you expected to succeed is the finding.
- `audit_anchor_id` — the record exists and is anchored.
- **the invocation history** — the record is *retrievable*, which is the property an auditor depends on and the one a successful call does not by itself establish.

Test the refusal too, and test it deliberately. Set the test application's posture ceiling low, or its per-invocation budget to something a real call will exceed, and confirm you get `sdk:aegis_sdk.GovernanceViolationError` rather than a success. A governance configuration that has never refused anything has not been shown to be capable of refusing anything — and "we never saw it deny" is a statement about your test inputs, not about your controls.

The discipline that makes all of this worth anything

A green check proves nothing until you have seen it go red. Before you trust a test as evidence that your extension works, establish that the same test fails when the extension is broken — remove the grant, drop the posture, point at the wrong agent id. If it passes both ways, it was never measuring what you thought.

This applies with particular force to refusal tests, because the failure mode is silent: a test asserting "this is refused" passes identically when the call is refused for the reason you intended and when it is refused because the agent id was a typo. Assert on *which* refusal — the exception type, and the status in `details["status_code"]` — not merely that one occurred.

Two more habits worth carrying:

Pair every zero with a positive control. "No invocations recorded" and "the history call returned nothing because I passed the wrong id" are the same output. Prove the call can return non-empty before you read an empty result as a finding.

Do not read a timeout as a denial. `sdk:aegis_sdk.ConnectionError` and `sdk:aegis_sdk.TimeoutError` mean the deployment did not answer. They are not evidence about permissions, existence, or state — and recording one as a refusal is how a whole test sweep comes back clean for the wrong reason.

All three habits are the same discipline, and `coc/guardrails/reading-a-measurement.md` is the artifact for it: name what would have proved you wrong before you cite what you measured.

Probing the deployment itself

Before any of the above, establish that the deployment you are pointed at is the one you think, and that you can reach it:

```
python -m aegis_sdk.coc.probe --base-url https://<your-deployment>
```

`sdk:aegis_sdk.coc` ships this alongside the handbook; the working procedure around it is `coc/skills/working-against-a-deployment.md`. **The probe deliberately does not test writes**, and that limitation is load-bearing for this part: the reachability of a *write* is frequently different from the read beside it, so a clean probe tells you nothing about whether you can register anything.

Establish the write side yourself, cheaply, before you start:

```
me = await client.auth.get_current_user()
```

`api:GET /api/v1/auth/me`. **If your personas list is empty you are holding an API key**, and a large part of the mutating surface is closed to it regardless of scopes — a documented open defect covered in 04.1. Register with a session unless you have proven the specific write is key-reachable. Discovering this at the end of a provisioning script is the expensive way, and `coc/guardrails/credential-reachability.md` is the cheap one.

After it ships — watching for drift

Testing establishes behaviour at a moment. Agents drift: behaviour moves away from what was configured and approved, without anything changing in the configuration.

- `api:GET /api/v1/agentic/agents/{id}/drift` — one agent.
- `api:GET /api/v1/agentic/drift/alerts` — the tenant.
- `api:POST /api/v1/agentic/agents/{id}/drift/escalate` and `api:POST /api/v1/agentic/agents/{id}/drift/recover` — the responses.

This is the surface most deployments never wire up, and it answers the question your test suite structurally cannot: *is this still doing what we approved?* A trust posture (02.4) assumes someone is asking it. If nobody is, the posture is a statement about the day it was set.

A pre-flight checklist

1. **auth/me** — which credential am I holding, and can it write?
2. **The probe** — is the deployment reachable and the one I meant?
3. **Draft-test the prompt** — does the model do the right thing, unsaved?
4. **Validate the structure** — for a pipeline, does the graph hold together?
5. **Invoke through a test application** — does the *governed* path work?
6. **Invoke past a deliberate bound** — does it refuse, and with the right error?
7. **Read the invocation history** — is the record retrievable, not merely created?
8. **Establish each of the above can fail** — remove the grant and watch step 5 go red. A check you have only ever seen pass is a check you have not yet read.

What is not settled

UNVERIFIED: whether `test_draft`'s separate test budget is shared across an organisation or is genuinely per-request. "Capped server-side" is stated; the sharing scope is not, and it decides whether a colleague iterating in parallel can exhaust your ability to test.

UNVERIFIED: whether an invocation refused by governance appears in `api:GET /api/v1/tool-agents/{id}/invocations`. If a refusal is recorded, that history is your evidence that controls fired; if only successes are recorded, you need a different surface for it, and you should find out which before an auditor asks. This is the same open question 03.1 records for the in-process side, at the registered-agent surface.

Next: 03.6 — Choosing between a tool, a skill, an agent and a pipeline

03.6 — Choosing between a tool, a skill, an agent and a pipeline

This part has described six extension shapes. They overlap enough that a reader can build almost anything with almost any of them, and differ enough that the wrong choice is expensive to unwind six months later. This chapter is the decision.

Read it before you build, not after. Every one of these shapes is cheap to create and none is cheap to migrate off, because what accumulates around them — grants, policies, invocation history, audit records, other people's dependencies — does not migrate with them.

The six shapes, in one table

shape	is	identity	governed at	reusable across agents
in-process tool	a function in the agent's runtime	none	the tool call, by a hook	no — it is code in one runtime
skill	a reusable instruction attached to agents	an id	not directly	yes, by design
agent	a configured worker with a prompt and a model	an id	its execution	n/a
tool agent	a shared capability others invoke	an id	the invoke, by the platform	yes, via grants
task agent	a delegation target another agent hands work to	an id	its own posture ceiling	yes
pipeline	a fixed multi-step graph	an id	per node	the graph is the reuse

The column that decides most arguments is the fourth. **A skill is not governed at its own boundary** — it shapes what an agent does, and the agent's governance is what applies. A tool agent *is* governed at its boundary, which is why it costs more to set up and why it is the right answer whenever the capability is shared.

The four questions, in order

Work down this list. The first one that answers decides.

1. Does more than one thing need to invoke it, with its own bounds?

Yes → a registered tool agent (03.3, 03.4).

This is the single strongest signal in the chapter. The moment a capability has two consumers who need different limits — different budgets, different postures, different data in scope — you need an object that can hold per-consumer bounds, and the tool agent plus grant plus invocation policy is that object. Nothing else here can express "the same capability, bounded differently per caller".

The tell that you got this wrong: you are about to create a second, nearly-identical copy of something because a second consumer needs slightly different limits.

No, only one agent uses it → keep going.

2. Is it *behaviour the agent performs*, or *instructions the agent follows*?

Instructions → a skill.

A skill is content: markdown instructions plus a declared list of required tools, attached to agents by assignment. It changes how the agent approaches a job. It does not execute anything itself. If what you want to add is "the agent should approach cash forecasting this way", that is a skill and it will be simpler than anything else you could reach for.

Behaviour → keep going. If what you want to add is "the agent can now query the treasury feed", instructions cannot do it. Something has to run.

3. Does it run inside the agent's own process, or is it called across a boundary?

Inside → an in-process tool (03.1). No registration, no id, no lifecycle. The hook governs the call. This is the lightest thing in this part and it is the right answer for a capability one agent uses and nobody else needs to discover, audit separately, or invoke on its own.

Across a boundary, and the thing on the other side is outside your deployment → an external agent.

`api:POST /api/v1/external-agents`, on `sdk:aegis_sdk.IntegrationsModule` — `client.integrations.create_external_agent(...)`. See § External agents below; it is a security decision, not a configuration one.

Across a boundary, and it is another agent doing work for this one → a task agent (see below).

4. Is the *shape* of the work fixed and known in advance?

Yes, and it is multi-step → a pipeline. A declared graph of nodes and connections, with a composition pattern.

No → an objective (02.5). Objectives decompose themselves. A pipeline that encodes a decomposition the platform would have derived is a maintenance burden you chose: every change to the work becomes a change to the graph.

This is the one place in this chapter where the right answer is frequently "none of these" — reach for a pipeline when you can draw the steps on a whiteboard and be confident they will still be those steps next quarter.

Skills, in more detail — and one trap

```
skill = await client.skills.create(
    name="cash-forecasting",
    category="reasoning",
    description="How this organisation forecasts overnight cash.",
    content_markdown="# Cash forecasting\n1. Read the overnight position...",
    tools_required=["treasury_feed"],
)
await client.skills.assign_agent_skill(agent.id, skill.id, priority=1)
```

`api:POST /api/v1/skills`, then `api:POST /api/v1/skills/agents/{id}/skills` to assign and `api:GET /api/v1/skills/agents/{id}/skills` to list what an agent has.

category is required and validated against a fixed set. The SDK exposes it as `sdk:aegis_sdk.core.skills.VALID_SKILL_CATEGORIES`:

```
coding · data · web · file · reasoning · custom
```

The SDK checks it client-side and raises `sdk:aegis_sdk.ValidationError` with an actionable message rather than letting the server return an opaque 422. A skill created without a valid category is not created at all.

`priority` on the assignment is an ordering, `0` being highest, and must be `>= 0`. It decides precedence when an agent holds several skills that speak to the same situation. Assigning a skill twice raises rather than silently re-ordering.

Skills cannot be forked: there is no skills duplicate route. The fork convention this part describes for other objects — `api:POST /api/v1/agents/{id}/duplicate`, for example — was never implemented for skills, so `skills duplicate()` raises `sdk:aegis_sdk.UnsupportedOperationError` rather than answering a 404. To fork a skill, `get` it and `create` a new one from the same fields; editing the shared one and surprising every agent that had it is the alternative.

⚠ The trap: `get_by_name` scans a window, not the whole set

```
skill = await client.skills.get_by_name("cash-forecasting")
if skill is None:
    skill = await client.skills.create(name="cash-forecasting", category="reasoning")
```

`get_by_name` returns `None` rather than raising when nothing matches, which makes it the obvious primitive for an idempotent provisioning script — the create-if-missing pattern 02.2 had to hand-roll for units.

But it is a client-side convenience, not a server-side lookup. It lists skills and scans the result for a name match, and the listing it scans is bounded. Once your organisation holds more skills than that window, an existing skill can sit outside it — `get_by_name` returns `None`, and the script above creates a duplicate.

Nothing errors. You end up with two skills of the same name, agents assigned to whichever one their provisioning run happened to create, and a change to "the" skill that reaches half of them.

```
# DO      keep your own registry of provisioned skill ids, keyed by name
# DO NOT  rely on get_by_name as an existence check in a growing organisation
```

If you need create-if-missing at scale, record the id when you create it and look up by id thereafter. The name is a label; the id is the identity.

Agents, versions, and the thing to do before you change one

Agents are covered in 03.2. One point belongs here because it is a *choice*, not a mechanic:

Cut a version before you change a production agent's prompt or configuration. `api:POST /api/v1/agents/{id}/versions` with a changelog, then `api:POST /api/v1/agents/{id}/versions/{v}/rollback` if the change was wrong. Rollback is the cheap way out of a bad prompt, and it exists only if you made the version.

Tool agents have no version surface. That asymmetry is a real input to the choice between them: if the thing you are building will have its prompt tuned repeatedly by someone who wants an undo, an agent gives you one and a tool agent does not. For a tool agent, your provisioning script under source control is the version history, which is another argument for building in code.

Task agents — the delegation target

```
agent = await client.task_agents.create(
    name="Invoice Totals Extractor",
    description="Extracts totals from an invoice document.",
    posture_ceiling="supervised",
)
```

`api:POST /api/v1/task-agents`, returning `sdk:aegis_sdk.modules.task_agents.TaskAgent`.
Updated at `api:PUT /api/v1/task-agents/{id}`, removed at `api:DELETE /api/v1/task-agents/{id}`, exercised at `api:POST /api/v1/task-agents/{id}/test` (03.5).

A task agent is a **non-delegate** agent: it does not stand in for a human in a role, which is what separates it from the delegate agents 03.2 covers. It is a unit of work another agent hands something to.

posture_ceiling is the field to set deliberately. It defaults to `delegated` — the *most* autonomous posture — which is the one default in this part that is permissive rather than restrictive. The SDK validates it client-side against `sdk:aegis_sdk.modules.task_agents.ALLOWED_POSTURE_CEILINGS`:

```
pseudo · supervised · shared_planning · continuous_insight · delegated
```

Set it down to what the work actually needs. A task agent created with defaults is a delegation target with the ceiling wide open, and the fact that a ceiling exists does not mean it was chosen.

`authority_level ( 1 – 5 )` is **informational and does not gate access** — clearance does that (02.3). Do not reach for it as a permission.

The `get` path enriches the record with `unit_name` and `reporting_chain`, which `list` does not carry — so if you are checking where a task agent sits in the organisation, read it individually rather than from the roster.

Tool agent versus task agent — the distinction people get wrong

They share the word "agent" and almost nothing else.

	tool agent	task agent
is	a capability something else invokes	a unit of work with a prompt
has	components, consumers, an envelope, invocations	a prompt, a posture ceiling, a test path
called by	an application holding a grant	another agent, by delegation
lifecycle	<code>create</code> → components → <code>change_status</code>	<code>create</code> → <code>test</code> → <code>update</code>
governed at	the invoke, with a full verdict	its own posture ceiling

The practical test: **does something *outside* the agent world call it?** An application, a console, a service — that is a tool agent, and it needs the grant model. Is it another agent handing off a sub-task? That is a task agent.

Getting this wrong is not fatal but it is expensive: a task agent has no consumers list, no impact analysis and no grant model, so the day you need to know who depends on it, there is no call that answers.

Specialists — the authoring surface

`client.specialist_system ( sdk:aegis_sdk.SpecialistSystemModule )` is the agent-authoring console's resource: `api:GET /api/v1/specialist-system/specialists`, `api:POST /api/v1/specialist-system/specialists`, and `api:POST /api/v1/specialist-system/specialists/{id}/clone`, returning `sdk:aegis_sdk.modules.specialist_system.Specialist`.

Two things to know. **Its wire shape is camelCase**, not the `snake_case` the rest of this part uses — `systemPrompt`, `planningStrategy`, `budgetLimit`, `checkpointFrequency`, `timeoutSeconds`. The SDK model accepts either spelling, but `update_specialist(**fields)` passes your keys through, so

use the camelCase names there. **And the server fills defaults on create** from its own default specialist, so `create_specialist(name=...)` gives you a fully-configured object whose configuration you did not choose. Read it back.

This surface is the console's, and it is the least SDK-complete of the six — the backend exposes considerably more than the specialists resource this module covers. If you are working here, expect to reach for HTTP directly (04.1) for the sub-resources.

External agents — the boundary case

`api:POST /api/v1/external-agents`, on `client.integrations`, returning `sdk:aegis_sdk.modules.integrations.ExternalAgent`. Invoked at `api:POST /api/v1/external-agents/{id}/invoke`, with history at `api:GET /api/v1/external-agents/{id}/invocations`.

```
ext = await client.integrations.create_external_agent(
    workspace_id=workspace_id,
    name="Support Triage Bot",
    platform="slack",
    webhook_url="https://...",
    auth_type="api_key",
    auth_config={...},
    budget_limit_daily=25.0,
    budget_limit_monthly=500.0,
    rate_limit_per_minute=60,
    rate_limit_per_hour=1000,
)
```

Note the money types here are **floats**, not the decimal strings the applications module takes (03.4). The two surfaces disagree, and the disagreement is easy to carry across from one provisioning function to the other. Check the signature.

`platform` is one of `teams`, `discord`, `slack`, `telegram`, `notion` or `custom_http`; `auth_type` one of `oauth2`, `api_key`, `bearer_token`, `basic`, `custom` or `none`. The webhook URL is validated server-side against server-side request forgery.

Treat registering one as a security change rather than a configuration change. It is the point at which your governed organisation acquires an edge to a system it does not govern. It carries credentials of its own, it reaches outside the deployment, and the budget and rate limits you set on it are the only bounds on that edge.

⚠ The four limits default to unlimited, expressed as `-1`

`budget_limit_daily`, `budget_limit_monthly`, `rate_limit_per_minute` and `rate_limit_per_hour` all default to `-1`, which means unlimited. So an external agent registered without them is an ungoverned outbound edge with a credential — created by a call that looks complete and raises nothing.

Two consequences worth stating separately:

Set all four, explicitly, every time. Do not rely on the defaults, and do not let a provisioning helper omit them.

Never compare these numerically. `-1` is smaller than every threshold you will ever set, so `if agent.budget_limit_daily < warn_at:` fires loudest on precisely the agent that has *no* limit. A sentinel drawn from the ordinary numeric range does not announce itself at the comparison site; test for `== -1` first. `coc/guardrails/sentinels-and-defaults.md` treats this class at length, and it is worth reading before you write any check over a limit field.

`sdk:aegis_sdk.modules.integrations.ExternalAgentInvokeResult` gives you `invocation_id`, `trace_id`, `status` and `output`; the per-invocation records (`sdk:aegis_sdk.modules.integrations.ExternalAgentInvocation`) carry `execution_time_ms`, `response_code`, `error_message` and `cost` — which is the surface to watch, because an external agent's failures are outside your deployment and this is where they become visible inside it.

The decision, condensed

- More than one consumer, each with its own bounds → **tool agent**.
- Instructions rather than execution → **skill**.
- One agent, in-process, nobody else needs it → **in-process tool**.
- Another agent hands it work → **task agent**.
- Fixed multi-step shape → **pipeline**. Not fixed → **objective**.
- Reaches outside your deployment → **external agent**, and treat it as security.

And one rule that outranks all of them: **if the capability has to be evidenced, it needs an identity**. An in-process tool has no consumers list, no invocation history and no impact analysis, so every audit question about it has to be answered by hand from your own logs. If someone will one day ask "who called this, when, under what authority, and what did it cost?", build the shape that can answer.

This is the last chapter of part 03. Next: Part 04 — The API surface.

Part 04 — The API surface

Audience: you are calling Aegis over HTTP directly, or you are debugging a call that failed. Most of the time the SDK is the better path — it is typed, it handles path encoding, and parts 02 and 03 are written against it. This part is for when you need the layer underneath: another language, a webhook receiver, a proxy, an authorization failure you cannot explain, or any question about what the client does under load.

⚠ **Be precise about what the SDK retries, because the scope is narrower than the word usually implies.** It retries a request that got **no answer** — a connection failure, a timeout — and it retries **no HTTP status at all**, including `429` and `503`. Counts and the arithmetic are in 04.5. If your integration needs status-level retry, that layer is yours to write.

#	Chapter	Read it when
04.1	Calling the API	You are calling it directly, or a 403 is not making sense
04.2	Errors and refusals	Something raised and you need to know whether to retry, fix, or file it
04.3	Credentials, and what a key is not	You are issuing, rotating or auditing API keys
04.4	Lists, filters and pagination	You are reading an organisation big enough to have a second page
04.5	Concurrency, timeouts and streams	You are running calls in parallel, or consuming an agent's output live
04.6	Reading a response honestly	A field is <code>None</code> and you are about to conclude something from it

Read 04.2 first if something has already gone wrong. It carries the status-to-exception table, the seven statuses with no branch at all, the five failures that are not in this client's exception hierarchy, and the four-way split — not reached, not known, refused, fault — that decides what you change next. A `try/except` written from the documentation alone lets all five past, and treats a `401` a fresh credential cannot clear as a credential problem.

Start here if you have a 403

Aegis has **two credential models**: personas belong to user sessions, scopes belong to API keys. A route gated on personas wants a session, and a wider scope will not substitute for one. Chapter 04.1 gives you a one-minute test that tells you which model a route follows.

Run it before you re-issue keys, widen scopes, or conclude your permission model is wrong — a 403 names the permission, not the layer that decided.

What the route anchors in this book mean

Every operation named in this handbook is written as an **api anchor** — the method and path in backticks behind an `api:` prefix, as in `api:GET /api/v1/agents` — and every one is checked by `python -m aegis_sdk.handbook.check` against the calls this package actually makes. A resolving anchor tells you the operation exists in the surface you were given. Where this book says something is *enforced*, that claim rests on observed behaviour, not on the anchor.

Run the check, and look routes up rather than deriving them. The quota **read** is `api:GET /api/v1/features/limits`, while the quota **write** is `api:POST /api/v1/billing/quotas/adjust` — **the read and the write of the same noun do not share a prefix**, and nothing in the module name predicts it.

Do not infer a route from a noun. Sibling operations on the same concept are not reliably siblings in the path. When you need the read for something whose write you know, look it up. Build the same habit into your own code: a plausible-looking route string is easy to write and considerably more expensive to discover later.

Next: 04.1 — Calling the API

04.1 — Calling the API

This chapter is about *who you are* when you call Aegis, and why a call is refused. Read it before you change a credential: most refusals here are answered by knowing which of two credentials you hold and which layer turned you away.

Two credentials, one door

Both JWTs and API keys enter through the same dependency, and they arrive there already resolved: validation happens in middleware, *before* the dependency runs. An `X-API-Key` header, or a bearer token beginning `sk_live_`, is validated against the API-key service, which stamps the organisation and a synthetic user id of the form `api_key:<key id>` onto the request.

Either credential is accepted the same way by the client (`sdk:aegis_sdk.ClientConfig`), and you can see which one you are holding at `api:GET /api/v1/auth/me`.

The two principals differ in ways that decide everything downstream:

field	API key	JWT
<code>id</code>	<code>api_key:<key id></code>	the user id
<code>organization_id</code>	the key's organisation	the token's <code>org_id</code> claim
<code>role</code>	<code>None</code>	the resolved role
<code>personas</code>	<code>[]</code>	the resolved personas
<code>auth_type</code>	<code>"api_key"</code>	absent entirely
<code>api_key_scopes</code>	the key's scopes	—

Read that table before you read anything else in this chapter. **Two of those cells are empty by design, and they decide which routes each credential reaches:** an API key has no role and no personas, ever, whatever it is scoped to. A key carries scopes; a session carries a role and personas.

The absence of `auth_type` on the JWT shape is the discriminator every gate uses. That is worth internalising, because it means gates test for `auth_type == "api_key"` and treat "no `auth_type`" as "human session". A principal constructed by some third path that forgets the key will be treated as a JWT caller.

A route's behaviour towards an API key is not a property of the API as a whole. Two routes that look alike from outside can admit a key on different terms, and some admit one only where the route asks for it explicitly — a refusal by default, which is the right default. So if a key works on one route and is refused on a sibling, that is not necessarily a scope problem. Treat the two routes as two questions and test each with the credential in your hand.

Claims in this chapter name **surfaces you can call** and symbols you can import, and you can re-verify every one of them against your own installed package:


```
python -m aegis_sdk.handbook.check
```

An operation you can call is evidence you can act on.

Which credential reaches which route

Personas belong to sessions; scopes belong to keys. That single sentence is the credential model, and it decides admission across the whole API.

A **persona** is a property of a human identity — `admin`, `architect` and the rest are things a *person* is within an organisation. An API-key principal is constructed with no personas and no role, unconditionally and by design: a machine credential is not a person, and inventing a persona for one would be a fail-open default in the place that can least afford it. A **scope** is the machine equivalent — a narrow, explicit grant of `:read` or `:write` over a named resource, attached to the key when it is issued.

So the routing is direct:

route is gated on	reached by
personas	a user session
API-key scopes	an API key carrying a satisfying scope
both	either, admitted at the router and scope-checked per route

Use a session for persona-gated calls and a key for scope-gated ones. A wider scope does not turn a key into a session: on a persona-gated path, scopes are not the question being asked, so widening one buys you nothing and leaves you holding a more powerful credential than the job needs.

Routers that accept both use a persona-or-API-key admission gate — the router decides whether the caller is admitted at all, and each route beneath it applies its own write-scope check. That is the shape to expect on mixed surfaces, and it is being extended across the remaining persona-gated routers [immediate roadmap].

Telling the two apart in one minute. When a call is refused and you are not sure which model a route follows, retry the identical request with a user session (`api:POST /api/v1/auth/login` , then the same call). If the session succeeds where the key did not, the route is persona-gated and a session is the credential it wants. Confirm what your own credential carries with `api:GET /api/v1/auth/me` — an API-key principal shows an empty persona list, which is the expected shape, not a misconfiguration.

Admission is layered, and that explains an otherwise baffling result. A router-level gate runs *before* any per-route gate. So a route can carry a perfectly correct per-route permission check that is never reached, because the router made the admission decision first.

What you observe: two routes on the same resource, one reachable with your credential and one not, with no difference in the permissions either route documents. The difference is which layer decided — and the error body does not say. If a route is refused where a sibling is not, the useful question is not

"which permission am I missing?" but "was I refused at the door, or at the desk?".

⚠ A scope check is a no-op for session callers

The scope gate returns early for anything that is not an API key. It is an API-key mechanism; against a user session it checks nothing at all.

The consequence, on routes whose only per-route gate is a scope check: the **sole** authority check for a human session is the router-level persona allowlist. There is no per-route permission check on that path.

Read that in both directions, because both matter. As a caller, a session may reach a route you expected your scopes to bound. As an integrator deciding what to hand a colleague, **a scope-bounded API key is not the weaker credential you may assume it is** — on some routes it is the only one that is actually bounded.

The two registries

These are two different vocabularies that share a spelling. Keep them apart.

The permission matrix — RBAC. Keyed by *role* (`tenant_admin` , `org_owner` , `org_admin` , `developer` , `viewer` , `member` , `app_operator` , `app_admin` , `user` , `operator` , `manager` , `architect` , `admin`), with granular actions plus a `*` wildcard. This is what a user session is judged against.

The API-key scope vocabulary — scopes, and only two buckets: `:read` and `:write` . Anything that is not a read resolves to write. This is what a key is judged against, and you can list what a given key carries with `api:GET /api/v1/api-keys` .

Neither is a subset of the other, and that is the whole difficulty. The concrete case that catches people:

- `roles:read` and `roles:write` are valid API-key scopes.
- **No `roles:` entry exists anywhere in the permission matrix.** `roles` , `units` , `knowledge` and `clearance` are all absent from it.

So "`roles:write` is not in the permission matrix" is a true statement that tells you nothing about whether the scope is valid. It is valid. The two registries answer different questions, and a name in one is not a name in the other.

The bridge between them is one-directional. A permission *resource* expands to a set of candidate scopes, and any one of them satisfies it:

```
organizations -> organizations, units, roles, knowledge
deployments   -> deployments, gateways
```

So a key scoped `roles:write` satisfies the permission `organizations:update`. **There is no reverse table:** you cannot map a scope back to a permission, so "what can this key do?" has no mechanical answer from the scope alone. Expect to determine it by trying the operation.

Two more rules worth knowing about scopes, both of which surprise people:

- **write implies read.** A key scoped `x:write` satisfies `x:read`. You do not need both, and asking for both does not make the key narrower.
- **a malformed scopes value denies.** If the scopes field is not a sequence, the answer is no — not "unrestricted". Fail-closed, as everywhere else here.

The same permission can be enforced two different ways

Two different internal guards enforce permissions on this platform. One consults **roles only**; the other consults roles **and attribute rules**. Which one guards a given route is not visible from the outside, and there is no field in any response that tells you.

What that means for you as a caller, which is the whole of what you can act on:

- A permission that is granted to your role and still refused is **not necessarily a misconfigured role**. An attribute rule may be denying you.
- **An attribute deny overrides a role allow.** That is the merge strategy — deny wins — so adding the role again will not clear it.
- **Two routes requiring what looks like the same permission can decide differently for the same caller**, because one consults the attribute rules and one does not. If that happens to you it is a real distinction, not a flake, and not something to retry your way out of.

Where a gate is role-only it is so by deliberate configuration — a documented choice, and a property of the platform's design rather than a gap.

The guard is chosen per route at the platform side, and the choice is not exposed on the response.

The error body names the permission, not the mechanism that evaluated it. So a permission granted to your role and *still* refused is not necessarily a bug in your role — an attribute rule may be denying you, and deny wins over a role allow.

So diagnose it from the outside, in this order. Confirm the permission is granted to your role. If it is and the call is still refused, treat an attribute rule as the live hypothesis and take it to whoever administers your organisation — they can see the rules and you cannot. The one thing not to do is widen the role and retry, which cannot succeed against an attribute deny and leaves the role permanently wider than it needed to be.

Tenancy

The field is `organization_id` on the principal — the key's organisation for an API key, the token's `org_id` claim for a session — and it is mirrored onto the request. You can read the organisation your credential resolves to at `api:GET /api/v1/auth/me`.

Tenancy is checked in more than one place, and that is the fact to carry. One of those refusals is worth knowing exactly: a caller with no organisation context is turned away with a **400** and the detail `"Authenticated caller has no organization context"`. Note the status — a *400*, not a 401 or 403. A caller who reads only the status code will classify this as a malformed request and go looking at their body. It is an identity problem.

The consequence: **a tenancy failure can surface as a 400, a 403, or an empty result set.** Those look like three different bugs and are one. When you meet any of the three, read the organisation your credential resolves to before you change anything else.

A scope and a tenant are different questions. Your organisation comes from the credential itself, never from what that credential is scoped to. A scope tells you which operations a key may attempt; it never tells you whose data the key reaches. So widening a scope is not a way to reach another organisation's data, and a scope that looks correct is no evidence that a tenancy refusal was about scopes at all.

Checklist for calling this API

1. **Know which credential you hold**, and what it lacks. Check with `api:GET /api/v1/auth/me`. An API key has no role and no personas — ever.
2. **If a call is refused, establish which layer refused it** before changing anything. Retry with the other credential type; the difference tells you more than the status code does.
3. **Do not widen a credential to solve a refusal you have not diagnosed.** On several paths a wider scope changes nothing, and you are left holding a more powerful key for no benefit.
4. **Never send an organisation id in a request body** expecting it to be honoured. Tenancy is derived from your credential, and a body-supplied value is at best ignored.
5. **Expect the same for any approver or decider identity.** It is derived from the authenticated session and pinned when the record is created — a body field naming an approver is not how approval works here, and this is deliberate.

Adding a route to the platform, rather than calling it, is covered by the platform's own internal documentation.

Where the rest of this part goes. This chapter is about *who you are* and *why you were refused*. The five that follow are about everything that happens once a call actually goes out: what each failure means and whether to retry it (04.2), what a key can and cannot hold over its life (04.3), how to read a collection without silently truncating it (04.4), what the client does under load and where its error handling stops (04.5), and which fields of a successful response you may actually act on (04.6).

Next: 04.2 — Errors and refusals

04.2 — Every failure this client can raise

There are **four** kinds of bad outcome, and telling them apart is the whole job:

	what happened	what you have measured
not reached	the deployment never answered	nothing — not permissions, not existence, not state
not known	it answered: <i>I do not know who is asking, in the way this operation requires</i>	your authentication, and nothing about your permissions
refused	it answered: <i>I know who you are, and the answer is no</i>	a governance verdict
fault	something broke — a malformed request, a server error, a contract the client cannot model	that somebody has to fix something

A refusal is the product working. Part 02 makes that claim; this chapter is where it becomes actionable. A refusal is evidence, and it belongs in your audit story rather than in your incident channel. The other three do not.

⚠ **The distinction that costs a day is *not known* versus *refused* — 401 versus 403**. They read alike, they are both "denied" in casual speech, and they have opposite remedies. A 401 says the platform could not establish who is asking to the standard this operation demands; reissuing a credential may be the answer, and no permission change will be. A 403 says it established exactly who you are and declined; reissuing the credential is wasted work.

An architect who learns "denied means 403" will misread the other one, and the misreading is expensive in both directions — see § *A refusal is not one status* below, which is the case where a perfectly valid key gets a 401.

The rest of this chapter is the mapping that lets you tell. For the diagnostic ladder that uses it — which rung to work, in what order, and what not to change until you have — see the refusal-diagnosis skill shipped beside this book, and the error-taxonomy guardrail it rests on.

The status-to-exception table

Every entry below comes from driving `sdk:aegis_sdk.Agentic0SClient`'s transport against each status in turn. Reproduce it on your own build; it takes under a second.

status	what the client does	kind
200 201 202	returns the parsed JSON body	—
204	returns <code>None</code>	—
400	<code>sdk:aegis_sdk.ValidationError</code>	fault (your request)

status	what the client does	kind
401	<code>sdk:aegis_sdk.AuthenticationError</code>	fault (your credential)
403	<code>sdk:aegis_sdk.AuthorizationError</code>	refusal
404	<code>sdk:aegis_sdk.NotFoundError</code>	either — see below
409	<code>sdk:aegis_sdk.ConflictError</code>	refusal
422	<code>sdk:aegis_sdk.ValidationError</code>	fault (your request)
423	<code>sdk:aegis_sdk.GovernanceViolationError</code>	refusal
429	<code>sdk:aegis_sdk.RateLimitError</code>	not reached, yet
451	<code>sdk:aegis_sdk.TrustViolationError</code>	refusal
5xx	<code>sdk:aegis_sdk.ServiceError</code>	fault (theirs)
anything else	<code>sdk:aegis_sdk.AgenticOSError</code> , message Unexpected status code: <n>	unclassified

Four of those are refusals and each says something different. `403` is an authorization decision — a principal was judged and found wanting. `409` is a **state** refusal: the request was well-formed and permitted, and the server declined to guess because the row moved, or because several eligible rows exist and none was named. `423` is a **governance** refusal: a policy, a budget, an approval requirement. `451` is a **trust** refusal: the delegated authority for this action does not hold. If you collapse all four into "permission denied" you lose the distinction your own evidence export depends on, because they are answered by four different people — and `409` by a different ACTION (re-read the row) rather than by a different person at all.

To explain one rather than infer it, use `client.governance_explain`. It walks the access chain and returns the step the decision stopped at:

```
why = await client.governance_explain.explain_access(
    role_id="role-analyst",
    knowledge_item={
        "id": "doc-9",
        "classification": "confidential",
        "unit_address": "D1-R1",
    },
    posture="supervised",
)
print(why.allowed, why.step_reached, why.reason)
```

Read `step_reached`: when `allowed` is `False` it names where the chain stopped, and `access_path` carries the route taken.

△ **It is a dry run, and it is not a record of the refusal you received.** It evaluates the item *you describe* — it reads no stored record, records no access, and a wrong or missing field changes the verdict. Reconstruct the item the refused call was about and re-run the chain. It answers about a role plus a knowledge item, not about a connector or a tool.

The same module carries the state readouts: `explain_envelope()` for a role's effective envelope through its ancestor chain, and `envelope_hydration_status()` / `envelope_coverage()` — a hydration pass that **skipped** a role leaves it on a fail-closed bootstrap default, so a skip reads exactly like a governance refusal and is silent everywhere else.

`404` is the ambiguous one. It means *this client asked for something the server does not have* — which covers a deleted record, a mistyped id, and a route that does not exist on this deployment. The client cannot separate them, and neither can you from the status alone.

A refusal is not one status

At least three different mechanisms refuse a caller, and they do not all produce a `403`. Knowing which you are looking at decides what you change next.

1. A permission-level denial gives `403`. The principal was identified, judged against what the route requires, and declined. This is the case the word "denied" makes people picture, and it is the only one of the three where looking at roles and permissions is the right next move.

2. Some surfaces are gated at the whole-router level, so even a *read* is refused for an API key. This is the open defect 04.1 documents, and it is worth restating in status terms: the refusal is a `403` carrying a message about personas, and it is **not about the route you called**. It is admission at the door. A read that looks harmless is refused exactly as a write would be, because nothing route-specific was ever consulted.

3. At least one control requires a verified interactive session and answers `401`, not `403`. This is the one that costs the day. Your key is valid, it is unexpired, it was accepted everywhere else this morning — and this operation answers `401` because it demands a kind of authentication a key cannot supply. The client raises `sdk:aegis_sdk.AuthenticationError`, whose own documentation says "API key is invalid or malformed / token has expired", and **both of those readings are wrong here**. Reissuing the key produces an identical `401`.

Design intent, not observable: which control demands a verified session, and what makes a session count as verified, is decided at the platform side. You cannot query it from here and this book will not invent a way to tell you.

What you can do from outside is the same sibling comparison 04.1 uses, with one extra rung: if a `401` survives a freshly-issued key, stop treating it as a credential problem and retry the identical call with an interactive session. If the session succeeds, you have found mechanism 3, and no key of any scope will ever pass.


```
# The discriminator, in full. Run all three; the pattern identifies the mechanism.
# key -> 401, fresh key -> 401, session -> 200 ... mechanism 3, use a session
# key -> 403, session -> 200 ... mechanism 2, use a session
# key -> 403, session -> 403 ... mechanism 1, look at permissions
```

Holding the right grants is not sufficient

The most confusing refusal on this API is the one where you hold every grant the operation names and are still refused — **with those grants never consulted**.

The mechanism is ordering. On at least one surface the identity gate runs *before* the permission check, so a caller who fails the first gate is turned away while the grants that would have satisfied the second sit inert. The verdict is a `403` whose message names something to obtain, and obtaining it changes nothing.

Your instinct on seeing that message will be to add permissions. **Adding permissions will not help**, and you will finish holding a wider credential than the job needs, permanently, having fixed nothing. This is the same trap 04.1 frames as *"was I refused at the door, or at the desk?"* — and the answer is not in the response body.

The observable signature, from your side, is precisely that: **a change that should have worked and did not**. Treat "I granted the thing it asked for and the refusal is byte-identical" as diagnostic rather than as a reason to grant more. Take it to whoever administers your organisation, who can see the ordering and the rules; you cannot.

The gaps, named — because the table above has a bottom row for a reason

Six statuses an integrator meets routinely have no branch at all. They fall to the base class carrying `Unexpected status code: <n>`, which reads like a platform malfunction and is nothing of the sort:

status	what it actually means	what a naive handler does
<code>402</code>	payment required	retries
<code>405</code>	wrong method for this path	retries
<code>410</code>	gone — permanently, not temporarily	polls forever
<code>413</code> <code>415</code> <code>428</code>	too large / wrong content type / precondition required	retries

`410` is the expensive one. It means the id will never come back, so a poll loop on it never terminates — precisely the shape a retry-by-default handler gets exactly wrong.

⚠ **`409` WAS IN THIS TABLE AND IS NOT ANY MORE, which changes what you CATCH and not what you DO.** It now arrives as `sdk:aegis_sdk.ConflictError` and sits in the table at the top of this chapter. The handling is unchanged: a conflict means your precondition has already failed, so re-

asserting it fails again. Catch it by type if that reads better; branch on `status_code` if you already do that everywhere else. Both work, and the second keeps working for every status that is still in the table above.

And the boundary is wide, not a corner case. That the client has no branch for these is checkable by you, in seconds, and is the fact this book anchors on. They are ordinary answers rather than exotic ones — a `410` on anything deliberately retired, a `405` from a route you assembled by hand — so a retry-by-default handler meets them on normal paths, not at the edges. What follows is concrete: **write the `410` branch**, because the client will not, and the base class will not tell you which one you have without reading the status.

⚠ `sdk:agis_sdk.PaymentError` exists, is exported, and is never raised. The transport has no `402` branch, and the whole package contains no site that raises it. Do not write an `except PaymentError` and believe you have covered billing failures; a `402` arrives as the base class.

Two more gaps are easy to miss because they look like success.

- Any `2xx` that is not `200`, `201`, `202` or `204` is raised as an error. A `203`, `205`, `206` or `207` becomes `Unexpected status code`. If a partial or multi-status response ever appears on a route you use, you will see it as a failure.
- No redirect is followed. The client is configured not to, so a `301`, `302`, `307` or `308` becomes `Unexpected status code: 307`. This is the single most confusing first-day failure: point the client at a base URL that redirects — `http` where the deployment wants `https`, a host that appends a trailing slash, a gateway that normalises paths — and **every call fails with a message that mentions nothing about redirects**. Check your base URL with a plain `curl -i` before you debug anything else.

What escapes the taxonomy entirely

`except AgenticOSError` is the idiom, and it is not complete. Four failures are not `sdk:agis_sdk.AgenticOSError` at all:

what happens	what is raised
a <code>200</code> , <code>201</code> or <code>202</code> whose body is empty, truncated, or not JSON	<code>json.JSONDecodeError</code>
a list route that returns an envelope where this client expects a bare array	<code>TypeError</code>
a client method that hands the transport a keyword it does not accept	<code>TypeError</code> , before anything is sent
any call on a client you already closed	<code>RuntimeError</code>
any transport failure during a stream	the raw underlying HTTP-library exception

The first is the one you will hit most often. A proxy or gateway that returns an HTML page with a `200`, or an endpoint that accepts with `202` and an empty body, both produce a decode error rather than anything in this book's vocabulary. The last is covered in 04.5: the streaming path has no error

translation, so a read timeout mid-stream arrives raw.

If you are writing a library on top of this one, catch `Exception` at your boundary and re-raise something of your own. If you are writing a script, at least know that a bare `except AgenticOSError` will let five things past.

The middle row deserves its own warning: some methods cannot execute at all

A client method builds a request by passing keyword arguments down to the transport. Anything the transport does not declare falls through to the underlying HTTP library, and there it either **raises**, or — worse — is **quietly accepted and does something else**. Neither is visible to a type checker, because the argument disappears into a catch-all before anything can check it.

Measured on this build: **seven call sites pass the transport a keyword it does not declare**. Two of them make their method impossible to call at all.

```
await client.compliance.export_audit(          # api:POST /api/v1/compliance/audit/export
    start_date="2024-01-01", end_date="2024-01-31", format="csv",
)
# TypeError: ... got an unexpected keyword argument 'raw_response'
```

`sdk:aegis_sdk.modules.ComplianceModule.export_audit` and `sdk:aegis_sdk.modules.ComplianceModule.export_soc2_evidence` both raise that, and — the part that matters for diagnosis — **zero requests reach the wire**. Verified by counting requests at a mock transport that would have answered `200`: both methods, 0 requests; a sibling on the same module that passes no stray keyword, 1 request.

Read that as a general instruction, not as two names to avoid. **When a call fails, establish whether a request was made before you look at the deployment at all**. A pre-wire failure is a client defect and no amount of credential, permission or network investigation will move it — and it is the one failure mode where the deployment is definitively not involved.

```
# `--transports-only` answers this offline, before any network
# python -m aegis_sdk.coc.probe --transports-only
#
# Or discriminate a single call: if AGENTIC_OS_DEBUG=true logs no
# "SDK Request:" line for it, nothing was sent.
```

The other five sites are the quieter half: a keyword the HTTP library *does* accept, which sends a form body or a file upload where a JSON body was intended, or attaches query parameters to a stream. Those fail on the wire rather than before it, so they surface as a `400` or a `422` that looks like your data is wrong. **UNVERIFIED**: whether each of the five is a defect or deliberate — some are on upload paths where a non-JSON body is correct. What is settled is that the type checker could not have told you either way.

The general lesson is the one worth carrying past this release: **a method being exported, documented and typed is not evidence that it has ever run**. Where this book documents a method's behaviour, it documents one that was traced end to end; where you are about to depend on one this book does not cover, call it once against a deployment you can afford to be wrong about before you build on it.

Two failures that happen before any request is sent

Neither is a deployment problem, and both are frequently reported as one.

`sdk:aegis_sdk.ConfigurationError` — no base URL. Nothing was contacted. Covered in 01.3.

`sdk:aegis_sdk.UnsupportedOperationError` — the method you called targets a server operation that does not exist. **Ten methods raise it unconditionally, before any request is made.** Six are on the trust surface: `sdk:aegis_sdk.trust.ChainsModule.suspend` and `sdk:aegis_sdk.trust.ChainsModule.reinstate`, the three delegation *read* methods, and a single-entry audit read. Four are on the usage surface: `sdk:aegis_sdk.revenue.usage.UsageModule.get_history` and `.get_breakdown`, and their duplicates `AnalyticsModule.usage_history` and `.usage_breakdown`. None of these recover, and retrying is meaningless: the operation is not implemented, not down.

🚫 **Until 2026-09-18 the two `revenue.usage` methods did not raise — they LIED.** They caught the `404` and returned an `EMPTY` result with no log and no warning, so a caller could not distinguish "no usage in this period" from "this endpoint does not exist". Both answers were the same empty value, on a billing surface. They now raise, which is the only honest answer available. Both were ALSO the two the bundled documentation taught you to call, so if you have code that expected `[]` from them, that code was reading a false negative.

⚠️ An earlier revision of this section described *two further* methods raising only for a `csv / xlsx` argument on the analytics exports. **That guard was removed** — it rested on the false premise that the transport has no raw-bytes path, which `_http.py` does have and two shipped methods already used. The count is ten, and the export methods are not among them.

That is not the whole of the dead surface. Separately, **nineteen methods carry a warning in their own docstring that the route they call currently returns `404`** — a known-dead path rather than a missing record. They are spread across agent executions, skills, objectives, sessions, analytics and audit-log reads. When a `NotFoundError` looks impossible against data you know exists, read the method's docstring before you doubt your data:

```
help(client.objectives.submit) # the warning is in the docstring, not the type
```

Contrast `sdk:aegis_sdk.ServiceUnavailableError`, which means the capability exists and the infrastructure is down. It is exported, and — like `PaymentError` — **no site in this package raises it.**

Reading the message, and why you should not

The exception's string is built from the first of the body's `detail`, `message` or `error` keys that is present. When an error body carries none of them, that value is `None` — and because the key **exists** and holds `None`, a `details.get("message", "some default")` returns `None` rather than your default. On a `403`:

the error body	<code>str(exc)</code>
<code>{"error": {"code": ..., "message": "real detail", ...}}</code> — what Aegis sends	<code>'real detail'</code>
<code>{"detail": "real detail"}</code>	<code>'real detail'</code>
<code>{}</code> or <code>{"foo": "bar"}</code>	the per-status default, e.g. <code>'Insufficient permissions'</code>
no body at all	<code>'Unknown error'</code>
<code><html>...</html></code> from a proxy	the raw HTML, to 500 characters

Corrected. The second and third rows used to read differently, and both corrections come from the same defect. This client parsed only the *flat* `{"detail": ...}` shape, while the platform wraps **every** error in the `{"error": {...}}` envelope on the first row — so `str(exc)` rendered a Python dict repr instead of the message, `exc.details["code"]` was `None` on every error the platform raises, and every structured field the server attached was discarded. The third row's old answer, a literal `'None'`, was the same bug reaching the empty-body case. Both are fixed; see § *Branching on a refusal*.

Build your own log line from the status rather than relying on the default: it tells a reader which of the four kinds they are looking at.

```
from aegis_sdk import AgenticOSError

try:
    await client.agents.list()
except AgenticOSError as exc:
    status = exc.details.get("status_code")
    detail = exc.details.get("message") or f"HTTP {status}"
    log.error("agents.list failed: %s (%s)", detail, status)
```

The **status code is on every raised error**, at `exc.details["status_code"]`, including the unclassified ones. That is the field to branch on — not the subclass, which cannot distinguish `410` from `418` from `405`, and which does not even exist for them. `exc.status_code` is the same value without the `KeyError` risk on an error the client raised locally, where there was no response to carry one.

Branching on a refusal

Every error the platform returns carries a machine-readable code, and some carry structured fields beside the message. Read those; do **not** parse the message. The message is prose written for a human and may be reworded in any release without notice — a handler that matches on its wording is a handler that breaks silently on a copy-edit.

attribute	what it holds
<code>exc.error_code</code>	the server's code for this refusal — <code>'FORBIDDEN'</code> , <code>'DEPENDENCY_UNAVAILABLE'</code> , ... — or <code>None</code> when the error did not come from a server response
<code>exc.server_details</code>	the structured fields attached beside the message; <code>{}</code> when none were sent
<code>exc.request_id</code>	the server's correlation id, when supplied. Quote it when reporting a refusal you cannot explain — it is what lets an operator find the same event on their side
<code>exc.status_code</code>	the HTTP status, or <code>None</code> for a local failure

```

from aegis_sdk import AgenticOSError

try:
    await client.agents.execute(agent_id, message="...")
except AgenticOSError as exc:
    if exc.error_code == "DEPENDENCY_UNAVAILABLE":
        # A named component is down. The request is retryable, and this is
        # NOT a statement about your agent.
        dependency = exc.server_details.get("dependency")
        log.warning("governance dependency down: %s (req %s)", dependency, exc.request_id)
    else:
        # A decision about this caller. Retrying changes nothing.
        log.error("refused: %s (%s)", exc.message, exc.error_code)

```

An absent field means UNDETERMINED, never the negative case

`exc.server_details` carries what the server sent on *this* response. A missing key means it sent nothing under that name — it does **not** mean the underlying condition is absent. Several refusal paths still serialise only a prose sentence, so their structured half arrives as `{}`. Record that as *undetermined* and say so, exactly as you would record a timeout as *not reached* rather than as *refused*.

The gap this book will not paper over — a refusal that is the platform's, not yours

Some denials are the governance apparatus reporting that it **could not be queried at all** — the constraint store was unreachable, so no restriction could be evaluated and the platform failed closed. That is categorically different from a restriction that was evaluated and applied to your agent: the first is an outage on their side and is retryable, the second is a decision about you and is not. The remedies have nothing in common.

Today you cannot tell those apart programmatically on agent execution. Both arrive as `403` with `error_code` `'FORBIDDEN'` and an empty `server_details`; the only thing that differs is the wording of the message, and matching on wording is the thing this section tells you not to do.

The client-side half of the channel is in place — the moment the platform attaches a discriminator beside the message, it arrives at `exc.server_details` with no change on your side and no new SDK release. What is missing is the platform half: the execute route serialises the enforcer's verdict as a

prose sentence and drops the structured fields the enforcer already computed. Closing it is a change to the Aegis API layer, not to this client, and it is tracked platform-side.

Until it closes, the honest handling is: treat a `403` on execution as a refusal (do not retry it), and when you need to know *which kind* it was, quote `exc.request_id` to whoever operates your deployment. They can see the verdict's structured basis in the audit record; you cannot see it from here, and this book will not invent a way to tell you.

△ A `422` puts a *list* where you expect a string. A field-validation failure carries a list of per-field objects, and that list is what lands in the message. `str(exc)` becomes the printed repr of a list of dicts, each with the field path and the reason. It is genuinely the most useful error body the API produces and the least readable thing to print. Iterate it rather than logging it:

```
except ValidationError as exc:
    problems = exc.details.get("message")
    if isinstance(problems, list):
        for p in problems:
            log.error(" %s: %s", ".".join(str(x) for x in p.get("loc", [])), p.get("msg"))
```

What to retry, and what never to

retry	<code>sdk:aegis_sdk.RateLimitError</code> (honour its <code>retry_after</code>), <code>sdk:aegis_sdk.ServiceError</code> , <code>sdk:aegis_sdk.ConnectionError</code> , <code>sdk:aegis_sdk.TimeoutError</code>
never	<code>400</code> <code>401</code> <code>403</code> <code>404</code> <code>409</code> <code>410</code> <code>422</code> <code>423</code> <code>451</code> , and every <code>sdk:aegis_sdk.UnsupportedOperationError</code>

Retrying a refusal produces a slower refusal and, eventually, a rate limit. Retrying a `409` re-asserts a precondition that has already failed.

△ The client does not do this for you, and 04.5 is blunt about the gap. It retries *transport* failures only — no HTTP status is ever retried, including `429` and `503`. `RateLimitError` carries a `retry_after` parsed from the response header, and nothing acts on it. If your integration needs status-level retry, you write it.

The one distinction to keep hold of

A `sdk:aegis_sdk.ConnectionError` or a `sdk:aegis_sdk.TimeoutError` is not evidence about permissions, about existence, or about state. It is the absence of an answer.

This matters most in a sweep. If you are walking a hundred routes to establish what your credential can reach, a timeout recorded as a denial — or worse, as an empty result — produces a report that looks clean for the wrong reason. Record "not reached" as its own outcome, distinct from both "allowed" and "refused", and count it. A sweep with unexplained gaps is a sweep with an unknown answer, and saying so is the whole of the discipline.

Next: 04.3 — Credentials, and what a key is not

04.3 — Credentials, and what a key is not

04.1 explains what the two credential types *are* and why a persona-gated route refuses a key. This chapter is the operational half: what a key can hold, what happens to it over its life, and what a permission failure looks like from where you are sitting.

One header, two credentials

Whatever you hold, this client sends it the same way:

```
Authorization: Bearer <the credential>
```

That is the whole mechanism. The client sends **no X-API-Key header** — the deployment may well accept one from a hand-rolled caller, but this package never produces it, so a rule you write against that header will not see SDK traffic.

The consequence for your code is convenient and slightly alarming: one client object can hold either kind, and swap between them mid-run.

```
client = AgenticOSClient.from_env()           # holding the provisioning key

token = await client.auth.login(email, password) # api:POST /api/v1/auth/login
client.set_auth_token(token.access_token)       # now holding a session
```

`sdk:aegis_sdk.AgenticOSClient.set_auth_token` and `sdk:aegis_sdk.AgenticOSClient.set_api_key` are the **same method under two names** — the "token" one is an alias. There is no separate token slot, no precedence rule, and nothing warns you that you have just replaced a long-lived provisioning credential with a session that will expire. If you need both, hold two clients:

```
provisioner = AgenticOSClient(base_url=URL, api_key=KEY)
operator    = AgenticOSClient(base_url=URL)
operator.set_auth_token((await provisioner.auth.login(e, p)).access_token)
```

`sdk:aegis_sdk.ClientConfig.with_api_key` builds a second config from the first if you would rather not repeat the settings.

Neither the repr nor the masked property will leak the credential. Printing the client shows a truncated form, and `sdk:aegis_sdk.AuthToken` hides both `access_token` and `refresh_token` from its own repr. That is a real protection and it is also a trap in the other direction: `print(token)` will not show you the token, so a debugging session that prints the object and sees nothing has not necessarily got nothing.

A key is a provisioning credential, not a person

This is the sentence to carry out of the chapter.

An API-key principal has an id of the form `api_key:<key id>`, no role and no personas — see the table in 04.1. That is not a defect of your particular key. **It is what a key is.** Three consequences follow, and each of them is somebody's wasted afternoon:

1. **A key cannot approve anything.** Approvals are attributable to a human, and the approver identity is derived from the authenticated session and pinned when the record is created. A key has no human behind it to attribute to.
2. **A key cannot reach a persona-gated route, at any scope.** The one-minute test is in 04.1; run it before you touch scopes.
3. **A key's `organization_id` is fixed at issue.** It is the key's organisation, not a claim you can vary. `api:POST /api/v1/auth/me/switch-org` exists for a session; a key has nothing to switch.

Read your own credential before reasoning about any refusal:

```
me = await client.auth.get_current_user() # api:GET /api/v1/auth/me
if not me.personas:
    ... # you are holding a key. Persona-gated routes are closed. Scopes will not help.
```

⚠ **That call can raise for exactly the credential it exists to diagnose.** `sdk:aegis_sdk.User` requires six fields — `id`, `email`, `name`, `organization_id`, `organization_name`, `role` — and a key principal is documented as carrying no role. Two of the three plausible wire shapes for a missing role raise a validation error; only `role: ""` parses.

A validation error here means the request was ANSWERED, not refused. It is a different finding from a `401` or a `403` and it points at a different owner. **UNVERIFIED:** which shape a real deployment sends. Settling it needs a live call, and this edition could not make one, so do not read either answer out of this book — make the call and read the body:

```
raw = await client._http.request("GET", "/api/v1/auth/me") # api:GET /api/v1/auth/me
print(raw) # personas, role, organization_id – unparsed, therefore readable
```

That deliberately reaches past the model. It is right for this one diagnosis and wrong everywhere else.

Nothing in this book depends on `get_current_user()` succeeding. It is a convenience, not a gate.

Creating a key, and the secret you will lose

```
key = await client.auth.create_api_key( # api:POST /api/v1/api-keys
    name="nightly-provisioning",
    scopes=["organizations:write", "agents:read"],
    expires_in_days=90,
)
print(key.key_prefix) # 'sk_live_' – an identifier, not a credential
```

🚫 The returned `sdk:aegis_sdk.APIKey` has no field for the key itself. Its fields are `id`, `name`, `key_prefix`, `scopes`, `created_at`, `expires_at`, `last_used_at` — and nothing else. A server response carrying the full secret is parsed into that model, the secret matches no declared field, and it is **silently discarded**. There is no error and no warning; you are left holding a prefix and an id, and the secret is not retrievable afterwards by design.

Given a response body carrying both `key` and `api_key`, neither attribute exists on the result, while the same body read before parsing contains both.

If you are provisioning keys from a script, take the raw body:

```
raw = await client._http.request(
    "POST", "/api/v1/api-keys",          # api:POST /api/v1/api-keys
    json_data={"name": "nightly-provisioning", "scopes": ["agents:read"]},
)
secret = raw.get("key") or raw.get("api_key")  # whichever your deployment emits
```

UNVERIFIED: which field name a real deployment uses for the secret, or whether it returns one at all — that is a property of the deployment and cannot be settled from this package. What *is* settled is that the typed method cannot give it to you whatever it is called.

Then treat it as a secret. Everything in 01.3 about environment variables applies: it belongs in your secret store, not in the provisioning log, and not in the audit trail of the script that made it.

What scopes are, briefly, and where the detail is

Two buckets — `:read` and `:write` — and **write implies read**, so asking for both does not make a key narrower. A malformed scopes value denies rather than permits. The scope vocabulary and the permission matrix are **different registries that share a spelling**, and the full account of how they relate, including the one-directional bridge between them, is in 04.1 § *The two registries*. Read it before you design a key hierarchy; the two-registry confusion is the most reliably repeated mistake on this API.

The one thing worth repeating here, because it inverts an assumption people carry in: on routes whose only per-route gate is a scope check, **a scope-bounded key is the more tightly bounded credential, not the looser one** — the scope gate returns early for a session and checks nothing. Do not reach for a human session on the assumption that it is the safer choice.

Read what a key actually carries with `api:GET /api/v1/api-keys`, or one key by id with `api:GET /api/v1/api-keys/{id}`:

```
for k in await client.auth.list_api_keys():
    print(f"{k.name:24} {k.key_prefix} scopes={k.scopes} expires={k.expires_at}")
```

As a key ages

Four fields describe a key's life, and the gaps between them are the useful part.

field	what it tells you
<code>created_at</code>	when it was issued — always present
<code>expires_at</code>	when it stops working, or <code>None</code> for never
<code>last_used_at</code>	when it was last presented, or <code>None</code> for never used
<code>scopes</code>	what it carries — an empty list is legal

⚠ **There is no status, active or revoked field on the model at all.** You cannot tell a live key from a revoked one by reading it. `expires_at` is the only lifetime signal the client exposes, and `None` there means *no expiry*, not *expired*. If your deployment revokes a key by deleting the record, absence from `api:GET /api/v1/api-keys` is your signal; if it revokes in place, this client gives you nothing to see it by. **UNVERIFIED:** which of those your deployment does.

`last_used_at` is the field to build a hygiene check on, and it is the one architects reliably forget they have:

```
from datetime import UTC, datetime, timedelta

cutoff = datetime.now(UTC) - timedelta(days=90)
for k in await client.auth.list_api_keys():
    if k.last_used_at is None and k.created_at < cutoff:
        print(f"never used in 90 days: {k.name} ({k.id})")
    if k.expires_at is None:
        print(f"no expiry set: {k.name} ({k.id})")
```

That is exactly the kind of assertion 02.1 argues the harness is for and the console cannot do. Run it on a schedule.

Rotation. `api:POST /api/v1/api-keys/{id}/regenerate` exists on this surface. **UNVERIFIED:** whether the old secret stops working immediately or overlaps — that decides whether you can rotate without a maintenance window, and it cannot be settled from the client. Establish it in a staging deployment before you need it in production. Retiring a key outright is `api:DELETE /api/v1/api-keys/{id}`; `sdk:aegis_sdk.auth.AuthModule.revoke_api_key` is an alias for the delete, not a separate softer operation.

Expiry is opt-in on this side. `expires_in_days` is optional at creation, the client supplies no default, and it is **omitted from the request body entirely** when you do not pass it — so whether an unexpiring key results is the deployment's decision, not the client's. **UNVERIFIED:** what yours does with the field absent. Pass it explicitly for anything living in CI, and set the rotation reminder in the same change; then you do not need to know.

When a permission failure arrives

You will see one of three things, and they want three different responses.

what you get	what it means	what to do
<code>sdk:aegis_sdk.AuthenticationError</code> (401)	two different things — see immediately below	reissue once ; if it recurs, stop and read on
<code>sdk:aegis_sdk.AuthorizationError</code> (403)	you were identified and refused	attribute it before changing anything
<code>sdk:aegis_sdk.GovernanceViolationError</code> (423) or <code>sdk:aegis_sdk.TrustViolationError</code> (451)	a policy or a trust constraint refused you	this is governance working — see 02.6

⚠ The **401** row is the one that wastes a day, and the exception's own documentation points the wrong way. `AuthenticationError` is documented as "invalid or malformed key / expired token", and that is only one of its two causes. The other is that **the operation requires a verified interactive session, and a key cannot supply one at any scope** — so a perfectly valid, unexpired, correctly-scoped key gets a **401** that looks exactly like a bad credential.

The discriminator is cheap and definitive: **reissue once, and if the fresh key gets the same 401, retry with a session**. A session that succeeds where two independent keys failed is mechanism 3 in 04.2, and no key will ever pass that control. Stop reissuing.

For the **403**, the attribution step is one retry with the other credential type, and the result table is in 04.1. The short version: a key refused where a session succeeds is a *reachability* problem no scope change will fix; both refused is a real authorization decision; a session that returns **401** means your session is bad and you have measured nothing.

🚫 **Do not widen a credential to clear a refusal you have not attributed.** In the reachability branch the added scopes are never read, so you gain nothing and are left holding a more powerful key permanently. In the authorization branch you have granted access without deciding it should be granted. Neither branch is improved by widening; only one is even affected by it.

Next: 04.4 — Lists, filters and pagination

04.4 — Lists, filters and pagination

An organisation of any size makes this the first thing you hit and the easiest thing to get quietly wrong. A list call that returns fifty rows out of four hundred does not fail. It succeeds, returns a plausible answer, and your create-if-missing script re-creates everything past the first page.

This chapter is about the four ways this client asks for a page, the many ways the API answers, and which of the fields you get back you may actually trust.

There is no one pagination convention. There are four

Across the client, **83 of its 841 asynchronous methods take a paging parameter**, in four distinct caller-facing shapes.

the parameters the method takes	methods	example
<code>limit + offset</code>	45	<code>sdk:aegis_sdk.core.SkillsModule.list</code>
<code>page + page_size</code>	18	<code>sdk:aegis_sdk.core.AgentsModule.list</code>
<code>limit</code> only — no way to advance	18	several memory and lookup reads
<code>page_size</code> only — no way to advance	2	<code>sdk:aegis_sdk.execution.ObjectivesModule.list</code>

The third and fourth rows are the ones to notice. Twenty methods let you ask for *more* and give you no way to ask for the *next*. On those, the only page you can ever read is the first one, and the only lever you have is to raise the limit. If the collection is larger than your limit, the remainder is not reachable through that method at all.

⚠ **Do not assume the parameter names are the wire names.** `agents.list` takes `page` and `page_size` and sends `limit` and `offset`, because that is what the route reads; sending `page` would be silently ignored and every call would return the same first page. The translation is the client doing you a service, and it means you cannot infer the HTTP contract from the Python signature. Where this book names a paging parameter, it is naming the **Python** one.

The default page is small, and it varies

Every paging method has a literal default. Across the 83:

38 x <code>limit = 50</code>	15 x <code>limit = 100</code>	4 x <code>page_size = 20</code>
14 x <code>page_size = 50</code>	4 x <code>limit = 10</code>	3 x <code>limit = 20</code>
	2 x <code>page_size = 10</code>	1 x <code>limit = 12</code>
	1 x <code>limit = 200</code>	1 x <code>limit = 1000</code>

So the modal default is **50** — a bit over sixty per cent of the paging surface — and the range runs from 10 to 1000. **Never rely on the default.** Pass the page size explicitly at every call site, so that the number is in your code where a reviewer can see it rather than in a signature they will not read.

Servers cap it too, and the cap is not uniform: `skills.list` documents a maximum of 100, `objectives.list` a maximum of 200. Asking for more than the cap does not error; you get the cap.

What comes back is not uniform either

There is no single list envelope. **This client reads the collection out of the response under 27 distinct key names**, of which `records` is the plurality (18 sites) and `items` a distant second (7). `agents`, `data`, `skills`, `policies`, `history`, `events`, `invoices`, `plans`, `members`, `nodes`, `permissions` and fifteen others each appear on their own routes. A handful of routes — API keys among them — return a **bare JSON array** with no envelope at all.

You do not usually have to care, because the typed methods unwrap it for you. You have to care in exactly two situations:

- **You are calling the raw transport**, in which case the key is yours to find. Print the response before you write the accessor. Do not guess it from the noun; the [part README](#) has the worked case of a read and a write on the same noun that share no path prefix, and envelope keys are no more predictable.
- **Your deployment returns a different envelope than this client expects.** A route that hands back `{"records": [...]}` where the client wants a bare array produces a raw `TypeError`, outside the exception taxonomy — see 04.2.

`PaginatedResponse` is a client construction, not a server one

Several methods return `sdk:aegis_sdk.PaginatedResponse`, which has five fields:

```
items total page page_size has_next
```

Only `items` and `total` are required. `page`, `page_size` and `has_next` are optional with defaults of `1`, `50` and `False`. And — this is the part that decides whether your paging loop is correct — **the client computes all of them itself, differently per module, from whatever the route actually returned.**

Three measured examples, all returning the same type:

method	total means	has_next is computed as
<code>sdk:aegis_sdk.core.AgentsModule.list</code>	the organisation-wide count	<code>offset + len(page) < total</code>
<code>sdk:aegis_sdk.core.SkillsModule.list</code>	the organisation-wide count	<code>offset + len(page) < total</code>

method	total means	has_next is computed as
<code>sdk:aegis_sdk.execution.ObjectivesModule.list</code>	the length of this page	<code>len(page) >= page_size</code> — a heuristic

Read the third row twice. On `objectives.list`, `total` is *not* an organisation-wide count and `page` is hard-coded to `1`, because the route does not paginate. A loop that reads `result.total` as "how many objectives exist" gets "how many objectives came back", which is a different number wearing the same name, and the two agree exactly when the answer does not matter.

 **The failure mode that will actually bite you.** Where a route omits `total`, the client falls back to the length of the page it received. `has_next` then evaluates `offset + len(page) < len(page)`, which is **false on the first page** — so a paging loop terminates after one page and reports success. Nothing errors, nothing warns, and the result looks exactly like a small collection.

That is not hypothetical arithmetic; it is the shape behind the create-if-missing hazard in 02.2, where a re-runnable provisioning script that reads only the first page re-creates everything it could not see.

A paging loop that does not lie to you

Two rules make it robust against every variation above: **advance by what you received**, and **stop when a page comes back short**, rather than trusting `total` or `has_next`.

```

async def all_of(fetch, page_size: int = 200):
    """Every row from a limit/offset method, without trusting `total`.

    `fetch` takes (limit, offset) and returns a PaginatedResponse.
    """
    out, offset = [], 0
    while True:
        page = await fetch(page_size, offset)
        out.extend(page.items)
        if len(page.items) < page_size:      # short page = last page
            return out
        offset += len(page.items)           # advance by RECEIVED, not by requested
        if offset > 100_000:                 # a bound, so a server bug cannot loop forever
            raise RuntimeError(f"paging did not terminate at offset {offset}")

skills = await all_of(lambda limit, offset: client.skills.list(limit=limit, offset=offset))

```

Three things that loop does deliberately:

- It advances by `len(page.items)`, not by `page_size`. If the server caps your requested size, advancing by the requested amount skips rows. Advancing by what arrived cannot.
- It stops on a short page, not on `has_next`. A short page is a fact about the response; `has_next` is an inference the client made from fields that may not have been sent.
- It has an upper bound. A route that ignores `offset` returns the same page forever and the loop never ends. The bound converts a hang into an error you can read.

For a `page / page_size` method the same shape applies with `page += 1`. For the twenty methods with no offset at all, there is no loop to write: raise the limit to the cap, and **if you get exactly the cap back, treat the result as truncated rather than complete** — because you cannot tell the difference from here.

Filters, and the ones that do nothing

Filtering is per-method and the parameters are not uniform either. `skills.list` takes `category`, `is_public` and `search`; `agents.list` takes `workspace_id`, `status` and `agent_type`; `objectives.list` takes **status** and **nothing else** — no `agent_id`, no `workspace_id`, and no page cursor.

⚠ **A parameter a route does not read is silently ignored.** It is not an error and there is no warning: the server matches the parameters it declares and drops the rest, so an unsupported filter returns the *unfiltered* first page. That is the more dangerous direction — you get more than you asked for while believing you were narrowed — and it is how a "list the objectives for this agent" call becomes "list the organisation's objectives" without saying so.

Two habits that catch it:

```
# 1. Assert the filter actually held, rather than assuming it.
page = await client.agents.list(status="active", page_size=200)
assert all(a.status == "active" for a in page.items), "the status filter did not apply"

# 2. Read the method's own docstring for the supported set before you rely on it.
help(client.objectives.list)
```

The first is worth writing into any sweep whose conclusion depends on the filter. A filter that silently no-ops turns a narrow question into a broad answer, and the broad answer is usually still plausible.

Two return shapes, and the one that fails at the point of use

01.1 names this and it is worth the reminder in a chapter about reading collections: some modules return typed models with attribute access, and the vertical-standup modules return raw dictionaries with subscript access. Within a single provisioning script you will handle both — typically a `dict` from the create and a model from the list, for the *same* noun.

```
created = await client.units.create(name="Treasury", unit_type="department")
unit_id = created["id"]                    # dict -> subscript

page = await client.org_standup.list_units(limit=200, offset=0)
names = [u.name for u in page.records]    # model -> attribute
```

Note that `list_units` returns `.records`, not `.items` — it is not a `PaginatedResponse`. When in doubt, `print(type(result))` once and stop guessing; the failure otherwise surfaces as a `TypeError` or `AttributeError` at the point of *use*, several lines from the call that decided it.

Next: 04.5 — Concurrency, timeouts and streams

04.5 — Concurrency, timeouts and streams

Everything in this client is asynchronous, and the shape of that matters once you are provisioning two hundred roles rather than one. This chapter is what the client does under load, what it does *not* do for you, and the one place — streaming — where its error handling stops.

One client, many calls

The client holds a connection pool and is meant to be shared. Create it once, use it concurrently, close it once.

```
import asyncio
from aegis_sdk import AgenticOSClient

async def main() -> None:
    async with AgenticOSClient.from_env() as client:          # one pool
        agents = await asyncio.gather(*[
            client.agents.get(agent_id) for agent_id in ids    # concurrently
        ])
    return agents
```

The pool allows **100 simultaneous connections with 20 kept alive** between calls. Sixty concurrent requests through one client all go in flight together — the client applies **no concurrency limit of its own** below the pool ceiling.

A client per call is the anti-pattern. Each one builds its own pool and TLS session, so a loop that constructs and discards clients pays a handshake per request and holds none of the connection reuse the pool exists for. Two hundred roles is one client and a `gather`, not two hundred clients.

Closing matters. `sdk:aegis_sdk.AgenticOSClient.close` releases the pool; the `async with` form calls it for you. A call on a client you already closed raises a plain `RuntimeError`, not anything in this book's exception vocabulary — see 04.2.

The timeout is one number, and it applies four times over

`AGENTIC_OS_TIMEOUT` (default **30.0** seconds) becomes the timeout for connecting, writing, reading and waiting for a pool slot — the same value for all four. There is no separate connect timeout and no separate read timeout.

That matters when you raise it. Setting `timeout=300` for one slow report also gives you a five-minute *connect* timeout, so a deployment that is simply unreachable now takes five minutes to say so, three times over. Prefer a second client for the slow work:

```
fast = AgenticOSClient.from_env()
slow = AgenticOSClient(base_url=fast.base_url, api_key=KEY, timeout=300.0)
```

`sdk:aegis_sdk.ClientConfig.with_base_url` and `sdk:aegis_sdk.ClientConfig.with_api_key` build a variant config if you would rather not repeat the settings.

What is retried — and it is less than you think

Counting HTTP attempts against each failure in turn, at the default `max_retries` of 3:

what fails	attempts made	total added delay
a connection error	3	2.5 s
a read or connect timeout	3	2.5 s
a pool timeout	3	2.5 s
a <code>500</code> or <code>503</code>	1	none
a <code>429</code> rate limit	1	none
a <code>403</code> refusal	1	none

△ **No HTTP status is ever retried. Only transport failures are.** This is the single most commonly mis-stated property of the client — including, until this edition, by parts of this book that described it as handling retries without saying which ones. It retries the case where the deployment *did not answer*. It does not retry the case where the deployment answered `503`.

`sdk:aegis_sdk.RateLimitError` carries a `retry_after` parsed from the response header, and **nothing acts on it**. The value is there for you to use; the client will not sleep on your behalf. If your integration needs status-level retry, you write it, and the table in 04.2 says which statuses may have it.

The backoff between transport attempts is fixed — roughly one second, then one and a half — and it is **not configurable from the environment**. Only `AGENTIC_OS_MAX_RETRIES` is. So the worst case for one unreachable call is `max_retries × timeout` plus the backoff: at the defaults, **about 92 seconds before a single call gives up**. For a `gather` of two hundred against a deployment that is down, that is a script which appears to hang for a minute and a half and then produces two hundred identical errors. Lower the timeout for anything you would rather have fail fast.

🛑 **Do not set `max_retries` to 0 to disable retries.** With `AGENTIC_OS_MAX_RETRIES=0` the retry loop never executes, **no request is made at all**, and every call raises `sdk:aegis_sdk.ServiceError` with the message `Unknown error after retries` — including calls that would have returned `200`. The failure looks like a broken deployment and is entirely local.

Use `1` for a single attempt with no retry. The constructor argument path happens to fall back to the default when given `0`, so this bites specifically through the environment variable and through `sdk:aegis_sdk.ClientConfig` — which is to say, through the two paths a deployment configures rather than the one a developer types.

What the client does not do for you

Stated plainly, because each of these is something integrators assume is handled:

- **No rate limiting.** Nothing throttles you below the pool's 100. A `gather` over a large collection will hit the deployment as hard as your event loop can manage, and the 429 you get back will not be retried.
- **No `Retry-After` compliance.** See above.
- **No idempotency keys.** The client sends no idempotency header on any request. A retried write is a second write, and the transport's own retry is on the transport layer — where a request that timed out *may still have been applied by the server*. For creates you care about, make the operation create-if-missing yourself, as 02.2 shows.
- **No request ordering.** `asyncio.gather` gives you results in argument order and says nothing about the order the server saw them in. If B depends on A, `await A` first; do not put them in one `gather` and hope.

A bounded worker is usually the right shape for anything above a few dozen calls:

```
async def bounded(client, ids, *, concurrency: int = 8):
    sem = asyncio.Semaphore(concurrency)

    async def one(agent_id):
        async with sem:
            return await client.agents.get(agent_id)

    return await asyncio.gather(*(one(i) for i in ids), return_exceptions=True)
```

`return_exceptions=True` is deliberate: without it, the first failure cancels the rest and you lose the results of every call that had already succeeded. With it, you get a list you can partition into results and failures — and per 04.2, you should partition failures further into refusals, faults, and *not reached*.

Streaming, and where the error handling stops

Some operations stream server-sent events rather than returning a body:

`sdk:aegis_sdk.core.AgentsModule.stream` for an agent execution,
`sdk:aegis_sdk.execution.SessionsModule.stream_events` and
`sdk:aegis_sdk.execution.SessionsModule.stream_messages` for a session. Each yields a dictionary with a `type` key.

```
async for event in client.agents.stream(agent_id, message="..."):
    if event["type"] == "content":
        print(event["content"], end="", flush=True)
    elif event["type"] == "done":
        break
```

The streaming path is a different code path from every other call in this client, and it behaves differently in four ways that all matter.

1. Its errors are not translated. A transport failure during a stream raises the **raw underlying HTTP-library exception**, not `sdk:aegis_sdk.TimeoutError` or `sdk:aegis_sdk.ConnectionError`. A read timeout mid-stream comes out untranslated. `except AgenticOSError` around an `async for` will not catch it.

Status codes *are* translated — a `403` or `500` on the initial response raises the same exception as anywhere else — but only for statuses at or above 400.

2. A redirect on a stream yields nothing and raises nothing. The status check covers 400 and above, so a `302` falls through to the event loop, which finds no events: an empty stream, clean exit, no error. A `204` does the same. **An empty stream is indistinguishable from a governed agent that produced no output**, which is exactly the wrong ambiguity to have here.

3. It is not retried. One attempt, then the exception. There is no reconnection and no resumption from a last-seen event.

4. The read timeout applies between events, not to the stream. The client's 30-second timeout is the gap it will tolerate *between* events. A long-running governed session that thinks for more than thirty seconds without emitting will time out mid-stream — and per point 1, it will do so with an untranslated exception. For anything you expect to run long, use a client with a larger timeout.

Two silent losses to guard against

Malformed event data is dropped without a word. A `data:` line that is not valid JSON is skipped; there is no error and no gap in the sequence you can detect. A stream of three events with the middle one malformed yields two, cleanly.

A truncated stream looks exactly like a complete one. The iteration ends either on a terminator event or when the connection closes, and both produce the same clean exit. There is no completion signal you can check afterwards.

The consequence is a rule: **decide completion from the events, never from the loop ending.**

```
done = False
async for event in client.agents.stream(agent_id, message=task):
    if event["type"] == "error":           # an error arrives as an EVENT
        raise RuntimeError(event.get("error", "the stream reported an error"))
    if event["type"] == "done":
        done = True
if not done:
    raise RuntimeError("stream ended without a completion event – treat as truncated")
```

⚠ **Note the first branch.** A failed execution is reported to you as an event of type `error`, on a stream that otherwise looks healthy. A consumer that filters for `content` and ignores everything else will print a partial answer and never learn that it was partial. This is the same shape as the truncation above and it is more likely: the error is *there*, in the data, and you dropped it.

The same trap has a sharper edge on `stream_events`, whose `event_types` argument filters **client-side**. Passing `event_types=["progress_update"]` discards the `error` event before you ever see it — the server sent it, and the filter you wrote to reduce noise removed the one event that mattered. If you

filter, always keep `error` in the list.

That argument filters on `type` values that are the server's, not the names in this chapter's prose — and it fails **silently** when you pass one that does not exist: nothing raises, you simply receive no such event, which is indistinguishable from a session that emitted none. The vocabulary is on `sdk:aegis_sdk.execution.SessionsModule.stream_events` ; check it there rather than guessing at a plausible name.

Next: 04.6 — Reading a response honestly

04.6 — Reading a response honestly

A call succeeded. You have an object. The question this chapter answers is which parts of it you may act on, and what a field that is *not there* is telling you.

That question is load-bearing here in a way it is not on an ordinary API. The objects you read describe an organisation's governance — who delegated what to whom, under what bound, and what was decided. **A field you read wrongly becomes a claim in an evidence export**, and the difference between "the server said no" and "the client did not model the answer" is exactly the difference between a finding and a mistake.

Required, optional, and the third thing

Across the 78 models this package exports there are **263 required fields and 226 optional ones** — so a bit under half of everything you can read is optional. Seven models have no required field at all.

"Required" and "optional" here are properties of **this client's model**, not promises by the deployment. That is the distinction the whole chapter turns on:

you see	it means
a required field, populated	the deployment sent it — the client refuses to construct the object otherwise
an optional field, populated	the deployment sent it
an optional field, <code>None</code>	the deployment sent <code>null</code> , or sent nothing at all — you cannot tell which
a required field missing	you never got an object; the call raised instead

The third row is the one to internalise. An optional field reading `None` collapses three different situations — the deployment does not have this value, the deployment does not implement this field, and this deployment version predates it — into one indistinguishable answer. If a decision depends on the difference, read the raw body:

```
raw = await client._http.request("GET", f"/api/v1/agents/{agent_id}")
print("field present in the response:", "model_id" in raw)
print("field value:", raw.get("model_id"))
```

Absent and `null` are different facts and only the raw body carries the difference. Reach for it for a diagnosis, not as a habit — the typed path is the one that will keep working.

What each of the models you will read most actually guarantees

model	required	notable optional
<code>sdk:aegis_sdk.User</code>	6 — <code>id</code> , <code>email</code> , <code>name</code> , <code>organization_id</code> , <code>organization_name</code> , <code>role</code>	<code>personas</code> , <code>status</code> , <code>mfa_enabled</code> , <code>last_login_at</code>
<code>sdk:aegis_sdk.Agent</code>	8 — including <code>agent_type</code> , <code>unit_type</code> , <code>status</code>	<code>model_id</code> , <code>system_prompt</code> , <code>capabilities</code> , <code>description</code> , <code>workspace_id</code>
<code>sdk:aegis_sdk.Objective</code>	9 — including <code>status</code> , <code>created_by</code> , <code>agent_id</code>	<code>priority</code> , <code>assigned_to</code> , <code>metadata</code> , <code>completed_at</code> , <code>workspace_id</code>
<code>sdk:aegis_sdk.TrustChain</code>	2 — <code>agent_id</code> , <code>genesis</code>	<code>delegations</code> , <code>status</code> , <code>human_origin</code>
<code>sdk:aegis_sdk.APIKey</code>	4 — <code>id</code> , <code>name</code> , <code>key_prefix</code> , <code>created_at</code>	<code>scopes</code> , <code>expires_at</code> , <code>last_used_at</code>
<code>sdk:aegis_sdk.AuthToken</code>	1 — <code>access_token</code>	<code>refresh_token</code> , <code>expires_in</code> , <code>expires_at</code> , <code>user</code>

Two rows deserve a second look.

TrustChain requires only two fields, and `delegations` — the thing you are almost certainly reading it *for* — is optional and defaults to empty. An empty delegation list therefore means either "this chain has no delegations" or "this response did not carry them", and the model cannot distinguish them. Do not report "no delegated authority" from an empty list without confirming the field was sent.

AuthToken requires only `access_token`. `expires_in` defaults to 3600 when the deployment sends nothing — a *client-side* default, not something the deployment told you. A refresh scheduler built on `token.expires_in` may be scheduling against a number nobody sent. Prefer `expires_at` when it is populated, and treat its absence as "unknown", not "one hour".

That is the general shape of the hazard: **a default is an answer the client invented**. It is indistinguishable, at the point of use, from an answer the deployment gave.

Fields the deployment sends and you never see

Unknown fields are silently dropped. Every model ignores keys it does not declare. Constructing a `sdk:aegis_sdk.User` with an extra field the model does not know about succeeds, and the attribute simply does not exist on the result.

This is the right default — a client that raised on a new server field would break on every deployment upgrade — and it has a real cost worth naming: **if your deployment is newer than your SDK, its new fields are invisible to you and nothing says so**. The typed path cannot show you what it does not know about. The raw body can, and comparing the two is a five-line check worth running once after any deployment upgrade:


```
from aegis_sdk import User

raw = await client._http.request("GET", "/api/v1/auth/me")
print("sent but not modelled:", sorted(set(raw) - set(User.model_fields)))
```

One field is dropped by design and costs you a credential: the one-time secret returned when you create an API key. That is 04.3, and it is the sharpest instance of this whole class.

Values the client rewrites on the way in

Two rewrites happen quietly and both are defensible. Know about them, because each turns a server value into a different client value.

An unrecognised agent type becomes `unknown`. If a deployment introduces an agent type this SDK release predates, `sdk:aegis_sdk.Agent` coerces it to `sdk:aegis_sdk.AgentType`'s `unknown` member rather than failing. That keeps one unfamiliar row from failing a whole list — the correct trade for a read model — and it means `agent_type == "unknown"` is not a state the platform has. It is this client telling you it is out of date. Treat it as a prompt to upgrade, not as a property of the agent.

Two fields on a streamed session message are generated locally. In

`sdk:aegis_sdk.execution.SessionsModule.stream_messages`, the wire events carry no message id and no timestamp, so the client synthesises both: `id` is a fresh random identifier and `created_at` is **your machine's clock at the moment of receipt**. Neither is a server fact. Do not correlate on that `id` — nothing on the deployment knows it — and do not put that `created_at` in an audit narrative, because it measures when your process read the event, not when the event happened.

The repr shows less than the object holds

Two fields are deliberately hidden from the printed form of their objects: `email` on `sdk:aegis_sdk.User`, and both tokens on `sdk:aegis_sdk.AuthToken`. They are present on the object and simply do not appear when you print it.

That protection is correct — a logged repr is how credentials and personal data escape — and it produces one specific confusion worth pre-empting. 01.3 tells you to `print(me)` as your first smoke test, and the email you are looking for will not be in the output. It is there:

```
me = await client.auth.get_current_user()
print(me)           # no email – hidden from the repr, by design
print(me.email)     # the value, when you have decided you want it in your logs
```

A checklist for reading any response

1. **Ask whether the field is required or optional** before you branch on it.

`type(obj).model_fields["x"].is_required()` answers it on your own build, and the answer is a property of your SDK version.

2. **Never read `None` as a fact about the deployment.** It is `null`, or absent, or not implemented — three different findings wearing one value.
 3. **Never read a default as an answer.** `expires_in` of 3600 and `page` of 1 are the client's inventions until you have confirmed the deployment sent them.
 4. **Never read an empty collection as "none exist"** unless the field was required. An empty `delegations`, an empty `personas` and an empty page all have two possible causes, and only one of them is about your data.
 5. **Drop to the raw body for a diagnosis, and only for a diagnosis.** It is the only place where present-versus-absent is visible, and it is the wrong long-term dependency, because it is the layer with no contract at all.
 6. **When a reading will become evidence, record what you actually observed** — the operation you called, the status you got, and the field you read — rather than the conclusion you drew from it. That distinction is the whole of 02.6 and it starts here, at the point where a `None` becomes a sentence in a report.
-

Next: Part 05 — The web console

Part 05 — The web console

Audience: anyone with a login — including you.

This part is written entirely from the running application: no code, no configuration, no assumed knowledge of how any of it is built. What it contains instead is a plain account of what you are looking at, screen by screen.

Two reasons an architect or operator should read it anyway. First, this is where *you* answer a held decision — approvals are attributable to a named human, and the console is where you are authenticated as yourself rather than as a provisioning key. Second, this is what everyone else in the organisation will ever see of the system you designed; chapter 05.5 in particular, on why things are invisible, is the screen-side face of the clearance model you set up in 02.3, and it is worth knowing what that model looks like from the outside.

The harness equivalent of everything here is part 02. Where this part says "click", part 02 says which call it makes.

The chapters

#	Chapter	What it answers
1	First login	What you see the first time, and what to do with it
2	Submitting an objective	How to ask for work, and why it asks you questions back
3	Your inbox	The things waiting for you, and which ones are urgent
4	When something is held	Why work stops, what "held" means, and where to look — including why the approvals queue can be empty even though something was held
5	What you can see, and why	Why some things are invisible to you, and why that is deliberate
6	The rest of your workspace	The five remaining entries in your navigation, and whether you can change anything in them

Three things worth knowing before you start

- 1. Aegis will stop and ask you things. That is the product working, not failing.** Agents here operate inside bounds someone set deliberately. When an agent reaches the edge of its bounds — it wants to spend more than its budget, or touch data outside its remit, or take an action classed as needing a human — it stops and waits for a person. You may be that person. Chapter 4 is about this and is the chapter most people need first.
- 2. You will not be able to see everything, and that is by design.** Aegis treats organisational units as knowledge boundaries. Being a colleague of someone does not grant you sight of their work. When you hit a boundary, the system tells you that you hit one rather than pretending the thing does not exist. Chapter 5 explains how to tell the two apart, and what to do about it.
- 3. Nothing you do here is invisible.** Every action — yours and every agent's — is recorded in a tamper-evident audit trail. This is not surveillance of you; it is the evidence that lets your organisation stand behind what its agents did. It also means that if something goes wrong, there is a record of what actually happened rather than a reconstruction.

About the screenshots

Every picture in this part was taken from a fictional company. That company is Northwind Manufacturing — its people, agents, objectives and figures invented, its email addresses on a reserved domain that cannot reach a real mailbox. Nothing in any image is any organisation's real data, and none of it came from a live deployment. That is a hard rule rather than a preference: blurring a real name does not remove it, and the only safe method is to photograph data that was never real.

Each marked placeholder names the exact screen, the state it has to be in, and who has to be logged in to see it. Read those as instructions: each tells you which screen the paragraphs beside it are describing, so the text stands on its own.

Some screens are photographed empty, and the caption says why. The demo company has an organisation, roles, agents and work; it does not have every kind of record. Chapter 4 makes the argument that matters here: an empty queue is not evidence that nothing was held.

Where an image does not quite match what you see, the difference will be your own organisation's configuration — its unit names, its clearances, what your role is allowed to do — not a different version of the software. The shape will be the same.

Next: 05.1 — First login

05.1 — First login

What actually happens when you sign in

You will not land where you expect. Almost everyone's first sign-in goes to a **Getting Started** walkthrough rather than to the main screen, and that is deliberate. This chapter walks the first ten minutes in order.

Signing in

The sign-in page is titled "**Sign in to Aegis**" and offers two ways in:

- **Email and password.** There is a "Remember me" tickbox and a show/hide control on the password field.
- **Single sign-on**, under a divider reading "**Or continue with**" — Google, Microsoft, Apple and GitHub.

Not all four sign-on buttons will necessarily work. If your organisation has not enabled one, it appears greyed out reading "**{Provider} unavailable**", and hovering explains that provider is not available on this deployment. If a button looks normal but fails, that is a different problem — see below.

Sign in to Aegis
Enter your credentials to continue

Email
you@example.com

Password
Enter your password

☐ Remember me

Sign In

OR CONTINUE WITH

Google unavailable

Continue with Microsoft

Apple unavailable

GitHub unavailable

Don't have an account? [Register](#)

*The sign-in page on the demo tenant. Three of the four sign-on buttons read "unavailable" and are greyed out; only **Continue with Microsoft** is live, because that is the only provider this deployment has enabled. Yours will differ. Note the eye icon inside the password field, and the "Register" link at the foot — both described above.*

Two more things to expect at sign-in:

- **Multi-factor authentication follows your organisation's settings.** Where your organisation has enabled it, you are challenged for a second factor after your password. [immediate roadmap]
- **Rate limiting surfaces as a message rather than a separate page.** Get the password wrong repeatedly, or trip a rate limit, and you get a red message box — often *"Too many requests. Please wait a moment and try again."* Wait, then try again.

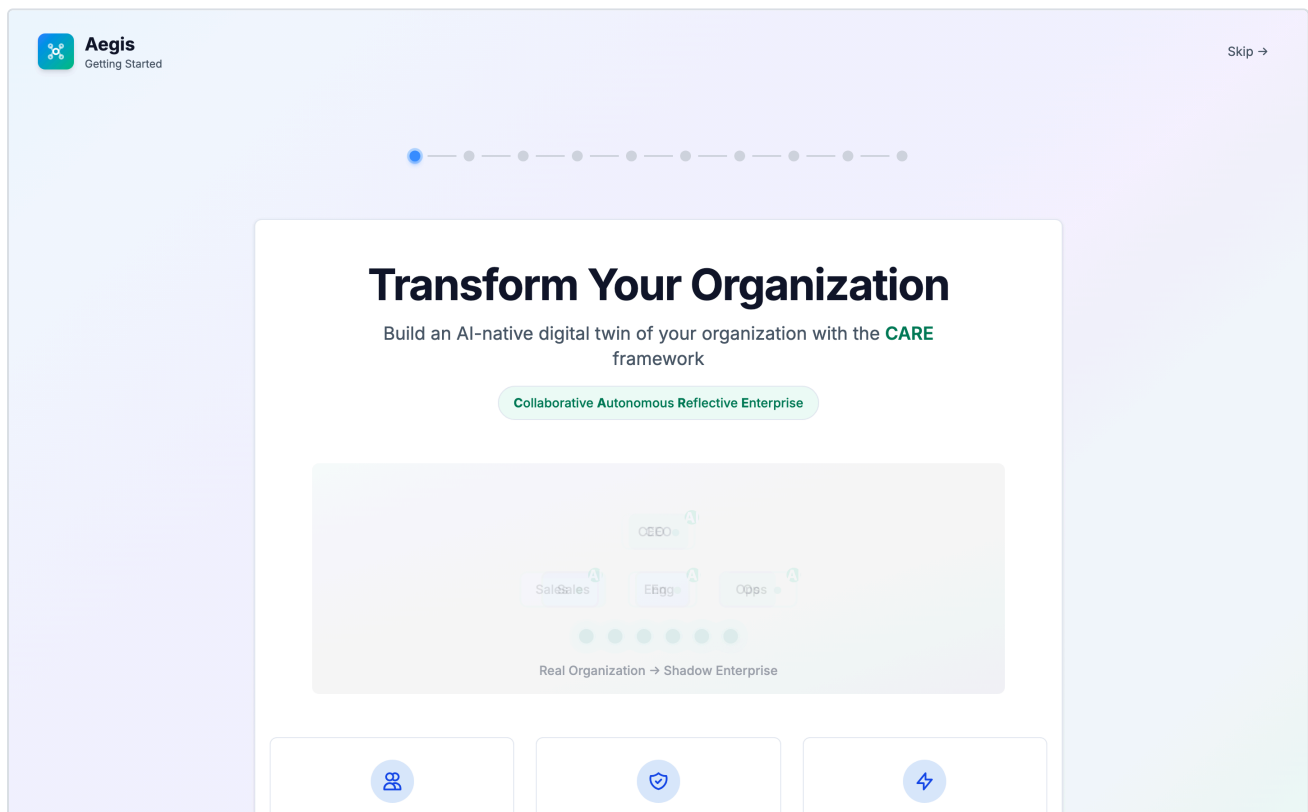
If **sign-in fails**, a red box appears in the corner titled **"Login failed"** with the reason underneath. The reason comes from the server, so it is usually specific. If you arrived back at the sign-in page after being bounced from single sign-on, the message is **"SSO Authentication Failed"** — start again, and if it recurs, it is a configuration matter for whoever set up your organisation, not something you can fix from this screen.

Validation on the form itself only runs when you press **Sign In** — it will not nag you while typing.

Getting Started: a walkthrough, not a setup form

Your first stop after signing in is a full-screen walkthrough. **It does not configure anything.** It is ten short screens explaining how the system thinks: what trust postures are, what a trust chain is, how agents are bounded, and what the adoption phases mean. One screen introduces the agent assigned to you, if you have one.

There is a **Skip** link in the top right. Skipping costs you nothing functionally — nothing is left unconfigured — but chapters 2 and 4 of this handbook will make considerably more sense if you spend the ten minutes.



The first screen of the walkthrough — this is where most people land instead of the main application. The row of dots is the progress indicator: twelve steps, and you are on the first. **"Skip" is in the top right** and works from any step. The panel introduces CARE — the Collaborative Autonomous Reflective Enterprise framework, an open standard published by the Terrene Foundation that Aegis implements.

The walkthrough is remembered per browser

Whether you finished or skipped it is remembered **in the browser you used**, not in your account. Sign in from a different browser, a different computer, or a private/incognito window, and the walkthrough appears again from the beginning.

Nothing has been reset and you have lost nothing. Press **Skip**.

Until the walkthrough is finished or skipped, it stands in front of everything else — you cannot navigate around it to the main screens.

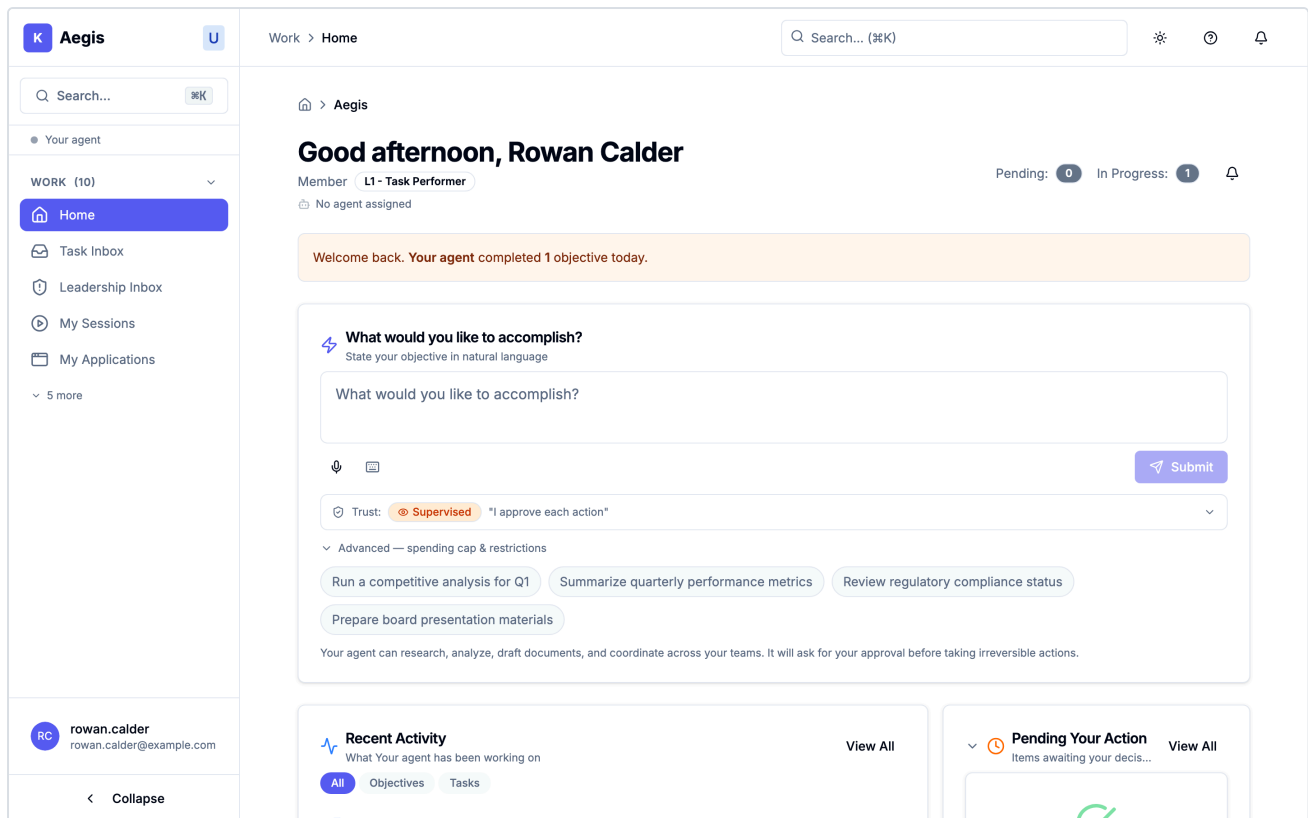
Your home screen

Past the walkthrough you arrive at your home screen, which greets you by name with the time of day and shows your level and role.

If you are brand new, most of it is empty, and the empty states are worth recognising so you do not think something is broken:

Panel	What a new account sees
Recent Activity	"No recent activity. Submit an objective to get started."
Pending Your Action	a green tick and "All caught up!"
In Progress	"No active objectives"
Agent status	"No agent assigned", if none has been

The orange summary banner that regular users see across the top does not appear until you actually have something pending or completed. Its absence is normal on day one.



Home, for a seeded demo user. Four things worth finding: the **greeting** with your role beneath it (`Member` , `L1 - Task Performer`) and `No agent assigned` ; the **objective box**, which is the main thing you will use; the **trust selector** under it, reading `Supervised - "I approve each action"` , which is where you choose how much latitude the agent has for this piece of work; and the counters top-right, here `Pending: 0` and `In Progress: 1` . The suggestion chips below the box are starting points, not limits.

The centre of the screen is a box headed "What would you like to accomplish?" That is the main thing you do here, and it is chapter 2.

Beneath it sit four example prompts — things like "Run a competitive analysis for Q1". Read them as **examples of the shape of a good request**, not as suggestions tailored to you or to your organisation. They are the same four for everybody.

Finding your way around

The left-hand sidebar is filtered to you. **You will not see every section this handbook or a colleague mentions**, and an item missing from your sidebar is usually not a fault — it is a section you have not been given. Chapter 5 covers how to tell that apart from something being broken.

Two navigation behaviours worth knowing now:

- Some items sit behind a "**Show more**" control in a section rather than at the top level. If you cannot find something named in this handbook, expand the section before concluding you lack access.

- **An unrecognised web address returns you to your usual landing page** rather than to an error screen. If you followed a link and ended up somewhere unexpected, the link was stale — nothing went wrong with your account.

Value Chains appears in the sidebar and opens shortly. [immediate roadmap]

If your screen does not match a colleague's

Two accounts with the same job title can legitimately see different things, because what you see depends on the role you were given, the organisational unit you sit in, and your clearance. That is the system working as designed, and chapter 5 explains the reasoning.

There is also a mechanical reason worth knowing: **what your account can see is cached in the browser**, so the same account in two different browsers can present differently until each refreshes. If your access looks wrong rather than merely narrow — a section you used yesterday has gone — sign out fully and sign back in, which refreshes it. If it persists after that, it is worth reporting.

Next: 05.2 — Submitting an objective

05.2 — Submitting an objective

Ask for an outcome, not for a task

The box on your home screen is headed "**What would you like to accomplish?**" and the prompt underneath asks you to state it in natural language. That is meant literally. You are describing a result you want, in a sentence or two, the way you would to a capable colleague.

You do not need to break the work down. The system does that, shows you its plan, and waits for you to approve it before doing anything.

Writing it

The box behaves like a message field:

- **Enter** submits. **Shift+Enter** starts a new line.
- **Cmd/Ctrl+K** jumps your cursor into it from anywhere on the page.
- It grows as you type, up to **2,000 characters**. A counter appears once you are past a hundred or so, and warns you as you approach the limit.
- There is a microphone control if you would rather speak it.

Once you have typed a few characters it will start offering completions based on what you are writing. These are genuine suggestions drawn from the system, unlike the four fixed examples shown on an empty box.

The two controls underneath, which are the important part

Directly below the box are two collapsed controls. Most people ignore them. They are the only place you set the terms of the work *before* it starts, and they are worth thirty seconds.

Trust — how much you want to be asked

A row reading **Trust:** followed by a badge and a plain-English phrase. Four choices, and the phrasing on screen is the clearest summary anyone has written:

Setting	What it means for you
Supervised	"I approve each action"
Shared Planning	"I review the plan"
Continuous Insight	"I monitor progress"
Delegated	"Full autonomy"

Expanding the control draws a spectrum from **more control** to **more autonomy**. Pick by how much you want to be interrupted, and how reversible the work is. Drafting a document is a different risk from sending it.

You may not be offered all four. Your organisation sets a ceiling, and anything above it is not shown. If **Delegated** is missing, that is a decision someone made for your organisation or your role — not a fault, and not something this screen can override.

Advanced — spending cap and restrictions

A collapsible section reading **"Advanced — spending cap & restrictions"**, which shows **"(set)"** when you have put something in it. Two fields:

- **Spending cap for this objective (USD)** — blank means no per-objective limit.
- **Blocked actions** — specific things this piece of work must not do.

Setting a cap here binds *this* objective. It is the cheapest way to be comfortable letting something run with less supervision: rather than approving each step, you bound what the whole thing may cost.

What happens when you press Enter

1. Your objective is saved, and you get a confirmation reading **"Objective created"** with a **View objective** link.
2. A short **initialisation** window appears, showing the trust chain being established, any sub-agents involved, and the constraints in force. This is the system writing down what it is allowed to do before it does anything.
3. You are taken into the work session, and the conversation starts.

[SCREENSHOT] id: `work-home-objective-compose` · route: `/work/home` · state: `northwind.objective.compose` · persona: `design_lead` *Must show:* the objective box with text entered, the Trust control expanded, and the Advanced section open with a spending cap set.

One message worth recognising: if your objective saves but the session fails to open, you will see **"Objective saved — could not open a work session"** and, importantly, *"You do not need to submit it again."* Take that at its word. Re-submitting produces a duplicate. Use **View objective** to find it.

It will ask you questions back

This surprises people, so: **being asked questions is the normal path, not an error**. Shortly after you submit, the agent posts something like *"I've received your objective... Let me analyze this and prepare some questions"*, and then a short numbered list of things it wants to understand.

Underneath the questions is the instruction that matters:

"Feel free to address these in your own words — you don't need to answer each one separately."

There is no form and no Skip button. You reply in the ordinary message box, in prose, the way you would to a colleague who asked three things in one email. Answer all of it, some of it, or say "use your judgement and proceed" — the work moves on either way. If your reply cannot be processed you will be told, and nothing is lost; you simply reply again.

Where the objective is broad, the questions are correspondingly broad — *"What format should the deliverable take?"* and similar — and the system says so before listing them. Treat those as a checklist that sharpens the brief.

Then it shows you the plan

Next comes a plan: a set of tasks, drawn as a graph, with a summary of **Tasks**, **Estimated time**, **Estimated cost** and **Agents**. Below it, a confirmation block headed **"Review & Approve Plan"**.

While a number is still being worked out it reads **"Estimating..."** or **"Assigning..."** rather than showing a misleading zero. Where the cost matters to you, set a spending cap as well as reading the figure — the cap is the thing that binds.

Approving raises a final check — **"Approve Execution Plan?"** — which spells out that the agent will delegate to specialist agents and consume resources, and repeats the estimated cost. Two buttons: **Review Plan Again** and **Approve & Start Execution**.

You can push back. Declining does not cancel anything. You are asked to describe the changes you want, the plan is revised, and there is an **Approve Original Plan** escape hatch if you change your mind.

Watching it run

Inside the session you will see status wording as things progress — **Initializing**, **Active**, **Paused**, **Completing**, **Completed**, **Failed**, **Cancelled** — with controls to pause, resume or end the work.

Elsewhere in the interface the same work is labelled by **what it needs from you**, which is more useful:

Label	Meaning
Needs Your Input	it is waiting on you
Review Plan	a plan is ready for approval
Decision Needed	something requires you to decide
Running	in progress, nothing needed
Completed / Failed / Paused / Cancelled	as they read

The first three pulse gently. Those are the ones to act on.

[SCREENSHOT] id: `work-session-in-flight` · route: `/work/sessions/:id` · state: `northwind.session.in_flight` · persona: `design_lead` *Must show:* the conversation with the clarification questions answered, the approved plan graph, and the session status chip.

On connection messages: you may see *"Reconnecting to stream..."* or a red banner offering **Reconnect**. Read the small print, because it is accurate — *"Your session continues on the server."* Your work is not lost when your connection drops. Messages you type while disconnected may not arrive, so re-send those.

When work stops and waits

If the agent reaches the edge of what it may do, the work stops and waits for a person. That is chapter 4, and it is the chapter most people need.

Next: 05.3 — Your inbox

05.3 — Your inbox

There are three of them, and they are not the same thing

This is the part of the interface most likely to confuse you, because "inbox" appears three times in the sidebar and the three lists hold genuinely different things. In short:

Where	What is in it	What you do there
Task Inbox	pieces of work — yours, and unclaimed work you could take	claim it, work on it, complete it, hand it back
Pending Requests	an agent has asked a human for something and is waiting	claim it and respond
Leadership Inbox	notifications routed to you because of your position	read them

The first two are things to **do**. The third is things to **know**.

You may not have all three. Pending Requests in particular is shown to a narrower set of people, and it sits behind the "**Show more**" control in the sidebar rather than at the top level — so check there before concluding you do not have it.

Task Inbox

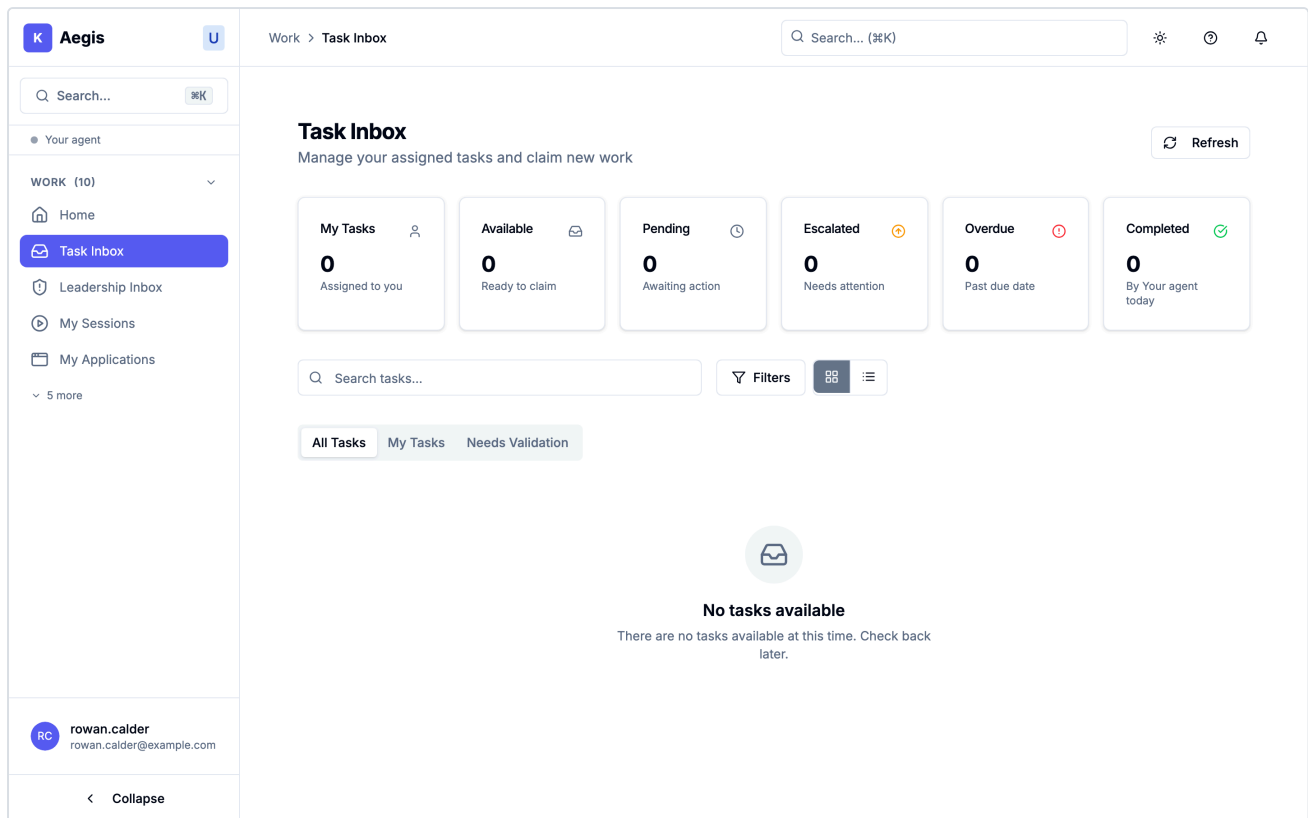
Six counters across the top — **My Tasks**, **Available**, **Pending**, **Escalated**, **Overdue**, **Completed** — and three tabs: **All Tasks**, **My Tasks**, and **Needs Validation**.

The distinction that matters is between work that is *yours* and work that is *available*. Unclaimed work sits in All Tasks; claiming it makes it yours and takes it off everyone else's list. There is a confirmation step, because claiming is a commitment other people can see.

On each card:

- **Claim** — take it.
- **Work on Task** — opens a working session and takes you into it.
- An overflow menu with **Complete**, **Reject**, **Release**, **Escalate** and **Delete**.

Release is the one people miss. If you claimed something you cannot finish, release it rather than leaving it to go overdue — it returns to the available pool. **Escalate** is for work that needs someone above you, not for work you simply do not want.



The Task Inbox on the demo tenant, and it is **empty** — all six counters at zero and "No tasks available. There are no tasks available at this time." Read it as "nothing for you here", never as "nothing here": a list emptied by your permissions and a genuinely empty list are the same picture. The six counters and the three tabs — All Tasks, My Tasks, Needs Validation — are the parts to learn now, because they are where the numbers will appear when there is work.

Reading urgency

Each card carries a priority — **Low, Normal, Medium, High** or **Urgent**, with the top two in red — and a due date that turns red and reads "**Overdue by ...**" once it passes. Work that has been escalated is badged as such.

The list does not sort by priority. Priority is shown, not enforced, so an Urgent item can sit below routine work. Sort or filter yourself, and do not assume the top of the list is the most important thing. (The **Pending Your Action** panel on your home screen *does* order by priority, which makes it the better first glance of the day.)

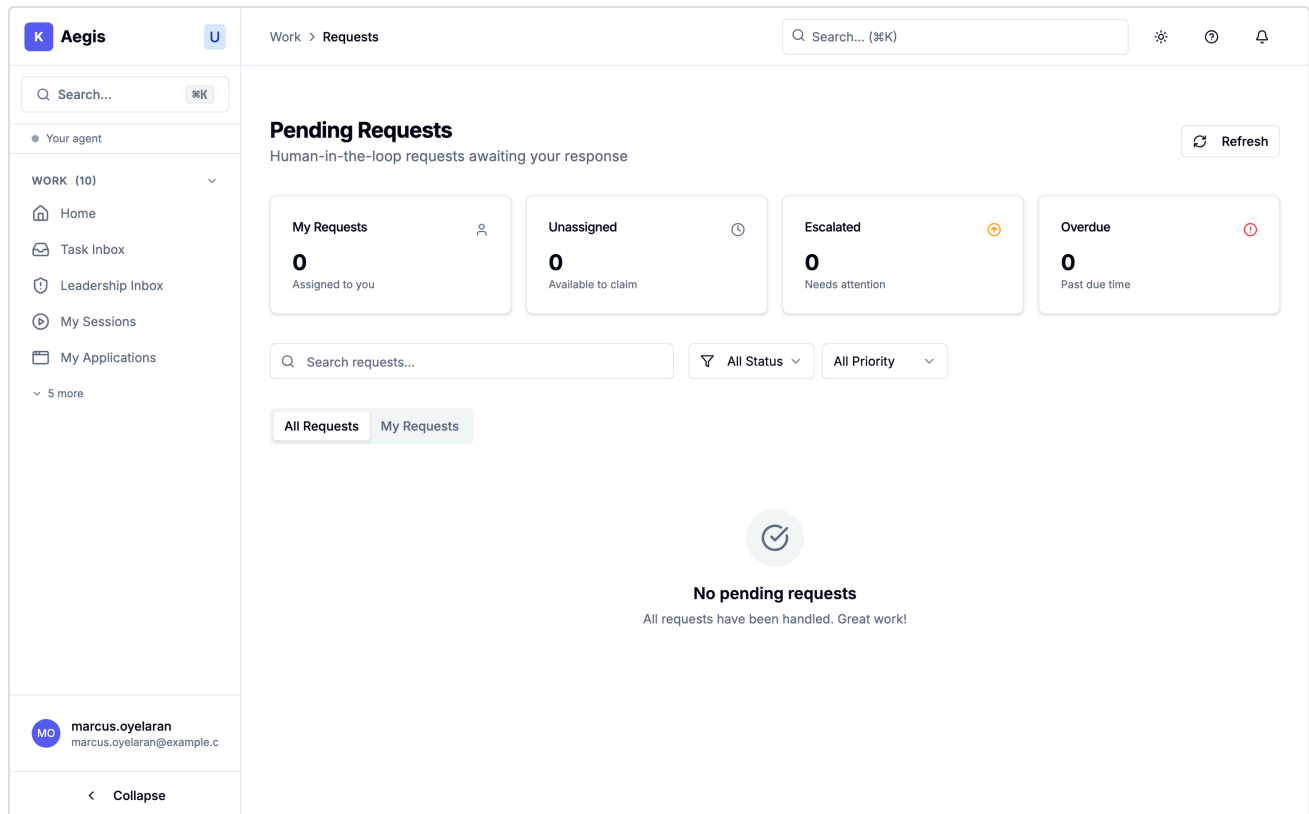
Pending Requests

Where an agent has stopped and asked a human for something. Four counters — **My Requests, Unassigned, Escalated, Overdue** — a search box, and filters for status and priority.

Rows are either assigned to someone, showing their avatar, or marked "**Unassigned**". The one action from the list is **Claim** on an unassigned row; clicking through opens the request in full.

Statuses you will see: **Pending, Assigned, In Progress, Completed, Escalated, Timeout, Cancelled**. **Timeout** means nobody answered in time — the request expired rather than being decided, and whatever was waiting on it did not get what it needed.

Empty, it reads *"All requests have been handled. Great work!"*



*Pending Requests, subtitled "Human-in-the-loop requests awaiting your response". Also empty on the demo tenant. The reassuring copy — "All requests have been handled. Great work!" — is worth noticing precisely because it is **not** evidence that nothing was requested of anyone. It reports on what you can see.*

Leadership Inbox

Notifications routed to you because of where you sit in the organisation. It polls every thirty seconds; there is no live push, so a brief delay is normal.

You can **Mark read** on a row, and toggle between **Show unread only** and **Show read**. That is all — it is a record to be aware of, not a queue to work. Rows carry a priority badge and a data-classification badge, and unread ones are marked.

The expanded row carries the underlying notification

Expanding a row shows the raw underlying notification, including a block of technical data. You will see lines labelled **Event** and **Routing** written in system vocabulary rather than plain English.

Read the summary, the priority and the classification badge; the technical block is there for when you are chasing something specific. If a notification matters and you need the event type decoded, the person who set up the routing can tell you what it means.

[SCREENSHOT] id: leadership-inbox-unread · route: /agentic/leadership-inbox · state: northwind.inbox.unread · persona: operations_director *Must show:* the unread/read toggle, priority and classification badges, and one row expanded — with any identifying values in the technical block obscured before publication.

What "empty" does and does not tell you

Every one of these lists shows a cheerful empty state when it has nothing for you: *"No tasks available"*, *"All caught up!"*, *"You're all caught up."*

Read those as **"nothing for you here"**, not as **"nothing here"**. These lists show what you are entitled to see, and they do not tell you when something has been filtered out — there is no "3 items not shown" notice anywhere. An empty list and a list emptied by your permissions look identical.

This matters in one specific situation: a colleague says they have sent you something, and your inbox is empty. That is not necessarily a bug and it is not necessarily them being wrong — it can be a visibility boundary. Chapter 5 is about how to tell, and what to do.

A practical order for the morning

1. **Home screen, Pending Your Action** — it is the only list ordered by priority.
2. **Task Inbox, My Tasks** — check Overdue before Available.
3. **Pending Requests** — an agent is blocked on each of these, so they cost more than they look. A request that times out is work that failed waiting.
4. **Leadership Inbox** — awareness, not action. Last.

If something in any of these lists is stopped and waiting rather than simply assigned to you, that is a **held** action, and it works differently. That is the next chapter.

Next: 05.4 — When something is held

05.4 — When something is held

Held means the system worked

An agent reached the edge of what it was allowed to do, stopped, and asked for a person. Nothing has failed. Nothing is broken. The bounds were set deliberately, and this is what reaching them looks like.

This is worth saying plainly because the experience does not feel like success. Work you asked for has stopped, and a queue is asking you to decide something. The instinct is to treat it as an obstruction. It is the opposite: an agent that *could not* be stopped at this point would be the thing to worry about.

The stopping itself is real and reliable — it is one of the controls this platform enforces rather than merely records.

Where held work lives

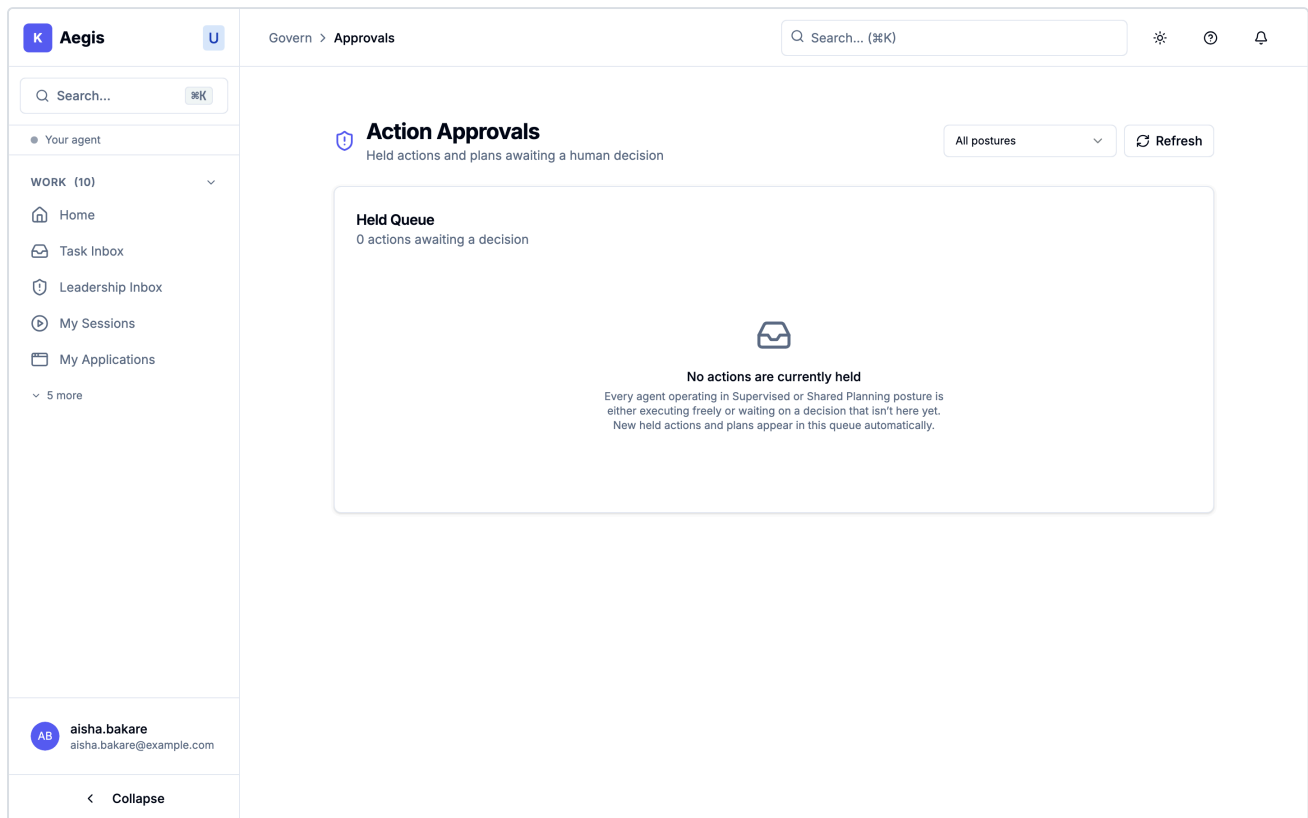
The queue lives at `/govern/approvals`, and the navigation entry that points at it sits inside the **GOVERN** section. The page admits both the `admin` and the `user` persona; a standing navigation entry for every persona it admits is on its way. [immediate roadmap]

In the meantime, go to `/govern/approvals` directly in the address bar and it will load. If it does not, you hold neither persona and you need an administrator rather than a different URL.

Action Approvals, subtitled *"Held actions and plans awaiting a human decision"*. The queue is headed **Held Queue** and refreshes itself every five seconds — there is a badge saying so, and it is accurate. You do not need to reload.

Empty, it reads **"No actions are currently held"**, and explains that agents under supervision are either running freely or waiting on a decision that is not in this queue.

[SCREENSHOT] id: `approvals-held-queue-countdown` · route: `/govern/approvals` · state: `northwind.approvals.pending` · persona: `operations_director` *Must show:* the Held Queue with one card — posture badge, request type, agent's reasoning, proposed action, and the countdown timer mid-way.



The approvals queue with nothing in it — the state this chapter warns you not to over-read. The page's own explanation is the important part: every agent in a supervised or `shared_planning` posture "is either executing freely or waiting on a decision that isn't here yet." **An empty queue is not proof that nothing was held.** Note the "All postures" filter top-right: if you are looking for something and not finding it, check that first.

Reading a held item

Each card shows:

- **Which agent** stopped, and **the objective** it was working on.
- **Its posture** — **Supervised** or **Shared Planning**. This is the reason it stopped: an agent under supervision must ask.
- **What kind of decision** — a **Single Action** or a whole **Execution Plan**.
- **Agent's reasoning** — the agent's own account of why it wants to do this.
- **Proposed action** — the specific thing it wants to do.
- **Time left** — a live countdown.

What the card tells you, and what it leaves to you

Three properties worth knowing, so you are not looking for things that are not there:

The reason is the agent's, not a policy's. "Agent's reasoning" is the agent explaining itself, and the posture badge is the whole answer to "why did this stop" — it stopped because it is supervised. Where the agent supplied no reasoning, the field reads **"No reasoning was provided."**

It does not name a decider. Anyone with access to the queue can act on any item in it, which keeps a held action from waiting on one person's availability. In practice that means **a held item is nobody's job unless someone makes it theirs** — if your team relies on this queue, decide between you who watches it.

"Requested by" shows an account identifier, not a person's name. You may have to ask who that is.

The countdown, and what running out means

Each card shows **Time left**, counting down, amber under a minute and red under thirty seconds. Expired, it reads **"Window closed"** with a blunt line: *"This request should have timed out already — decide now or the agent will fail with a timeout error."*

Take that literally. **A held action that is not decided does not wait forever and does not proceed — it fails.** The work does not quietly continue without approval, which is correct, but it also does not sit there indefinitely. Nobody deciding is itself a decision, and it is the worst one available: the work fails having consumed whatever it already spent.

If the timer cannot be worked out you will see **"Deadline unknown"**. Treat that as urgent rather than as no deadline.

Deciding

Two buttons: **Approve** and **Reject**. Afterwards you get a short confirmation — *"Action approved / The agent has been notified and will proceed."* or *"Action rejected / The agent has been notified and will not proceed."*

If it was **your** request, you also get **Withdraw request**, which cancels it. That is the right move when you have changed your mind or realised the agent misunderstood — better than rejecting, because it does not read as a judgement on the agent's proposal.

You cannot approve your own request

The separation between requesting and approving is deliberate, and it is one of the things that makes the audit trail worth anything. The refusal is enforced on the server: pressing **Approve** on a card you raised yourself returns a red message reading **"Could not approve this action"**.

Use **Withdraw request** if you want your own request gone, and find a colleague if it genuinely needs approving.

Your work seems stuck. What to check.

Start with the approvals queue.

If your item is there, it is genuinely waiting for a person. Approve or reject it and the *same* action resumes from where it stopped; nothing is re-run and nothing is lost. There is a **five-minute** window by default — past it the request times out and the work does not proceed. If you are waiting on somebody else, tell them. Do not assume they were notified.

🛑 **An empty queue does not mean nothing was held.** Work can stop *because* a human decision was required and there was nobody to ask. That stop, and the reason for it, are reported on your session view alongside the ones that reach the queue. [immediate roadmap]

Approvals queue	What it means	What to do
Your item is there	Held and waiting — someone must decide	Approve or reject, within five minutes
Empty, session still shows active	It stopped because nobody could be asked, or it is still working	Re-submit under a posture with a human in the loop, which routes the same stop to you
Empty, session completed	Nothing was held	—

The one-sentence version: *the approvals queue shows the work that stopped to wait for you; a stop with nobody to wait for is reported on the session — so read both.*

⚠️ **Why the re-submit works.** A posture with a human in the loop routes the same stop into the queue, where you can see it and act on it. That turns a question you are waiting on into a decision you can make.

Where a held state is recorded

Whether an action of yours is held is recorded and is retrievable through the approvals interface, so an administrator — or you, on the approvals page — can answer "is anything of mine held?" directly.

What this is for

Every one of these decisions is recorded — who asked, what was proposed, who decided, and when — in a tamper-evident trail. That record is the point.

Your organisation is accountable for what its agents do. That accountability cannot be handed to a supplier, a platform or an agent, and "the AI did it" has never been a defence. What Aegis provides is the **proof**: bounded mandates, actions that stop at the boundary, and an evidence trail showing that a named human decided the things that needed a human. You keep the accountability; the system gives you what you need to show how you discharged it.

That is why a held action is worth the interruption. Approving something here is not administrative friction — it is the moment the record captures a human making a decision, which is exactly what an auditor, a regulator or an insurer will later ask you to demonstrate.

Next: 05.5 — What you can see, and why

05.5 — What you can see, and why

Being a colleague does not make you an audience

Most systems assume that if you work somewhere, you can see what happens there, and they restrict the exceptions. Aegis assumes closer to the opposite: your organisation is made of units, units are boundaries, and what happens inside one does not automatically flow to another.

This is not a security posture bolted onto a collaboration tool. It is how the structure works. Someone sitting beside you, on the same floor, at the same grade, may legitimately be unable to see the work you are doing — and you theirs.

The practical consequence is that **"I cannot see it"** and **"it does not exist"** are different sentences. This chapter is mostly about telling which one you are in.

The four ways a boundary shows up

You will meet these in roughly descending order of frequency.

1. It simply is not in your sidebar

By far the most common, and the least alarming: the section is not there. You do not get an error because you never reach a page to be refused from. Whole areas of the product are invisible to you.

Check the "Show more" control in each section before concluding anything. A surprising amount of what people report as "I don't have access" is one click under a collapsed list.

2. A page tells you the area is restricted

If you follow a link into somewhere you are not entitled to be, you get a card headed **"Access Restricted"** which names what the area requires and what you currently have, and suggests contacting your administrator. There is a **Go Back** button.

This message uses internal vocabulary — it will say something like *"This area requires admin persona. Your current persona is user."* Do not be put off by the wording. It is telling you the area needs a different role from yours, and it is naming both so that a request to your administrator can be specific. Quote it verbatim when you ask; it is precisely the information they need.

3. A thing you followed a link to will not load

Open a link to something specific — a session, a task, an escalation — that you are not entitled to see, and you get a load-failure screen with a line about not having permission, and a **Retry** button.

The Retry button will not help. It offers to try again, but permission is not a transient condition. If the message mentions permission, retrying it forever changes nothing; ask for access instead.

4. A list is simply shorter

The quiet one. Lists show what you are entitled to see and say nothing about the rest. There is **no "3 items hidden" notice anywhere in the product.**

An empty inbox and an inbox emptied by your permissions are the same screen.

The question this chapter exists to answer

A colleague says they sent me something. I cannot see it. Is it broken?

Work through it in this order:

1. **Is the section collapsed?** Expand "Show more" in the relevant sidebar section.
2. **Do you have a filter or tab set?** Task Inbox in particular defaults to views that hide things — check All Tasks, and clear any status or priority filter.
3. **Did you follow a link and land somewhere unexpected with no error?** Mistyped or stale links do not produce a "page not found" screen; they quietly return you to your home page. That silent bounce means the link was bad, not that you lack access.
4. **Ask your colleague what they can see.** If they see it and you do not, and steps 1–3 are clear, you are almost certainly on the other side of a boundary — a different unit, a different clearance, or work that was never shared across.
5. **Then ask your administrator,** quoting anything the screen said verbatim.

The order matters because the first three are things you can fix in under a minute, and the last two involve other people's time.

One place the system does tell you

On your home screen, next to your agent information, there is a small **"N agents visible"** indicator you can click. It opens a **Visibility Breakdown** — a short explanation that what you see is scoped to your role and reporting line, and that it includes your direct reports, people below them, and any cross-unit bridges that apply to you.

It is the only place in the product that proactively tells an ordinary user their view is narrowed rather than complete. **It covers agents only** — not tasks, not sessions, not knowledge. But it is worth clicking once, because it makes the principle concrete: you are seeing a scoped view, and the scope has a shape you can ask about.

Why it is built this way

Three reasons, and the third is the one that matters to you personally.

It is structural, not administrative. Knowledge boundaries follow the organisation's real shape, so access does not have to be granted and revoked by hand every time someone moves. There is no long list of exceptions for someone to forget to update.

It bounds agents, not just people. The same boundaries constrain what an agent can reach. An agent working for another unit cannot pull your unit's material into its context, which is what stops a capable agent becoming a general purpose leak. The boundary that occasionally inconveniences you is the same one doing that job.

It makes the record meaningful. Because access is bounded and every crossing is recorded, your organisation can show precisely who and what could reach a given piece of information. That is only worth something if the boundaries are real — a system where everyone can see everything has nothing to attest to.

That last point is the honest answer to "why is this making my life harder". Your organisation remains accountable for what its people and agents do with information, and that accountability cannot be delegated to a platform. What the system provides is the evidence — a defensible account of who could see what, and who actually did. The boundaries are what make that account true.

What to do when a boundary is wrong

Sometimes it genuinely is. You have changed roles, or you have been brought into a piece of work and nobody adjusted anything.

Access is granted by whoever administers Aegis for your organisation rather than from these screens. Go to them, and give them:

- **what you were trying to reach** — the page, or the link you followed;
- **exactly what the screen said**, including any role or permission it named;
- **what you need to do**, in terms of the work rather than the system.

That last one matters most. Access here is granted in terms of roles, units and clearances — not page by page — so "I need to see the Q3 supplier objectives our team is running" gets you a correct answer, and "please give me admin" usually gets you either too much or nothing.

Next: 05.6 — The rest of your workspace. If something has stopped and you are not sure why, 05.4 is the one to re-read — most confusion about this product resolves into either a held action nobody has decided, or a boundary nobody explained.

05.6 — The rest of your workspace

Chapters 1 to 5 cover the things you will do most days: sign in, ask for work, work your inbox, deal with something that stopped, and understand why parts of the system are invisible to you.

Your navigation has five more entries. None of them is hidden from you, and none of them is covered above, so this chapter walks each one briefly — what it is for, what you will see, and whether you can change anything there.

Read this as a tour, not a manual. Most people use one or two of these regularly and never open the rest. Skim it once so nothing in your sidebar is a mystery, then come back when you need one.

My Sessions

A session is one continuous piece of work an agent did for you. When you submit an objective and an agent starts working, that run is a session. This page lists yours.

You can filter the list four ways, and the counts beside each filter tell you how many fall into it:

- **All** — everything, however old.
- **Active** — running right now. This also counts sessions that are still starting up, so a brand-new session appears here immediately rather than seeming to vanish for a few seconds.
- **Paused** — stopped part-way and able to resume. A session waiting on your approval is the usual reason (chapter 4).
- **Completed** — finished, whether it succeeded or not.

If the list is empty you get "**No sessions found**". As chapter 5 explains, an empty list means *nothing you are entitled to see*, which is not always the same as *nothing happened*.

You can delete a session from here. It asks you to confirm first. Deleting removes the session from your list — it does **not** erase the audit record of what the agent did, which is kept independently and is not yours to remove. That distinction is the point of the audit trail, not a limitation of this screen.

[SCREENSHOT] id: `work-sessions-mixed` · route: `/work/sessions` · state: `northwind.sessions.mixed_status` · persona: `field_service_lead` *Must show:* the four filters with non-zero counts, including at least one paused session, so the filter row is legible as a summary.

My Applications

An application here is a piece of software your organisation runs that agents can act on your behalf within. If your organisation has not set any up, or has not assigned you to one, this page will be empty and you can ignore it.

If you have been assigned some, this page is where you see them and, where you are permitted, request changes to them. It carries a second section, **My Pending Change Requests**, listing changes you have asked for that have not yet been decided. A request sitting here is waiting on a person, not lost.

EATP Tasks

EATP is the Enterprise Agent Trust Protocol — one of the open standards Aegis implements, published by the Terrene Foundation. You do not need to know anything about the standard to use this page. What matters to you is that it gives tasks a consistent shape across every system that speaks it, so a task means the same thing here as it does in another tool your organisation runs.

The page opens with four counts, which is usually all you need:

- **Total Pending** — everything outstanding, across everyone.
- **My Tasks** — the subset assigned to you. This is the number to act on.
- **Overdue** — shown in red. Past its due date.
- **Completed** — shown in green.

Below the counts is the task list itself. If your organisation does not use the EATP task surface, expect these to sit at zero; that is a configuration choice.

[SCREENSHOT] id: `work-eatp-tasks-with-overdue` · route: `/work/eatp/tasks` · state: `northwind.eatp.tasks_one_overdue` · persona: `field_service_lead` *Must show:* the four count tiles with a non-zero red Overdue tile, so the colour coding is legible.

EATP Processes

A process is a longer-running piece of work made of several tasks. This page shows the ones you own or take part in, with a search box, a list of the active ones, and a button to start a new one where you are permitted to.

One thing here will look different for you than it does in this handbook. The labels on this page come from your organisation's own vocabulary rather than being fixed in the software. If your organisation calls these things *workflows*, or *cases*, or *engagements*, that is the word you will see — on the heading, on the search box, on the button, and in the empty state. The page works identically whatever it is called. So if your sidebar does not say "Processes", you are still in the right place.

Teams

Read-only, for most people. This page shows the teams in your organisation: who is grouped with whom. Ordinary members can read it — you can see your organisation's team structure without being an administrator.

What you will not have is the ability to change it. The controls that create, rename or re-staff a team are reserved for administrators, so you may see buttons that are unavailable to you. That is the expected state, not a permissions problem to report.

Teams appears twice in the navigation config — once under **WORK** and once under **GOVERN** — and both point at the same page. Which of them *you* see depends on your persona: the WORK entry is the one ordinary users get, the GOVERN one is administrators'. If a colleague describes reaching Teams by a route you do not have, that is why, and you have not found two different screens.

⚠ **A general note for this whole part.** Being *allowed* to open a page and being *shown a link to it* are separate things. Where this handbook says "you will find X in the sidebar", it means the navigation entry exists for an ordinary user, not merely that the page would admit you. If the menu item is not there, check that distinction before concluding you lack permission.

If something in your sidebar is not in this handbook

Your navigation is filtered by what you are allowed to use, so two people in the same organisation see different sidebars. If you have an entry this handbook does not describe, the likeliest explanation is that you hold a role with administrative reach — in which case the screen you are looking at changes how the system behaves rather than how you work inside it, and the corresponding harness path is in part 02 rather than here.

Part 06 — Architecture

Audience: the lead engineer designing a system against Aegis. Parts 02 to 05 answer *how do I do this*. This part answers *what is this* — the platform's internals as you need them in order to design: the split between its Trust Plane and its Execution Plane, the governance objects and how they relate, what a request and a run actually pass through, where enforcement happens, and what this SDK can and cannot reach.

It assumes you are past orientation and have made at least one real call. It does not assume you have seen the platform's source, and it never asks you to: every structural claim here is one you can settle from what you do hold. Where that evidence runs out, the chapter says so rather than filling the gap.

The chapters

#	Chapter	You leave able to
06.1	What core and the SDK are	Say where the boundary is, and which half of the system a given responsibility belongs to
06.2	The request path	Trace a call through the platform, and predict which layer will refuse it
06.3	The governed organisation	Design the structure your agents live in, and know what each object constrains
06.4	Trust and the audit spine	Decide what will be provable later, and design for the decision the platform will actually make
06.5	The execution model	Place work correctly in the run lifecycle, and design for the way a run ends
06.6	Designing against the platform	Choose where your integration attaches, and know what is guaranteed before you build on it

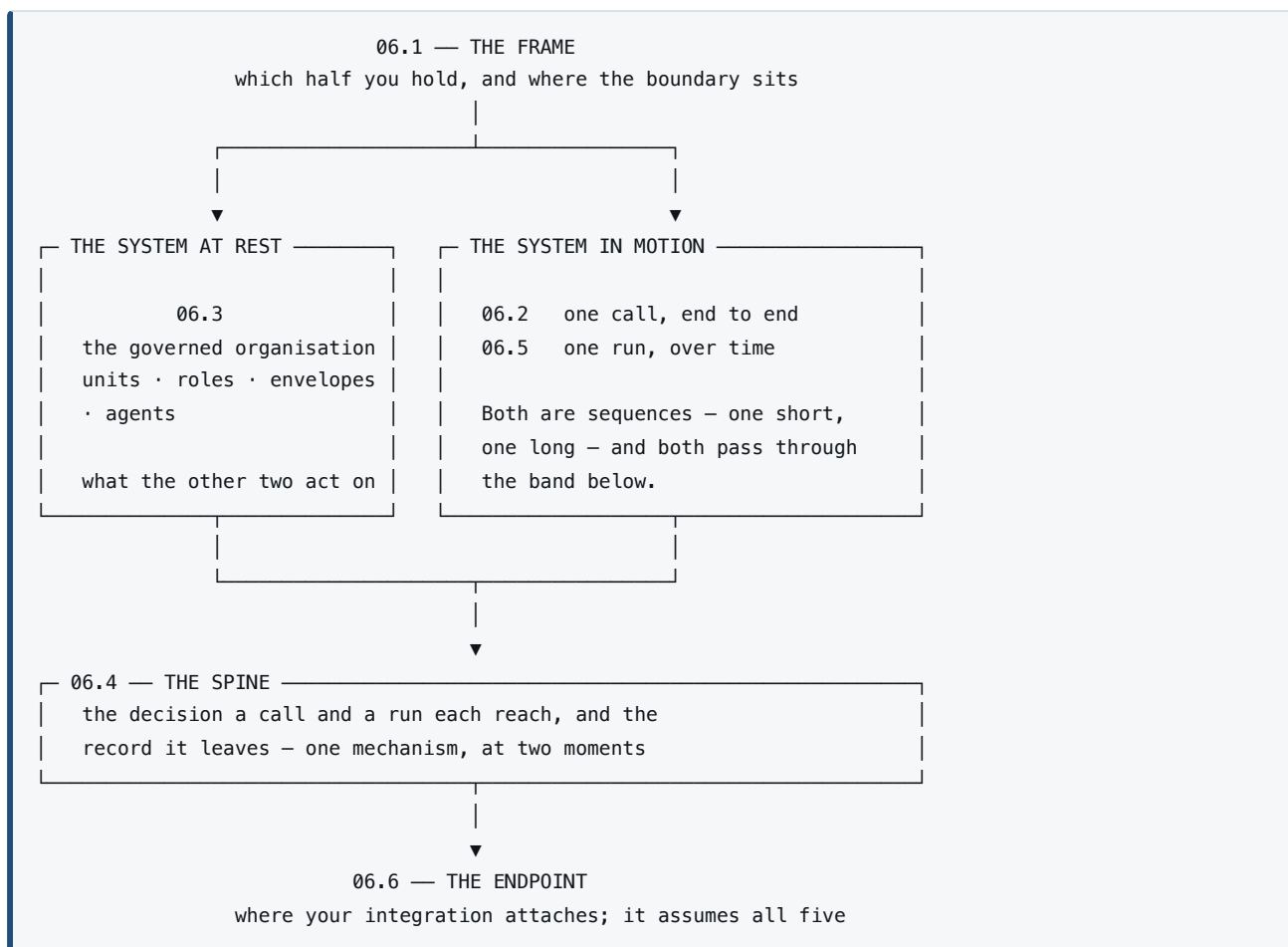
The system the part describes

Two views of the same six chapters, and neither is redundant. This first one is the system they describe. What is above the line is yours; what is below it belongs to the deployment, and you reach it only through the calls you make.

```
aegis, from where you stand
|
|— what you hold ————— 06.1
|   |— the client — one object, pointed at one deployment
|   |— a credential, scoped to something
|
|— what the deployment holds
|   |— the request path ————— 06.2
|       seven layers, from your module's call to the typed response
|   |— the governance objects ————— 06.3
|       organisation
|       |— unit — role — envelope — agent
|       |       |— trust chain — posture
|   |— the Trust Plane ————— 06.4
|       whether an action is permitted, and the record it leaves
|   |— the Execution Plane ————— 06.5
|       work entering, running, carried, and stopped
|
06.6 — where your integration attaches; it assumes all five above
```

The three axes, and the two chapters that bracket them

The second view is how the six divide the system up. 06.1 frames the whole and 06.6 closes it; between them the part runs on three axes — the system at rest, the system in motion, and the spine both of them pass through.



Three things follow from that shape, and each decides how you read the rest.

06.1 frames everything else. Every later chapter is a statement about one side of the boundary it draws, and a statement read without the boundary is easy to misplace — most expensively by concluding that something the platform owns is yours to enforce, or the reverse.

06.4 is a spine, not a topic. Both the request path and the execution model pass through it: a call reaches a decision, and a run reaches several. It sits between them rather than under either, because the decision mechanism and what it leaves behind — `sdk:aegis_sdk.TrustAuditEntry` is the shape of that record — are one thing seen at two moments.

Reading out of order is fine, with one exception. Once 06.1 has landed, 06.2 through 06.5 can be taken in any order. 06.6 should not be: it is the only chapter you are meant to act on directly, and it assumes the other five.

Where this part sits relative to the others

Part 02 is the working reference: it tells you how to stand an organisation up and run work through it. Part 06 does not repeat any of that. What it adds is the system those steps operate on — the reason they come in that order, the reason the shapes are those shapes, and the reason some of them cannot be rearranged. That is what you need when no recipe covers your case.

The practical test is the question you are asking. *"How do I establish a trust chain?"* is part 02. *"Why does authority have to originate with a person at all?"* is part 06, and the answer changes what you can design. *"Why did this call come back refused?"* starts at part 04, which carries the error taxonomy; 06.2 explains what the call passed through on its way to the refusal, which is what tells you whether the answer is to change the credential or to change the design.

That third question is worth one concrete step. A refusal does not identify which layer produced it, and the layers are not interchangeable — a credential problem and a policy problem look alike from outside. So the useful first move is to establish who you are: `api:GET /api/v1/auth/me` reports the principal you are currently holding, as distinct from the one you assumed. Read 06.2 with that in hand and the refusal stops being opaque.

If you can read only one chapter first, read 06.6 and let it send you backwards. It is written to be usable that way.

Two ideas this part keeps returning to

Which half holds it. You hold a client — one object, `sdk:aegis_sdk.Agentic0SClient`, configured against a base URL you supply, since the package ships no default endpoint — and a credential. The deployment holds the state, the policy, the decision, and the record. Almost every structural question in this part reduces to asking which side of that line a thing sits on. The client's configuration surface, `sdk:aegis_sdk.ClientConfig`, is entirely about the connection — which deployment, with which credential, under which timeout and retry policy. None of it is about the organisation's content: that is all asked for at the time, per call.

Existing is not the same as enforced. An operation this client declares is an operation that exists in the surface you were given. It is not evidence that anything is checked when you call it. The distinction sounds pedantic until you apply it: this client declares `api:GET /api/v1/agents`, and that declaration tells you the operation is part of the surface. It cannot tell you what the server checked before answering, and it would read identically if the answer were *nothing*. This is the one architectural claim that gets made wrongly most often, because a route that resolves, a field that is accepted and a status that appears in an enum all *look* like guarantees. They are not. Where a chapter claims something is enforced, it rests on observed behaviour or it is labelled — never on the mere existence of the surface.

How this part knows what it knows

This part describes the platform's internals, and it is worth being precise about how — because the source is not part of what you were given.

Architecture is normally read from the inside: open the tree, follow the call, see which component ends up holding the state. None of that is available here. What is available is the outside — the operations this client performs, the symbols the package exports, the shapes calls return, and the behaviour you can observe against a running deployment. Everything this part says about the plane split, about the governance objects and their relationships, and about where enforcement happens is written against that evidence and bounded by it. Where a claim about the deployment's internals outruns what an

outside read can reach — and some do, because the split between the planes is a split of responsibility and not an observable boundary — the chapter marks it as a reading rather than asserting it in the same voice. Three things in particular do not come with it:

- **Which line enforces a rule.** You can establish the layering, and this part does. You cannot open it and read the check — and a rule that is declared and never consulted looks identical from outside to one that is enforced, because both are simply operations that exist.
- **Why a boundary falls where it does.** Some boundaries are load-bearing and some are historical. The behaviour is the same either way, so no amount of observation separates them. Where a chapter offers a reason, it is a reading.
- **Anything on a path with no surface.** A capability with no operation behind it is invisible from here — not absent, just unreachable, and the two are indistinguishable in what you can measure.

So the markers matter more in this part than anywhere else in the book. Where a claim rests on observed behaviour it is stated as observation. Where it explains intent it is marked *design intent, not observable*. Where it could not be settled from what is reachable it says **UNVERIFIED**. And where the shipped behaviour is wrong it says **open defect** and describes the symptom, because a structural explanation of a system that does not match the running one is worse than no explanation: it will be believed.

A chapter in this part that uses none of those markers should be read as claiming more than an outside read can support.

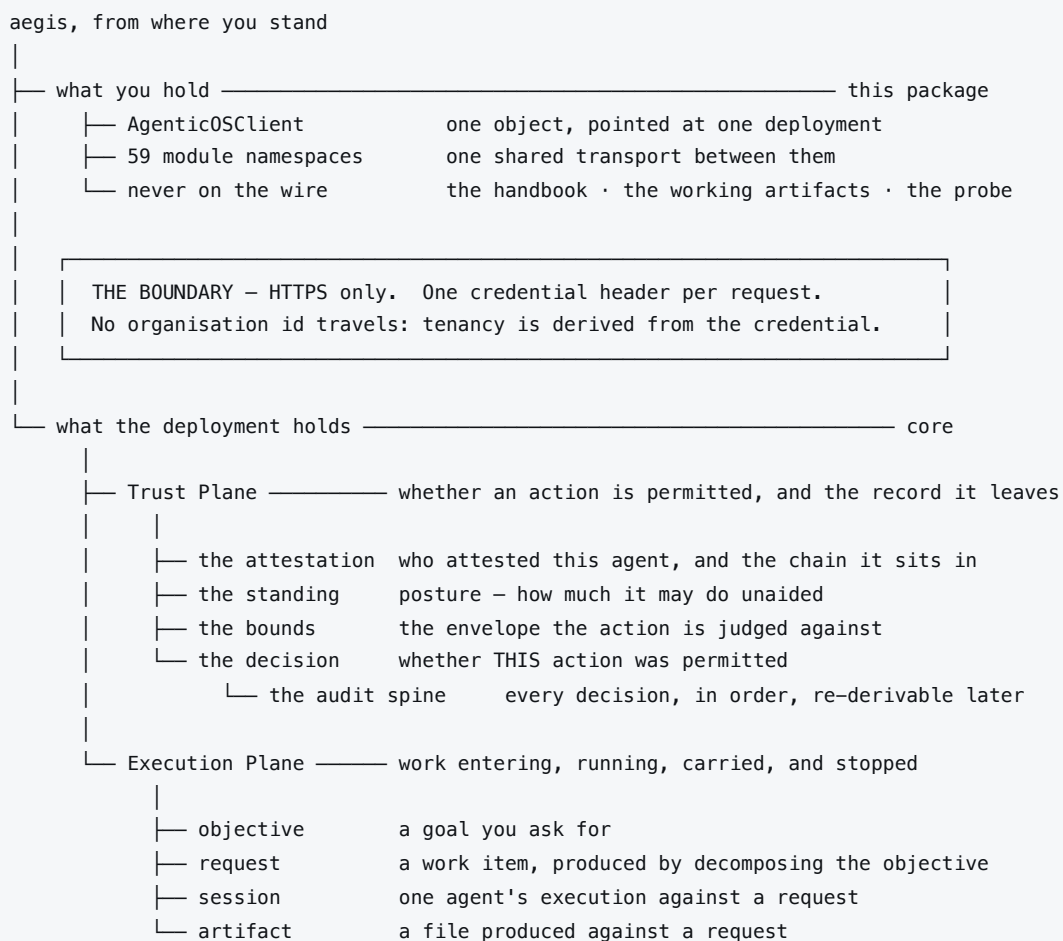
Next: 06.1 — What core and the SDK are

06.1 — What core and the SDK are

Two things arrived together and they are not the same kind of thing. One is a running system with an internal structure you are about to design against. The other is a client for it. Almost every architectural mistake made against this platform traces back to treating the second as if it were the first — most expensively by concluding that something the deployment owns is yours to enforce, or the reverse.

This chapter draws the boundary and the split behind it. Everything later in this part is a statement about one side of one of them.

The shape, in one picture



Read the middle band carefully. It is the part of the package that **never crosses**. Knowing which band a thing is in is the difference between an integration that works and one that quietly measures the wrong machine.

The plane split

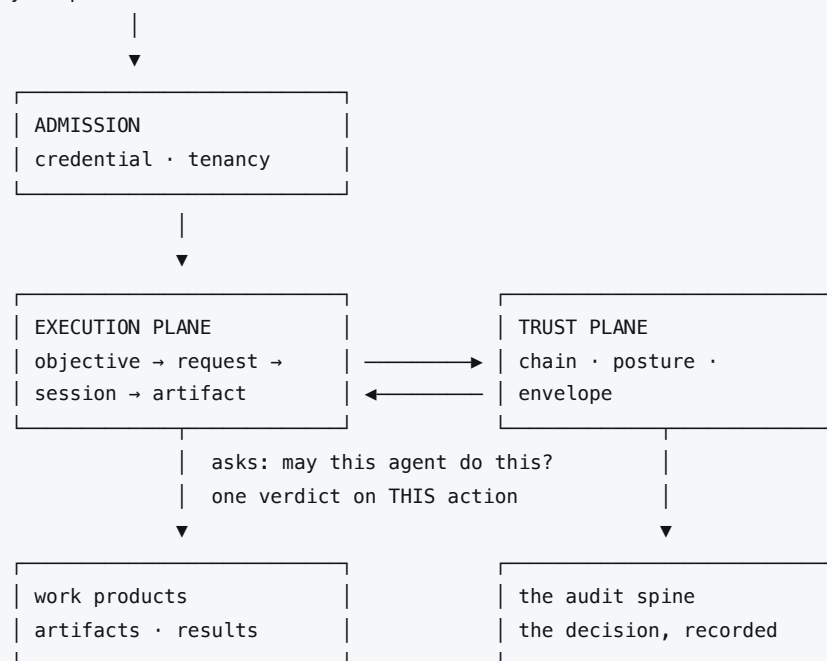
This is the single most load-bearing idea in the part, so it is worth stating plainly: **the deployment is organised into two planes with different responsibilities, and the distinction is about what each one is for, not about where either one runs.**

The Trust Plane decides. Its subject is authority: whether a given agent may take a given action, and whether that can be established at all. It owns the attestation that made the agent, the chain that shows where its authority came from, and the posture that says how much of that authority it may use unsupervised. When it decides, it judges the action against the envelope that bounds what the role's delegate may do — a bound the *structure* authors and this plane consults. Its **product is a verdict and a record**: `api:POST /api/v1/trust/verify` is the decision as a call, `api:GET /api/v1/trust/audit` is the record afterwards, and `api:GET /api/v1/trust/health` reports whether the plane itself is answering — the client's own documentation describes that one as an **unauthenticated** snapshot, which is worth knowing before you decide how to monitor a governed deployment.

The Execution Plane does the work. Its subject is the work itself: goals accepted, decomposed, executed, and turned into files. Its **product is work products** — an `api:POST /api/v1/objectives` becomes requests, requests become `sdk:aegis_sdk.execution.SessionsModule` runs, runs produce `sdk:aegis_sdk.execution.ArtifactsModule`.

The relationship between them runs one way. Execution asks; trust answers. The Trust Plane does not do work, and the Execution Plane does not decide its own authority. That is the whole of the contract, and it is what lets you reason about either half on its own:

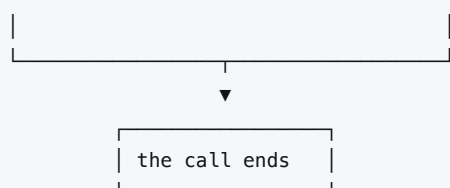
your process · the client



Both refusals end at the same place, for different reasons:

refused at the door
no plane was reached
– from ADMISSION

refused here
the run ends
– from the TRUST PLANE



Three consequences follow, and each is a design decision rather than trivia.

The two planes fail differently, and only one of them fails closed. When the Trust Plane cannot be reached, there is no verdict to give — and a governance system that permitted on *no answer* would be worthless, so the refusal is the safe reading. What you observe is a refusal. What actually happened is an outage, and it is retryable, where a real refusal is not. That distinction is 04.2's, and it is the one place this architecture can cost you a day: read a refusal on a governed action as *the apparatus could not be consulted* before you read it as *the platform decided about you*.

The two products are different evidence, and you need both. The record is the Trust Plane's — what was decided, on what basis, in what order. The work product is the Execution Plane's. A compliance story built only from artifacts is a story about outputs, with nothing to say about authority; a story built only from decisions has nothing to show for the work. Design for both from the start, because the record is not reconstructible after the fact.

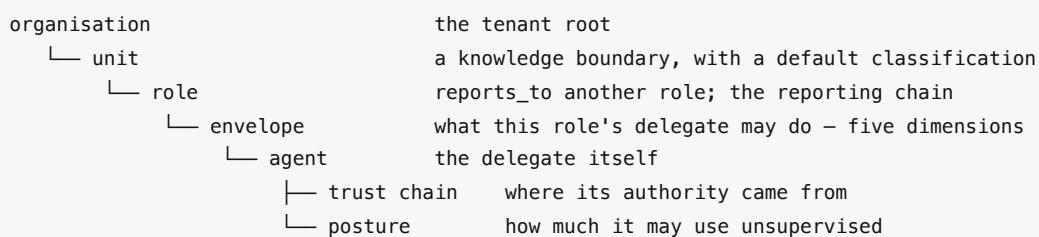
A refusal at a plane is not the same as a refusal at the door. Access control decides whether your *call* is admitted. The Trust Plane decides whether an *agent's action* is permitted. They produce overlapping statuses and have completely different remedies. 06.2 puts both on one path so you can tell which one you are looking at.

⚠ **The split is a split of responsibility. Whether it is a split of machines is not observable from here.** The two planes may be one process, two services, or two clusters; nothing you can reach distinguishes those, and nothing in your design should depend on the answer. What you *can* rely on is the direction of asking and the shape of what each one produces — and that is the part that binds you.

The Trust Plane gets a chapter of its own — 06.4 is the decision mechanism and the audit spine; the execution lifecycle is 06.5. This chapter only needs you to know which half owns what.

The governance objects, and which plane owns each

Before the planes are useful you need to know what they operate on. The objects have a containment order, and it is the deployment's, not yours to rearrange:



object	the question it answers	plane	read it at
organisation, unit, role	what the structure is	neither — it is the context both planes read	sdk:aegis_sdk.standup.0 rganizationsModule , sdk:aegis_sdk.standup.0 rganizationRolesModule

object	the question it answers	plane	read it at
envelope	what may this role's delegate do?	authored in the structure; consulted by the Trust Plane when it decides	<code>sdk:aegis_sdk.RoleEnvelopesModule</code>
agent	who is the delegate?	acted by the Execution Plane, bounded by the Trust Plane	<code>sdk:aegis_sdk.core.AgentsModule</code>
authority	who attested this agent?	Trust Plane	<code>sdk:aegis_sdk.trust.AuthoritiesModule</code>
chain	where did its authority come from?	Trust Plane	<code>sdk:aegis_sdk.trust.ChainsModule</code>
posture	how much may it do unaided, now?	Trust Plane	<code>sdk:aegis_sdk.TrustPosture</code>
decision	was <i>this</i> action permitted?	Trust Plane — and it is a record, not a setting	<code>api:GET /api/v1/trust/audit</code>
objective, request, session, artifact	what work was asked for, and what came out	Execution Plane	<code>sdk:aegis_sdk.execution.ObjectivesModule</code>

Two things about that table are worth carrying before you meet the chapters that develop it.

An agent is the one object both planes touch, from opposite sides. The Execution Plane acts as it; the Trust Plane bounds it. That is why an agent can be perfectly healthy and still unable to do the thing you asked — the two planes are answering different questions about it.

"Decision" is an object, not an event that vanishes. It is written down, addressable, and re-derivable. That is unusual enough to be worth noticing: most systems treat a permission check as a step, and this one treats it as a record you can go back and read.

What the SDK is not

Three non-identities, each of which someone assumes at least once.

It is not the deployment's source code. The package contains no server. Measured: no HTTP framework is imported anywhere in it, by any form of import — every occurrence of one is a comment *about* the shape core answers with. You were not given core, and nothing here is a copy of it.

It is not an evaluator. No plane runs in your process. Nothing here decides whether a call is allowed, resolves an envelope, evaluates a clearance, or consults a chain. When you read a verdict, you are reading a response. That is why `sdk:aegis_sdk.modules.GovernanceExplainModule` is a set of *calls* — `api:POST /api/v1/governance/explain-access` and its siblings — rather than a local reasoning step: explaining a decision is itself a request to the plane that made it.

It is not a place to keep the truth. Every read is a round trip, and a value you read is a snapshot from the moment core answered.

The client caches nothing — measured: no response cache, no memoised read anywhere in the transport or the modules. That removes one class of bug entirely: there is no local copy that can disagree with the server, and nothing to invalidate. **But it does not make a read fresh**, because the deployment caches, and a cached answer is still an answer.

Some surfaces are explicit about it, and they are the ones to model on.

`sdk:aegis_sdk.modules.LlmProvidersModule` reads a provider's model list from `api:GET /api/v1/llm/models` and the result carries `from_cache`, `cached_at` and a `fallback` flag; a separate call, `api:POST /api/v1/llm/providers/{}/refresh`, forces the recalculation. Where a surface offers you that shape, read the flag rather than assuming freshness — and where it does not, treat the read as a snapshot with no guarantee about its age, because you have nothing to check.

 **The client's route table is a belief, not a specification.** Every `api:` anchor in this book resolves against *the calls this package makes*. If the client and the deployment disagree about a path, the client is wrong in the same direction and nothing here notices. The check that ships with this book verifies existence, never enforcement — an operation can exist and check nothing at all. Treat "this route is declared" and "this route does a thing" as the two separate claims they are.

What you reach, and how

The client presents **59 namespaces carrying 828 public members between them**. The number is measured, and it is large enough that the useful question is not what exists but which plane a namespace serves.

family	reaches	plane	examples
trust	whether an action is permitted, and the record	Trust Plane	<code>sdk:aegis_sdk.trust.ChainsModule</code> , <code>sdk:aegis_sdk.trust.AgentTrustModule</code>
structure	the organisation the planes read	neither	<code>sdk:aegis_sdk.standup.OrganizationRolesModule</code> , <code>sdk:aegis_sdk.RoleEnvelopesModule</code>
work	what runs, and what it produced	Execution Plane	<code>sdk:aegis_sdk.execution.ObjectivesModule</code> , <code>sdk:aegis_sdk.execution.SessionsModule</code>
lineage	what produced what, and from what	neither — it observes both	<code>sdk:aegis_sdk.dataflow.DataFlowModules</code>
commercial	subscription, quota, license	neither — it bounds the tenant	<code>sdk:aegis_sdk.revenue.SubscriptionsModule</code>

Four details of the surface's shape will save you time.

Some namespaces are grouped and some are flat. `trust`, `revenue` and `dataflow` are containers: `client.trust.chains`, not `client.chains`. The rest are direct: `client.agents`, `client.objectives`. Nothing distinguishes the two by name.

The name is not the surface. `client.roles` and `client.role_admin` are different APIs over different routes with different return shapes — one returns untyped dictionaries, the other typed models. The client's own comments say so. When two similarly-named namespaces exist, read both before assuming one is an alias.

Some surfaces require a kind of credential you may not hold. Several modules — `sdk:aegis_sdk.modules.CredentialsModule`, `sdk:aegis_sdk.modules.EmergencyBypassModule`, `sdk:aegis_sdk.modules.KillSwitchModule` and `sdk:aegis_sdk.modules.PromotionsModule` — refuse an API key on *every* route, reads included. That is not a scope problem and no scope fixes it.

`client.trust.observability` is the plane looking at itself. It aggregates chain and verification metrics and returns a health snapshot of the Trust Plane — `sdk:aegis_sdk.trust.TrustObservabilityModule`. If you are deciding whether a refusal was a decision or an outage, that is the surface that separates them, and 06.4 is where it is developed.

The part of the package that never crosses

The middle band of the first diagram is easy to skip past, and skipping it is how you lose a day. Beside the client the package ships:

- **this handbook**, which explains behaviour;
- **the working artifacts** — agent briefs, skills, guardrails, and a set of enforcing hooks that run in your own CLI session rather than against core;
- **a probe**, `python -m aegis_sdk.coc.probe`, a runnable reachability check;
- and **a local orchestration subpackage**, `sdk:aegis_sdk.nexus`.

That last one is the one to be careful with, and it is worth stating exactly.

🚫 `sdk:aegis_sdk.nexus` does not touch your deployment — and its most convincing-looking verb returns success without doing anything. Measured: the subpackage imports no HTTP client, no socket, no subprocess, and no container or cluster tooling anywhere.

`sdk:aegis_sdk.nexus.DeploymentManager` 's `deploy()` validates a configuration object, then returns a `sdk:aegis_sdk.nexus.DeploymentResult` with `success=True` and a set of endpoints synthesised as `http://localhost:<port>` from the manifest. It deploys nothing. Its own source says the real implementation would talk to Kubernetes or Docker; the shipped one does not.

Nothing is broken relative to what that code claims internally. The hazard is entirely in the reading: a method called `deploy()`, returning a result object, next to configuration models named for production tiers, invites the conclusion that you have a deployment tool. You have a **configuration model**. Use it to *describe* a shape — and do not build a workflow on the belief that calling it changed anything anywhere.

UNVERIFIED: whether this subpackage is intended for partners at all, or is an internal library that happened to travel in the wheel. Nothing in the package says, and the question is not answerable from what you were given.

The handbook, the working artifacts and the probe are in the same band: they are things you *read and run locally*. None of them is a client for core, and none of them makes a governance decision. A hook running in your session can refuse what *your tooling* does — that is a real control and it is worth wiring — but it is not the deployment's enforcement, and it does not bind an agent that is already running somewhere else.

Where the standards sit

Core implements four open specifications published by the Terrene Foundation under CC BY 4.0. Core implements them; it does not own them, and this package is a client for the implementation rather than a home for the specifications.

The vocabulary is not decoration — it is how you predict names, and the plane split itself is one of the four standards' central claims.

specification	what it contributes	where it shows up
CARE	the dual plane — Trust Plane and Execution Plane — with the posture ladder and the four-zone verdict	the split itself; <code>sdk:aegis_sdk.TrustPosture</code>
PACT	the organisation model: units, roles, D/T/R addressing, envelopes, clearance	<code>sdk:aegis_sdk.standup.OrganizationalRolesModule</code> , <code>sdk:aegis_sdk.RoleEnvelopesModule</code>
EATP	the trust protocol: attestation, chains, delegation, revocation, the trust verdicts	<code>sdk:aegis_sdk.TrustChain</code> , <code>api:POST</code> <code>/api/v1/trust/establish</code>

specification	what it contributes	where it shows up
CO	the method — <i>how</i> to work with the system, including this book	the working artifacts beside this handbook

Read the third column as naming where the vocabulary is *expressed*, never where it is enforced. A module named for a concept means core implements that concept and this client can reach it. Enforcement is the deployment's — the same fact as everything else on this page, arrived at from a different direction.

What you can reach, in one paragraph

You can call any operation this client declares, with a credential core accepts, and you will get back one of three things: a parsed payload, raw bytes, or a typed error. You can read the state core holds and change it to the extent the governance on it permits; you can read a decision the Trust Plane made, and ask it to explain one. You cannot evaluate a governance decision locally, boot a copy of core, read core's source, or reach anything the API does not expose. And you cannot, from this position, see *which* component refused you — only that something did, and with what status. Where the two halves disagree, core is right: the client is a belief about it, and this book is a belief about the client, both re-checkable against the build in your hands.

Next: 06.2 — The request path

06.2 — The request path

06.1 drew the boundary and named the two planes. This chapter walks a call across it — every layer it passes through, the point at which each kind of decision is actually taken, and what happens to the request when one of them says no.

The point of the walk is prediction. A refusal does not tell you which layer produced it, and the layers have different remedies: one is answered by a different credential, one by a different permission, one by a different design, and one is not about you at all. Knowing the order is what lets you tell them apart from the outside.

The layers, in order

```
one call, and the layers it passes through
|
|— in your process —————
|   1 the module      builds the path, percent-encoding every identifier
|   2 the transport   one credential header · SDK + API version · Content-Type
|                     retries transport failures only – never an HTTP status
|
|— THE BOUNDARY · everything below is the deployment's —————
|
|— in the deployment
|   3 ADMISSION ——— ENFORCED, before any route is consulted
|                     the credential is resolved, the organisation is resolved
|                     router-level gates run here – so a refusal here is a
|                     statement about the D00R, not about the route you called
|                     → 401 (no usable credential) · 400 (no organisation context)
|                     → 403 (refused at the door)
|
|   4 AUTHORIZATION — ENFORCED, per route
|                     the permission the route names, against the caller's role
|                     attribute rules may also deny, and a deny beats a role allow
|                     → 403
|
|   5 THE TRUST PLANE — ENFORCED, for an AGENT's action – not for your API call
|                       chain · posture · envelope → one verdict on THIS action
|                       → 403 / 423 / 451 · and the decision is written down
|
|   6 THE HANDLER ——— the work happens
|                       or the request is refused as malformed
|                       → 400 / 422
|
|   7 THE RESPONSE ——— back across the boundary
|                       JSON · raw bytes · none (204) · or a typed exception
```

Eight claims are compressed into that picture, and each of the rest of this chapter's sections is one of them, derived.

⚠ **Where that ordering comes from, stated so you know what you are relying on.** Layers 1 and 2 are this package's own behaviour, and every claim about them here is measured against the build in your hands. Layers 3 to 5 are on the deployment's side, and **no SDK can measure the order in which a server runs its own checks**. That half of the picture is derived from two things: what the client's own documentation reports about admission and authorization, and the observable *consequences* — which status a refusal carries, which fields the error envelope does or does not populate, and what a caller receives when the same call is retried with the other credential type. The layer order is the reading that makes those consequences consistent, and it matches what 04.1 and 04.2 describe. Treat it as a well-supported model, not as something you can open and read.

🛑 **Layers 3 and 4 decide about *your call*. Layer 5 decides about an *agent's action*.** They are the two enforcement points on this path and they are easy to conflate because they produce overlapping statuses. The sentence to keep: **admission and authorization are access control on the caller; the Trust Plane is governance on the governed**. A 403 can come from either, and the request body will not say which — which is why the rest of this chapter teaches the shape of the evidence rather than the shape of the response.

Construction: the client will not guess where core is

`sdk:aegis_sdk.Agentic0SClient` requires a base URL. There is no default host, and the omission is deliberate enough that it raises `sdk:aegis_sdk.ConfigurationError` before anything else happens.

Construction comes down to two decisions — **which deployment** you are pointing at, and **which credential** you present — and you supply them in one of three ways: directly, as a whole `sdk:aegis_sdk.ClientConfig`, or by letting `Agentic0SClient.from_env()` read them from the environment. The third is the one with architectural weight, because it is the only path where a *deployment* configures the client rather than a developer typing it into a script — which is what makes the same code promotable between environments. The mechanics, the credential vocabulary and the provisioning patterns belong to 04.3; what belongs here is what the choice decides.

The supported environment names are `AEGIS_`-prefixed; the older `AGENTIC_0S_` names are still read and emit a one-time deprecation warning, and when both are set the `AEGIS_` value wins.

The configuration surface is small, and it is the whole of what you control about the connection:

setting	default	note
<code>timeout</code>	<code>30.0</code>	applied to connect, write, read and pool waits alike
<code>max_retries</code>	<code>3</code>	attempts, not retries-after-the-first
<code>retry_backoff</code>	<code>1.5</code>	base of an exponent, not seconds
<code>verify_ssl</code>	<code>True</code>	leave it
<code>debug</code>	<code>False</code>	logs requests, with bodies scrubbed

`sdk:aegis_sdk.ClientConfig.with_base_url` and `ClientConfig.with_api_key` build a variant without repeating the rest — useful when one job needs a longer timeout than everything else.

🚫 `max_retries=0` does not mean "do not retry". It means "do not make the request." The retry loop is the send loop, so zero iterations send nothing and every call raises `sdk:aegis_sdk.ServiceError` with the message `Unknown error after retries` — including calls that would have returned `200`. Use `1` for a single attempt. The constructor happens to substitute the default for a literal `0`, so this bites through the environment variable and through the config object, which are the two paths a deployment configures rather than the one a developer types.

Layer 1 — the module builds a request

A module namespace is stateless. It holds the transport and a set of path templates, and that is all: `sdk:aegis_sdk.core.AgentsModule` has no agents in it.

Every interpolated identifier is percent-encoded **at the call site**, before it is spliced into the path, so a value containing `/` or `..` cannot shift which route you address. Measured through the transport:

```
get("abc-123")    -> /api/v1/agents/abc-123
get("a/b")        -> /api/v1/agents/a%2Fb
get("../..../admin") -> /api/v1/agents/..%2F..%2Fadmin
```

The last line is the point. An id is a *segment*, and a slash inside one never becomes a route separator.

⚠️ **Encoding is applied to the raw value, so an already-escaped id gets double-escaped.** `%2F` in an id becomes `%252F` on the wire, because the `%` itself is encoded. If you are storing ids in escaped form, unescape before you pass them. Nothing errors — you simply address a different resource, or get a `404` that looks like a missing record.

Not every call site in the package is a request. Some methods raise `sdk:aegis_sdk.UnsupportedOperationError` unconditionally, because the operation they target does not exist. Those never reach the wire, so **no layer above is consulted** and retrying is meaningless. This is an **open defect** enumerated in 04.2, and it is the purest case of the rule that gives this whole chapter its value: **when a call fails, establish that a request was made before you investigate the deployment at all.**

Layer 2 — the transport types and signs it

This is the layer that touches the network, and it is shared by every module — which is why the behaviour below is uniform no matter whose method you called.

Measured on the wire, a `GET` from a client holding an API key carries:

header	value
X-API-Key	the key, verbatim
X-SDK-Version	the package version
X-API-Version	v1
User-Agent	AgenticOS-SDK/<version>
Accept	application/json
Content-Type	application/json — set per request, including on bodyless GET s

The credential is routed by its shape, not by what you intended. A value beginning `sk_live_` goes into `X-API-Key` ; anything else goes into `Authorization: Bearer` . Both are accepted by core, and the client picks the canonical channel for each kind. Measured both ways.

🚫 **There is exactly one credential on the wire at a time, and the client is careful about it.**

`set_api_key` rewrites both headers on every call: the one being set is written, and the other is *removed*. That matters because the same slot holds either an API key or a session JWT, and a swap that left the old header in place would present two identities and let the deployment authenticate as whichever it checked first. Measured: a key swap retires `Authorization` ; a token swap retires `X-API-Key` .

No organisation identifier is sent. Measured: the header set above is complete — there is no tenant or org header on any request. This is the transport half of layer 3, and it is the architectural reason a body field naming an organisation is at best ignored: **the caller does not get to say who it is**. Tenancy is derived from the credential and resolved on the deployment. To see which organisation your credential actually resolves to, ask: `api:GET /api/v1/auth/me` .

Content-Type is applied per request, not once on the connection, and precedence runs caller-supplied first, then whatever type the body's own encoder computes, then JSON. That ordering is the fix for an upload failure in which a client-wide JSON type overrode the multipart boundary a file upload needs — the deployment could not parse the form and answered `422` on every upload call while the module tests, which stubbed the transport, stayed green. The rule to carry: **if you send a body the client does not type for you, and you care how it is labelled, set the header yourself**.

Layer 2, continued — the loop that sends it

The transport sends, and on failure decides whether to send again — **and it decides by verb first**, because a retried write is a second write. Measured at the default `max_retries` of 3, for an idempotent verb (`GET` , `HEAD` , `OPTIONS` , `PUT` , `DELETE`):

what failed	attempts	added delay
a connect, read or pool timeout	3	2.5 s
a network or request error	3	2.5 s

what failed	attempts	added delay
any HTTP status — 500 , 429 , 403 alike	1	none

`POST` and `PATCH` are **not** retried on any of those failures — they are sent once, and the timeout or connection error is raised. A `POST /organization-units` that times out *after* the server created the unit is indistinguishable, at this layer, from one that never arrived, and units cannot be renamed or merged, so the retry that looked harmless was the expensive direction. The verb is the only discriminator available here: there is no idempotency key to read, so a verb that is not idempotent gets no second attempt.

The backoff is `retry_backoff ** attempt`, so roughly one second then one and a half, and it is not configurable from the environment. Only `max_retries` is.

No HTTP status is retried. The loop covers the case where the deployment *did not answer*; it does not cover the case where it answered `503`. That distinction is load-bearing at this point on the path: **a retry is only sound where nothing happened, and this client can only tell that a reply is missing — never that a request was not applied.** If your integration needs status-level retry, you write it, and 04.5 has the measured arithmetic and the statuses worth retrying.

Design intent, not observable: the reason the client declines to retry statuses is a reading, not a measurement — the transport sends no idempotency header, so a retried write is simply a second write. Treat the reading as the book's explanation of a behaviour you can measure directly in the table above.

Layer 3 — admission, and why a refusal here lies to you

This is the first layer on the deployment's side, and it runs **before your route is consulted**. What happens here is coarse: the credential is resolved, the organisation is resolved, and router-level gates decide whether you are admitted at all.

The consequences are the ones that cost people days, and they are worth stating as a list because they all present as "I do not have permission":

- **A refusal at admission is not about the route you called.** The route's own permission check was never reached, so what the route documents is irrelevant to your refusal.
- **A read can be refused exactly as a write would be.** Where a router-level gate is what refused you, nothing route-specific was consulted, so the read/write distinction never entered the decision. This is one of the platform behaviours 04.2 records as an open defect, and it is scoped: it applies to the surfaces that carry such a gate, not to every route.
- **The message may name something to obtain, and obtaining it changes nothing.** The identity gate runs before the permission check on at least one surface, so grants that would have satisfied the later check sit inert.

The door/desk distinction is 04.1's, including the one-minute test that settles which you have. What this chapter adds is *where* on the path it happens: at layer 3, before anything you can see in the route's own documentation.

The status here is not what you expect, and that is the trap. A caller with no organisation context is refused with a `400`, not a `401` or `403`. A caller reading only the status code classifies that as a malformed request and goes looking at their body. It is an identity problem.

Layer 4 — authorization, the per-route decision

If admission lets you through, the route's own requirement is checked against what your principal holds. This is where the two registries of 04.1 become load-bearing — roles for a session, scopes for a key — and where a scope check turns out to be a no-op for a session caller, because it is an API-key mechanism.

Two properties decide what you can predict from outside:

There is more than one guard, and they do not agree by construction. One consults roles only; the other consults roles *and* attribute rules. Which one guards a given route is not visible in any response. So a **permission granted to your role and still refused is not necessarily a misconfigured role** — an attribute rule may be denying you, and a deny overrides a role allow, so adding the role again will not clear it.

Two routes requiring what looks like the same permission can decide differently for the same caller, because one consults the attribute rules and one does not. That is a real distinction, not a flake, and retrying is not a remedy for it.

Layer 5 — the Trust Plane, where a governed action is decided

This is the enforcement point the rest of this book exists to describe, and it is the one most often mistaken for access control. The distinction is the subject, not the mechanism:

	layers 3 and 4	layer 5
decides about	the caller — you, holding a credential	an agent's action — the delegate, mid-run
the question	may this principal call this route?	may <i>this agent</i> do <i>this thing</i> , given its authority?
what it consults	roles, scopes, attribute rules	the chain, the posture, the envelope
the artifact	—	a decision, written down

The Trust Plane's answer is not a boolean the platform computes and discards. It resolves into CARE's four-zone verification gradient — four values and exactly four, delivered as the verdict's `level`:

<code>auto_approved</code>	proceed, and record it
<code>flagged</code>	proceed, and surface it for attention
<code>held</code>	stop, and wait for a human
<code>blocked</code>	refuse

Three things about that output govern what you can build on it.

held is not a soft **blocked**. Under an enforced-execution path a held action **ends the run** rather than queueing politely. Plan for a held action to terminate the work, not to pause it — the mental model has the incident where that mattered, and 06.5 is where you design for it.

Thoroughness can upgrade a verdict. A stricter setting promotes `auto_approved` to `flagged`, so raising it produces more *signal*, not merely more refusals. If your flagged count rises after you tighten something, that is the feature.

The verdict is reachable as a call, and as a record. `api:POST /api/v1/trust/verify` is the decision on demand; `api:GET /api/v1/trust/audit` is what it left behind. That second one is unusual and worth designing around: most systems treat a permission check as a step, and this one treats it as an object you can go back and read.

Where the refusal statuses come from

There is no status that uniquely means "the Trust Plane refused you", and **403** is not the authorization-only status it looks like. Three statuses map to three distinct exceptions, and the first of them is shared:

status	exception	what it means
403	<code>sdk:aegis_sdk.AuthorizationError</code>	an authorization decision at layers 3 or 4 — or a governance verdict that arrived with no structured half
423	<code>sdk:aegis_sdk.GovernanceViolationError</code>	a governance refusal: a policy, a budget, an approval requirement
451	<code>sdk:aegis_sdk.TrustViolationError</code>	a trust refusal: the delegated authority for this action does not hold

⚠ **That first row is the sharpest trap on this page, so read it twice.** A refusal on a governed action can arrive as a bare `403` carrying `error_code: "FORBIDDEN"` and nothing else — byte-for-byte the shape of an authorization refusal from layer 3 or 4 — and **nothing in the response says which one you have**. So collapsing governance into "denied" is worse than lossy: on those paths there is nothing to collapse, and the two are answered by different changes made by different people.

🚫 **And the two causes *inside* that `403` are also indistinguishable.** A refusal caused by the governance apparatus being unreachable and one caused by the agent's own declared restriction arrive identically — same status, same code, both with an empty structured half — and the only thing that differs is the wording of a message you should not parse. This is the fail-closed property from 06.1 wearing a refusal's clothes: the platform could not evaluate, so it declined. **Retrying is the correct response to the first and the wrong response to the second.** Until the structured discriminator reaches you, the honest handling is to treat a refusal on a governed action as a refusal, and quote the request id to whoever operates the deployment when you need to know which kind it was. Full treatment in 04.2.

Layer 6 — the handler, and what a refusal leaves behind

If nothing refused you, the work happens. This is where a malformed body is caught (`400` , `422`) — after admission and authorization, which is why a `422` tells you your *request* was wrong and says nothing about your standing.

What happens to the refused request is: nothing. There is no partial application to compensate for and no queue it sits in. The layers above run before the handler, so a refusal at any of them means the handler was never entered, and the request is gone. The response you get back is the canonical error envelope, and the two fields that matter are the machine-readable code and the request id — the correlation handle that lets somebody who was not there find the same event on their side.

That is a stronger guarantee than it sounds, and it is worth naming as one. The common alternative is a request that half-applied and returned an error, which is where compensation logic gets invented. Here, a refusal is clean: either the handler ran or it did not.

Design intent, not observable: that ordering is a design choice, not something you can measure from the response. What you can measure is its consequence — a refused call leaves no work product — and that is the part your design may rely on.

Layer 7 — the response, and its three success shapes

A `2xx` does not always give you back a dictionary. Measured:

status	body	you receive
<code>200</code> <code>201</code> <code>202</code>	JSON	the parsed value — a <code>dict</code> or a <code>list</code>
<code>200</code> <code>201</code> <code>202</code>	not JSON (an HTML error page, a PDF, a CSV)	<code>bytes</code> , the raw body
<code>202</code> with an empty body	—	<code>b""</code> , not <code>None</code>
<code>204</code>	—	<code>None</code>

The bytes row is the one that catches people. A gateway or proxy answering `200` with an HTML page gives you `bytes` ; so does an endpoint that emits a file, and so does a success whose body is empty. The client decides by *whether the body parses*, never by the `Content-Type` header — a JSON body served under a wrong type still parses, and the client will not change behaviour on a path that currently works.

If you get `bytes` where you expected a mapping, you are looking at an unparsed body, and the next move is to check your base URL for a redirect before you check anything else. That trap, and the one-minute test that identifies it, are 04.2's.

How the error becomes a typed exception

Every error the deployment returns is re-wrapped into one envelope before it leaves:

```

{"error": {"code": "FORBIDDEN",
          "message": "Constraint violation: ...",
          "details": {...},
          "request_id": "..."}}

```

The transport recognises an `error` key whose value is an object as that envelope and reads the fields one level in. Anything else — including an `error` key holding a plain string, which some servers use as the message — takes the flat `{"detail": ...}` path unchanged, so a gateway or a non-Aegis server answering in a different shape still parses as it always did.

Then it maps the status to a class and projects the envelope onto it. Measured on a `403` carrying the envelope above:

attribute	value
the exception's class	<code>sdk:aegis_sdk.AuthorizationError</code>
<code>exc.message</code>	the envelope's message — a string, not a dict
<code>exc.error_code</code>	<code>"FORBIDDEN"</code>
<code>exc.server_details</code>	<code>{"unverifiable_scope": "apparatus"}</code> — the structured fields
<code>exc.request_id</code>	the server's correlation id
<code>exc.status_code</code>	<code>403</code>

Branch on those attributes; never on the message text. `sdk:aegis_sdk.Agentic0SError` is the base every one of them inherits from, and its own documentation is explicit that a **missing** key in `server_details` means the server sent nothing under that name — never that the underlying condition is absent.

The transport-level failures are typed too, so `except Agentic0SError` covers them: `sdk:aegis_sdk.TimeoutError` and `sdk:aegis_sdk.ConnectionError` for the two ways the deployment can fail to answer, and `sdk:aegis_sdk.RateLimitError` for a `429`, which carries a `retry_after` read from the response header. **The client does not act on that number** — it is there for you.

The typed model, and the exception that is not one

Most methods end by constructing a model from the parsed body. That construction is strict, and it is the last thing that can fail — in a way the taxonomy above does not cover.

Measured: driving `client.agents.get()` with a body missing required fields, or carrying an unexpected enum value, raises `pydantic.ValidationError` — which is a `ValueError`, and is **not** an `sdk:aegis_sdk.Agentic0SError`. An `except Agentic0SError` block lets it straight through.

That is worth internalising rather than working around. It means a wire-shape change on core surfaces to you as a validation error naming the fields that moved, and it means the typed models are a real contract rather than a decorative annotation. `sdk:aegis_sdk.Agent` and `sdk:aegis_sdk.User` are

both strict about required fields and about enumerated values — `unit_type` accepts exactly two strings, `status` an exact set.

△ **The models you send are strict in the other direction too: they carry fields you did not set.**

Measured: a create call given four arguments put **eight** fields in the request body, filling in `unit_type`, `agent_subtype`, `a2a_enabled` and `is_shadow_agent` from the client's own defaults. A body you assume equals the arguments you passed is not the body that goes out. When a create behaves unexpectedly, read the request body — the debug log prints it, scrubbed.

The envelope the deployment sends, and who unwraps it

Many endpoints wrap their payload as `{"data": ...}`, sometimes beside a pagination sibling like `{"total": 12}`. A client that reads fields straight off the outer object finds none of them, and the failure is quiet: a list reads as empty and a single object reads as missing.

The transport never unwraps. Doing it there would change the return shape of every method at once, including the many that are correct today and the endpoints that answer with a bare list or with a payload that legitimately owns a `data` field of its own.

Unwrapping is therefore opt-in per call site, and it is guarded: a response is treated as an envelope only when its `data` key is accompanied by nothing outside a known set of pagination siblings. Measured — `{"data": [...], "total": 12}` unwraps; `{"version": 1, "data": ...}` does not, because `data` there is one field of the answer rather than a wrapper around it.

Measured across the whole package: **exactly one module opts in** —

`sdk:aegis_sdk.core.PipelinesModule`. Everything else hands you the deployment's body as it arrived.

That single fact explains a class of confusion worth naming: **two methods on different modules can return the same conceptual noun in different shapes**, one unwrapped and one not. Before you subscript a result, know which module it came from. The shape hazard itself — and the habit of checking the type once at any call site you are unsure about — belongs to the [orientation chapter](#); what a *field* of a returned model does and does not guarantee is 04.6's.

Authentication is a request like any other

There is no bypass path. `api:POST /api/v1/auth/login` returns a nested envelope — a `user` object and a `tokens` object — and the module flattens `tokens` onto the returned `sdk:aegis_sdk.AuthToken` while attaching the parsed `sdk:aegis_sdk.User` beside it, so you do not need a second round trip. If the body is not that shape, the module raises `sdk:aegis_sdk.Agentic0SError` naming the keys it expected and the keys it received, rather than a bare `KeyError`.

🚫 **Logging in does not authenticate the client. The install step is yours.**

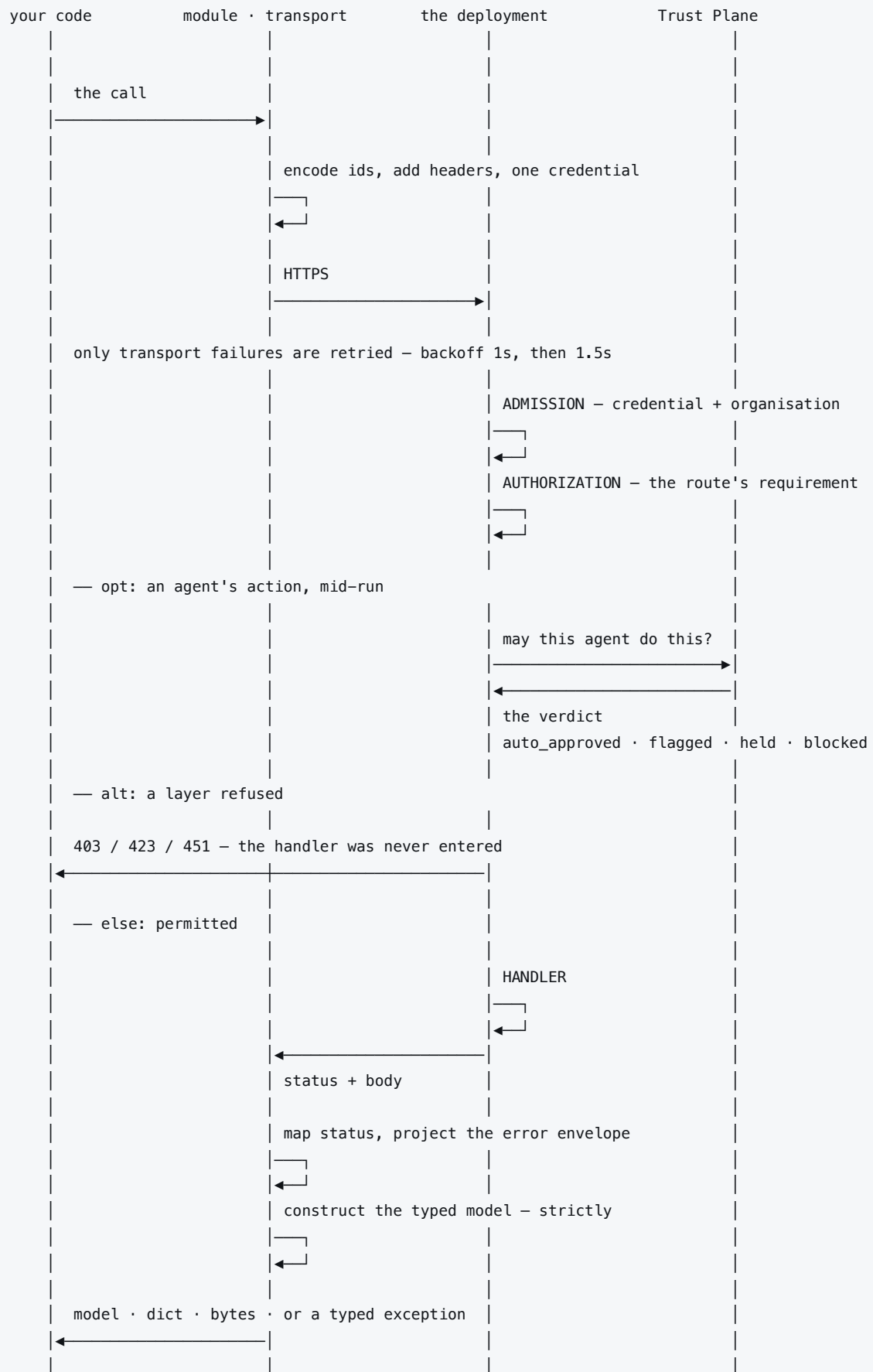
`client.auth.login()` returns a token; it does not adopt it. The client's configuration already holds either an API key or a session JWT, never both, and it changes only when you tell it to. Measured: after a `login()` on a client constructed with an API key, the next request still carried the key. Skipping the install step is the difference between a session and a key — presented as whichever you had before, and resolved at layer 3. The call itself, and the two ways to install what it returns, are 04.3's.

Refresh follows the same rule, with one more consequence. `api:POST /api/v1/auth/refresh` returns a new token pair, and the refresh token is single-use: rotation retires **both** halves of the pair it replaces, so the previous access token is revoked immediately rather than at its expiry. Of two concurrent refreshes presenting the same refresh token, exactly one succeeds and the other raises `sdk:aegis_sdk.AuthenticationError`.

The client never refreshes for you. Nothing watches for a `401` and retries with a fresh token. A long-running script that outlives its access token will start failing on a clock, and the failure will arrive as `sdk:aegis_sdk.AuthenticationError` on calls that were working a minute ago.

The path again, as a sequence

The same seven layers, from your line of code to your exception — this form carries the fan-out and the returns that the tree at the top of the chapter flattens:



What the path does not do for you

Stated plainly, because each of these is something integrators assume is handled. None of it is a defect; all of it is yours to build:

- **No enforcement.** Nothing on this path decides anything in your process. The only refusal your own code produces is the strict model construction, and that is a shape check, not a governance one.
- **No status retry.** See layer 2. `RateLimitError` even carries the number to wait, and nothing reads it.
- **No token refresh.** See above.
- **No idempotency key.** A retried write would be a second write, so the transport does not retry one — `POST` and `PATCH` are sent once. What its retry still covers is the case where the deployment may not have received the first request, and only for an idempotent verb, where a second send cannot double-apply.
- **No rate limiting.** Nothing throttles you below the connection pool's ceiling.
- **No ordering guarantee.** `asyncio.gather` returns results in argument order and says nothing about the order the deployment saw them in.

Everything above is checkable from the build in your hands. Everything about *what the deployment checks* — whether a permission is consulted, whether an attribute rule denies you, whether the Trust Plane was reachable when it answered — is not, and this book marks it where it appears rather than letting you assume it.

Next: 06.3 — The governed organisation

06.3 — The governed organisation

This chapter is the object model. Not how to create things — 02.2 and 02.3 do that — but what the things *are*, how they nest, and which of them your own concepts map onto.

The organising idea is worth stating plainly, because it changes where you look for a problem: **the organisation chart is not metadata about the system, it is the addressing scheme of the system**. An envelope resolves through it, a clearance attaches at a node in it, a trust chain follows one edge of it, and a knowledge item is owned by a node in it. When a governance decision comes out differently from what you expected, the answer is usually "some ancestor contributes a bound you did not account for", and the reason you can compute that answer at all is that the structure is the input.

If you are designing against this, the question to hold onto is which of these objects carries authority and which is only context. Most of them are context. A small number of them decide whether an action happens, and those are the ones whose lifecycle you have to get right — because a context object that is wrong produces a confusing org chart, while an authority object that is wrong produces a bound that reads as a bound and stops nothing.

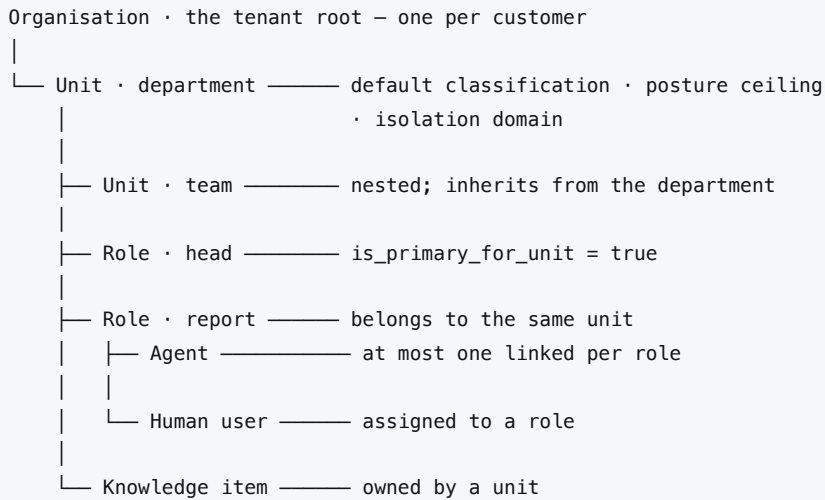
role in the system	objects	what follows
the context both planes read	organisation, unit, role, team	wrong values mislead; nothing is enforced by them directly
the authority objects	authority, trust chain, posture, envelope, clearance	their values change a verdict — this is where enforcement attaches
the acted-upon objects	agent, knowledge item, position	bounded by the authority objects; they do not bound themselves

The middle row is the one to study, and note where those five attach: the **envelope and clearance hang off a role**, and the **chain and posture hang off an agent**, while an authority is the signing root above both. A role is therefore context *and* the anchor the bounds are bolted to — the seat survives whoever occupies it, which is the property that makes a re-org survivable. That division is set out precisely in 06.4.

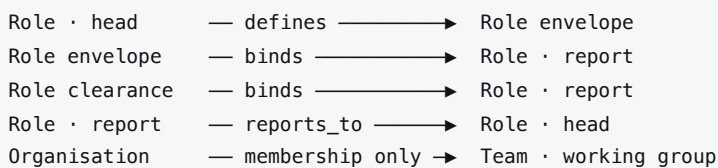
06.1 assigns each object to a plane and 06.5 shows what the execution side does with them. This chapter stays on the shared substrate both planes sit on, and on the authority objects in it, because that is the layer you cannot change by writing code — only by describing it correctly.

The containment tree

CONTAINMENT – solid: delete the parent and the child has nowhere to live.



BINDING – dotted: an envelope does not CONTAIN a role, it CONSTRAINS one. Dotted edges over the same objects as the tree above.



The two nodes those edges reach:

Role envelope — defined BY a supervising role, FOR a target role
Role clearance — what the role may see

Team is deliberately NOT in the containment tree above. It has no parent-child edge, only the membership edge listed here – nothing lives inside a team, and deleting one takes no object with it.

The solid edges are containment: delete the parent and the child has nowhere to live. The dotted edges are bindings: an envelope does not *contain* a role, it constrains one.

Organisation — the tenant root

Everything else in this chapter is scoped to one. `api:POST /api/v1/organizations` creates it; `api:GET /api/v1/organizations/{id}` reads it back, and `api:GET /api/v1/auth/me/organizations` lists the ones your principal can see — which is how you discover you are holding credentials for more than one.

The tenant boundary is enforced by *answering as not-found*, not by refusing. A unit, role, chain, or agent belonging to another organisation comes back 404, never 403.

`sdk:aegis_sdk.OrganizationsModule` is the thin typed write path;

`sdk:aegis_sdk.OrgStandupModule` is where the list reads live, which is the split chapter 02.2 warns about.

Plan for one organisation per customer, and treat a second one as a migration you have not decided to make yet. The scoping is total, so anything you build across two organisations is something you are maintaining outside the platform's own model.

Units — the boundary that carries a classification

A unit is where the structure acquires *semantics*: it is simultaneously an organisational container, a knowledge boundary, and the carrier of a default data classification. `api:POST /api/v1/organization-units`, and `sdk:aegis_sdk.OrganizationUnitsModule` for the typed create/get pair.

Three fields on a unit do most of the work, and all three cascade down the tree:

field	what it does
<code>default_classification</code>	the level knowledge created in this unit inherits unless overridden
<code>max_trust_posture</code>	the posture ceiling for agents running as roles in this unit
<code>isolation_domain</code>	the tenant-defined hard-isolation plane the unit sits in

Read `effective_max_trust_posture`, never `max_trust_posture`. `api:GET /api/v1/organization-units/{id}/posture-ceiling` returns both, and the declared value is only half the picture: a ceiling cascades, so a child can never exceed its parent *whatever its own value reads*. Setting a permissive ceiling on a leaf under a restrictive branch changes nothing, and nothing tells you that. The setter, `api:PUT /api/v1/organization-units/{id}/posture-ceiling`, has a genuinely dangerous asymmetry worth carrying into your code: **omitting the argument leaves the ceiling unchanged, while passing an explicit null clears it** — and clearing a ceiling widens what the unit may do. Those two are one keyword apart at the call site and opposite in effect.

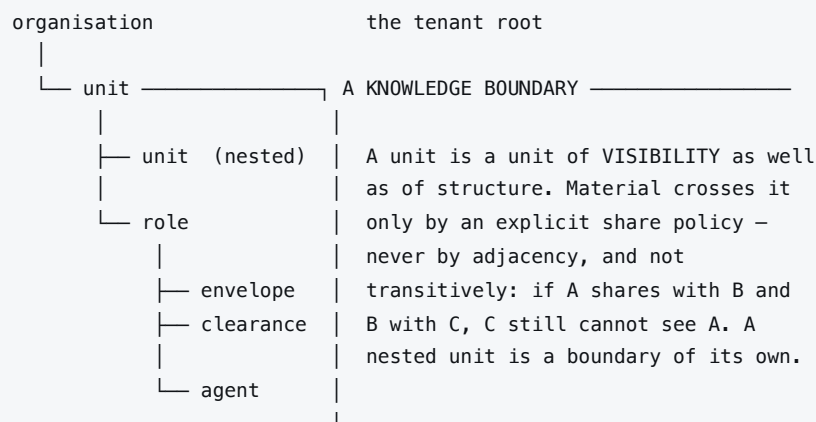
`isolation_domain` deserves a warning about defaults. The registry of planes is tenant-supplied — `api:GET /api/v1/organization-units/isolation-domains` — and **canonical Aegis ships zero planes**, which is the client's own wording for it. Read that as a claim about the product rather than about your estate: a deployment that has configured planes is no longer the canonical one, and an empty default tells you nothing about what your own deployment holds. *Design intent, not observable*: the grading of planes is a deployment's own doctrine about what must not touch what, so shipping an empty registry keeps the product generic and leaves the decision where the knowledge is. The setter replaces the whole set rather than merging, which is what makes *de-registering* a plane expressible at all — and also means a careless write silently removes every plane you did not re-list.

Two hierarchies, and they are not the same hierarchy

This is the structural fact most likely to cost you a day, so it gets its own heading — and its own diagram, because the two relations are easy to read as one until you see them drawn over the same objects.

— 1 — CONTAINMENT — what a thing LIVES INSIDE —

Carries: default classification · posture ceiling · isolation plane



envelope	what the delegate may DO – five dimensions
clearance	what the role may SEE
agent	the acting party. Its trust chain records where its authority came from; its posture, how much of that it may use unaided.

— 2 — REPORTING CHAIN — what a thing ANSWERS TO —

Carries: envelope composition · trust lineage · who delegates to whom

```
role —reports to→ role —reports to→ role → (no manager ⇒ CHAIN ROOT)
```

△ These are DIFFERENT RELATIONS over the same objects, and they can disagree. A role's manager may sit in a different unit from the one that contains it. The two are edited separately, and the divergence surfaces in the audit export rather than in anything that fails loudly.

Read the boundary column first. The knowledge boundary is drawn as a **region**, not as another edge in the tree, because visibility is not the containment relation — a unit's subtree does not become readable because it is nested inside something readable, and it does not become readable because a sibling is. Then note that panel 2 shares objects with panel 1 but not its shape: a role has exactly one containing unit and exactly one manager, and **those two facts are independent**.

There is a **unit tree** (`parent_unit_id`) and there is a **reporting chain** (`reports_to_role_id`). They are both real, both editable, and they can disagree. They do different jobs:

- The **unit tree** is where knowledge boundaries, classifications and posture ceilings live. It is read top-down: `api:GET /api/v1/organization-units/tree`, `/roots`, and per-unit `/{{id}}/children`, `/{{id}}/ancestors`, `/{{id}}/descendants`.
- The **reporting chain** is where authority and lineage live. Envelopes compose along it, and trust chains follow it. It is read with `api:GET /api/v1/organization-roles/{{id}}/reporting-chain` and `/{{id}}/direct-reports`, or as a whole picture at `api:GET /api/v1/admin/roles/hierarchy`.

A role with no `reports_to_role_id` is a **chain root**: it anchors its own lineage instead of inheriting one. That is legitimate, and it is also how an accidental orphan happens.

Creating a unit auto-creates that unit's head role, and **the auto-created head role arrives with no manager**. So a provisioning script that builds a tree but never parents the heads produces a set of units whose subtrees are each a separate authority structure, unattached from each other and from the top. No call fails. Nothing warns you.

The validators will not catch it, and it is worth knowing exactly what each one covers so you do not rely on the wrong one. `api:GET /api/v1/organization-units/validate` checks the **unit hierarchy** for orphaned units and circular references — units, not roles. `api:POST /api/v1/organization-builder/validate` checks a proposed structure for circular dependencies, missing fields, invalid relationships and constraint conflicts. Neither is described as a reporting-chain test.

So check it yourself, and check the right thing. `has_primary_role` on `api:GET /api/v1/organization-units/{id}/summary` tells you the head role exists; it does not tell you whether that head is *attached*. For that, walk the reporting chain — `api:GET /api/v1/organization-roles/{id}/reporting-chain` — on every role you expected to have a manager. A chain that comes back with a single entry is a root, and on a role you meant to nest, that is the finding.

Moving a unit is not a small edit

`api:POST /api/v1/organization-units/{id}/move` re-parents everything beneath the unit, and the response's cascade summary is the honest measure of what that touched: posture ceilings recompute, knowledge-share policies revalidate, bridges re-derive, and **trust chains are revoked and recreated**. Read that summary before treating a move as tidying up the chart. A re-parent is a governance event.

Roles — the accountability anchor

A role is the node almost everything else attaches to. It belongs to exactly one unit, it is what a human is assigned to, what an agent is linked to, what an envelope constrains and what a clearance is granted on. `api:POST /api/v1/organization-roles`, or the fuller twelve-operation surface on `sdk:agis_sdk.modules.RolesModule`.

field	what it actually does
<code>organization_unit_id</code>	the unit it belongs to — structural, exactly one
<code>reports_to_role_id</code>	the reporting edge; null makes it a chain root
<code>is_primary_for_unit</code>	marks the unit's head role
<code>authority_level</code>	1 (staff) ... 5 (C-suite) — informational
<code>auto_generate_agent</code>	creates a delegate agent with the role; defaults <code>true</code>
<code>is_external</code>	marks a role representing an outside party

authority_level does not gate access. It is seniority, and it is a tempting thing to build a permission model on. What gates seeing is clearance, what gates doing is the envelope, and neither of them reads `authority_level`. A level-5 role with no clearance sees less than a level-1 role with clearance — that is the intended behaviour, not a bug to work around.

It does gate *assignment*, and that half is enforced rather than advisory: a caller cannot attach anyone — including themselves — to a role whose `authority_level` sits above the caller's own effective ceiling. The rule closes the path of self-assigning into an existing high-authority role that somebody else legitimately created, which is why it is stated as a ceiling on the caller rather than as a property of the role.

A role holds at most one delegate agent, which is why the link calls take the role as the identifier:

`api:POST /api/v1/organization-roles/{id}/link-agent` and the bodyless `{id}/unlink-agent`. Linking does not widen anything — the agent acts inside the envelope the role already carries — so the link is a *naming* of who acts as this role, not a grant.

That is also why a role with nobody in it is a normal state rather than an incomplete one. The platform tracks vacancy explicitly, and a role can be stood up, linked to an agent, and be doing work before a human is ever appointed to it.

The address — one name for a position in the structure

Units and roles both carry an `address`, and it is the compact way to say *where in the structure this is* — a sequence of typed, numbered segments such as `D1-R1-T1-R2`. You do not have to decode it yourself: the platform will read one back to you in plain language with `api:POST /api/v1/governance/describe-address`, which is the right way to learn the grammar on your own deployment.

Half of this is read and half is not, and the difference is the part that matters. That a two-segment address names a *unit* is read: the explainer's own examples pass a field literally called `unit_address` holding `D1-R1`, so the association is the client's rather than this chapter's. What is **UNVERIFIED** is narrower, and it is what a parser would trip on — nothing this chapter could read expands the letters into domain, team and role, and nothing states that the **segment count** is a grammar rule rather than a coincidence of the examples. Do not carry "two segments means a unit, four means a role" out of this page: you have seen the pattern twice, not been told it. `api:POST /api/v1/governance/describe-address` settles it in one call, against your own deployment.

The address is worth knowing about for two reasons. It is **the identifier that survives renames** (titles get edited, addresses get recomputed), and it is **the input to envelope explanation**: `api:POST /api/v1/governance/explain-envelope` takes an address and returns the prose account of how that role's effective envelope was composed down through its ancestors. That is the single most useful call in this chapter when a bound is not what you expected, because it shows you the composition rather than making you reconstruct it.

`api:GET /api/v1/organization-units/validate` checks the tree for structural violations before you commit to one. Role-grammar violations — a head role not flagged as primary, a role whose address is null, an auto-generated role with no shadow agent — are enumerable at `api:GET /api/v1/governance/probe-corrupted-roles`, which is worth running after any bulk provisioning pass.

Agents — what occupies a role

An agent is the acting party. It is not a user, and it is not a role; it is the thing that runs, and a role gives it its position. `sdk:aegis_sdk.AgentType` and `sdk:aegis_sdk.AgentSubtype` are the two axes, and they are genuinely different questions.

Type separates what the agent *is* from a lifecycle standpoint, and the vocabularies are wider than the create surface — a distinction that matters because you will see values in listings you cannot create:

creatable through the API	minted by the platform itself
<code>chat</code> , <code>task</code> , <code>pipeline</code> , <code>custom</code>	<code>shadow</code> , <code>pseudo</code> , <code>tool</code> , <code>esa</code>

Presence in the right-hand column is not something you provision; those agents come into existence because some *other* object did. A tool agent is created by registering a tool. **A unit's head role carries a shadow agent — and that causal direction is inferred here rather than read.** The role model exposes a `shadow_agent_id` , and the platform raises

`shadow_agent_id_null_despite_auto_generate` when it is absent, which establishes that the two are linked; *which* of them brings the other into existence is not stated anywhere this chapter could read. Reading `agents.list()` and finding an agent you never made is normal, and finding it does not mean someone made it behind your back.

Subtype is behavioural — `specialist` , `manager` , `esa` , `pseudo` , `governance` — and describes the part the agent plays rather than its container.

Status (`sdk:aegis_sdk.AgentStatus`) is the lifecycle: `draft` , `active` , `archived` , `deprecated` , `suspended` , `revoked` . Treat `suspended` and `revoked` as reachable rather than exotic; an agent is transitioned to `suspended` by the platform when something goes wrong, which is exactly the moment your code is running and reading.

🚫 **`sdk:aegis_sdk.UnitType` is not the unit tree's unit type.** That enum is `atomic` / `composite` and classifies a *work* unit — a different domain that happens to share the word. The organisational unit type is the string `department` or `team` on an organisation unit. Two meanings, one word, and the enum name points at the one you probably did not mean.

Positions — a stance, which is not a structure

One object here is genuinely outside the org chart, and it is easy to miss because it sounds like it belongs in it. A **position** is a signed stance an agent holds on behalf of a principal — an opinion, a recommendation, a commitment — and the platform treats it as a claim that needs a human's authority before anyone may act on it. `sdk:aegis_sdk.modules.PositionsModule` .

It is created **fail-closed**. A new position is `unratified` and therefore not binding; the represented principal ratifies it (`api:POST /api/v1/positions/{id}/ratify`) before it means anything, and may retract it afterwards (`/api/v1/positions/{id}/retract`). Consumers read it through `api:POST`

`/api/v1/positions/{id}/consume` , which returns the position *only if it is binding* — so a design that reads a position and then decides for itself whether to trust it is re-implementing a check the platform already enforces.

Branch on the derived `is_binding` flag rather than comparing `binding_status` to a string. The status is the stored lifecycle state; the flag is the platform's own derived answer, and it is the same derivation `consume` enforces — so reading the flag is reading the value that actually decides.

🚫 **The positions surface admits a user session only.** It answers 403 for a client built with an API key, because an API-key principal carries no role and no personas by design. Use OAuth configuration for anything that touches it.

⚠️ **A caution on actor identity, because the decision surfaces are not uniform.** Some derive the actor from your session and take no actor argument. Others do not: the approvals surface **requires** a `reviewed_by` value and the client puts it in the **request body**, and the emergency and knowledge surfaces likewise take an `approver_role_id` or a `reviewer_id` . What the client sends is observable. **What the server does with a caller-supplied identity is not** — nothing reachable from here distinguishes "the server records the session principal" from "the server records the string you sent", and those two have very different consequences for an audit trail. This book's rule about anchors applies exactly here: a method existing and a field being accepted say nothing about what is enforced. **If the provenance of a recorded approval matters to your audit, settle it against your own deployment before you rely on it.**

The bounds that attach to a role

Two independent limits attach to the same node, and designing as though they were one is the most common modelling error on the platform. 02.3 has the task-level treatment; the structural statement is this:

- An **envelope** bounds what a delegate may *do*. It is authored by a *defining role* for a *target role* — `api:POST /api/v1/role-envelopes` , via `sdk:aegis_sdk.RoleEnvelopesModule` — and it is composed along the reporting chain by **monotonic tightening**, an intersection and never a union.
- A **clearance** bounds what a role may see — `api:POST /api/v1/role-clearances` , reached through the trust-posture module rather than through a module named for clearances.

The structural consequence is worth drawing out, and it is a **derivation** from the tightening rule rather than a separate guarantee: because a target's bounds are always a subset of its definer's, **the envelope graph is a DAG rooted in the reporting chain, not a free-form overlay**. You cannot express an escalation by adding an envelope. Widening is not a thing the model can say. If a delegate genuinely needs to exceed its bounds, the answer is a human decision, not a second envelope — and if you find yourself wanting one, that is the signal you are modelling the wrong structure.

Both take a **draft/active distinction**, and in both cases creating is not granting: an envelope defaults to `draft` and does not enforce until activated, and a new clearance lands in a pending vetting state and is not in force until approved. These are the same shape in two places, and the failure mode is identical — a provisioning script that creates fifty of something and walks away has produced an organisation that lists as bounded and enforces nothing.

Where classification meets clearance

Units carry a **default** classification; knowledge items carry an **explicit** one. Material created in a unit without an explicit level inherits the unit's default, which is why a unit created with the default value is a decision rather than a non-decision.

Knowledge itself is a tree, not a flat set. Each item carries a `path` that expresses containment — `api:POST /api/v1/knowledge`, with lifecycle operations on `sdk:aegis_sdk.KnowledgeModule` and the fuller review and share surface on `sdk:aegis_sdk.modules.KnowledgeGovernModule`. An item's `owner_unit_id` points it at a unit, and that ownership is what makes unit boundaries real: material sharing across a boundary does not follow adjacency on the chart, it follows a knowledge-share policy somebody wrote (`api:POST /api/v1/knowledge-share-policies`). **Sharing is not transitive** — if A shares with B and B shares with C, C still cannot see A's material, because every hop is its own policy. Designs that assume otherwise promise a capability that does not exist, in the safe direction.

A read is permitted when the role's clearance, the item's classification (plus compartment, at the top two levels), and any applicable share policy all line up. A failure in any of them presents identically, which is why `api:POST /api/v1/governance/explain-access` exists — it returns the step the evaluation reached and the access path taken, so you can see *which* of the three refused you instead of guessing.

Teams — a membership list, not a boundary

`api:POST /api/v1/teams`, members at `api:POST /api/v1/teams/{id}/members`. A team is a working group: a set of people, with no classification of its own and no knowledge-boundary semantics. It sits *beside* the containment tree rather than in it.

The practical rule: **if you want a boundary, you want a unit**. Teams are the right construct for "these people work together"; they are the wrong construct for "these people and nobody else may see this", and reaching for one there produces an access model that is more permissive than you believe it is.

Vocabulary — the ontology

Finally, the tenant has a vocabulary. `api:POST /api/v1/ontology/preset/{id}` applies a preset — replacing custom terms with that preset's defaults — `api:GET /api/v1/ontology/presets` and `api:GET /api/v1/ontology/config/{id}` read what is in force, and `sdk:aegis_sdk.OntologyModule` wraps them.

What is documented is that applying a preset **replaces the terms**. That the objects underneath are unaffected is **UNVERIFIED** — it is the behaviour you would expect of a vocabulary layer, and the safe assumption until you check it on your own deployment, but this chapter did not establish it. If your integration keys on a term rather than on an identifier, confirm the two independently.

Set it early. It is nearly free at provisioning time and disruptive once people have learned the old words, which makes it a decision worth making deliberately rather than by never making it.

The model, once

object	answers	nests in	bounded by
Organisation	whose tenant is this?	—	—
Unit	where in the boundary tree?	a unit, or none (root)	default classification, ceiling, isolation plane
Role	who is accountable here?	exactly one unit	envelope, clearance, reporting edge
Agent	who acts?	a role (at most one)	envelope and clearance, via the role
User	which human?	a role, by assignment	role + its bounds
Team	who works together?	nothing — membership only	—
Position	what does an agent assert?	nothing — a stance	ratification by the represented principal
Knowledge item	what is the governed material?	a path, owned by a unit	classification + compartment + share policy
Envelope / Clearance	what may this role do / see?	attached to a role	the definer's own bounds (tightening only)

Next: 06.4 — Trust and the audit spine

06.4 — Trust and the audit spine

02.4 showed you how to establish a chain, move a posture and revoke one. This chapter is about the shape underneath those calls: what trust *is* as an object here, how a decision to permit or refuse is actually reached, how standing can be withdrawn, and what survives an action so that somebody who was not there can reconstruct it later.

The single most useful thing to carry into your design is this: **trust here is not a boolean the platform computes about an agent. It is a set of first-class objects you can read, compose and argue with** — a lineage that names where authority came from, a standing that moves and can be explained, and a decision that can be re-derived after the fact. Everything below follows from taking that seriously.

Four objects that people collapse into one

object	answers	mutable?	read it at
Authority	who attested this agent?	created, deactivated	<code>sdk:aegis_sdk.trust.AuthoritiesModule</code>
Chain	where did its authority come from?	established, revoked	<code>sdk:aegis_sdk.trust.ChainsModule</code>
Posture	how much may it do unaided, now?	moves both ways	<code>sdk:aegis_sdk.trust.PosturesModule</code>
Decision	was <i>this</i> action permitted?	— it is a record	<code>api:POST /api/v1/trust/verify</code>

They are separable in practice, not just in principle. An agent can hold a valid chain and a restrictive posture — that is the normal state of anything newly provisioned. An authority can be retired while chains established under it continue to exist. A chain can be live and a particular action still refused, because the decision consults the envelope and the posture as well.

The authority — the signing root

A chain is not self-originating. An **organisational authority** is the signing root a chain is established under: agents are attested *by* an authority, and that relationship is recorded in the chain's genesis. Enumerate them at `api:GET /api/v1/trust/authorities`, create one with `api:POST /api/v1/trust/authorities`, and read the agents established under one at `api:GET /api/v1/trust/authorities/{id}/agents`.

Deactivating an authority (`api:POST /api/v1/trust/authorities/{id}/deactivate`) is not a rename — it is a **security transition that every chain established under it inherits**, which is why the platform requires a reason and records it on the deactivation entry. Treat it like a revocation, not like housekeeping.

Two honest limits on the read surface. The established-agent count on the display routes is computed by scanning trust chains, and it is **absent rather than zero** when the platform could not compute it — an unknown count and a genuine zero are different facts, and the model deliberately does not collapse them. And `list_agents` itself walks a bounded page of chains, so on a large tenant it can return a partial answer **with nothing in the response marking it as partial**.

The chain — lineage, and its one nesting trap

Establishing is `api:POST /api/v1/trust/establish`, returning `sdk:aegis_sdk.EstablishedTrustChain`. The body names the agent, the authority, a list of **capability labels** and a list of **constraint labels** — both are `list[str]`, not dictionaries; the actual bounds live in envelopes, which is 02.3. Expiry defaults to 365 days, and chains genuinely expire.

Two properties of this call are worth restating because they constrain what you can build rather than merely how you call it. **Human origin is derived server-side from the authenticated caller** and cannot be supplied — the parameter is accepted and ignored. *Design intent, not observable*: an identity a client can assert is an identity a client can forge, so the one fact the lineage must not be able to lie about is taken from the session. The practical consequence is that **the human on the record is whoever's credentials ran the script**, which is why provisioning under a shared machine key produces a tenant whose entire authority structure descends from a service account.

🚫 **Identity is NESTED one level down, and this is the trap.** On the full lineage document returned by `api:GET /api/v1/trust/chains/{id}`, `id`, `agent_id` and `authority_id` live on `genesis`, **not** at the top level. A reader that looks for those three names a level too high finds none of them — and on a permissive parser, reads three nulls beside fully-populated capability rows, which looks like a response problem rather than a lookup one. `sdk:aegis_sdk.TrustChain` is the flat summary shape and `sdk:aegis_sdk.EstablishedTrustChain` is the establishment shape; the lineage document is a third, richer shape again. For the narrow projection, when the lineage is more than you need, use `api:GET /api/v1/trust/chains/{id}/summary`.

The delegation path is `api:GET /api/v1/trust/chains/{id}/delegation-path`, returning `sdk:aegis_sdk.DelegationPath` — the ordered walk from the human root down to this agent, its depth, and the ceiling in force. Note the two addressings in play: **chains are keyed by agent**, so what looks like a chain id in these routes is an agent id, and an agent with no chain returns an *empty path rather than an error*. `None` there means "no chain", never "failed to load".

One more read worth knowing about before you depend on it: the summary projection is **clearance-gated under selective disclosure**. Some fields — the two parties and the bridge origin — may be *absent* rather than null when your clearance does not reach them, while `id`, `status` and `created_at` are always visible. That means a null on a gated field is ambiguous between "not disclosed to you" and "genuinely absent", and the response cannot tell you which. Do not build a security decision on reading absence.

Standing — posture as a moving object

A posture is not a property of the chain; it is the current answer to *how much may this agent do without a human in the loop*, and it moves. Read it at `api:GET /api/v1/agents/{id}/trust-posture`. The five levels are CARE-aligned and lowercase — `sdk:aegis_sdk.TrustPosture` — and 02.4 has the table.

The read returns more structure than people expect, and two fields in it are load-bearing:

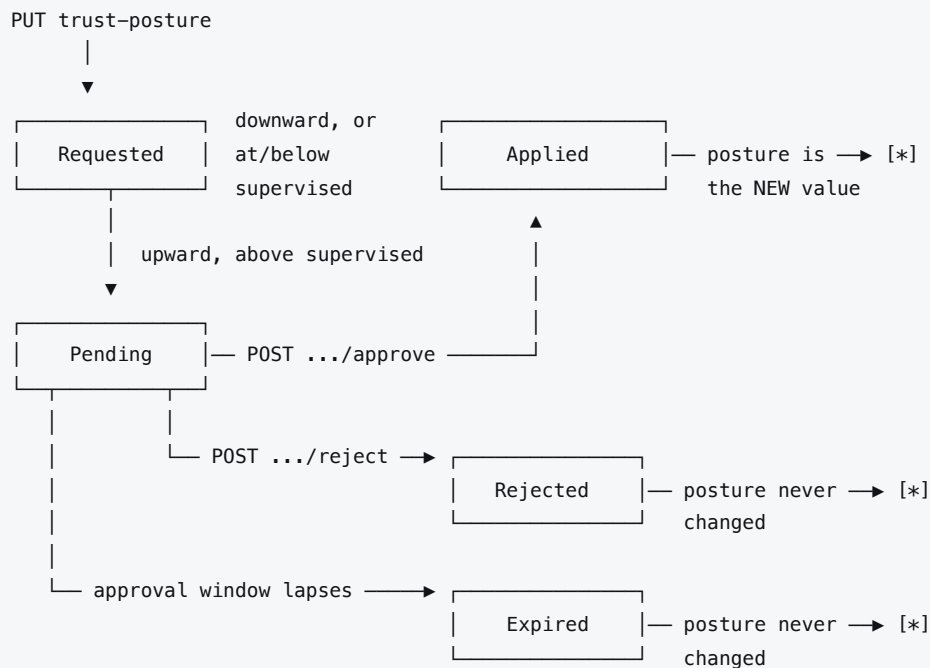
- **posture** is the **EFFECTIVE posture**. If an override is active, this is the override's value, not the agent's own.
- **base_posture** is what the agent reverts to when the override lapses.

So "what posture is this agent at" has two correct answers depending on whether you mean *now* or *by default*, and an emergency restriction implemented as an override is precisely the case where they differ. `sdk:aegis_sdk.PostureOverride` models the override row itself, including who initiated it, why, and when it expires.

Changing it: one endpoint, two outcomes

Both the governed progression request and the administrative override go through the *same* call — `api:PUT /api/v1/agents/{id}/trust-posture` — which decides internally whether the change applies immediately or is held for a human. There is no separate override endpoint, and the two are gated on the same permission.

The return shape exists because the two outcomes demand **opposite next actions**, and inferring them is what goes wrong:



While a change is Pending the agent is **STILL RUNNING** at the old posture, and `result.posture` reports the OLD value. Do not re-issue the same transition to "make it take" – it records nothing new and comes back flagged.

The two routes drawn above are `POST /api/v1/agents/{id}/trust-posture/approve` and `POST /api/v1/agents/{id}/trust-posture/reject`. The agent id is elided in the picture; the `/trust-posture` resource segment is not.

Branch on `approval_pending`. When it is `True`, `result.posture` is the posture the agent is **still running at** – reporting it as the new one is the single most common integration defect in this area, and it is a reporting defect that compounds: a caller that retries after a timeout files a *second* pending request, because the endpoint has no idempotency key. Re-issuing the same transition while one is pending records nothing new and comes back flagged – check `api:GET /api/v1/agents/{id}/trust-posture/pending` before every request, and treat a non-null answer as *already asked* rather than as an obstacle.

A pending change is decided at `api:POST /api/v1/agents/{id}/trust-posture/approve` or the matching `.../reject`. Both accept an explicit approval id, and past one pending request the server **refuses rather than picking one** – which is the right behaviour, and means your code should never let a second request accumulate in the first place.

Where the evidence lives

There is a whole read surface for *why* an agent's standing is what it is, and it is worth more to an auditor than the posture value itself. The eligibility verdict is `api:GET /api/v1/agents/{id}/trust-posture/evaluate`, and it is the **only** call that answers "may this agent progress" – no posture read and no change response carries an eligibility verdict. The behavioural rates it is computed from are at `.../trust-posture/metrics`; the basis is at `.../trust-posture/evidence`; and the trail of how the agent got here is at `.../trust-posture/history`. `sdk:aegis_sdk.TrustPostureModule` holds the per-agent reads.

Look at the history endpoint before you decide whether your governance is real. If every entry in it is an override, the graduated progression was decorative: the agent was placed at its autonomy level rather than having earned it, and an agent that has never been observed under supervision has no evidence behind its autonomy — which is exactly the thing you would need to show.

The decision path — how a permit is reached

`api:POST /api/v1/trust/verify` is the operation the enforcement path calls on an agent's behalf. It takes an agent, an **action** (a string like `write` or `execute`), and a target addressed as a **kind plus an optional instance**: `resource_type` says what sort of thing is being acted on, `resource_id` says which one. The kind is required, and the client will refuse rather than invent one — *design intent, not observable*: the platform writes that value into its audit trail, so a fabricated placeholder would read as one the caller had named.

The result is a decision plus the platform's reasoning: `sdk:aegis_sdk.TrustVerificationResult` carries `allowed` and `reason`, and the richer shape carries `capabilities_matched` and — the one to read when a request was refused — `constraints_violated`. Two traps in the shape: `chain_id` and `constraints_applied` are inherited from a base model and are **always empty on this route**, so reading `constraints_applied` for the reason a denial happened tells you nothing. Read `constraints_violated`. And caller-supplied `context` is accepted and ignored: the evaluation runs against the chain's own recorded constraints, never against anything the request asserts about itself.

Asking why, without doing the thing

`api:POST /api/v1/governance/explain-access` — on `sdk:aegis_sdk.modules.GovernanceExplainModule` — returns the same decision as a trace rather than a verdict: whether it was allowed, the reason, **the step the evaluation reached**, and the access path taken. That step name is the useful field on a refusal, because it tells you *where* the chain stopped.

Read its two subject modes carefully, because they are not the same kind of answer. A **knowledge item** subject is a dry run: nothing is accessed, nothing is recorded, and the verdict is only as good as the item description you passed — it is not evidence that a real access would be permitted. A **tool** subject is the opposite: it reads the refusals that were actually persisted for that tool's work unit, so the step and reason are that gate's own words rather than a recomputation. And a tool subject with no refusal on record reports the *absence of a record*, which is not the same as a determination that the tool is permitted. That distinction is stated in the response itself; believe it.

One question worth asking on a schedule

`api:GET /api/v1/governance/envelope-coverage` answers not *"do envelopes exist"* but *"would they stop anything"*. It is the closest thing here to a readiness check on the governance layer, and it is honest in a way that makes it usable: an envelope that resolves to allow-all is counted in its own bucket rather than as coverage, and the report carries a **completeness flag** — when the measurement did not fully happen, the counts are a floor and the report says which part was missed. Check completeness first, or you will quote a partial number as a total.

The same honesty appears one level down in the hydration status (`api:GET /api/v1/governance/envelope-hydration-status`): the envelopes a hydration pass **skipped** are surfaced rather than silently dropped, and a skipped envelope means the engine fell back to a restrictive bootstrap default. A skip is fail-closed, not a hole — but it is also invisible everywhere else, which is why it has its own read.

Where enforcement actually attaches

Two questions, and both have a precise answer that is shorter than people expect.

To which object? To two, and the division is the important part. The bounds split by *whose* they are:

attached to	what it carries	why there
the role	envelope, clearance	they describe the <i>seat</i> , so they survive whoever occupies it
the agent	trust chain, posture	they describe the <i>occupant</i> — its lineage, and how far it may go unaided

Both are consulted at the same decision (Gate 2 below), and the split is what makes the two facts independent: **swapping the agent in a role changes the agent's standing and leaves the role's bounds untouched**, which is exactly what makes a seat reusable and a re-org survivable. It is also why linking an agent to a role grants nothing — the role's bounds were already there and the agent brings only its own. The organisation and the unit sit above both and contribute *ceilings* that propagate down; neither carries a value that is consulted directly.

At which moment? At the decision — not at the declaration. Every authority object has a declared state and a consulted state, and they are not the same moment and are frequently not the same value. An envelope that exists in `draft` is *declared* in every listing you will read and is *consulted* by nothing. A clearance awaiting vetting is declared and is not consulted. That gap is the single most common way a governance design ends up looking complete and enforcing nothing, and the diagram below is the whole of it.

WHERE A BOUND IS CONSULTED – and where it is only DECLARED

— THE DECISION PATH — each step is a place the answer can change —————

an agent forms an intent to act

```
|
|  nothing consults a bound here – an intent is not yet a request
```

▼

GATE 1 · ADMISSION

Is the CALL admitted? credential · tenancy

Refused here, no plane is reached and no agent action was decided.

1

▼

```
┌ GATE 2 · THE TRUST DECISION    ★ ENFORCEMENT ATTACHES HERE ★ ───────────
```

```
|
```

```
|   trust chain    is there authority, and is it still valid?
```

```
|   posture        how much may be used unaided, RIGHT NOW?
```

```
|   envelope       does the bound permit THIS action?
```

```
|
```

```
|
```

```
|   ⇒ ONE verdict, on THIS action:    permit /  refuse
```

1

- |— PERMIT → the action proceeds. The platform is no longer in the path; the record is written afterwards.

1

- └ REFUSE $\rightarrow$ the action does not happen, and the reason names the constraint that refused it.

DECLARED, BUT NOT CONSULTED AT GATE 2 – none of these changes a verdict:

· an envelope in DRAFT	it appears in every listing and is not in force until activated
· a clearance PENDING vetting	it exists; the role still cannot see
· a trust SCORE or CARE budget	a projection for humans – the enforcement path does not read it
· an envelope resolving to ALLOW-ALL	counted in coverage as a FAILURE bucket, precisely because it stops nothing

Two things to take from the shape. The gates are **sequential and different**: a refusal at Gate 1 is a statement about your credential, and a refusal at Gate 2 is a statement about the agent — they arrive with overlapping status codes and have completely different remedies, which is 06.1's point about plane-versus-door made concrete. And the second list is not a list of bugs. Each of those objects is in the state it is in by design; they are simply *not* part of the decision, and a design that treats their existence as coverage has misread which half of the system it is looking at.

⚠ **Whether Gate 2 fires before the action is runtime-dependent.** 06.2 is explicit about this and it is not a detail: the hook model that puts the decision in front of the tool call is the `claude_agent_sdk` runtime. The default agent runtime has **no pre-execution gate**, and the platform warns at boot that it does not. The decision still exists and is still recorded — what changes is whether anything *stops*. Establish which runtime your deployment runs before you design a control that depends on a refusal happening.

What is enforced, and what is only recorded

This is the table to design against, because treating a recording surface as an enforcement surface is how a system ends up governed on paper only.

surface	enforced?	what that means for you
<code>verify</code> — the trust decision	enforced	the path the runtime calls; a refusal stops the action
envelope resolution	enforced	bounds what a delegate may do; fail-closed on a skip
posture materialisation	enforced	the agent's own permission set is written from it
clearance vs classification	enforced	gates reads; failure presents as "cannot see"
trust score / CARE budget reads	not enforced — display only	the enforcement path does not consult them
posture progression <i>metrics</i>	inputs, not a verdict	they are what eligibility is computed <i>from</i>
pipeline pre-flight validation	partial	constraints are not evaluated on that route today
audit log	a record	it tells you what happened; it does not prevent it

⚠ **Read the middle column as what each surface is *for*, not as a measurement this chapter took.** Every row rests on what the surface says about itself and on what you can observe when a call is refused — but **no client can demonstrate that a control actually stops an action**. An anchor proves a surface exists; it cannot prove it refuses anything, and a control that silently stopped enforcing looks identical to one that never stopped. The four `enforced` rows are the platform's own account of its decision path. They are the rows most likely to be lifted into a design document as fact, which is why the caution sits here rather than below.

Three of those deserve expanding, because each is a place where the surface looks stronger than it is.

The scoring reads are display-only, and the module says so. The A–F grade and the five-dimension CARE budget (`api:GET /api/v1/trust/agents/{id}/trust-score` , `/care-budget`) are projections for humans. The platform's enforcement path does not consult them. **design intent, not observable:** a

score that gates an action is a score an attacker optimises against, so it is kept out of the decision path. Design accordingly — never gate your own logic on a grade.

Pipeline validation checks less than its name suggests. Pre-flight validation across a multi-agent pipeline (`api:POST /api/v1/trust/pipeline/validate` , `sdk:aegis_sdk.trust.PipelineTrustModule`) confirms every agent holds an active chain carrying the capabilities its step requires. Its `violated_constraints` field is **always empty because constraints are not evaluated on that route** — so an empty list there means *not checked*, not *none violated*. Read `all_valid` as a statement about chains and capabilities only.

Trust metrics are a sample on a large tenant. `api:GET /api/v1/trust/metrics` computes its chain and verification counts from a bounded page of records, and the response carries **no marker distinguishing a sample from a total**. For an organisation with more chains than that page holds, every figure is the most recent slice rather than the population. The compliance report (`api:GET /api/v1/trust/compliance/{id}`) scans further and *declares when it could not* — so when a number has to be defensible, that is the surface to reach for, and the honest move is to say which one you used.

Withdrawal — revocation, and the shapes that confuse people

`api:POST /api/v1/trust/revoke/{id}/cascade` is the permanent, cascading removal. `sdk:aegis_sdk.CascadeRevocationResult` reports what it revoked. The cascade is not optional: **every revocation route cascades**, so there is no non-cascading revoke to reach for, and revoking one agent revokes everything that derived authority from it.

Look before you cut. `api:GET /api/v1/trust/revoke/{id}/impact` returns the `sdk:aegis_sdk.RevocationImpact` preview — how many agents are affected, and whether active workloads are at risk. That preview is the difference between removing one agent and silently removing a department's, and it costs one call. `api:POST /api/v1/trust/revoke/by-human/{id}` revokes everything originating from a particular person, which is what you want when someone leaves; `api:POST /api/v1/trust/revoke-delegation` removes a single delegation edge.

△ **A cascade can fail to complete, and that is a real, discoverable state.** `api:GET /api/v1/trust/revoke/jobs/incomplete` lists cascade revocations that did not finish — and **until a job here is resumed, agents it named may still hold active trust**. `api:POST /api/v1/trust/revoke/jobs/{id}/resume` retries it, idempotently, and reports whether the resume itself timed out again. Read the two counts on a job — completed against total — rather than its status string alone, because the status does not tell you how far it got. **Put this listing on a schedule.** A revocation you believe landed and which did not is the most expensive possible version of this system's failure modes.

The reversible brake is a posture, not a chain state. There is no operation that suspends a trust chain; the methods that appear to be one exist only as deprecation shims and always raise. If you need to stop an agent acting without permanently ending its lineage, the honest tool is a *downward* posture

override — `api:PUT /api/v1/agents/{id}/trust-posture` — which applies immediately rather than waiting on approval, and which you can lift afterwards. Use revocation for the decision you mean to be permanent.

The audit spine — two logs, and they answer different questions

There are two distinct audit surfaces, they are written by different subsystems, and conflating them is how an evidence request gets answered with the wrong document.

	platform audit	trust audit
read at	<code>api:GET /api/v1/audit/logs</code>	<code>api:GET /api/v1/trust/audit</code>
records	user actions, resource changes	trust decisions — who delegated what, on what basis
module	<code>sdk:aegis_sdk.modules.ObserveAuditModule</code>	<code>sdk:aegis_sdk.trust.AuditModule</code>
answers	"who changed this?"	"why was this agent allowed?"

Both are queryable and exportable — `api:GET /api/v1/audit/export` for the platform log, and per-human filtering on the trust side at `api:GET /api/v1/trust/audit/by-human/{id}`.

Walking a decision back to the human who authorized it

The trust audit's most valuable operation is `api:GET /api/v1/trust/audit/trace/{id}`, which takes a single audit entry and walks its lineage **backwards to the root human source** — the delegation chain, its depth, and the person at the top. This is the operation that discharges the accountability claim: an organisation can point at an action and name the human whose authority it descended from.

🚫 Read the `verified` flag on a trace with its own warning in hand. That field reports signature verification but **defaults to true for an entry that carries no signature at all**. It therefore means *"no signature check failed"*, not *"this entry is cryptographically verified"* — an unsigned entry and a validly signed one are indistinguishable on that field alone. Check whether the record carries a signature before treating the flag as evidence, and do not report a trace as cryptographically verified on the strength of it.

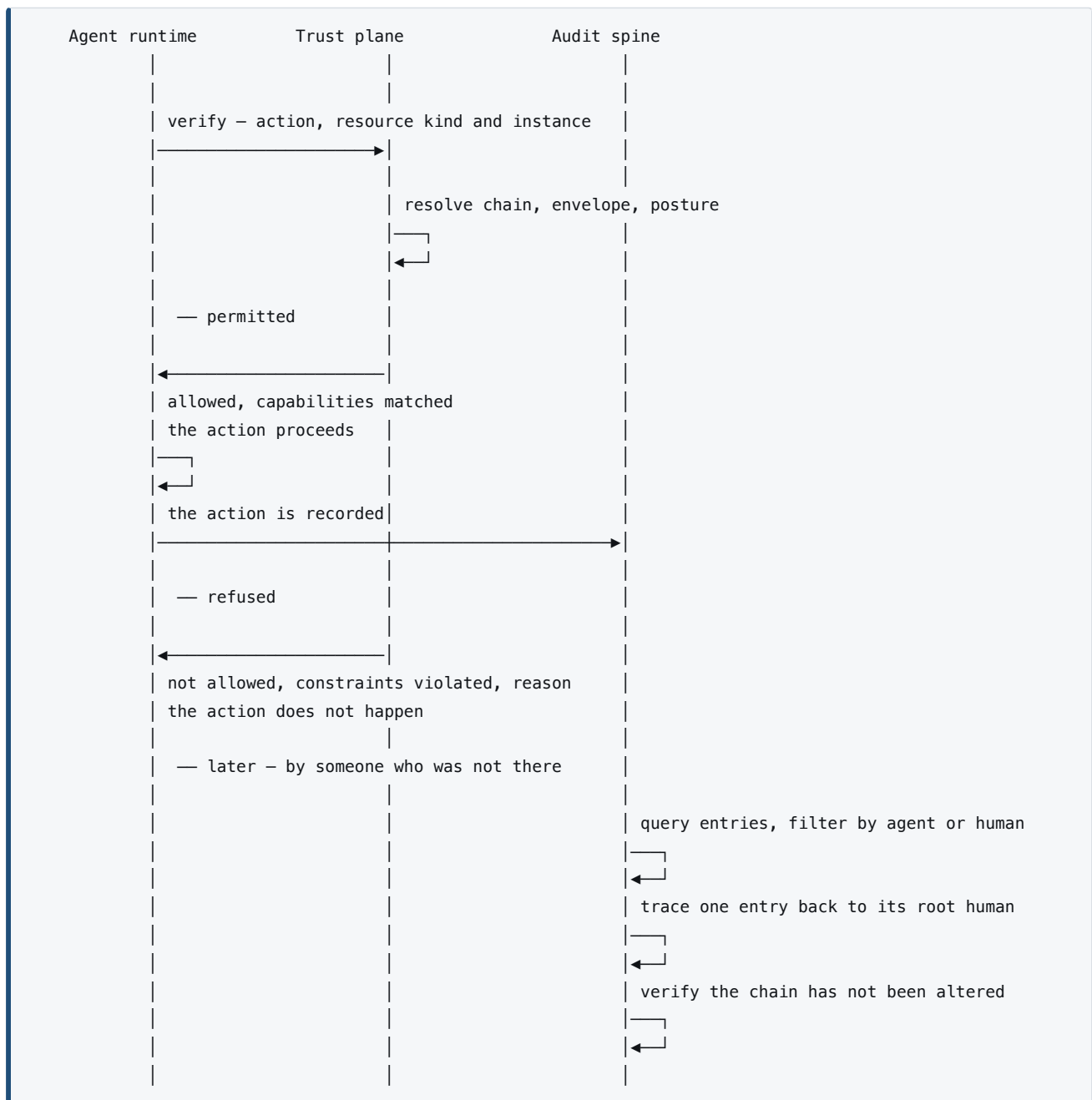
Is the log itself intact?

The audit log is presented as tamper-evident, and there are two different reads for that claim. `api:GET /api/v1/compliance/audit/verify` performs the verification walk — a cryptographic hash-chain check with optional start and end bounds — and returns whether it was valid, how many entries were checked, and the id of the first invalid entry if tampering was found. That is an **on-demand** check you run.

`api:GET /api/v1/compliance/chain/integrity` is the *other* half: it reads the result of the platform's own periodic verification sweep. It does not trigger a sweep. And it has a cold-start trap worth knowing — **check the last-verified timestamp before reading the ok flag**, because a deployment that has not yet run a sweep reports *not ok*, which is a "nothing has been checked yet" finding and not a "something is broken" one. Those two produce the same boolean and opposite alarms.

Reconstructing an action, end to end

The decision diagram earlier answered *where a bound is consulted*. This one answers a different question — *what happens when* — and the gap between the two halves of it is the whole reason an audit spine exists.



The shape to notice is that the decision and the record are **different witnesses**. The decision is about an intent — a tool name and its arguments, or an action and a resource kind — made before anything happened. The record is written after. Neither substitutes for the other, and a system that can produce one and not the other cannot answer the question an auditor will actually ask, which is not "was something enforced" but "show me that it was".

What this spine does not establish

Stated as plainly as the rest, because these are the places a design goes wrong by assuming more than is there.

- **A permitted decision is a decision about an intent, not a guarantee about an outcome.** No response body exists when the verdict is made.
- **A green governance check is not a statement about your configuration being correct.** Coverage reports readiness; they do not know what you meant.
- **The audit log records; it does not prevent.** Tamper-*evidence* is a property of the hash chain, not a claim that nothing can be changed before it is verified.
- **The trust registry's discovery surface reads chains, not agents** — see `sdk:aegis_sdk.trust.TrustRegistryModule` and `api:POST /api/v1/trust/registry/discover`. An agent with no chain is **invisible to discovery** even if it exists and is running, so an empty discovery result is not evidence that no such agent exists.
- **A signed lineage is not a trusted one.** Signature verification tells you the record was not altered; it says nothing about whether the authority that signed it should have.

UNVERIFIED in this edition: the precise conditions under which the platform runs its periodic chain-verification sweep — interval, trigger and whether an operator can schedule one — could not be settled from what is reachable here. `api:GET /api/v1/compliance/chain/integrity` reports the *result* of a sweep; it does not describe the schedule. If the freshness of that verification matters to your controls, read `last_verified_at` on a schedule of your own and alarm on staleness, rather than assuming a cadence.

Next: 06.5 — The execution model

06.5 — The execution model

Parts 02 and 04 told you what to call. This is the shape underneath: what a unit of work *is*, what happens to it between submission and result, who can be holding it at any moment, and what you can actually observe while it runs.

Read it before you design anything that submits work, watches it, or answers a question about it. Most of the surprises are in the second half.

The split that shapes everything: who decides, who executes

Before any object, there is one division. Aegis is built on **two planes**:

plane	what lives there	the question it answers
Trust Plane	envelopes, clearances, postures, delegations, approvals, revocations	<i>what is the agent allowed to do?</i>
Execution Plane	objectives, requests, tool calls, sessions, artifacts	<i>what does the agent actually do, inside what it was allowed?</i>

The division of labour is the whole design in five words: **the human decides, the AI executes**. A person sets the bounds and answers what the system escalates to them. They do not approve every action — that would make throughput a function of their attention, and the system would get slower as it got more capable.

What the split buys is a single structural property: **an agent cannot widen its own mandate, because the mandate is not decided where the work happens**. That is not a policy anyone has to remember to enforce. It is why the never-delegated set below contains *changing the constraints* and *changing the governance* — an agent at maximum autonomy still cannot reach either.

 **Do not read the two planes as two services.** This is the reading that gets made and it is wrong. There is no trust-plane process to point at, no second cluster, and no database split — the split is between **who decides** and **who executes**, and it is realised inside the same deployment, across many components. A design that assumes a network boundary between the planes will build a hop that does not exist, and — worse — will locate the human in the wrong place. The human is on the loop in the trust plane, not in a queue beside it.

Work is an object graph, not a call

Nothing here is a function you call and get a result from. An objective is accepted, decomposed, executed by one or more agents, and produces files. Five objects carry that, and they nest in a direction that is not the obvious one:

object	what it is	how it is addressed
objective	a goal you ask for	api:POST /api/v1/objectives , read back at api:GET /api/v1/objectives/{}
request	one work item, produced by decomposition; some need a person	api:GET /api/v1/objectives/{}/requests
task graph	that decomposition as nodes and edges	api:GET /api/v1/objectives/{}/task-graph
session	one agent's execution context against a request	api:POST /api/v1/sessions/from- task , read at api:GET /api/v1/sessions/{}
artifact	a file produced against a request	api:POST /api/v1/artifacts , read at api:GET /api/v1/artifacts/{}

The artifact edge points at the request, not the objective. That is the one relationship people get backwards, and it has consequences at the end of this chapter. api:GET /api/v1/objectives/{}/artifacts exists, but it is a roll-up across the objective's requests — it is not where an artifact lives.

A **session** is the unit of "what an agent actually did": messages, artifacts, and any subagents it spawned, readable at api:GET /api/v1/sessions/{}/subagents and streamed live (below).

The second family: work units are definitions, not instances

Objectives are *things you asked for*. A **work unit** is a reusable, versioned definition you keep and execute on demand — closer to a function than a job. It has its own lifecycle, its own configuration and its own version chain:

	objective	work unit
lifetime	one piece of work	reusable across many runs
versioning	none	api:GET /api/v1/work- units/{}/versions
what runs	a decomposition into requests	the unit itself
executed via	confirm-implementation	api:POST /api/v1/work- units/{}/run or api:POST /api/v1/work-units/{}/execute

The two execution routes are genuinely different and you should pick deliberately. run is synchronous and hands back a run result. execute drives an agent: it takes a message, returns a run_id with cost_usd , cycles_used and tokens_used , and continues across turns if you pass a session_id . Only execute produces something you can cancel mid-flight, with the run_id .

A work unit also carries an **execution configuration** — runtime, model, execution mode, budget ceiling in USD, cycle cap, timeout, memory depth, and the allowed-tool list — read and written at api:GET /api/v1/work-units/{}/execution-config . If you are building a provisioning script, this is the object

you set once and version, rather than restating on every call.

`sdk:aegis_sdk.modules.WorkObjectivesModule` wraps this surface; the objective-side methods live on `sdk:aegis_sdk.execution.ObjectivesModule`.

The lifecycle, and the collapse in the middle of it

Here is the real state vocabulary an objective moves through on the wire:

The main line – each state hands off to the next:

[*] → draft → pending → plan_review → confirmed → executing

Every transition, with the ones an OPERATION establishes marked in the third column. The unmarked rows are the ordering this book reads from the state names and the way the routes are named: design intent, not observable.

from	to	established by
[*]	draft	
draft	pending	
pending	plan_review	
plan_review	confirmed	
plan_review	executing	confirm-implementation
confirmed	executing	confirm-implementation
executing	decision	the run stops to ask
executing	completed	
executing	failed	
executing	cancelled	stop
decision	executing	revise / more_analysis
decision	completed	proceed
decision	cancelled	reject
completed	[*]	
cancelled	[*]	
failed	[*]	

Three of the sixteen are terminal, and `decision` is the only state that can return to `executing` – the loop in the middle of the lifecycle.

Edges carrying an operation name are established by that operation's documented transition. The unlabelled edges are the ordering this book reads from the state names and the way the routes are named; no single route documents them, so treat the sequencing as **design intent, not observable**.

Now the part that matters. 🚫 **open defect** — the client's own module documentation calls this a known limitation and a tracked follow-up, so it is declared rather than accidental; it is still the thing most likely to shape your design wrongly.

`sdk:aegis_sdk.ObjectiveStatus` declares six values, and the client maps those nine states onto them:

wire state	what <code>ObjectiveStatus</code> reports
<code>draft</code>	<code>DRAFT</code>
<code>pending</code> , <code>plan_review</code> , <code>confirmed</code>	<code>PENDING</code>
<code>executing</code> , <code>decision</code>	<code>IN_PROGRESS</code>
<code>completed</code> / <code>cancelled</code> / <code>failed</code>	<code>COMPLETED</code> / <code>CANCELLED</code> / <code>FAILED</code>

Two collapses, and both bite:

- `plan_review` , `confirmed` and `pending` are one value. "Nobody has looked at it", "a human is reviewing the plan" and "the plan is approved and waiting to launch" are indistinguishable.
- `decision` reads as `IN_PROGRESS` . An objective that has *stopped and is waiting for a person* is reported by the typed model as one that is running.

So an operator dashboard that branches on the `status` field of `sdk:aegis_sdk.Objective` cannot show that anything is waiting on a human. That is the single most consequential fact on this page.

And the mapping fails open on a state it does not recognise. The nine above are the client's belief about the platform's vocabulary; a status this client has never heard of resolves to `PENDING` , not to an error. So a state the platform adds later arrives looking like work nobody has started, rather than looking like something your client does not understand.

The workaround is cheap and you should build it in from the start: **read the status off the progress route instead.** `api:GET /api/v1/objectives/{}/progress` returns a raw map carrying the unmapped `status` string alongside `overall_progress` , `steps` , `graph_nodes` , `elapsed_ms` , `estimated_remaining_ms` , `cost` and `constraint_dimensions` . The typed `sdk:aegis_sdk.Objective` does not carry the raw value — it is dropped during normalisation and the model has no field to hold it.

Standing advice, stated once for the whole book: **treat the SDK's status enums as a convenience for coarse filtering, never as the thing you branch on for control flow.** Where a raw-dict method exposes the wire value, prefer it.

How work starts

Creation does not start anything, and the gap is wider here than anywhere else in the platform. Three separate hand-offs stand between "created" and "running":

1. Clarification, if the objective needs it. The platform can generate questions before decomposing. Generation is asynchronous: `api:POST /api/v1/objectives/{}/clarify/trigger` returns as soon as the work is queued, and the questions are not in the response. Poll `api:GET /api/v1/objectives/{}/clarification-status` or watch the progress stream.

Read the `status` field on that poll, because **an empty question list does not mean there is nothing to ask.** `generating` means not yet. `failed` means generation did not finish, and an explanatory `message` is present only in that case — the questions are empty for a reason that is *not* "no clarification was needed". Only `completed` settles it.

Answering triggers the next stage. `api:POST /api/v1/objectives/{}/clarify` submits answers and starts decomposition as a background task; the free-form variant accepts one block of text and does the same. Recording an answer is not a passive act here.

2. Decomposition. The objective becomes a task graph — nodes, edges, an estimated total in minutes and dollars. Read the result at `api:GET /api/v1/objectives/{}/task-graph`. `trigger_execution` starts this and is safe to repeat: it answers either `processing`, or `already_decomposed` with a `task_count`. Branch on that string rather than treating a repeat call as an error.

3. Confirmation. `api:POST /api/v1/objectives/{}/confirm-implementation` takes no body, moves `plan_review / confirmed` to `executing`, and **launches execution as a background task**. Read the last four words carefully: the call returns when the work has been *launched*, not when it has been done. Everything you learn about the run from here on comes from a poll or a stream.

This is the same *describing* versus *effecting* split you meet on draft envelopes, unsubmitted objectives and unapplied change requests — by now it should read as the platform's consistent habit rather than a quirk.

Where a bound is actually consulted

⚠ **The mechanism below is the platform's account of itself, not something a client can observe.** None of it is derivable from the SDK surface: you can see that a call was refused, never which object consulted what, and no route returns the composition rules. It is set out because you are designing against it, and it is labelled because a design resting on an unlabelled description rests on a reading.

This is the section to read twice if you are designing anything that depends on a control — because the answer is not *"at the API"* and it is not *"at the tool"*. It is **both, and they are different moments, with different strengths**, and one of the two is not guaranteed to be present at all.

Four governance objects are read on the execution path. They are the trust plane's values, and the execution plane only ever reads them:

object	what it contributes	who sets it
role envelope	the bounds — five constraint dimensions	the role's owner, in the trust plane
posture	how much of those bounds the agent may use unsupervised	the posture ladder; effective value is the <i>tightest</i> of the agent's own, its unit's ceiling, and what the objective selected
trust chain	where its authority came from, and whether it is still valid	established once, revoked from the trust plane
kill-switch state	whether it may run at all	an operator, in an emergency

A key property of the envelope axis, and it is the one designs get wrong: **bounds compose by intersection, never by union**. Adding an envelope narrows; it never widens. A delegate cannot hold more than the role defining it holds. If you are relying on an escalation path that *adds* permission, there

is no such path.

And the adjudication itself resolves into **exactly four zones** — there is no fifth, and nothing "advisory" among them:

zone	what happens
auto_approved	runs, and is recorded
flagged	runs, and is highlighted for later review
held	stops and waits for a person
blocked	refused, with a reason

The path, with the enforcement points marked

Read this as the shape of one action, from your call to the record. The thing to notice is the vertical position of each box relative to the line marked *side effect*.

⚠ **The boxes are documented; their arrangement is this chapter's reading of them.** Each layer and each gate is stated somewhere reachable — the runtime census below is the best-evidenced part of this page — but the *order* assembled here is a model of how an action passes through them, **not a trace anyone ran**. Nothing you can call returns the sequence. Read the vertical positions as a design claim to reason from rather than as a measurement, and expect a refusal to tell you that you were kept out rather than which box did it.



Everything above the side-effect line is **prevention**. Everything below it is **containment and evidence**. Which side a given verdict lands on is decided by your runtime, not by your policy — and that is the next section, because it is the single most consequential fact on this page.

The pre-execution gate is runtime-dependent, and the default does not have one

Aegis registers **six** agent runtimes and they do **not** have the same enforcement properties. Exactly one carries a genuine per-tool pre-dispatch gate: it can return a refusal *before* the tool runs, and it fails closed when the check itself is unavailable. Three declare no pre-execution gate at all. Two declare a

conditional path that attaches only when a further switch is turned on.

The one with the real gate is `claude_agent_sdk`, and it is not the shipped default. The default is `kaizen_native`, whose conditional gate is off unless the deployment enables it — so **in a stock deployment there is no pre-execution tool gate at all**, and what remains is the event boundary below the line. The platform does not hide this: it emits a warning at boot naming the condition — the default runtime has no pre-execution gate, and agent rows with no explicit runtime execute with no control adjudicating an action before its side effect. A deployment that requires the gate can be configured to **refuse to start** without it rather than warn, which converts the property from aspirational into structural.

⚠ **This is a deployment fact, not an SDK one, and your integration cannot change it.** You can read readiness for the *envelope* half of the question at `api:GET /api/v1/governance/envelope-coverage`; whether a pre-execution gate is attached is answered by the deployment's own configuration and its boot log, not by any route in this client. **UNVERIFIED** from the SDK surface: which configuration your deployment is running. Ask before you build a claim on it.

And the event boundary is not a substitute for it. A verdict decided from a tool-use event is a *record*, not a control — the honest statement is that it stops the run, withholds the action from everything downstream, and tells you accurately what was attempted and refused. On every runtime but the one with the real gate, the first side effect may already have landed by the time the verdict exists. That is not a bug in the adjudication, which is correct and fails closed; it is *when* the adjudication is consulted relative to the effect.

🚫 **Do not design around a refusal that happens before the effect unless you have confirmed you are on the runtime that provides it.** The failure this prevents is expensive and silent: a control you believe is preventive, which is in fact a very good record of something that already happened.

Where the strongest layer actually is

⚠ **Same basis as the section above.** Which layer holds is argued from the platform's description of its own behaviour and from the custody property you can reason about from the client; it is not a trace of an action being stopped.

Pre-execution interception is the *weakest* layer here, which is counter-intuitive enough to be worth stating plainly. The layers that hold on **every** runtime are:

1. **Credential custody.** The agent never holds the secret it acts with. A compromised agent — prompt-injected, jailbroken, or simply wrong — still cannot read the credential, so the worst it can do is *make requests*, and requests are adjudicated. This does not depend on any interception being correct, which is why it is the layer to lead with.
2. **Admission at the boundary.** Identity, organisation, role and permission decide who may call what at all.
3. **The kill switch.** Four scopes — a single agent, one person's work, a class of workflow, or the whole estate — in one of two modes: *hard stop* terminates in-flight work now, *drain* refuses new work and lets in-flight settle inside a bounded window before force-stopping stragglers. It is checked at

several independent execution surfaces, fails closed if the state cannot be read, and **does not expire on its own** — clearing it is an explicit act.

4. **Envelope and posture adjudication**, which computes correctly and fails closed, subject to the timing question above.
5. **The audit chain and revocation**, which bound what you can prove and what you can stop after the fact — 06.4 owns that axis.

Two of these fail *toward* refusal in ways worth knowing, because they change what your integration sees: the kill-switch gate answering "gated" when it cannot read its own state, and a never-delegated check that raises being treated as *held* rather than as permission.

What a verdict does to work in flight

△ This is the least observable claim in the chapter, and the one most likely to shape a design wrongly, so read its basis before its content. Nothing here is derivable from the SDK surface and nothing here was measured: it is the platform's description of its own verdict semantics, and in particular the claim that a side effect already landed is *not undone* is a statement about an event boundary you cannot see from your side. Design as though the boundary is where the platform says it is — and, where the distinction matters to you, confirm it against your own deployment rather than against this page.

Knowing where a bound is consulted is half the question. The other half is what happens to work that is already running when the answer comes back unfavourable.

A **held** action genuinely suspends and waits. It is not logged-and-continued. The waiting coroutine blocks until a person answers, and on approval **the same action object is re-emitted downstream** — so approval *resumes* the action rather than replaying it. If no queue is available to hold in, the hold is treated as a **denial**, on the explicit reasoning that a hold with nowhere to go is a denial and not an approval. That is fail-closed, and it is the right default: design your integration so a missing queue can never be read as consent.

A **blocked** action is refused with a reason, and nothing downstream proceeds. The reason travels with the verdict; surface it rather than collapsing it to a boolean, because it is the only thing that tells an operator which bound was hit.

A **never-delegated action is held regardless of posture**. The seven-action set is checked on the surfaces that speak *business actions*, and where a verdict has already been computed as approve, the never-delegated check **overrides it to held**. Two entries make the set self-protecting: changing the constraints and changing the governance are themselves never delegated, so no agent can reach the machinery that bounds it. **UNVERIFIED**: whether every path to a tool boundary passes one of those surfaces. If one does not, an action from that set could reach a boundary that cannot recognise it. That question is open, and it is the right one to ask of any deployment you are relying on.

And the honest limit, which is the whole point of this section: on every runtime except the one with a genuine pre-dispatch gate, a **held** verdict stops the **run** — everything downstream of the action does not happen — but it does not undo a side effect that already landed. Read that as: *containment of the*

run, and a durable record of the attempt, which is real value. Do not read it as prevention. Where a run is long and autonomous, a hold is a hard stop near the edge of its envelope, which is why 02.5's advice to decompose work so the boundary-crossing part is its own small request is a design constraint rather than a style preference.

What a running execution is

The platform tracks execution at three different resolutions, and they are not views of one record:

resolution	what you read	what it is good for
objective	<code>api:GET /api/v1/objectives/{}/progress</code>	how far along, what it has cost, which constraint dimensions are under pressure
task graph	<code>api:GET /api/v1/objectives/{}/task-graph</code>	what the work is broken into, and which node is stuck
session	<code>api:GET /api/v1/sessions/{}</code>	what an agent did: messages, artifacts, subagents

Not everything that runs is an addressable record, and this is worth knowing before you build a poller. `api:POST /api/v1/agents/{}/execute` is a *synchronous* chat completion: it either returns a result or raises. Agent executions are not persisted as individually-addressable objects, so the client's `get_execution`, `list_executions` and `cancel_execution` have nothing to address. 🚫 **open defect** — those three methods are declared by the client, and **each carries a warning in its own documentation saying the call answers 404**. Note what that rests on: the client's account of itself, not a call this chapter made. One call would settle it, and a docstring is not a measurement — this book's own gate says an anchor proves a surface exists and never that it behaves. Whether the three should be present at all is a fair question; the answer that matters to you is that **a method existing is not evidence a route exists** — see 06.6, where that distinction decides how much you may trust any anchor in this book, including this page's.

That leaves three things you can stop, and they are different things:

- the **objective** — `api:POST /api/v1/objectives/{}/stop`, described as terminal and not resumable;
- a **work-unit execution** — cancel takes a `run_id`, and only works while an autonomous execution is still running;
- a **session** — pause, resume or end at `api:POST /api/v1/sessions/{}/pause` and its siblings. Prefer pausing a session to revoking an agent: revocation cascades and does not undo.

Watching it: three streams, three vocabularies

Everything long-running is observable over Server-Sent Events, and there is more than one stream. They do not share an event vocabulary, so one consumer cannot serve all three:

stream	route	events
objective progress	api:GET /api/v1/objectives/{}/progress/stream	execution_started , step_started , step_completed , progress_update , execution_completed , execution_error , heartbeat
session	api:GET /api/v1/sessions/{}/stream	progress_update , subagent_spawned , cost_update , objective_completed , objective_failed , escalation_created , agent_auto_claimed , agent_auto_executing , cancelled , error
agent execution	api:POST /api/v1/agents/{}/execute/stream	start , content , done , error
escalations	api:GET /api/v1/agent-escalations/stream	pending human decisions

Read the right-hand column as three different *granularities*, not three spellings of one: the agent stream emits model tokens, the objective stream emits plan steps, and the session stream emits what the agent is doing. Pick by the question you are answering.

Two mechanical traps, both of which cost an afternoon:

🚫 **The objective progress stream authenticates by query parameter, and only by query parameter.** It takes a `?token=`, because a browser `EventSource` cannot send an `Authorization` header — and it does **not** fall back to the bearer header the rest of this client uses. The client defaults the parameter to its configured API key when you omit it, which is convenient and worth knowing when you are debugging why the same credential works everywhere else.

🚫 **The session stream has no server-side filter.** Its `event_types` argument is applied client-side, so every event crosses the wire and is discarded after arrival. If you are budgeting bandwidth on a high-volume session, filtering is not buying you anything.

For anything you need to keep rather than react to, poll the progress route. The streams are for liveness; the progress and task-graph reads are for state.

Where a human gets asked to decide

There are **six** places work can stop for a person, and they are six different records with six different resolutions. A design that assumes one queue is the most common integration mistake in this platform.

what is waiting	the record	where you read it	how it is answered
the agent reached a decision point	an objective decision	api:GET /api/v1/decisions/pending	api:POST /api/v1/decisions/{}/decide , or from the objective at api:POST /api/v1/objectives/{}/decide
an action is held by the verification gradient	an approval request	api:GET /api/v1/approvals/pending	api:POST /api/v1/approvals/{}/approve (or reject, or approve-modified)
the agent is out of its depth	an escalation	api:GET /api/v1/agent-escalations/pending	api:POST /api/v1/agent-escalations/{}/resolve
a step was routed to people	a pseudo request	api:GET /api/v1/pseudo-requests	claim at api:POST /api/v1/pseudo-requests/{}/claim , then respond
a finished request needs sign-off	an inbox item	api:GET /api/v1/agentic/inbox	api:POST /api/v1/agentic/requests/{}/validate — approve, reject or revise
the organisation itself is changing	a change request	api:GET /api/v1/change-requests	see 02.6

An empty approvals queue means nothing about the other five. That is stated in 02.6 from the operator's side; here is the structural reason — each is a different table with its own inbox route, and nothing in the platform aggregates them.

Two of these have admission rules that a design can run into late:

- **A decision is addressed to a named person, not to whoever holds write authority.** The pending list is scoped server-side to the authenticated caller's own user id and organisation, with no parameter to list or override it — so "show me all pending decisions" is not a query the API answers. Resolving one requires being the assignee or an organisation administrator; a correctly-credentialed user resolving someone else's still gets refused. And writes on that surface **reject an API key outright**, because the platform authorises them from the caller's role and an API-key principal carries no role.
- **A pseudo request is claimed, and claiming is how you avoid duplicate work.** Two people answering the same unclaimed request is the ordinary failure, and a claimed request held by someone who has moved on is invisible to everyone else.

What happens to the work while it waits

The structural answer, and it is not what most people assume: **the work holds a state; it does not occupy a queue.** An objective at `decision` is not running in the background waiting to be unblocked. It has stopped, and the direction it resumes in is decided by the option that is chosen:

- an option resolving to *proceed* completes the objective;
- one resolving to *reject* cancels it;
- *revise* or *more_analysis* returns it to `executing`.

So "what happens while it waits" is the honest answer **nothing** — which is exactly why an unanswered hold is expensive and why the deadline fields below are weaker than they look.

Three caveats, each of which changes what you build:

⚠ **A recorded deadline is not an enforced one.** A decision carries an `expires_at`. The client's own documentation says plainly that this is a deadline the platform records, not a commitment to act at that instant, and that a past `expires_at` on a still-pending decision should be read as *not yet swept* — re-read the status rather than assuming the platform resolved it. **Do not build a timeout on it.** If you need work to move when a human does not answer, that is a scheduler you own.

Escalations carry an `expiresAt` on the same terms, alongside status, resolution and reasoning fields that stay empty until somebody answers. **UNVERIFIED:** what the platform actually does when either deadline passes. Reading those fields tells you the deadline is not a promise; it does not tell you what the sweeper does, and this book will not guess.

And the caution applies in reverse when you are *building* the queue: a decision your code reads and never answers is a piece of work that stops permanently. Whatever you integrate, integrate the notification as well as the read.

Who can carry the work

Not every agent is the same kind of thing, and the differences decide what you can ask of one. Two independent axes — what the agent *is* (`sdk:aegis_sdk.AgentType`) and what role it plays (`sdk:aegis_sdk.AgentSubtype`) — and a third field that is not an agent axis at all.

type	what it is	how it runs
<code>chat</code> , <code>task</code> , <code>pipeline</code> , <code>custom</code>	the four you can create	<code>api:POST /api/v1/agents/{}/execute</code> , or the stream above
<code>shadow</code>	a role agent — the delegate attached to a seat	assigned objectives (02.7)
<code>pseudo</code>	a human in the loop, behind an agent-shaped interface	routes a task to people
<code>tool</code>	a shared capability other applications invoke	governance-gated invocation
<code>esa</code>	minted by the platform's own machinery	—
<code>unknown</code>	a sentinel for a type a future release adds	—

Only the first four are creatable. `sdk:aegis_sdk.core.AgentsModule` will not create a shadow, pseudo, tool or esa agent: those are minted server-side, and the read enum accepts them so that listing an organisation that has them does not fail. `unknown` exists so a future value arrives as a sentinel rather than a validation error — a design that branches exhaustively on this enum should have a default branch.

Those types differ in more than name:

- **A tool agent is invoked, not executed.** `api:POST /api/v1/tool-agents/{}/invoke` runs it *through governance* — trust, posture and budget are all checked, and a refusal is a governance refusal rather than an error. It can run a registered built-in tool with no model round-trip at all, in which case the result reports that dispatch path and a zero cost. Two behaviours worth designing around: an unregistered tool name is refused outright rather than answered by the model, and declaring a governance category that was never delegated to the agent is refused regardless of the agent's posture. The client raises before the request on a malformed tool call, deliberately — see 06.6, because that is the clearest example in the package of a boundary being held client-side.
- **A task agent exists for agent-to-agent delegation** and is testable before you rely on it, at `api:POST /api/v1/task-agents/{}/test`. It is not a delegate: it carries no role envelope and its posture ceiling is an explicit field.
- **A pseudo agent is the human-in-the-loop work unit.** It presents the same interface as an autonomous agent; behind it, requests route to human operators over the channels you configure, at `api:POST /api/v1/work-units/pseudo/{}/route`.

Pools hold role slots, not people. Membership is keyed by role, so whoever occupies the role is in the pool and a re-org moves the membership with the seat (`sdk:aegis_sdk.modules.AgentPoolsModule` for the registry, and the operational claiming and load distribution surface alongside it). If you were about to build a roster, build it as a role mapping instead or every re-org becomes a data migration.

Lastly: **unit_type is not agent_type**. `sdk:aegis_sdk.UnitType` — atomic and composite — is the work-unit classification, and it travels in a field of its own. A previous release of this client sent `agent_type="atomic"` and the server rejected it; the two vocabularies look adjacent and are unrelated.

Subagents exist at runtime but not as a request. Sessions list what an agent spawned at `api:GET /api/v1/sessions/{}/subagents`, and spawns arrive on the session stream as `subagent_spawned` — but there is no request-side route to trigger one. If your design needs to direct a subagent, it is directed by the agent, not by you.

Which model each agent is running is ON the stream, not behind a second call. Every agent-activity event carries `data.model` — the resolved model of the agent that emitted it, read through `sdk:aegis_sdk.execution.SessionsModule.stream_events`. Agents within one session do not share a model, so a multi-agent view has to read it per event rather than resolve it once per session. It is a string and is `""` when unresolved: treat empty as *unknown*, never as a default, because collapsing the two is how a view ends up attributing one agent's model to another.

The output end, and where it attaches

Artifacts are the product of all this, and three rules about them are structural rather than incidental.

An artifact attaches to a request, in a workspace. The request must be in your organisation *and* attached to a workspace; if it is not, the upload is refused with a conflict.

`sdk:aegis_sdk.execution.ArtifactsModule` is the surface.

🚫 **You cannot choose the workspace, and that is deliberate.** There is no `workspace_id` argument on the upload route, and the server would ignore it if you sent one. The workspace is derived from the request, and the workspace is what decides how highly the upload is classified. Letting a caller name it would let them choose whose classification mark the artifact carries. Design your upload path around the request you are attaching to, not around a folder you want to file it in.

A new version inherits its predecessor's classification as a floor. Superseding at `api:POST /api/v1/artifacts/{}/supersede` takes the next version number and cannot lower the mark. Version history is at `api:GET /api/v1/artifacts/{}/versions`, and there is one behaviour that looks like corruption and is not: **each version is clearance-checked individually**, so a version you may not see is simply omitted and the numbers arrive with a gap — `1, 2, 4`. The values are as stored, not renumbered. A gap is information, not a fault.

Reads are clearance-checked, and the refusals are asymmetric on purpose. A clearance you do not reach is refused with a 403; another organisation's artifact is reported as **not found**, never as forbidden — so a 404 does not confirm that something exists. Downloads return raw bytes rather than parsed JSON, and can fail in two distinct ways that a retry loop should treat differently: the record exists but its bytes are unreachable, which is a service condition, versus a file too large to serve, which a retry will never fix.

Where the next page goes

This chapter described what is guaranteed by the shape of the system. Not all of it is. The next page is the one to read before you commit to a design: which of these behaviours are enforced by a boundary you can rely on, which are only conventional, where your integration attaches, and the failure modes that cost the most to find late.

Next: 06.6 — Designing against the platform

06.6 — Designing against the platform

This is the last chapter of part 06 and the only one you are meant to act on directly. The first five describe a system. This one is where that description turns into a decision about your integration.

Three questions, in the order they cost you money if you get them wrong:

1. **Where does my integration attach?** There are more surfaces than most people expect, and picking the wrong one is usually discovered late.
2. **What may I assume?** Some of what looks like a guarantee is a convention the client maintains for you.
3. **What will surprise me?** A short list of the failures that are expensive to find late, each with the symptom you will actually see.

If you read one chapter of this part before building, read this one and let it send you backwards. It is written to be usable that way.

Where your integration attaches

Start from what your code is *doing*, not from which module name looks likely. The six intents below are mutually exclusive in practice, and each lands on a surface with a different shape of guarantee — which is the point of drawing them apart.

What is your integration actually doing?

— putting work in	→ Objective – decomposes into requests 02.5, 06.5 Work unit – a versioned definition you execute, not a job 06.5
— watching work run	→ Progress stream – plan-step granularity, poll for state 06.5 Session stream – what the agent is doing, different vocabulary 06.5 Audit and reasoning traces – after the fact 02.6, 04.5
— being the human who answers	→ Six separate queues. Nothing aggregates them. 06.5, 02.6
— adding a capability	→ Skills, tools, pipelines, agents part 03 Tool agent – invoked through governance, not executed 06.5
— changing the organisation	→ Org, units, roles, envelopes 02.2, 02.3 Change requests – approved is not applied 02.6
— taking output out	→ Artifacts – clearance-checked, classified by the workspace, never by the caller 06.5

Every leaf in that diagram is expanded somewhere in this book, and each names where. The two worth reading twice before you build anything: **the human-decision surfaces are six queues with no aggregator**, and **the artifact's classification is decided by the workspace you attach to, not by the argument you pass**.

The client-side view of this map is `sdk:aegis_sdk.AgenticOSClient` : one object, configured by `sdk:aegis_sdk.ClientConfig` , carrying every module as an attribute. If you are orienting yourself in a REPL, that object is the index.

What you can reach, and what you cannot

There is a hard line between the system's *state* and its *configuration*, and it is not the same line as "what has a route". Your integration sits on one side of it, and the most important properties in this book are on the other.

you hold: a client · a credential · a base URL

REACHABLE – anything behind an HTTP route your credential admits

work	objectives · requests · work units · sessions
output	artifacts · downloads · version history
humans	decisions · approvals · escalations · pseudo requests inbox · change requests
trust	chains · delegations · postures · revocation
evidence	audit log · trust audit · compliance verify and export
emergency	kill switch · emergency bypass
structure	organisations · units · roles · envelopes · teams
readiness	envelope coverage · hydration status · explain-access

the credential boundary

refused, not invisible – you learn you were kept out,
not that the surface is absent

NOT REACHABLE – configuration, and the platform's own internals

the deployment's posture

which agent runtime is in force
whether a pre-execution tool gate is attached
whether outbound LLM content is classified
the encryption and signing posture
which environments are guarded

the platform's internals

which component refused you, and at which layer
whether a control that exists is enforcing or merely present
any capability with no route in front of it

Each reachable band above is a real surface, not a theme:

`sdk:aegis_sdk.modules.WorkObjectivesModule` for work,
`sdk:aegis_sdk.execution.ArtifactsModule` for output, `sdk:aegis_sdk.ApprovalsModule` for the human queues, `sdk:aegis_sdk.trust.ChainsModule` for authority, `api:POST /api/v1/kill-switch/activate` for the emergency stop, and `sdk:aegis_sdk.standup.OrganizationsModule` for the structure. Everything the client carries about the deployment is in `sdk:aegis_sdk.ClientConfig` — a base URL, a credential, a timeout — and nothing else. That short list is the whole of what you decide about the system before you call it, and it is worth contrasting with the lower half of the diagram.

One caution about that lower half: it is an argument from absence. Everything in the *not reachable* band is there because no route in this client reaches it — and that cannot distinguish *the capability does not exist* from *this client does not expose it*. From where you stand those two are indistinguishable, and the difference matters depending on what you do next: if you were planning to ask the platform team for access, the second reading is the one that has an answer.

Two consequences follow, and they are the design content of this section.

Your integration inherits the enforcement posture; it does not set it. The runtime choice, the presence of a pre-execution tool gate, the egress classification policy and the cryptographic posture are all **deployment configuration**. Nothing in this client writes them. That means the single most consequential question about your system — *does a bound get consulted before the effect, or after it?* — is answered by whichever team configured the deployment, and is not answerable from your code. 06.5 states the mechanism and the default; what it cannot tell you is what **your** deployment runs.

Treat that as a question to ask, not an assumption to carry:

- Which agent runtime is in force, and does it carry a pre-execution tool gate?
- Is the deployment configured to refuse to start without one, or only to warn?
- Is outbound content classification actually live, and what classifier is installed?
- Which credential types does each surface in my path admit?

The one readiness read you *can* make is `api:GET /api/v1/governance/envelope-coverage` — it answers not *"do bounds exist"* but *"would they stop anything"* — and 06.4 covers how to read it honestly. Everything else on that list is a conversation.

A refusal tells you that you were kept out; it does not tell you by what. The credential boundary is a wall you can see, which is the good case. The enforcement layers inside a run are not: when work stops, nothing in the response distinguishes a posture ceiling from an envelope dimension from a never-delegated override from a kill switch. *Derived, not observed here*: that opacity is inherited from what the client documents about its refusals, not from a refusal this chapter provoked — treat it as the model to design against and let your own first held run confirm it. The record layer knows, and 06.4 has the reads that ask it. Build the habit of asking *which* bound, rather than treating the first plausible one as the answer.

Guaranteed by the boundary, or only conventional?

This is the distinction this chapter exists to draw, and it is not visible from the call signatures. Both columns below behave identically when everything is working. They diverge on the day something is not.

Guaranteed means: the thing that holds the enforcement is outside your process. You cannot get around it by calling the route yourself, and it is not something you could neglect to configure.

Conventional means: the behaviour is maintained by this client, or by a field the platform records without acting on. It is real, it is useful, and a design that depends on it is depending on a contract that the server does not enforce on your behalf.

There is a third answer, and it is the one that costs the most to get wrong: **runtime-dependent**. The behaviour is guaranteed *on some runtimes and not on others*, decided by deployment configuration you cannot see from here.

behaviour	which	why
Clearance on artifact reads	guaranteed	the server refuses a read your clearance does not reach; the client cannot bypass it
Another organisation's artifact reads as <i>not found</i>	guaranteed	a deliberate refusal shape, so a 404 never confirms existence
Classification of an uploaded artifact	guaranteed	derived from the request's workspace; there is no argument to set it
Tenancy of anything you touch	guaranteed	derived from your credential, checked at more than one layer
An API key is refused on persona-gated routes	guaranteed	the key is built with no role and no personas; the refusal is the design
Governance on a tool-agent invocation	guaranteed	trust, posture and budget are checked server-side; fail-closed if the governance service is unavailable
A <code>held</code> verdict stopping the run	guaranteed	it suspends and waits for a person; with no queue to hold in it fails closed to a denial
A <code>blocked</code> verdict refusing the action	guaranteed	decided before anything downstream proceeds
A verdict landing <i>before</i> the side effect	runtime-dependent	only one runtime carries a genuine pre-execution tool gate, and it is not the shipped default
A bound preventing an effect that already happened	impossible	below the event boundary a verdict is a record; it stops the run, not the effect
Outbound LLM content being classified	runtime-dependent	the mechanism ships; the policy is the deployment's, and the default is a pass-through
Trust-chain and audit records	guaranteed	see 06.4
The audit chain's integrity check	guaranteed	<code>api:GET /api/v1/compliance/audit/verify</code> is the operation that makes the trail evidence
Objective status	conventional	<code>sdk:aegis_sdk.ObjectiveStatus</code> is a <i>lossy</i> projection of nine wire states onto six values

behaviour	which	why
Quota adequacy	conventional	the client's check reads a cached quota list and does the arithmetic locally
Pagination	conventional	some endpoints do not paginate at all and still return the envelope
A <code>decision</code> 's <code>expires_at</code>	conventional	recorded, not enforced — a past deadline on a pending decision means unswept
Stream filtering	conventional	the session stream's <code>event_types</code> argument is applied client-side
Priority	conventional	an integer here, a four-value tier on the wire
Error signalling	partly conventional	some refusals arrive as <code>success: false</code> inside a <code>200</code>

The two that surprise people most, in full

Status is a projection, and it collapses the two states you most need to tell apart.

`sdk:aegis_sdk.ObjectiveStatus` maps `plan_review` and `confirmed` and `pending` onto one value, and maps `decision` — *stopped, waiting for a person* — onto `IN_PROGRESS`. An operator view built on the typed model cannot show that anything is waiting on a human. Read the status off `api:GET /api/v1/objectives/{}/progress`, which returns the wire string. Full table in 06.5.

The quota check is a local calculation, and its unknown-input branch fails open.

`sdk:aegis_sdk.revenue.QuotasModule` reads the organisation's quotas at `api:GET /api/v1/features/limits` and caches them; the check method compares `remaining` against the amount you name and answers yes or no from that cached copy. Two consequences you should design around rather than discover:

- 🚫 **It is a check-then-act with no atomicity.** Another caller can consume the headroom between your check and your create. The check tells you the answer as of the last read, not that the next write will succeed — so treat a pass as permission to *try*, and handle the refusal at the write.
- 🚫 **An unrecognised resource type answers "allowed".** If the type you pass is not a quota the platform tracks, the check returns allowed with a limit and remaining of zero. That is the client's documented branch, not a bug in your call — and a documented fail-open is still a fail-open. A design that gates on that answer is safe exactly as long as the spelling is right and nothing about the response tells you when it is wrong.

Whether the platform *additionally* enforces a quota when you actually attempt the write is **UNVERIFIED** here. The client exposes a read of the limits and a local judgement on them; it does not expose the refusal. Do not build a capacity plan on the assumption either way without testing it against your deployment.

Pagination is the quiet one. `sdk:aegis_sdk.PaginatedResponse` is a clean envelope — `items`, `total`, `page`, `page_size`, `has_next` — and some endpoints that return it do not paginate at all. On those, `total` is the length of the page you were handed rather than a count of anything, `page` is always 1, and `has_next` is a heuristic comparing the number of records against the page size. A consumer that loops on `has_next` over such an endpoint will either miss records or repeat them, depending on which way the boundary falls. Check that the endpoint actually accepts a page parameter before you write a pagination loop around it. This is the same family as the status collapse: **the shape of the response is not a statement about the endpoint's behaviour.**

The failure modes that cost the most to find late

△ The first four turn on where enforcement actually happens, and that is the least observable thing in this part. They are written from the platform's account of its own decision path rather than from observed refusals, so treat each as a hypothesis to test against your own deployment rather than as a measured fact. The symptom column is the half you can check directly.

Each of these is a design that looks right, works in a demo, and fails once real volume or a real refusal arrives. The symptom is what you will actually observe.

The first four are about **where enforcement happens**, and they lead the list because they are the ones that cannot be discovered by testing harder. A control at the wrong layer behaves identically to a correct one right up until the question is asked by someone who will not accept a demonstration.

1. Assuming enforcement sits at the layer you designed against. *Symptom:* a control that refuses reliably in a demo and does not refuse in production, or the reverse; a test that proves your integration handles a refusal on a path where no refusal is ever produced. *Diagnosis:* enforcement here is layered, and the layers do not live in the same place. Admission at the boundary is a different layer from adjudication inside the run, which is a different layer again from the tool gate — 06.5 maps where each is consulted relative to the effect. Before you depend on a control, establish *which* layer produces it, and confirm that layer is on the path your work actually takes. A refusal you can reproduce proves a layer exists; it does not prove it is the layer you think it is.

2. Assuming a `held` verdict prevented the action. *Symptom:* a run that stopped, a hold that was answered, and an external system that had already been changed. Or an audit that reads clean on the record and surprising in the world. *Diagnosis:* this is the most expensive misreading available on this platform. A hold stops the run and everything downstream of it. On every runtime except the one with a genuine pre-dispatch gate, the first side effect may already have landed before the verdict exists. The verdict is **containment and evidence, not prevention** — real value, and a different claim. If your design needs prevention, it needs the runtime that provides it, and you need to have confirmed you have it rather than assumed it.

3. Assuming the deployment has a pre-execution gate. *Symptom:* a governance claim you cannot support, discovered in the room where you have to support it. *Diagnosis:* the shipped default runtime has no pre-execution tool gate, and the platform says so out loud at boot rather than leaving it to be discovered. Of the six runtimes, one carries the gate. Whether *your* deployment is on it is a

configuration fact, not a code fact, and no route in this client answers it. Ask; and if the answer matters to what you are promising, have the deployment configured to refuse to start without the gate rather than merely warn about it.

4. Assuming outbound content is classified. *Symptom:* a design that depends on no unclassified content leaving the process, and content leaving unclassified. *Diagnosis:* the classification mechanism ships, is fail-closed inside a guarded environment, and covers the LLM calls your own process makes — but in the canonical default no environment is guarded and no classifier is installed, so it is a no-op pass-through. Both halves are the deployment's to supply. It also cannot cover a runtime that spawns a subprocess, by construction: 06.2 has that boundary and it is the reason some designs must disable those runtimes rather than police them.

5. Building control flow on the typed status enums. *Symptom:* work that is stopped on a human reads as running, so nothing escalates and nothing reminds anyone. You find out when a request has been sitting for a week. *Diagnosis:* read the wire status from a raw-dict route and keep your own state machine. Treat the enums as filters for coarse queries.

6. Treating a 403 as a scope problem. *Symptom:* a refusal whose message names personas you do not have, on a route that works for a colleague using a session. You go and inspect the key's scopes and find nothing wrong — because nothing there is wrong. *Diagnosis:* on persona-gated routes the scopes are never consulted. Retry the identical call with a user session (`api:POST /api/v1/auth/login` , then the same call). If the session succeeds, no amount of scope editing will move it, and widening the key leaves you holding a stronger credential for no benefit. Confirm what the credential carries at `api:GET /api/v1/auth/me` .

7. One queue where there are several. *Symptom:* the approvals queue is empty, so the operator reports all clear, while a pending decision and an escalated request sit unanswered for weeks. *Diagnosis:* there is no aggregating endpoint — each kind of waiting is its own record with its own inbox route. If you build one check, build the sweep across `api:GET /api/v1/approvals/pending` , `api:GET /api/v1/decisions/pending` , `api:GET /api/v1/agent-escalations/pending` , `api:GET /api/v1/pseudo-requests` , `api:GET /api/v1/agentic/inbox` , `api:GET /api/v1/posture/pending-approvals` and `api:GET /api/v1/change-requests` . 02.6 has the same list from the operator's side, and both stop short of claiming it is exhaustive: an organisation can wait on things this book never enumerated, and nothing in the platform will tell you that you missed one.

8. Assuming a hold resumes on its own. *Symptom:* a decision nobody answered, and work that never completed. No error anywhere. *Diagnosis:* the work holds a state; it does not occupy a queue. A decision's deadline is recorded, not enforced. If nobody answers, nothing happens — so the scheduler that nudges a human is yours to write, not the platform's to provide.

9. Adopting a workflow on a method that is declared but not served. *Symptom:* a well-shaped method that answers 404 every time — or, worse, one you wire into a retry path and never notice. *Diagnosis:* this client declares operations the server does not serve. Whether they should be declared is a fair question; the useful move is to call each method your design depends on **once**, against a real deployment, before you build on it. The known cases are marked where you would look for them — `.. warning:` blocks in the package's own method docstrings, which is exactly what `help()` on any method in this SDK shows you — and the family is recognisable: whole-object **update**, **submit**, and **list**

operations on resources whose real lifecycle is driven by a background task rather than by a request. If a method's docstring says a call 404s, believe it, and design the workflow around the route it names instead.

10. Reading `approve` as `applied`. *Symptom:* a change that was approved and never took effect; a system where the review says done and the state says otherwise. *Diagnosis:* approve and apply are separate steps on the change-request surface. Same shape as draft envelopes and unsubmitted objectives — describing and effecting are always two operations here.

11. Assuming you can name an artifact's classification, or file it where you want. *Symptom:* an upload accepted with an unexpected classification, or refused because the request is not attached to a workspace. *Diagnosis:* the workspace comes from the request and there is no argument for it. Choose the request you attach to; it chooses everything else.

12. Reading a version gap as corruption. *Symptom:* `api:GET /api/v1/artifacts/{}/versions` returns `1, 2, 4`. *Diagnosis:* each version is clearance-checked individually, so a version you may not see is omitted and the numbers are as stored, not renumbered. A gap is information about your clearance, not a fault.

13. Treating a failure signal as the only failure signal. *Symptom:* a write that reports success and did not happen; or the opposite — you handle exceptions and miss the refusals that arrive inside `200` bodies. *Diagnosis:* several operations here report a refusal in the body rather than by raising, so a successful transport is not a successful operation. Read the documented return shape of each write, not only its raise list. `sdk:aegis_sdk.Agentic0SError` and its subclasses carry the status when one IS raised — `409` now arrives as `sdk:aegis_sdk.ConflictError` — but the refusals this entry is about never reach your `except` clauses at all.

14. Depending on a guard this client holds. *Symptom:* a check that works from your code and not from a direct HTTP call — and you discover it when someone else's integration does the thing you thought was refused. *Diagnosis:* the clearest example is tool-agent invocation, where this client raises *before* sending anything if a tool call is malformed, precisely because the server would otherwise answer from the model and report success. That protection is a property of the SDK, not of the platform. If your design's safety rests on it, and anything else in your estate can reach the same route without the SDK, the safety is not where you think it is.

What to do differently, in one list

Design practice that follows from the fourteen above:

- 1. Establish which layer enforces what you are promising.** For each guarantee your design rests on, name the layer that produces it and confirm that layer is on the path your work actually takes. The first four failure modes are all this one question, asked of four different controls, and none of them can be settled by testing harder.
- 2. Branch on wire values, not on client enums.** Where a raw-dict route exposes the platform's own vocabulary, use it for anything that decides behaviour.
- 3. Prefer the stream for liveness and a poll for state.** They answer different questions and the streams do not keep a history.

4. **Treat every declared operation as unverified until you have called it**, and treat every anchor in this book the same way — including the ones on this page. An anchor proves the surface exists in the client you were given; it says nothing about what the server checks when you call it.
5. **Own the states the platform does not hold**. Your integration's timeout, its nudge, its retry and its exhaustion handling are all yours. Nothing in the product schedules them.
6. **Build the sweep before you build any single queue**. Retrofitting an aggregator onto surfaces you integrated one at a time is worse than building the sweep first, and the sweep is the only view an operator can act on.
7. **Verify what you will later have to prove**. The integrity check over the audit chain is worth running on a schedule from day one, because a verification taken after an incident is worth a fraction of a series taken before it (02.6 covers this and is worth the re-read).
8. **Write the check before you need the answer**. Whatever your organisation will one day be asked to demonstrate, build the read for it now — while you still remember why it matters.

What this chapter cannot establish

It draws a line between guaranteed and conventional, and the line is only as sharp as an outside read allows. Three limits, stated rather than implied:

- **"Guaranteed" here means the enforcement is not in your process**. It does not mean the enforcement is correct, complete, or free of defects. A control that is genuinely server-side can still be wrong; 04.1 documents one that is.
- **The conventional column is a list of what this client does, not an exhaustive list of everything the platform does not enforce**. There is a path with no surface anywhere in this book, and a behaviour nothing exposes cannot be classified here at all — it is invisible, not absent.
- **Nothing on this page was verified against a running deployment by writing it**. Every claim derives from the client's own declared behaviour. Where that was not enough, the page says **UNVERIFIED** rather than choosing the reading that makes the sentence tidier.

You have reached the end of part 06. If you arrived here first, the part index maps what to read next; the chapters are ordered as reading dependencies, and each one is a statement about one side of the boundary 06.1 draws.

Next: Part 07 — Deployment architecture

Part 07 — Deployment architecture

Audience: the lead engineer on a partner integration — you are designing against a deployment whose internals you do not operate. You will not run its nodes, hold its cloud credentials, or open its cluster. You will point a client at it over HTTPS, and you will be asked to answer for what it does. This part is the shape of the thing you are attaching to.

What it is. A reference architecture for a production Aegis deployment on **Azure** — that is what this part targets, and naming the cloud is a statement about the reference rather than about your estate. It covers the components one is built from, what each is for, who owns what, and where the boundaries fall; it answers three questions in order — what am I deploying, what do I own, and where does the line sit between me and the platform.

The reference deployment runs on managed Kubernetes in Azure: a managed cluster, a container registry, an ingress edge, one or more application namespaces, an ephemeral job and agent execution namespace, a managed relational database with a vector extension, a cache, object storage, a secret store, and your identity provider. Three of those are specifically Azure's — the managed cluster, the container registry and workload identity federation. The rest are generic: a namespace, a network policy, and a secret referenced at runtime rather than baked in all exist under some name on every platform, and the chapters keep them generic and say so where the distinction matters.

Design intent, not observable — that **your** deployment has this shape. It is the shape the chapters describe, and the deployment you are actually attached to is the one you can check. 07.1 names each of them and says what it is for; its ownership table is the one you will keep coming back to.

What it is not. An inventory of any particular running estate. A given deployment will differ from this reference in detail, and where the two disagree, the one you are attached to is the real one. Every resource name in this part is a placeholder — `<cluster-name>`, `<registry>.azurecr.io`, `<app-namespace>` — because the names belong to the deployment rather than to the architecture.

Nothing here rests on a file you were not given, which is the same rule the rest of the book follows — and it costs this part something real. It describes infrastructure from outside, so much of what it says is how a deployment is *designed* rather than something you can measure from your own machine. The convention for marking that is set out at the foot of this page, and it is worth reading before the chapters.

Where your code attaches

Your process holds a client configured against a base URL you supply, and that base URL is the edge.

`sdk:aegis_sdk.ClientConfig` is where the endpoint, the credential and the timeout policy live, and

`sdk:aegis_sdk.Agentic0SClient` is the object that presents them.

The client half of that is checkable, and worth stating flatly: there is one configured address, and no code path in this client that aims at any other. Nothing else in this part is reachable from your process — not the cluster, not the network, not the database. The other half — that the edge is the deployment's *only* public entry point — is a claim about how the deployment is built rather than something your side can check, and 07.4 is where it is made and marked.

One more thing to expect, so that meeting it later is not a surprise: a deployment **may** offer a second, non-public path into the cluster — a namespace-scoped administrative path its operators use. 07.2 describes it, and marks whether a given deployment provides one as a question to ask up front rather than discover during an incident. It is not a second door for your integration: nothing in the SDK reaches it, and the edge remains the only address your code has.

That gives you one check worth running on day one. A deployment pointed at the wrong cluster, or answering as a principal you did not expect, looks identical to a correct one until you ask: `api:GET /api/v1/auth/me` reports which principal your credential resolves to on the deployment you are actually talking to. Establish that before you debug anything else.

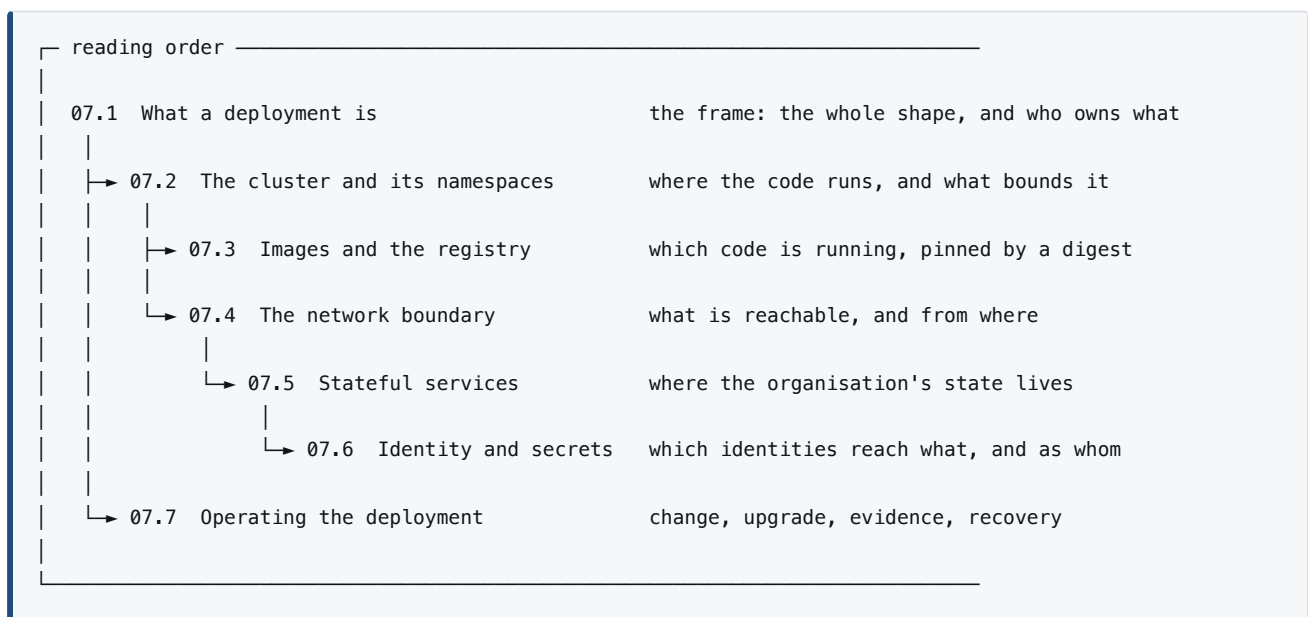
The chapters

#	Chapter	Read it when
07.1	What a deployment is	First. The whole shape in one place, and the ownership table that draws the line between what you own and what the platform operates
07.2	The cluster and its namespaces	You are asking where the processes actually run, or what a namespace does and does not separate
07.3	Images and the registry	You need to know which code is running, or you are planning an upgrade or a rollback
07.4	The network boundary	You are drawing your own security boundary, or someone asks what is reachable from where
07.5	Stateful services	You are reasoning about where the organisation's state lives, or what a backup and a restore are worth
07.6	Identity and secrets	You hold a credential for your own integration, or you are auditing how anything authenticates

#	Chapter	Read it when
07.7	Operating the deployment	You are about to go to production, and want the checklist and the questions to put to your operator

How the seven chapters relate

They are not seven topics. They are one deployment described in layers — where the code runs, what encloses it, and what it holds — and knowing which layer a question belongs to is most of the work of finding the answer.



Read it top to bottom. Each branch is a **reading dependency**, not traffic — what has to be understood before the next chapter lands. The chapters nest where one chapter's subject is the context for the next, and four things follow from the shape:

- **07.1 is the frame.** Every chapter below it is a detail of it, and the ownership table there is what gives each later boundary statement its meaning: a boundary you do not control is a different kind of fact from one you do.
- **07.2 and 07.3 are where the code runs, and which code it is.** 07.4 reads after both: a boundary drawn without knowing what it separates is a diagram, not a design.
- **07.4 is the hinge.** 07.5 and 07.6 both hang off it — where the state lives, and which identities may reach what. The network decides who may *try*; those two chapters are about what is there and who succeeds.
- **07.7 is the only chapter you act on over time**, and it assumes the rest: a rollout is a digest change from 07.3, and a restore is 07.5's subject. It is also where the questions the earlier chapters told you to ask are collected.

Where this part sits relative to the others

Part 02 is the working reference: it shows you how to stand an organisation up and run work through it, and this part does not repeat any of that. Part 04 is the API surface — how you call a deployment over HTTP, what a credential is, what an error looks like. Part 06 is the platform's architecture: what the system *is*, and why its boundaries fall where they do.

This part is the deployment those three run on. Part 06 describes a system you never see the inside of; this part describes the machine that holds it — the cluster, the network, the state, the identities, and the operations that change them over time.

The practical test is the question you are asking. *"How do I establish a trust chain?"* is part 02. *"Why did this call come back refused, and what does this error mean?"* is part 04. *"What is actually running, who can change it, and what happens to it during an upgrade?"* is this part — and it is the one your security review will ask, because it is the one that decides what you can promise about the deployment you do not operate.

What a reference architecture can and cannot tell you

This section is the one to read if you read nothing else here, because a reference shape is easy to mistake for a description of your deployment.

What it can tell you:

- The components a production deployment is built from, and what each is for before how it is configured.
- Where ownership usually falls — which parts you administer inside, which the platform operates, and which are genuinely shared.
- Where your code attaches, and what it presents when it does.
- The shape of the operations a deployment undergoes — rollout, upgrade, rotation, restore — and what each one touches.

What it cannot tell you:

- **It cannot tell you what your deployment is.** A reference shape is a design; a running deployment is an instance of it. Where they differ, yours is the one you are attached to, and only whoever operates it can tell you which case you are in.
- **It cannot tell you where a boundary is enforced.** From your side a refusal is a refusal. Which component produced it is not visible, and a refusal that arrives from a different layer than you assumed will send you to change the wrong thing.
- **It cannot tell you which boundaries are structural and which are historical.** Some are load-bearing and some are the residue of how a system grew. The behaviour is identical either way, so no amount of watching from your position separates them.
- **It cannot tell you that a control does anything.** A resource that exists is not a check that runs; this is the book's standing warning, and it applies twice over to infrastructure you cannot instrument yourself.

- **It cannot tell you the numbers.** Any figure here is an illustrative example and is labelled as one. Real deployments vary, and a number you cannot source is worse than no number.
- **It cannot tell you your operator's choices.** Egress policy, retention windows, recovery objectives and capacity ceilings are set by whoever runs the deployment, and it is precisely there that real deployments differ from the reference.

So the markers matter more in this part than anywhere else in the book. Where a claim rests on something you can observe, it is stated as observation. Where it explains how the deployment is *designed*, it is marked *design intent*, *not observable*. Where it could not be settled from outside, it says **UNVERIFIED**. Where the shipped behaviour is wrong it says **open defect**, with the symptom you will recognise it by.

A chapter in this part that uses none of those markers should be read as claiming more than an outside read can support.

07.7 closes with the checklist to run through before production and with the questions to put to whoever operates your deployment — which is the right destination for everything this page has just said it cannot answer.

Next: 07.1 — What a deployment is

07.1 — What a deployment is

Aegis is not something you install on a machine you already own. You are given a **deployment** — a running platform at an address someone hands you — and you administer *inside* it rather than operating the processes underneath it. The distinction is the whole chapter: almost everything that surprises a new integrator comes from assuming they are running the software when in fact they are using it.

Every later chapter in this part is a detail of the shape below.

What actually arrives

Three things, and it is worth being precise about which is which, because they come from different places and have different lifetimes.

- **A running platform.** The services are already up, already configured, already storing data. You do not start them, and in the normal case you have no way to. Your work happens at the API and inside the namespace you are given.
- **An endpoint.** One HTTPS base URL, and it is the only address you need. It is the same address your browser uses and the same one your SDK uses; there is no separate administrative port to find.
- **A credential.** A key or token issued to your organisation, presented by your integration. What it can do is scoped by the roles your organisation has granted it, and that scoping is enforced server-side — see 07.6 for how it is issued, scoped and revoked.

What does not arrive: infrastructure credentials. You are not given a database password, a storage key, or a cluster administrator's kubeconfig, because you do not need one to integrate. If you are asked to design against a deployment and you find yourself looking for those, the design has drifted into operating it.

The components, and what each is for

Names vary between deployments and the ones below are generic roles, not a inventory of any particular estate. What does not vary is the job each one does.

Component	What it is for
Managed Kubernetes cluster	Where the platform's own services and yours actually run. Managed means the control plane is not your problem.
Container registry	Holds the platform's images. The cluster pulls from it; nothing is built on the cluster.
Ingress / edge	Terminates TLS, owns the public hostname, and is the single door into the deployment.
Application namespace(s)	Where the running services live, separated from the platform's own controllers.

Component	What it is for
Job / agent execution namespace	Ephemeral, short-lived workloads — the things that run once and exit.
Managed relational database	Where your organisation's state lives, permanently. Also where embeddings live.
Cache	Makes reads fast. Losing it costs latency, not data.
Object storage	Artifacts and evidence — the large, immutable, long-retained things.
Secret store	Holds the credentials the platform itself needs, referenced rather than embedded.
Identity provider	Who the humans are, and the federation that lets workloads hold identity without holding secrets.

The ordering matters less than the split: the first five are the *substrate*, the last five are *state and identity*. 07.5 covers the state half and 07.6 covers identity, because those are the two places a design goes wrong quietly.

The shape in one picture

Read this as the map for the rest of the part. The SDK's attach point is drawn deliberately outside the boundary, because that is where it is: your code talks to this deployment over one HTTP edge and reaches nothing else directly.



The state and identity components are not reachable from your network, and that is a design property rather than a configuration detail — 07.4 draws the line properly.

Who owns what

This is the table to come back to. The honest answer is that ownership is **split**, and most integration friction comes from assuming a row is entirely one column or the other.

Thing	Platform owns	You own	Notes
Cluster, node pools, control plane	yes	no	You never schedule pods directly.
Namespaces and their quotas	creates them	works inside them	Quota values are usually negotiable; the boundary is not.
Platform images and their digests	yes	no	You do not build or pin these — see 07.3.
Your images, if you run any	registry, pull path	build and publish	Only if your deployment runs custom workloads at all.
Database engine, version, backup policy	yes	no	You choose what to store and how to index it.
Your organisation's schema and data	no	yes	Entirely yours, including retention decisions.
Network boundary and egress policy	default posture	requests for change	See 07.4 — this is where real deployments differ most.
Identity provider	runs it	configures your users	Federation into it is usually a shared task.
Your API credential	issues it	holds and rotates it	Scoped by the roles you grant it.
Upgrade timing	proposes and performs	your window preference	See 07.7.
Audit evidence	produces and retains it	reads it	Retention and access differ from operational logs — 07.7.

Design intent, not observable: that the third column is exactly what the fourth says and no more. You cannot verify the split from outside the deployment; you can only verify the parts that surface at the API. The column is written so you know what to ask, not so you can act as though it were settled.

A working rule for the ambiguous rows: if it changes what the platform *is*, the platform owns it and you request a change. If it changes what your organisation *means* — its units, roles, envelopes, knowledge — you own it and you do it through the API.

Where the SDK attaches

There is exactly one attachment point: **the edge**, over HTTP, at a base URL you supply. Your client is configured with it through `sdk:aegis_sdk.ClientConfig`, and nothing else in the SDK reaches the deployment by another route.

That single point is why the rest of this part matters less to your day-to-day code than you might expect: once the base URL and the credential are right, everything above is the platform's problem. It is also why a misconfigured deployment is hard to diagnose from the client side — a base URL pointing at the wrong estate produces a working client talking to the wrong system.

You can settle that question directly. `api:GET /api/v1/auth/me` answers which principal you authenticated as, which is the cheapest way to confirm both halves at once — the deployment you reached and the identity you reached it with.

This is a reference shape

Say it plainly, because the rest of the part depends on the reader holding it: **this is a reference architecture, not a description of a running estate**. A given deployment may differ in detail — a component may be absent, an equivalent substituted, a service pinned to a different engine. Where the differences matter to you, they matter because they change what you can observe or control, and this part names those places as they come up.

Three ways to find out which parts of this shape your deployment actually has:

1. **Ask your operator, and ask specifically.** "Do I get my own namespace, and what is its quota?" is answerable. "How does it work?" is not.
2. **Test what surfaces at the API.** Health operations, your own identity, and the error taxonomy are all observable; `api:GET /api/v1/auth/me` above is the simplest example.
3. **Read the rest of this part and notice which claims are marked.** Anything marked as design intent is a claim about construction you cannot check from where you sit. Treat those as questions for your operator rather than as facts about your deployment.

Design intent, not observable: the component split above, and the ownership boundary that follows from it. The marker is not hedging — a reference architecture that presented itself as an inventory would be wrong the moment any deployment deviated, and wrong invisibly.

Next: 07.2 — The cluster and its namespaces

07.2 — The cluster and its namespaces

Underneath everything in this part is a Kubernetes cluster. You will rarely touch it, and you never administer it — but you will meet its boundaries constantly, because they are what your workloads run inside. This chapter is about those boundaries and, more usefully, about which of them are real.

Managed, and what "managed" covers

The cluster is **managed**: the control plane — the API server, the scheduler, the controllers that keep the declared state true — is operated by the platform. That is a real removal of work, and it is worth being precise about what it removes and what it does not.

- **Covered:** control-plane availability, version upgrades, certificate rotation for the control plane, and the operational burden of keeping the scheduler healthy. You do not patch a managed control plane.
- **Not covered:** anything running *in* the cluster. Your workloads, their resource requests, their behaviour under load, and whether they are healthy are not the platform's problem by virtue of the cluster being managed. "Managed" describes the control plane, not your services.

A managed control plane also means the cluster's API server is not something you reach directly. You interact with the deployment through the edge described in 07.4 or, where a deployment offers it, through a namespace-scoped administrative path. Which of those your deployment provides is a question worth asking up front rather than discovering during an incident.

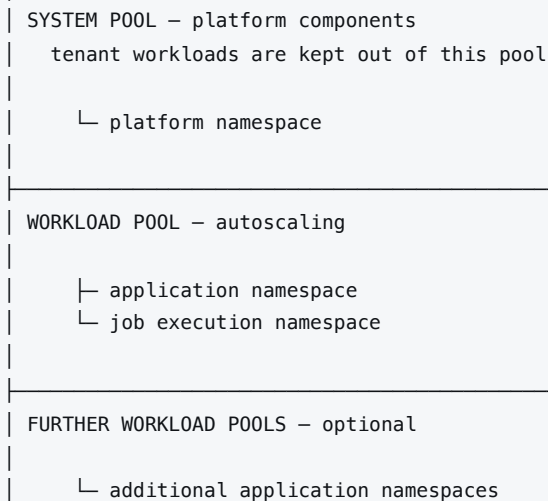
Node pools, and why there is more than one

The nodes the workloads land on are grouped into **pools**, and the reference shape has at least two kinds:

- **A system pool**, carrying the platform's own components — the controllers, ingress, and the services that make the cluster work at all.
- **One or more workload pools**, carrying application services and the ephemeral job execution described below.

The split exists for one reason: **the platform's own components must not be starved by tenant workloads**. If everything shared one pool, a workload that consumed the pool's capacity would take the cluster's own control components down with it — and the failure would look like the platform being broken rather than like a workload being greedy.

Autoscaling is the normal state, not an event. Pools grow and shrink as demand moves. A consequence worth internalising: a pod that cannot be scheduled is not necessarily a quota problem — it may be a node that has not come up yet, and the two have different fixes and different durations.



Read it as: the pools are the top-level grouping, and namespaces live *inside* them. The separation is not a network path — it is the placement rule that keeps tenant workloads out of the system pool, which is enforced as described below.

The number and naming of pools is the deployment's. The reference shape is one system pool plus one or more workload pools; a specific deployment may carry more, sized differently. Do not plan against a pool count — plan against the rule that separates them.

The namespace model

Namespaces are how work is separated. The reference shape uses three kinds:

Namespace kind	What runs there	Who puts things there
Platform	Controllers, ingress, the services that run the deployment itself	The platform, exclusively
Application	The long-running services that serve the API	The platform; you configure what they serve
Job execution	Ephemeral, short-lived workloads — the things that run once and exit	The platform, on demand

The separation is worth having even though it is not a security boundary (next section), because it gives you three useful things: independent quota, a meaningful blast radius when something misbehaves, and a resource view that separates "the platform is unhealthy" from "my work is unhealthy".

A namespace is not a security boundary

This is the point readers most often get wrong, so it is stated flatly:

A namespace is a naming and quota boundary. It is not a security boundary. Two pods in different namespaces on the same cluster are not, by virtue of that fact, isolated from one another. Namespaces do not create a network partition, do not prevent one workload from reaching another's endpoints, and do not by themselves stop a compromised workload from doing anything the node permits.

What actually enforces isolation, where it exists, sits elsewhere:

Enforcement	Where it lives	What it actually separates
Network policy	Inside the cluster, applied to namespaces and pods	Which pods may open connections to which others
Private endpoints	The deployment's network	The data plane from the public internet — 07.4
Workload identity	The identity provider, federated into the cluster	What a pod may authenticate <i>as</i> , and therefore what it may reach — 07.6
Authorization at the API	The application layer	What a principal may do, regardless of network position — part 04

The practical consequence: **a namespace boundary should be treated as an organisational convenience, and every real control assumed to be somewhere else.** If your design depends on "it is in a different namespace, so it cannot reach us", the design depends on something a namespace does not provide.

Which of the enforcement mechanisms above your deployment applies, and how strictly, is the operator's configuration. Network policy in particular is present in the reference shape and enforced to a degree each deployment sets. That is a question for your operator, and one of the more important ones to ask.

Quotas and limits, and what hitting one looks like

Two different mechanisms, and conflating them is a common source of confusion:

- **Quotas** cap the total a namespace may consume — how much CPU, memory, and how many objects. A quota is a namespace-level ceiling.
- **Limits** bound an individual container. A limit is what happens to one pod.

You hit	Symptom	What it means
A quota	New workloads are refused or stay pending	The namespace is at its ceiling. Existing work keeps running.
A memory limit	The container is restarted	The pod exceeded its own bound. This is per-pod, not namespace-wide.
A CPU limit	The container is throttled, not killed	Slower responses, no restart. Easily mistaken for a performance bug.

You hit	Symptom	What it means
Pool capacity	Pods stay pending until nodes arrive	Not a quota at all — the pool is growing, or cannot.

The third row is the one that costs people days: a throttled container returns slow responses and logs nothing, so it reads as an application problem. If latency rises under load and nothing else changed, check throttling before profiling.

Placement: how tenant workloads stay off the system pool

The rule that separates the pools is enforced with the cluster's own placement mechanisms:

- **Taints and tolerations.** The system pool carries a taint; only components that explicitly tolerate it may be scheduled there. Tenant workloads do not carry that toleration, so they cannot land there.
- **Node affinity.** Workloads are steered toward the pools intended for them, rather than being left to the scheduler's default preference.

These are the mechanism, and they are why the pool separation in the diagram above is a rule rather than a hope. You will normally not configure them — but if your deployment runs custom workloads, and you are asked to supply a scheduling constraint, this is the machinery your request is expressed in.

The reference shape uses the two above; a specific estate may add admission policy on top of them.

Next: 07.3 — Images and the registry

07.3 — Images and the registry

Everything the deployment runs is a container image, and every image comes from a registry. That much is unremarkable. What is worth a chapter is **how an image is named when it is deployed**, because the difference between the two obvious ways of naming one is the difference between a release you can reason about and a release you cannot.

A tag is a label. A digest is an identity.

A registry lets you refer to an image two ways:

```
<registry>.azurecr.io/<image>:1.4.0  
<registry>.azurecr.io/<image>@sha256:<digest>
```

The first is a **tag**. The second is a **digest**. They look like variations on the same idea. They are not.

	tag	digest
What it names	a label someone chose	the exact bytes of one image
Can it change?	yes — pushed over, silently	no — it <i>is</i> the content hash
Reproducible?	only if nobody ever retags	always
Good for	humans reading a list	deploying, auditing, rolling back
Failure mode	two different images, same name, weeks apart	none — a digest cannot drift

A mutable tag is not a release. If a deployment is pinned to a tag, then the thing running in your environment today is not provably the thing that was tested last week, even though every artefact of the process — the manifest, the deployment record, the change ticket — says `1.4.0`. Nothing has to go wrong for this to be true; a rebuild and a re-push under the same tag is enough, and it leaves no trace anyone would think to look for.

A digest is what makes "we tested this exact thing" a true sentence. When you see a pod running `@sha256:...`, that string names one image and no other, and it names the same image in every environment and every registry copy.

Design intent, not observable: that your deployment pins by digest rather than by tag. You cannot read the running pod's image reference from outside, and this is a reasonable question to put to your operator precisely because the answer is not visible from where you sit.

Build once, then promote

The reference pipeline builds an image **once** and then moves *that same image* through the environments:

1. **Build** — from a specific source revision, in a pipeline, into an image.

2. **Publish** — pushed to the registry and identified by its digest.
3. **Promote** — the *same digest* is referenced in each environment in turn, as it passes whatever gate that environment applies.

Why not build per environment, which is easier to wire up? Because a rebuild is a different image. Different base layer, possibly different dependency resolution, different build timestamps — all invisible, all real. So "we tested it in staging, then built for production" means you tested something and deployed something else, and the only reason it usually works is that the two are *usually* identical rather than *guaranteed* identical. Promoting a digest removes the word "usually".

This is also what makes the chain auditable: build happened once, at one time, from one revision, and everything after that is a reference to it.



The label beside each hop is the subject of the next two sections: a different identity acts at each one, and none of them is a password sitting in a file.

The cluster does not hold a registry password

A cluster that must pull a private image has to authenticate to the registry to do it. The obvious way is an **image pull secret** — a credential stored in the cluster and handed to every node that pulls. The reference architecture does **not** do this, and the reason is not tidiness:

A stored pull credential is a long-lived secret with cluster-wide reach. It sits in the cluster, it is readable by anything that can read secrets there, it must be rotated by hand, and when it leaks the blast radius is the whole registry rather than one workload.

Instead, the cluster authenticates as a **managed identity** — a workload identity federated from the platform's identity provider. There is no password to store, because there is no password: the identity is issued to the workload at runtime and is scoped to what that workload should be able to reach.

That mechanism is 07.6's subject, and it is deliberately the same mechanism used for every other cloud resource a pod might need. One identity model, not one per service.

Design intent, not observable: the federation is the reference shape's construction. A specific deployment may still hold a pull secret; if you are auditing one, this is a question worth asking directly, because the two designs have very different exposure when a credential leaks.

Supply chain: what to require before you promote

Two properties, both about trusting an image you did not build:

- **Provenance** — evidence of *how* the image was produced: which source revision, which build, which pipeline. Provenance answers "where did this come from?" and makes the answer checkable rather than asserted.
- **Signature** — evidence of *who* produced it, verifiable against a key you trust. A signature answers "should I accept this?" and is what lets a policy say *reject anything unsigned* rather than *check the tag and hope*.

They are complementary. Provenance without a signature tells you a story you cannot authenticate; a signature without provenance tells you who signed something without telling you what is in it.

What an operator should require before promoting an image into an environment that matters: that it is pinned by digest, that its signature verifies against a trusted key, and that its provenance names a source revision the operator recognises. A tag that looks right is not on that list, and the reason is the first section of this chapter.

Design intent, not observable: whether your deployment enforces these, or merely supports them. Enforcement is admission policy inside the cluster and is not visible from the client side — another question for your operator, and one whose answer changes how much the previous paragraph is worth.

Rollback is re-pinning, not rebuilding

This follows directly from digests, and it is the practical payoff:

To roll back, reference the previous digest. The image still exists, it is still the exact bytes that ran before, and the operation is a change of reference — minutes, and no build.

Contrast the tag-based version of the same operation, which is where rollback usually goes wrong: you cannot roll *back* to a tag, because the tag does not remember. You can only redeploy the tag and hope it still points where you think it does, or reconstruct the earlier build and deploy something that is *approximately* what ran before.

The corollary is worth acting on: a rollback you have never rehearsed is a hypothesis. Knowing the previous digest exists is not the same as having deployed it. If the answer to "what do we do if this goes wrong" is a procedure nobody has executed, the honest state of that control is untested — 07.7 returns to this in the pre-production checklist.

Which versions you can and cannot see

These are different questions and it is easy to conflate them:

- **Your own integration's version** is entirely observable. `sdk:aegis_sdk.__version__` reports it from your own environment, and it is what you quote when asking whether your client is compatible with a deployment.
- **The deployment's running image digests** are not observable from your integration. You reach the platform over HTTP; you do not get to read what it is running.

UNVERIFIED: whether any deployment exposes its own version through a supported operation. The health operations in 07.7 report liveness rather than build identity, and this document does not claim a version endpoint exists. If you need the deployed version, ask your operator — that is the reliable route, and it is not a workaround.

Next: 07.4 — The network boundary

07.4 — The network boundary

A deployment has a public surface and a private one, and almost all the architecture's security value sits in the line between them. This chapter draws that line, names what crosses it, and is honest about the one crossing where the reference shape and real deployments diverge most.

One public door

The public surface is deliberately tiny:

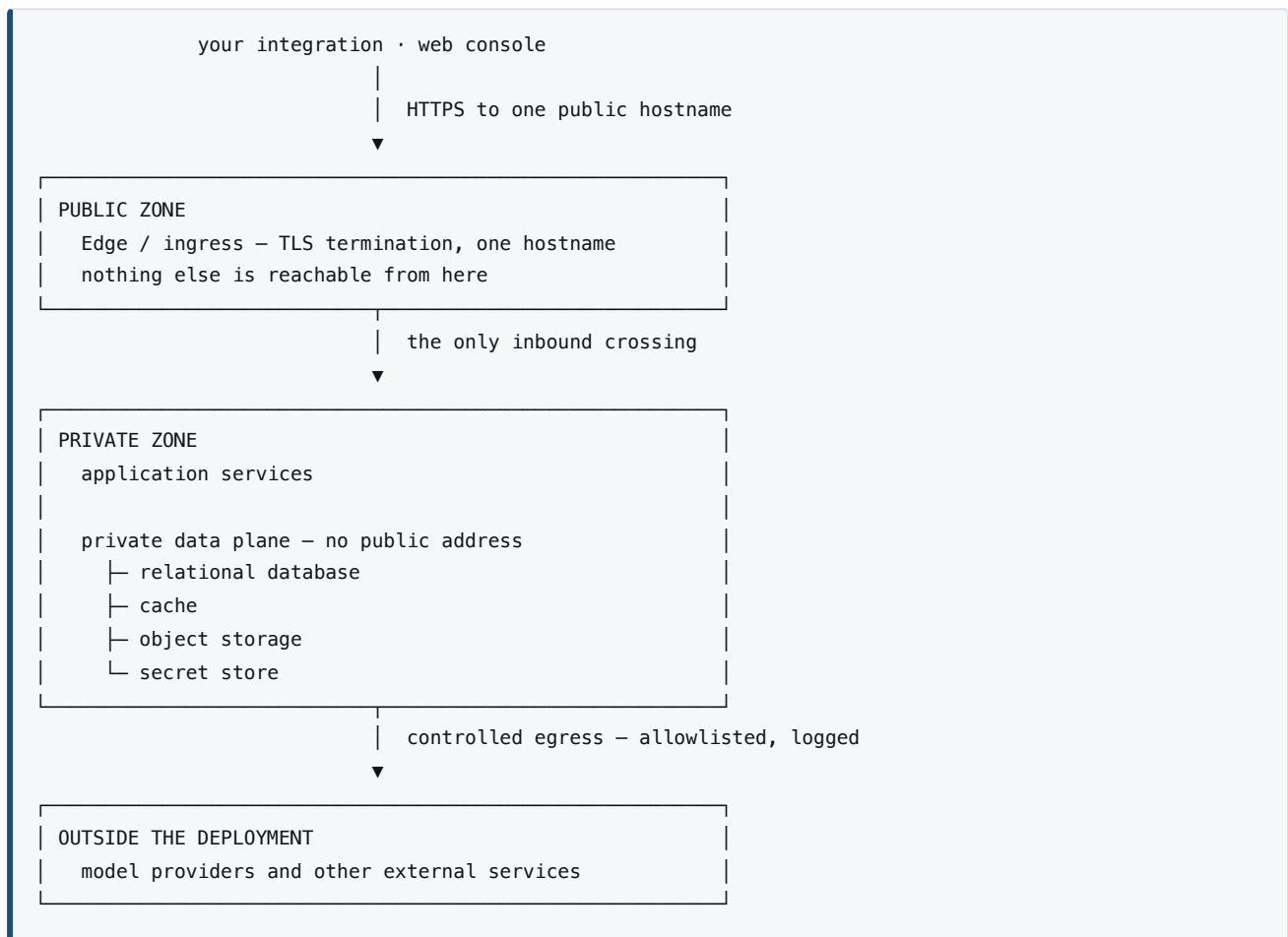
- **One hostname**, serving the API. This is the address your integration points at and the address the web console is served from.
- **TLS termination at the edge**. Certificates are presented and managed at the boundary, and your connection ends there. *Design intent, not observable*: what the hops behind the edge look like — re-encrypted or not — is a construction property, and 07.5 covers what the reference shape expects of the data plane specifically.
- **Nothing else**. There is no public database port, no public storage endpoint, no administrative interface on a second hostname.

From your side the entire boundary is one string: the base URL you give

`sdk:aegis_sdk.ClientConfig`. That is the whole of what the SDK can reach, and it is worth appreciating as a design statement rather than a convenience — there is no second address and no code path in this client that aims at one.

Design intent, not observable: that the hostname is the only public entry point. You can verify the *client* half of that claim trivially — the SDK has one configured base URL and no other route out. You cannot verify from your side that no second door exists elsewhere, so the claim stays marked.

What crosses the line



Two crossings are drawn and they behave differently. **Inbound**, one edge admits traffic and passes it to the application services. **Outbound**, a single controlled path is the only way anything leaves. Everything else in the diagram is inside a zone and never crosses.

The private plane

The database, the cache, object storage and the secret store are reachable **only over private endpoints inside the deployment's network**. They are not on the public internet, and they do not have public addresses that merely happen to be firewalled — the reference shape removes them from public reachability entirely.

Why this matters more for a governance product than for a typical web application. Three of these four components hold the things the product's value depends on:

- The **database** holds the organisation's structure, its envelopes, its clearance model, and the record of what was permitted. It is simultaneously the system of record and a map of the organisation's authority structure.
- **Object storage** holds artifacts and evidence — the material an auditor would be shown.
- The **secret store** holds the credentials that let the platform act as itself.

A publicly reachable data plane would mean the audit evidence — the thing the platform exists to produce — was an internet-exposed asset. The compromise that matters is not disruption; it is that someone could alter or read the record whose whole purpose is to be trustworthy. That is why the data plane sits behind private endpoints rather than behind authentication alone.

Design intent, not observable: that private endpoints are the mechanism used, rather than a network-level restriction that achieves the same reachability outcome. Either satisfies the property; the reference shape uses private endpoints, and a specific deployment may achieve it differently.

Segmentation inside the private zone

Within the private zone, not everything can reach everything:

- **Namespace-to-namespace.** Network policy governs which pods may open connections to which others. The application services reach the data plane; the ephemeral job namespace reaches what its work requires and no more. *Design intent, not observable, and unevenly true in practice:* network policy is frequently present in the reference shape and variably enforced — the same reason 07.2 refuses to treat a namespace as a security boundary. Do not design against this without confirming it.
- **Workload-to-data-plane.** Reachability is paired with **identity**: being on the network is not the same as being authorised. A service that can open a TCP connection to the database still has to authenticate to it — see 07.6. The network limits who may *try*; identity decides who *succeeds*.

Hold both, because they fail differently. A network rule that is too permissive is a latent exposure. An identity that is too permissive is an immediate one.

Egress is where deployments actually differ

Be direct about this, because it is the section most likely to be untrue of the deployment in front of you.

Agents in this platform call **model providers**, and those providers are outside the deployment. So there is real outbound traffic carrying real content, and how that traffic is controlled is the single largest variable between deployments.

The reference posture:

- **Egress is not default-allow.** Outbound traffic leaves through a controlled path, and destinations are allowlisted rather than the whole internet being reachable.
- **It is a governed path.** What leaves is attributable. An agent's call to a model provider is the same class of event as any other externally-visible action the platform takes, and it is recorded as such rather than being invisible plumbing.
- **It is the crossing that carries your data outward.** Inbound traffic is requests from you; outbound traffic may be your organisation's content being sent to a third party in order to answer one of those requests. That asymmetry is why this crossing deserves more scrutiny than the inbound one.

Where the reference shape and reality part company: allowlisting egress is a real operational burden. Provider endpoints change, regions differ, and an over-tight policy produces outages that look like application failures. Many deployments therefore run something looser than the above — sometimes substantially looser.

This is exactly why the next section is worth reading rather than skimming.

What to ask your operator, in this order:

1. Is egress allowlisted, or is it default-allow with logging?
2. Which destinations are on the allowlist, and who changes it?
3. Is the agent-to-provider path recorded, and where does that record go?

UNVERIFIED: the egress posture of any particular deployment. This chapter describes the reference shape; it cannot see the policy applied in an estate you were handed, and the difference between those two answers is significant enough that guessing is not reasonable.

What the boundary does not do

Two honest limits, stated so the chapter is not read as more than it is:

- **The boundary is not an authorization model.** It decides what can be *reached*, not what may be *done*. A request that arrives cleanly over the public hostname still faces every check the platform applies — see part 04. Network position is not authority, and no part of this architecture treats it as such.
- **A boundary you cannot see is a boundary you must ask about.** Everything in this chapter except the single base URL is a claim about construction. The marked claims are not hedging; they are the accurate epistemic status of a reader who was given an endpoint rather than a cluster.

Next: 07.5 — Stateful services

07.5 — Stateful services

Everything that survives a restart lives here. The workloads in 07.2 are designed to be replaced; these are the components that must not lose anything, and they are consequently the ones where an operator's choices matter most and are least visible from outside.

State does not live on the cluster

The platform's own services run on the cluster. Its **state does not**. That is a deliberate separation and worth understanding, because it explains a great deal about how the deployment behaves.

Nodes are cattle, not pets: they are created, drained and destroyed as the pools autoscale. Anything stored only on a node's local disk is therefore coupled to that node's lifetime. So the stateful components are **managed services outside the cluster** — the database, the cache, and object storage are reached *over the network* by the services that use them, rather than running as pods alongside them.

The practical consequences you will meet:

- **A pod being rescheduled loses nothing.** It was never holding the data.
- **The cluster can be rebuilt without the data being rebuilt,** because the data was never on it.
- **Database availability and cluster availability are different things.** One can be healthy while the other is not, and the symptoms differ — worth knowing before an incident rather than during one.

The relational database

This is where the organisation's state lives: its structure, its roles and their envelopes, its clearance model, its objectives and their history, and the record of what was permitted. **Managed** means the platform operates it — engine version, patching, replication, backup — and you do not. You own what is *in* it.

Embeddings live in the database

This is the detail that most affects a design decision, so it is stated separately.

The database carries a vector extension, and embeddings are stored in it — not in a separate vector service. When an agent or a search performs a semantic lookup, that lookup runs against the same database that holds the organisation's records, with the same transactional and backup characteristics as everything else.

The operation this shows up in is `api:GET /api/v1/knowledge/search`. That this client declares it is something you can check from your own machine. *Design intent, not observable:* which store the embeddings actually live in is a property of the deployment, and nothing available to you establishes it.

Three consequences worth planning around:

- **One store, one backup, one recovery point.** There is no second system whose consistency with the first has to be reasoned about. A record and the embedding derived from it share a recovery point by construction.
- **The engine is constrained.** You cannot move this deployment onto an arbitrary relational engine or an arbitrary major version. The engine must be one that supports the vector extension, and a version upgrade is gated on that extension being available for the new version. *Design intent, not observable*: this constraint is real, and which engines satisfy it is a question for your operator rather than something inferable from here.
- **Embedding cost is storage cost.** As the corpus grows, so does the database. Capacity planning for knowledge is database capacity planning, not a separate line item.

The cache

The cache exists to make reads fast. Its correctness property is the useful thing to know:

Losing the cache degrades performance, not correctness. Nothing durable lives in it. A cold cache means slower responses while it refills — temporarily worse latency, the same answers.

That is the honest general statement, and it comes with an honest caveat:

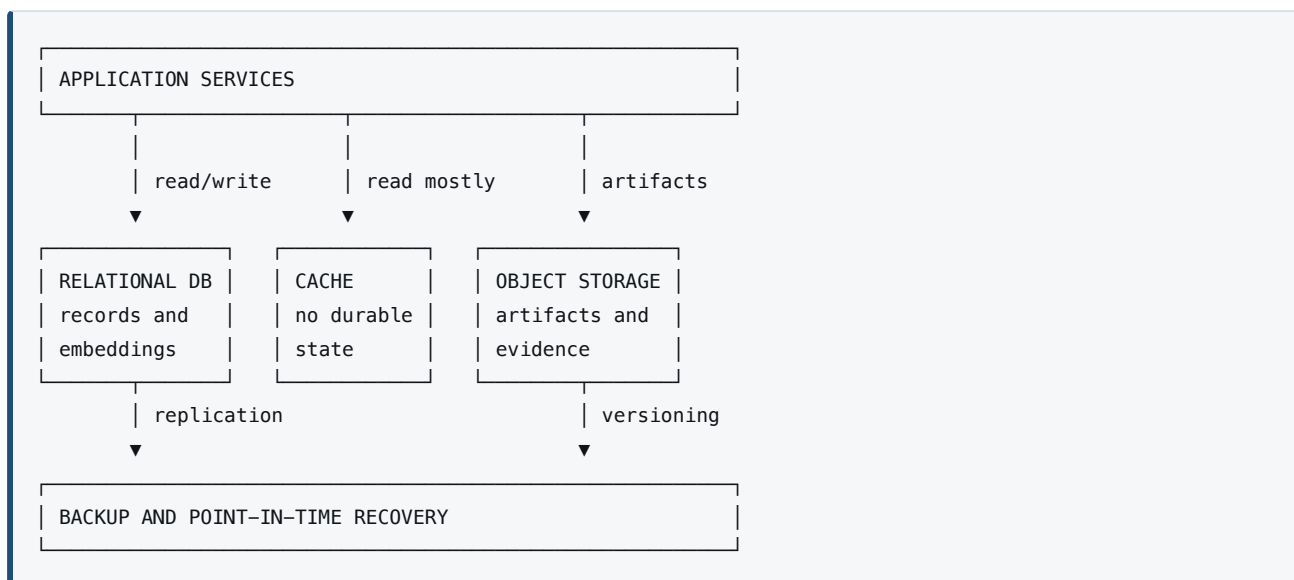
Any system has exceptions to that rule, and you should confirm which apply here. A cache that holds anything authoritative — a rate-limit counter, a session that is not also persisted, a lock — is no longer merely an optimisation, and losing it changes behaviour rather than speed. The reference shape keeps authoritative state out of the cache, but *design intent, not observable* applies: you cannot verify from outside what a given deployment actually stores there.

If you are designing for failure, this is worth one direct question to your operator: does losing the cache lose anything that is not reconstructible? A "no" is what the architecture intends. Anything other than a clear "no" should be treated as a real answer.

Object storage

Artifacts and evidence — the large, immutable, long-retained material. A few properties that matter:

- **It is not the system of record for structure.** Roles, envelopes and history live in the database. Object storage holds the bulkier things: uploaded artifacts, captured evidence, exports.
- **It is written to be retained, not overwritten.** The reference shape treats stored objects as effectively immutable; the way to change one is to write a new one. That is what makes it evidence rather than storage.
- **Retention is a policy, not an accident.** How long evidence is kept, and who can delete it, are decisions the deployment makes — and if you have compliance obligations of your own, they are decisions you should confirm rather than assume.



The absence is the point of the diagram: the cache is the one component with no arrow into the protected copy, because there is nothing in it that cannot be rebuilt. The database and object storage both have one — replication and versioning respectively.

Backup and recovery

The reference expectation: automated backups, and **point-in-time recovery** — the ability to restore to a chosen moment, not merely to last night. The distinction is the difference between "we lost a day" and "we lost an hour", and it is the second you want when something is deleted at 2pm.

Recovery objectives — how much loss is tolerable (the recovery point) and how long recovery may take (the recovery time) — are **targets an operator sets**, not guarantees a platform ships. They are choices with costs attached, and different deployments will have made different ones. 07.7 covers what to ask.

What an operator must verify rather than assume:

- That backups are **actually running**, not merely configured. A policy that exists and a job that executes are different facts.
- That the retention window is what was intended, and that the oldest available restore point is where you think it is.
- That a restore has been **performed**, into a real environment, within recent memory.

A backup nobody has restored is a hypothesis. It is a common and understandable one — the configuration looks right, the job reports success, the storage has bytes in it — and none of those observations distinguishes a working backup from an unusable one. The only instrument that settles it is a restore that produces a usable database. Until that has happened, the correct description of your disaster-recovery posture is *untested*, which is not the same as *bad* but must not be reported as *good*.

Design intent, not observable: all of it, from your position. This is the chapter where the gap between what a reference architecture describes and what you can check is widest, and it is why the next section exists.

Encryption

Encryption at rest and in transit are both expected in the reference shape:

- **In transit.** Traffic reaches the edge over TLS (07.4), and connections to the data plane are encrypted inside the private network as well as outside it. Encrypting only the public hop would leave the private network — which is not a trust boundary in the sense 07.2 describes — carrying plaintext.
- **At rest.** Stored data is encrypted by the managed service that holds it. This is a property of the platform, not something you configure.

Your operator's key management may extend this. A deployment can use customer-managed keys so that the keys protecting your data are held in your control rather than the platform's, which changes who can read the data at the storage layer. That is a genuine control with real operational consequences — losing such a key is a different class of incident — and whether it is in place is a question of policy, not something visible from your integration.

UNVERIFIED — and marked deliberately: the specific encryption configuration, key custody arrangements, and recovery objectives of any particular deployment. Every claim in this section is about construction. The reader who checks them by asking has done the right thing; the reader who treats them as established because they appear in a reference architecture has not.

Next: 07.6 — Identity and secrets

07.6 — Identity and secrets

Most of a deployment's actual security posture reduces to one question: **which identities exist, and which of them are holding a password**. This chapter is about the answer the reference architecture gives, and about a contrast you can draw for yourself before you finish reading it.

Two identities, and they are different

Keep these apart, because they are conflated constantly and the conflation leads to bad reasoning about who can do what.

	Operator identity	Workload identity
Whose is it?	A person's	A workload's
What is it for?	Administering the deployment	A service reaching a resource
How does it authenticate?	Interactively, through the identity provider	With a short-lived token issued to the pod
Lifetime	As long as the person's access is valid	Minutes, refreshed automatically
Can it be shared?	It should not be, and it is attributable	It is not a thing anyone shares
Blast radius if misused	Whatever that person's roles allow	Whatever that one workload is permitted

Neither is privileged over the other in principle. They are simply **different kinds of principal**, and an architecture that keeps them separate can answer "who did this?" unambiguously. One that merges them — where a workload authenticates as a person, or a person's credential is baked into a service — cannot, and that is the failure this separation exists to prevent.

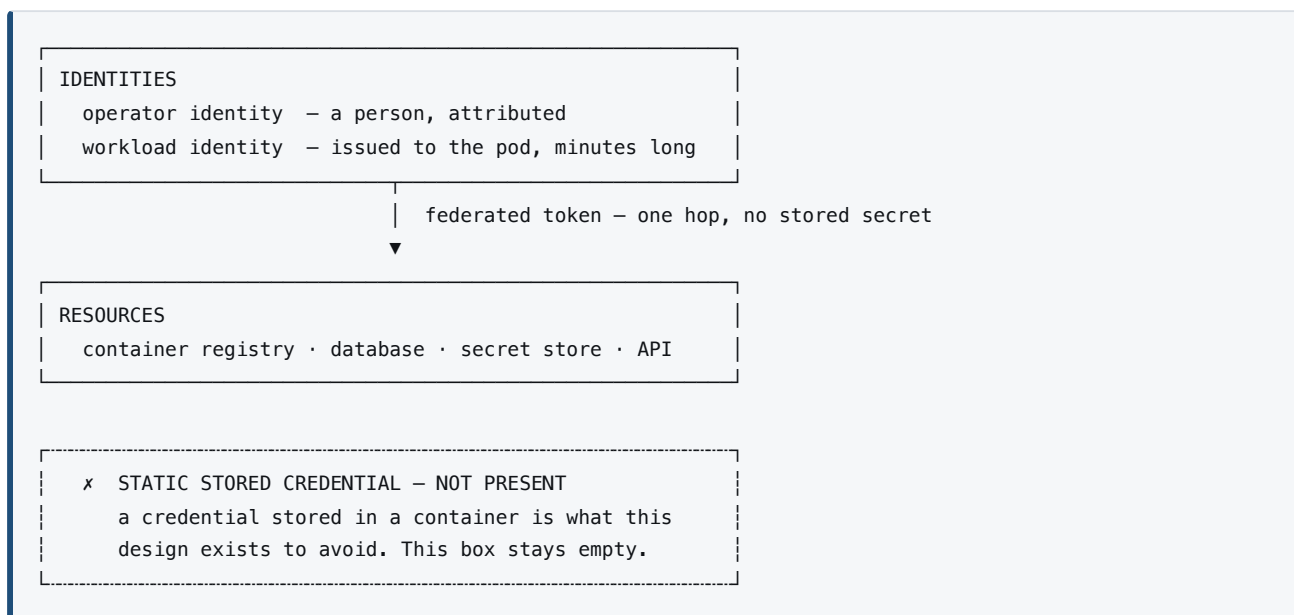
A pod holds no long-lived secret

This is the single most important sentence in this chapter:

A pod authenticates to cloud resources as a federated managed identity, which means there is no long-lived credential stored inside the container.

Say the negative form too, because it is what the architecture is actually built to avoid: **a stored credential in a container is the thing this design exists to prevent**. A password in an environment variable, a key in a mounted file, a token in an image layer — each is a secret with a long life, many places to leak from, and no automatic end. Anything that can read the container's environment or filesystem reads it; anything that can read the image layer reads it forever.

Workload identity federation replaces that with a token the pod requests and receives at runtime, issued because the platform can attest *which workload is asking*. The credential is short-lived, scoped to the resource being reached, and never written anywhere durable.



The dashed box at the bottom is the one that is *absent*, and it is drawn deliberately: the contrast between a federated hop and a stored credential is the entire argument of this section. If you take one thing from this part, take that this box must stay empty.

Design intent, not observable: that no workload in a given deployment holds a stored credential. This is exactly the kind of claim that is true by construction in the reference shape and unverifiable from your integration — you cannot see inside a container. It is also, for that reason, one of the most valuable questions to put to your operator directly.

The secret store, and what lives in it

A secret store still exists, and that is not a contradiction — federation removes the *need* for stored credentials, it does not remove every secret. What actually lives there:

- **The platform's own credentials** for the resources that genuinely cannot issue short-lived tokens — third-party services that only accept a static key, for instance.
- **Provider keys**, where a model provider requires an API key rather than accepting a federated identity. This is a common and unavoidable case, and it is precisely why the store exists.
- **Configuration that is sensitive but not a credential** — endpoints and parameters whose disclosure would be a problem.

The rule that governs all of them: a secret is referenced at runtime, never baked in. It is read when the workload needs it, from the store, rather than being written into an image or a deployment manifest. The distinction is not stylistic — a secret in an image is in every copy of that image and survives every rotation of the original, while a referenced secret is one thing in one place, replaceable without rebuilding anything.

Design intent, not observable: what your deployment's store actually contains, and whether anything in it should have been federated instead. There is no outside vantage point for this, and the honest position is that it is a construction property you are told about rather than one you check.

Rotation, and what it costs a running workload

Rotation is where the two designs above diverge in a way you will feel operationally, so it is worth stating as a comparison rather than a list.

What rotates	Trigger	Effect on a running workload
Workload identity tokens	Automatic, continuously	None. The pod asks for the next one. This is the common case.
Federated identities' trust configuration	Platform change	A pod restart, at a time the platform chooses.
Third-party provider keys	On a schedule, or on suspicion	A restart or a reload, depending on how the workload reads it.
Your application credential	You decide	None, if you issue the new one before revoking the old.

The first row is the payoff of the whole design: **the credential that is used most often is the one that never needs a maintenance window**, because it was never stored and its rotation is invisible.

The last row is the one you control, and it is covered next.

Your credential, and what you can verify about it

Your integration holds an application credential, presented by the client —

`sdk:aegis_sdk.AgenticOSClient` is where that happens, and it is the only credential your code handles.

Three properties to understand about it:

- **Issued to your organisation, scoped by roles.** What it can do is the union of what the roles granted to it permit. Nothing about the credential itself confers authority; the scoping is enforced server-side, on every request.
- **Attributable.** Actions taken with it are recorded as having been taken by that principal. That is what makes the audit trail mean anything — an entry naming a credential that ten systems share is not evidence of much.
- **Revocable.** Revocation is immediate, and it is the reason a leaked credential is recoverable rather than terminal.

You can settle which principal you are, at any time, with `api:GET /api/v1/auth/me`. This is more useful than it sounds during an incident: when a call is refused and the reason is not obvious, the first question is whether you are who you think you are, and that operation answers it directly rather than by inference.

What you can verify from outside, and what you cannot:

Claim	Verifiable by you?
Which principal your credential authenticates as	Yes — <code>api:GET /api/v1/auth/me</code>

Claim	Verifiable by you?
What your credential is permitted to do	Yes, by trying — refusals are the evidence
Who else holds a credential for your organisation	Partly — ask; it is your organisation's, not the platform's
That no pod holds a stored secret	No — take it on trust, and ask
How rotation is performed, and when it last happened	No — ask your operator

Design intent, not observable: the third row rests on the ownership split in 07.1's table, and that table is itself construction. Whether your organisation or your operator keeps the credential records is a question to put to them, not an answer this book can give you.

The bottom two rows are the ones to notice. They are the claims this architecture most wants you to believe and the ones you are least able to check, which is exactly why they are listed rather than left implicit.

UNVERIFIED: every construction claim in this chapter, for any specific deployment. The reference shape is described; the estate you were handed may differ, and no part of this document can see into it. Asking is not a workaround here — for these particular questions, it is the only instrument there is.

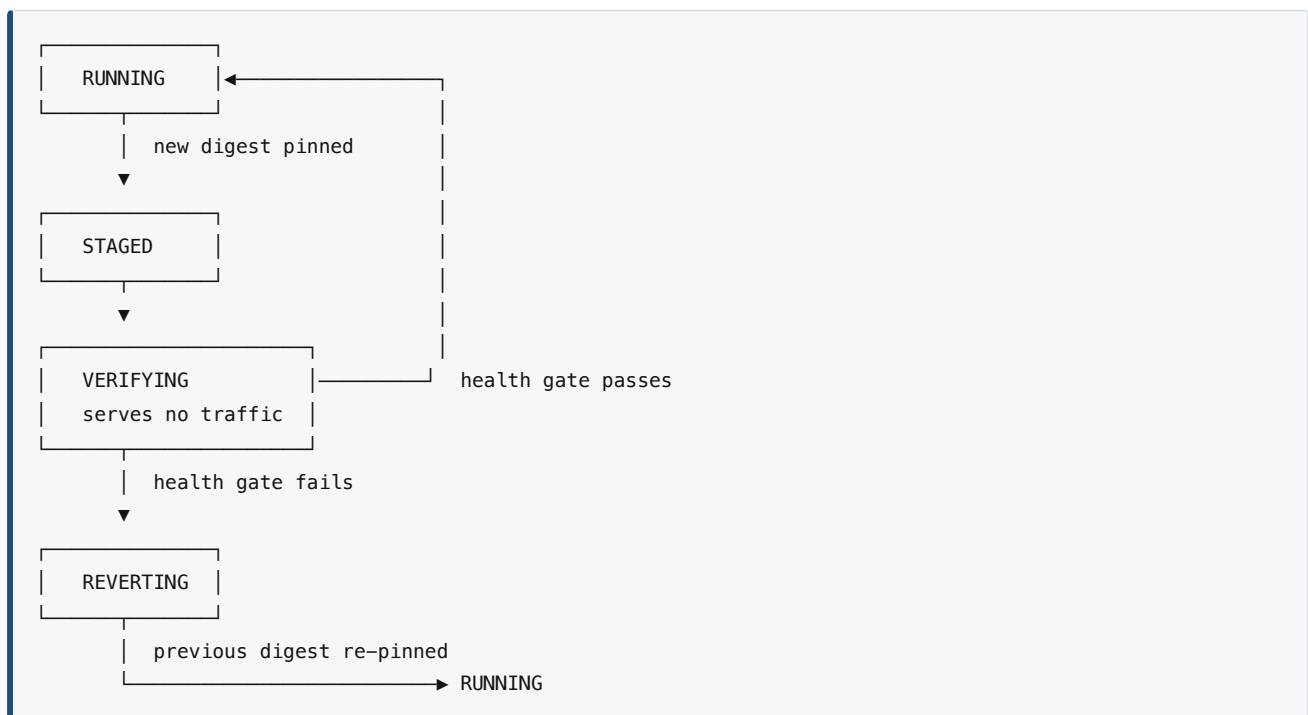
Next: 07.7 — Operating the deployment

07.7 — Operating the deployment

This chapter is about what happens after the design is settled: changing the thing, watching it, sizing it, and recovering it. It closes with a checklist you can actually run through, and that checklist is the part to come back to.

Change: rollout, health gate, rollback

A change to the platform is not a file copy. It is a **rollout**: new instances of a service are started alongside the old ones, checked, and only then does traffic move. If the check fails, traffic never moves and nothing is lost.



The states matter more than the arrows. **Verifying is a state in which the new version exists and serves nothing** — that is what makes the gate meaningful, and it is why a failed rollout is usually an absence of change rather than an incident.

Health gating decides the transition: the new instances must demonstrate they are working before they receive traffic. What counts as working is a per-service definition, and *design intent*, not *observable* applies to its details.

Rollback ties directly back to 07.3. Because the deployed reference is a **digest** rather than a tag, reverting is re-pinning the previous digest — the exact bytes that ran before, still in the registry, still verifiable. No rebuild, no reconstruction, no "approximately what ran before". This is the operational payoff of the digest discipline, and it is the reason that chapter insisted on it.

Upgrades, and what you control

Design intent, not observable: platform upgrades are **initiated by the platform**, not by you. That follows from 07.1's boundary — you administer inside the deployment, and the platform is operated for you — and it is the reference shape's operating model rather than something you can read from your own integration.

What the platform decides: what is upgraded, and that an upgrade is available. **What is usually negotiable:** when it happens.

The reference expectation is that upgrades are scheduled into an agreed window rather than applied at the platform's convenience — an upgrade is a change to something you are integrating against, and applying one during your launch week is a decision somebody should make deliberately.

What a partner can reasonably ask for:

- Advance notice of a change, and what it contains.
- A maintenance window that respects your own change freezes.
- A statement of whether the change is backward-compatible for the operations you use.
- A rollback position if something you depend on regresses — which 07.3 makes a real possibility rather than a hope.

Your own changes are separate. If your deployment runs workloads you own, those roll out on your schedule, through the same rollout and rollback machinery. The distinction is worth holding: the platform upgrades itself, you upgrade what you put on it.

Design intent, not observable: the notice period, the window policy, and who makes the call. These are contractual and operational rather than architectural, and they vary. Ask.

Observability: two kinds of record, and they are not interchangeable

A common and expensive mistake is assuming that because a system emits logs, its audit evidence is in them. It is not, and the two differ on every axis that matters.

	Audit evidence	Operational telemetry
Answers	Who did what, and was it permitted	Is the system healthy, and is it fast
Produced by	The governance layer, deliberately	The runtime, as a by-product
Retention	Long, and set by policy	Short, and set by cost
Access	Restricted, and itself auditable	Broad — engineers read it daily
Completeness	Intended to be complete for its scope	Sampled, dropped, rotated
If you lose it	You cannot evidence your governance	You lose situational awareness

Design intent, not observable: every row above. The split between the two columns is the architecture's intent, and the retention, access and completeness properties are how a deployment is built rather than something you can measure from your integration. Read "long" and "restricted" as the shape to confirm with your operator, not as figures you were given.

Evidence is a product artifact; telemetry is an operational one. The governance record — approvals, holds, refusals, the decisions and who made them — is the material an auditor is shown, retained for as long as an auditor may ask for it. Telemetry is how the platform is run, and it is allowed to be lossy in ways evidence is not.

Check that your own retention obligations are met by the evidence store rather than by a log pipeline. If you have a requirement to retain a record of governance decisions for a period, it is the first column that satisfies it, and its retention is a policy you should confirm rather than a property you can assume from the fact that you can see log lines.

What you can check from outside: whether the governance services are responding at all. `api:GET /api/v1/trust/health` and `api:GET /api/v1/compliance/health` are the operations this client declares for that question — that they exist and that you can call them is verifiable from your own machine, and what a healthy response actually reports is not. They are a reasonable first call in any diagnostic sequence, because before you debug a refusal it is worth knowing whether the service that would have made the decision is up.

Be precise about what a health check establishes, though, because it is easy to over-read. A healthy response means the service is responding. It does not mean governance is configured the way you intended, that your envelopes are the ones you think, or that any particular control is enforced. Those are different questions with different instruments, and treating a green health check as evidence for them is how a misconfiguration survives a monitoring system that was watching the wrong thing.

Capacity: what scales, and what does not

Scales with demand	Does not scale
Application service replicas	The relational database's write throughput
Node pool size (autoscaling)	The vector extension's index build cost
Cache capacity	Retention requirements — evidence grows and is kept
Job execution concurrency	Storage growth, which is monotonic

Design intent, not observable: the directions above — which way each limit moves when it is reached — as well as the thresholds themselves. That storage only grows, and that write throughput scales up rather than out, are design properties of the reference shape rather than measurements you were handed.

The row that catches people is storage. Evidence and artifacts are retained on purpose, so object storage grows and never shrinks. That is the intended behaviour rather than a leak, and it means a capacity plan that assumes a steady state will be wrong in the only direction that matters.

Which limits are structural, and therefore worth designing around rather than arguing with:

- **Database write throughput** is bounded by the managed instance's size. It scales up, not out, and it does so on a change request rather than automatically.
- **Vector index maintenance** is the most expensive routine database operation, and it is the one most sensitive to corpus size. Adding knowledge is not free at the storage layer.
- **Connection counts** are bounded by the database, and every additional service replica consumes some. More replicas is not unconditionally more throughput, and past a point it is less.

Design intent, not observable: the specific thresholds. The reference shape says what scales and what does not; only your operator can say at what point each one bites, and that is a better question to ask before a launch than after.

Disaster recovery: objectives, not guarantees

Two numbers define a recovery posture, and both are **things an operator chooses**, not properties a platform has:

- **Recovery point objective** — how much data loss is tolerable. "One hour" means a restore may lose the last hour of changes.
- **Recovery time objective** — how long recovery may take. "Four hours" means four hours of degraded or absent service is an accepted outcome.

They are targets bought with cost and complexity, so different deployments will have chosen differently, and neither is more correct in the abstract.

What to ask your operator — and these are the questions, not a preamble to them:

1. What are the recovery point and recovery time objectives for this deployment?
2. When was a restore last **performed** — not configured, performed — and what did it demonstrate?
3. Where does a restore land, and does it land somewhere you can validate before it replaces anything?
4. Who decides to invoke a recovery, and how are you told?

Question 2 is the one that matters. As 07.5 puts it, a backup nobody has restored is a hypothesis, and a recovery objective that has never been tested against is a number rather than a capability.

UNVERIFIED — and this is the chapter's largest one: every objective, policy and threshold above, for any particular deployment. A reference architecture can tell you what to ask and why it matters. It cannot tell you your operator's answers, and no amount of reading this part substitutes for the conversation.

Pre-production checklist

Run through this before you depend on a deployment. Each item is answerable, and together they cover the gaps this part has flagged.

From your side, at the API:

- ☐ The base URL points at the deployment you intend, and `api:GET /api/v1/auth/me` names the principal you expect.
- ☐ `api:GET /api/v1/auth/me/organizations` lists the organisations you expect — and no others.
- ☐ `api:GET /api/v1/trust/health` and `api:GET /api/v1/compliance/health` both respond.
- ☐ Your credential's scope is what you think: attempt one operation you expect to be **refused**, and confirm it is. A permission model you have only tested in the permissive direction is untested.
- ☐ Your error handling treats a refusal as a first-class outcome rather than an unexpected failure — a governance product refuses things as normal operation.
- ☐ You can revoke the credential, and you know who to tell if it leaks.

From your operator, before you commit:

- ☐ The recovery point and recovery time objectives, and when a restore was last actually performed.
- ☐ The egress posture: allowlisted or default-allow, and whether the agent-to-provider path is recorded.
- ☐ The upgrade policy: notice period, window, and whether you can request a freeze.
- ☐ The retention policy for audit evidence, against your own obligations.
- ☐ The support path and escalation route, in writing.

One item that is easy to skip and should not be: confirm that the person answering these questions is able to answer them. "I will find out" is a fine answer. A confident answer about a deployment the respondent has not inspected is not, and this part has marked a dozen claims as unverifiable-from-outside precisely so you can tell the two apart.

Next: Part 08 — The capability catalogue

Part 08 — The capability catalogue

Audience: the architect or evaluator who needs to know everything the platform can do. Parts 01 to 05 teach the working path — the sequence you follow to stand an organisation up and run work through it. Part 06 is the architecture, and part 07 is the deployment. None of them enumerates the surface. This part does.

What it is. A complete, grouped inventory of the capabilities Aegis exposes: what each one is for, which SDK module drives it, which operations it publishes, and how the pieces relate. It is organised by capability area rather than by module, because the question an architect arrives with is "*can it do X?*" and not "*what is in `modules/`?*" — so the areas are named for the work, and the module that serves each one is named inside it.

What it is not. A tutorial. Nothing here walks you through a first session or explains why an envelope exists — parts 01 and 02 do that, and this part links to them rather than repeating them. It is also not a substitute for 04.1 on how a call is made, or 04.2 on what a refusal means. Read this part to find out **what exists**; read those to find out **how to call it and what comes back**.

The surface is large. It is one client, one credential, and one base URL, and every capability below is reachable through the same object you configured in 01.3.

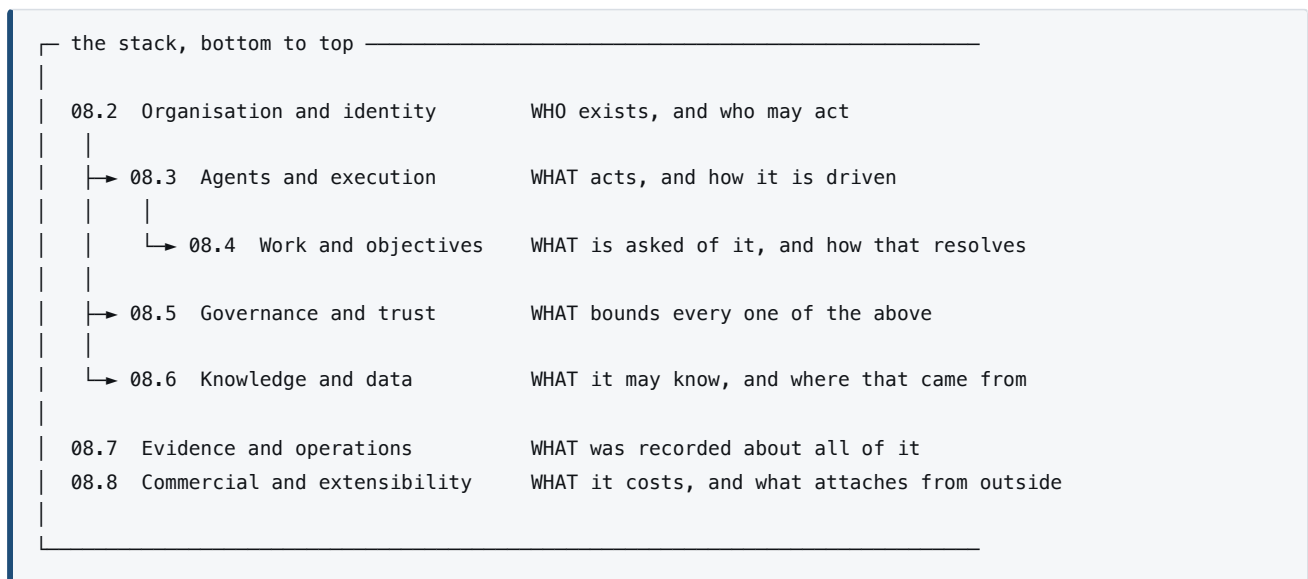
The chapters

#	Chapter	You leave able to
08.1	How to read this catalogue	Navigate the taxonomy, map a capability to the module that serves it, and check reach on your own deployment
08.2	Organisation and identity	Enumerate everything that models who the organisation is and who may act inside it
08.3	Agents and execution	Enumerate every kind of agent, how work reaches one, and every way one can be driven
08.4	Work and objectives	Enumerate the work-shaped objects — objectives, work units, requests, directives — and their lifecycles

#	Chapter	You leave able to
08.5	Governance and trust	Enumerate every control that bounds an agent, and every surface that explains or overrides one
08.6	Knowledge and data	Enumerate what the organisation knows, how it is classified and reviewed, and where it came from
08.7	Evidence and operations	Enumerate what the platform records about itself and how you get it out
08.8	Commercial and extensibility	Enumerate the metering, licensing, and third-party-integration surfaces

How the seven catalogue chapters relate

They are not seven topics chosen for convenience. They are the platform's own layering, read from the bottom up — each chapter's subject is the thing the next one acts on.



Read it as: 08.2 is the foundation, because a role is what a trust chain anchors to and a unit is what an envelope scopes. 08.3 and 08.6 both hang off it — an agent is linked to a role, and knowledge is classified against the clearance a role holds. 08.4 sits on 08.3 because an objective decomposes into work that an agent executes. **08.5 cuts across all of them**, which is why it is drawn to the side rather than in the chain: an envelope bounds a role, an agent, a session and a bridge alike. 08.7 and 08.8 are orthogonal — the first observes the whole stack, the second surrounds it.

The practical consequence for reading order: if you are checking Aegis against a requirements list, read 08.2 and 08.5 first. Most requirements that look like they are about agents turn out to be about the bounds on agents, and those are in 08.5.

Where this part sits relative to the others

Part 02 is the working reference — it shows you the sequence. Part 03 is how you add a capability the platform does not ship. Part 04 is the call mechanics. Part 06 is the architecture, and part 07 is the deployment.

This part is the **index of what exists**. It deliberately says less about each capability than the part that owns it, and more about the set as a whole. Where a chapter here names something part 02 teaches, it links there rather than re-explaining it.

The practical test is the question you are asking. *"How do I establish a trust chain?"* is 02.4 — it is a procedure. *"What does this error mean?"* is 04.2 — it is call mechanics. *"Does Aegis support cascade revocation, retention policies, and per-unit posture ceilings?"* is this part — it is an inventory question, and it is the one a procurement review, a security questionnaire, or a build-versus-buy decision actually asks.

The shape of every capability entry

Every area in 08.2 through 08.8 is presented the same way, so you can scan for the one you need rather than reading through:

1. **A paragraph** saying what the capability is for and what it is not.
2. **A capability table** — one row per distinct thing it can do.
3. **The SDK entry point**, as a resolvable symbol.
4. **The operations**, as resolvable routes.
5. **A short example** in the house style, so the call shape is concrete.

Where a capability has a lifecycle — a thing that is created, moved through states, and retired — the states are given as a table rather than as prose, because a state you did not know existed is the one that will surprise you in production.

Next: 08.1 — How to read this catalogue

08.1 — How to read this catalogue

This chapter is the key to the six that follow. It gives you the taxonomy the catalogue is organised by, the rule for mapping a capability onto the module that serves it, and — the part worth doing before you trust any of it — the commands that check what **your** deployment actually exposes.

It is short on purpose. The value of this part is in 08.2 through 08.8; this page exists so those chapters can be scanned rather than read, and so that a capability you cannot find is a capability you can rule out rather than one you are unsure about.

Everything in this part is reachable from one object. There is no second client, no admin SDK, no separate management plane.

The three kinds of thing in the catalogue

Every entry in this part is one of three kinds, and knowing which you are looking at tells you where the failure will show up.

kind	what it is	how it fails
Record	Something the platform stores and you address by id — a role, an agent, a policy	Fails at create time, loudly, with a validation error naming the field
Lifecycle	A record that moves through states — a bridge, an objective, a posture	Fails at a transition: the call returns a refusal naming the state it was in
Projection	A computed view over other records — effective constraints, an access matrix, a trust score	Never fails; returns a value computed from whatever inputs exist, which is why an empty input set reads as a permissive answer

The third row is the one to internalise. A projection over an incomplete organisation returns successfully and looks authoritative — an effective-envelope call against a role whose ancestors carry no envelopes returns a shape, not an error. When a governance answer surprises you, check whether you are reading a projection and what it was computed from.

Two surfaces, two sizes — and both numbers are correct

There are two different denominators in this catalogue and an evaluator will meet both. They answer different questions, and treating either as "the size of Aegis" produces a wrong conclusion.

surface	size	what it is
What the platform serves	~1,254 authored operations	1,133 are HTTP route operations a running deployment lists; the rest are reached through other governed surfaces

surface	size	what it is
What this client declares	933 operations	The operations the SDK has a method for — the anchorable set

The client is deliberately smaller than the platform. It covers the paths an architect drives; the remainder are served over HTTP and reached directly. So a capability described here without an anchored route is not a gap in the platform and not a defect in the client — it is a surface you call the other way, and 04.1 is how.

Every `api:` anchor in this part comes from the 933. Where this catalogue describes something the client has no method for, it says so and routes you to part 04 rather than inventing an anchor.

⚠ **A route listing you generate yourself will not match these numbers.** This catalogue counts operations, not per-method variants: an introspection dump counts each HTTP verb on a path separately and includes framework-generated endpoints, so it reads higher. **Authored operations is the denominator a catalogue should use**, and it is the one above.

The taxonomy, and why it is not the module list

The catalogue is grouped by **what the capability is for**, not by which module implements it. Those two groupings genuinely differ, and following the module layout would scatter related capabilities across chapters.

chapter	the question it answers	the areas inside it
08.2	Who exists, and who may act?	Organisations, units, roles, teams, users, SSO, invitations, API keys, RBAC
08.3	What acts, and how is it driven?	Agents of every kind, pools, sessions, pipelines, skills, tools, A2A, MCP
08.4	What is being asked, and how does it resolve?	Objectives, work units, requests, directives, decisions, escalation
08.5	What bounds all of it?	Trust chains, postures, envelopes, clearances, policy, bridges, bypass
08.6	What may it know, and where did that come from?	Knowledge, categories, reviews, connectors, classification, lineage
08.7	What was recorded?	Audit, compliance, analytics, metrics, notifications, webhooks, settings
08.8	What does it cost, and what attaches externally?	Billing, licensing, applications, external agents, gateways, surfaces

The eighteen capability areas, and where each one is

The seven catalogue chapters group eighteen distinct capability areas. If you arrived with a requirements list organised by area rather than by chapter, this is the index to read:

#	capability area	chapter
1	Identity, access and tenancy	08.2
2	PACT organisational modelling	08.2
3	Trust plane — chains, delegation, revocation	08.5
4	Trust posture and the verification gradient	08.5
5	Constraint envelopes, clearance and compartments	08.5
6	Bridges and information barriers	08.5
7	Agents, pools and the agent catalogue	08.3
8	Objectives, work and sessions	08.4
9	Human-on-the-loop — approvals, escalations, interventions	08.4
10	Emergency controls and kill switch	08.5
11	Knowledge, retrieval and ingestion	08.6
12	Connectors, integrations, A2A and MCP	08.3
13	Execution runtime and LLM governance	08.3 · 08.8
14	Audit, compliance and independent assurance	08.7
15	Observability, analytics and drift	08.7
16	Commercial — licensing, billing, entitlement	08.8
17	Platform administration and extensibility	08.8
18	Developer surface	08.1 · part 04

Five of the eighteen areas land in 08.5, which is why that chapter is the longest and why it is the one to read first against a governance requirement. Areas 3 to 6 and 10 are all controls, and a requirements list that separates them is describing one apparatus from five angles.

One capability appears in two chapters when two chapters genuinely own a piece of it, and the entry says so. Trust posture is the clearest case: the *agent* side of it is in 08.3 because posture is what decides whether an agent may act autonomously, and the *governance* side is in 08.5 because posture is one of the controls an architect sets. Neither chapter repeats the other; each links across.

Mapping a capability to the module that serves it

Every area names its entry point as a resolvable symbol. Two shapes appear, and the difference is only how the SDK organises itself:

shape	example	reached as
Top-level module	<code>sdk:aegis_sdk.modules.pools.PoolModule</code>	<code>client.pools</code>
Namespaced module	<code>sdk:aegis_sdk.trust.chains.ChainModule</code>	<code>client.trust.chains</code>

There are two namespaces, `trust` and `revenue`, plus a `dataflow` namespace for lineage. Everything else hangs directly off the client. When an entry names `sdk:aegis_sdk.trust.postures.PosturesModule`, the call is `await client.trust.postures.get(...)`.

`sdk:aegis_sdk.AgenticOSClient` is the object that presents all of them, and `sdk:aegis_sdk.ClientConfig` is where its endpoint and credential live.

```
from aegis_sdk import AgenticOSClient, ClientConfig

client = AgenticOSClient(
    ClientConfig(
        base_url="https://aegis.example.com",
        api_key="<your key>",
        timeout=30.0,
    )
)

roles = await client.roles.list()
chains = await client.trust.chains.list()
usage = await client.revenue.billing.get_usage()
```

Checking reach on your own deployment

A catalogue is a statement about the platform. What **your** credential can reach on **your** deployment is a different question, and it is one you can answer yourself rather than take on trust.

Two commands. Both ship inside the SDK you were given.

```
# What does this package declare, and which routes does my credential reach?
python -m aegis_sdk.coc.probe --base-url https://aegis.example.com --api-key "$AEGIS_API_KEY"

# Offline: does every module actually perform a request, or does one fabricate a response?
python -m aegis_sdk.coc.probe -- transports-only
```

The probe derives its operation set from this package's own call sites rather than from a written list, calls the subset that is safe to call blind — `GET` with no path parameters — and reports where your credential and the route disagree.

Read its exit code, not just its output. The four values are distinct and collapsing them is how a failed run gets banked as a clean one:

exit	meaning
0	Clean — every probed route behaved as the catalogue says
1	Findings — at least one route was refused, or a transport was simulated
2	Usage — the invocation itself was wrong; nothing was measured
3	Undetermined — a credential failed to authenticate, so nothing is known

 **Exit 3 is not a pass.** An expired token and a perfectly reachable API produce the same empty finding set, so the probe refuses to report a zero it did not earn. If you see `3`, fix the credential and run it again — a catalogue claim checked against an unauthenticated run has been checked against nothing.

The failure mode to expect: a probe run with a key that lacks a scope reports refusals across a whole family and reads like the capability is absent. It is not — it is unreachable *by that credential*. 04.3 is where that distinction is drawn, and it is worth reading before you conclude anything from a run with findings.

Checking the book against the package

The handbook ships with its own checker. Every route and symbol named in this part is resolved against the package you hold — so a capability named here that your build does not carry is caught mechanically rather than discovered at runtime.

```
python -m aegis_sdk.handbook.check # prints OK, or FAIL with a finding per problem
```

It resolves every `api:` route against the operation set derived from the package's call sites, resolves every `sdk:` symbol by import, and follows every internal link. Run it after upgrading the SDK: a route that stops resolving is the earliest signal that a capability moved.

Reading an operations list

Each area lists its operations in the anchored form, for example `api:GET /api/v1/organization-roles/{id}` — method, then path, with `{id}` standing for any path parameter. The lists are representative rather than exhaustive for the largest families: `/agents` alone publishes 47 operations and `/trust` another 47, and a chapter that printed every one would be a worse reference than one that names the shape and the notable exceptions.

Where a family is listed in full, the chapter says so. Where it is not, assume the CRUD five — list, create, get, update, delete — exist even when only the interesting ones are named, and use the SDK module's own method list as the enumeration. That is the honest granularity: the module is the surface you call, and its methods are the catalogue at its finest grain.

```
# The module's own methods are the finest-grained catalogue there is
import aegis_sdk.modules.pools as pools

print([m for m in dir(pools.PoolsModule) if not m.startswith("_")])
```

A note on vocabulary

Aegis implements four open standards, and their vocabulary is load-bearing throughout this part. Postures are always lowercase — `pseudo`, `supervised`, `shared_planning`, `continuous_insight`, `delegated` — and the five constraint dimensions are **Financial**, **Operational**, **Temporal**, **Data Access** and **Communication**. 02.3 and 06.3 are where those are taught; this part uses them as given.

Next: 08.2 — Organisation and identity

08.2 — Organisation and identity

This chapter enumerates everything that answers **who exists** and **who may act**. It is the foundation of the catalogue: a trust chain anchors to a role, an envelope scopes to a unit, an agent links to a role, and knowledge is cleared against the clearance a role holds. Nothing above this layer means anything without it.

Two distinct things live here and they are easy to conflate. The **organisation model** — units, roles, teams — is a description of the business: it persists, it is addressed positionally, and it exists whether or not anyone is employed. The **identity model** — users, credentials, SSO, permissions — is about the humans and machines that authenticate. A role is a seat; a user is an occupant; the seat exists first and outlives every occupant.

Get the organisation model right before anything else, because every control in 08.5 attaches to it.

Organisations

The tenant boundary. Everything else in this catalogue exists inside exactly one organisation, and no call crosses that line.

capability	what it does
Create	Stand up a new tenant with its own units, roles, agents and evidence
List / get	Enumerate organisations your principal is a member of
Membership	A user may belong to several; one is active per session
Switch	Move the active session to another organisation you are a member of
Settings	Per-organisation configuration — see 08.7

Entry points: `sdk:aegis_sdk.standup.organizations.OrganizationsModule` for the create/get/list path, and `sdk:aegis_sdk.modules.auth_users.AuthUsersModule` for membership and switching.

Operations: `api:POST /api/v1/organizations` · `api:GET /api/v1/organizations` · `api:GET /api/v1/organizations/{id}` · `api:GET /api/v1/auth/me/organizations` · `api:POST /api/v1/auth/me/switch-org`

```
org = await client.organizations.create(
    name="Northwind Logistics",
    slug="northwind",
)

memberships = await client.auth_users.auth_organizations()
await client.auth_users.auth_switch_org(organization_id=org["id"])
```


🚫 **An organisation is the isolation boundary, not a folder.** Two organisations on one deployment share no knowledge, no agents, no trust chains and no evidence. Creating a second organisation to separate two departments gives you two estates that cannot cooperate — use units for that, and read 06.3 before deciding.

Units — the containment tree

Units are departments and teams: the containers that give the organisation its shape. They nest, they carry posture ceilings, and they are what an isolation domain is drawn around.

capability	what it does
Create / update / delete	Standard record lifecycle on a unit
Tree	The whole hierarchy in one call, rather than walked one parent at a time
Roots	The top-level units, for rendering or for starting a walk
Ancestors / children / descendants	Relative navigation from any unit
Move	Re-parent a unit; the addresses of everything beneath it are recomputed
Summary	A rolled-up view of one unit — counts, roles, agents
Validate	Check the hierarchy for structural problems before relying on it
Posture ceiling	Cap the autonomy of every agent beneath this unit — see 08.5
Isolation domains	Declare which parts of the tree may not see one another

Entry point: `sdk:aegis_sdk.standup.units.OrganizationUnitsModule` for the basic lifecycle; `sdk:aegis_sdk.modules.org_standup.OrgStandupModule` carries the tree, navigation, move, ceiling and isolation surface.

Operations: `api:POST /api/v1/organization-units` · `api:GET /api/v1/organization-units` · `api:GET /api/v1/organization-units/tree` · `api:GET /api/v1/organization-units/roots` · `api:GET /api/v1/organization-units/validate` · `api:GET /api/v1/organization-units/{id}` · `api:GET /api/v1/organization-units/{id}/ancestors` · `api:GET /api/v1/organization-units/{id}/children` · `api:GET /api/v1/organization-units/{id}/descendants` · `api:GET /api/v1/organization-units/{id}/summary` · `api:POST /api/v1/organization-units/{id}/move` · `api:GET /api/v1/organization-units/{id}/posture-ceiling` · `api:PUT /api/v1/organization-units/{id}/posture-ceiling` · `api:GET /api/v1/organization-units/isolation-domains` · `api:PUT /api/v1/organization-units/isolation-domains` · `api:DELETE /api/v1/organization-units/{id}`

```

tree = await client.org_standup.get_unit_tree()
issues = await client.org_standup.validate_hierarchy()

await client.org_standup.move_unit(
    unit_id=treasury_id,
    new_parent_id=finance_id,
)

```

The failure mode is a move you do not re-validate. Re-parenting recomputes positional addresses beneath the moved unit, and an envelope or a trust chain written against the old address is now attached to a path that no longer describes the reporting line. Nothing errors — the records are still valid. Call `validate_hierarchy` after any structural change, and read the result before declaring the re-organisation done.

Roles — the accountability anchors

A role is a position, not a person. It persists across occupants, it is what an agent is linked to, and it is the anchor every trust chain terminates at.

capability	what it does
Create / update / delete	Standard record lifecycle
Hierarchy	The reporting tree, distinct from the unit containment tree
Reporting chain	Everyone above a given role, in order
Direct reports	Everyone immediately beneath it
Assign / unassign user	Seat a human in the role, or vacate it
Link / unlink agent	Bind an agent to the role's authority
Constraints	The envelope that bounds this role — see 08.5
Validate	Check one role's structural integrity
Without agents	Find roles that have no agent linked — the coverage gap query

Entry points: `sdk:aegis_sdk.standup.roles.OrganizationRolesModule` for create/get/list, `sdk:aegis_sdk.modules.roles.RolesModule` for the agent/user-linking surface, and `sdk:aegis_sdk.modules.org_standup.OrgStandupModule` for the reporting navigation.

Operations: `api:POST /api/v1/organization-roles` · `api:GET /api/v1/organization-roles` · `api:GET /api/v1/organization-roles/{id}` · `api:PUT /api/v1/organization-roles/{id}` · `api:DELETE /api/v1/organization-roles/{id}` · `api:GET /api/v1/organization-roles/without-agents` · `api:GET /api/v1/organization-roles/{id}/reporting-chain` · `api:GET /api/v1/organization-roles/{id}/direct-reports` · `api:GET /api/v1/organization-roles/{id}/constraints` · `api:GET /api/v1/organization-roles/{id}/validate` · `api:POST /api/v1/organization-roles/{id}/assign-user` · `api:POST /api/v1/organization-`

roles/{id}/unassign-user · api:POST /api/v1/organization-roles/{id}/link-agent · api:POST /api/v1/organization-roles/{id}/unlink-agent · api:GET /api/v1/roles/hierarchy · api:GET /api/v1/roles/{id}/agents · api:GET /api/v1/roles/{id}/users

The /roles family is an alias twin of /organization-roles carrying the same record through a shorter path, and it publishes its own full CRUD surface: api:GET /api/v1/roles · api:POST /api/v1/roles · api:GET /api/v1/roles/{id} · api:PUT /api/v1/roles/{id} · api:DELETE /api/v1/roles/{id} · api:POST /api/v1/roles/{id}/agents · api:DELETE /api/v1/roles/{id}/agents/{agent_id} · api:POST /api/v1/roles/{id}/users · api:DELETE /api/v1/roles/{id}/users/{user_id}

The two families address the same records, so a role created through one is visible through the other. Pick one and use it consistently — code that mixes them works correctly and reads as though two different things are being managed.

```
uncovered = await client.org_standup.get_roles_without_agents()

chain = await client.org_standup.get_reporting_chain(role_id=controller_id)
reports = await client.org_standup.get_direct_reports(role_id=cfo_id)

await client.org_standup.assign_user_to_role(
    role_id=controller_id,
    user_id=user["id"],
)
```

⚠ **The containment tree and the reporting tree are different trees.** A role lives inside a unit (containment) and reports to another role (reporting), and the two do not have to agree — a matrixed specialist sits in one department and reports into another. Authority and trust follow the **reporting** chain. Code that walks the unit tree to compute authority produces a plausible answer that is wrong for every matrixed role in the organisation.

Teams

A lighter-weight grouping than a unit: a working group with members, used where you need collaboration structure without a new node in the containment tree.

capability	what it does
Create / get / list	Standard record surface
Add member	Attach a user to the team

Entry point: sdk:aegis_sdk.standup.teams.TeamsModule .

Operations: api:POST /api/v1/teams · api:GET /api/v1/teams · api:GET /api/v1/teams/{id} · api:POST /api/v1/teams/{id}/members

Users

The humans. A user authenticates, holds permissions, occupies zero or more roles, and belongs to one or more organisations.

capability	what it does
Create / update / delete	Standard record lifecycle
List / get	Enumerate the directory
Me	The authenticated principal, as the deployment sees it
Reset password	Administrative credential reset
Permissions	What this principal may do — see RBAC below

Entry point: `sdk:aegis_sdk.modules.auth_users.AuthUsersModule` .

Operations: `api:POST /api/v1/users` · `api:GET /api/v1/users` · `api:GET /api/v1/users/{id}` · `api:PUT /api/v1/users/{id}` · `api:DELETE /api/v1/users/{id}` · `api:GET /api/v1/users/me` · `api:POST /api/v1/users/{id}/reset-password`

```
me = await client.auth_users.users_me()
directory = await client.auth_users.users_list(limit=100)
```

Authentication and sessions

How a principal proves who it is. Two credential kinds, and they are not interchangeable — 04.3 owns the distinction in full.

capability	what it does
Login / logout	Exchange credentials for a session bearer token
Refresh	Extend a session without re-presenting the password
Register	Self-service account creation, where the deployment permits it
Me	Which principal the presented credential resolves to
Verify email	Complete the email-confirmation step
Start trial	Onboarding entry point for a new tenant

Entry point: `sdk:aegis_sdk.auth.client.AuthModule` .

Operations: `api:POST /api/v1/auth/login` · `api:POST /api/v1/auth/logout` · `api:POST /api/v1/auth/refresh` · `api:POST /api/v1/auth/register` · `api:GET /api/v1/auth/me` · `api:GET /api/v1/auth/verify-email/{token}` · `api:POST /api/v1/onboarding/start-trial`

```
token = await client.auth.login(email="ops@example.com", password="<secret>")
principal = await client.auth.get_current_user()
```

api:GET /api/v1/auth/me is the first call to make when anything looks wrong. A deployment pointed at the wrong environment, and a credential resolving to a principal you did not expect, both look exactly like a correct setup until you ask. Establish which principal you are before debugging anything downstream.

API keys

Machine credentials, for integrations and unattended work.

capability	what it does
Create	Mint a key; the secret is returned once and not again
List / get	Enumerate keys and their metadata, never their secrets
Update	Change a key's description or scope set
Regenerate	Rotate the secret, keeping the key record
Revoke / delete	Withdraw a key immediately

Entry point: `sdk:aegis_sdk.auth.client.AuthModule` — the key surface lives alongside login.

Operations: `api:POST /api/v1/api-keys` · `api:GET /api/v1/api-keys` · `api:GET /api/v1/api-keys/{id}` · `api:PATCH /api/v1/api-keys/{id}` · `api:DELETE /api/v1/api-keys/{id}` · `api:POST /api/v1/api-keys/{id}/regenerate`

```
created = await client.auth.create_api_key(
    name="nightly-reconciliation",
    scopes=["objectives:read", "artifacts:read"],
)
# created carries the secret exactly once – store it now
```



An API key cannot reach a persona-gated route, and no scope configuration changes that.

Some surfaces require a human session because the decision they record is a human's. Widening a key's scopes to clear a refusal on one of those is the wrong fix — the probe in 08.1 is how you tell the two refusal kinds apart on your own deployment.

Single sign-on

Federated identity, where the organisation's own provider is the source of truth for who may log in.

capability	what it does
List providers	Which SSO provider types this deployment supports

capability	what it does
Create / update / delete connection	Configure a provider for this organisation
Initiate	Begin an OIDC-style sign-in flow
SAML initiate	Begin a SAML sign-in flow
Link account	Bind an existing local user to a federated identity
Identities	Enumerate the federated identities bound to a principal

Entry point: `sdk:aegis_sdk.modules.auth_users.AuthUsersModule` .

Operations: `api:GET /api/v1/sso/providers` · `api:GET /api/v1/sso/connections` · `api:POST /api/v1/sso/connections` · `api:GET /api/v1/sso/connections/{id}` · `api:PUT /api/v1/sso/connections/{id}` · `api:DELETE /api/v1/sso/connections/{id}` · `api:GET /api/v1/sso/initiate/{provider}` · `api:GET /api/v1/sso/saml/initiate/{provider}` · `api:GET /api/v1/sso/auth/{provider}` · `api:POST /api/v1/sso/link` · `api:GET /api/v1/sso/identities`

Invitations

Bringing a new human into an existing organisation.

capability	what it does
Create	Issue an invitation, returning a token to deliver out of band
List	Outstanding invitations and their states
Accept	Redeem an invitation and join the organisation

Operations: `api:POST /api/v1/invitations` · `api:GET /api/v1/invitations` · `api:POST /api/v1/invitations/{id}/accept`

RBAC — permissions and platform roles

Distinct from organisation roles, and the distinction matters. An **organisation role** is a seat in the business with authority and an envelope. An **RBAC role** is a bundle of permissions that decides which API operations a principal may call. A user has both, and they answer different questions.

capability	what it does
List permissions	Every permission the platform defines
List / get roles	The permission bundles available
Add / remove permission	Change what a bundle grants
Delete role	Retire a bundle
Get user permissions	The effective permission set for one principal

capability	what it does
Check permission	Ask directly whether a principal may perform one action

Entry point: `sdk:aegis_sdk.modules.governance.GovernanceModule` .

Operations: `api:GET /api/v1/rbac/permissions` · `api:GET /api/v1/rbac/roles` · `api:GET /api/v1/rbac/roles/{id}` · `api:POST /api/v1/rbac/roles/{id}` · `api:DELETE /api/v1/rbac/roles/{id}` · `api:DELETE /api/v1/rbac/roles/{id}/{permission}` · `api:GET /api/v1/rbac/users/{id}` · `api:POST /api/v1/rbac/check-permission` · `api:GET /api/v1/auth/permissions`

```
allowed = await client.governance.check_permission(
    user_id=user["id"],
    permission="objectives:create",
)
effective = await client.governance.get_user_permissions(user_id=user["id"])
```

The failure mode is assuming one gate. A call can be refused because RBAC does not grant the permission, *or* because the envelope bounds the action, *or* because clearance does not reach the data. Three independent controls, three different fixes. `check_permission` answers only the first — a `True` from it is not a prediction that the call will succeed.

Compiling and deploying an organisation

The organisation you describe and the organisation that runs are two artifacts, and compilation is what turns the first into the second. This is the surface that generates agents, trust chains and constraints in bulk rather than one at a time.

capability	what it does
Validate	Check a described structure before committing to it
Preview	See what a compilation would produce, without producing it
Compile	Turn the described organisation into its runtime form
Deploy	Put a compiled organisation into service
Deployment status	Where a deploy has reached
Generate agents	Create the role agents the structure implies
Generate trust chains	Create the delegation chains the reporting tree implies
Generate constraints	Create the envelopes the structure implies
Templates	Prebuilt organisation shapes to start from
Import YAML	Build an organisation from a declarative file
Rollback	Return to the previous deployed structure

capability	what it does
Verify integration	Confirm the deployed structure is internally consistent

Entry point: `sdk:aegis_sdk.modules.org_standup.OrgStandupModule` .

Operations: `api:POST /api/v1/organization-builder/validate` · `api:POST /api/v1/organization-builder/preview` · `api:POST /api/v1/organization-builder/compile` · `api:POST /api/v1/organization-builder/deploy` · `api:GET /api/v1/organization-builder/deployment-status` · `api:POST /api/v1/organization-builder/generate-agents` · `api:POST /api/v1/organization-builder/generate-trust-chains` · `api:POST /api/v1/organization-builder/generate-constraints` · `api:GET /api/v1/organization-builder/templates` · `api:GET /api/v1/organization-builder/templates/{id}` · `api:POST /api/v1/organization-builder/import-yaml` · `api:POST /api/v1/organization-builder/rollback/{id}` · `api:GET /api/v1/organization-builder/verify-integration`

```
issues = await client.org_standup.validate_structure()
preview = await client.org_standup.preview_compilation()
result = await client.org_standup.compile_organization()
await client.org_standup.generate_trust_chains()
```

This is the surface an architect uses most and reaches for last. The temptation is to create units and roles one call at a time, then wonder why no agent has a trust chain. Generation is a deliberate step: the structure does not produce its runtime objects until you ask it to, and a structurally perfect organisation with no generated chains runs nothing.

Ontology

The vocabulary layer — what this organisation calls its own concepts. A preset adapts the platform's generic nouns to an industry's.

capability	what it does
List presets	The vocabularies available
Apply preset	Adopt one for this organisation
Get / update config	Read or adjust the active vocabulary

Entry point: `sdk:aegis_sdk.standup.ontology.OntologyModule` .

Operations: `api:GET /api/v1/ontology/presets` · `api:POST /api/v1/ontology/preset/{id}` · `api:GET /api/v1/ontology/config/{id}` · `api:PUT /api/v1/ontology/config/{id}`

Summary — what this chapter covered

area	entry point	scale
Organisations	<code>standup.organizations</code> · <code>modules.auth_users</code>	3 + membership ops
Units	<code>standup.units</code> · <code>modules.org_standup</code>	16 operations
Roles	<code>standup.roles</code> · <code>modules.roles</code> · <code>modules.org_standup</code>	26 operations
Teams	<code>standup.teams</code>	4 operations
Users	<code>modules.auth_users</code>	7 operations
Auth and sessions	<code>auth.client</code>	9 operations
API keys	<code>auth.client</code>	6 operations
SSO	<code>modules.auth_users</code>	11 operations
Invitations	<code>modules.auth_users</code>	3 operations
RBAC	<code>modules.governance</code>	8 operations
Compile and deploy	<code>modules.org_standup</code>	13 operations
Ontology	<code>standup.ontology</code>	4 operations

Next: 08.3 — Agents and execution

08.3 — Agents and execution

This chapter enumerates **what acts** and **how it is driven**. It is the largest area in the catalogue:

`/agents` alone publishes 47 operations, and the agent kinds below are genuinely different objects rather than variations on one.

The distinction that organises everything here is between an **agent** — a durable, governed actor linked to a role — and an **execution** — one bounded run of one. An agent persists, carries a posture and an envelope, and appears in the audit spine; an execution is a single act by that agent, with inputs, a status, and a result. Confusing the two produces the most common category error in this part of the platform: treating an execution as if it carried authority of its own.

Every agent here is bounded by the controls in 08.5, without exception. This chapter says what they can do; that one says what stops them.

The agent kinds

Five distinct kinds. They are not interchangeable, and choosing the wrong one is a design error you feel later rather than at create time.

kind	what it is	linked to a role?
Agent	The general actor — configured, executable, versioned	Optionally
Role agent	An agent bound to an organisational role, acting with that role's authority	Yes, definitionally
Delegate agent	An agent acting on behalf of a role-holder under an explicit delegation	Yes
Task agent	A narrow agent for one repeated task shape	Usually not
Tool agent	A governed wrapper around a capability other agents invoke — see 08.8	Not directly
Pseudo agent	A human in an agent-shaped slot, reached through a channel	Yes

The pseudo agent is the one most often missed on a first reading of the platform. It exists so that a step requiring human judgement occupies the same position in a workflow as an automated one — the work routes to a person through a channel, their answer returns in the same shape, and the audit spine records it identically. 02.7 covers role agents in depth.

Agents — the core surface

capability	what it does
Create / update / delete	Standard record lifecycle
List / get	Enumerate and address agents
Duplicate	Copy an agent's configuration as a starting point
Accessible	The agents this principal may actually use — RBAC- and authority-capped
Status	A live status roll-up across agents
Summary	One agent's rolled-up state
Presets	Prebuilt agent configurations
Execute	Run the agent once, synchronously
Execute (stream)	Run it with incremental output — see 04.5
Execution status	Where a run has reached
List / get executions	The run history for one agent
Cancel execution	Stop a run in flight
Test draft	Exercise an unsaved configuration without creating the agent
Analyze objective	Ask which agents suit a stated objective
Versions	Create, list, get and roll back agent configuration versions
Contexts	Attach, update and remove durable context the agent reads
Tools	Attach, update and remove the capabilities the agent may invoke
Subagents	The agents this one may spawn
Connectors / data sources	The external data this agent is wired to — see 08.6
Trust posture	Read, move and evidence this agent's autonomy — see 08.5
Register from manifest	Create an agent from a declarative manifest, by value or by upload

Entry point: `sdk:aegis_sdk.core.agents.AgentsModule` , with
`sdk:aegis_sdk.core.agents.AgentVersionsModule` ,
`sdk:aegis_sdk.core.agents.AgentContextsModule` and
`sdk:aegis_sdk.core.agents.AgentToolsModule` for the three sub-surfaces.

Operations — the core lifecycle: `api:POST /api/v1/agents` · `api:GET /api/v1/agents` · `api:GET /api/v1/agents/{id}` · `api:PUT /api/v1/agents/{id}` · `api:DELETE /api/v1/agents/{id}` · `api:POST /api/v1/agents/{id}/duplicate` · `api:GET /api/v1/agents/accessible` · `api:GET /api/v1/agents/status` · `api:GET /api/v1/agents/presets` · `api:GET /api/v1/agents/{id}/summary`

Execution: `api:POST /api/v1/agents/{id}/execute` · `api:POST /api/v1/agents/{id}/execute/stream` · `api:GET /api/v1/agents/{id}/execute/status` · `api:GET /api/v1/agents/{id}/executions` · `api:GET /api/v1/agents/{id}/executions/{execution_id}` · `api:POST /api/v1/agents/{id}/executions/{execution_id}/cancel` · `api:POST /api/v1/agents/{id}/test-draft` · `api:POST /api/v1/agents/analyze-objective`

Composition: `api:GET /api/v1/agents/{id}/versions` · `api:GET /api/v1/agents/{id}/versions/{version_id}` · `api:POST /api/v1/agents/{id}/versions` · `api:POST /api/v1/agents/{id}/versions/{version_id}/rollback` · `api:GET /api/v1/agents/{id}/connectors` · `api:GET /api/v1/agents/{id}/data-sources` · `api:GET /api/v1/agents/{id}/contexts` · `api:GET /api/v1/agents/{id}/contexts/{context_id}` · `api:POST /api/v1/agents/{id}/contexts` · `api:PUT /api/v1/agents/{id}/contexts/{context_id}` · `api:DELETE /api/v1/agents/{id}/contexts/{context_id}` · `api:GET /api/v1/agents/{id}/tools` · `api:POST /api/v1/agents/{id}/tools` · `api:PUT /api/v1/agents/{id}/tools/{tool_id}` · `api:DELETE /api/v1/agents/{id}/tools/{tool_id}` · `api:GET /api/v1/agents/{id}/subagents` · `api:POST /api/v1/agents/register-from-manifest` · `api:POST /api/v1/agents/register-from-manifest/upload`

```
agent = await client.agents.create(
    name="reconciliation-analyst",
    agent_type="role",
    organization_role_id=controller_role_id,
)

result = await client.agents.execute(
    agent_id=agent["id"],
    inputs={"period": "2026-Q3", "ledger": "general"},
)

history = await client.agents.list_executions(agent_id=agent["id"], limit=20)
```

⚠ `api:GET /api/v1/agents` and `api:GET /api/v1/agents/accessible` answer different questions. The first lists what exists; the second lists what this principal may drive, capped by RBAC and by authority. Building a picker on the first produces a list where selecting some entries returns a refusal — and the refusal arrives at execute time, not at list time, which is the worst place for a user to meet it.

Delegate agents

An agent acting on behalf of a role-holder, under a delegation that is itself a governed record. Distinct from a role agent: a role agent **is** the role's actor, while a delegate agent acts **for** someone occupying it.

capability	what it does
Create / update / delete	Standard lifecycle
List / get	Enumerate delegations
For unit	Every delegate agent beneath one unit
Activate / deactivate	Turn a delegation on or off without destroying it
Activity	What this delegate has done
Health	Whether it is functioning
Trust chain	The delegation chain that authorises it
Dashboard summary	A roll-up across delegates

Entry point: `sdk:aegis_sdk.core.agents.AgentsModule` — the delegate surface is served by the same module.

Operations: `api:POST /api/v1/delegate-agents` · `api:GET /api/v1/delegate-agents` · `api:GET /api/v1/delegate-agents/{id}` · `api:PATCH /api/v1/delegate-agents/{id}` · `api:DELETE /api/v1/delegate-agents/{id}` · `api:GET /api/v1/delegate-agents/dashboard-summary` · `api:GET /api/v1/delegate-agents/for-unit/{unit_id}` · `api:POST /api/v1/delegate-agents/{id}/activate` · `api:POST /api/v1/delegate-agents/{id}/deactivate` · `api:GET /api/v1/delegate-agents/{id}/activity` · `api:GET /api/v1/delegate-agents/{id}/health` · `api:GET /api/v1/delegate-agents/{id}/trust-chain`

```
delegates = await client.agents.list_delegate_agents_for_unit(unit_id=finance_id)
health = await client.agents.get_delegate_agent_health(delegate_agent_id=d_id)
```

Deactivation is not revocation. A deactivated delegate stops acting; its trust chain still exists and its past acts remain attributed. If you need the authority withdrawn rather than paused, that is revocation and it lives in 08.5.

Task agents and specialists

Two narrower kinds. A **task agent** is scoped to one repeated task shape. A **specialist** is a reusable expert configuration that can be cloned into several agents.

capability	task agents	specialists
Create / update / delete	yes	yes
List / get	yes	yes
Test	yes	—
Clone	—	yes

Entry points: `sdk:aegis_sdk.modules.task_agents.TaskAgentsModule` and `sdk:aegis_sdk.modules.specialist_system.SpecialistSystemModule`.

Operations: `api:POST /api/v1/task-agents` · `api:GET /api/v1/task-agents` · `api:GET /api/v1/task-agents/{id}` · `api:PUT /api/v1/task-agents/{id}` · `api:DELETE /api/v1/task-agents/{id}` · `api:POST /api/v1/task-agents/{id}/test` · `api:GET /api/v1/specialist-system/specialists` · `api:POST /api/v1/specialist-system/specialists` · `api:GET /api/v1/specialist-system/specialists/{id}` · `api:PUT /api/v1/specialist-system/specialists/{id}` · `api:DELETE /api/v1/specialist-system/specialists/{id}` · `api:POST /api/v1/specialist-system/specialists/{id}/clone`

Pseudo agents — humans in an agent-shaped slot

The capability that makes a human step indistinguishable, structurally, from an automated one. Work routes to a person through a configured channel; their response returns in the agent response shape.

capability	what it does
Create / update / delete	Configure the human-backed slot, its operators and its channel
List / get	Enumerate them
Test channel	Confirm the channel delivers before relying on it
Route task	Send a task to the human behind the slot
Requests	The queue of outstanding human requests
Claim / respond / reassign	The operator-side lifecycle of one request
Stats	Queue depth and response behaviour

Entry point: `sdk:aegis_sdk.modules.pseudo_agents.PseudoAgentsModule`, with the request-queue surface on `sdk:aegis_sdk.modules.pools.PoolsModule`.

Operations: `api:POST /api/v1/work-units/pseudo` · `api:GET /api/v1/work-units/pseudo` · `api:GET /api/v1/work-units/pseudo/{id}` · `api:PATCH /api/v1/work-units/pseudo/{id}` · `api:DELETE /api/v1/work-units/pseudo/{id}` · `api:POST /api/v1/work-units/pseudo/test-channel` · `api:POST /api/v1/work-units/pseudo/{id}/route` · `api:GET /api/v1/pseudo-requests` · `api:GET /api/v1/pseudo-requests/stats` · `api:POST /api/v1/pseudo-requests/{id}/claim` · `api:POST /api/v1/pseudo-requests/{id}/respond` · `api:POST /api/v1/pseudo-requests/{id}/reassign`

```
await client.pseudo_agents.test_channel(
    pseudo_agent_id=approver_slot_id,
)
await client.pseudo_agents.route_task(
    pseudo_agent_id=approver_slot_id,
    task={"summary": "Confirm Q3 write-off", "amount_cents": 4_200_00},
)
```

🚫 **Test the channel before the work depends on it.** A pseudo agent whose channel is misconfigured accepts routed tasks and queues them against a person who is never notified. Nothing errors, the queue grows, and the symptom is work that sits in `pending` forever with no failure anywhere to point at.

Pools — how work finds an actor

A pool is a set of agents or operators that work is distributed across, with claiming, escalation, and load management. This is the routing layer between a request and whoever handles it.

capability	what it does
Create / update / delete	Standard lifecycle
Members	Add, list and remove the actors in the pool
Summaries / user pools	Roll-ups, and the pools one user belongs to
Pending tasks	What is waiting, globally or per pool
Claim / release task	Take a task, or put it back
Extend / cancel claim timeout	Manage a held claim's clock
Pause / resume / drain	Stop intake, restart it, or let the pool finish and empty
Health / statistics / metrics	Operational state of the pool
Load distribution / rebalance	See how work is spread, and redistribute it
Escalation config	What happens when a task is not handled in time
Pending escalations / escalate	The escalation queue and manual escalation
Acknowledge / history / stats	The operator side of escalation
Utilization	How heavily the pool is used

Entry point: `sdk:aegis_sdk.modules.pools.PoolsModule` . Agent-only pools have a narrower surface at `sdk:aegis_sdk.modules.agent_pools.AgentPoolsModule` .

Operations — pool lifecycle: `api:POST /api/v1/pools` · `api:GET /api/v1/pools` · `api:GET /api/v1/pools/{id}` · `api:PUT /api/v1/pools/{id}` · `api:DELETE /api/v1/pools/{id}` · `api:GET /api/v1/pools/summaries` · `api:GET /api/v1/pools/user/{user_id}` · `api:GET /api/v1/pools/{id}/members` · `api:POST /api/v1/pools/{id}/members` · `api:DELETE /api/v1/pools/{id}/members/{member_id}`

Work distribution: `api:GET /api/v1/pools/tasks/pending` · `api:GET /api/v1/pools/{id}/tasks/pending` · `api:POST /api/v1/pools/tasks/{task_id}/claim` · `api:POST /api/v1/pools/tasks/{task_id}/release` · `api:POST /api/v1/pools/{id}/operations/extend_timeout` · `api:POST`

`/api/v1/pools/{id}/operations/cancel_timeout` · `api:POST`
`/api/v1/pools/{id}/operations/rebalance` · `api:GET`
`/api/v1/pools/{id}/operations/load_distribution`

Operational control: `api:POST /api/v1/pools/{id}/pause` · `api:POST /api/v1/pools/{id}/resume` ·
`api:POST /api/v1/pools/{id}/drain` · `api:GET /api/v1/pools/{id}/health` · `api:GET`
`/api/v1/pools/{id}/metrics` · `api:GET /api/v1/pools/{id}/statistics`

Agent pools: `api:GET /api/v1/agent-pools` · `api:POST /api/v1/agent-pools` · `api:GET`
`/api/v1/agent-pools/{id}` · `api:PUT /api/v1/agent-pools/{id}` · `api:DELETE /api/v1/agent-`
`pools/{id}` · `api:GET /api/v1/agent-pools/{id}/members` · `api:POST /api/v1/agent-`
`pools/{id}/members` · `api:DELETE /api/v1/agent-pools/{id}/role-members/{member_id}`

```
pending = await client.pools.list_pool_pending_tasks(pool_id=trriage_pool_id)
await client.pools.claim_task(task_id=pending["tasks"][0]["id"])

await client.pools.drain_pool(pool_id=trriage_pool_id) # finish, then stop
health = await client.pools.get_pool_health(pool_id=trriage_pool_id)

# Agent-only pools are the narrower surface, addressed separately
roster = await client.agent_pools.list_members(pool_id=analysis_pool_id)
```

Pause and drain are different, and the difference bites during an incident. Pause stops the pool acting on anything, including work already claimed. Drain stops new intake and lets in-flight work finish. Reaching for pause when you meant drain abandons work mid-execution; reaching for drain when you meant pause lets the thing you are trying to stop keep running to completion.

Sessions — stateful, multi-turn work

A session is a durable conversation with an agent: it accumulates messages, artifacts, context and memory, and it can spawn subagents.

capability	what it does
Create from task	Start a session from an existing task
Initialize agent	Start one by naming the agent directly
Get / list / my	Address sessions, all or your own
Messages	Read the transcript, or send into it
Stream messages / events	Consume the session live
Artifacts	Attach outputs to the session, or read them
Context / memory	What the session is carrying
Subagents	Spawn and enumerate child agents inside the session
Pause / resume / end	Session lifecycle control

Entry point: `sdk:aegis_sdk.execution.sessions.SessionsModule` .

Operations: `api:POST /api/v1/sessions/from-task` · `api:POST /api/v1/sessions/initialize-agent` · `api:GET /api/v1/sessions` · `api:GET /api/v1/sessions/my` · `api:GET /api/v1/sessions/{id}` · `api:GET /api/v1/sessions/{id}/messages` · `api:POST /api/v1/sessions/{id}/messages` · `api:GET /api/v1/sessions/{id}/stream` · `api:GET /api/v1/sessions/{id}/artifacts` · `api:POST /api/v1/sessions/{id}/artifacts` · `api:GET /api/v1/sessions/{id}/context` · `api:GET /api/v1/sessions/{id}/memory` · `api:GET /api/v1/sessions/{id}/subagents` · `api:POST /api/v1/sessions/{id}/subagents` · `api:POST /api/v1/sessions/{id}/pause` · `api:POST /api/v1/sessions/{id}/resume` · `api:POST /api/v1/sessions/{id}/end`

```
session = await client.sessions.create_from_task(task_id=task["id"])
await client.sessions.send_message(session_id=session["id"], content="Proceed.")

async for event in client.sessions.stream_events(session_id=session["id"]):
    handle(event)
```

A session that is never ended is not merely untidy. It continues to hold context and count against whatever the deployment budgets sessions by. End them explicitly; `pause` is for work you intend to resume.

Pipelines — declarative multi-step execution

Where a session is conversational, a pipeline is a graph: nodes, connections, and a deterministic execution order.

capability	what it does
Create / update / delete	Standard lifecycle
Duplicate	Copy a pipeline as a starting point
Save graph	Replace the node-and-edge structure
Node types	Which node kinds this deployment offers
Validate	Check the graph before running it
Execute	Run it
Executions	List and get runs
Cancel execution	Stop a run in flight

Entry point: `sdk:aegis_sdk.core.pipelines.PipelinesModule` .

Operations: `api:POST /api/v1/pipelines` · `api:GET /api/v1/pipelines` · `api:GET /api/v1/pipelines/{id}` · `api:PUT /api/v1/pipelines/{id}` · `api:DELETE /api/v1/pipelines/{id}` · `api:POST /api/v1/pipelines/{id}/duplicate` · `api:PUT /api/v1/pipelines/{id}/graph` · `api:GET /api/v1/pipelines/node-types` · `api:POST`

```
/api/v1/pipelines/{id}/validate · api:GET /api/v1/pipelines/{id}/executions · api:GET
/api/v1/pipelines/{id}/executions/{execution_id} · api:POST
/api/v1/pipelines/{id}/executions/{execution_id}/cancel
```

Standalone execution control sits alongside: `api:POST /api/v1/executions/start` · `api:GET /api/v1/executions/{id}` · `api:POST /api/v1/executions/{id}/stop` · `api:GET /api/v1/executions/history/{id}`

Registered workflows — the named, reusable graphs a deployment publishes — are read-only from the client: `api:GET /api/v1/workflows` · `api:GET /api/v1/workflows/{id}`. A workflow is what a pipeline is built from; you enumerate them to discover what the deployment offers, and compose pipelines against them.

```
await client.pipelines.validate(pipeline_id=p_id)
run = await client.pipelines.execute(pipeline_id=p_id, inputs={"batch": "2026-09"})
```

Skills

Reusable capability definitions assigned to agents — what an agent knows how to do, as distinct from the tools it may call.

capability	what it does
Create / update / delete	Standard lifecycle
List / get / by name	Address skills
Duplicate	Not served — <code>get</code> then <code>create</code> instead
Agent skills	List and assign the skills one agent holds

Entry point: `sdk:aegis_sdk.core.skills.SkillsModule`.

Operations: `api:POST /api/v1/skills` · `api:GET /api/v1/skills` · `api:GET /api/v1/skills/{id}` · `api:PUT /api/v1/skills/{id}` · `api:DELETE /api/v1/skills/{id}` · `api:GET /api/v1/skills/agents/{agent_id}/skills` · `api:POST /api/v1/skills/agents/{agent_id}/skills`

⚠ Skills have **no** duplicate route, unlike agents and pipelines, so `skills.duplicate()` raises `sdk:aegis_sdk.UnsupportedOperationError` rather than answering a 404.

Tools

The invocable capabilities, and the validation surface for them. 03.1 is how you write one.

capability	what it does
List / get / count	Enumerate what is available
Categories	How tools are grouped

capability	what it does
Danger levels	The risk classification a tool carries
Validate	Check a tool definition
Batch	Resolve several tools in one call

Entry point: `sdk:aegis_sdk.modules.tools.ToolsModule` .

Operations: `api:GET /api/v1/tools` · `api:GET /api/v1/tools/{id}` · `api:GET /api/v1/tools/count` · `api:GET /api/v1/tools/categories` · `api:GET /api/v1/tools/danger-levels` · `api:POST /api/v1/tools/validate` · `api:POST /api/v1/tools/batch`

```
available = await client.tools.list(category="data")
levels = await client.tools.danger_levels()
```

Read `danger_levels` before granting a tool to an agent. The classification is what an envelope's Operational dimension is written against, and a tool attached without regard to it is bounded by whatever the envelope happened to say.

Agent-to-agent (A2A)

Discovery and invocation across agent boundaries — including agents this deployment does not own.

capability	what it does
Discover	Find agents matching a capability requirement
Model card	Read one agent's published capability description
Workers	The workers behind an agent
Route	Pick the right agent for a request without invoking it
Invoke	Call one agent
Orchestrate	Drive a multi-agent interaction

Entry point: `sdk:aegis_sdk.modules.a2a.A2AModule` .

Operations: `api:POST /api/v1/a2a/discover` · `api:GET /api/v1/a2a/agent/{agent_id}/card` · `api:GET /api/v1/a2a/workers/{agent_id}` · `api:POST /api/v1/a2a/route` · `api:POST /api/v1/a2a/invoke` · `api:POST /api/v1/a2a/orchestrate`

```
candidates = await client.a2a.discover(capability="invoice-extraction")
chosen = await client.a2a.route(task={"kind": "extract", "document_id": doc_id})
result = await client.a2a.invoke(agent_id=chosen["agent_id"], payload=payload)
```

MCP — model context protocol

Registering external MCP servers and binding their tools into agents.

capability	what it does
Register / update / delete server	Manage an MCP server registration
List / get registrations	Enumerate registered servers
Bind / bind inline	Attach a server's tools to an agent, by reference or inline
List bindings / unbind	Manage those attachments
Datasets	Create, list, get, update and delete MCP-exposed datasets

Entry point: `sdk:aegis_sdk.modules.mcp.McpModule` .

Operations: `api:GET /api/v1/datasets` · `api:POST /api/v1/datasets` · `api:GET /api/v1/datasets/{id}` · `api:PUT /api/v1/datasets/{id}` · `api:DELETE /api/v1/datasets/{id}`

```
reg = await client.mcp.register_server(
    name="ledger-mcp",
    url="https://mcp.internal.example.com",
    transport="http",
)
await client.mcp.bind(agent_id=agent["id"], registration_id=reg["id"])
```

The three MCP surfaces, and which one you are on

MCP appears at three places in Aegis, and they are different surfaces with different audiences. Confusing them is the reason a tool you can see from one side is absent from the other.

surface	what it exposes	audience
External MCP serving	3 tools — <code>query_knowledge</code> , <code>list_datasets</code> , <code>read_dataset</code> , each with per-tool authorization	An outside MCP client
The enterprise MCP server	Tools for callers inside the organisation, reached under governance rather than from outside	Governed enterprise callers
The internal tool bridge	10 tools — <code>decompose_objective</code> , <code>route_task</code> , <code>escalate</code> , <code>delegate_to_agent</code> , <code>query_knowledge</code> , <code>check_directive</code> , <code>create_deliverable</code> , <code>create_presentation</code> , <code>submit_upstream</code> , <code>invoke_task_agent</code>	Agents inside the platform

The external surface is deliberately narrow. Three read-shaped tools, each separately authorized — it is knowledge retrieval for an outside client, not a remote control for the organisation. The ten-tool internal bridge is what agents inside the platform use to decompose work, escalate, and delegate, and it is not reachable from outside.

`query_knowledge` appears on both the external surface and the internal bridge, and it is the same capability under the same clearance rules — an external caller does not get a wider view of knowledge than an internal agent with the same clearance would. **The failure mode to expect is an outside integrator asking why they cannot call `delegate_to_agent`** : it is an internal bridge tool, it was never externally exposed, and no authorization change reaches it.

The agentic dashboard

The operator-facing roll-up across everything in this chapter — the inbox, the activity feed, and agent drift.

capability	what it does
Dashboard stats	The headline numbers
Inbox	What is waiting for this operator
Activity feed	What has been happening
Get / claim / validate request	The operator lifecycle on one request
Pools	Pool state from the operator's view
Agent drift	Whether an agent's behaviour has moved from its baseline
Escalate / recover drift	Act on a drifting agent
Drift alerts	The queue of drift signals
Reasoning traces	Why an agent decided what it decided

Entry point: `sdk:aegis_sdk.modules.agentic_dashboard.AgenticDashboardModule` .

Operations: `api:GET /api/v1/agentic/dashboard/stats` · `api:GET /api/v1/agentic/dashboard/activity-feed` · `api:GET /api/v1/agentic/inbox` · `api:GET /api/v1/agentic/pools` · `api:GET /api/v1/agentic/pools/{id}` · `api:GET /api/v1/agentic/requests/{id}` · `api:POST /api/v1/agentic/requests/{id}/claim` · `api:POST /api/v1/agentic/requests/{id}/validate` · `api:GET /api/v1/agentic/agents/{agent_id}/drift` · `api:POST /api/v1/agentic/agents/{agent_id}/drift/escalate` · `api:POST /api/v1/agentic/agents/{agent_id}/drift/recover` · `api:GET /api/v1/agentic/drift/alerts` · `api:GET /api/v1/reasoning-traces`

```
inbox = await client.agentic_dashboard.inbox()
drift = await client.agentic_dashboard.get_agent_drift(agent_id=agent["id"])
traces = await client.agentic_dashboard.list_reasoning_traces(agent_id=agent["id"])
```

In the console: the inbox, the activity feed and the drift alerts are the three panels an operator lives in. 05.3 covers the inbox screen.

Summary — what this chapter covered

area	entry point	scale
Agents	<code>core.agents</code>	47 operations
Delegate agents	<code>core.agents</code>	12 operations
Task agents	<code>modules.task_agents</code>	6 operations
Specialists	<code>modules.specialist_system</code>	6 operations
Pseudo agents	<code>modules.pseudo_agents</code> · <code>modules.pools</code>	12 operations
Pools	<code>modules.pools</code> · <code>modules.agent_pools</code>	32 operations
Sessions	<code>execution.sessions</code>	17 operations
Pipelines	<code>core.pipelines</code>	16 operations
Skills	<code>core.skills</code>	8 operations
Tools	<code>modules.tools</code>	7 operations
A2A	<code>modules.a2a</code>	6 operations
MCP and datasets	<code>modules.mcp</code>	5 + binding
Agentic dashboard	<code>modules.agentic_dashboard</code>	13 operations

Next: 08.4 — Work and objectives

08.4 — Work and objectives

This chapter enumerates **what is being asked** and **how it resolves**. Where 08.3 catalogued the actors, this catalogues the work that reaches them — and the distinction is sharper here than anywhere else in the platform, because Aegis carries several work-shaped objects that look similar in a listing and behave completely differently.

Four of them, and choosing the wrong one is a design error rather than a preference:

object	what it is	who creates it
Objective	A stated outcome, decomposed by the platform into work	A human, usually
Work unit	A reusable, versioned unit of executable work	An architect
Request	One piece of work assigned to one actor	Decomposition
Directive	A standing instruction that binds until acknowledged or superseded	A role-holder

An objective is a *goal*; a work unit is a *capability*; a request is an *assignment*; a directive is an *instruction*. **The objective is the entry point for most work, and everything else in this chapter is either what it decomposes into or what constrains it.**

02.5 is the procedure. This is the inventory.

Objectives

The primary work surface. You state an outcome; the platform clarifies it, decomposes it into a task graph, executes it through agents, and stops to ask when it reaches something it may not decide alone.

capability	what it does
Create / update	State an objective, or revise it
List / get / recent	Enumerate and address objectives
Templates	Prebuilt objective shapes
Submit	Hand the objective to the platform to begin work
Clarify	The platform asks; you answer, structured or freeform
Clarification status	Whether it is waiting on you
Task graph	The decomposition — what it intends to do, and in what order
Trigger execution	Begin executing the graph
Implement	Drive the implementation phase

capability	what it does
Confirm implementation	Accept what was produced
Progress / progress stream	Where it has reached, polled or streamed
Pause / resume / stop	Lifecycle control on a running objective
Retry node	Re-run one failed step without restarting the objective
Decide	Answer a decision the objective is blocked on
Decisions	Every decision this objective has raised
Requests	The assignments it decomposed into
Complete / check completion / completion status	Finish it, or ask whether it can finish
Artifacts / download	What it produced
Participants	Who and what has been involved
Summary	The rolled-up view

Entry points: `sdk:aegis_sdk.execution.objectives.ObjectivesModule` for the core lifecycle, and `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` for clarification, the task graph and progress streaming.

Operations — lifecycle: `api:POST /api/v1/objectives` · `api:GET /api/v1/objectives` · `api:GET /api/v1/objectives/{id}` · `api:PUT /api/v1/objectives/{id}` · `api:GET /api/v1/objectives/recent` · `api:GET /api/v1/objectives/templates` · `api:POST /api/v1/objectives/{id}/submit` · `api:POST /api/v1/objectives/{id}/trigger-execution` · `api:POST /api/v1/objectives/{id}/implement` · `api:POST /api/v1/objectives/{id}/confirm-implementation` · `api:POST /api/v1/objectives/{id}/pause` · `api:POST /api/v1/objectives/{id}/resume` · `api:POST /api/v1/objectives/{id}/stop` · `api:POST /api/v1/objectives/{id}/complete` · `api:POST /api/v1/objectives/{id}/check-completion` · `api:GET /api/v1/objectives/{id}/completion-status`

Clarification and decisions: `api:POST /api/v1/objectives/{id}/clarify` · `api:POST /api/v1/objectives/{id}/clarify/trigger` · `api:POST /api/v1/objectives/{id}/clarify/respond` · `api:POST /api/v1/objectives/{id}/clarify/respond-freeform` · `api:GET /api/v1/objectives/{id}/clarification-status` · `api:POST /api/v1/objectives/{id}/decide` · `api:GET /api/v1/objectives/{id}/decisions`

Progress and output: `api:GET /api/v1/objectives/{id}/task-graph` · `api:GET /api/v1/objectives/{id}/progress` · `api:GET /api/v1/objectives/{id}/progress/stream` · `api:POST /api/v1/objectives/{id}/nodes/{node_id}/retry` · `api:GET /api/v1/objectives/{id}/requests` · `api:GET /api/v1/objectives/{id}/artifacts` · `api:GET /api/v1/objectives/{id}/download` · `api:GET /api/v1/objectives/{id}/participants` · `api:GET /api/v1/objectives/{id}/summary`

Two administrative overrides sit alongside the normal lifecycle and bypass it: `api:POST /api/v1/objectives/{id}/admin-status` sets an objective's status directly, and `api:POST /api/v1/objectives/{id}/admin-tasks` injects tasks into its graph. Both are recovery tools for an objective stuck in a state the normal transitions cannot leave — **an objective moved by `admin-status` has not done the work its new status implies**, so use them to unblock and then re-drive, never to mark something complete.

```
objective = await client.objectives.create(
    title="Close the Q3 books",
    description="Reconcile all ledgers, produce the trial balance, flag variances over $10k.",
)
await client.objectives.submit(objective_id=objective["id"])

status = await client.work_objectives.get_clarification_status(
    objective_id=objective["id"],
)
if status["pending"]:
    await client.work_objectives.respond_to_clarification(
        objective_id=objective["id"],
        question_id=status["questions"][0]["id"],
        answer="Use the consolidated ledger, not the per-entity ones.",
    )

graph = await client.work_objectives.get_task_graph(objective_id=objective["id"])
```

🛑 **Create and submit are two calls, and an unsubmitted objective does nothing.** `create` records the intent; `submit` hands it to the platform. An objective created and never submitted sits in the listing looking exactly like a live one, with no progress and no error to explain why — because nothing has gone wrong, nothing has started.

The clarification loop is the second place work silently stalls. The platform asks a question and waits; if nothing polls `api:GET /api/v1/objectives/{id}/clarification-status` and nothing answers, the objective is blocked indefinitely. It is not failed, so it does not appear in a failure query. Poll the status, or consume the progress stream, which carries the clarification event.

Work units

Reusable, versioned units of executable work. Where an objective is a one-off goal, a work unit is a capability you define once and run repeatedly — with version history, an execution configuration, and run records.

capability	what it does
Create / update / delete	Standard lifecycle
List / get / available	Enumerate, including which are available to run
Run / execute	Invoke it
Cancel execution	Stop a run in flight

capability	what it does
Runs	The execution history
Execution config	Read and adjust how it runs
Versions	Create, list, get, compare, restore and delete versions

Entry point: `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` .

Operations: `api:POST /api/v1/work-units` · `api:GET /api/v1/work-units` · `api:GET /api/v1/work-units/{id}` · `api:PUT /api/v1/work-units/{id}` · `api:DELETE /api/v1/work-units/{id}` · `api:GET /api/v1/work-units/available` · `api:POST /api/v1/work-units/{id}/run` · `api:POST /api/v1/work-units/{id}/execute` · `api:POST /api/v1/work-units/{id}/execute/cancel` · `api:GET /api/v1/work-units/{id}/runs` · `api:GET /api/v1/work-units/{id}/execution-config` · `api:PATCH /api/v1/work-units/{id}/execution-config` · `api:GET /api/v1/work-units/{id}/versions` · `api:POST /api/v1/work-units/{id}/versions` · `api:GET /api/v1/work-units/{id}/versions/{version_id}` · `api:GET /api/v1/work-units/{id}/versions/compare` · `api:POST /api/v1/work-units/{id}/versions/{version_id}/restore` · `api:DELETE /api/v1/work-units/{id}/versions/{version_id}`

```
unit = await client.work_objectives.create_work_unit(
    name="monthly-variance-report",
    description="Produce the variance report for one period.",
)
await client.work_objectives.create_work_unit_version(work_unit_id=unit["id"])

run = await client.work_objectives.run_work_unit(
    work_unit_id=unit["id"],
    inputs={"period": "2026-09"},
)
```

Version before you change a work unit that is in service. Editing in place changes the behaviour of every future run with no record of what it used to do, and a run record from last month then describes a unit that no longer exists in that form. Create a version, then edit.

Requests

One piece of work assigned to one actor. Requests are what objectives decompose into and what pools distribute.

capability	what it does
List / get	Enumerate and address requests
Claim / release	Take ownership, or give it back
Complete	Finish the request
Escalate	Raise it to whoever is above

capability	what it does
Findings	Record and read what the work discovered
Decompose	Break a request into smaller ones
Submit deliverable	Attach the output that satisfies it

Entry point: `sdk:aegis_sdk.execution.requests.RequestsModule` .

Operations: `api:GET /api/v1/requests` · `api:GET /api/v1/requests/{id}` · `api:POST /api/v1/requests/{id}/claim` · `api:POST /api/v1/requests/{id}/release` · `api:POST /api/v1/requests/{id}/complete` · `api:POST /api/v1/requests/{id}/escalate` · `api:GET /api/v1/requests/{id}/findings` · `api:POST /api/v1/requests/{id}/findings` · `api:POST /api/v1/objectives/{id}/requests/{request_id}/complete` · `api:POST /api/v1/objectives/{id}/requests/{request_id}/decompose` · `api:POST /api/v1/objectives/{id}/requests/{request_id}/submit-deliverable`

```
mine = await client.requests.list(assigned_to_me=True)
await client.requests.claim(request_id=mine["records"][0]["id"])
await client.requests.add_finding(
    request_id=mine["records"][0]["id"],
    summary="Ledger 4400 has 12 unmatched entries.",
    severity="high",
)
await client.requests.complete(request_id=mine["records"][0]["id"])
```

⚠ **A claimed request that is neither completed nor released holds its claim until the pool's timeout expires.** Until then no other actor can take it, and it does not appear in any pending queue. An agent that crashes mid-request leaves exactly this state — the work is invisible rather than failed. Pool health in 08.3 is where you see it.

Directives

Standing instructions. A directive is published to an audience, must be acknowledged, and binds until it expires, is revoked, or is superseded. It is the mechanism for organisation-wide instruction that outlives any single objective.

capability	what it does
Create / update / delete	Author a directive
List / get	Enumerate them
Publish	Put it into force
Acknowledge	Record that a recipient has read and accepted it
Batch acknowledge	Acknowledge on behalf of many
Acknowledgment status	Who has and has not acknowledged

capability	what it does
Expire	End it at its natural end
Revoke	Withdraw it immediately
Supersede	Replace it with a newer directive, preserving the lineage

Entry point: `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` .

Operations: `api:POST /api/v1/directives` · `api:GET /api/v1/directives` · `api:GET /api/v1/directives/{id}` · `api:PUT /api/v1/directives/{id}` · `api:DELETE /api/v1/directives/{id}` · `api:POST /api/v1/directives/{id}/publish` · `api:POST /api/v1/directives/{id}/acknowledge` · `api:POST /api/v1/directives/{id}/batch-acknowledge` · `api:GET /api/v1/directives/{id}/acknowledgments` · `api:POST /api/v1/directives/{id}/expire` · `api:POST /api/v1/directives/{id}/revoke` · `api:POST /api/v1/directives/{id}/supersede`

```
directive = await client.work_objectives.create_directive(
    title="Freeze discretionary spend",
    body="No new commitments above $5,000 without CFO sign-off until year end.",
)
await client.work_objectives.publish_directive(directive_id=directive["id"])

status = await client.work_objectives.get_acknowledgment_status(
    directive_id=directive["id"],
)
```

An unpublished directive binds nobody, the same shape as an unsubmitted objective. And a directive that is revoked rather than superseded leaves no pointer to what replaced it — use `supersede` when there is a successor, so the lineage survives for whoever audits the decision later.

Change requests

A proposal to change something, with a negotiation lifecycle: submit, approve, deny, counter-propose, accept the counter, and apply.

capability	what it does
Create	Draft the proposed change
Submit	Put it forward for decision
Approve / deny	Decide it
Counter-propose	Offer a modified version
Accept counter	Take the counter-proposal
Apply	Effect the approved change
List / get	Enumerate them

Entry point: `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` .

Operations: `api:POST /api/v1/change-requests` · `api:GET /api/v1/change-requests` · `api:GET /api/v1/change-requests/{id}` · `api:PATCH /api/v1/change-requests/{id}/submit` · `api:PATCH /api/v1/change-requests/{id}/approve` · `api:PATCH /api/v1/change-requests/{id}/deny` · `api:PATCH /api/v1/change-requests/{id}/counter-propose` · `api:PATCH /api/v1/change-requests/{id}/accept-counter` · `api:PATCH /api/v1/change-requests/{id}/apply`

Approval and application are separate steps on purpose. An approved change request has a decision recorded against it and has changed nothing. Forgetting `apply` produces a change everyone believes is in force and no system reflects — the most expensive failure in this chapter, because the audit trail says approved and the behaviour says otherwise.

Decisions

The surface where held work waits for a human answer. This is the decision gradient made concrete: an objective that reaches something it may not decide alone raises a decision, and the decision blocks until answered.

capability	what it does
List pending	Every decision waiting on a human
Get	One decision in full
Options	The choices available, with their consequences
Decide	Answer it
Cancel	Withdraw the decision request

Entry point: `sdk:aegis_sdk.modules.decision_points.DecisionsModule` .

Operations: `api:GET /api/v1/decisions/pending` · `api:GET /api/v1/decisions/{id}` · `api:GET /api/v1/decisions/{id}/options` · `api:POST /api/v1/decisions/{id}/decide` · `api:POST /api/v1/decisions/{id}/cancel`

```
pending = await client.decisions.list_pending()
for d in pending["records"]:
    options = await client.decisions.get_options(decision_id=d["id"])
    await client.decisions.decide(decision_id=d["id"], option_id=options[0]["id"])
```

Review decisions

Distinct from decision points: a review decision is the recorded verdict of a review, with conditions that may be marked met later.

capability	what it does
List / get	Enumerate review verdicts

capability	what it does
Latest for request	The most recent verdict on one request
Mark conditions met	Record that a conditional approval's conditions are satisfied

Entry point: `sdk:aegis_sdk.modules.decision_points.ReviewDecisionsModule` .

Operations: `api:GET /api/v1/review-decisions` · `api:GET /api/v1/review-decisions/{id}` · `api:GET /api/v1/review-decisions/latest/{request_id}` · `api:PATCH /api/v1/review-decisions/{id}/conditions`

```
verdict = await client.review_decisions.get_latest(request_id=request["id"])
if verdict["conditional"]:
    await client.review_decisions.mark_conditions_met(decision_id=verdict["id"])
```

A conditional approval is not an approval until its conditions are marked met. The verdict reads `approved` in every listing while the conditions are outstanding, so the work it gates stays blocked with nothing showing why.

Approvals

The general approval queue — the human gate that several other surfaces route into.

capability	what it does
List pending	Everything awaiting approval
Get	One approval in full
Approve / reject	Decide it
Modify	Approve with changes

Entry point: `sdk:aegis_sdk.standup.approvals.ApprovalsModule` .

Operations: `api:GET /api/v1/approvals/pending` · `api:GET /api/v1/approvals/{id}` · `api:POST /api/v1/approvals/{id}/approve` · `api:POST /api/v1/approvals/{id}/reject`

Escalation

What happens when work is not handled in time. Escalation is configured per pool and fires on a timer, but can also be triggered manually.

capability	what it does
Get / update config	The escalation chain and its timings for one pool
Pending	Escalations currently in flight

capability	what it does
Escalate	Raise a task manually
Acknowledge	Accept an escalated task
Reset / cancel timer	Manage the escalation clock on one task
History	What has escalated, and to whom
Stats	Escalation rates and patterns

Entry point: `sdk:aegis_sdk.modules.pools.PoolsModule` , with the objective-side surface on `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` .

Operations: `api:GET /api/v1/escalation/config/{pool_id}` · `api:PUT /api/v1/escalation/config/{pool_id}` · `api:GET /api/v1/escalation/pending` · `api:GET /api/v1/escalation/stats` · `api:POST /api/v1/escalation/tasks/{task_id}/escalate` · `api:POST /api/v1/escalation/tasks/{task_id}/acknowledge` · `api:POST /api/v1/escalation/tasks/{task_id}/reset-timer` · `api:POST /api/v1/escalation/tasks/{task_id}/cancel-timer` · `api:GET /api/v1/escalation/tasks/{task_id}/history`

An escalation chain whose last link is vacant terminates silently. The task escalates upward until it runs out of chain and then stops, still unhandled and no longer in any pending queue anyone watches. Check the chain against the reporting structure in 08.2 after any re-organisation.

Agent escalations

Distinct from task escalation: this is an agent stopping to ask, because it has reached the edge of what it may decide. It carries the trigger, the options, a recommendation, and the constraint state at the moment it stopped.

capability	what it does
List pending	Agents currently waiting on a human
Stream	Consume escalations as they happen
Get	One escalation, with its full context
Resolve	Answer it and let the agent continue
Cancel	Withdraw it

Entry point: `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` .

Operations: `api:GET /api/v1/agent-escalations/pending` · `api:GET /api/v1/agent-escalations/stream` · `api:GET /api/v1/agent-escalations/{id}` · `api:POST /api/v1/agent-escalations/{id}/resolve` · `api:POST /api/v1/agent-escalations/{id}/cancel`

```

async for esc in client.work_objectives.stream_agent_escalations():
    detail = await client.work_objectives.get_agent_escalation(escalation_id=esc["id"])
    await client.work_objectives.resolve_agent_escalation(
        escalation_id=esc["id"],
        decision="approve",
        rationale="Within the revised Q3 ceiling.",
    )

```

Interventions

Acting on a running session from outside it — the operator's hand on the wheel.

capability	what it does
List sessions	Sessions available to intervene in
Get / history	One session's intervention state and past interventions
Pause / resume	Halt or restart the session
Tighten constraints	Narrow the envelope on a session already running
Fence	Confine the session to a reduced scope

Entry point: `sdk:aegis_sdk.modules.work_objectives.WorkObjectivesModule` .

Operations: `api:GET /api/v1/interventions/sessions` · `api:GET /api/v1/interventions/{id}` · `api:GET /api/v1/interventions/{id}/history` · `api:POST /api/v1/interventions/{id}/pause` · `api:POST /api/v1/interventions/{id}/resume` · `api:POST /api/v1/interventions/{id}/tighten-constraints` · `api:POST /api/v1/interventions/{id}/fence`

```

await client.work_objectives.tighten_constraints(
    session_id=session["id"],
    constraints={"financial": {"max_amount_cents": 50_000}},
)

```

Tightening is one-way within a session. Constraints narrow; they do not widen back. If you need the original bounds restored, that is a new session — which is the intended behaviour, because an operator who can widen mid-run has removed the envelope's meaning.

Promotions

Moving something to a higher state of trust or scope, with an approval gate and rules that can automate the decision.

capability	what it does
Create / list / get	The promotion record
Approve / reject	Decide it

capability	what it does
Execute	Effect an approved promotion
Rules	Create, list, get, update and delete automation rules

Entry point: `sdk:aegis_sdk.modules.promotions.PromotionsModule` .

Operations: `api:POST /api/v1/promotions` · `api:GET /api/v1/promotions` · `api:GET /api/v1/promotions/{id}` · `api:POST /api/v1/promotions/{id}/approve` · `api:POST /api/v1/promotions/{id}/reject` · `api:POST /api/v1/promotions/{id}/execute` · `api:GET /api/v1/promotions/rules` · `api:POST /api/v1/promotions/rules` · `api:GET /api/v1/promotions/rules/{id}` · `api:PUT /api/v1/promotions/rules/{id}` · `api:DELETE /api/v1/promotions/rules/{id}`

```
promotion = await client.promotions.create(subject_id=agent["id"], target="delegated")
await client.promotions.approve(promotion_id=promotion["id"])
await client.promotions.execute(promotion_id=promotion["id"]) # without this, nothing moved
```

As with change requests, **approve and execute are separate** — and for the same reason, and with the same failure mode.

Positions

A lightweight claim-and-consume primitive: a position is staked, ratified or retracted, and consumed once.

capability	what it does
Create	Stake the position
Get	Read it
Ratify	Confirm it
Retract	Withdraw it
Consume	Spend it — single-use by construction

Entry point: `sdk:aegis_sdk.modules.positions.PositionsModule` .

Operations: `api:POST /api/v1/positions` · `api:GET /api/v1/positions/{id}` · `api:POST /api/v1/positions/{id}/ratify` · `api:POST /api/v1/positions/{id}/retract` · `api:POST /api/v1/positions/{id}/consume`

```
position = await client.positions.create(subject_id=claim_subject_id)
await client.positions.ratify(position_id=position["id"])
await client.positions.consume(position_id=position["id"]) # single-use; the second call fails
```

A position is spent, not read. `consume` succeeds once; a retry after a partial failure is refused, so treat the consume as the commit point and record its result rather than re-deriving it.

Artifacts

The outputs. Artifacts are versioned, downloadable, and supersedable — the durable record of what work produced.

capability	what it does
Create	Record an artifact
List / get	Enumerate and address them
Versions	The version history of one artifact
Supersede	Replace it, keeping the lineage
Download	Retrieve the content
Delete	Remove it

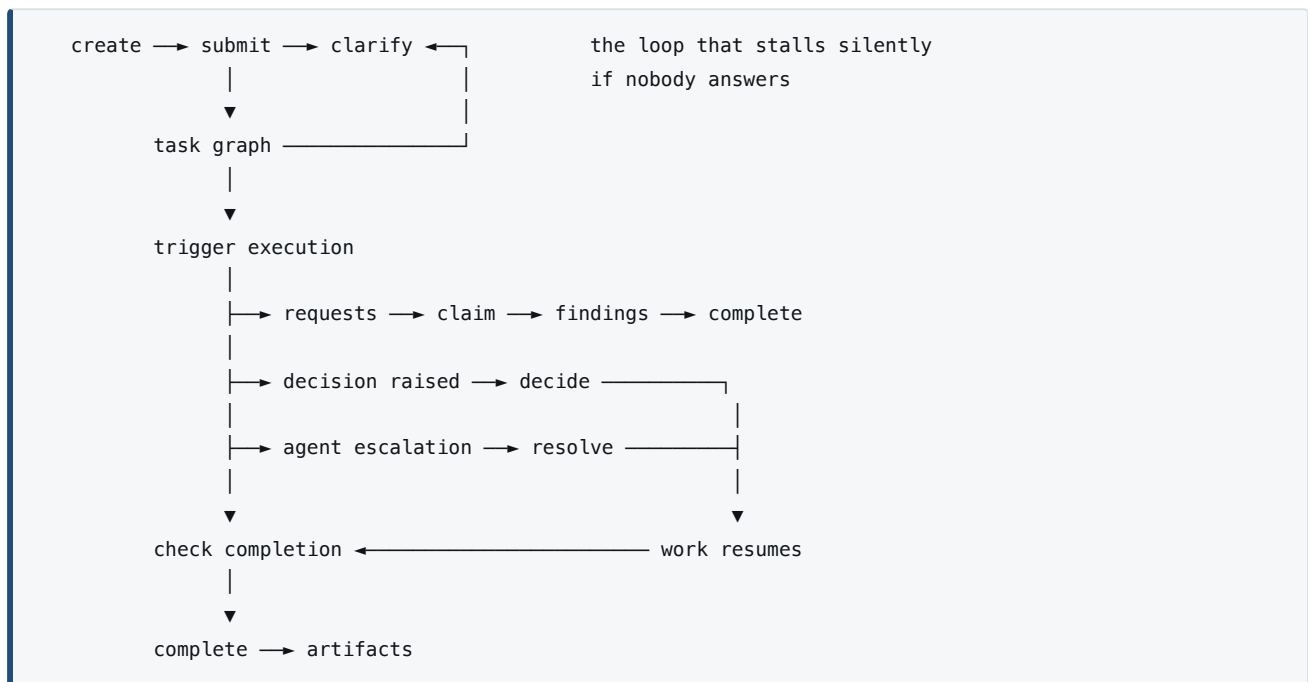
Entry point: `sdk:aegis_sdk.execution.artifacts.ArtifactsModule` .

Operations: `api:POST /api/v1/artifacts` · `api:GET /api/v1/artifacts` · `api:GET /api/v1/artifacts/{id}` · `api:GET /api/v1/artifacts/{id}/versions` · `api:GET /api/v1/artifacts/{id}/download` · `api:POST /api/v1/artifacts/{id}/supersede` · `api:DELETE /api/v1/artifacts/{id}`

```
artifacts = await client.objectives.get_artifacts(objective_id=objective["id"])
content = await client.artifacts.download(artifact_id=artifacts[0]["id"])
```

Supersede rather than delete when an artifact was wrong. Deleting removes the evidence that the wrong version ever existed, which is precisely what an auditor asks about. Superseding keeps both and records which replaced which.

The work lifecycle, end to end



Read it as: the left column is the objective's own progress, and the two branches in the middle are the places it stops and waits for a human. **Both branches are blocking and neither is a failure**, which is why an objective that has stopped does not appear in any error query. If work has gone quiet, check `api:GET /api/v1/decisions/pending` and `api:GET /api/v1/agent-escalations/pending` before looking for a fault.

Summary — what this chapter covered

area	entry point	scale
Objectives	<code>execution.objectives</code> · <code>modules.work_objectives</code>	37 operations
Work units	<code>modules.work_objectives</code>	25 operations
Requests	<code>execution.requests</code>	11 operations
Directives	<code>modules.work_objectives</code>	12 operations
Change requests	<code>modules.work_objectives</code>	9 operations
Decisions	<code>modules.decision_points</code>	5 operations
Review decisions	<code>modules.decision_points</code>	4 operations
Approvals	<code>standup.approvals</code>	4 operations
Escalation	<code>modules.pools</code>	9 operations
Agent escalations	<code>modules.work_objectives</code>	5 operations
Interventions	<code>modules.work_objectives</code>	7 operations

area	entry point	scale
Promotions	<code>modules.promotions</code>	11 operations
Positions	<code>modules.positions</code>	5 operations
Artifacts	<code>execution.artifacts</code>	7 operations

Next: 08.5 — Governance and trust

08.5 — Governance and trust

This chapter enumerates **what bounds everything else**. It is the chapter to read if you are evaluating Aegis against a governance requirement, because almost every requirement that sounds like it is about agents turns out to be about the controls on agents, and they are all here.

The controls are not one mechanism with several settings. They are **five independent gates**, each answering a different question, and a call must pass all of them:

gate	answers	catalogued in
Permission	May this principal call this operation?	08.2 — RBAC
Trust chain	Does this agent hold delegated authority at all?	here — trust chains
Posture	How autonomously may it act?	here — postures
Envelope	Within what financial, temporal and operational bounds?	here — envelopes
Clearance	May it see this classification of data?	here + 08.6

A refusal names one gate, and fixing a different one changes nothing. That is the single most useful thing to carry out of this chapter: when a call is refused, establish which gate refused it before adjusting anything. 02.3 and 06.4 teach the model; this is the surface.

Trust chains

The spine. A trust chain records that authority was delegated from a human origin, through roles, to an agent — and every act an agent takes is attributable back along it. An agent with no chain has no authority to do anything.

capability	what it does
Establish	Create a chain from an authority to an agent
List / get	Enumerate and address chains
Verify	Check a chain is intact and valid right now
Delegation path	The full path from origin to this agent
Summary	One chain's rolled-up state
Agent trust context	Everything the platform knows about one agent's trust position
Revoke	Withdraw a chain
Revoke cascade	Withdraw it and everything that depends on it
Revoke by human	Withdraw everything traceable to one human origin

capability	what it does
Revocation impact	What a revocation would affect, before you do it
Incomplete jobs / resume	Find and finish a revocation that did not complete
Suspend / reinstate	Pause a chain without destroying it
Delegate	Extend authority down the chain
Revoke delegation	Withdraw one link rather than the whole chain

Entry points: `sdk:aegis_sdk.trust.chains.ChainsModule`,
`sdk:aegis_sdk.trust.delegations.DelegationsModule` and
`sdk:aegis_sdk.trust.revocation.RevocationModule`.

Operations: `api:POST /api/v1/trust/establish` · `api:GET /api/v1/trust/chains` · `api:GET /api/v1/trust/chains/{id}` · `api:GET /api/v1/trust/chains/{id}/delegation-path` · `api:GET /api/v1/trust/chains/{id}/summary` · `api:POST /api/v1/trust/verify` · `api:POST /api/v1/trust/delegate` · `api:POST /api/v1/trust/revoke-delegation` · `api:POST /api/v1/trust/revoke` · `api:POST /api/v1/trust/revoke/{id}/cascade` · `api:POST /api/v1/trust/revoke/by-human/{human_id}` · `api:GET /api/v1/trust/revoke/{id}/impact` · `api:GET /api/v1/trust/revoke/jobs/incomplete` · `api:POST /api/v1/trust/revoke/jobs/{job_id}/resume` · `api:GET /api/v1/trust/agents/{agent_id}/trust-context`

```
chain = await client.trust.chains.establish(
    authority_id=cfo_authority_id,
    agent_id=agent["id"],
)
verdict = await client.trust.chains.verify(chain_id=chain["id"])

# Always ask before you cascade
impact = await client.trust.chains.analyze_revocation_impact(chain_id=chain["id"])
if impact["affected_agent_count"] < 50:
    await client.trust.revocation.revoke(chain_id=chain["id"], cascade=True)
```



Measure the blast radius before a cascade revocation. `api:GET`

`/api/v1/trust/revoke/{id}/impact` exists because a cascade from high in the tree can silently stop every agent beneath it. The symptom is not an error — it is an organisation where nothing executes and every agent reports valid configuration, because the configuration *is* valid and the authority behind it is gone.

A revocation that does not complete leaves a partial state. `api:GET`

`/api/v1/trust/revoke/jobs/incomplete` is how you find it and `api:POST /api/v1/trust/revoke/jobs/{job_id}/resume` is how you finish it. Check this after any large cascade: a half-revoked tree has some agents stopped and some still running on authority you intended to withdraw.

Authorities

The origin points of trust. An authority is the organisational source a chain descends from.

capability	what it does
Create / update	Define an authority
List / get	Enumerate them
Deactivate	Retire one
Agents	Every agent descending from this authority
Display views	Presentation-shaped views for a console

Entry point: `sdk:aegis_sdk.trust.authorities.AuthoritiesModule` .

Operations: `api:POST /api/v1/trust/authorities` · `api:GET /api/v1/trust/authorities` · `api:GET /api/v1/trust/authorities/{id}` · `api:PATCH /api/v1/trust/authorities/{id}` · `api:POST /api/v1/trust/authorities/{id}/deactivate` · `api:GET /api/v1/trust/authorities/{id}/agents` · `api:GET /api/v1/trust/authorities/ui` · `api:GET /api/v1/trust/authorities/ui/{id}`

Postures — how autonomously an agent may act

The CARE posture gradient. Five levels, lowercase, and each one is a statement about how much a human stays in the loop.

posture	what it means
<code>pseudo</code>	Not an autonomous actor — a human in an agent-shaped slot
<code>supervised</code>	Acts, but a human reviews before anything takes effect
<code>shared_planning</code>	Plans with a human; executes the agreed plan
<code>continuous_insight</code>	Acts autonomously with continuous human visibility
<code>delegated</code>	Acts autonomously within its envelope

Posture is not set once and left. It is **progressed** — an agent earns a higher posture by accumulating evidence, and the platform evaluates whether it is eligible.

capability	what it does
Get	One agent's current posture
Update	Set it directly
History	Every posture this agent has held
Evaluate progression	Whether it currently qualifies for a higher posture

capability	what it does
Upgrade eligibility	The eligibility answer on its own
Request upgrade	Ask for a promotion, raising an approval
Pending approvals	Posture changes awaiting a human
Approve / reject transition	Decide one
Evidence	What supports the current posture
Metrics	The measures progression is evaluated against
Override	Set a posture outside the normal progression
Posture ceiling	The cap a unit imposes on everything beneath it

Entry points: `sdk:aegis_sdk.trust.postures.PosturesModule` and `sdk:aegis_sdk.modules.trust_posture.TrustPostureModule`.

Operations: `api:GET /api/v1/agents/{id}/trust-posture` · `api:PUT /api/v1/agents/{id}/trust-posture` · `api:GET /api/v1/agents/{id}/trust-posture/history` · `api:GET /api/v1/agents/{id}/trust-posture/evaluate` · `api:GET /api/v1/agents/{id}/trust-posture/upgrade-eligibility` · `api:GET /api/v1/agents/{id}/trust-posture/evidence` · `api:GET /api/v1/agents/{id}/trust-posture/metrics` · `api:GET /api/v1/agents/{id}/trust-posture/pending` · `api:POST /api/v1/agents/{id}/trust-posture/request-upgrade` · `api:POST /api/v1/agents/{id}/trust-posture/approve` · `api:POST /api/v1/agents/{id}/trust-posture/reject` · `api:GET /api/v1/posture/pending-approvals` · `api:GET /api/v1/organization-units/{id}/posture-ceiling` · `api:PUT /api/v1/organization-units/{id}/posture-ceiling`

```
posture = await client.trust.postures.get(agent_id=agent["id"])
eligibility = await client.trust_posture.check_upgrade_eligibility(agent_id=agent["id"])

if eligibility["eligible"]:
    await client.trust_posture.request_upgrade(
        agent_id=agent["id"],
        target_posture="continuous_insight",
    )
```

⚠ A unit's posture ceiling silently caps every agent beneath it. An agent set to `delegated` inside a unit ceilinged at `supervised` operates as `supervised`, and its own posture field still reads `delegated`. The agent's record is not wrong — the effective posture is the minimum of the two, and only the ceiling call tells you what is actually in force.

Envelopes and constraints — the five dimensions

An envelope bounds what an actor may do, across five dimensions:

dimension	bounds
Financial	Spend, commitments, transaction size
Operational	Which actions, how many, against what
Temporal	When, and for how long
Data Access	Which data, at which classification
Communication	Who and what it may talk to

Envelopes compose down the hierarchy and **tighten monotonically**: a child can never be wider than its parent, and the effective envelope is the intersection of every envelope from the origin down.

capability	what it does
Role envelopes	Create, get, list, update, delete the standing bounds on a role
Activate / suspend	Put a role envelope into or out of force
Agent constraints	Read and set the bounds on one agent
Effective constraints	The composed result — what is actually in force
Inherited constraints	What this agent gets from above
Validate	Check a proposed constraint set before applying it
Organisation defaults	The baseline every new actor starts from
Envelope defaults	The platform's own default shape
Gradient rules	How constraint breaches map onto the decision gradient

Entry points: `sdk:aegis_sdk.standup.envelopes.RoleEnvelopesModule` and `sdk:aegis_sdk.modules.trust_posture.TrustPostureModule`.

Operations: `api:POST /api/v1/role-envelopes` · `api:GET /api/v1/role-envelopes` · `api:GET /api/v1/role-envelopes/{id}` · `api:PUT /api/v1/role-envelopes/{id}` · `api:DELETE /api/v1/role-envelopes/{id}` · `api:POST /api/v1/role-envelopes/{id}/activate` · `api:POST /api/v1/role-envelopes/{id}/suspend` · `api:GET /api/v1/constraints/agents/{agent_id}` · `api:PUT /api/v1/constraints/agents/{agent_id}` · `api:GET /api/v1/constraints/agents/{agent_id}/effective` · `api:GET /api/v1/constraints/agents/{agent_id}/inherited` · `api:POST /api/v1/constraints/agents/{agent_id}/validate` · `api:GET /api/v1/constraints/gradient-rules` · `api:GET /api/v1/constraints/organizations/{org_id}/defaults` · `api:PUT /api/v1/constraints/organizations/{org_id}/defaults` · `api:GET /api/v1/envelopes/defaults`

```

envelope = await client.role_envelopes.create(
    organization_role_id=controller_role_id,
    financial={"max_amount_cents": 500_000, "period": "month"},
    temporal={"business_hours_only": True},
    data_access={"max_classification": "confidential"},
)
await client.trust_posture.activate_role_envelope(envelope_id=envelope["id"])

effective = await client.trust_posture.get_effective_constraints(agent_id=agent["id"])

```

🚫 **A created envelope is not an active envelope.** `create` records it; `activate` puts it in force. A provisioning script that creates envelopes and never activates them produces an organisation that looks fully bounded in every listing and enforces nothing. Nothing errors.

Read the effective constraints, never the agent's own. The agent-level constraint call returns what was set *on that agent*; the effective call returns the intersection of everything above it. They differ whenever any ancestor carries a tighter bound, which is the normal case in a real hierarchy — and the difference is exactly the part that will refuse a call you expected to succeed.

Clearances — what an actor may see

Clearance is the Data Access dimension made explicit, and it is independent of authority. A junior role can hold a high clearance; a senior one can hold a low one.

capability	what it does
Create / update / delete	Manage a role's clearance record
List / get	Enumerate clearances
Approve / reject	Decide a requested clearance

Entry point: `sdk:aegis_sdk.modules.trust_posture.TrustPostureModule` .

Operations: `api:POST /api/v1/role-clearances` · `api:GET /api/v1/role-clearances` · `api:GET /api/v1/role-clearances/{id}` · `api:PUT /api/v1/role-clearances/{id}` · `api:DELETE /api/v1/role-clearances/{id}` · `api:POST /api/v1/role-clearances/{id}/approve` · `api:POST /api/v1/role-clearances/{id}/reject`

```

clearance = await client.trust_posture.create_role_clearance(
    organization_role_id=analyst_role_id,
    max_classification="secret",
    compartments=["treasury"],
)
await client.trust_posture.approve_role_clearance(clearance_id=clearance["id"])

```

Posture caps clearance. An agent at `supervised` does not reach `secret` data however cleared its role is — the effective clearance is the minimum of the role's clearance and the posture's ceiling. Raising the clearance on a low-posture agent changes nothing, and it is the most common wasted fix in

this area.

Access policy

Attribute-based policy over resources, separate from RBAC's operation-level permissions.

capability	what it does
Create / update / delete	Author a policy
List / get	Enumerate policies
Evaluate	Ask what a policy set decides for a given request
Assign / unassign	Attach a policy to a principal
User policies	Which policies apply to one user
References	What a policy refers to
Validate conditions	Check a policy's conditions are well-formed
Validate conflicts	Find policies that contradict one another
Resolve conflict	Decide between conflicting policies

Entry point: `sdk:aegis_sdk.modules.knowledge_govern.KnowledgeGovernModule` .

Operations: `api:POST /api/v1/policies` · `api:GET /api/v1/policies` · `api:GET /api/v1/policies/{id}` · `api:PUT /api/v1/policies/{id}` · `api:DELETE /api/v1/policies/{id}` · `api:POST /api/v1/policies/evaluate` · `api:POST /api/v1/policies/{id}/assign` · `api:DELETE /api/v1/policies/assignments/{id}` · `api:GET /api/v1/policies/user/{user_id}` · `api:GET /api/v1/policies/{id}/references` · `api:POST /api/v1/policies/validate-conditions` · `api:POST /api/v1/policies/validate-conflicts` · `api:POST /api/v1/policies/resolve-conflict`

Run **`validate-conflicts`** after adding a policy, not when something breaks. Two policies that contradict resolve deterministically but not necessarily the way either author intended, and the symptom is an access decision nobody can explain from reading either policy alone.

Policy clauses and compilation

The declarative layer above policy: clauses are authored, compiled into a coherent policy set, previewed, and applied as a unit.

capability	what it does
List / create clause	Author policy clauses
Retract clause	Withdraw one
Compile	Turn the clause set into an applicable policy set
List / get runs	The compilation history

capability	what it does
Apply run	Put a compiled run into force

Entry point: `sdk: aegis_sdk.modules.governance.GovernanceModule` .

Operations: `api:GET /api/v1/policy-clauses` · `api:POST /api/v1/policy-clauses` · `api:POST /api/v1/policy-clauses/{id}/retract` · `api:POST /api/v1/compilation-runs` · `api:GET /api/v1/compilation-runs` · `api:GET /api/v1/compilation-runs/{id}` · `api:POST /api/v1/compilation-runs/{id}/apply`

```
run = await client.governance.compile_policy_clauses()
detail = await client.governance.get_compilation_run(run_id=run["id"])
await client.governance.apply_compilation_run(run_id=run["id"])
```

Compile and apply are separate, as everywhere else in the platform. A compilation run that is never applied is a policy set that exists and governs nothing.

Governance explanation

The surface that answers *why* — indispensable when a refusal is not obvious, and the thing to reach for before changing any control.

capability	what it does
Explain access	Why a principal can or cannot reach a resource
Explain envelope	How an effective envelope was composed, and from what
Describe address	What a positional address refers to
Envelope coverage	Which roles have envelopes and which do not
Envelope hydration status	Whether the envelope data is fully loaded
Probe corrupted roles	Find roles whose governance data is inconsistent

Entry point: `sdk: aegis_sdk.modules.governance_explain.GovernanceExplainModule` .

Operations: `api:POST /api/v1/governance/explain-access` · `api:POST /api/v1/governance/explain-envelope` · `api:POST /api/v1/governance/describe-address` · `api:GET /api/v1/governance/envelope-coverage` · `api:GET /api/v1/governance/envelope-hydration-status` · `api:GET /api/v1/governance/probe-corrupted-roles`

```

why = await client.governance_explain.explain_access(
    principal_id=agent["id"],
    resource_id=knowledge_item_id,
    action="read",
)
# why names the gate that decided, not just the verdict

coverage = await client.governance_explain.envelope_coverage()

```

envelope_coverage is the query to run after standing an organisation up. It reports which roles carry no envelope — and a role with no envelope is not a role with no bounds, it is a role whose composition has a gap, which fails closed and produces refusals that look arbitrary.

Bridges — authorised cross-boundary interaction

Isolation domains stop units seeing one another. A bridge is the governed exception: an explicit, envelope-bounded channel between two roles. Three kinds, by how long they last and how they are approved.

kind	lasts	approved by
Standing bridge	Indefinitely, with periodic review	Authorisation, then activation
Scoped bridge	For one workspace or objective	Approval, with expiry and renewal
Ad-hoc bridge	One interaction	Approval per instance

A bridge's envelope is the intersection of both sides — the most restrictive wins. Two roles bridged together do not gain each other's reach; each is bounded by whichever side is tighter.

Standing bridges

capability	what it does
Create / update / delete	Standard lifecycle
List / get / for unit	Enumerate, including per unit
Authorize	Approve the bridge's existence
Activate	Put it into force
Suspend / revoke	Pause or end it
Check interaction	Ask whether a specific interaction is permitted
Submit review	Record the periodic review
Overdue	Bridges whose review has lapsed

Entry point: `sdk:aegis_sdk.modules.bridges.StandingBridgesModule` .

Operations: `api:POST /api/v1/standing-bridges` · `api:GET /api/v1/standing-bridges` · `api:GET /api/v1/standing-bridges/{id}` · `api:PUT /api/v1/standing-bridges/{id}` · `api:DELETE /api/v1/standing-bridges/{id}` · `api:GET /api/v1/standing-bridges/for-unit/{unit_id}` · `api:GET /api/v1/standing-bridges/overdue` · `api:POST /api/v1/standing-bridges/{id}/authorize` · `api:POST /api/v1/standing-bridges/{id}/activate` · `api:POST /api/v1/standing-bridges/{id}/suspend` · `api:POST /api/v1/standing-bridges/{id}/revoke` · `api:POST /api/v1/standing-bridges/{id}/check-interaction` · `api:POST /api/v1/standing-bridges/{id}/review`

Scoped bridges

capability	what it does
Create / update / delete	Standard lifecycle
List / get	Enumerate
For workspace / for objective	Bridges scoped to one context
Approve / reject	Decide it
Add / remove unit	Change which units participate
Participants	Who is in it
Extend / renew	Push out the expiry
Complete / cancel / expire	End it

Entry point: `sdk:aegis_sdk.modules.bridges.ScopedBridgesModule` .

Operations: `api:POST /api/v1/scoped-bridges` · `api:GET /api/v1/scoped-bridges` · `api:GET /api/v1/scoped-bridges/{id}` · `api:PUT /api/v1/scoped-bridges/{id}` · `api:DELETE /api/v1/scoped-bridges/{id}` · `api:GET /api/v1/scoped-bridges/for-workspace/{workspace_id}` · `api:GET /api/v1/scoped-bridges/for-objective/{objective_id}` · `api:GET /api/v1/scoped-bridges/{id}/participants` · `api:POST /api/v1/scoped-bridges/{id}/approve` · `api:POST /api/v1/scoped-bridges/{id}/reject` · `api:POST /api/v1/scoped-bridges/{id}/add-unit` · `api:DELETE /api/v1/scoped-bridges/{id}/participants/{participant_id}` · `api:POST /api/v1/scoped-bridges/{id}/extend` · `api:POST /api/v1/scoped-bridges/{id}/renew` · `api:POST /api/v1/scoped-bridges/{id}/complete` · `api:POST /api/v1/scoped-bridges/{id}/cancel` · `api:POST /api/v1/scoped-bridges/{id}/expire`

Ad-hoc bridges

capability	what it does
For unit	Ad-hoc bridges touching one unit
Pending for	Awaiting one principal's decision
Approve / reject	Decide it

capability	what it does
Activate	Put it into force

Entry point: `sdk:aegis_sdk.modules.bridges.AdHocBridgesModule` .

Operations: `api:GET /api/v1/ad-hoc-bridges/for-unit/{unit_id}` · `api:GET /api/v1/ad-hoc-bridges/pending-for/{principal_id}` · `api:POST /api/v1/ad-hoc-bridges/{id}/approve` · `api:POST /api/v1/ad-hoc-bridges/{id}/reject` · `api:POST /api/v1/ad-hoc-bridges/{id}/activate`

```
bridge = await client.bridges.standing.create(
    role_a_id=treasury_lead_id,
    role_b_id=risk_lead_id,
    purpose="Daily liquidity position exchange",
)
await client.bridges.standing.authorize(bridge_id=bridge["id"])
await client.bridges.standing.activate(bridge_id=bridge["id"])

overdue = await client.bridges.standing.overdue()
```

Authorise and activate are two steps for standing bridges, and an authorised but inactive bridge passes every existence check while carrying no traffic. `api:GET /api/v1/standing-bridges/overdue` is the other one to watch: a bridge whose review has lapsed is still carrying traffic, and the lapse is visible nowhere else.

Emergency bypass

The governed way to exceed an envelope when circumstances demand it. Every bypass is time-bounded, approved, and auditable — it is not a switch that turns governance off.

capability	what it does
Create	Request a bypass, stating scope and duration
Approve / reject	Decide it
Revoke	End it early
List / list active	Every bypass, or only those in force
Get	One bypass in full

Entry point: `sdk:aegis_sdk.modules.emergency.EmergencyBypassModule` .

Operations: `api:POST /api/v1/emergency-bypass/create` · `api:GET /api/v1/emergency-bypass` · `api:GET /api/v1/emergency-bypass/active` · `api:GET /api/v1/emergency-bypass/{id}` · `api:POST /api/v1/emergency-bypass/{id}/approve` · `api:POST /api/v1/emergency-bypass/{id}/reject` · `api:POST /api/v1/emergency-bypass/{id}/revoke`

```
bypass = await client.emergency_bypass.create(
    agent_id=agent["id"],
    reason="Settlement window closes in 40 minutes; ceiling blocks the transfer.",
    duration_hours=4,
    scope={"financial": {"max_amount_cents": 2_000_000}},
)
await client.emergency_bypass.approve(bypass_id=bypass["id"])

still_open = await client.emergency_bypass.list_active()
```

🚫 **Bypasses expire; check `active` rather than assuming they have.** A bypass that was approved for an incident and never revoked keeps its widened bounds until its clock runs out, and nothing announces that it is still in force. `api:GET /api/v1/emergency-bypass/active` belongs in whatever you review daily.

Kill switch

The blunt instrument. Where a bypass widens bounds, the kill switch stops activity outright.

capability	what it does
Activate	Stop the scoped activity
List open	Every active kill-switch activation
Get	One activation in full
Clear	Lift it

Entry point: `sdk:aegis_sdk.modules.emergency.KillSwitchModule` .

Operations: `api:POST /api/v1/kill-switch/activate` · `api:GET /api/v1/kill-switch/open` · `api:GET /api/v1/kill-switch/{id}` · `api:POST /api/v1/kill-switch/{id}/clear`

An uncleared kill switch is the explanation for an organisation that has gone quiet. It leaves no error anywhere — agents simply do not act. Check `api:GET /api/v1/kill-switch/open` early when nothing is happening and nothing is failing.

Trust observability

The measurement surface over everything above.

capability	what it does
Health	Whether the trust fabric is sound
Metrics / export	Trust measures, readable or exportable
Compliance report	The trust position formatted for a reviewer

capability	what it does
Agent trust score	One agent's composite trust measure
CARE budget	What the agent has left across the gradient dimensions
Capabilities	What this agent is actually able to do
Capability summary	The rolled-up version
Trust summary / with-trust	Agent records enriched with their trust position
Registry	Register, discover, heartbeat and read agent metadata
Pipeline trust	Validate a pipeline's agents before running it
ESA config	External trust-service configuration and connection test

Entry points: `sdk:aegis_sdk.trust.observability.TrustObservabilityModule` ,
`sdk:aegis_sdk.trust.agent_trust.AgentTrustModule` ,
`sdk:aegis_sdk.trust.registry.TrustRegistryModule` ,
`sdk:aegis_sdk.trust.pipeline.PipelineTrustModule` and `sdk:aegis_sdk.trust.esa.ESAModule` .

Operations: `api:GET /api/v1/trust/health` · `api:GET /api/v1/trust/metrics` · `api:GET /api/v1/trust/metrics/export` · `api:GET /api/v1/trust/compliance/{id}` · `api:GET /api/v1/trust/agents/{agent_id}/trust-score` · `api:GET /api/v1/trust/agents/{agent_id}/care-budget` · `api:GET /api/v1/trust/agents/{agent_id}/capabilities` · `api:GET /api/v1/trust/agents/{agent_id}/capability-summary` · `api:GET /api/v1/trust/agents/{agent_id}/constraints` · `api:GET /api/v1/trust/agents/{agent_id}/trust-summary` · `api:GET /api/v1/trust/agents/{agent_id}/with-trust` · `api:POST /api/v1/trust/registry/agents` · `api:GET /api/v1/trust/registry/agents/{agent_id}` · `api:POST /api/v1/trust/registry/agents/{agent_id}/heartbeat` · `api:POST /api/v1/trust/registry/discover` · `api:POST /api/v1/trust/pipeline/validate` · `api:GET /api/v1/trust/pipeline/{pipeline_id}/agents/{agent_id}` · `api:GET /api/v1/trust/esa/config` · `api:PUT /api/v1/trust/esa/config` · `api:POST /api/v1/trust/esa/test-connection`

```
health = await client.trust.observability.get_health()
budget = await client.trust.agents.get_care_budget(agent_id=agent["id"])
await client.trust.pipeline.validate(pipeline_id=p_id)
```

Trust audit

The append-only record of every trust event. The audit spine proper is in 08.7; this is its trust-specific face.

capability	what it does
Query	Search trust events

capability	what it does
Record	Append an entry
By human	Everything traceable to one human origin
Trace	Follow one act back along its chain

Entry point: `sdk:aegis_sdk.trust.audit.AuditModule` .

Operations: `api:GET /api/v1/trust/audit` · `api:POST /api/v1/trust/audit` · `api:GET /api/v1/trust/audit/by-human/{human_id}` · `api:GET /api/v1/trust/audit/trace/{event_id}`

```
trail = await client.trust.audit.trace(event_id=suspicious_event_id)
by_person = await client.trust.audit.query(human_id=departing_user_id)
```

by-human is the query a leaver process needs. It returns everything traceable to one human origin, which is both the audit answer and the input to `api:POST /api/v1/trust/revoke/by-human/{human_id}` .

The five gates, as a checklist

When a call is refused, work down this list rather than adjusting the first thing that looks relevant:

ask	with
1. Does the principal hold the permission?	<code>api:POST /api/v1/rbac/check-permission</code>
2. Does the agent have a valid chain?	<code>api:POST /api/v1/trust/verify</code>
3. Is its effective posture high enough?	<code>api:GET /api/v1/agents/{id}/trust-posture</code> + the unit ceiling
4. Do the effective constraints permit it?	<code>api:GET /api/v1/constraints/agents/{agent_id}/effective</code>
5. Does clearance reach the data?	<code>api:POST /api/v1/governance/explain-access</code>

Or ask the platform directly: `explain_access` names the gate that decided, which turns a five-step diagnosis into one call.

Next: 08.6 — Knowledge and data

08.6 — Knowledge and data

This chapter enumerates **what the organisation knows** and **where that came from**. Knowledge in Aegis is not a document store bolted onto an agent platform — it is a governed asset with a classification, a review lifecycle, a sharing policy, and a lineage, and every one of those is a first-class surface.

The reason it works this way is the one the rest of the catalogue keeps returning to: an agent's clearance decides what it may read, and a knowledge item without a classification cannot be cleared for. **Classification is what makes knowledge reachable, not what restricts it** — an unclassified item is not maximally available, it is unresolvable.

Four distinct things live here, and they are easy to run together:

object	what it is
Knowledge item	A governed piece of what the organisation knows
Connector	A live link to an external data system
Data source	A registered, described source of data
Workspace	A collaborative container for documents and work

02.3 teaches the clearance model this chapter's classification surface serves.

Knowledge items — the core lifecycle

A knowledge item moves through a review lifecycle before it is available, and every transition is recorded.

state	meaning
Draft	Authored, not yet submitted
Under review	Submitted; a reviewer is assigned
Changes requested	Returned to the author
Approved	Reviewed and accepted, not yet published
Published	Available to anyone cleared for it
Archived	Withdrawn from availability, retained for evidence

capability	what it does
Create / update / delete	Standard record lifecycle
List / get / search	Enumerate, address and search
Submit for review	Move it into the review lifecycle
Assign reviewer	Name who decides

capability	what it does
Approve / reject / request changes	The reviewer's three verdicts
Publish	Make it available
Create version	Snapshot the current content
Create revision	Start a new editable revision from a published item
Archive / unarchive	Withdraw or restore availability
Review queue	Everything awaiting review

Entry points: `sdk:aegis_sdk.standup.knowledge.KnowledgeModule` for the simple create/get/publish/list path, and `sdk:aegis_sdk.modules.knowledge_govern.KnowledgeGovernModule` for the full governance surface.

Operations: `api:POST /api/v1/knowledge` · `api:GET /api/v1/knowledge` · `api:GET /api/v1/knowledge/{id}` · `api:PUT /api/v1/knowledge/{id}` · `api:DELETE /api/v1/knowledge/{id}` · `api:GET /api/v1/knowledge/search` · `api:POST /api/v1/knowledge/{id}/submit` · `api:POST /api/v1/knowledge/{id}/assign-reviewer` · `api:POST /api/v1/knowledge/{id}/approve` · `api:POST /api/v1/knowledge/{id}/reject` · `api:POST /api/v1/knowledge/{id}/request-changes` · `api:POST /api/v1/knowledge/{id}/publish` · `api:POST /api/v1/knowledge/{id}/version` · `api:POST /api/v1/knowledge/{id}/create-revision` · `api:POST /api/v1/knowledge/{id}/archive` · `api:POST /api/v1/knowledge/{id}/unarchive` · `api:GET /api/v1/knowledge-reviews`

```

item = await client.knowledge.create(
    title="Q3 close procedure",
    content=procedure_text,
    classification="confidential",
    category_id=finance_category_id,
)
await client.knowledge_govern.submit_for_review(knowledge_id=item["id"])
await client.knowledge_govern.assign_reviewer(
    knowledge_id=item["id"],
    reviewer_id=controller_user_id,
)
await client.knowledge_govern.approve(knowledge_id=item["id"])
await client.knowledge.publish(knowledge_id=item["id"])

```

 **Approved is not published.** Approval records a reviewer's verdict; publication makes the item available. An approved-but-unpublished item is invisible to every agent that should be able to read it, and appears correct in every listing — the review shows complete, the content is there, and no agent can reach it.

Archive rather than delete when an item was superseded. Deleting removes the record that the organisation ever held that knowledge, which is exactly the question an audit asks. Archiving withdraws availability and keeps the evidence.

Knowledge categories

The taxonomy items are filed under. Categories are how knowledge is scoped to parts of the organisation.

capability	what it does
Create / update / delete	Manage the taxonomy
List	Enumerate categories

Entry point: `sdk:aegis_sdk.modules.knowledge_govern.KnowledgeGovernModule` .

Operations: `api:GET /api/v1/knowledge-categories` · `api:POST /api/v1/knowledge-categories` · `api:PUT /api/v1/knowledge-categories/{id}` · `api:DELETE /api/v1/knowledge-categories/{id}`

Knowledge share policies

Who may see what, expressed as a policy rather than per-item permissions. Share policies carry their own review cadence.

capability	what it does
Create / update / delete	Author a sharing policy
List / get	Enumerate them
Due for review	Policies whose review has come round
Suspend / reactivate	Take a policy out of force and back

Entry point: `sdk:aegis_sdk.modules.knowledge_govern.KnowledgeGovernModule` .

Operations: `api:GET /api/v1/knowledge-share-policies` · `api:POST /api/v1/knowledge-share-policies` · `api:GET /api/v1/knowledge-share-policies/{id}` · `api:PUT /api/v1/knowledge-share-policies/{id}` · `api:DELETE /api/v1/knowledge-share-policies/{id}` · `api:GET /api/v1/knowledge-share-policies/due-for-review` · `api:POST /api/v1/knowledge-share-policies/{id}/suspend` · `api:POST /api/v1/knowledge-share-policies/{id}/reactivate`

```
due = await client.knowledge_govern.list_share_policies_due_for_review()
```

due-for-review is the query that keeps sharing honest. A share policy written for a project that ended two years ago is still granting access, and nothing surfaces that except this call.

Content reviews and templates

A general review workflow over content, distinct from the knowledge lifecycle above — and templates to author consistently against.

capability	reviews	templates
Create / list / get	yes	yes
Update / delete	delete	yes
Submit	yes	—
Approve / reject / reopen	yes	—

Entry point: `sdk:aegis_sdk.modules.knowledge_govern.KnowledgeGovernModule` .

Operations: `api:GET /api/v1/content-reviews` · `api:POST /api/v1/content-reviews` · `api:GET /api/v1/content-reviews/{id}` · `api:DELETE /api/v1/content-reviews/{id}` · `api:POST /api/v1/content-reviews/{id}/submit` · `api:POST /api/v1/content-reviews/{id}/approve` · `api:POST /api/v1/content-reviews/{id}/reject` · `api:POST /api/v1/content-reviews/{id}/reopen` · `api:GET /api/v1/content-templates` · `api:POST /api/v1/content-templates` · `api:GET /api/v1/content-templates/{id}` · `api:PUT /api/v1/content-templates/{id}` · `api:DELETE /api/v1/content-templates/{id}`

Connectors — live links to external systems

A connector is a configured, testable link to a system outside Aegis. Agents query through connectors rather than holding credentials themselves.

capability	what it does
Create / update / delete	Manage a connector
List / get / types	Enumerate connectors and the kinds available
Test	Confirm it connects, before anything depends on it
Validate	Check the configuration is well-formed
Health	Whether it is working now
Schema	What the far side exposes
Metadata	Descriptive information about the source
Query	Read through it
Attach / detach	Wire it to an agent, or unwire it
Instances	Per-use configured instances of a connector

Entry point: `sdk:aegis_sdk.modules.connectors.ConnectorsModule` .

Operations: `api:POST /api/v1/connectors` · `api:GET /api/v1/connectors` · `api:GET /api/v1/connectors/{id}` · `api:PUT /api/v1/connectors/{id}` · `api:DELETE /api/v1/connectors/{id}` · `api:GET /api/v1/connectors/types` · `api:POST /api/v1/connectors/{id}/test` · `api:POST /api/v1/connectors/{id}/validate` · `api:GET`

`/api/v1/connectors/{id}/health` · `api:GET /api/v1/connectors/{id}/schema` · `api:GET /api/v1/connectors/{id}/metadata` · `api:POST /api/v1/connectors/{id}/query` · `api:POST /api/v1/connectors/{id}/attach` · `api:DELETE /api/v1/connectors/instances/{id}`

```

connector = await client.connectors.create(
    name="general-ledger",
    connector_type="postgres",
    config={"host": "ledger.internal", "database": "gl"},
    credential_id=stored_credential_id,
)
check = await client.connectors.test(connector_id=connector["id"])
await client.connectors.attach(connector_id=connector["id"], agent_id=agent["id"])

```

△ **validate** and **test** answer different questions. Validation checks the configuration is well-formed without touching the far side; test actually connects. A connector that validates and does not test is misconfigured in a way that only shows up when an agent queries it mid-objective — and the failure surfaces there, attributed to the agent rather than to the connector.

Credentials belong in the credential store, not in connector config. 08.8 catalogues that surface; referencing a stored credential by id keeps it rotatable without touching every connector that uses it.

Data source registry

Registered, described sources — the catalogue of where data comes from, distinct from the live connectors that read it.

capability	what it does
Create / update / delete	Manage a registration
List / get	Enumerate registered sources

Entry point: `sdk:aegis_sdk.modules.knowledge_govern.KnowledgeGovernModule` .

Operations: `api:GET /api/v1/data-source-registry` · `api:POST /api/v1/data-source-registry` · `api:GET /api/v1/data-source-registry/{id}` · `api:PUT /api/v1/data-source-registry/{id}` · `api:DELETE /api/v1/data-source-registry/{id}`

Data governance — classification, policy, consent

The controls over data as data, independent of the knowledge items that carry it.

Classifications

The levels data is labelled with, and what handling each requires.

capability	what it does
Create / update / delete	Define a classification
List / get	Enumerate them, with handling requirements

Operations: `api:GET /api/v1/data-governance/classifications` · `api:POST /api/v1/data-governance/classifications` · `api:GET /api/v1/data-governance/classifications/{id}` · `api:PUT /api/v1/data-governance/classifications/{id}` · `api:DELETE /api/v1/data-governance/classifications/{id}`

Data policies

capability	what it does
Create / update / delete	Author a data policy
List / get	Enumerate them
Evaluate	Ask what the policy set decides for an access

Operations: `api:GET /api/v1/data-governance/policies` · `api:POST /api/v1/data-governance/policies` · `api:GET /api/v1/data-governance/policies/{id}` · `api:PUT /api/v1/data-governance/policies/{id}` · `api:DELETE /api/v1/data-governance/policies/{id}` · `api:POST /api/v1/data-governance/policies/evaluate`

Consent

Where data subjects have a say, consent is a tracked record with a withdrawal path — the GDPR-shaped surface.

capability	what it does
Record	Capture a consent
List / get	Enumerate consents
Check	Ask whether consent covers a given use
Withdraw	Revoke a consent

Operations: `api:GET /api/v1/data-governance/consents` · `api:POST /api/v1/data-governance/consents` · `api:GET /api/v1/data-governance/consents/{id}` · `api:POST /api/v1/data-governance/consents/check` · `api:POST /api/v1/data-governance/consents/{id}/withdraw`

Retention

capability	what it does
Create / update	Define how long data is kept

capability	what it does
Get	Read one retention rule

Operations: `api:POST /api/v1/data-governance/retention` · `api:GET /api/v1/data-governance/retention/{id}` · `api:PUT /api/v1/data-governance/retention/{id}`

Compliance-driven retention policies — the ones a framework requires, with an execution surface — are catalogued in 08.7.

Entry point for all four: `sdk:aegis_sdk.modules.governance.GovernanceModule` .

```
decision = await client.governance.evaluate_access(
    principal_id=agent["id"],
    resource_id=dataset_id,
    action="read",
)
covered = await client.governance.check_consent(
    subject_id=subject_id,
    purpose="analytics",
)
```

Check consent before the use, not after the complaint. `consents/check` is cheap and the alternative is discovering the gap during a subject access request.

Lineage — where data came from and what depends on it

Two lineage surfaces, serving different questions.

Data governance lineage is a graph of nodes and edges you build explicitly: this dataset derived from those inputs.

capability	what it does
Create node / edge	Record a data asset and a derivation
Graph	Read the whole graph
Get node	One node in full
Upstream	Everything this derived from
Downstream	Everything derived from this
Impact	What changing this would affect

Operations: `api:POST /api/v1/data-governance/lineage/nodes` · `api:POST /api/v1/data-governance/lineage/edges` · `api:GET /api/v1/data-governance/lineage/graph` · `api:GET /api/v1/data-governance/lineage/nodes/{id}` · `api:GET /api/v1/data-governance/lineage/nodes/{id}/upstream` · `api:GET /api/v1/data-governance/lineage/nodes/{id}/downstream` · `api:GET /api/v1/data-governance/lineage/nodes/{id}/impact`

Entry points: `sdk:aegis_sdk.modules.governance.GovernanceModule` and `sdk:aegis_sdk.dataflow.graph.LineageGraphModule` .

Invocation lineage is recorded for you: which agent invoked what, with what data, on whose behalf.

capability	what it does
List / get	Enumerate invocation lineage records
Graph	The invocation graph
Export	Extract it for external analysis
Redact user	Remove one subject's data from the lineage record

Entry point: `sdk:aegis_sdk.dataflow.lineage.InvocationLineageModule` .

Operations: `api:GET /api/v1/lineage` · `api:GET /api/v1/lineage/{id}` · `api:GET /api/v1/lineage/graph` · `api:GET /api/v1/lineage/export` · `api:DELETE /api/v1/lineage/user/{user_id}`

```
upstream = await client.dataflow.graph.upstream(node_id=report_node_id)
impact = await client.governance.get_lineage_impact(node_id=source_node_id)

records = await client.dataflow.lineage.list(agent_id=agent["id"], limit=100)
await client.dataflow.lineage.redact_user(user_id=erasure_request_user_id)
```



`api:DELETE /api/v1/lineage/user/{user_id}` is the erasure surface, and it is not reversible. It exists to satisfy a right-to-erasure request. Running it removes that subject from the lineage record permanently — confirm the request is genuine before calling it, because there is no restore.

Impact analysis before a schema change is the cheapest version of this chapter.

`lineage/nodes/{id}/impact` answers what breaks, and the alternative is finding out from the downstream consumer.

Datasets

Named, addressable data collections — the unit agents and MCP bindings consume.

capability	what it does
Create / update / delete	Manage a dataset
List / get	Enumerate them

Entry point: `sdk:aegis_sdk.modules.mcp.McpModule` .

Operations: `api:GET /api/v1/datasets` · `api:POST /api/v1/datasets` · `api:GET /api/v1/datasets/{id}` · `api:PUT /api/v1/datasets/{id}` · `api:DELETE /api/v1/datasets/{id}`

Workspaces

Collaborative containers: members, work units, documents and messages in one governed scope. A workspace is also a bridge scope — see 08.5.

capability	what it does
Create / update / delete	Standard lifecycle
List / get	Enumerate and address workspaces
Archive / restore	Withdraw from active use, and bring back
Add / update / remove member	Manage membership and roles within it
Add / remove work unit	Attach executable work to the workspace
Attach / list / detach document	Manage its documents

Entry point: `sdk:aegis_sdk.modules.workspaces.WorkspacesModule` .

Operations: `api:POST /api/v1/workspaces` · `api:GET /api/v1/workspaces` · `api:GET /api/v1/workspaces/{id}` · `api:PATCH /api/v1/workspaces/{id}` · `api:DELETE /api/v1/workspaces/{id}` · `api:POST /api/v1/workspaces/{id}/archive` · `api:POST /api/v1/workspaces/{id}/restore` · `api:POST /api/v1/workspaces/{id}/members` · `api:PATCH /api/v1/workspaces/{id}/members/{member_id}` · `api:DELETE /api/v1/workspaces/{id}/members/{member_id}` · `api:POST /api/v1/workspaces/{id}/work-units` · `api:DELETE /api/v1/workspaces/{id}/work-units/{work_unit_id}` · `api:POST /api/v1/workspaces/{id}/documents` · `api:GET /api/v1/workspaces/{id}/documents` · `api:DELETE /api/v1/workspaces/{id}/documents/{document_id}`

```
ws = await client.workspaces.create(
    name="Q3 close",
    description="Working space for the Q3 close objective.",
)
await client.workspaces.add_member(workspace_id=ws["id"], user_id=controller_id)
await client.workspaces.add_work_unit(workspace_id=ws["id"], work_unit_id=unit["id"])
```

Archiving a workspace does not end its bridges. A scoped bridge created for a workspace outlives the workspace's archival unless it is completed or expired — check `api:GET /api/v1/scoped-bridges/for-workspace/{workspace_id}` as part of closing one down, or the channel stays open after the reason for it is gone.

How the pieces connect



Read it as: the top half is **data in motion** — where it comes from, what consumes it, what derives from what. The bottom half is **knowledge at rest** — what the organisation has written down, and who may read it. They meet at the clearance check, which is the one gate both paths pass through, and which is catalogued in 08.5.

Summary — what this chapter covered

area	entry point	scale
Knowledge items	<code>standup.knowledge</code> · <code>modules.knowledge_govern</code>	17 operations
Knowledge categories	<code>modules.knowledge_govern</code>	4 operations
Share policies	<code>modules.knowledge_govern</code>	8 operations
Content reviews	<code>modules.knowledge_govern</code>	8 operations
Content templates	<code>modules.knowledge_govern</code>	5 operations
Connectors	<code>modules.connectors</code>	14 operations
Data source registry	<code>modules.knowledge_govern</code>	5 operations
Data governance	<code>modules.governance</code>	26 operations
Lineage	<code>dataflow.lineage</code> · <code>dataflow.graph</code> · <code>modules.governance</code>	12 operations
Datasets	<code>modules.mcp</code>	5 operations
Workspaces	<code>modules.workspaces</code>	15 operations

Next: 08.7 — Evidence and operations

08.7 — Evidence and operations

This chapter enumerates **what the platform recorded about itself** and **how you get it out**. It is the part of the catalogue a compliance reviewer reads first, and the part an architect reaches for when something has already gone wrong.

The organising distinction is between **evidence** and **telemetry**, and they are not the same asset. Evidence is the append-only, verifiable record of what happened and who authorised it — it is produced for an auditor, it does not change, and its integrity is itself checkable. Telemetry is measurement: aggregated, sampled, useful for operating the system, and not something you would put in front of a regulator.

Aegis provides the proof; the deploying organisation keeps the accountability. That is the whole purpose of this chapter's surface — it exists so that your organisation can evidence its own governance to whoever asks, not so that the evidence-keeping becomes someone else's problem.

The audit spine

The append-only record. Every governed act lands here, attributed along the trust chain that authorised it.

capability	what it does
List logs	Query the audit record
Get log	One entry in full
User activity	Everything one principal did
Resource history	Everything that happened to one resource
Export	Extract a set of entries for external retention

Entry point: `sdk:aegis_sdk.modules.observe_audit.ObserveAuditModule` .

Operations: `api:GET /api/v1/audit/logs` · `api:GET /api/v1/audit/logs/{id}` · `api:GET /api/v1/audit/users/{user_id}` · `api:GET /api/v1/audit/resources/{resource_type}/{resource_id}` · `api:GET /api/v1/audit/export`

```
history = await client.observe_audit.resource_history(
    resource_type="objective",
    resource_id=objective["id"],
)
activity = await client.observe_audit.user_activity(
    user_id=departing_user_id,
    since="2026-01-01T00:00:00Z",
)
```

The trust-specific face of the same spine — `api:GET /api/v1/trust/audit/trace/{event_id}` and `api:GET /api/v1/trust/audit/by-human/{human_id}` — is catalogued in 08.5. Use the trust surface when the question is *by what authority*; use this one when the question is *what happened to this thing*.

Alert thresholds

Turning the audit stream into notification: thresholds that fire, and a lifecycle for what fires.

capability	what it does
List / create thresholds	Define what should raise an alert
Acknowledge	Record that someone has seen it
Resolve	Close it out

Entry point: `sdk:aegis_sdk.modules.observe_audit.ObserveAuditModule` .

Operations: `api:GET /api/v1/alerts/thresholds` · `api:POST /api/v1/alerts/thresholds` · `api:POST /api/v1/alerts/{id}/acknowledge` · `api:POST /api/v1/alerts/{id}/resolve`

An acknowledged alert is not a resolved alert, and a dashboard that counts only unacknowledged ones hides every problem someone has looked at and not fixed. Query both states.

Compliance

The largest evidence surface: frameworks, controls, assessment, scoring, evidence generation, and reports.

Frameworks and assessment

capability	what it does
List / get frameworks	Which compliance frameworks this deployment tracks
Framework controls	The individual controls within one
Run assessment	Evaluate the organisation against a framework
Score	The current compliance score
Dashboard	The compliance position at a glance
Events	Compliance-relevant events
Violations	Where the organisation is out of compliance
Acknowledge alert	Record attention on a compliance alert
Health	Whether the compliance subsystem itself is sound

Audit verification

capability	what it does
Verify audit	Check the audit record has not been tampered with
Chain integrity	Verify the cryptographic chain over the record

capability	what it does
List audit entries	The compliance view of the audit spine
Export audit	Extract audit evidence

SOC 2 and HIPAA

capability	what it does
SOC 2 status	Where the organisation stands
Generate SOC 2 evidence	Produce the evidence pack
Export SOC 2 evidence	Extract it
HIPAA status / settings	Whether HIPAA mode is on, and how it is configured
Enable / disable HIPAA	Turn the regime on or off

Retention and reports

capability	what it does
List / get retention policies	What is kept, and for how long
Create / update policy	Define retention
Execute retention	Apply it — deletes what has aged out
Create / list / get report	Compliance reports
Report summary	The headline view
Report PDF	The report as a document
Export report	Extract it

Entry point: `sdk:aegis_sdk.modules.compliance.ComplianceModule` .

Operations — frameworks and assessment: `api:GET /api/v1/compliance/frameworks` · `api:GET /api/v1/compliance/frameworks/{id}` · `api:GET /api/v1/compliance/frameworks/{id}/controls` · `api:POST /api/v1/compliance/assess` · `api:GET /api/v1/compliance/score` · `api:GET /api/v1/compliance/dashboard` · `api:GET /api/v1/compliance/events` · `api:GET /api/v1/compliance/violations` · `api:POST /api/v1/compliance/alerts/{id}/acknowledge` · `api:GET /api/v1/compliance/health`

Verification: `api:GET /api/v1/compliance/audit/verify` · `api:GET /api/v1/compliance/chain/integrity` · `api:GET /api/v1/compliance/audit/entries` · `api:POST /api/v1/compliance/audit/export`

SOC 2 and HIPAA: `api:GET /api/v1/compliance/soc2/status` · `api:POST /api/v1/compliance/soc2/evidence` · `api:GET /api/v1/compliance/soc2/evidence/export` · `api:GET /api/v1/compliance/hipaa/status` · `api:GET /api/v1/compliance/hipaa/settings` ·

`api:POST /api/v1/compliance/hipaa/enable` · `api:POST /api/v1/compliance/hipaa/disable`

Retention and reports: `api:GET /api/v1/compliance/retention/policies` · `api:POST /api/v1/compliance/retention/execute` · `api:GET /api/v1/compliance/reports` · `api:POST /api/v1/compliance/reports` · `api:GET /api/v1/compliance/report/summary` · `api:GET /api/v1/compliance/reports/{id}.pdf` · `api:POST /api/v1/compliance/export`

```
integrity = await client.compliance.get_chain_integrity()
if not integrity["valid"]:
    raise RuntimeError("audit chain integrity check failed")

await client.compliance.run_assessment(framework="soc2")
evidence = await client.compliance.generate_soc2_evidence(
    period_start="2026-07-01",
    period_end="2026-09-30",
)
```

🚫 **Verify the chain before you present the evidence, not after.** `api:GET`

`/api/v1/compliance/chain/integrity` is what makes an evidence pack worth anything — an export taken from an unverified record is a document, not evidence. The check is cheap and it is the one thing a reviewer will ask you whether you ran.

🚫 `api:POST /api/v1/compliance/retention/execute` **deletes**. It applies the retention policies, which means it removes records that have aged past their window. Run `api:GET /api/v1/compliance/retention/policies` and read what is about to be applied first — a policy with a wrong window destroys evidence you are required to hold, and the deletion is not reversible.

Enabling HIPAA mode changes platform behaviour, not just a flag. It tightens handling requirements across data surfaces. Enable it in a non-production environment first and confirm your integrations still function — the symptom of enabling it late is a set of refusals on paths that worked yesterday.

Analytics

Aggregate measurement over work, agents, pools and cost. This is telemetry — use it to operate the system, not to evidence it.

capability	what it does
Overview	The headline numbers
Summary	A composed multi-section view
Tasks	Task throughput and outcomes
Agents / agent performance	Per-agent measurement
Top agents	The most active
Pools / pool utilization	How heavily pools are used

capability	what it does
Costs / cost breakdown	What it is costing, and where
Export costs / metrics	Extract for external analysis
Trends	Completion rate over time
Usage history / breakdown	Consumption over time and by dimension
SLA	Service-level measurement
HITL	How much human-in-the-loop involvement is occurring
Verification gradient	The distribution of decisions across the gradient
Workspace / team metrics	Measurement scoped to a workspace or team

Entry point: `sdk:aegis_sdk.modules.analytics.AnalyticsModule` .

Operations: `api:GET /api/v1/analytics/overview` · `api:GET /api/v1/analytics/summary` · `api:GET /api/v1/analytics/tasks` · `api:GET /api/v1/analytics/agents` · `api:GET /api/v1/analytics/agents/{agent_id}/performance` · `api:GET /api/v1/analytics/top-agents` · `api:GET /api/v1/analytics/pools` · `api:GET /api/v1/analytics/pools/{pool_id}/utilization` · `api:GET /api/v1/analytics/costs` · `api:GET /api/v1/analytics/export` · `api:GET /api/v1/analytics/trends/completion-rate` · `api:GET /api/v1/analytics/sla` · `api:GET /api/v1/analytics/hitl` · `api:GET /api/v1/analytics/verification-gradient` · `api:GET /api/v1/analytics/workspaces/{workspace_id}` · `api:GET /api/v1/analytics/teams/{team_id}`

```
overview = await client.analytics.overview(period="30d")
gradient = await client.analytics.verification_gradient()
costs = await client.analytics.cost_breakdown(group_by="agent")
```

verification-gradient is the most under-used call in this chapter. It shows how decisions are distributing across auto-approved, flagged, held and blocked — and a governed organisation whose gradient is 99% auto-approved is either very well tuned or not enforcing anything. The distribution is the diagnostic; the count of blocked actions alone is not.

Metrics

Lower-level than analytics: the operational measures, including a recording surface for your own.

capability	what it does
Dashboard	The operational dashboard numbers
Summary	Rolled-up measures
Timeseries	Measures over time
Executions	Execution-level measurement
Errors	The top errors

capability	what it does
Agent metrics	Per-agent operational measures
Deployment metrics	Per-deployment measures — see 08.8
Record	Push your own metric

Entry point: `sdk:aegis_sdk.modules.metrics.MetricsModule` .

Operations: `api:GET /api/v1/metrics/dashboard` · `api:GET /api/v1/metrics/summary` · `api:GET /api/v1/metrics/timeseries` · `api:GET /api/v1/metrics/executions` · `api:GET /api/v1/metrics/errors` · `api:GET /api/v1/metrics/agents/{agent_id}` · `api:GET /api/v1/metrics/deployments/{deployment_id}` · `api:POST /api/v1/metrics/record`

```
await client.metrics.record_metric(
    name="reconciliation.unmatched_entries",
    value=12,
    labels={"ledger": "4400", "period": "2026-09"},
)
errors = await client.metrics.top_errors(period="24h")
```

Invocation analytics

Measurement scoped to invocations — who called what, how often, at what cost. This is the surface that answers commercial questions about usage.

capability	what it does
Summary	Invocation totals
By application	Grouped by the calling application
By agent	Grouped by the agent invoked
SDK overview	Measurement of SDK-originated traffic

Entry point: `sdk:aegis_sdk.modules.metrics.MetricsModule` .

Operations: `api:GET /api/v1/invocation-analytics/summary` · `api:GET /api/v1/invocation-analytics/by-application` · `api:GET /api/v1/invocation-analytics/by-agent` · `api:GET /api/v1/sdk-metrics/overview`

Notifications

Getting things in front of people. Notifications have channels, per-user preferences, and a live stream.

capability	what it does
List / get	The notification queue

capability	what it does
Mark read (one, many, all)	Clear notifications
Delete (one, many, all)	Remove them
Stats	Queue depth and read rates
List / create / update / delete channel	Manage delivery channels
Get channel / test channel	Inspect one, and confirm it delivers
Get / update preferences	Per-user notification settings
Stream	Consume notifications live over SSE

Entry point: `sdk: aegis_sdk.modules.notifications.NotificationsModule` .

Operations: `api:GET /api/v1/notifications` · `api:GET /api/v1/notifications/{id}` · `api:PATCH /api/v1/notifications/{id}/read` · `api:PATCH /api/v1/notifications/read` · `api:PATCH /api/v1/notifications/read-all` · `api:DELETE /api/v1/notifications/{id}` · `api:DELETE /api/v1/notifications` · `api:DELETE /api/v1/notifications/all` · `api:GET /api/v1/notifications/stats` · `api:GET /api/v1/notifications/channels` · `api:POST /api/v1/notifications/channels` · `api:GET /api/v1/notifications/channels/{id}` · `api:PUT /api/v1/notifications/channels/{id}` · `api:DELETE /api/v1/notifications/channels/{id}` · `api:POST /api/v1/notifications/channels/{id}/test` · `api:GET /api/v1/notifications/preferences` · `api:PUT /api/v1/notifications/preferences` · `api:GET /api/v1/notifications/stream/sse`

```
await client.notifications.test_channel(channel_id=ops_channel_id)

async for note in client.notifications.stream():
    handle(note)
```

Test the channel when you create it. An untested notification channel accepts every notification and delivers none, and the symptom is a held decision nobody answers — which reads as an unresponsive team rather than as a broken channel. This is the same failure shape as an untested pseudo-agent channel in 08.3, and it is worth checking both together.

Webhooks

Outbound integration: the platform calls you when something happens.

capability	what it does
Create / update / delete	Manage a webhook
List / get	Enumerate them
Event types	Which events you may subscribe to
Test	Fire a test delivery

capability	what it does
Deliveries	The delivery history for one webhook
Retry delivery	Re-send a failed delivery
Rotate secret	Change the signing secret

Entry point: `sdk:aegis_sdk.modules.webhooks.WebhooksModule` .

Operations: `api:POST /api/v1/webhooks` · `api:GET /api/v1/webhooks` · `api:GET /api/v1/webhooks/{id}` · `api:PUT /api/v1/webhooks/{id}` · `api:DELETE /api/v1/webhooks/{id}` · `api:GET /api/v1/webhooks/events` · `api:POST /api/v1/webhooks/{id}/test` · `api:GET /api/v1/webhooks/{id}/deliveries` · `api:POST /api/v1/webhooks/deliveries/{delivery_id}/retry` · `api:POST /api/v1/webhooks/{id}/rotate-secret`

```
hook = await client.webhooks.create(
    url="https://ops.example.com/aegis-events",
    events=["objective.completed", "decision.raised", "trust.revoked"],
)
await client.webhooks.test(webhook_id=hook["id"])

failed = await client.webhooks.get_deliveries(webhook_id=hook["id"], status="failed")
```

⚠ **Check deliveries, not just the webhook record.** A webhook whose endpoint started returning errors keeps its `active` state and stops delivering. The webhook listing looks healthy; the deliveries listing is where the failures are. Rotate the secret on any suspicion of exposure — it is a single call and it invalidates every signature an attacker might have captured.

Settings

Organisation and user configuration, with backup, export, import and an audit log of its own.

capability	what it does
Get / update organisation	Organisation-wide settings
Get / update user	Per-user settings
List / create / delete backup	Snapshot settings
Restore backup	Return to a snapshot
Export	Extract settings
Import / preview import	Apply settings from elsewhere, previewing first
Audit log / get entry	What changed, when, and by whom
Revert audit entry	Undo one settings change

capability	what it does
Site config	Public-facing configuration

Entry point: `sdk:aegis_sdk.modules.settings.SettingsModule`.

Operations: `api:GET /api/v1/settings/organization` · `api:PUT /api/v1/settings/organization` · `api:GET /api/v1/settings/user` · `api:PUT /api/v1/settings/user` · `api:GET /api/v1/settings/backups` · `api:POST /api/v1/settings/backups` · `api:POST /api/v1/settings/backups/restore/{backup_id}` · `api:DELETE /api/v1/settings/backups/{backup_id}` · `api:POST /api/v1/settings/export` · `api:POST /api/v1/settings/import` · `api:POST /api/v1/settings/import/preview` · `api:GET /api/v1/settings/audit-log` · `api:GET /api/v1/settings/audit-log/{id}` · `api:POST /api/v1/settings/audit-log/revert` · `api:GET /api/v1/site-config` · `api:PUT /api/v1/site-config`

```
await client.settings.create_backup(label="pre-Q4-config-change")

preview = await client.settings.preview_import(payload=incoming_settings)
if not preview["conflicts"]:
    await client.settings.import_settings(payload=incoming_settings)
```

 **Preview every settings import.** `import/preview` reports the conflicts an import would resolve silently, and a settings import that overwrites a governance-relevant value is a control change with no approval attached to it. Take a backup first; the settings audit log tells you what changed, and `audit-log/revert` undoes one entry — but only if you know which one.

Putting evidence together

The sequence that produces a defensible evidence pack, in order:

step	call
1. Verify the record is sound	<code>api:GET /api/v1/compliance/chain/integrity</code>
2. Assess against the framework	<code>api:POST /api/v1/compliance/assess</code>
3. Read what failed	<code>api:GET /api/v1/compliance/violations</code>
4. Generate the evidence	<code>api:POST /api/v1/compliance/soc2/evidence</code>
5. Extract it	<code>api:GET /api/v1/compliance/soc2/evidence/export</code>
6. Extract the underlying audit	<code>api:POST /api/v1/compliance/audit/export</code>

Step 1 is the one that is skipped and the one that matters: an evidence pack whose chain was never verified proves that the platform *reported* something, not that the record is intact.

Summary — what this chapter covered

area	entry point	scale
Audit spine	<code>modules.observe_audit</code>	5 operations
Alert thresholds	<code>modules.observe_audit</code>	4 operations
Compliance	<code>modules.compliance</code>	28 operations
Analytics	<code>modules.analytics</code>	16 operations
Metrics	<code>modules.metrics</code>	8 operations
Invocation analytics	<code>modules.metrics</code>	4 operations
Notifications	<code>modules.notifications</code>	18 operations
Webhooks	<code>modules.webhooks</code>	10 operations
Settings	<code>modules.settings</code>	16 operations

Next: 08.8 — Commercial and extensibility

08.8 — Commercial and extensibility

This chapter closes the catalogue with the two surfaces that surround everything else: **what the platform costs** and **what attaches to it from outside**.

They sit together because they share a property the rest of the catalogue does not. Every other chapter describes things inside the governed organisation — roles, agents, work, knowledge. These describe the boundary: metering, which measures what crossed it; and integration, which is the crossing itself. An external agent, a gateway, a registered surface and a third-party application are all ways something outside becomes something the organisation can govern.

Everything that attaches from outside is governed on the way in. An external agent is bounded by an envelope like any other; an application holds explicit, revocable grants; a tool agent publishes a capability under a policy. There is no side door.

03.4 teaches the grant model this chapter inventories.

What the commercial surface is actually selling

Worth stating plainly before the metering tables, because it is the thing most often misread at a procurement review.

You keep the accountability. Aegis gives you the proof. Your organisation is accountable for what its agents do, and that accountability cannot be delegated to a platform, a vendor, or a model — "the AI did it" is not a defence anywhere that matters. What this platform provides is the **evidence** that your governance was real: bounded mandates, tamper-evident audit, fail-closed permissions, and attestable lineage, against an external open standard.

So the artifact the commercial surface produces is a **proof artifact** — a signed, exportable record of conformance, audit chain and human gates that a CISO can put in front of an auditor, or an insurer can underwrite against. It makes your residual risk documentable. It does not move your risk onto someone else, and no offering here should be read as doing so.

The practical consequence for an evaluator: the question to ask of this chapter is *"can we evidence our governance to a third party?"* and not *"who is on the hook?"* The second question has the same answer it had before Aegis — you are — and 08.7 is where the answer to the first one is produced.

Billing and metering

What the organisation is consuming, what it will cost, and the payment surface.

capability	what it does
Usage / usage details	What has been consumed, summarised or itemised
Quotas	The consumption ceilings in force
Update / adjust quota	Change a ceiling

capability	what it does
Periods / current period / get	Billing periods
Close period	Finalise a period
Estimate cost	Price a hypothetical before committing to it
Pricing	The rate card
Plans	Available plans
Summary	The billing position at a glance
Setup / remove payment method	Manage payment instruments
Set default payment method	Choose which one is used
Billing contact	Who receives billing correspondence
Usage alerts	Thresholds that warn before a ceiling is hit
Metered items / reconcile	Attach and reconcile metered usage

Entry point: `sdk:aegis_sdk.revenue.billing.BillingModule` .

Operations: `api:GET /api/v1/billing/usage` · `api:GET /api/v1/billing/usage/details` · `api:GET /api/v1/billing/quotas` · `api:PUT /api/v1/billing/quotas/{id}` · `api:POST /api/v1/billing/quotas/adjust` · `api:GET /api/v1/billing/periods` · `api:GET /api/v1/billing/periods/current` · `api:GET /api/v1/billing/periods/{id}` · `api:POST /api/v1/billing/periods/{id}/close` · `api:POST /api/v1/billing/estimate` · `api:GET /api/v1/billing/pricing` · `api:GET /api/v1/billing/plans` · `api:GET /api/v1/billing/summary` · `api:POST /api/v1/billing/setup-payment-method` · `api:DELETE /api/v1/billing/payment-methods/{id}` · `api:POST /api/v1/billing/payment-methods/{id}/default` · `api:GET /api/v1/billing/contact` · `api:PUT /api/v1/billing/contact` · `api:GET /api/v1/billing/alerts` · `api:PUT /api/v1/billing/alerts` · `api:POST /api/v1/billing/metered-items` · `api:POST /api/v1/billing/metered-reconcile`

```
current = await client.revenue.billing.get_current_period()
usage = await client.revenue.billing.get_usage()

estimate = await client.revenue.billing.estimate_cost(
    workload={"agent_executions": 50_000, "knowledge_items": 2_000},
)

await client.revenue.billing.update_usage_alerts(
    thresholds=[{"resource": "agent_executions", "percent": 80}],
)
```

⚠ **Set usage alerts before you need them.** A quota reached mid-objective stops work, and the refusal arrives attributed to the agent that happened to be running — not to the quota. The alert at 80% is what turns an outage into a conversation.

Estimate before a large workload, not after. `estimate` prices a hypothetical against the live rate card, which is the only way to answer "what will this campaign cost" without running it.

Subscriptions

The plan the organisation is on, and the self-service surface for changing it.

capability	what it does
Current	The active subscription
Plans	What is available
Subscribe	Take a plan
Upgrade	Move to a higher plan
Cancel / reactivate	End a subscription, or resume it
Usage	Consumption against the plan
Invoices	Billing documents
Portal	A hosted self-service session

Entry points: `sdk:aegis_sdk.revenue.subscriptions.SubscriptionsModule` ,
`sdk:aegis_sdk.revenue.plans.PlansModule` and
`sdk:aegis_sdk.revenue.invoices.InvoicesModule` .

Operations: `api:GET /api/v1/subscriptions/current` · `api:GET /api/v1/subscriptions/plans` ·
`api:POST /api/v1/subscriptions/subscribe` · `api:PUT /api/v1/subscriptions/upgrade` ·
`api:POST /api/v1/subscriptions/cancel` · `api:POST /api/v1/subscriptions/reactivate` ·
`api:GET /api/v1/subscriptions/usage` · `api:GET /api/v1/subscriptions/invoices` · `api:POST /api/v1/subscriptions/portal`

```
sub = await client.revenue.subscriptions.get()
comparison = await client.revenue.plans.compare_tiers()
```

Cancellation is not immediate termination. A cancelled subscription runs to the end of its period and `reactivate` reverses it within that window. Treating cancel as a kill switch leaves the organisation running and billing until the period closes.

Licences

Where the deployment is licensed rather than subscribed — generation, validation, and revocation.

capability	what it does
Generate	Issue a licence
Validate	Check one is genuine and current

capability	what it does
Revoke	Withdraw it
Status	The licence position
Usage	Consumption against one licence
Editions	The licence editions available

Entry point: `sdk:aegis_sdk.revenue.licenses.LicensesModule` .

Operations: `api:POST /api/v1/licenses/generate` · `api:POST /api/v1/licenses/validate` · `api:POST /api/v1/licenses/{id}/revoke` · `api:GET /api/v1/licenses/status` · `api:GET /api/v1/licenses/{id}/usage` · `api:GET /api/v1/licenses/editions`

🚫 **A licence governs the deployment, not this SDK.** The harness you are reading is obtained by cloning it and working inside the directory — there is no package index entry for it, no version to pin, and nothing to install. What a licence covers is the platform your client points at.

The distinction matters because one nearby thing genuinely is an installed package: **kailash-enterprise** is the runtime Aegis is *built on*, and it is a dependency of the platform rather than something an architect adds to their own project. Two different artifacts, one of which is installed and one of which is cloned — reaching for a package manager to obtain this SDK is the error the distinction exists to prevent.

Features and quotas

Feature gating — whether a capability is available on this plan — and the quota surface that bounds consumption of it.

capability	what it does
Check	Is this feature available to this organisation?
Limits	The limits this plan imposes
Tiers	What each tier includes
Get / update quota	Read and change a quota
Check limit	Whether a specific limit has been reached

Entry points: `sdk:aegis_sdk.revenue.features.FeaturesModule` ,
`sdk:aegis_sdk.revenue.quotas.QuotasModule` and `sdk:aegis_sdk.revenue.usage.UsageModule` .

Operations: `api:GET /api/v1/features/check` · `api:GET /api/v1/features/limits` · `api:GET /api/v1/features/tiers`

```

available = await client.revenue.features.check(feature="advanced_lineage")
if not available["enabled"]:
    fall_back_to_basic_lineage()

```

Check the feature before building the path that depends on it. A feature check that runs only in the error handler means the first time you learn a capability is unavailable is when a user hits it.

Applications, grants and policy

An application is a third party the organisation has admitted: it holds explicit grants over named tool agents, operates under an invocation policy, and every call it makes is attributable to it.

capability	what it does
Create / update / delete	Manage an application
List / get / summary	Enumerate and inspect
Change status	Move it between states
Archive	Retire it
Freeze status	Whether it is frozen, and why
Grants	List, grant and revoke access to named tool agents
Policy	Read and set the invocation policy
Delegation matrix	Which of its operators may invoke what
Operators	List, add and remove the humans behind it
Invocations	What it has called
Audit events	Its governed act history
Notifications	What it has been told

Entry point: `sdk:aegis_sdk.modules.applications.ApplicationsModule` .

Operations: `api:POST /api/v1/applications` · `api:GET /api/v1/applications` · `api:GET /api/v1/applications/{id}` · `api:PUT /api/v1/applications/{id}` · `api:DELETE /api/v1/applications/{id}` · `api:GET /api/v1/applications/{id}/summary` · `api:PATCH /api/v1/applications/{id}/status` · `api:GET /api/v1/applications/{id}/freeze-status` · `api:GET /api/v1/applications/{id}/grants` · `api:POST /api/v1/applications/{id}/grants` · `api:DELETE /api/v1/applications/{id}/grants/{grant_id}` · `api:GET /api/v1/applications/{id}/policy/{policy_id}` · `api:PUT /api/v1/applications/{id}/policy/{policy_id}` · `api:GET /api/v1/applications/{id}/delegation-matrix` · `api:PUT /api/v1/applications/{id}/delegation-matrix` · `api:GET /api/v1/applications/{id}/operators` · `api:POST /api/v1/applications/{id}/operators` · `api:DELETE`

```
/api/v1/applications/{id}/operators/{operator_id} · api:GET
/api/v1/applications/{id}/invocations · api:GET /api/v1/applications/{id}/notifications ·
api:GET /api/v1/applications/{id}/audit-events
```

```
app = await client.applications.create(
    name="treasury-dashboard",
    description="Read-only liquidity view for the treasury team.",
)
await client.applications.grant_tool_agent(
    application_id=app["id"],
    tool_agent_id=liquidity_reader_id,
)
matrix = await client.applications.get_delegation_matrix(application_id=app["id"])
```

🛑 **Revoking a grant is what stops an application; deleting the application is not.** A deleted application's outstanding grants are the thing to check — and `api:GET /api/v1/applications/{id}/invocations` is how you establish what it actually did before you retire it. Retire in that order: read the invocations, revoke the grants, then delete.

The delegation matrix is where over-broad access hides. Grants say which tool agents the application may reach; the matrix says which of its operators may invoke them. An application with narrow grants and a permissive matrix gives every operator the full grant set, which is rarely what was intended.

Tool agents — publishing a governed capability

A tool agent is a capability other agents and applications invoke, wrapped in governance. It is the primary extension point of the platform.

capability	what it does
Create / update / delete	Manage a tool agent
List / get	Enumerate them
Change status	Move it through its lifecycle
Invoke	Call it
Envelope summary	The bounds it operates within
Consumers	Who is using it
Invocations	What it has been asked to do
Components	List, add and remove the parts it is built from
Impact	What would be affected by changing it

Entry point: `sdk:aegis_sdk.modules.tool_agents.ToolAgentsModule` .

Operations: `api:POST /api/v1/tool-agents` · `api:GET /api/v1/tool-agents` · `api:GET /api/v1/tool-agents/{id}` · `api:PUT /api/v1/tool-agents/{id}` · `api:PATCH /api/v1/tool-agents/{id}/status` · `api:POST /api/v1/tool-agents/{id}/invoke` · `api:GET /api/v1/tool-agents/{id}/envelope-summary` · `api:GET /api/v1/tool-agents/{id}/consumers` · `api:GET /api/v1/tool-agents/{id}/invocations` · `api:GET /api/v1/tool-agents/{id}/components` · `api:POST /api/v1/tool-agents/{id}/components` · `api:DELETE /api/v1/tool-agents/{id}/components/{component_id}` · `api:GET /api/v1/tool-agents/{id}/impact`

```
impact = await client.tool_agents.get_impact(tool_agent_id=ta_id)
consumers = await client.tool_agents.list_consumers(tool_agent_id=ta_id)
```

Check **impact** and **consumers** before changing a published tool agent. It is a shared capability by design, so a change to its behaviour propagates to every consumer silently — they are not notified, and their next invocation gets the new behaviour. 03.3 owns the lifecycle discipline.

External agents

Agents that live outside this deployment and are invoked across the boundary. They are registered, governed and metered like anything else.

capability	what it does
Create / update / delete	Register and manage one
List / get	Enumerate them
Invoke	Call across the boundary
Invocations	What has been called, and what came back

Entry point: `sdk:aegis_sdk.modules.integrations.IntegrationsModule` .

Operations: `api:POST /api/v1/external-agents` · `api:GET /api/v1/external-agents` · `api:GET /api/v1/external-agents/{id}` · `api:PATCH /api/v1/external-agents/{id}` · `api:DELETE /api/v1/external-agents/{id}` · `api:POST /api/v1/external-agents/{id}/invoke` · `api:GET /api/v1/external-agents/{id}/invocations`

```
ext = await client.integrations.create_external_agent(
    name="partner-risk-scorer",
    endpoint_url="https://partner.example.com/score",
    auth_type="api_key",
    credential_id=partner_credential_id,
)
result = await client.integrations.invoke_external_agent(
    external_agent_id=ext["id"],
    payload={"counterparty_id": cp_id},
)
```

🚫 **An external agent's governance fields are separate permissions on purpose.** Its budget, rate limits and credentials are not ordinary update fields — changing them is a governance act, and the platform gates them accordingly. If an update is refused where the general update succeeded, that is the gate working, not a bug.

Gateways and deployments

Gateways front external traffic; deployments are the runnable units the platform manages.

capability	gateways	deployments
Create	yes	yes
List	yes	yes
Get	—	yes
Start / stop	—	yes
Redeploy	—	yes

Entry point: `sdk:aegis_sdk.modules.integrations.IntegrationsModule` .

Operations: `api:GET /api/v1/gateways` · `api:POST /api/v1/gateways` · `api:GET /api/v1/deployments` · `api:POST /api/v1/deployments` · `api:GET /api/v1/deployments/{id}` · `api:POST /api/v1/deployments/{id}/start` · `api:POST /api/v1/deployments/{id}/stop` · `api:POST /api/v1/deployments/{id}/redeploy`

```
await client.integrations.redeploy_deployment(deployment_id=d_id)
metrics = await client.metrics.deployment_metrics(deployment_id=d_id)
```

Per-deployment measurement is catalogued in 08.7. The platform-level deployment architecture — clusters, images, networks — is part 07, and is a different subject from these records.

Credentials

The secret store. Credentials are stored once, referenced by id everywhere, and rotated without touching the things that use them.

capability	what it does
Store	Put a secret in
List / get	Enumerate metadata — never the secret
Retrieve	Read a secret, as a governed act
Rotate	Replace the secret, keeping the reference stable
Revoke	Withdraw it immediately

capability	what it does
Delete	Remove the record
Types / statuses	The kinds available and the states one can be in

Entry point: `sdk:aegis_sdk.modules.credentials.CredentialsModule` .

Operations: `api:POST /api/v1/credentials` · `api:GET /api/v1/credentials` · `api:GET /api/v1/credentials/{id}` · `api:POST /api/v1/credentials/{id}/retrieve` · `api:POST /api/v1/credentials/{id}/rotate` · `api:POST /api/v1/credentials/{id}/revoke` · `api:DELETE /api/v1/credentials/{id}` · `api:GET /api/v1/credentials/types` · `api:GET /api/v1/credentials/statuses`

```
cred = await client.credentials.store(
    name="partner-api-key",
    credential_type="api_key",
    value=secret_value,
)
# Reference it by id from connectors and external agents; rotate in one place
await client.credentials.rotate(credential_id=cred["id"], value=new_secret)
```

Rotation is the reason to reference rather than embed. A secret pasted into a connector configuration and an external agent's auth block must be changed in both places and in whatever else copied it; a stored credential referenced by id is rotated once. `retrieve` is itself a governed, audited act — if you find it being called on a hot path, something is re-reading a secret it should hold.

Surface registry

The catalogue of what this deployment publishes: which surfaces exist and which are registered as available.

capability	what it does
Manifest	The full surface manifest
List / get registrations	What is registered
Create / update / delete registration	Manage a registration

Entry point: `sdk:aegis_sdk.modules.surfaces.SurfacesModule` .

Operations: `api:GET /api/v1/surface-registry` · `api:GET /api/v1/surface-registry/registrations` · `api:GET /api/v1/surface-registry/registrations/{id}` · `api:POST /api/v1/surface-registry/registrations` · `api:PATCH /api/v1/surface-registry/registrations/{id}` · `api:DELETE /api/v1/surface-registry/registrations/{id}`

```
manifest = await client.surfaces.manifest()
```

`manifest` is worth calling once against any deployment you are new to — it is the deployment's own statement of what it publishes, which is a useful cross-check against this catalogue.

LLM providers and models

Which models the deployment may use, what they cost, and whether they are healthy.

capability	what it does
List / create / update provider	Manage provider configuration
Provider health	Whether a provider is reachable
Refresh provider cache	Re-read a provider's model list
List models / all models	What is available
Validate model	Check a model name is usable
Model metadata	Capabilities and limits of one model
Rate card / pricing	What each model costs

Entry point: `sdk:aegis_sdk.modules.llm_providers.LlmProvidersModule` .

Operations: `api:GET /api/v1/llm/providers` · `api:POST /api/v1/llm/providers` · `api:PUT /api/v1/llm/providers/{id}` · `api:GET /api/v1/llm/providers/{id}/health` · `api:POST /api/v1/llm/providers/{id}/refresh` · `api:GET /api/v1/llm/models` · `api:GET /api/v1/llm/models/all` · `api:POST /api/v1/llm/models/validate` · `api:GET /api/v1/llm/models/{provider}/{model}/metadata` · `api:GET /api/v1/llm/pricing`

```
health = await client.llm_providers.check_provider_health(provider_id=p_id)
ok = await client.llm_providers.validate_model(provider="openai", model=model_name)
rates = await client.llm_providers.get_rate_card()
```

Validate a model name before configuring an agent with it. An agent configured against a model the provider has retired fails at execution time, and the error surfaces as an agent failure rather than as a configuration problem. `refresh` re-reads the provider's list, which is what you want after a provider deprecates something.

Administrative surface

Cross-cutting administration: pools, agent assignments, and the access views that answer *who can reach what*.

capability	what it does
Pools	Create, list, get, update, delete and archive pools
Pool stats	Statistics for one pool

capability	what it does
Agent assignments	List, get, create and delete assignments
Bulk delete assignments	Remove many at once
Role hierarchy	The administrative view of the role tree
Access tree	Who can reach what, as a tree
Access matrix	The same, as a matrix
Access audit	The access-review record
Export access	Extract the access picture for review

Entry point: `sdk:aegis_sdk.modules.admin.AdminModule` .

Operations: `api:GET /api/v1/admin/pools` · `api:POST /api/v1/admin/pools` · `api:GET /api/v1/admin/pools/{id}` · `api:PUT /api/v1/admin/pools/{id}` · `api:DELETE /api/v1/admin/pools/{id}` · `api:POST /api/v1/admin/pools/{id}/archive` · `api:GET /api/v1/admin/pools/{id}/stats` · `api:GET /api/v1/admin/agent-assignments` · `api:POST /api/v1/admin/agent-assignments` · `api:GET /api/v1/admin/agent-assignments/{id}` · `api:DELETE /api/v1/admin/agent-assignments/{id}` · `api:POST /api/v1/admin/agent-assignments/bulk-delete` · `api:GET /api/v1/admin/roles/hierarchy` · `api:GET /api/v1/admin/access/tree` · `api:GET /api/v1/admin/access/matrix` · `api:GET /api/v1/admin/access/audit` · `api:POST /api/v1/admin/access/export`

```
matrix = await client.admin.get_access_matrix()
export = await client.admin.export_access(format="csv")
```

The access matrix is the artefact an access review actually needs. It answers "who can reach what" in one call, where reconstructing the same picture from roles, grants, clearances and envelopes takes a working day and gets it wrong. Export it on whatever cadence your review requires.

Presentations

Generating a presentable view of platform state — for a report, a review, or a stakeholder.

capability	what it does
Create	Generate a presentation
Get	Retrieve one

Entry point: `sdk:aegis_sdk.modules.presentation.PresentationModule` .

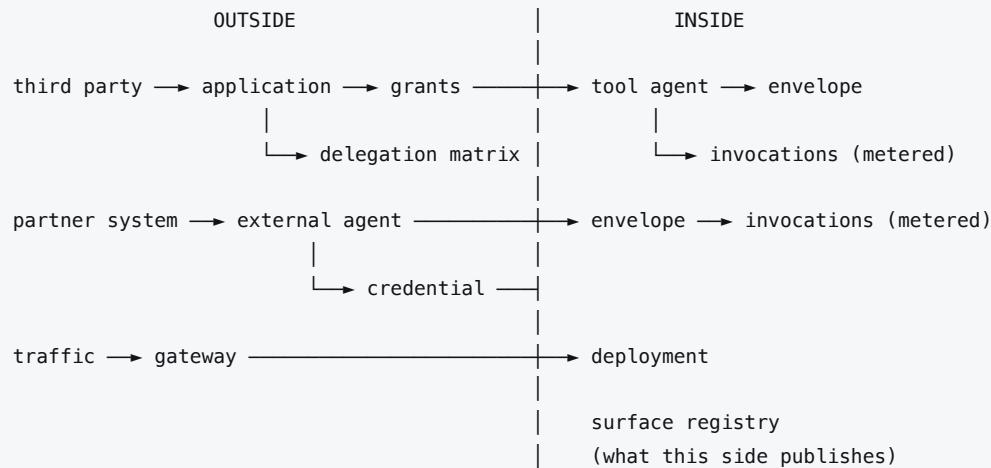
Operations: `api:POST /api/v1/presentations` · `api:GET /api/v1/presentations/{id}`

```

deck = await client.presentation.create(
    subject="quarterly-governance-review",
    period={"start": "2026-07-01", "end": "2026-09-30"},
)
rendered = await client.presentation.get(presentation_id=deck["id"])

```

Where the boundary is drawn



Read it as: everything crossing the line left-to-right passes through a governed object — a grant, an envelope, a credential reference — and everything that crosses is metered on the way. **There is no path from the left column to the right that skips the middle**, which is what makes an external integration something the organisation can evidence rather than something it has to trust.

Summary — what this chapter covered

area	entry point	scale
Billing	revenue.billing	22 operations
Subscriptions	revenue.subscriptions · revenue.plans · revenue.invoices	9 operations
Licences	revenue.licenses	6 operations
Features and quotas	revenue.features · revenue.quotas · revenue.usage	3 operations
Applications	modules.applications	21 operations
Tool agents	modules.tool_agents	13 operations
External agents	modules.integrations	7 operations
Gateways, deployments	modules.integrations	8 operations
Credentials	modules.credentials	9 operations

area	entry point	scale
Surface registry	<code>modules.surfaces</code>	6 operations
LLM providers	<code>modules.llm_providers</code>	10 operations
Admin	<code>modules.admin</code>	17 operations
Presentations	<code>modules.presentation</code>	2 operations

The catalogue, closed

You have now seen the whole of the client-driven surface — all 933 operations this SDK declares, across eighteen capability areas: identity and organisation in 08.2, the actors in 08.3, the work in 08.4, the controls in 08.5, what it knows in 08.6, what it recorded in 08.7, and what surrounds it here.

The platform itself serves more than this — roughly 1,254 authored operations against the client's 933 — and the remainder are reached over HTTP rather than through an SDK method, by the path 04.1 sets out. That is a difference in surface size, not a missing capability, and 08.1 explains which denominator answers which question.

If you arrived with a requirements list, 08.5 is the chapter to re-read — most requirements that sound like they are about capability turn out to be about control, and that is the chapter where Aegis differs most from a platform that added governance afterwards. Five of the eighteen capability areas land there.

If you arrived to build, go back to 02.2 and start there. The catalogue tells you what exists; part 02 is the order to do it in.

Next: Part 09 — The governance architecture

Part 09 — The governance architecture

Audience: the architect who has to satisfy an auditor. Parts 02 to 05 tell you how to operate the platform. Part 06 tells you what the platform is. This part tells you what the platform *governs*, end to end — the complete control surface, in one argument, in the order the controls actually compose.

It is written for the conversation you will eventually have with someone who does not care how the system is built and cares enormously whether it can be shown to have behaved. That person will ask four things: what bounded this agent, who decided it could act, what was recorded, and how do I know the record is intact. This part answers all four, and names the object that answers each.

One idea governs the whole part, and it is worth carrying from the first page: **governance here is not a layer wrapped around a capability — it is the same object seen from the other side.** A role envelope is simultaneously what a delegate *may* do and what it *can* do. A clearance is simultaneously what a role knows and what it may not. A trust chain is simultaneously the authority an agent holds and the ceiling on it. Nothing in this part is a restriction bolted onto something that would otherwise be unbounded, and designs that treat it that way end up with two systems that disagree.

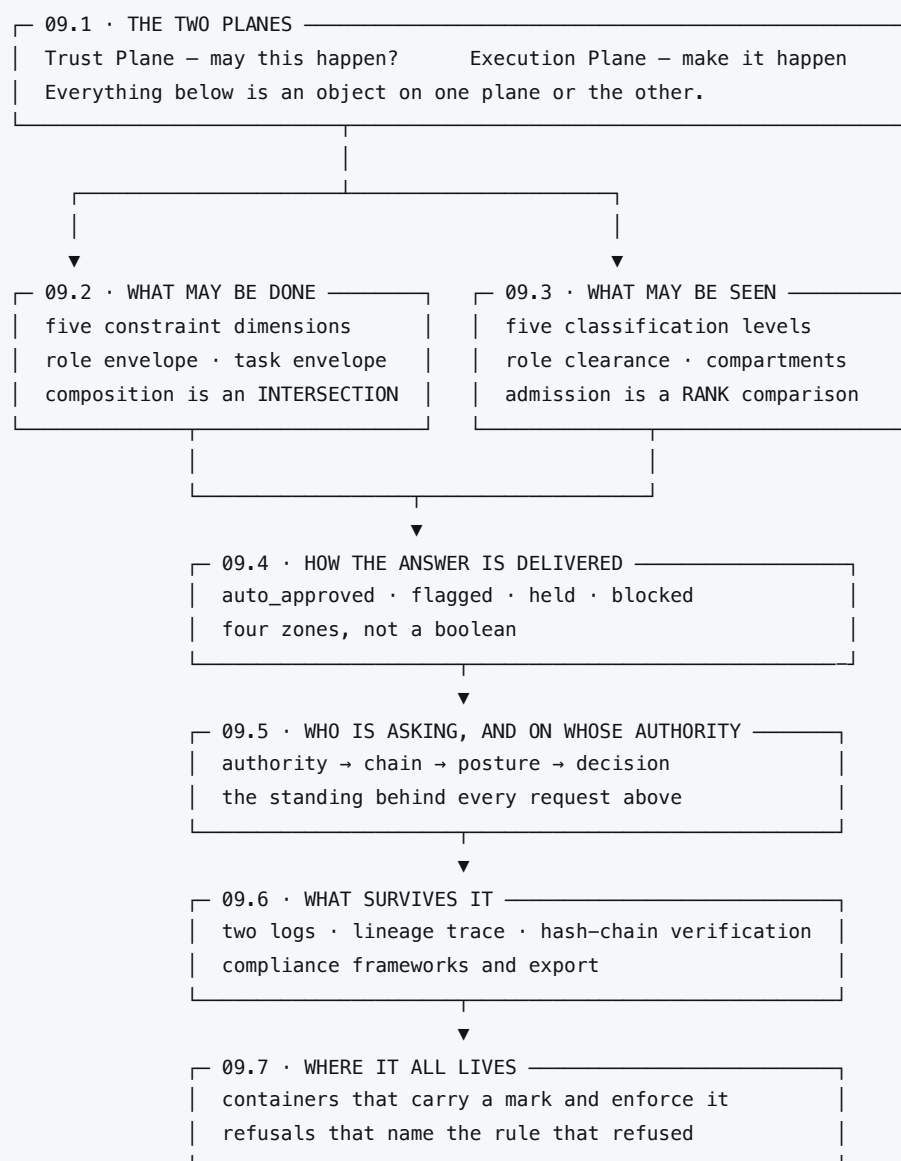
The chapters

#	Chapter	You leave able to
09.1	The two planes	Say which plane a decision belongs to, and why the split is the product rather than a feature
09.2	Envelopes and constraints	Design a bound that composes correctly down a hierarchy, and know when it starts enforcing
09.3	Clearance and classification	Model who sees what, and predict which of three checks refused a read
09.4	The decision gradient	Place an action in the right zone, and design the human step that resolves it
09.5	Trust chains and postures	Establish, move, suspend and withdraw an agent's standing, and evidence each

#	Chapter	You leave able to
09.6	The audit spine and evidence	Produce the artefact an auditor asks for, and show that it has not been altered
09.7	Containment and explainable refusal	Keep classified material inside a container that can enforce it, and read a refusal correctly

How the seven compose

THE GOVERNANCE STACK – read top to bottom; each layer consumes the one above



Read it as: 09.1 establishes the two halves. 09.2 and 09.3 are the two independent bounds — *what may be done* and *what may be seen* — and they are genuinely separate axes that people collapse constantly. 09.4 is how a verdict reaches a human. 09.5 is the standing every request in the three

layers above rests on. 09.6 is what is left afterwards. 09.7 is the container discipline that makes all six survivable when material of different sensitivities sits side by side.

Where this part sits relative to the others

Part 02 is the working reference — how to create an envelope, grant a clearance, establish a chain. Part 06 is the architecture — what the platform is and how a request moves through it. This part is neither. It is the **governance argument**: why these controls and not others, how they compose, which one refuses in a given case, and what you can show afterwards.

The practical test is the question you are asking.

"How do I activate a role envelope?" is 02.3 — a procedure, with the call.

"My delegate has an active envelope and still cannot read the document. Which control refused?" is 09.3, which carries the three-way check and the order to test it in.

"My auditor wants to see that an agent's autonomy was earned rather than assigned. What do I give them?" is 09.5 and 09.6 together — the posture history and evidence surfaces, then the export that makes them portable.

If you read only one chapter, read 09.1. Every other chapter is a statement about one plane or the other, and a control read without knowing which plane it sits on is the single most common way a governance design ends up looking complete and enforcing nothing.

Two things this part keeps returning to

Declared is not enforced. Every governance object in Aegis has a state in which it exists and a state in which it is consulted, and they are different states with different names. An envelope in `draft`. A clearance awaiting vetting. A policy authored but not compiled. Each appears in every listing you will read and changes no decision. This is not a defect in the model — it is what makes review possible — but a provisioning script that creates the objects and never advances them produces an organisation that looks fully governed and is not. The state field is the only thing that distinguishes the two, so assert on it.

The organisation stays accountable; Aegis supplies the proof. Nothing in this part transfers responsibility for what your agents do. What it gives you is the ability to *show* what bounded them, who decided, and what happened — bounded mandates, tamper-evident records, fail-closed permissions, attestable lineage. That is the product: governance you can prove to an auditor, against an external published standard, rather than autonomy you have branded as safe.

Aegis is Integrum's commercial implementation of four open standards published by the Terrene Foundation under CC BY 4.0 — **CARE** (the governance philosophy and its two planes), **PACT** (the organisational architecture and its envelopes), **EATP** (the trust protocol and its classification ladder) and **CO** (the orchestration methodology). The vocabulary in this part is theirs. Where a name looks like a platform invention, it is almost always a term of art from one of the four, and reading the specification will tell you more about the intent than any amount of API inspection.

Next: 09.1 — The two planes

09.1 — The two planes

Every action an agent takes in Aegis passes through two distinct systems, and the entire governance architecture is a consequence of keeping them apart. One decides whether the action is permitted and records that it was decided. The other does the work. CARE calls them the **Trust Plane** and the **Execution Plane**, and the split is not an implementation convenience — it is the claim the product makes.

This chapter is the frame for the six that follow. It is not a tour of the platform's internals; 06.1 draws the client/deployment boundary and 06.2 traces a call through the layers. What this chapter establishes is *which plane a given control belongs to*, and why a control placed on the wrong one produces a system that passes every review and governs nothing.

The single idea to carry out: **the harness is what an agent can do and what it may do, inseparably**. Not two systems, one of which constrains the other. One object, seen from two sides.

Why the split exists at all

The intuitive design for AI governance is a permission layer in front of a model: the model is the capability, the layer is the restriction, and you tune the layer until the risk is acceptable. That design has a property nobody wants — the capability is defined independently of the restriction, so every new capability arrives unbounded by default and someone has to remember to bound it.

Aegis inverts it. An agent's capability is its envelope, its clearance, its chain and its posture. There is no underlying unbounded agent that these constrain. A role with no envelope does not have unlimited authority; it has undefined authority, which the composition treats as a refusal (09.2). A role with no clearance does not see everything; it sees `public` and nothing else (09.3). An agent with no trust chain is not an ungoverned agent — it is invisible to trust discovery and refused at the decision (09.5).

That inversion is what makes autonomy *scalable* rather than merely permitted. Control here is not the tax paid on capability; it is the mechanism by which a wider grant becomes safe enough to make. Ungoverned capability does not scale — it stops at the first thing nobody dares let it touch.

🚫 **Never describe an Aegis control as only a guardrail, or only a capability.** Both halves are true of every object in this part, and naming one half describes a different system. An envelope grants a delegate the authority to spend and bounds the amount, in one declaration. A tool binding confers reach and, through the same object, decides the limit. Design documents that split them end up with a capability shipped without its bound — an exposure — or a bound with no capability behind it, which is ceremony.

What each plane owns

	Trust Plane	Execution Plane
question	may this happen, and on whose authority?	make it happen

Read it as: the verdict is reached *before* the work, over an intent — a tool name and its arguments, an action and a resource kind. The record is written *after*. They are two different witnesses to one event, and neither substitutes for the other. A system that can produce one and not the other cannot answer the question an auditor actually asks, which is never "was something enforced" but "show me that it was".

The three planes people expect, and the one that is not there

Architects arriving from a policy-engine background reach for a third plane: a **Policy Plane** holding rules, separate from the Trust Plane that evaluates them. Aegis does not have one, deliberately, and knowing why saves a design cycle.

Policies in Aegis are not free-standing documents consulted at decision time. They are **compiled into the objects the decision already reads**. An organisation-level policy with an `envelope` enforcement tier is intersected into the affected agents' constraint envelopes through the same monotonic-tightening path every other envelope uses; from that moment the constraint engine enforces it at every invocation, with no separate policy lookup. Authoring is `api:POST /api/v1/policies` and `api:POST /api/v1/policies/{id}/assign`; the compiled result is read back where every other bound is read, at `api:GET /api/v1/constraints/agents/{id}/effective`.

```
policy = await client.governance.create_policy(
    name="Treasury spend ceiling",
    node_address="D1-R1-D1-R1",
    enforcement_tier="envelope",
    policy_body={
        "max_cost_usd": "500.00",
        "action_constraints": {"prohibited": ["wire_transfer_external"]},
    },
)
await client.governance.list_policies()
```

Three enforcement tiers exist, and the difference between them is the difference between a rule and a control:

tier	what it does	binding?
<code>soft</code>	advisory rule text injected into the agent's instructions	no — honestly labelled non-binding
<code>envelope</code>	intersected into the constraint envelope; the engine enforces it at every invocation	yes, hard
<code>gate</code>	targets the four-zone verification gradient, recorded onto the agent's posture configuration	yes, at the gradient (09.4)

🚫 **A financial or action-limit policy authored at `soft` is not a control.** The tier is named honestly and the platform refuses to pretend otherwise: a spend cap enforced by a sentence in a prompt is a request, not a ceiling. The symptom when this goes wrong is the worst possible one — the policy appears in `api:GET /api/v1/policies`, reads correctly to every reviewer, and the agent exceeds it without anything erroring, because nothing was ever asked to stop it. Author limits at `envelope`.

The absence of a Policy Plane is what makes the guarantee checkable. If policies lived in their own plane, "is this policy in force?" would be a question about two systems agreeing. Because they compile into the envelope, it is a question about one object, and `api:GET /api/v1/governance/envelope-coverage` can answer it mechanically.

The five harness classes

CARE's dual plane is a philosophical claim. What implements it in a running deployment is a **harness** — and the harness has parts, each of which grants something and bounds something in the same breath.

class	what it grants	what it bounds
intent	the role's purpose, its job description, what it is for	everything that is not that purpose
skills	a competence the role can exercise	the scope that competence may be exercised in
rules	the invariants the deployment holds	the shapes an agent may not produce
tools	reach into a system outside the platform	whether the binding is enabled, and under what envelope
knowledge	what an agent knows	what it may not know, by classification

Read the table as five instances of one pattern, not five different mechanisms. In each row the two columns are the same declaration read in opposite directions. A tool binding is the clearest case: the object that gives an agent reach into an external system is the same object whose enabled flag decides the limit, and there is no second object to consult. Part 03 covers authoring them; what matters here is the shape.

⚠️ **An empty harness class is never neutral.** A role with no skills is not a role with general competence — it is a role whose competence is undeclared. A deployment with no rules is not a permissive deployment; it is one where nothing expresses what a wrong answer looks like. When you audit a class and find it empty, the finding is *which* — capability left unbounded, or control left ungranted — and the two have opposite remedies.

Scaling autonomy means thickening the harness, never loosening it. When the answer to "this agent needs to do more" is to remove a bound, the result is an agent that does more of what nobody is watching. The answer that works is more harness: a wider envelope with a tighter gradient, a higher

posture with richer evidence behind it, a broader clearance with compartments that scope it. 09.5 is the whole of that argument, made on the posture axis.

Which plane refused you

This is the practical payoff of the split, and it is worth one concrete procedure because the two refusals arrive with overlapping status codes and have completely different remedies.

```
# Establish who you are holding, as distinct from who you assumed
curl -H "Authorization: Bearer $TOKEN" "$AEGIS/api/v1/auth/me" # the principal
```

`api:GET /api/v1/auth/me` reports the principal currently in force. Read the refusal against it:

symptom	plane	what it means	remedy
401 / 403 naming a scope	neither — the door	the credential does not carry the scope the route requires	mint a key with the scope (04.3)
403 naming a constraint	Trust Plane	the bound refused this action for this agent	widen the envelope deliberately, or route through a human decision
403 naming clearance or presenting as an empty result	Trust Plane	the reader may not see the material	grant and approve the clearance (09.3)
409 naming a state	Trust Plane	the object exists but is not in force — <code>draft</code> , <code>pending</code>	activate or approve it
500 / timeout	Execution Plane	the work failed; nothing about authorisation is implied	retry, or read the execution record

The distinction that costs the most when it is missed is the second row against the first. A scope refusal is a statement about *your credential* and no amount of governance change will fix it. A constraint refusal is a statement about *the agent's bounds* and no amount of credential change will fix it. Both read `403 Forbidden` from outside, and the only reliable discriminator is the body: a constraint refusal names the constraint.

The client raises distinct exception types along the same line, which is the cheapest discriminator available to code rather than to a human: `sdk:aegis_sdk.AuthorizationError` for the door, `sdk:aegis_sdk.GovernanceViolationError` when a governance bound refused, and `sdk:aegis_sdk.TrustViolationError` when the refusal came from the trust decision itself. Catching them separately is worth the three lines — a handler that catches the base error and retries will retry a governance refusal forever, because nothing about the passage of time changes an envelope.

`api:POST /api/v1/governance/explain-access` is the tool for the second and third rows. It is a **dry run**: it evaluates a role against a knowledge item you describe and returns the decision as a trace rather than a verdict — whether access would be allowed, the reason, the `step_reached`, and the path

it took. The step name is the useful field, because it tells you *where* the evaluation stopped rather than merely that it did.

```
trace = await client.governance_explain.explain_access(
    agent_id=agent_id,
    knowledge_item_id=document_id,
)
```

Nothing is accessed and nothing is recorded by that call, so it answers *what would the model decide* rather than *why was my request refused*. The second question belongs to the audit spine (09.6).

In the console: the two planes are separate areas by design. Governance, trust and approvals live under the governance section; objectives, sessions and runs live under work. When an operator says "the agent did nothing", the first question is which of those two areas shows the event — a held action appears in the approvals queue and never in the run history, which reads as silence from the work side.

Where each plane surfaces in the client

The client you hold is partitioned along the same line, and knowing which namespace a module lives in is a reliable shortcut for knowing which plane a question belongs to.

namespace	plane	what it carries
<code>sdk:aegis_sdk.trust</code>	Trust	authorities, chains, postures, delegations, revocation, the trust audit
<code>sdk:aegis_sdk.modules.GovernanceModule</code>	Trust	policies, permissions, classifications, consent, lineage
<code>sdk:aegis_sdk.modules.GovernanceExplainModule</code>	Trust	the explain surfaces — access, envelope, address
<code>sdk:aegis_sdk.standup.RoleEnvelopesModule</code>	Trust	envelope authoring and activation
<code>sdk:aegis_sdk.standup.ApprovalsModule</code>	Trust	the human decision surface
<code>sdk:aegis_sdk.modules.ComplianceModule</code>	Trust	frameworks, reports, retention, chain integrity
<code>sdk:aegis_sdk.execution</code>	Execution	objectives, requests, sessions, artifacts
<code>sdk:aegis_sdk.WorkObjectivesModule</code>	Execution	work as it is planned and carried
<code>sdk:aegis_sdk.modules.ObserveAuditModule</code>	the spine	the platform record of what changed

Two entries repay a second look. `sdk:aegis_sdk.modules.ObserveAuditModule` is deliberately placed in neither plane — the audit spine is written *by* both and owned by neither, which is 09.6's subject. And `sdk:aegis_sdk.modules.ComplianceModule` sits on the Trust Plane rather than alongside it, because a compliance framework here is a set of controls over the governance objects rather than a reporting layer draped over them.

You hold one client — `sdk:aegis_sdk.Agentic0SClient`, configured through `sdk:aegis_sdk.ClientConfig` against a base URL you supply — and both planes are reached through it. The configuration surface is entirely about the connection: which deployment, with which credential, under which timeout and retry policy. None of it is about the organisation's content, which is asked for per call.

The harness classes, worked

The five classes are easiest to see on one role. Take a treasury analyst whose job is to reconcile settlement positions and draft, but never send, outbound wires.

class	the grant	the bound
intent	"reconcile settlement positions and draft outbound wires for review"	everything outside reconciliation and drafting
skills	position reconciliation, wire drafting	no settlement execution competence is declared, so none is exercisable
rules	the invariants — a wire always names a counterparty, a reconciliation always balances	shapes the agent may not produce
tools	a binding to the positions system, read-only	the binding's envelope bounds which accounts, and its enabled flag decides whether it exists at all
knowledge	the treasury desk's procedures at <code>confidential</code>	nothing at <code>secret</code> , because the role's clearance stops there

Read down the second column and you have the role's capability. Read down the third and you have its governance. **They are the same five declarations.** There is no sixth object holding the restrictions, which is why there is no way to remove the restrictions and keep the role.

The operational consequence is worth stating because it changes how you provision: **when a role needs to do something new, the change is always additive and always in both columns.** Add the skill and its scope. Add the tool binding and its envelope dimension. Add the knowledge and the clearance that reaches it. A change that touches only one column is the one to catch in review — a skill with no scope, or a scope with no skill.

Why governance is the product

It is tempting to read this part as the safety documentation for a capability product. It is the other way round, and the distinction has practical consequences for what you build.

A competitor bolting a permission layer onto a model has built a different thing, not a cheaper version of this one. Their capability exists independently of their controls, so their controls can be removed and the capability survives. Here they cannot: remove an agent's envelope and the composition refuses; remove its chain and the decision refuses; remove its clearance and it sees `public`. The system degrades toward *doing nothing*, not toward *doing anything*.

That is the property the architecture exists to deliver, and it is the one to carry into your own design. **When you extend the platform, extend both halves in the same change.** A new tool with no envelope dimension covering it is an exposure. A new gate on a surface nothing can invoke is ceremony. The chapters that follow are, between them, the vocabulary for keeping the two together.

Summary

you want to know	plane	the call
which principal am I holding	the door	<code>api:GET /api/v1/auth/me</code>
which organisations can I act in	the door	<code>api:GET /api/v1/auth/me/organizations</code>
switch the acting organisation	the door	<code>api:POST /api/v1/auth/me/switch-org</code>
may this agent take this action	Trust	<code>api:POST /api/v1/trust/verify</code>
why a read would be refused	Trust	<code>api:POST /api/v1/governance/explain-access</code>
what bounds this agent right now	Trust	<code>api:GET /api/v1/constraints/agents/{id}/effective</code>
is the governance layer enforcing	Trust	<code>api:GET /api/v1/governance/envelope-coverage</code>
author a rule that binds	Trust	<code>api:POST /api/v1/policies at envelope tier</code>
which agents exist	Execution	<code>api:GET /api/v1/agents</code>
what changed, and who changed it	the spine	<code>api:GET /api/v1/audit/logs</code>

Next: 09.2 — Envelopes and constraints

09.2 — Envelopes and constraints

An envelope is the answer to *what may this delegate do*. It is the Trust Plane's bound on action, it is authored by a supervising role rather than by the delegate, and it composes down a hierarchy by intersection — which is the whole of its safety property and the source of every surprise it produces.

02.3 shows you how to create and activate one. This chapter is about the algebra: the five dimensions the bound is expressed in, the three layers an effective bound is composed from, why the composition can only ever tighten, and what it means when a link in the chain is missing. Clearance — *what may this delegate see* — is a separate axis with a separate algebra and is 09.3.

The idea to carry out: **an envelope is not a list of permissions, it is a ceiling, and every ceiling beneath it is lower**. Designs that treat envelopes as grants — "add an envelope to let this agent do more" — are expressing something the model cannot represent, and the failure is silent.

The five constraint dimensions

CARE defines five, and they are the complete vocabulary. Every bound on action in Aegis is expressed as one of them; there is no sixth, and a limit that does not fit one of the five is a limit the model is not carrying.

dimension	canonical name	bounds	typical shape
Financial	<code>financial</code>	spend — amounts, currency, budget	<code>{"max_amount": 100000, "currency": "USD"}</code>
Operational	<code>operational</code>	which actions are permitted or blocked	<code>{"blocked_actions": ["wire_transfer_external"]}</code>
Temporal	<code>temporal</code>	when, and for how long	window, duration, expiry
Data Access	<code>data_access</code>	which data may be reached	prohibited fields, scopes
Communication	<code>communication</code>	who or what may be contacted	internal-only, recipient allowlists

The names above are the canonical CARE spellings, lowercase, and they are what you write in `constraint_config`. Three of the five are worth a sentence of design guidance each, because they are the ones people under-use:

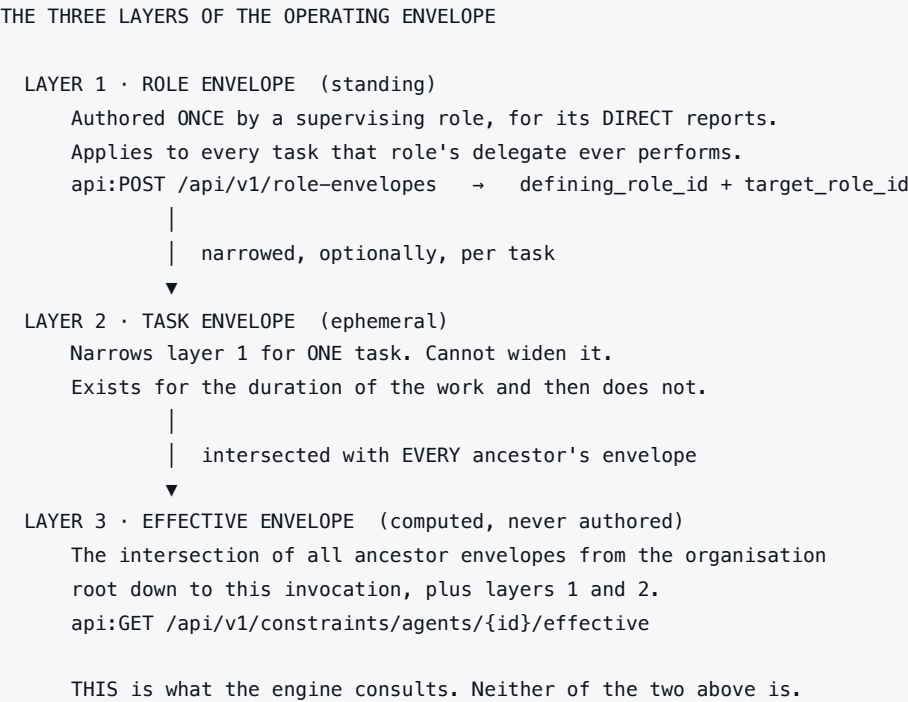
- **Temporal** is the dimension that makes an emergency widening safe. A bound with no temporal component cannot expire, so every widening becomes permanent by default and somebody has to remember to undo it. 09.4 § "The emergency path" is built on this.
- **Data Access** is not clearance. Clearance answers *is this reader permitted to see material at this classification*; Data Access answers *may this delegate reach this data at all, whatever its classification*. A delegate can hold `secret` clearance and still be refused a table its envelope excludes.

- **Communication** is the dimension most often left empty, and an empty Communication constraint on an agent with outbound tool bindings is the clearest case of a capability shipped without its bound. If the agent can send email, the envelope should say to whom.

⚠ **Aegis persists several historical spellings of these names and translates them at their boundaries.** An organisation-level policy may carry `data` where a role envelope carries `data_access` ; a wire shape may carry `transaction` , which CARE treats as a sub-type of Financial. They name the same dimensions and the platform relates them. What matters for your own authoring is that you write the canonical five and read the composed result from `api:GET /api/v1/constraints/agents/{id}/effective` , which resolves them for you rather than leaving you to match spellings.

Three layers, one effective bound

A delegate's actual bound is never a single object. It is the composition of three layers, and knowing which layer you are looking at is the difference between debugging the right thing and debugging a symptom.



Read it as: you author layers 1 and 2; the platform computes layer 3; the decision consults only layer 3. The consequence people trip on is that an envelope can be perfectly correct at layer 1 and produce a bound at layer 3 that looks nothing like it, because an ancestor three levels up is tighter.

Three read surfaces separate the layers, and using the right one is most of the debugging:

you want to know	read	what it answers
what I authored	api:GET /api/v1/role-envelopes/{id}	one envelope, as written

you want to know	read	what it answers
what came down from above	<code>api:GET /api/v1/constraints/agents/{id}/inherited</code>	the ancestors' contribution
what actually binds	<code>api:GET /api/v1/constraints/agents/{id}/effective</code>	the composition — trust this one

```

effective = await client.governance.get_agent_constraints(
    agent_id=agent_id,
    resolution="effective",
)
inherited = await client.governance.get_agent_constraints(
    agent_id=agent_id,
    resolution="inherited",
)

```

The failure mode to design against: a supervisor tightens their own envelope to respond to an incident, and every delegate beneath them tightens immediately and without any of their envelopes being touched. That is the model working exactly as intended, and it is a genuinely useful lever. It is also a surprise if you did not expect it, and the symptom is a fleet of agents that begin refusing actions they performed yesterday, with no change to any of their own objects. Before concluding that a delegate's envelope is broken, read its inherited layer.

Monotonic tightening — the property everything rests on

The composition rule has one sentence: **a child envelope must be tighter than or equal to its parent, on every dimension, and the platform refuses a write that is not.** Composition is an intersection, never a union.

```

proposed = await client.governance.validate_agent_constraints(
    agent_id=agent_id,
    constraints={"financial": {"max_amount": 250000, "currency": "USD"}},
)

```

`api:POST /api/v1/constraints/agents/{id}/validate` checks a proposed change against the composition before you make it — which is the call to reach for whenever a bound change is going to be applied programmatically, because the refusal it gives you is cheap and the refusal the write gives you arrives after a partially-provisioned organisation exists.

Two consequences shape every design built on this:

You cannot build an escalation path by adding an envelope. Widening is not expressible in the model. If a delegate genuinely needs to exceed its bounds, the answer is a human decision (09.4) or a time-bounded emergency widening authored by someone whose own envelope is wide enough to contain it

— never a second envelope. Attempts to express escalation as an envelope produce one of two symptoms: the write is refused, which is the good case, or the envelope is written at a level nobody consults, which is the bad one.

A bound is only as wide as the narrowest ancestor. This is obvious stated directly and consistently surprising in practice, because organisational hierarchies are deep and people reason about the two roles either side of a delegation rather than the whole chain. `api:POST /api/v1/governance/explain-envelope` exists for exactly this: it returns the composition as a trace, so you can see which ancestor supplied each binding term.

 **A missing ancestor envelope is a DENY, not a skip.** When the composition walks the reporting chain and finds a role with no envelope, it does not continue to the next ancestor — it fails closed. A gap in the chain means governance is undefined at that level, and undefined is not permissive here. The symptom is an agent that refuses everything with a constraint reason, several levels below the role whose envelope was never authored, and the remedy is to author the missing one rather than to widen the delegate.

`api:GET /api/v1/governance/probe-corrupted-roles` is the sweep for this class — roles whose envelope state cannot be resolved — and it is worth running after any bulk organisational change.

draft does not enforce, and that is the most expensive thing in this chapter

An envelope is created in one of two states, and `create` defaults to `draft`.

```
envelope = await client.role_envelopes.create(
    defining_role_id=head_role_id,
    target_role_id=analyst_role_id,
    constraint_config={
        "financial": {"max_amount": 100_000, "currency": "USD"},
        "operational": {"blocked_actions": ["wire_transfer_external"]},
        "communication": {"internal_only": True},
    },
    status="draft",
)
await client.org_standup.activate_role_envelope(envelope["id"])
```

`api:POST /api/v1/role-envelopes` creates it; `api:POST /api/v1/role-envelopes/{id}/activate` puts it in force; `api:POST /api/v1/role-envelopes/{id}/suspend` takes it back out. A `draft` envelope is a description of an intention. It appears in `api:GET /api/v1/role-envelopes`, it reads correctly to every human who looks at it, and it changes no decision.

This is the failure mode to design against, stated plainly: a provisioning script that creates envelopes and never activates them produces an organisation that looks fully bounded in every listing and enforces nothing. Nothing errors. There is no warning. The only thing that distinguishes the two states is the `status` field, so assert on it as part of provisioning rather than as part of review:

```

envelopes = await client.org_standup.list_role_envelopes()
for e in envelopes.records:
    assert e.status == "active", f"envelope {e.id} is {e.status}, not enforcing"

```

Pass `status="active"` at creation when the envelope is fully specified. Use `draft` deliberately, when a human is going to review it before it binds — not by accident, and never as the default in automation.

Two further fields on the envelope are worth knowing before you author one. `clearance_ceiling` caps how far the target role's clearance can carry *inside this delegation*, which is the one place the two axes of this part meet. `verification_defaults` sets how thoroughly the delegate's actions are checked, which is the envelope's handle on the gradient in 09.4.

Defaults, ceilings and the organisation root

Not every bound is authored per-role. Three surfaces set bounds that apply before any role envelope is consulted.

surface	scope	read	write
organisation constraint defaults	every agent in the tenant	<code>api:GET /api/v1/constraints/organizations/{id}/defaults</code>	<code>api:PUT /api/v1/constraints/organizations/{id}/defaults</code>
envelope defaults	the template a new envelope starts from	<code>api:GET /api/v1/envelopes/defaults</code>	—
unit posture ceiling	every agent under a unit	<code>api:GET /api/v1/organization-units/{id}/posture-ceiling</code>	<code>api:PUT /api/v1/organization-units/{id}/posture-ceiling</code>

The posture ceiling is the one that behaves differently from the others and the one worth understanding here even though postures are 09.5's subject. It cascades the same way envelopes do — a child unit cannot set a higher ceiling than its parent — and it caps how autonomous *any* agent beneath that unit may become, regardless of what that agent has individually earned. It is the organisational brake, and it is the correct lever when the answer to "how much autonomy should this department have" is different from "how much has this particular agent earned".

Per-agent bounds are read back at `api:GET /api/v1/trust/agents/{id}/constraints` and written at `api:PUT /api/v1/constraints/agents/{id}`. For a tool agent specifically, `api:GET /api/v1/tool-agents/{id}/envelope-summary` is the compact projection — which dimensions are bounded, and how tightly — and is the one to put on a dashboard.

Tightening in an incident

Envelopes are also the incident control, and this is where the temporal dimension earns its place.

`api:POST /api/v1/interventions/{id}/tighten-constraints` narrows a running session's bounds without stopping it, which is frequently what you want: the agent keeps working on the part of the task

that is still in scope, and the part that is not begins refusing.

```
await client.interventions.tighten_constraints(
    intervention_id=session_id,
    constraints={"financial": {"max_amount": 0, "currency": "USD"}},
)
```

The sibling controls are `api:POST /api/v1/interventions/{id}/pause`, `api:POST /api/v1/interventions/{id}/resume` and `api:POST /api/v1/interventions/{id}/fence`, with the history at `api:GET /api/v1/interventions/{id}/history`. Choose between them by blast radius: tightening is surgical and leaves the work running, pausing stops one session, and the kill switch (09.4) stops a class of work across the tenant.

△ **Tightening an ancestor is not the same instrument as tightening a session.** The first is permanent until undone and propagates to every descendant; the second is scoped to one run. In an incident the temptation is to reach for the ancestor because it is broader, and the cost lands afterwards — the incident ends, the session ends with it, and the ancestor's tightening stays in force silently bounding work nobody connected to the incident. If you tighten an ancestor, author the restoration in the same change.

A composition, worked

Four roles on one reporting chain, each with an active envelope, and one delegate at the bottom. This is the whole algebra in one example, and the answer is not the one most people predict.

COMPOSING FOUR ENVELOPES DOWN ONE CHAIN

role		financial.max_amount	operational.blocked_actions
CEO	(D1-R1)	10,000,000	[]
CFO	(...-D1-R1)	2,000,000	["equity_issuance"]
Treasury lead	(...-T1-R1)	500,000	["wire_transfer_external"]
Analyst	(...-T1-R2)	750,000	[]
EFFECTIVE for the analyst		500,000	["equity_issuance", "wire_transfer_external"]

FINANCIAL is an intersection of RANGES → the LOWEST ceiling wins.
The analyst's own 750,000 is IGNORED: it is wider than the treasury lead's 500,000, and a delegate cannot exceed its supervisor. The write that authored it was accepted because 750,000 is below the CFO's 2,000,000 — the refusal only fires when a value exceeds the IMMEDIATE parent.

OPERATIONAL is an intersection of PERMITTED SETS → blocks ACCUMULATE.
Every ancestor's block survives all the way down. The analyst cannot issue equity, and nobody in the chain below the CFO can make that possible again.

Read it as: **the two dimensions compose in the same direction and it does not look the same**. A financial ceiling gets lower; a block list gets longer. Both are the intersection. The confusion is that one reads as a number shrinking and the other as a list growing, and an operator who has internalised "tightening means the number goes down" will look at a lengthening block list and read it as a widening.

Two things fall out of the worked example directly.

A locally-valid envelope can be entirely inert. The analyst's 750,000 was a legal write and contributes nothing to any decision. It will appear correct in `api:GET /api/v1/role-envelopes/{id}` forever. The only surface that reveals it is the effective composition, which is why that read is the one to trust when debugging.

Removing a bound in the middle of a chain does not restore anything below it. Lifting the treasury lead's 500,000 does not give the analyst 750,000 — it gives them 2,000,000, the CFO's ceiling, because the analyst's own envelope was never the binding term. Widening a chain is not a local operation, and the safe procedure is to read the effective composition before and after rather than to predict it.

Is the layer enforcing anything at all?

Two reads answer the question an auditor will ask about envelopes as a class, rather than about any one of them, and both belong on a schedule.

`api:GET /api/v1/governance/envelope-coverage` answers not *"do envelopes exist"* but *"would they stop anything"*. It is honest in the way that makes it usable: an envelope that resolves to allow-all is counted in its own bucket rather than as coverage, and the report carries a completeness flag. Read the completeness flag first — when the measurement did not fully complete, the counts are a floor rather than a total, and quoting a floor as a total is how a coverage number ends up in a compliance document and stays there.

```
coverage = await client.governance.get_envelope_coverage()
hydration = await client.governance.get_envelope_hydration_status()
```

`api:GET /api/v1/governance/envelope-hydration-status` is the layer beneath it. Hydration is the pass that materialises an envelope into the form the engine consults, and the envelopes a pass **skipped** are surfaced rather than silently dropped. A skipped envelope means the engine fell back to a restrictive bootstrap default — fail-closed, so not a hole, but also not what you authored, and invisible everywhere else.

The failure mode these two exist to catch: an organisation where every role has an envelope, every envelope is `active`, and the coverage report shows most of them resolving to allow-all because the constraint configuration was generated from a template nobody filled in. Every listing is green. Nothing refuses anything. The coverage bucket is the only surface that distinguishes it from a correctly bounded organisation.

Reading an envelope in the console

In the console: envelopes appear under the governance area, on the role they bound rather than on the role that authored them — which is the reverse of how you create them. The `status` chip is the field to look at first; a `draft` envelope renders identically to an active one apart from that chip. The effective composition is shown on the agent rather than on the role, because composition is a property of the invocation and roles do not invoke.

Summary

you need	the object	the call
bound a delegate's actions	role envelope	<code>api:POST /api/v1/role-envelopes</code>
put that bound in force	activation	<code>api:POST /api/v1/role-envelopes/{id}/activate</code>
see what actually binds an agent	effective envelope	<code>api:GET /api/v1/constraints/agents/{id}/effective</code>
see what came from above	inherited constraints	<code>api:GET /api/v1/constraints/agents/{id}/inherited</code>
test a change before making it	validation	<code>api:POST /api/v1/constraints/agents/{id}/validate</code>
understand a composition	envelope trace	<code>api:POST /api/v1/governance/explain-envelope</code>
bound a whole department's autonomy	unit posture ceiling	<code>api:PUT /api/v1/organization-units/{id}/posture-ceiling</code>
narrow a live session	intervention	<code>api:POST /api/v1/interventions/{id}/tighten-constraints</code>
know whether the layer enforces	coverage	<code>api:GET /api/v1/governance/envelope-coverage</code>

Next: 09.3 — Clearance and classification

09.3 — Clearance and classification

Clearance is the answer to *what may this role see*. It is the second of the two independent bounds in the Trust Plane, it runs on a five-level ladder inherited from EATP, and it is the axis most often collapsed into something it is not.

This chapter is the complete read-access model: the ladder and its ranks, how a role's clearance is granted and approved, what compartments add at the top two levels, how an agent's *effective* clearance is computed from its role's clearance and its posture, and the three-way check a read actually passes through. 09.2 covered the action axis; the two compose but never substitute.

The idea to carry out: **authority level does not enter into read access at all**. A C-suite role with no clearance sees less than a staff role with clearance. This is the single most common modelling error on the platform, it survives review because the org chart looks right, and the symptom is a senior person reporting an empty screen.

The ladder

Five levels, lowercase, ordered. They are EATP's, and Aegis uses them unmodified.

level	rank	display	compartment required?
public	0	Public (C0)	no
restricted	1	Restricted (C1)	no
confidential	2	Confidential (C2)	no
secret	3	Secret (C3)	yes
top_secret	4	Top Secret (C4)	yes

Every access decision on this axis is a rank comparison: the reader's effective clearance rank against the item's classification rank, with `>=` admitting. At the top two levels a rank comparison alone is not sufficient and compartments supply the need-to-know dimension — see § Compartments below.

Two properties of the ladder are load-bearing and neither is obvious:

The levels are fixed and validated client-side. Anything outside the five raises before a request is made. That is deliberate: a typo'd level is a bound that silently matches nothing, and catching it at the client costs one round-trip rather than one incident.

Unknown is not public. An item whose classification cannot be resolved — absent, or a value no level maps — is treated as *unknown*, and unknown denies at every egress boundary. This is the opposite of the intuitive default and it is the right one: "we do not know what this is" must never resolve to "the least sensitive thing there is". 09.7 carries the full treatment, because it is where the consequences land.

Granting a clearance, and the state it lands in

```
clearance = await client.trust_posture.create_role_clearance(
    role_id=analyst_role_id,
    max_clearance="confidential",
    justification="Handles cash-position reporting for the Treasury desk.",
)
await client.trust_posture.approve_role_clearance(clearance["id"])
```

`api:POST /api/v1/role-clearances` creates it. Note it lives on `sdk:aegis_sdk.TrustPostureModule`, not on a `clearances` module of its own — clearance and posture are read together often enough that they share a surface.

Four rules bind the write, and each has a distinct failure signature:

1. **The levels are fixed.** Anything outside the five raises client-side, before a request is made.
2. **justification is required by the server at confidential and above.** Omitting it fails server-side, which is a different error from the one above and reads differently — a `422` rather than a `ValueError`.
3. **secret and top_secret require compartments.** A level alone does not grant sight at those two; see below.
4. **A new clearance lands in vetting_status="pending" and is not yet in force.**

 **The fourth rule is the expensive one, and it is the same shape as the draft envelope in 09.2.** Creating a clearance is not granting it. Approval and rejection are separate governance steps — `api:POST /api/v1/role-clearances/{id}/approve` and `api:POST /api/v1/role-clearances/{id}/reject` — and until approval the role reads at `public`. A provisioning script that creates clearances and walks away has staffed an organisation whose roles cannot see their own material, and the symptom is **an empty screen rather than an error**. There is nothing in the UI, nothing in the logs, and nothing in the clearance listing that reads as wrong; the row exists, it names the right level, and `vetting_status` is the only field that says it is not in force.

Read them back with `api:GET /api/v1/role-clearances` and `api:GET /api/v1/role-clearances/{id}`; amend with `api:PUT /api/v1/role-clearances/{id}`; remove with `api:DELETE /api/v1/role-clearances/{id}`. Assert on `vetting_status` in provisioning the same way you assert on envelope `status`:

```
clearances = await client.trust_posture.list_role_clearances()
for c in clearances.records:
    assert c.vetting_status == "active", f"clearance {c.id} is {c.vetting_status}"
```

The separation is the point rather than an inconvenience. Clearance is the one governance object whose grant a machine should not be able to complete unattended, and the approval step is where a human takes responsibility for it. The console is where a reviewer approves a pending clearance; the harness is where you create the fifty that need reviewing.

Compartments — the need-to-know dimension

At `secret` and `top_secret`, a rank comparison is not an access model. Two roles both cleared to `secret` are not thereby cleared to each other's material, and the mechanism that expresses this is the compartment.

```
clearance = await client.trust_posture.create_role_clearance(
    role_id=treasury_lead_role_id,
    max_clearance="secret",
    compartments=["treasury-positions", "counterparty-exposure"],
    justification="Desk lead; requires position and exposure detail.",
)
```

The rule has two halves and both must hold:

- the reader's clearance rank must reach the item's classification rank, **and**
- at `secret` / `top_secret`, the item's compartment must be one the reader is specifically cleared for.

A `secret`-cleared role with no membership in the item's compartment is refused, and so is a `secret` item that carries no compartment at all — the latter fails closed rather than being treated as uncompartmented material. An item at those levels without a compartment is an item whose need-to-know boundary was never declared, and serving it would be enforcing a rank comparison while presenting it as need-to-know isolation.

△ **A container that cannot express compartments cannot hold compartmented material.** Several object kinds in Aegis carry a classification and no compartment — and their writable vocabulary is therefore capped at `{public, restricted, confidential}`. This is not a limitation to work around; it is the model refusing to offer protection it cannot deliver. A control that reads stronger than what runs is worse than a narrower one that is honest. 09.7 § "A container may only hold what it can enforce" is the general statement.

The return value of the clearance create is the raw persisted row, in which `compartments_json` is a JSON *string* rather than a deserialised list. Parse it if you need it; indexing it as a list produces a character.

Effective clearance — clearance capped by posture

A role's clearance is not an agent's clearance. An agent acting in that role has an **effective clearance**, and it is the lower of two things:

```
effective_clearance = min( role.max_clearance , posture_ceiling[agent.posture] )
```

The posture ceiling is fixed by CARE and is not configurable per agent:

posture	maximum effective clearance
pseudo	public
supervised	restricted
shared_planning	confidential
continuous_insight	secret
delegated	the role's full clearance

Read that table as the answer to a question people ask the wrong way round. It is not "what clearance does this posture grant" — a posture grants nothing. It is "how much of the seat's clearance may an agent at this standing actually carry". A newly provisioned agent at `supervised` sitting in a role cleared to `secret` reads at `restricted`, and that is correct: the agent has not yet demonstrated anything, and the seat's clearance is a property of the seat.

There is a third cap. An envelope's `clearance_ceiling` (09.2) narrows how far the target role's clearance carries *inside that particular delegation*, which lets a supervisor delegate a task without delegating the whole of the delegate's sight.

The failure mode this produces: an agent that could read a document yesterday and cannot today, with no change to the document and no change to the role's clearance — because an emergency posture override (09.5) moved the agent down a level and the ceiling moved with it. The remedy is not to raise the clearance. Read the posture first.

Classifying the material

The other half of the comparison is the item's own classification. Knowledge is the principal classified object and carries the full model — classification, compartment, containment path, and a review lifecycle.

```
policy = await client.knowledge.create(
    title="Treasury Approval Policy",
    content_markdown="# Approval Policy\nAll wires above $50k require sign-off.",
    knowledge_type="policy",
    path="/treasury/policies/approvals",
    classification="confidential",
    owner_unit_id=unit_id,
)
await client.knowledge.publish(policy["id"])
```

`api:POST /api/v1/knowledge` then `api:POST /api/v1/knowledge/{id}/publish`. `knowledge_type` is one of `policy`, `procedure`, `reference`, `faq`; `path` is a containment path and is how material is organised and addressed; `classification` uses the same five levels, with the same compartment requirement at the top two.

Creating knowledge does not publish it — a third instance of the shape this part keeps returning to. For anything going through review the lifecycle has more stages than create-and-publish, and each is its own call: `api:POST /api/v1/knowledge/{id}/submit`, `api:POST`

`/api/v1/knowledge/{id}/assign-reviewer` , `api:POST /api/v1/knowledge/{id}/approve` , `api:POST /api/v1/knowledge/{id}/request-changes` , `api:POST /api/v1/knowledge/{id}/reject` , and `api:POST /api/v1/knowledge/{id}/archive` with `api:POST /api/v1/knowledge/{id}/unarchive` as its inverse. Versioning is `api:POST /api/v1/knowledge/{id}/version` and `api:POST /api/v1/knowledge/{id}/create-revision` . The review queue itself is `api:GET /api/v1/knowledge-reviews` .

Beyond knowledge, `sdk:aegis_sdk.modules.GovernanceModule` carries the data classification registry — `api:GET /api/v1/data-governance/classifications` , `api:POST /api/v1/data-governance/classifications` , `api:PUT /api/v1/data-governance/classifications/{id}` — which is where field- and dataset-level classifications live for material that is not a knowledge item. The two share the ladder.

Units are boundaries, and sharing is explicit

Classification is one of two things that gate a read. The other is containment: material belongs to a unit, and units are knowledge boundaries.

Material does not flow between units because the two are adjacent on the org chart. It flows because somebody wrote a share policy: `api:POST /api/v1/knowledge-share-policies` , listed at `api:GET /api/v1/knowledge-share-policies` , suspended and reactivated at `api:POST /api/v1/knowledge-share-policies/{id}/suspend` and `api:POST /api/v1/knowledge-share-policies/{id}/reactivate` .



Sharing is not transitive. If A shares with B and B shares with C, C does not see A's material. Every hop is its own policy and its own decision. A design that assumes transitivity produces an access model more permissive on paper than in practice — the safe direction, but it means a capability you promised a stakeholder does not exist, and the symptom is a user who can see the material through one route and not through the route they were told to use.

`api:GET /api/v1/knowledge-share-policies/due-for-review` belongs on a schedule. Share policies are exactly the object that gets created for a project and outlives it, and a review cadence is the only thing that catches one.

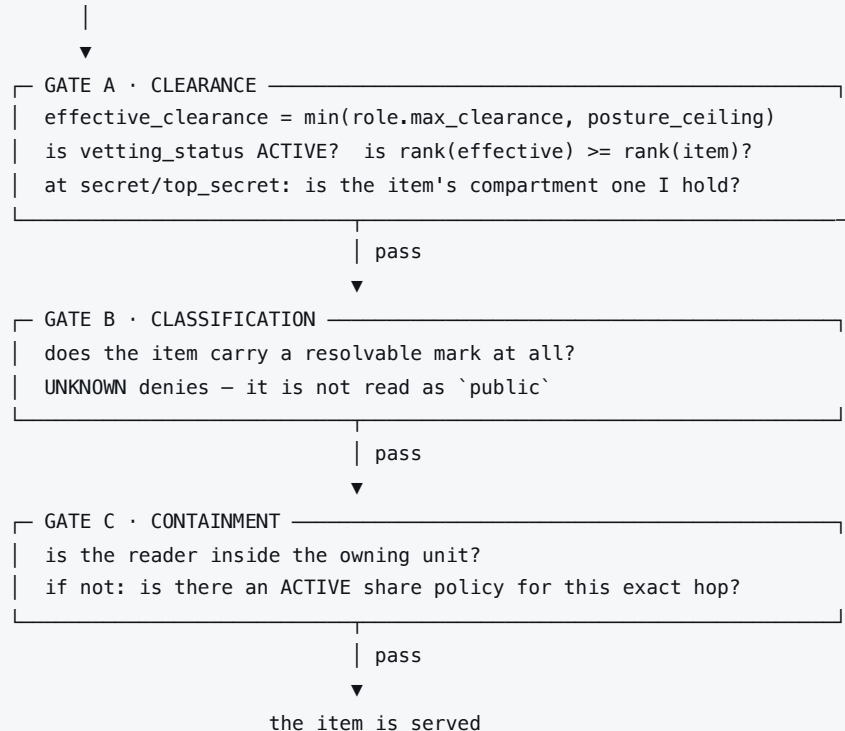
For the broader containment question — which units are isolated from which at the tenancy level — `api:GET /api/v1/organization-units/isolation-domains` reads the current partition and `api:PUT /api/v1/organization-units/isolation-domains` sets it.

The three-way check

When a read is evaluated, three things must line up.

A READ, EVALUATED – three gates, one message

reader asks for item X



A FAILURE IN ANY ONE OF THE THREE PRESENTS IDENTICALLY:

"you cannot see this" – and the three are not distinguished in the message.

Read it as: three independent conditions, one indistinguishable symptom. That is deliberate — a refusal that told you *which* gate refused would let a reader map the classification of material they cannot see, and existence is itself information at the top two levels.

It does mean you need an order to debug in, and the order is not the order the gates run in. **Check clearance first**, because `vetting_status="pending"` is the most common cause and the cheapest to confirm. Then classification, because an unmarked item is the second most common and is invisible from the reader's side. Containment last, because a missing share policy is the one a human usually remembers not creating.

`api:POST /api/v1/governance/explain-access` short-circuits the whole procedure when you hold the right credential: it evaluates a **role against a knowledge item** and returns the decision as a trace — allowed or not, the reason, the `step_reached` (the phase name the evaluation stopped at), and the access path taken.

```
trace = await client.governance_explain.explain_access(
    agent_id=agent_id,
    knowledge_item_id=document_id,
)
```

🚫 **Read `step_reached`, not `allowed`, when you are diagnosing.** `allowed` tells you the outcome you already observed; `step_reached` tells you *which of the three gates above the evaluation stopped at*, which is the whole reason to call it. A trace that stops at the clearance phase and one that stops at

containment produce the same `allowed: false` and send you to different remedies.

⚠ **It is a DRY RUN, and that is the property to design around.** The call evaluates the item you describe: nothing is read from any stored refusal record, nothing is accessed, and nothing is recorded. A wrong or missing field in the subject you pass changes the verdict — so a trace is only as good as the description it was given, and a clean result is evidence about the model's decision on that description, not about any access that actually happened. To find out what happened, read the audit spine (09.6); this surface answers *what would the model decide*, and the two questions are not interchangeable.

Where classification meets the rest of the model

Three cross-references, each of which is a place a design goes wrong by treating this axis in isolation.

Classification is a property of the field, not of the verb. A write operation that returns a row is a read surface. Every mutation return path in the platform applies the same redaction the read path applies, which is why an agent that creates a record containing a field above its clearance gets that field back redacted rather than echoed. Design your own integrations the same way: the caller having just supplied a value is not a reason to return it to them.

Retrieval filters before it ranks. When classified material backs a retrieval surface, the clearance filter is applied before retrieval rather than after. Post-retrieval filtering leaks through ranking scores and result counts — a `restricted`-cleared agent could infer the existence of `secret` documents by watching how many results come back. The filter belongs in the query.

Lineage carries classification forward. Derived material inherits the high-water mark of everything it was derived from, so a summary of a `secret` input is `secret` even if the summary itself reads innocuously. `api:GET /api/v1/data-governance/lineage/graph` and `api:GET /api/v1/data-governance/lineage/nodes/{id}/upstream` are the read surfaces for the general case; 09.7 is where the rule is stated in full.

In the console: clearances appear under the governance area and a pending one shows in the approvals queue for whoever holds the reviewing persona. The clearance surface is deliberately gated behind an administrative persona — a director cannot read or grant clearances, which is why "I can see the role but not its clearance" is a correct outcome rather than a permissions bug.

Summary

you need	the object	the call
let a role see classified material	role clearance	<code>api:POST /api/v1/role-clearances</code>
put that clearance in force	approval	<code>api:POST /api/v1/role-clearances/{id}/approve</code>
add need-to-know at <code>secret</code> +	compartments on the clearance	<code>compartments=[...]</code> at create
mark material	knowledge classification	<code>api:POST /api/v1/knowledge</code>

you need	the object	the call
put material in force	publication	<code>api:POST /api/v1/knowledge/{id}/publish</code>
let material cross a unit boundary	share policy	<code>api:POST /api/v1/knowledge-share-policies</code>
find share policies that outlived their project	review queue	<code>api:GET /api/v1/knowledge-share-policies/due-for-review</code>
find out which gate would refuse	access dry run	<code>api:POST /api/v1/governance/explain-access</code>

Next: 09.4 — The decision gradient

09.4 — The decision gradient

The Trust Plane does not return a boolean. It returns a position on a four-zone gradient, and the zone determines not only whether the action proceeds but who, if anyone, is told about it and what they can do.

This chapter is the delivery half of the governance argument. 09.2 and 09.3 established what bounds an action and what bounds a read; this chapter is what happens at the moment those bounds are consulted — the four zones, who decides in each, how a held action reaches a human, and the two escape hatches (emergency widening and the kill switch) that exist for the cases the gradient cannot resolve on its own.

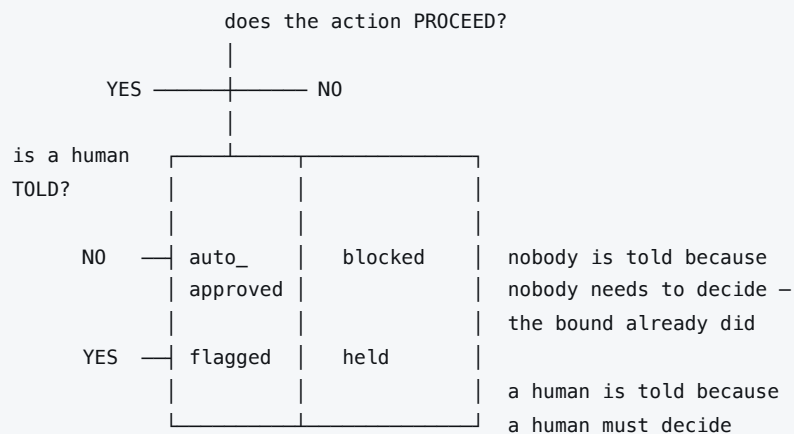
The idea to carry out: **a binary approve/reject collapses three genuinely different outcomes into two, and the one it loses is the useful one.** `flagged` — proceed, but tell somebody — is what makes autonomy survivable at scale, because it is the zone that produces oversight without producing a queue.

The four zones

zone	agent behaviour	human involvement	latency cost
<code>auto_approved</code>	proceeds autonomously	none — observable after the fact	none
<code>flagged</code>	proceeds, human notified	notified; can intervene	none to the agent
<code>held</code>	pauses for approval	yes — a bounded, specific decision	blocks until decided
<code>blocked</code>	cannot proceed	none — denied automatically	immediate refusal

The ordering is not a severity scale, and reading it as one leads to a design where everything drifts toward `held`. The correct reading is **two independent questions**:

THE GRADIENT IS TWO AXES, NOT ONE DIAL



The LEFT column costs the agent nothing. The RIGHT column stops the work.
The BOTTOM row costs a human's attention. The TOP row does not.

A design that only uses the diagonal (auto_approved / blocked) has thrown away both of the zones that carry information.

Read it as: **flagged** buys you oversight for free, and **held** buys you a decision at the cost of a human's attention and the agent's latency. The two costs are genuinely different and should be spent on different things. Flag what you want to *know about*; hold what you want to *decide*.

🚫 **Never implement `if approved: proceed else: deny` where the gradient applies.** The binary form has no way to express "this happened and you should look at it", so the actions that warrant attention either get promoted to **held** — producing an approval queue nobody can keep up with, which is abandoned within a fortnight — or demoted to **auto_approved**, where nothing records that anyone wanted to know. Both failure modes look like a working system until the incident review asks who saw it.

Which zone an action lands in

Three things determine the zone, and they compose in this order:

1. **The action's own risk classification**, from the gradient rule set. Read the rules in force at `api:GET /api/v1/constraints/gradient-rules`.
2. **The agent's posture**. A **supervised** agent holds far more of its surface than a **delegated** one; this is most of what a posture *means* (09.5).
3. **The envelope's `verification_defaults`**, which is the supervising role's handle on how thoroughly its delegate's actions are checked (09.2).

An organisation-level policy authored at the **gate** tier targets this gradient directly, carrying `{"zone": "flagged"|"held"|"blocked", "condition": "<text>"}`. Note the absence: **auto_approved** is not a valid gate-tier target, and that is correct rather than an omission — a policy that wants auto-approval is a policy that should not exist, since the absence of a gate policy already produces auto-approval.

```

policy = await client.governance.create_policy(
    name="Outbound wires are held",
    node_address="D1-R1-D1-R1",
    enforcement_tier="gate",
    policy_body={"zone": "held", "condition": "action is an external wire transfer"},
)

```

`api:GET /api/v1/analytics/verification-gradient` is the aggregate view — how the tenant's actions actually distributed across the four zones over a window. It is the single most useful number for answering *is our governance calibrated*, and the shape to look for is not a target percentage but a **non-degenerate distribution**. All four zones should have traffic. A tenant whose actions are 99%

`auto_approved` has a gradient that is not doing anything; a tenant with a large `held` fraction has a gradient that operators are about to route around.

```

distribution = await client.analytics.get_verification_gradient(days=30)

```

Held — the human decision, and what makes it bounded

A `held` action does not stop the agent forever. It creates a specific decision, for a specific human, with a specific set of options, and it waits.

```

pending = await client.approvals.list_pending()
for item in pending.records:
    detail = await client.approvals.get(item.id)

```

surface	route	what it carries
the queue	<code>api:GET /api/v1/approvals/pending</code>	everything awaiting this principal
one item	<code>api:GET /api/v1/approvals/{id}</code>	the action, the agent, the reason it was held
approve	<code>api:POST /api/v1/approvals/{id}/approve</code>	releases the action
reject	<code>api:POST /api/v1/approvals/{id}/reject</code>	refuses it; the agent is told why

Decision points — a richer form, where the human is choosing between options rather than approving a single action — have their own surface: `api:GET /api/v1/decisions/pending`, `api:GET /api/v1/decisions/{id}`, and `api:GET /api/v1/decisions/{id}/options` for the choices themselves. The objective-scoped view is `api:GET /api/v1/objectives/{id}/decisions`.

The property that makes a hold bounded rather than a blocker is that the decision is *specific*. A human is not being asked "should this agent be allowed to operate"; they are being asked "should this particular wire, for this amount, to this counterparty, go". That is a decision a domain expert can make in seconds without understanding the platform, and it is why the hold zone scales at all.

△ **A hold that nobody owns is an agent that stopped.** The decision routes to a human, and when the seat that human occupies is empty the routing walks *up* the reporting chain to the superior rather than stopping. That is the designed behaviour and it is what stops a vacancy severing a subtree. What it does not do is invent an approver: a hold routed to a chain whose upper reaches are also vacant sits in a queue with no reader, and the symptom is an objective that never completes and never errors. Put `api:GET /api/v1/approvals/pending` age on a dashboard.

Escalation — when a hold ages

Holds that are not answered escalate, and escalation is its own surface rather than a property of the hold.

surface	route
what is currently escalating	<code>api:GET /api/v1/escalation/pending</code>
the escalation chain for one task	<code>api:GET /api/v1/escalation/tasks/{id}/history</code>
escalation configuration for a unit	<code>api:GET /api/v1/escalation/config/{id}</code>
aggregate	<code>api:GET /api/v1/escalation/stats</code>
acknowledge, stopping the clock	<code>api:POST /api/v1/escalation/tasks/{id}/acknowledge</code>

Agent-side escalations — an agent asking for help rather than a hold ageing out — are `api:GET /api/v1/agent-escalations/pending`, `api:GET /api/v1/agent-escalations/{id}` and the live feed at `api:GET /api/v1/agent-escalations/stream`.

The design guidance here is one sentence: **escalation timeouts should be shorter than the work's own deadline, and the chain should be shorter than you think.** A three-hop escalation chain with a four-hour timeout per hop is a twelve-hour delay on a decision somebody could have made immediately, and the operational response to that is invariably to widen the envelope so the hold stops happening — which removes the control rather than fixing the latency.

Who may approve — distinctness, not seniority

Two properties bind every approval in the platform, and both are about identity rather than rank.

The approver's identity is derived server-side. It comes from the authenticated session, never from a field in the request body. An identity a client can assert is an identity a client can forge, and an approval is exactly the record that must not be able to lie about who made it.

Self-approval is refused by distinctness, not by policy. The requester and the approver must be different principals. This is checked at the identity level rather than at the role level, so holding two credentials does not produce two approvers, and a service identity is never an eligible approver for a human decision.

🚫 **Do not implement dual control by requiring two approvals from a queue anyone can read.** Two approvals from the same person under two credentials is one approval, and a design that counts rows rather than distinct principals will count it as two. Where genuine dual control is required, the platform's distinctness check is the mechanism — build on it rather than beside it.

The self-approval identity is pinned at creation time rather than re-resolved when the decision is made. That matters in a re-org: an approval created while someone held a role, decided after they left it, is still evaluated against the identity that created it. Re-resolving from current role occupancy would let a role change retroactively legalise a self-approval.

The emergency path

Sometimes the correct answer is that the bound is wrong *right now* and the work cannot wait for the bound to be re-authored. The platform has a first-class path for this and it is deliberately uncomfortable.

```
bypass = await client.emergency.create(
    agent_id=agent_id,
    reason="Counterparty settlement failure; manual wire required within the hour.",
    duration_hours=4,
    widened_constraints={"financial": {"max_amount": 250000, "currency": "USD"}},
)
```

surface	route
request one	api:POST /api/v1/emergency-bypass/create
what is active right now	api:GET /api/v1/emergency-bypass/active
all of them	api:GET /api/v1/emergency-bypass
one	api:GET /api/v1/emergency-bypass/{id}
approve / reject	api:POST /api/v1/emergency-bypass/{id}/approve api:POST /api/v1/emergency-bypass/{id}/reject
end one early	api:POST /api/v1/emergency-bypass/{id}/revoke

Four properties make it safe enough to exist:

1. **It auto-expires, on a hard timer.** Default four hours, maximum seventy-two. Expiry is enforced deterministically rather than by an application-level check that happens to run — a bypass whose window closes while no request is in flight still closes.
2. **The approval level scales with the duration.** Up to four hours is the immediate supervisor's call; four to twenty-four goes two levels up; twenty-four to seventy-two requires C-suite. Beyond seventy-two hours it is not an emergency and the answer is to change the envelope properly.
3. **It cannot widen beyond the approver's own envelope.** The monotonic rule from 09.2 still holds — an emergency widening is bounded by whoever authorised it, so nobody can grant authority they do not themselves have.

4. It carries distinct audit anchors and a post-incident review obligation within seven days. The record is deliberately separate from ordinary approval records so that "how often do we bypass" is a question with an answer.

🚫 **An emergency bypass with no expiry is not an emergency bypass.** The whole safety property is the hard timer; a widening that persists is simply a wider envelope, authored through a path with less review than the envelope would have had. The platform will not create one without an expiry. The failure mode to watch for in your own tooling is a scheduled job that re-creates the bypass as it lapses — which produces a permanent widening whose audit trail reads as a series of four-hour emergencies, and is the one shape a reviewer scanning `api:GET /api/v1/emergency-bypass` will spot immediately.

The kill switch

The gradient handles actions. The kill switch handles situations, and it is the only control in this part that operates on a class of work rather than on a single decision.

```
await client.kill_switch.activate(  
  scope="agent",  
  target_id=agent_id,  
  reason="Anomalous outbound volume; halting pending investigation.",  
)  
open_switches = await client.kill_switch.list_open()
```

`api:POST /api/v1/kill-switch/activate` opens one, `api:GET /api/v1/kill-switch/open` lists what is currently halted, `api:GET /api/v1/kill-switch/{id}` reads one, and `api:POST /api/v1/kill-switch/{id}/clear` ends it.

Choose between the four stop controls by blast radius and reversibility:

control	stops	reversible?	reach for it when
tighten constraints	one session's scope, partially	yes, by re-widening	part of the work is still in scope
pause	one session	yes, <code>resume</code>	you need time to look
kill switch	a class of work across the tenant	yes, <code>clear</code>	you do not yet know the blast radius
revoke	an agent's standing, permanently	no	the decision is meant to be final

The last row is 09.5's, and the distinction between it and the kill switch is the one that matters in an incident. A kill switch is a brake; revocation is a permanent removal that cascades to everything that derived authority from the revoked agent. Reaching for revocation because it feels more decisive is the mistake, and it is not undoable.

In the console: held actions and decision points both surface in the operator inbox (05.3), and a held action is what 05.4 is about from the human side. Flagged actions do not appear there — by design, since they did not stop — and are read from the activity surfaces instead. "The agent did nothing and there is nothing in my inbox" usually means `blocked` rather than `held`: a blocked action produces no decision because there is nothing to decide.

The delegation matrix — the gradient for an application

An application invoking agents on a user's behalf has its own version of the same question: which agent may it invoke, for which user, in which zone. That is the **delegation matrix**, and it is the gradient applied at the application boundary rather than at the action boundary.

```
matrix = await client.applications.get_delegation_matrix(application_id=app_id)
await client.applications.update_delegation_matrix(
    application_id=app_id,
    matrix=updated,
)
```

`api:GET /api/v1/applications/{id}/delegation-matrix` reads it and `api:PUT /api/v1/applications/{id}/delegation-matrix` sets it. The application's own record of what it actually did is `api:GET /api/v1/applications/{id}/audit-events`, which is the surface to reach for when the question is "what did this integration invoke last Tuesday" rather than "what is it permitted to invoke".

The matrix is worth calling out separately because of a specific confusion: **an application's grant and an agent's envelope are different bounds and both apply**. An application authorised to invoke an agent does not thereby widen that agent — the agent's envelope, posture and clearance are unchanged, and the invocation is decided the same way any other invocation is. What the matrix decides is whether the invocation is attempted at all.

The failure mode is granting through the matrix and expecting the agent to become capable. The symptom is an integration that is correctly authorised, makes the call successfully, and receives a constraint refusal — which reads as a platform inconsistency and is the two bounds doing exactly their separate jobs.

Designing a gradient that survives contact

Four pieces of guidance, in the order they matter.

Start narrow and widen on evidence. A new agent's surface should be mostly `held`, and each action class should move to `flagged` and then `auto_approved` as the posture evidence accumulates. That progression is the whole of 09.5, and doing it in the other direction — starting permissive and tightening after an incident — produces a control whose first calibration data point is the incident.

Flag generously; hold sparingly. Flagging costs the agent nothing and gives you a record and a notification. It is the correct zone for the large class of actions that are almost certainly fine and that somebody would want to know about afterwards.

Measure the queue, not the policy. A `held` zone whose queue ages is a control that is about to be removed. Watch queue depth and age on `api:GET /api/v1/approvals/pending` and treat a growing queue as a calibration failure rather than a staffing one.

Make the emergency path pleasant enough to use. The failure mode of a painful bypass process is not that people wait — it is that they find a route that does not go through it, and that route has no audit trail at all. Four hours, one approver, one form field, and a review afterwards is a process people follow.

Summary

you need	the zone or control	the call
know an action happened	<code>flagged</code>	gate-tier policy, <code>zone: flagged</code>
decide an action yourself	<code>held</code>	<code>api:GET /api/v1/approvals/pending</code>
refuse an action outright	<code>blocked</code>	gate-tier policy, or the envelope refuses it
see how the gradient is calibrated	aggregate	<code>api:GET /api/v1/analytics/verification-gradient</code>
read the rules in force	gradient rules	<code>api:GET /api/v1/constraints/gradient-rules</code>
widen a bound for hours, not permanently	emergency bypass	<code>api:POST /api/v1/emergency-bypass/create</code>
see what is currently widened	active bypasses	<code>api:GET /api/v1/emergency-bypass/active</code>
halt a class of work now	kill switch	<code>api:POST /api/v1/kill-switch/activate</code>
stop one session and keep it	intervention	<code>api:POST /api/v1/interventions/{id}/pause</code>

Next: 09.5 — Trust chains and postures

09.5 — Trust chains and postures

Every decision in the three preceding chapters rests on a question those chapters take for granted: *on whose authority is this agent acting, and how much may it do unaided?* Those are two separate objects with two separate lifecycles, and this chapter is both of them end to end — established, verified, moved, suspended, withdrawn, and evidenced.

02.4 shows you the calls. 06.4 covers the object shapes and the decision path. This chapter is the *governance* argument: what standing means, how it is earned rather than assigned, what withdrawing it actually removes, and what an auditor can be shown.

The idea to carry out: **trust here is not a boolean the platform computes about an agent — it is a set of first-class objects you can read, compose and argue with.** A lineage that names where authority came from, a standing that moves and can be explained, and a decision that can be re-derived after the fact.

Four objects that people collapse into one

object	answers	mutable?	read at
Authority	who attested this agent?	created, deactivated	<code>sdk:aegis_sdk.trust.AuthoritiesModule</code>
Chain	where did its authority come from?	established, revoked	<code>sdk:aegis_sdk.trust.ChainsModule</code>
Posture	how much may it do unaided, now?	moves both ways	<code>sdk:aegis_sdk.trust.PosturesModule</code>
Decision	was <i>this</i> action permitted?	— it is a record	<code>api:POST /api/v1/trust/verify</code>

They are separable in practice, not merely in principle, and every combination occurs. An agent can hold a valid chain and a restrictive posture — the normal state of anything newly provisioned. An authority can be retired while chains established under it continue to exist. A chain can be live and a particular action still refused, because the decision consults the envelope and the posture as well as the chain.

The authority — the signing root

A chain is not self-originating. An **organisational authority** is the signing root a chain is established under: agents are attested *by* an authority, and that relationship is recorded in the chain's genesis.

```
authority = await client.trust.authorities.create(
    name="Treasury Operations Authority",
    description="Attests agents operating inside the Treasury desk.",
)
agents = await client.trust.authorities.list_agents(authority["id"])
```


Enumerate them at `api:GET /api/v1/trust/authorities`, create one with `api:POST /api/v1/trust/authorities`, amend with `api:PATCH /api/v1/trust/authorities/{id}`, and read the agents established under one at `api:GET /api/v1/trust/authorities/{id}/agents`.

🛑 **Deactivating an authority is a security transition that every chain established under it inherits.** `api:POST /api/v1/trust/authorities/{id}/deactivate` is not a rename and not housekeeping — which is why the platform requires a reason and records it on the deactivation entry. Treat it like a revocation: decide the blast radius first, using `api:GET /api/v1/trust/authorities/{id}/agents`, and expect every agent in that list to be affected.

How many authorities should a tenant have? One per accountability boundary, not one per department and not one overall. The test is whether you would ever want to retire one of them independently — if two groups of agents would always be retired together, they share an authority; if retiring one group should leave the other standing, they do not.

The chain — lineage from a human root

Establishing is `api:POST /api/v1/trust/establish`, returning `sdk:aegis_sdk.EstablishedTrustChain`.

```
chain = await client.trust.chains.establish(
    agent_id=agent_id,
    authority_id=authority["id"],
    capabilities=["read_positions", "draft_settlement"],
    constraints=["treasury_desk_only"],
)
```

The body names the agent, the authority, a list of **capability labels** and a list of **constraint labels**. Both are lists of strings, not dictionaries — the actual bounds live in envelopes (09.2), and these labels are how the chain describes what it was established *for*. Expiry defaults to 365 days, and chains genuinely expire.

Two properties constrain what you can build rather than merely how you call it:

Human origin is derived server-side from the authenticated caller and cannot be supplied. An identity a client can assert is an identity a client can forge, and the one fact a lineage must not be able to lie about is where the human authority came from. The practical consequence is that **the human on the record is whoever's credentials ran the script** — so provisioning an entire organisation under a shared machine key produces a tenant whose whole authority structure descends from a service account, which is exactly the finding an auditor is looking for. Provision under a named principal.

Chains are keyed by agent. What looks like a chain id in the chain routes is an agent id, and an agent with no chain returns an *empty path* rather than an error. `None` there means "no chain", never "failed to load".

read	route	shape
all chains	<code>api:GET /api/v1/trust/chains</code>	<code>sdk:aegis_sdk.TrustChain</code> summaries
the full lineage document	<code>api:GET /api/v1/trust/chains/{id}</code>	genesis, delegations, capabilities
the narrow projection	<code>api:GET /api/v1/trust/chains/{id}/summary</code>	the clearance-gated summary
the walk to the human root	<code>api:GET /api/v1/trust/chains/{id}/delegation-path</code>	<code>sdk:aegis_sdk.DelegationPath</code>
a delegate agent's chain	<code>api:GET /api/v1/delegate-agents/{id}/trust-chain</code>	the chain for one delegate

⚠ **Identity is nested one level down on the full lineage document.** On the document returned by `api:GET /api/v1/trust/chains/{id}`, `id`, `agent_id` and `authority_id` live on `genesis`, not at the top level. A reader that looks a level too high finds none of them — and on a permissive parser reads three nulls beside fully-populated capability rows, which presents as a response problem rather than as a lookup one. `sdk:aegis_sdk.TrustChain` is the flat summary shape, `sdk:aegis_sdk.EstablishedTrustChain` is the establishment shape, and the lineage document is a third, richer shape again.

The summary projection is **clearance-gated under selective disclosure**: some fields may be *absent* rather than null when your clearance does not reach them, while `id`, `status` and `created_at` are always visible. A null on a gated field is therefore ambiguous between "not disclosed to you" and "genuinely absent", and the response cannot tell you which. Do not build a security decision on reading absence.

Delegation between agents is `api:POST /api/v1/trust/delegate`, returning `sdk:aegis_sdk.TrustDelegationRecord`; a single delegation edge is removed with `api:POST /api/v1/trust/revoke-delegation`. Bulk chain generation during org standup is `api:POST /api/v1/organization-builder/generate-trust-chains`.

The posture — standing, and how it moves

A posture is not a property of the chain. It is the current answer to *how much may this agent do without a human in the loop*, and it moves in both directions.

posture	what it means	clearance ceiling
pseudo	deterministic behaviour only; no autonomous judgement	<code>public</code>
supervised	acts, with a human approving the consequential paths	<code>restricted</code>

posture	what it means	clearance ceiling
<code>shared_planning</code>	plans with a human, executes within the agreed plan	<code>confidential</code>
<code>continuous_insight</code>	acts autonomously, with continuous human visibility	<code>secret</code>
<code>delegated</code>	acts on delegated authority within its envelope	the role's full clearance

Read it at `api:GET /api/v1/agents/{id}/trust-posture`, which returns `sdk:aegis_sdk.TrustPostureInfo`. Two fields in that response are load-bearing and are routinely confused:

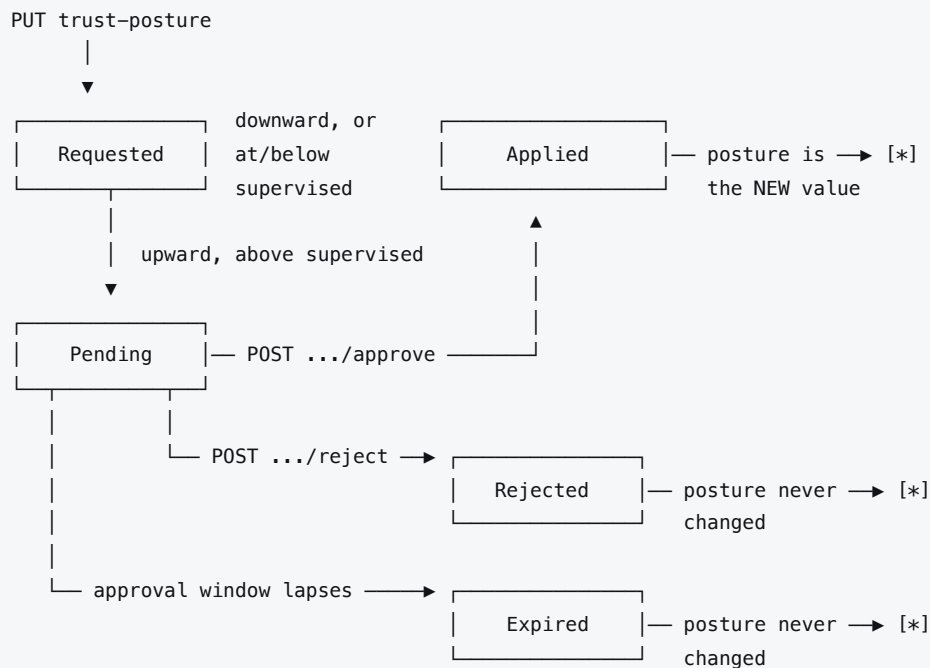
- **`posture` is the EFFECTIVE posture.** If an override is active, this is the override's value, not the agent's own.
- **`base_posture` is what the agent reverts to** when the override lapses.

So *what posture is this agent at* has two correct answers depending on whether you mean **now** or **by default**, and an emergency restriction implemented as an override is precisely the case where they differ. `sdk:aegis_sdk.PostureOverride` models the override row itself — who initiated it, why, and when it expires.

The unit posture ceiling from 09.2 composes here: an agent's effective posture is the lowest of its own posture, its unit's effective ceiling, and any posture selected for the particular piece of work. A child unit cannot raise its ceiling above its parent's, so the organisational brake cascades the same way envelopes do.

Changing a posture — one endpoint, two outcomes

Both a governed progression request and an administrative override go through the *same* call, `api:PUT /api/v1/agents/{id}/trust-posture`, which decides internally whether the change applies immediately or is held for a human. There is no separate override endpoint and the two are gated on the same permission.



While a change is Pending the agent is STILL RUNNING at the old posture, and the result reports the OLD value. Do not re-issue the same transition to "make it take" – it records nothing new and comes back flagged.

Read it as: **branch on `approval_pending`**. When it is `True`, `result.posture` is the posture the agent is *still running at*. Reporting it as the new one is the single most common integration defect in this area, and it compounds — a caller that retries after a timeout files a *second* pending request, and past one pending request the server refuses rather than picking one.

The remedy is one read before the write: check `api:GET /api/v1/agents/{id}/trust-posture/pending` and treat a non-null answer as *already asked* rather than as an obstacle. The decision routes are `api:POST /api/v1/agents/{id}/trust-posture/approve` and `api:POST /api/v1/agents/{id}/trust-posture/reject`, both of which accept an explicit approval id. `api:POST /api/v1/agents/{id}/trust-posture/request-upgrade` is the agent-initiated form, and `api:GET /api/v1/posture/pending-approvals` is the tenant-wide queue.

```

pending = await client.trust_posture.get_pending(agent_id=agent_id)
if pending is None:
    result = await client.trust_posture.update(
        agent_id=agent_id,
        posture="continuous_insight",
        reason="Six weeks at shared_planning with zero constraint violations.",
    )

```

Downward always applies immediately. That asymmetry is the whole design: a restriction must never wait on an approval, because the moment you need one is the moment nobody is available to grant it. It is also what makes a downward override the correct reversible brake — see § Withdrawal below.

Progression — autonomy earned, and the evidence for it

There is a whole read surface for *why* an agent's standing is what it is, and to an auditor it is worth more than the posture value.

read	route	what it answers
eligibility verdict	<code>api:GET /api/v1/agents/{id}/trust-posture/evaluate</code>	may this agent progress?
upgrade eligibility	<code>api:GET /api/v1/agents/{id}/trust-posture/upgrade-eligibility</code>	to what, and what is missing
behavioural rates	<code>api:GET /api/v1/agents/{id}/trust-posture/metrics</code>	what eligibility is computed <i>from</i>
the basis	<code>api:GET /api/v1/agents/{id}/trust-posture/evidence</code>	the observations behind the rates
how it got here	<code>api:GET /api/v1/agents/{id}/trust-posture/history</code>	every transition, and why

`api:GET /api/v1/agents/{id}/trust-posture/evaluate` is the **only** call that answers "may this agent progress". No posture read and no change response carries an eligibility verdict, and inferring one from the metrics is how a progression gets made on numbers that were never compared to a threshold.

```
verdict = await client.trust_posture.evaluate(agent_id=agent_id)
metrics = await client.trust_posture.get_metrics(agent_id=agent_id)
history = await client.trust_posture.get_history(agent_id=agent_id)
```

🚫 **Look at the history endpoint before you decide whether your governance is real.** If every entry in it is an override, the graduated progression was decorative: the agent was *placed* at its autonomy level rather than having earned it. An agent that has never been observed under supervision has no evidence behind its autonomy — which is exactly the thing you would be asked to show, and the absence reads the same in the API as a well-earned posture does in the posture field alone.

Design a progression path deliberately. The shape that works is: provision at `supervised`, accumulate observations under a gradient weighted toward `held` (09.4), evaluate on a cadence rather than on demand, and progress one level at a time with the evidence attached to the request's reason. The shape that does not work is provisioning at `delegated` because the agent needs to be useful on day one — which produces an agent with maximal standing and an empty evidence surface, and no way to narrow it later that does not read as a punishment.

Verification — how a permit is actually reached

`api:POST /api/v1/trust/verify` is the operation the enforcement path calls on an agent's behalf. It takes an agent, an **action** (a string such as `write` or `execute`), and a target addressed as a **kind plus an optional instance**: `resource_type` says what sort of thing is being acted on, `resource_id` says which one.

```
decision = await client.trust.verify(  
    agent_id=agent_id,  
    action="write",  
    resource_type="knowledge_item",  
    resource_id=document_id,  
)
```

The kind is required and the client refuses rather than inventing one — the platform writes that value into its audit trail, so a fabricated placeholder would read as one the caller had named.

The result is `sdk:aegis_sdk.TrustVerificationResult`, carrying `allowed` and `reason`, with the richer shape carrying `capabilities_matched` and — the field to read when a request was refused — `constraints_violated`.

⚠ Read `constraints_violated`, not `constraints_applied`. The latter is inherited from a base model and is always empty on this route, so reading it for the reason a denial happened tells you nothing and tells you so silently. Caller-supplied `context` is likewise accepted and ignored: the evaluation runs against the chain's own recorded constraints, never against anything the request asserts about itself.

For multi-agent work, `api:POST /api/v1/trust/pipeline/validate` pre-flights a whole pipeline: it confirms every agent holds an active chain carrying the capabilities its step requires. Read `all_valid` as a statement about chains and capabilities — the envelope bounds are evaluated at the decision, not on this route — and per-agent detail at `api:GET /api/v1/trust/pipeline/{id}/agents/{agent_id}`.

What is display and what is enforcement

Two read surfaces look like controls and are projections for humans. `api:GET /api/v1/trust/agents/{id}/trust-score` returns a grade; `api:GET /api/v1/trust/agents/{id}/care-budget` returns the five-dimension CARE budget. Neither is consulted by the enforcement path.

That is the correct design — a score that gates an action is a score an adversary optimises against — and the consequence for you is one rule: **never gate your own logic on a grade**. Gate on the decision. The score is for a human deciding whether to look more closely; the decision is for a machine deciding whether to proceed.

The genuinely useful agent-scoped reads are `api:GET /api/v1/trust/agents/{id}/capabilities` (what the chain established), `api:GET /api/v1/trust/agents/{id}/constraints` (the bounds in force), `api:GET /api/v1/trust/agents/{id}/trust-context` (`sdk:aegis_sdk.AgentTrustContext`,

the composed picture) and `api:GET /api/v1/trust/agents/{id}/trust-summary`.

Withdrawal — the reversible brake and the permanent one

Two different operations, and choosing the wrong one is expensive in opposite directions.

The reversible brake is a posture, not a chain state. There is no operation that suspends a trust chain. If you need to stop an agent acting without permanently ending its lineage, the honest tool is a *downward* posture override — `api:PUT /api/v1/agents/{id}/trust-posture` — which applies immediately rather than waiting on approval, and which you can lift afterwards.

Revocation is permanent and cascades. `api:POST /api/v1/trust/revoke/{id}/cascade` removes an agent's standing and everything that derived authority from it, returning `sdk:aegis_sdk.CascadeRevocationResult`. The cascade is not optional: every revocation route cascades, so there is no non-cascading revoke to reach for.

```
impact = await client.trust.revocation.get_impact(agent_id=agent_id)
if impact.affected_agent_count > 1:
    ... # decide deliberately before proceeding
result = await client.trust.revocation.revoke_cascade(agent_id=agent_id)
```

Look before you cut. `api:GET /api/v1/trust/revoke/{id}/impact` returns `sdk:aegis_sdk.RevocationImpact` — how many agents are affected, and whether active workloads are at risk. That preview is the difference between removing one agent and silently removing a department's, and it costs one call.

`api:POST /api/v1/trust/revoke/by-human/{id}` revokes everything originating from a particular person, which is what you want when someone leaves. `api:POST /api/v1/trust/revoke` is the general form.

🚫 **A cascade can fail to complete, and that is a real, discoverable state.** `api:GET /api/v1/trust/revoke/jobs/incomplete` lists cascade revocations that did not finish — and **until a job there is resumed, agents it named may still hold active trust.** `api:POST /api/v1/trust/revoke/jobs/{id}/resume` retries it idempotently and reports whether the resume itself timed out again. Read the two counts on a job — completed against total — rather than its status string alone, because the status does not tell you how far it got. **Put this listing on a schedule.** A revocation you believe landed and which did not is the most expensive possible version of this system's failure modes: the agent is gone from every dashboard and still holds standing.

Discovery, health and the tenant-wide view

read	route	what it answers
registry discovery	<code>api:POST /api/v1/trust/registry/discover</code>	which agents are discoverable

read	route	what it answers
one registry entry	<code>api:GET /api/v1/trust/registry/agents/{id}</code>	its registration
liveness	<code>api:POST /api/v1/trust/registry/agents/{id}/heartbeat</code>	the agent is still there
plane health	<code>api:GET /api/v1/trust/health</code>	is the trust plane serving
tenant metrics	<code>api:GET /api/v1/trust/metrics</code>	chain and verification counts
exportable metrics	<code>api:GET /api/v1/trust/metrics/export</code>	the same, portable
compliance report	<code>api:GET /api/v1/trust/compliance/{id}</code>	the defensible scan

Two notes on reading these honestly. **The discovery surface reads chains, not agents** — an agent with no chain is invisible to discovery even if it exists and is running, so an empty discovery result is not evidence that no such agent exists. And `api:GET /api/v1/trust/metrics` **computes from a bounded window** of recent records, which makes it the right surface for a dashboard and the wrong one for a figure that has to be defensible; the compliance report scans further and declares when it could not, which is why it is the one to reach for when a number is going into a document.

In the console: the trust area shows authorities, chains and postures on separate surfaces, and the posture surface is where progression requests appear for approval. An agent showing an effective posture different from its base posture renders the override explicitly, with its expiry — which is the fastest way to answer "why can it not do that today".

Summary

you need	the object	the call
a signing root	authority	<code>api:POST /api/v1/trust/authorities</code>
give an agent standing	chain	<code>api:POST /api/v1/trust/establish</code>
see where its authority came from	delegation path	<code>api:GET /api/v1/trust/chains/{id}/delegation-path</code>
know how autonomous it is now	posture	<code>api:GET /api/v1/agents/{id}/trust-posture</code>
move it, up or down	posture change	<code>api:PUT /api/v1/agents/{id}/trust-posture</code>

you need	the object	the call
know whether a move is earned	eligibility	<code>api:GET /api/v1/agents/{id}/trust-posture/evaluate</code>
show that it was earned	history + evidence	<code>api:GET /api/v1/agents/{id}/trust-posture/history</code>
ask whether an action is permitted	verification	<code>api:POST /api/v1/trust/verify</code>
stop it reversibly	downward override	<code>api:PUT /api/v1/agents/{id}/trust-posture</code>
see what a revocation would cost	impact preview	<code>api:GET /api/v1/trust/revoke/{id}/impact</code>
end it permanently	cascade revocation	<code>api:POST /api/v1/trust/revoke/{id}/cascade</code>
catch a cascade that did not finish	incomplete jobs	<code>api:GET /api/v1/trust/revoke/jobs/incomplete</code>

Next: 09.6 — The audit spine and evidence

09.6 — The audit spine and evidence

Everything in the four preceding chapters decides something. This chapter is about what is left afterwards — what is recorded, by which subsystem, how it is queried, how it is shown to be intact, and how it becomes the artefact somebody outside your organisation will accept.

This is where the product's actual claim is discharged. Aegis does not carry your accountability for what your agents do; it gives you the proof that lets you discharge it — bounded mandates you can show, human gates you can evidence, a lineage that terminates at a named person, and a record whose integrity is demonstrable rather than asserted.

The idea to carry out: **the decision and the record are different witnesses to one event, and neither substitutes for the other**. The decision is about an intent and is made before anything happens. The record is written after. A system that can produce one and not the other cannot answer the question an auditor actually asks, which is never "was something enforced" but "show me that it was".

Two logs, two questions

There are two distinct audit surfaces, they are written by different subsystems, and conflating them is how an evidence request gets answered with the wrong document.

	platform audit	trust audit
read at	<code>api:GET /api/v1/audit/logs</code>	<code>api:GET /api/v1/trust/audit</code>
records	user actions, resource changes	trust decisions — who delegated what, on what basis
module	<code>sdk:aegis_sdk.modules.ObserveAuditModule</code>	<code>sdk:aegis_sdk.trust.AuditModule</code>
answers	"who changed this?"	"why was this agent allowed?"
subject	a resource and the principal who touched it	an agent and the authority it acted under

```
changes = await client.observe_audit.list_logs(  
    resource_type="role_envelope",  
    limit=100,  
)  
decisions = await client.trust.audit.query(  
    agent_id=agent_id,  
    limit=100,  
)
```

The platform log is the one to reach for when the question is about a *change*: who edited this envelope, who approved this clearance, who moved this role. Single entries are `api:GET /api/v1/audit/logs/{id}`, a resource's whole history is `api:GET /api/v1/audit/resources/{type}/{id}`, and a principal's activity is `api:GET /api/v1/audit/users/{id}`.

The trust log is the one to reach for when the question is about an *action*: why was this agent permitted to do that, what chain was in force, what capabilities matched. `api:POST /api/v1/trust/audit` records an entry; `api:GET /api/v1/trust/audit/by-human/{id}` filters to everything that originated with one person.

Two narrower surfaces exist and are worth knowing so you do not go looking for their content in the two above. `api:GET /api/v1/settings/audit-log` records configuration changes, with `api:POST /api/v1/settings/audit-log/revert` to undo one. `api:GET /api/v1/admin/access/audit` records administrative access. And `api:GET /api/v1/applications/{id}/audit-events` scopes to one application's grants and invocations.

⚠ "The audit log" is not one thing, and an evidence request phrased that way needs a clarifying question before it is answered. The failure mode is an auditor asking for "the audit trail for this decision", being handed the platform log, finding nothing about the decision in it, and concluding the decision was not recorded. It was — in the other log. Name which surface you are quoting whenever you quote one.

Walking a decision back to the human who authorised it

The trust audit's most valuable operation takes a single entry and walks its lineage **backwards to the root human source** — the delegation chain, its depth, and the person at the top.

```
trace = await client.trust.audit.trace(entry_id=entry_id)
```

`api:GET /api/v1/trust/audit/trace/{id}` is the operation that discharges the accountability claim in practice. An organisation can point at an action an agent took, six months later, and name the human whose authority it descended from — not by inference from an org chart, but from the chain that was actually in force at the time.

That is the single most useful thing in this chapter, and it is worth building your evidence process around it. When an incident review asks "who authorised this", the answer is a call rather than a reconstruction.

🚫 Read the `verified` flag on a trace with its own warning in hand. That field reports signature verification and **defaults to true for an entry that carries no signature at all**. It therefore means "*no signature check failed*", not "*this entry is cryptographically verified*" — an unsigned entry and a validly signed one are indistinguishable on that field alone. Check whether the record carries a signature before treating the flag as evidence, and never report a trace as cryptographically verified on the strength of it.

Is the log itself intact?

The audit log is tamper-evident, and there are two different reads for that claim. They answer different questions and are frequently swapped.

read	route	what it does
on-demand verification	<code>api:GET /api/v1/compliance/audit/verify</code>	performs the hash-chain walk now, over an optional range
the last sweep's result	<code>api:GET /api/v1/compliance/chain/integrity</code>	reads the result of the platform's own periodic verification

```
walk = await client.compliance.verify_audit(start_id=first, end_id=last)
integrity = await client.compliance.get_chain_integrity()
```

`api:GET /api/v1/compliance/audit/verify` returns whether the chain was valid, how many entries were checked, and the id of the first invalid entry if tampering was found. The entry count is the number to quote — "valid" over an unstated range is a claim about an unknown population.

`api:GET /api/v1/compliance/chain/integrity` does not trigger a sweep; it reports the last one.

🚫 **Check the last-verified timestamp before reading the ok flag.** A deployment that has not yet run a sweep reports *not ok* — which is a "nothing has been checked yet" finding and not a "something is broken" one. Those two produce the same boolean and opposite alarms, and the timestamp is the only thing that distinguishes them. Read `last_verified_at` on a schedule of your own and alarm on staleness as well as on the flag; a sweep that stopped running six weeks ago reports `ok` forever.

`api:GET /api/v1/compliance/audit/entries` is the compliance-scoped entry listing, and `api:POST /api/v1/compliance/audit/export` produces a portable copy of it.

Compliance frameworks

The platform models compliance frameworks as first-class objects with controls, rather than as a report format. That distinction matters because it makes the question *which control does this evidence satisfy* answerable rather than editorial.

```
frameworks = await client.compliance.list_frameworks()
controls = await client.compliance.get_framework_controls(framework_id="soc2")
```

read	route
which frameworks are modelled	<code>api:GET /api/v1/compliance/frameworks</code>
one framework	<code>api:GET /api/v1/compliance/frameworks/{id}</code>
its controls	<code>api:GET /api/v1/compliance/frameworks/{id}/controls</code>
the current posture against all of them	<code>api:GET /api/v1/compliance/dashboard</code>
a single score	<code>api:GET /api/v1/compliance/score</code>

read	route
open findings	api:GET /api/v1/compliance/violations
the event stream behind them	api:GET /api/v1/compliance/events
service health	api:GET /api/v1/compliance/health

SOC 2 has its own surfaces because the evidence package is a recurring deliverable rather than a query: `api:GET /api/v1/compliance/soc2/status` reports where you stand, `api:POST /api/v1/compliance/soc2/evidence` generates the package, and `api:GET /api/v1/compliance/soc2/evidence/export` retrieves it.

HIPAA is a mode rather than a report, because it changes platform behaviour rather than describing it: `api:POST /api/v1/compliance/hipaa/enable` and `api:POST /api/v1/compliance/hipaa/disable` switch it, `api:GET /api/v1/compliance/hipaa/settings` reads the configuration, and `api:GET /api/v1/compliance/hipaa/status` reads the current state.

```
await client.compliance.enable_hipaa()
status = await client.compliance.get_hipaa_status()
```

⚠ **Enabling a compliance mode is a change to how the deployment behaves, and it is recorded as one.** Treat it as you would an envelope change rather than as a settings toggle: decide it deliberately, record why, and expect the change itself to appear in the platform audit log. The symptom of treating it casually is a deployment whose compliance posture nobody can explain the history of, which is a worse position than not having enabled it.

Assessments and reports are `api:POST /api/v1/compliance/assess` to run one, `api:POST /api/v1/compliance/reports` to generate a report, `api:GET /api/v1/compliance/reports` to list them, `api:GET /api/v1/compliance/report/summary` for the condensed form, and `api:GET /api/v1/compliance/reports/{id}.pdf` for the document an auditor will actually accept. `api:POST /api/v1/compliance/export` is the general export. Alerts raised by an assessment are acknowledged at `api:POST /api/v1/compliance/alerts/{id}/acknowledge`.

Retention

Records that are kept forever are a liability as well as an asset, and retention is modelled explicitly rather than left to infrastructure.

surface	route
compliance retention policies	api:GET /api/v1/compliance/retention/policies
execute a retention pass	api:POST /api/v1/compliance/retention/execute
data-governance retention rules	api:POST /api/v1/data-governance/retention
read one	api:GET /api/v1/data-governance/retention/{id}
amend one	api:PUT /api/v1/data-governance/retention/{id}

The design guidance is short and consequential: **retention policy is a governance decision, not an operational one, and the person who sets it should not be the person who runs it.** A retention pass deletes evidence. Authoring the policy and executing it are separate calls precisely so they can be separately authorised, and a deployment where the same automation does both has removed the step where somebody notices the window is wrong.

Consent and lineage

Two data-governance surfaces belong in an evidence chapter because they answer questions auditors ask that the audit logs cannot.

Consent records that a subject permitted a use, and is queried rather than inferred. `api:POST /api/v1/data-governance/consents` records one, `api:POST /api/v1/data-governance/consents/check` tests whether a given use is covered, `api:GET /api/v1/data-governance/consents` and `api:GET /api/v1/data-governance/consents/{id}` read them back, and `api:POST /api/v1/data-governance/consents/{id}/withdraw` withdraws one.

```
covered = await client.governance.check_consent(  
    subject_id=subject_id,  
    purpose="model_training",  
)
```

Lineage records where data came from and where it went.

read	route	answers
the whole graph	<code>api:GET /api/v1/data-governance/lineage/graph</code>	the shape
one node	<code>api:GET /api/v1/data-governance/lineage/nodes/{id}</code>	what it is
what fed it	<code>api:GET /api/v1/data-governance/lineage/nodes/{id}/upstream</code>	provenance
what it fed	<code>api:GET /api/v1/data-governance/lineage/nodes/{id}/downstream</code>	propagation
blast radius	<code>api:GET /api/v1/data-governance/lineage/nodes/{id}/impact</code>	what a change or a withdrawal touches

Nodes and edges are declared with `api:POST /api/v1/data-governance/lineage/nodes` and `api:POST /api/v1/data-governance/lineage/edges`.

The **impact** read is the one that earns its place. When a consent is withdrawn or a source is found to have been mis-classified, the question is not "what is this" but "what did this touch" — and answering it by search rather than by lineage is how a remediation misses the derived artefact that carries the same sensitivity under a different name. 09.7 carries the classification half of the same argument.

What an evidence request actually looks like

Four questions cover almost every request, and each has one primary surface.

the question	the surface	what you hand over
"What bounded this agent?"	<code>api:GET /api/v1/constraints/agents/{id}/effective</code>	the composed envelope, plus its ancestors from <code>.../inherited</code>
"Who decided it could act?"	<code>api:GET /api/v1/trust/audit/trace/{id}</code>	the lineage walk to a named human
"What was recorded?"	<code>api:GET /api/v1/audit/export</code> and <code>api:GET /api/v1/trust/audit</code>	both logs, scoped to the window
"How do I know the record is intact?"	<code>api:GET /api/v1/compliance/audit/verify</code>	the hash-chain walk, with its entry count

```
bundle = {
  "bounds": await client.governance.get_agent_constraints(
    agent_id=agent_id, resolution="effective"
  ),
  "authority": await client.trust.audit.trace(entry_id=entry_id),
  "decisions": await client.trust.audit.query(agent_id=agent_id),
  "integrity": await client.compliance.verify_audit(),
}
```

A fifth question arrives in any engagement that has read 09.5: *"was this agent's autonomy earned?"*

The answer is `api:GET /api/v1/agents/{id}/trust-posture/history` alongside `api:GET /api/v1/agents/{id}/trust-posture/evidence`, and it is the one most likely to surprise you the first time you run it.

 **Run the evidence bundle before the audit, not during it.** Every surface above is a query, and every query has a shape you will want to have seen once already. The findings that cost the most are structural rather than substantive — a tenant whose whole authority structure descends from a service account, a posture history that is all overrides, a chain-integrity sweep that has never run. None of those is a breach and all three are difficult to explain under time pressure. All three are visible in a single afternoon.

One event, both logs

The clearest way to internalise the split is to follow a single action through it. An analyst agent drafts an outbound wire; the gradient holds it; a treasury lead approves; the wire is drafted.

ONE ACTION – WHAT EACH LOG RECORDS

TRUST AUDIT

the agent's chain was resolved
the envelope permitted a draft
the posture placed it in `held`
–
–
the decision was re-evaluated and
permitted
–

answers: on whose authority,
against which bounds

PLATFORM AUDIT

–
–
–
the approval row was created
the lead approved it (who, when)
–
the draft artifact was created

answers: who did what,
to which resource

NEITHER log contains the other's rows. The approval is a RESOURCE CHANGE and lives in the platform log; the decision it unblocked is a TRUST DECISION and lives in the trust log. An evidence bundle for this single action needs BOTH, plus the trace that joins them.

Read it as: the join between the two logs is the action, and the operation that performs the join is the lineage trace. That is why the trace matters more than either listing — it is the only surface that takes a point in one log and walks to the authority behind it.

The record shape itself is `sdk:aegis_sdk.TrustAuditEntry`, and queries against it are expressed as `sdk:aegis_sdk.TrustAuditQuery`. Three fields on an entry do most of the work in an evidence conversation:

field	what it settles
the agent and the action	<i>what was decided</i> , precisely enough to match against a policy
the chain in force at the time	<i>on whose authority</i> — and it is the chain as it was, not as it is now
the reason	<i>why</i> the verdict went the way it did, in the engine's own words

The middle one is the property people assume and should verify: the entry records the chain **as it stood at the decision**, so a later revocation does not retroactively change what the record says was permitted. That is what makes the log usable as evidence of a past state rather than as a view of the present one.

Exports, and what makes them portable

`api:GET /api/v1/audit/export` produces the platform log in a form you can hand over. `api:GET /api/v1/trust/metrics/export` does the same for trust metrics. `api:GET /api/v1/compliance/soc2/evidence/export` produces the SOC 2 package.

Two properties make an export defensible rather than merely large:

It names its range. An export with no stated window is a claim about an unknown population, and the first thing a competent reviewer will ask is what it excluded. State the window in the request and carry it into whatever you hand over.

It is paired with an integrity walk over the same range. `api:GET /api/v1/compliance/audit/verify` accepts start and end bounds for this reason. An export plus a verification over the same bounds is a materially stronger artefact than either alone, because together they say *this is what happened, and here is why you can believe the record was not altered*.

In the console: the compliance area carries the dashboard, the framework status surfaces and the report list, and generated reports appear there for download rather than arriving by any other route. The audit surfaces are deliberately read-only in the console — the only audit-adjacent write is the settings revert, which is itself recorded.

Summary

you need	the surface	the call
who changed this resource	platform audit	<code>api:GET /api/v1/audit/resources/{type}/{id}</code>
why this agent was allowed	trust audit	<code>api:GET /api/v1/trust/audit</code>
the human behind an action	lineage trace	<code>api:GET /api/v1/trust/audit/trace/{id}</code>
everything from one person	by-human filter	<code>api:GET /api/v1/trust/audit/by-human/{id}</code>
prove the log is unaltered	hash-chain walk	<code>api:GET /api/v1/compliance/audit/verify</code>
the last sweep's verdict	chain integrity	<code>api:GET /api/v1/compliance/chain/integrity</code>
where you stand against a framework	compliance dashboard	<code>api:GET /api/v1/compliance/dashboard</code>
the SOC 2 package	evidence export	<code>api:GET /api/v1/compliance/soc2/evidence/export</code>
a document to hand over	report PDF	<code>api:GET /api/v1/compliance/reports/{id}.pdf</code>
what a withdrawal would touch	lineage impact	<code>api:GET /api/v1/data-governance/lineage/nodes/{id}/impact</code>
bound how long records are kept	retention policy	<code>api:POST /api/v1/data-governance/retention</code>

Next: 09.7 — Containment and explainable refusal

09.7 — Containment and explainable refusal

The six preceding chapters govern *decisions*. This one governs *places* — the containers that classified material sits in, what a container is allowed to hold, and what happens when material tries to leave one. It closes with the other half of the same discipline: a refusal that names the rule that refused it, because a control the operator cannot diagnose is a control the operator routes around.

This is the newest and most consequential part of the governance model, and it is where the classification axis from 09.3 stops being a property of individual items and becomes a property of the places they live.

The idea to carry out: **a workspace is a clearance-bearing container, not a label**. It carries a mark, that mark is the high-water of everything inside it, and admission to the container is a floor test against that mark. Designs that treat a workspace as a filing convention — a string you match on — produce a system where the container advertises a ceiling it does not hold.

Two quantities, two algebras

This is the architectural point of the whole chapter, and getting it wrong inverts the control rather than weakening it.

quantity	question	algebra	direction
containment	what does this container hold?	<code>max()</code> — a high-water mark	monotonically non-decreasing
admission	may this principal be inside it?	a floor test, <code>>=</code>	binds at every mark above the bottom

A container's mark is raised by `max()` on every attach and is **never lowered by a detach**. Admission compares the principal's effective clearance against that mark and admits when the clearance reaches it — `>=`, not `>`, so an exact-match clearance is admitted, which is the common case.

❌ **Intersection is the right algebra for envelopes and the wrong one for classification.** 09.2 composes bounds by intersection because a delegate must never exceed its supervisor. Applying the same instinct here — taking `min()` over member clearances to decide a container's mark — is **declassification by invite**: a workspace holding `confidential` documents would relabel itself downward the moment a `public`-cleared member joined. The two quantities look similar enough that one function gets reused for both, and the symptom is a container whose mark falls as its membership grows.

The identity matters too. `max()` over the empty set is `public` — the bottom of the lattice — because an empty container holds nothing to protect. At that mark the floor test is a tautology: every clearance reaches `public`, so an empty workspace refuses nobody. That is correct, and it is why the mark rising is what makes the container start enforcing.

The artifact mark

An artifact carries a `classification` drawn from the same five-level EATP ladder as everything else in 09.3: `public`, `restricted`, `confidential`, `secret`, `top_secret`.

```
artifact = await client.artifacts.create(
    request_id=request_id,
    name="settlement-reconciliation.xlsx",
    content=content,
    classification="confidential",
)
```

`api:POST /api/v1/artifacts` creates one, `api:GET /api/v1/artifacts` lists, `api:GET /api/v1/artifacts/{id}` reads the metadata, `api:GET /api/v1/artifacts/{id}/download` retrieves the bytes, and `api:DELETE /api/v1/artifacts/{id}` removes one. Objective- and session-scoped listings are `api:GET /api/v1/objectives/{id}/artifacts` and `api:GET /api/v1/sessions/{id}/artifacts`, with `api:POST /api/v1/sessions/{id}/artifacts` for the session write path.

🚫 An unmarked artifact is **UNKNOWN**, and unknown denies at egress. It is not `public`. This is the opposite of the intuitive default and it is the one that makes the model sound. "We do not know what this is" must never resolve to "the least sensitive thing there is", because the population of unmarked material is exactly the population nobody has looked at. The mark is nullable precisely so that unknown is representable — folding it to `public` would mint a capability rather than record an absence.

There is a third state worth knowing about, because the two failure modes have different remedies. A mark can be **absent** — the artifact carries nothing — or **present but unmappable**, where a value is stored that no level recognises. Both deny, and they deny with distinct reasons, because the operator fix differs: mark the artifact, versus correct the value. A report that collapses them sends somebody to the wrong remedy.

Lineage is not versioning

Artifacts carry two different edges to other artifacts, and they answer two different questions. Confusing them is the single most consequential modelling error in this chapter.

field	question	used for
<code>supersedes_artifact_id</code>	which row did this replace?	versioning — the chain of revisions
<code>derivation_input_ids_json</code>	which classified material went into this?	lineage — the basis for the mark

`derivation_input_ids_json` is a JSON array of `{"kind": ..., "id": ...}` objects, where `kind` is `artifact` or `knowledge`:

```

derived = await client.artifacts.create(
    request_id=request_id,
    name="counterparty-summary.md",
    content=content,
    derivation_inputs=[
        {"kind": "knowledge", "id": policy_id},
        {"kind": "artifact", "id": position_extract_id},
    ],
)

```

Only lineage makes a derived mark defensible. A summary of a `secret` input is `secret` — and without the lineage edge there is nothing that makes that true except somebody remembering to say so. The versioning edge cannot stand in for it: a new version of an artifact is not a statement about what went into the artifact, and reading it as one produces a derived document whose mark reflects its predecessor and not its sources.

Version history is `api:GET /api/v1/artifacts/{id}/versions`, and a new version is created with `api:POST /api/v1/artifacts/{id}/supersede`.

The high-water rule

A derived artifact's classification is the **maximum rank** over three sets:

THE HIGH-WATER RULE

```

classification = max( rank ) over

    { the declared level, if one was declared }
    u { the predecessor's mark, if this supersedes something }
    u { the resolved level of EVERY derivation input }

```

WORKED: a ``secret`` position extract + a ``public`` published policy
declared as ``restricted``

```
max( restricted(1), secret(3), public(0) ) = secret(3)
```

The artifact is ``secret``. The declaration did not lower it;
declarations only ever contribute a candidate.

THE PREDECESSOR IS A FLOOR. A new version cannot fall below the mark of the version it replaces — a revision that dropped its predecessor's mark would be the laundry via the version chain: publish once at ``secret``, supersede at ``public``, and the content has been declassified by an edit.

Read it as: **every path that could produce a lower mark is closed.** Mixing a `secret` input with a `public` one yields `secret`. Declaring a lower level than the inputs justify yields the inputs' level. Superseding a marked artifact yields at least its predecessor's mark. There is no ordering of operations

that produces a downgrade.

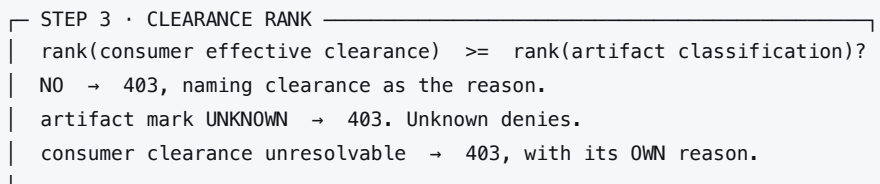
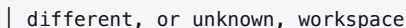
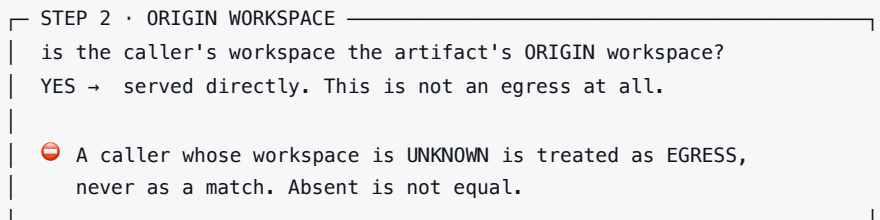
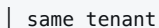
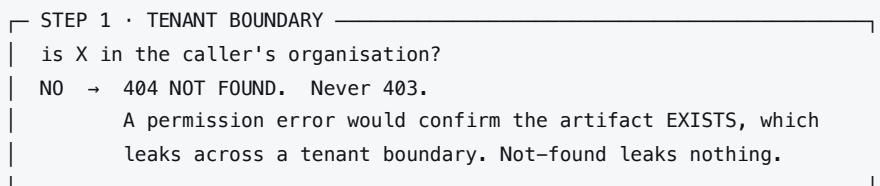
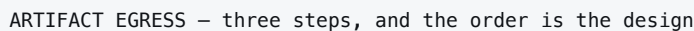
Two refusals fall out of the rule and both are deliberate:

An unresolvable input refuses the write. If a named derivation input cannot be resolved to a classification, the artifact is not written. The alternative — failing closed to `top_secret` — silently produces an artifact nobody can ever read, which presents as data loss rather than as a refusal. A loud error naming the input is the honest failure.

An unrecognised input kind refuses rather than being skipped. A skipped input contributes nothing to the maximum, which is a downgrade wearing the appearance of tolerance.

Egress — three steps, in order

A read of an artifact from outside its origin workspace is an **egress**, and the decision has three steps that must run in this order.



Step 2 is a **SHORTCUT**, not the gate. Step 3 is the gate.

Read it as: **equality is the shortcut; rank is the gate**. In-workspace reads are fast because they skip the comparison, not because the comparison would have passed. That has a direct consequence for how much the container's mark matters, and it is the reason the rest of this chapter exists: since an in-

workspace read is not clearance-checked, the thing standing between a `public`-cleared member and a `confidential` artifact is **whether they were admitted to the workspace at all**.

The two unresolved cases deny with *distinct* reasons — the artifact is unmarked versus the consumer's clearance could not be established — because the operator remedies are different and collapsing them into one "denied" makes the fix unguessable.

⚠ **A cross-tenant read returns not-found, and that is not an error message to improve.** The temptation when an integration hits a 404 it believes should be a 403 is to make the platform more helpful. Do not: the 404 is the control. A permission error on a cross-tenant read confirms the row exists, which is information the caller is not entitled to and which no amount of body redaction puts back.

The container mark, and what raises it

A workspace carries `contains_classification` — the high-water mark of every document and artifact attached to it.

operation	effect on the mark
attach a document or write an artifact	raised to <code>max(current, attached)</code>
attach something lower-classified	unchanged
detach anything	unchanged — a detach never lowers
re-derive, deliberately	recomputed, with an actor, a reason and an audit row

Raising is cheap; lowering is deliberate. Raising happens as a side effect of ordinary work and requires nothing of the caller. Lowering has exactly one path, it demands a named actor and a non-blank reason, and it writes an audit row. That asymmetry is the whole safety property: a declassification is an act somebody performs and answers for, not a consequence of a detach.

```
await client.workspaces.attach_document(  
    workspace_id=workspace_id,  
    knowledge_id=document_id,  
)  
documents = await client.workspaces.list_documents(workspace_id=workspace_id)  
await client.workspaces.detach_document(  
    workspace_id=workspace_id,  
    knowledge_id=document_id,  
)
```

Document attach, list and detach operate on the workspaces router: `api:POST /api/v1/workspaces/{id}/documents`, `api:GET /api/v1/workspaces/{id}/documents` and `api:DELETE /api/v1/workspaces/{id}/documents/{document_id}`.

❌ The listing is filtered by the CALLER's own clearance, because workspace membership is not a read grant. Being inside a container tells you the container's mark; it does not entitle you to every item in it. A member cleared to `restricted` in a workspace marked `confidential` sees the subset they are cleared for, and the absent rows are absent rather than redacted. The symptom of expecting otherwise is two members comparing screens and concluding the platform lost a document.

Admission — the floor test

Adding a member to a workspace is an admission decision, and it runs the floor test:

```
effective_clearance(principal) >= workspace.contains_classification
```

```
await client.workspaces.add_member(  
    workspace_id=workspace_id,  
    user_id=user_id,  
    role="contributor",  
)
```

`api:POST /api/v1/workspaces/{id}/members` adds one, `api:PATCH /api/v1/workspaces/{id}/members/{member_id}` amends the membership, and `api:DELETE /api/v1/workspaces/{id}/members/{member_id}` removes it.

Three properties of the test are worth stating precisely:

`>=`, not `>`. A `confidential`-cleared principal may join a workspace holding `confidential` content. Using `>` would deny the exact-match case, which is the common one, and would produce a model where a clearance can only ever admit you to material strictly below it.

Both principal kinds route through the same resolver. An agent's clearance is posture-capped (09.3); a human's is not, because humans have no posture and evaluating one through the agent formula collapses their ceiling. The admission surface takes the principal *kind* rather than a pre-resolved clearance, so the resolver owns that difference and three call sites cannot each get it slightly wrong.

An unestablished clearance is not a low one, and it denies. If the principal's effective clearance cannot be established at all, the admission is refused — with a different message from a clearance that was established and was too low, and the same outcome. Admitting on an unestablished clearance is the one direction a retry cannot undo.

The refusal message names the **workspace's mark** and deliberately does not name the refused principal's clearance level. That is not terseness: echoing the level would hand a caller a clearance-enumeration oracle over third parties — attempt an add per user against a marked workspace and read each level out of the refusal. The mark is a property of a row the caller can already read; the other party's clearance is not.

A container may only hold what it can enforce

A workspace carries a classification and no compartment. From 09.3, `secret` and `top_secret` require compartment-level need-to-know, and a rank comparison is not a need-to-know check.

So the workspace's writable vocabulary is `{public, restricted, confidential}`, and material above that is **refused at the door rather than approximated by rank**.

WHY REFUSING BEATS APPROXIMATING

Suppose a ``secret`` document is attached to a workspace, and the platform responds by raising the mark as far as the container can express – ``confidential`` – and logging the shortfall.

Admission now requires ``confidential``.

A ``confidential``-cleared principal is admitted.

They are **INSIDE** the workspace, so their read is not clearance-checked (egress step 2).

They read the ``secret`` document in place.

The breach happens. A line appears in a file.

The alternative – raising the mark to ``secret`` – is worse, not better: the mark would then **READ** as need-to-know isolation with none of the compartment membership behind it. A control that reads stronger than what runs is more dangerous than a narrower one that is honest.

Therefore: REFUSE. 422, naming the document and the mark that cannot express it.

This is the general principle and it applies well beyond workspaces: **do not offer a protection you cannot enforce**. The refusal is a smaller capability and an honest one, and it makes the gap visible at the moment somebody tries to rely on it rather than at the incident review.

Two properties that make the gate real

The container is never caller-chosen. The workspace an artifact belongs to is derived server-side from the request it was produced for — it is not a parameter of the create call. A caller who picks the container picks whose classification mark gets raised, which is a governance hole rather than a convenience. When the request carries no workspace, the write is refused rather than falling back to some other identifier; substituting a different kind of id into the workspace slot trades a refusal for an outage two layers down.

The gate runs before any side effect. The container capability check happens before the bytes reach storage and before the row is written, so a refusal leaves nothing behind. Ordering it the other way produces the worst artefact of all: a stored object whose row was refused, or a row whose bytes were never written — each of which looks like corruption rather than like a control.

⚠ **Build your own write paths the same way.** If your integration produces classified material, route it through the platform's artifact write rather than storing it yourself and registering it afterwards. The register-afterwards shape has a window in which the material exists and is unmarked, and that window is exactly the population `unknown` was designed to deny.

Explainable refusal

A refusal that says only "denied" is a refusal an operator will work around, because working around it is the only action available. Every refusal in this part names the rule that refused it, and the status code is the first half of that answer.

status	what it means here	remedy
403	egress denied — clearance rank, or an unknown mark	mark the artifact, or raise the reader's clearance
404	tenant mismatch, or genuinely absent	check the tenant; the two are deliberately identical
409	the object exists but is not in a state that permits this	activate, approve, or resolve the pending change
422	the sensitivity cannot be expressed by this container	use a container that can hold it, or reclassify

The reason strings carry the second half. An egress refusal distinguishes *the artifact is unmarked* from *your clearance could not be established* from *your clearance is below the mark*, because those are three different fixes performed by three different people.

For the broader question — *which gate would refuse, and where would the evaluation stop* — `api:POST /api/v1/governance/explain-access` evaluates a role against a knowledge item and returns the decision as a trace: allowed or not, the reason, the `step_reached`, and the access path taken.

```
trace = await client.governance_explain.explain_access(
    agent_id=agent_id,
    knowledge_item_id=document_id,
)
```

🚫 **It is a DIAGNOSTIC YOU RUN, never a lookup of what happened.** The call is a dry run over the subject you describe: nothing is read from any stored refusal, nothing is accessed, nothing is recorded, and a wrong or missing field in the description changes the verdict. It answers *what would the model decide about this role and this item* — not *why was this request refused*. For what actually happened, the audit spine (09.6) is the surface, and reading a dry run as a record of a past refusal is the mistake that sends a remediation after a cause the live request never hit.

Read `step_reached`, not `allowed`. `allowed` restates the outcome; the step name is the phase the evaluation stopped at, and it is the only field that narrows a refusal to a gate. Its companion `api:POST /api/v1/governance/describe-address` resolves a positional address in the organisation tree to the

objects that govern it, which is the read to reach for when the refusal named an ancestor you cannot place.

🚫 **A refusal that cannot be diagnosed is a refusal that gets disabled.** This is the operational argument for everything in this section, and it is not a nicety. The sequence is always the same: the control refuses, nobody can establish why, the work is urgent, somebody widens the bound that seems most likely — and now the control is gone *and* the original cause is still there. Name the rule in the refusal and the sequence does not start.

Designing with containment

Four pieces of guidance, in the order they matter.

Partition by sensitivity, not by project. A workspace's mark is the high-water of its contents, so a workspace mixing `public` onboarding material with one `confidential` analysis is marked `confidential` and every future member needs that clearance. The remedy is not to lower the mark; it is to have put the analysis somewhere else.

Expect the mark to ratchet, and design the membership for where it will end up. Marks rise as work proceeds and never fall on their own. A workspace provisioned with members cleared exactly to its current mark will start refusing additions the first time something more sensitive lands. Provision membership against the mark you expect, not the mark you have.

Declare lineage at the moment of creation. The derivation inputs are what make a derived mark defensible, and they are available at the moment the derived artifact is produced and reconstructible only with effort afterwards. An integration that creates artifacts without declaring inputs is producing material whose sensitivity has to be re-established by hand.

Re-derive deliberately, and rarely. The re-derivation path exists because a container's mark can legitimately become wrong — the document that raised it was detached and nothing else in the container justifies the level. Use it with a real reason written for somebody reading the audit row in a year, and treat a deployment that re-derives on a schedule as a deployment that has turned the one audited declassification path into an unattended one.

In the console: a workspace's mark appears on the workspace rather than on its documents, and a member add that the floor test refuses surfaces as a refusal with the mark named. The document list inside a workspace is the clearance-filtered one, so two members legitimately see different lists — which is worth saying out loud to operators, because the first report is invariably "documents are missing".

Summary

you need	the mechanism	the call
mark an artifact	classification at create	<code>api:POST /api/v1/artifacts</code>

you need	the mechanism	the call
make a derived mark defensible	derivation inputs	<code>derivation_inputs=[...] at create</code>
a new version that cannot declassify	supersede	<code>api:POST /api/v1/artifacts/{id}/supersede</code>
read the revision chain	versions	<code>api:GET /api/v1/artifacts/{id}/versions</code>
retrieve the bytes, clearance-checked	download	<code>api:GET /api/v1/artifacts/{id}/download</code>
put a document in a container	attach	<code>api:POST /api/v1/workspaces/{id}/documents</code>
see what is in a container, filtered	list documents	<code>api:GET /api/v1/workspaces/{id}/documents</code>
take one out without declassifying	detach	<code>api:DELETE /api/v1/workspaces/{id}/documents/{document_id}</code>
admit a principal to a container	member add, floor-tested	<code>api:POST /api/v1/workspaces/{id}/members</code>
find out which gate would refuse	access dry run	<code>api:POST /api/v1/governance/explain-access</code>
place an address in the tree	address description	<code>api:POST /api/v1/governance/describe-address</code>

Next: Part 10 — The organisation structure framework

Part 10 — The organisation structure framework

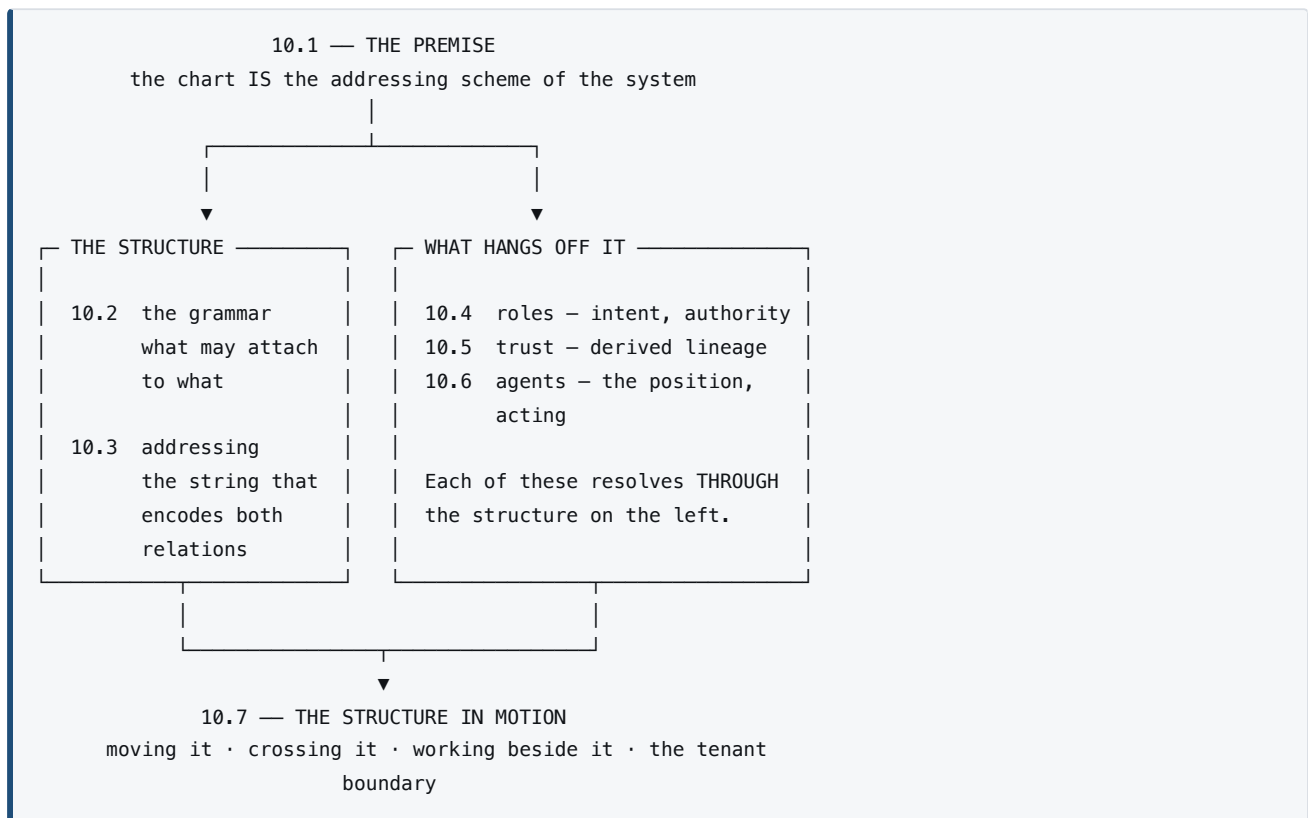
Audience: the architect designing the organisation their agents will live in. This part is PACT's organisational architecture as Aegis implements it — the grammar that constrains the structure, the addressing scheme derived from it, and the five things that resolve through it. Aegis is Integrum's commercial implementation of PACT, one of four open standards published by the Terrene Foundation under CC BY 4.0.

It assumes 06.3, which names the objects and says what each one constrains. This part answers the next question: **given those objects, how do you describe *your* organisation so the governance comes out the way you meant it?**

The chapters

#	Chapter	You leave able to
10.1	The organisation is the system	Say what a structural decision costs, and stop treating the chart as metadata
10.2	The D/T/R grammar	Build a structure the platform will accept, and know why each refusal is a saving
10.3	Addressing	Read an address, compute its algebra in your own code, and key on the right field
10.4	Roles, authority and intent	Fill the fields that actually decide behaviour, and stop using authority as a gate
10.5	Trust chains and delegation	Trace authority to a person, and withdraw it completely when you need to
10.6	Role agents	Provision agents by describing positions, and set autonomy through the right lever
10.7	Re-orgs, bridges and workspaces	Restructure without detaching governance, and cross a boundary deliberately

How the chapters relate



Read it as: 10.1 argues the premise, 10.2 and 10.3 are the structure itself, 10.4 through 10.6 are what the structure produces, and 10.7 is what happens once the organisation starts changing — which it will, before you finish provisioning it.

Three things follow from that shape.

10.2 and 10.3 are one idea in two forms. The grammar says a container attaches through its head role; the address is that rule rendered as a string. Reading either alone leaves the other looking arbitrary.

10.4 through 10.6 run in one direction. A role is described, its agent is derived from the description, and its authority is derived from its reporting edge. Nothing in that chain is configured independently, which is why a misbehaving agent is nearly always a role with an empty field.

10.7 is not an appendix. Every structure gets restructured, and the cascade a move triggers is the best test of whether the structure was described well. Read it before you provision, not after.

Where this part sits relative to the others

Part 02 is the working reference for standing an organisation up — the calls, in order, with the gotchas. Part 06 is the system-level picture: the planes, the objects, the request path. This part is neither. It is the **design framework**: the constraints that make a structure legal, the derivations that make it governable, and the failure modes that make a plausible structure the wrong one.

The practical test is the question you are asking. *"How do I create a unit and its head role?"* is part 02. *"Which of these objects carries authority and which is only context?"* is 06.3. *"Should this group be a team unit in the tree, or a flat Team beside it — and what do I lose either way?"* is part 10, and the answer changes what you can enforce for the lifetime of the deployment.

Two neighbouring parts sit either side of this one. Part 09 is the **governance argument** — this part says what a bound attaches to and how it is derived from the structure; part 09 says how bounds compose into a decision and what an auditor can be shown. Part 08 is the **inventory**: when you need the full set of organisational operations rather than the design reasoning behind them, 08.2 enumerates it.

One more routing note, because it is the question most often asked at the wrong address. *"Why did this agent get refused?"* is a request-path question — part 06 carries the layers and part 04 the error taxonomy. But *"why is this agent's bound narrower than the envelope I wrote?"* is a structure question, and it has a single answer: some ancestor on the reporting chain contributes a tighter bound. `api:POST /api/v1/governance/explain-envelope` returns that composition directly, and 10.3 is where to read about it.

Two ideas this part keeps returning to

Two relations, one structure. A role is contained by exactly one unit and reports to at most one role, and those are independent facts. Containment carries classification, posture ceilings and knowledge boundaries. Reporting carries envelopes and trust lineage. The address encodes both, which is why one string answers both kinds of question — and why assuming one relation implies the other is the most expensive mistake available here.

The most restrictive bound wins, everywhere. Envelopes intersect down the reporting chain and can never widen. Posture ceilings cascade as a minimum. A bridge crossing takes the tightest posture of the three parties. Workspace admission is a floor the joiner must clear. There is no path through this model along which a bound gets wider — so a design that needs one is a design that needs a different structure, not an exception.

Next: 10.1 — The organisation is the system

10.1 — The organisation is the system

This chapter is the premise the rest of the part rests on. It is not a tour of the objects — 06.3 does that, and this part assumes you have read it — but an argument about what kind of thing the organisation chart *is* on this platform, and what changes in your design once you accept it.

The claim is short and it is not a metaphor: **the organisation chart is not metadata about the system, it is the addressing scheme of the system.** Every governance decision the platform makes resolves through it. An envelope composes along one of its edges. A clearance attaches at one of its nodes. A trust chain is derived from one of its relations. A knowledge item is owned by one of its containers. There is no second structure the platform consults and no override table sitting beside the chart.

So describing your organisation is not configuration, it is the largest engineering decision you will make on this platform. A schema you got wrong produces queries that are awkward; a structure you got wrong produces governance that is wrong — bounds that resolve to the wrong ancestor, delegations that reach where you did not intend, knowledge that crosses a boundary you thought you had drawn. And it produces all of that silently, because a structure the platform can address is a structure it will happily operate.

What the organisation carries

Six distinct things hang off the structure, and none of them has a parallel mechanism. If the structure is wrong, each of them is wrong in a different way.

what resolves through the structure	the relation it uses	what goes wrong when the structure is wrong
operating envelopes	the reporting edge	a delegate is bounded by an ancestor you did not intend
trust chains	the reporting edge	authority traces to the wrong human
posture ceilings	the containment edge	an agent is permitted autonomy the unit never granted
knowledge boundaries	the containment edge	material is visible where it should not be
classification defaults	the containment edge	new material is born at the wrong sensitivity
addresses	both, encoded into one path	every one of the above resolves against the wrong node

Read the middle column carefully, because it is the structural fact that costs most: **two of these run on the reporting relation and three run on the containment relation, and those are different relations over the same objects.** A role belongs to exactly one unit and reports to exactly one other role, and those two facts are independent. The chart you draw on a whiteboard collapses them into one set of lines; the platform does not.

The practical consequence is that "this role sits under Finance" is an ambiguous sentence, and the ambiguity has governance weight. If it means *contained by Finance*, the role inherits Finance's classification default and posture ceiling. If it means *reports to the head of Finance*, the role's envelope composes down from that role's envelope and its trust chain descends from that role's agent. Usually you want both. Sometimes you deliberately want one — a security officer contained in Engineering but reporting to the General Counsel is a real and supported shape. **What you must not do is assume one implies the other.**

Why one address rather than two trees

A design that kept containment and reporting as two separate graphs would be simpler to write and impossible to reason about, because every governance question would require walking both and reconciling the answers. Aegis instead encodes both into a single traversable path — the **address** — so that one string answers *where in the structure is this, under whose authority, inside which boundary*.

ONE STRUCTURE, TWO RELATIONS, ONE ADDRESS

BOD	L0	governance root
└ D1 CEO Office	L1	a CONTAINER
└└ D1-R1 CEO	L2	the container's HEAD ROLE
└└└ D1-R1-R2 Staff Officer	L3	a role reporting in at CEO level
└└└ D1-R1-D1 CFO Office	L3	a container attached THROUGH the CEO
└└└└ D1-R1-D1-R1 CFO	L4	its head role
└└└ D1-R1-T1 Audit Committee	L3	a TEAM UNIT, attached through the CEO

The ``-R1-`` in the middle of every branch is not decoration. A child container attaches to its parent's HEAD ROLE, never to the parent container directly. That is why ``D1-R1-D1`` is expressible and ``D1-D1`` is not — the containment edge is forced to pass through a person.

Read it as: the address is a path down the containment tree, but every hop between two containers passes through the head role of the upper one. That single construction is what fuses the two relations. Containment is the sequence of **D** and **T** segments; authority is the sequence of **R** segments they are threaded through.

This is why the grammar in 10.2 is enforced rather than recommended, and why the addressing algebra in 10.3 works at all. An organisation that violated the grammar could not be given an address, and an organisation that cannot be given an address cannot have its governance computed.

Describing an organisation is a compile, not a write

You do not assemble the structure by writing rows and hoping they cohere. The platform takes a described structure, validates it, computes everything derived from it, and deploys the result — and each of those is a separate call you can run independently.

step	call	what it establishes
validate	<code>api:POST /api/v1/organization-builder/validate</code>	the structure is legal before anything lands
preview	<code>api:POST /api/v1/organization-builder/preview</code>	what the structure will look like once built
compile	<code>api:POST /api/v1/organization-builder/compile</code>	addresses, ceilings and envelopes are derived
deploy	<code>api:POST /api/v1/organization-builder/deploy</code>	the compiled structure becomes the live one
roll back	<code>api:POST /api/v1/organization-builder/rollback/{id}</code>	a deployment is reverted as a unit

Three of the derived artifacts have their own generation calls, which is worth knowing because it tells you what "derived" actually covers: `api:POST /api/v1/organization-builder/generate-agents`, `api:POST /api/v1/organization-builder/generate-constraints`, and `api:POST /api/v1/organization-builder/generate-trust-chains`. **Agents, constraint envelopes and trust chains are all outputs of the structure, not inputs to it.** You do not author a trust chain and then find a place for it in the chart; you describe the chart and the chains follow from it (10.5).

For a structure you already hold in a file, `api:POST /api/v1/organization-builder/import-yaml` takes it in one pass, and `api:GET /api/v1/organization-builder/templates` lists the shapes the deployment ships. `api:GET /api/v1/organization-builder/deployment-status` reports where a build got to.

The failure mode here is validating nothing and deploying anyway. A structure with a headless container is not rejected at write time — the container is real, its rows exist, and every listing shows it. What happens instead is that the compile cannot address it, so the container and everything beneath it drops out of the computed governance while remaining perfectly visible in the chart. The symptom is an agent that answers `404` to a governance question about a unit you can see, or an envelope explanation that stops higher up the tree than you expected. Run `api:GET /api/v1/organization-units/validate` before a deploy, and `api:GET /api/v1/governance/probe-corrupted-roles` after any bulk provisioning pass; both are seconds and both name the node.

The structure survives the people

A role is a **position**, not a person. It is created before anyone occupies it, it persists when the occupant leaves, and it is what every bound attaches to. The occupant is an attribute of the position, set by `api:POST /api/v1/organization-roles/{id}/assign-user` and cleared by `api:POST /api/v1/organization-roles/{id}/unassign-user`.

That split is what makes the structure durable under the thing organisations do most, which is change who is in which chair. A departure clears an assignment; it does not disturb an envelope, a clearance, a trust chain, an address, or a subtree. The governance you described stays described.

It also means an empty chair is an ordinary state rather than a broken one. **Roles are born vacant** — a container's head role is created with the container, before anyone is appointed — and the platform is built to operate that way. 10.4 develops what vacancy does and does not change; the short version is that it changes exactly one thing, and it is not the one most people expect.

The unit is three boundaries at once

A container is not only a container. Three fields on a unit cascade down the containment tree, and each makes the unit a different kind of boundary. You get all three whenever you create one, which is why "I just need somewhere to group these" is never quite what a unit does.

field	what it decides
<code>default_classification</code>	the sensitivity material created in this unit is born at, absent an explicit level
<code>max_trust_posture</code>	the autonomy ceiling for agents running as roles in this unit
<code>isolation_domain</code>	the hard-isolation plane the unit sits in

Creating a unit with the default classification is a decision, not the absence of one. Every knowledge item created inside it inherits that level, and the inheritance is silent. A unit stood up for a sensitive function at the lowest default produces correctly-classified-looking material at the wrong sensitivity for as long as nobody looks.

△ **Read the *effective* posture ceiling, never the declared one.** `api:GET /api/v1/organization-units/{id}/posture-ceiling` returns both, and a ceiling cascades — a child can never exceed its parent's effective value whatever its own field reads. Setting a permissive ceiling on a leaf beneath a restrictive branch changes nothing, and nothing tells you that. 10.6 covers the composition and the setter's sharp edge.

The isolation-domain registry is tenant-supplied: `api:GET /api/v1/organization-units/isolation-domains` reads what your deployment has configured and `api:PUT /api/v1/organization-units/isolation-domains` sets it. The setter **replaces the whole set rather than merging**, which is what makes de-registering a plane expressible at all — and also means a careless write silently removes every plane you did not re-list.

Standing a structure up

The sequence below is the shape of every provisioning script that works, and the ordering is not stylistic. Each step depends on the one before it: a child unit cannot attach until its parent has a head role, and a head role cannot be attached to a reporting chain that does not exist yet.

```

# 1. The tenant root.
org = await client.organizations.create(name="Northwind Industrials")

# 2. The root department. Its head role is created with it, vacant and primary.
ceo_office = await client.organization_units.create(
    name="CEO Office",
    unit_type="department",
    default_classification="confidential",
)
ceo_role_id = ceo_office["head_role"]["id"]

# 3. Give the head role its intent and its band. It is a chain ROOT – it has
#   no reporting edge, and that is correct at the top of the organisation.
await client.roles.update(
    role_id=ceo_role_id,
    title="Chief Executive Officer",
    authority_level=5,
    job_description="Accountable for the whole of Northwind's operations...",
)

# 4. Now a child department can attach – through the CEO role, per the grammar.
finance = await client.organization_units.create(
    name="Finance",
    unit_type="department",
    parent_unit_id=ceo_office["id"],
    default_classification="confidential",
)

# 5. Attach ITS head to the reporting chain. The platform does not infer this,
#   and skipping it is the defect that produces a detached authority island.
await client.roles.update(
    role_id=finance["head_role"]["id"],
    title="Chief Financial Officer",
    authority_level=5,
    reports_to_role_id=ceo_role_id,
)

```

The step people skip is 5, and skipping it fails nothing. The department exists, it is contained correctly, it has a head, it addresses correctly, and its head reports to nobody — so every envelope beneath it composes from a root rather than from the CEO, and every trust chain beneath it traces to a different origin than you intended. The check is one call: `api:GET /api/v1/organization-roles/{id}/reporting-chain` on a role you meant to nest, returning a single entry.

What the structure does not decide

It is as useful to know what the chart is *not* load-bearing for, because designing these into it produces containers that exist for no governance reason.

this is not decided by the structure	it is decided by
what a role may see	its clearance, and the item's classification
how autonomous an agent is	its posture, capped by the unit's ceiling

this is not decided by the structure	it is decided by
who works together on something	workspace membership — an orthogonal axis
who may cross to another part of the org	a bridge, explicitly authorised
what vocabulary your tenant uses	the ontology, set once at provisioning

The last is worth setting early. `api:GET /api/v1/ontology/presets` lists the available vocabularies, `api:POST /api/v1/ontology/preset/{id}` applies one, and `sdk:aegis_sdk.OntologyModule` wraps them. It is nearly free at provisioning time and disruptive once people have learned the old words — which makes it a decision worth making deliberately rather than by never making it.

Four consequences for your design

Each of these is a decision you will otherwise make by accident.

Model your real accountability, not your product taxonomy. The structure is the authority graph, so a node in it should correspond to something a person is answerable for. Team units named after subsystems, containers created to group documents, and roles minted to hold a permission all produce a chart that reads sensibly and governs strangely — because the platform will faithfully compose envelopes along edges you drew for filing reasons.

Draw the containment boundary where you want a knowledge boundary. A unit is simultaneously an organisational container, a classification default and a visibility boundary, and you get all three or none. If you want material fenced, you want a unit; if you only want people grouped, see 10.7, which covers the constructs that group without fencing.

Decide the reporting edge deliberately, including where it is absent. A role with no reporting edge is a **chain root** — it anchors its own lineage instead of inheriting one. For the top of the organisation that is correct. Everywhere else it is an orphan, and the two are indistinguishable in every listing. `api:GET /api/v1/organization-roles/{id}/reporting-chain` returning a single entry is the check: on a role you meant to nest, that single entry is the finding.

Expect to restructure, and design so restructuring is survivable. Re-parenting a unit recomputes addresses, revalidates knowledge-share policies, re-derives bridges and rebuilds trust chains beneath it. That is a governance event with a cascade, not a chart edit, and 10.7 is about making it one you can absorb. Structures that anchor on roles rather than on containers survive it best, which is why bridges anchor on roles.

The idea, once

the conventional reading	what is actually true here
the org chart describes the system	the org chart is the addressing scheme of the system
roles are labels on people	roles are positions ; people are an attribute of them

the conventional reading	what is actually true here
"under Finance" is one fact	containment and reporting are two relations, independently set
trust is configured	trust is derived from the reporting relation
restructuring is a chart edit	restructuring is a cascade across five derived artifacts
an empty seat is a gap	an empty seat is an ordinary state the platform operates through

Next: 10.2 — The D/T/R grammar

10.2 — The D/T/R grammar

The organisation is one tree with three node types, and there is exactly one structural rule governing how they attach. This chapter is that rule: what the three types are, what the rule says, why it is enforced at write time rather than checked later, and what the structures it forbids would have cost you.

The rule is worth stating before anything else, because everything in the rest of this part is downstream of it:

A containment node — a Department or a Team — must be immediately followed by exactly one Role, its head, before any further Department, Team or Role can attach beneath it.

The one-sentence rationale is the one to carry: **containers do not make decisions, people do**. Every department has a head. Every team unit has a lead. Every budget, every delegation and every escalation terminates at a person who is accountable for it. The grammar is that principle made structural — it is not possible to express an organisation on this platform in which authority passes through a container without passing through a person.

The single idea to carry out of this chapter: the grammar is not a style guide, it is the precondition for your organisation being addressable at all.

The three node types

type	symbol	what it is	persistence
Department	D	a knowledge container and organisational division	persistent
Team	T	a knowledge container and working group	fluid; may be temporary
Role	R	a person's position and accountability anchor	the position persists; the occupant changes

D and T are both **units**, distinguished by the `unit_type` field, whose only two values are `department` and `team`. They are structurally interchangeable in the grammar — everything the rule says about D it says about T — and they differ in intent and in one placement restriction covered below. Both are created through `api:POST /api/v1/organization-units` and read through `api:GET /api/v1/organization-units/{id}`; `sdk:aegis_sdk.OrganizationUnitsModule` is the typed path.

🚫 **"Team" names two unrelated things on this platform, and only one of them is in this tree.** A **team unit** is the **T** node above: a container with an address, a head role, a classification default and a posture ceiling. A **Team** — the flat entity at `api:GET /api/v1/teams`, with its own identifier and a membership list — is not in the D/T/R tree at all, has no address, no head role and no boundary semantics. They are separate constructs that share a word. **This part says "team unit" whenever it means the T node**, and names the other one explicitly. 10.7 covers when you want which; the short version is that only the team unit fences anything.

R is created through `api:POST /api/v1/organization-roles`, with the fuller surface on `sdk:aegis_sdk.modules.RolesModule`.

⚠️ **unit_type has exactly two values, and there is no executive**. The root of the organisation is an ordinary department — conventionally the CEO Office, addressed **D1**. An earlier generation of this model had a third type for the top of the chart; it is gone, and the root department replaces it. A structure that tries to express "this is the executive layer" as a type is expressing it in the wrong place: seniority is a property of roles (10.4), not a kind of container.

🚫 **sdk:aegis_sdk.UnitType is not the organisational unit type.** That enum classifies a *work* unit — a different domain that happens to share the word. The organisational unit type is the string `department` or `team` on an organisation unit. Two meanings, one word, and the enum name points at the one you probably did not mean.

The rule, stated as a grammar

Eight attachment sequences are legal and four are not. The four illegal ones are exactly the cases where a container attaches directly to a container.

VALID	INVALID
D-R a department and its head	D-D department under department
D-R-D a department beneath that head	D-T team unit under department
D-R-T a team unit beneath that head	T-T team unit under team unit
D-R-R another role at that level	T-D department under team unit
T-R a team unit and its lead	
T-R-T a team unit beneath that lead	
T-R-D a department beneath that lead	
T-R-R another role at that level	

Read the valid column as one pattern: every container is followed by its head role, and everything else attaches to the HEAD ROLE — not to the container. The invalid column is the same pattern with the person removed.

Read it as: **R** is unrestricted in what may follow it — any combination of **D**, **T** and **R**. **D** and **T** are restricted to exactly one thing: their head role. The asymmetry is the whole rule.

Two corollaries fall straight out and are worth having explicitly.

Exactly one primary role per unit. The head role is the one carrying `is_primary_for_unit = true`. A unit with two would have two candidate attachment points for its children and an ambiguous address; a unit with none cannot attach children at all. The platform refuses both: a second primary on a unit that already has one is rejected, and so is unsetting the only primary on a unit that would be left headless.

A team unit may not be the organisation's root. `D` and `T` are otherwise interchangeable, and this is the exception. The root of the tree is a department; a team unit with no parent is refused. A team unit is a working group inside an organisation, and an organisation whose top node is a working group has no durable container for the classification defaults and posture ceilings that cascade from the root.

Enforcement is at write time, and it is a refusal

The grammar is checked when you write, not when you compile. Four things are refused, and each refusal names the node and what to do about it.

you attempt	refused because
a <code>unit_type</code> other than <code>department</code> or <code>team</code>	the grammar has two container types
a team unit with no parent	the organisation's root is a department
any unit beneath a team unit that has no head role	the child would have no attachment point
a unit beneath any parent that has no primary role	the <code>D/T-R</code> invariant — create the parent's head first
a second primary role on a unit that already has one	the attachment point must be unambiguous
unsetting the only primary role on a unit with children	it would leave the subtree unattachable

The last two are the ones that catch provisioning scripts. A script that creates units in one pass and roles in a second will hit the head-role refusal on its very first child unit, which is the good outcome — the refusal arrives at the write that would have produced the broken structure, not three steps later.

Creating a unit auto-creates its head role, which is what makes the grammar liveable rather than tedious. You create the department; the platform creates the role that heads it, marks it primary, and leaves it vacant with its vacancy recorded as a structural placeholder. The seat exists, it is addressable, and its agent is active from that moment (10.6) — it simply has no human in it yet.

⚠ **The auto-created head role arrives with no reporting edge.** It is contained correctly and it is attached to nothing. That is the one thing the grammar does *not* do for you, and it is the most common structural defect in a freshly-provisioned organisation: a set of units whose subtrees are each a separate authority structure, unattached from each other and from the top. No call fails and nothing warns you. Set `reports_to_role_id` on every head role you create, and verify with `api:GET /api/v1/organization-roles/{id}/reporting-chain` — a chain that comes back with a single entry is a root, and on a role you meant to nest, that is the finding.

```
# Create the container; its head role is created with it, vacant and primary.
unit = await client.organization_units.create(
    name="Treasury",
    unit_type="department",
    parent_unit_id=finance_unit_id,
    default_classification="confidential",
)

# Attach the head to the reporting chain – the platform does not infer this.
await client.roles.update(
    role_id=unit["head_role"]["id"],
    reports_to_role_id=cfo_role_id,
    authority_level=3,
)
```

What the forbidden structures would have cost

D–D looks harmless. It is the shape every filing system has, and the instinct to reach for it is strong — you want a "Finance" container with "Treasury" and "Payables" inside it and no particular person in between. The grammar refuses it, and the three things that refusal buys are each load-bearing.

Authority would have no anchor. An envelope is defined by a role *for* a role, and a trust chain runs from an agent *to* an agent (10.5). Under **D–D**, a delegation crossing from Finance to Treasury would have no role at the boundary to originate from — so either the platform invents one, or authority crosses the boundary unattributed. Both are worse than a refusal.

Escalation would have a hole in it. When a decision exceeds a delegate's bounds it escalates to the superior. Under **D–D** the superior of everything in Treasury is a *container*, which cannot approve anything, so escalation would have to skip a level — silently widening who sees the decision.

The address would be ambiguous. **D1–D1** cannot be parsed as a position in the authority graph, because the graph is defined by the role segments. An organisation that could produce such an address could not have its governance computed (10.3), and the failure would land at compile time, far from the write that caused it.

 **The refusal is the cheap outcome; the expensive one is the structure that gets built around it.**

A team unit that hits the headless-parent refusal and "resolves" it by re-parenting the child somewhere it does fit has not satisfied the grammar, it has moved the work under an authority that was never meant to hold it — and every envelope and trust chain beneath will compose from that wrong ancestor, correctly, forever. When a write is refused, create the missing head role. Do not find a different parent.

Three role flags that change what a role is for

Beyond `is_primary_for_unit`, two flags alter a role's participation in the structure. Neither is cosmetic.

flag	what it marks	structural effect
<code>is_primary_for_unit</code>	the unit's head role	the attachment point for the unit's children; exactly one per unit
<code>is_external</code>	a board member, advisor, outside counsel	governance participation only; receives no operational delegate agent
<code>is_coordinator</code>	a cross-team coordinator	a non-primary role bound to a bridge-only, zero-financial envelope

`is_external` is how a board is expressed. A board is an **L0 governance root** — it sits above the organisation's root department, its member roles carry `is_external = true`, and it is the signing authority for the organisation-level envelope. External roles can hold governance agents that approve, observe and audit; they do not get operational delegate agents, and the platform enforces that rather than trusting you to remember it. **The failure mode a flag prevents here is a board seat that can execute work** — which reads, in every listing, like an ordinary senior role.

`is_coordinator` is the construct for a person who works across team units without reporting into each of them. It is deliberately non-primary: a coordinator is not the head of anything, and making one primary would put a cross-cutting role at a structural attachment point. Its envelope is bridge-scoped with no financial dimension — it can traverse, it cannot spend.

Department or team unit — choosing between the two containers

The grammar treats `D` and `T` identically, so the choice between them is a modelling decision rather than a structural one. Three differences guide it.

axis	Department	Team unit
persistence	durable — outlives the work it was created for	fluid — may be created for an effort and retired
head role	"Head of ..."	"Lead of ..."
placement	may be the organisation's root	may not be the root

Both are knowledge boundaries, both carry a classification default and a posture ceiling, and both cascade those to what they contain. **So the choice is about lifespan and about how the organisation talks about itself**, not about what the container enforces.

The practical rule: reach for a department when the boundary should outlive its current membership and its current purpose, and for a team unit when it should not. A "Q3 Migration" team unit that is archived when the migration completes is the construct behaving as intended; a "Q3 Migration" department is a boundary that will still be there in three years, still carrying a classification default, still cascading a ceiling, with nobody remembering why.

⚠ **Neither is the construct for "these people work together but nothing is fenced"**. Both fence. 10.7 covers the constructs that group without fencing — and it also covers the vocabulary collision waiting here, because a **team unit** in the grammar and a **team** on the teams surface are two different things

sharing one word.

Provisioning a structure without fighting the grammar

A script that creates all the units first and all the roles second will be refused on its first child unit, because the parent has no head role yet. The order that works is depth-first, and each level completes before the next begins.

THE ORDER THAT SATISFIES THE GRAMMAR

for each unit, top-down:

1. create the unit → head role auto-created, vacant, primary
2. set the head role's intent → title, authority_level, job_description
3. attach the head role → reports_to_role_id = parent's head role
4. create any non-primary roles in this unit
5. recurse into this unit's children

Step 3 is the one with no automatic counterpart. Steps 1 and 2 concern CONTAINMENT and the platform helps with both; step 3 concerns REPORTING and the platform will not infer it from the containment you just built.

Read it as: the grammar guarantees you a head role, and guarantees it is in the right container. It guarantees nothing about who that head answers to, because that is the other relation (10.1) and it is genuinely a separate decision — a role contained in one unit and reporting into another is a legitimate shape the platform will not second-guess.

For a structure you already hold, `api:POST /api/v1/organization-builder/import-yaml` takes the whole thing in one pass and does the ordering for you, and `api:POST /api/v1/organization-builder/validate` will tell you whether it is legal before anything lands.

Checking a structure you did not build

Three reads answer three different questions, and they are not substitutes for each other.

call	answers
<code>api:GET /api/v1/organization-units/validate</code>	is the unit hierarchy structurally sound — orphans, cycles?
<code>api:POST /api/v1/organization-builder/validate</code>	is a proposed structure legal before it is built?
<code>api:GET /api/v1/governance/probe-corrupted-roles</code>	which roles violate the grammar — no address, head not marked primary?

`api:GET /api/v1/organization-units/{id}/summary` reports `has_primary_role`, which tells you a head exists. **It does not tell you the head is attached** — that is the reporting-chain read above, and conflating the two is how a disconnected subtree passes a review. A unit can have a head role, satisfy every grammar check, and sit in its own authority island.

For the shape of what you have, `api:GET /api/v1/organization-units/tree` returns the whole containment tree, `api:GET /api/v1/organization-units/roots` the top nodes, and `api:GET /api/v1/admin/roles/hierarchy` the reporting picture as a whole.

```
# Before a deploy: does the structure hold together at all?
curl -s "$AEGIS/api/v1/organization-units/validate"      # orphans + circular refs
curl -s "$AEGIS/api/v1/governance/probe-corrupted-roles" # grammar violations on roles

# After provisioning: is every head role actually attached?
curl -s "$AEGIS/api/v1/organization-roles/$ROLE/reporting-chain" # length 1 == a root
```

The grammar, once

question	answer
What may follow a <code>D</code> or a <code>T</code> ?	Exactly one <code>R</code> — its head role. Nothing else.
What may follow an <code>R</code> ?	Any combination of <code>D</code> , <code>T</code> and <code>R</code> .
How many primary roles per unit?	Exactly one — enforced on create, on update, and on move.
Can a team unit be the root?	No. The root is a department.
Where does a child unit attach?	To its parent's head role , never to the parent container.
Who creates the head role?	The platform, when the unit is created — vacant, primary, and active.
What does the platform <i>not</i> do for you?	Attach that head role to a reporting chain.
When is the grammar checked?	At write time. The refusal arrives at the write that would break it.

Next: 10.3 — Addressing

10.3 — Addressing

Every unit and every role carries an `address` — a compact string naming its position in the structure. This chapter is the address in full: how it is composed, what the composition guarantees, the algebra you can compute over it in your own code, and the one consistency rule that makes all of it mean something.

The address is worth understanding rather than treating as an opaque key, for two reasons. It is **the identifier that survives renames** — titles get edited, addresses get recomputed from structure — and it is **the input to the platform's own governance explanations**, which take an address and tell you how a bound or an access decision was reached. An integration that keys on titles breaks when someone edits a title; an integration that keys on addresses breaks only when the structure actually changes, which is the event you wanted to know about.

The single idea to carry out: an address encodes containment and authority in one path, so the string alone answers questions that would otherwise require walking the tree.

The segment format

An address is a sequence of typed, numbered **segments** joined with `-`. Each segment is one letter from `D`, `T`, `R` followed by an integer.

```
D1-R1-D1-R1-T1-R2
| | | | |
| | | | └─ R2 the second role in that team unit – not its lead
| | | └─── T1 the first team unit beneath the CF0
| | └───── R1 the CF0 – the head role of that department
| └────── D1 the first department beneath the CEO
└──────── R1 the CEO – the head role of the root department
          D1 the root department
```

Read right to left to answer "who is this?", left to right to answer "under whose authority?". Both readings are available because both relations are in the same string.

Read it as: the `D` and `T` segments are the containment path; the `R` segments are the authority path threaded through it. `D1-R1-D1` means *the first department beneath the head role of the root department* — and the `-R1-` in the middle is the structural expression of 10.2's grammar. It is why `D1-D1` is not merely discouraged but unrepresentable.

How the numbers are assigned

The numbering is deterministic, which matters because it means the same structure always produces the same addresses. Four rules produce every address in the tree.

position	rule
root units	<code>D</code> and <code>T</code> are counted by separate counters , ordered by unit name

position	rule
roles within a unit	the primary role is always R1 ; the rest follow, ordered by title
child units	attach as <code><parent unit address>-R1-<D T><n></code> , ordered by unit name
child unit numbering	again separate D and T counters , scoped to that parent

Three consequences fall out of this that are easy to get wrong.

D and T numbering are independent. A unit containing one department and one team unit produces `...-R1-D1` and `...-R1-T1` , not `D1` and `T2` . Counting the children to predict a number will be wrong whenever both types are present.

R1 is the head role, always, by construction rather than by convention. It is not "the first role created" and not "the most senior"; it is the role carrying `is_primary_for_unit` . Non-primary roles in the same unit are `R2` , `R3` , ..., ordered by title. So `D1-R1-R2` is a second role reporting in at CEO level — a staff officer, a chief of staff — and it is *not* the head of anything.

Child units attach to `-R1-` , never to the container. This is the mechanism described in 10.2 rendered as a string operation. It is also why a unit with no primary role is structurally incomplete: there is no `-R1-` to attach through, so the unit and everything beneath it drops out of address computation entirely.

⚠ **A headless unit does not produce a bad address — it produces no address, and its whole subtree produces none either.** The unit stays visible in every listing, its rows are real, and it simply has no position in the governance graph. The symptom is a governance explanation that stops higher up the tree than you expected, or a unit that appears in `api:GET /api/v1/organization-units/tree` and resolves in no envelope composition. `api:GET /api/v1/organization-units/validate` names it; `api:GET /api/v1/governance/probe-corrupted-roles` names roles carrying a null address for the same reason.

Two bounds on the structure

The structure is bounded in depth and it is cycle-checked, and both are refusals rather than warnings.

Maximum depth is 50 segments. That is an extremely deep organisation — a fifty-segment address is twenty-five levels of container, each with its head role — and hitting it in a structure you intended almost always means something is wrong upstream. Treat it as a tripwire, not as a budget.

Cycles are detected during traversal. A unit whose parent chain returns to itself is refused with the cycle named. This is reachable by a re-parent operation that would make a unit its own ancestor, and it is refused at the move rather than discovered later.

The algebra you can compute yourself

An address is a string with an algebra, and the whole algebra is derivable from the string. You do not need a round trip to answer any of these — which matters when you are filtering a page of results client-side, or deciding whether to make a call at all.

question	operation on the address	example
What kind of thing is this?	read the last segment's letter	<code>...-T1</code> → a unit · <code>...-R2</code> → a role
How deep is it?	count the segments	<code>D1-R1-T1-R2</code> → 4
What is its parent?	drop the last segment	<code>D1-R1-T1-R2</code> → <code>D1-R1-T1</code>
What is its ancestry?	every prefix, root-first, inclusive of itself	<code>D1-R1-T1</code> → <code>D1</code> , <code>D1-R1</code> , <code>D1-R1-T1</code>
Is A beneath B?	A equals B, or A starts with <code>B</code> + <code>-</code>	<code>D1-R1-T1</code> is beneath <code>D1-R1</code> — and beneath itself
Which department is it in?	truncate at the first <code>T</code> segment	<code>D1-R1-D2-R1-T1-R2</code> → <code>D1-R1-D2-R1</code>
Which team unit is it in?	take from the last <code>T</code> segment onward	<code>D1-R1-D2-R1-T1-R2</code> → <code>T1-R2</code>
Are two roles in the same team unit?	strip each trailing <code>R</code> segment and compare the remainder	<code>D1-R1-T1-R1</code> and <code>D1-R1-T1-R2</code> → yes
What is their common ancestor?	the longest shared prefix, cut at a segment boundary	<code>D1-R1-D1-R1</code> and <code>D1-R1-T1</code> → <code>D1-R1</code>

Two of those have a sharp edge worth stating outright, because they are the two most likely to be implemented slightly wrong and then behave correctly in testing.

🚫 **The descendant test is descendant-or-equal, and ancestry is inclusive.** A node is beneath itself and appears in its own ancestry list. That is deliberate — it is what makes "everything at or below this address" a single test rather than a union of two — and it is the opposite of a strict prefix check. An implementation that excludes the node itself will silently drop the node you were asking about from every subtree operation, which looks like an off-by-one and is really a semantics error.

The prefix test also needs the trailing `-`. Comparing `"D1-R1-T1".startsWith("D1-R1")` without it means `D1-R12` tests as a descendant of `D1-R1`, which it is not. Split on segment boundaries or include the separator; do not compare raw prefixes.

```
# Filter a page of roles to one subtree, client-side, with no extra call.
def is_at_or_below(address: str, root: str) -> bool:
    return address == root or address.startswith(root + "-")

roles = await client.roles.list(limit=200)
under_finance = [
    role for role in roles["items"]
    if role.get("address") and is_at_or_below(role["address"], "D1-R1-D2-R1")
]
```


What the platform computes for you

Three calls take an address and return something you could not derive from the string, because they read the live structure rather than parsing a path.

call	takes	returns
<code>api:POST /api/v1/governance/describe-address</code>	an address	the position in plain language — what this address names
<code>api:POST /api/v1/governance/explain-envelope</code>	an address	how that role's effective envelope was composed down its ancestors
<code>api:POST /api/v1/governance/explain-access</code>	a subject and an item	which step of the access evaluation refused, and the path taken

`describe-address` is the right way to learn the grammar against your own deployment rather than against this page — pass an address you hold and read what comes back.

`explain-envelope` is the single most useful call in this part. When a bound is not what you expected, the question is almost always "which ancestor contributed this?", and reconstructing the answer by hand means walking the reporting chain and intersecting envelopes at every hop. The explainer returns the composition. Reach for it before you reach for the tree reads.

`explain-access` does the same job for a read that was refused. A clearance failure, a classification failure and a missing share policy all present identically from outside — the call tells you which of the three refused you instead of leaving you to guess (10.7).

```
# A delegate is bounded more tightly than its own envelope says. Ask why.
explanation = await client.governance.explain_envelope(
    address="D1-R1-D2-R1-T1-R2",
)
# The response carries the composition, ancestor by ancestor — read it
# instead of reconstructing the intersection yourself.
```

Walking the structure through the API

When you need the live tree rather than the string, the reads split by relation — and the split is the one 10.1 insists on.

relation	reads
containment	<code>api:GET /api/v1/organization-units/tree</code> · <code>/roots</code> · <code>/id/children</code> · <code>/id/ancestors</code> · <code>/id/descendants</code>

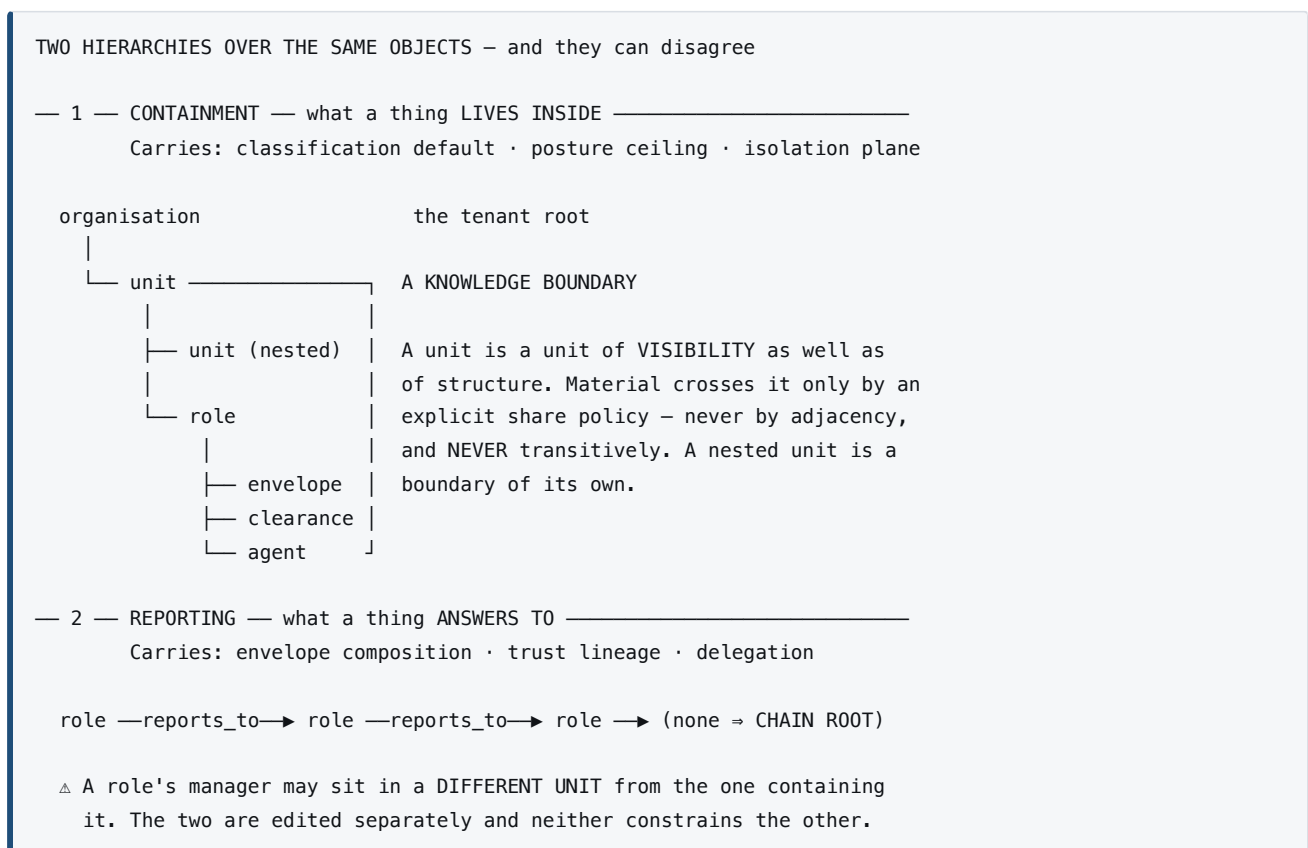
relation	reads
reporting	api:GET /api/v1/organization-roles/{id}/reporting-chain · /{id}/direct-reports · api:GET /api/v1/admin/roles/hierarchy

🚫 **Never walk containment to answer a question about authority.** The ancestors of a *unit* are its containers; the ancestors of a *role's authority* are the roles it reports to, and those are different sequences over the same tree. A delegation check, an envelope composition, an escalation target and a trust lineage all run on the reporting relation. Using `api:GET /api/v1/organization-units/{id}/ancestors` for any of them produces an answer that is well-formed, plausible, and about the wrong relation — which is the worst combination, because nothing about the response looks wrong.

The address is what lets you avoid the choice in the common case: its `R` segments *are* the authority path, so parsing the address answers authority questions without a walk at all.

Two hierarchies, drawn over the same objects

The reason the address has to encode both relations is that the two hierarchies are genuinely different shapes, and they can disagree. This is the structural fact most likely to cost you a day, so it is worth seeing drawn.



Read the boundary in panel 1 as a **region**, not as another edge — visibility is not the containment relation. A unit's subtree does not become readable because it is nested inside something readable, and it does not become readable because a sibling is. Then note that panel 2 shares objects with panel

1 but not its shape: a role has exactly one containing unit and exactly one manager, and **those two facts are independent**.

The address is what reconciles them into one identifier. Its **D** and **T** segments are panel 1; its **R** segments are panel 2. That is why a single string can answer both kinds of question — and why reaching for the wrong one of the two read families above returns a confident answer about the relation you did not ask about.

Using the algebra: three things it is actually for

The operations in the table above are not trivia. Three common jobs reduce to them, and each avoids a round trip or a wrong answer.

Scoping a query to a subtree. Anything at or below an address is a single prefix test, so a page of results can be filtered client-side without a second call. This is the descendant-or-equal semantics earning its keep: "this unit and everything under it" is one predicate, not a union.

Deciding whether a crossing is even needed. Two roles in the same team unit do not need a bridge to collaborate; two roles in different departments may. The same-team-unit test answers that from two strings, before you go looking for a bridge that might not exist (10.7).

Finding where two parts of the organisation meet. The lowest common ancestor of two addresses is the node whose authority covers both — which is the role that can legitimately decide something spanning them, and therefore the right escalation target for a dispute between them.

```
# Where do these two positions meet, and who can decide across them?
def lowest_common_ancestor(a: str, b: str) -> str | None:
    seg_a, seg_b = a.split("-"), b.split("-")
    shared = []
    for x, y in zip(seg_a, seg_b):
        if x != y:
            break
        shared.append(x)
    return "-".join(shared) or None

# "D1-R1-D1-R1" (CFO) and "D1-R1-D2-R1" (CTO) meet at "D1-R1" — the CEO.
meeting_point = lowest_common_ancestor(cfo_address, cto_address)
```

△ **Cut at segment boundaries, not at characters.** A naive longest-common-string over **D1-R1-D1** and **D1-R1-D12** returns **D1-R1-D1**, which is a real address and the wrong answer. Split first, compare segment by segment, and join back.

The consistency rule

One invariant ties the address to the rows it is derived from:

A stored address must agree with the structure it came from — with **parent_unit_id** on a unit, and with both **organization_unit_id** and **reports_to_role_id** on a role.

The platform maintains this by **always recomputing**, never by patching. Sequential numbering at every level means a single move can change addresses anywhere in the tree — re-parenting a unit can renumber its former siblings — so there is no such thing as a safe partial recomputation. A structural change triggers a full recompute for the organisation, and a compile that cannot address every representable unit is a structural error rather than a quiet omission.

Two practical consequences for your integration:

Addresses are stable under renames and unstable under moves. Editing a unit's name or a role's title does not change any address. Moving a unit changes its address, every address beneath it, and potentially its former siblings' addresses too. If you cache addresses, invalidate on move (10.7); renames are free.

Do not construct an address by hand and expect it to resolve. You can *parse* an address, and the algebra above is yours to compute. You cannot *mint* one — the numbering is assigned by the structure, and a string of the right shape naming a position that does not exist is simply a string. Read addresses from the objects that carry them.

⚠ **A unit's `path` field is not its address.** Units also carry a materialised name path — a slug trail like `/ceo/cfo/finance` — used for fast lookups by name. It contains no `R` segments, knows nothing about roles, and expresses containment only. It is a convenience for human-facing search. **The address is the governance identifier; the path is not**, and keying authorisation logic on the path reads containment while claiming to read authority.

Addressing, once

property	answer
Segment shape	<code>D</code> , <code>T</code> or <code>R</code> followed by an integer; joined with <code>-</code>
Head role	always <code>R1</code> in its unit
Non-primary roles	<code>R2</code> , <code>R3</code> , ... ordered by title
Child unit attachment	<code><parent unit address>-R1-<D T><n></code>
<code>D</code> and <code>T</code> numbering	separate counters, at every level
Depth ceiling	50 segments
Descendant test	descendant- or-equal ; requires the trailing <code>-</code>
Ancestry	every prefix, root-first, inclusive
Stable under	renames, title edits, occupant changes
Unstable under	moves — recomputed for the organisation, not patched in place
Derived, not minted	read it from the object; constructing one by hand names nothing
Not the address	the unit <code>path</code> field — containment only, no roles, no authority

Next: 10.4 — Roles, authority and intent

10.4 — Roles, authority and intent

A role is the node almost everything attaches to. It belongs to exactly one unit, it reports to at most one other role, it is what a human is assigned to and what an agent acts as, and it is what an envelope constrains and a clearance is granted on. This chapter is the role in full: what it holds, what its job description actually *does*, what its authority level does and does not gate, and why an empty one is an ordinary state rather than a gap.

Three things in this chapter are commonly misread in the same direction — as weaker than they are, or as stronger. The job description looks like documentation and is executable intent. The authority level looks like a permission and is not one. Vacancy looks like an outage and changes exactly one thing.

The single idea to carry out: a role is a position the organisation holds open, and everything the platform knows about what that position is *for* lives in fields on the role, not in the person occupying it.

The role, field by field

`api:POST /api/v1/organization-roles` creates one; `api:PUT /api/v1/organization-roles/{id}` updates it; `sdk:aegis_sdk.modules.RolesModule` is the typed surface. The fields divide cleanly into four groups, and the groups behave very differently.

field	what it actually does
structure	
<code>organization_unit_id</code>	the unit it belongs to — structural, required, exactly one
<code>reports_to_role_id</code>	the reporting edge; absent makes it a chain root
<code>is_primary_for_unit</code>	marks the unit's head role — exactly one per unit
<code>address</code>	its position, derived — see 10.3
<code>direct_reports_count</code>	how many roles report to it
intent	
<code>title</code>	the position's name — this field is <code>title</code> , not <code>name</code>
<code>description</code>	a short human-facing summary
<code>job_description</code>	the primary content of the role agent's system prompt — see below
<code>responsibilities_json</code>	what this position is answerable for; carried into the prompt in full
<code>approval_authority_json</code>	what this position may approve, and up to what limits
<code>required_skills_json</code>	the skills the position calls for
<code>required_capabilities_json</code>	the capabilities the position calls for
authority and bounds	

field	what it actually does
<code>authority_level</code>	1 (Staff) ... 5 (C-Suite) — seniority, not an access gate
<code>constraint_template_id</code>	the constraint template this role's bounds start from
<code>constraint_overrides_json</code>	this role's deviations from that template
occupancy	
<code>assigned_user_id</code>	the human currently in the seat
<code>is_vacant</code>	whether the seat is empty — roles are born vacant
<code>vacancy_reason</code>	why it is empty; a closed set of three values
<code>auto_generate_agent</code>	create the delegate agent with the role; defaults on, forced off when external
<code>shadow_agent_id</code>	the delegate agent acting as this role — at most one
<code>is_external</code>	a board member or advisor; governance participation only
<code>is_coordinator</code>	a cross-team coordinator; non-primary, bridge-scoped envelope
<code>status</code>	<code>active</code> or <code>archived</code>

Note what is *not* on a role. **There is no clearance field** — a clearance is a separate object granted on the role (`api:POST /api/v1/role-clearances`), with its own vetting lifecycle. **There is no posture field** — posture belongs to the agent, and the role's authority level is what determines the posture its agent is provisioned at. Looking for either on the role and finding nothing is the expected outcome, not a gap.

The job description is executable

This is the field most likely to be treated as documentation, and it is the one with the most direct runtime effect. **`job_description` becomes the primary content of the role agent's system prompt.** Every organisational role has an agent (10.6); that agent's instructions are assembled from the role, and the job description is the body of them.

The assembly is ordered, and the order is the priority:

THE ROLE AGENT'S SYSTEM PROMPT, IN ASSEMBLY ORDER

1. IDENTITY who this agent is – the role's title, its authority band, and the autonomy posture that band implies
2. JOB DESCRIPTION the role's `job\_description`, carried in as the PRIMARY CONTENT. This is the bulk of the prompt.
3. RESPONSIBILITIES every entry in `responsibilities\_json`, in full – the list is not truncated or summarised
4. APPROVAL
AUTHORITY what this position may approve, derived from `approval\_authority\_json`: bare permissions, and numeric limits rendered as "up to N"
5. ESCALATION where decisions beyond scope go – derived from `reports\_to\_role\_id`. A role with a manager escalates to it; a chain root escalates only critical strategic decisions upward.
6. CONSTRAINT
AWARENESS the agent is told it operates inside an envelope and that the envelope is enforced, not advisory

Sections 2 and 4 are omitted when their fields are empty. Section 5 changes shape depending on whether the reporting edge exists.

Read it as: **the fields you fill in are the agent's instructions**. A role whose `job_description` is blank produces an agent that knows its title, its responsibilities and its escalation path and nothing about what the job actually is. A role whose `responsibilities_json` is an empty array produces an agent with no statement of what it is answerable for.

Two consequences follow, and the second is the one that costs.

Write the job description for the agent, not for a recruiter. "Owns the month-end close, reconciles intercompany balances, and escalates any variance above the approval threshold to the CFO" is instructions. "Seeking a motivated finance professional" is not, and it will be carried into the prompt verbatim.

The failure mode is a structurally perfect organisation full of agents with nothing to do. Every unit has a head, every head reports correctly, every address resolves, every envelope composes — and each agent's prompt is a title and an escalation path, because the intent fields were left empty during provisioning and nothing requires them. Nothing errors. The agents respond; they respond generically. The tell is agent output that could have come from any role in the organisation.


```
# The intent fields are the agent's instructions. Fill them at create time.
role = await client.roles.create(
    organization_unit_id=treasury_unit_id,
    title="Treasury Analyst",
    authority_level=2,
    reports_to_role_id=head_of_treasury_id,
    job_description=(
        "Monitors daily cash position across all operating accounts. "
        "Prepares the weekly liquidity forecast. Executes approved "
        "sweeps within the standing limit and refers anything above it."
    ),
    responsibilities_json=[
        "Daily cash position reporting",
        "Weekly liquidity forecast",
        "Execution of approved account sweeps",
    ],
    approval_authority_json={"account_sweep": 250000, "counterparty_onboarding": False},
)
```

Authority level — what it is, and what it is not

`authority_level` is an integer from 1 to 5.

level	band
1	Staff
2	Manager
3	Director
4	VP
5	C-Suite

 **Authority level does not gate knowledge access. Clearance does.** This is the single most consequential misreading of the whole model. A level-1 legal secretary can hold secret clearance; a level-5 sales VP may hold only restricted. A level-5 role with no clearance sees strictly less than a level-1 role with one — that is the intended behaviour, not a defect to work around. Building a permission model on `authority_level` produces an access policy that is well-formed, intuitive, and enforces nothing the platform agrees with.

It is equally important that **authority level does not constrain the reporting edge at all** — not depth, not direction. Peer reporting between two level-5 roles is legal. A senior specialist reporting into a junior project lead is legal. Reporting chains may be arbitrarily deep regardless of the five-value range: a chain running Chairman → CEO → CFO → Controller → AP Manager → AP Clerk is an ordinary six-level structure, and the bounded range says nothing about it. Matrix and dotted-line shapes are first-class. The two real controls on the reporting edge are cycle prevention and the escalation ceiling below, and neither reads the parent-versus-child relationship.

What authority level genuinely drives is five things, and each is worth knowing because each has a visible effect.

It is a ceiling on what a caller may grant. A caller cannot create a role, set a role's authority, or assign a user to a role whose authority sits above the caller's own. Additionally, a caller may not change the authority level of a role they are themselves assigned to. Together these close the path of self-promoting into an existing senior position that somebody else legitimately created — which is why the rule is expressed as a ceiling on the *caller* rather than a property of the role.

It determines the role agent's behavioural subtype. Levels 1–3 provision a `specialist` ; levels 4–5 provision a `manager` . That is a statement about the part the agent plays, not about its permissions.

It maps one-to-one onto the autonomy posture the agent is provisioned at.

authority level	posture
1	<code>pseudo</code>
2	<code>supervised</code>
3	<code>shared_planning</code>
4	<code>continuous_insight</code>
5	<code>delegated</code>

The mapping fails closed: anything outside 1–5 resolves to `pseudo` , the most restrictive posture. Because there is no posture field on the role itself, this mapping is how a provisioning choice about autonomy is expressed — 10.6 covers what the posture then does, and how it is changed afterwards through the posture surface rather than by editing the level.

It is a floor on knowledge-share policies — on the requester side. A share policy carries a `min_authority_level` , and a role below it is refused by that policy. Read the direction carefully: it bounds who may *receive* under the policy, not who may author it.

It tiers emergency approvals and blast-radius controls. An emergency envelope widening requires an approver at or above a level set by its duration — up to four hours needs a Manager, four to twenty-four a Director, twenty-four to seventy-two a VP. Beyond seventy-two hours it is not an emergency. Kill-switch scopes are tiered the same way, widening from a single agent through to the whole estate as the required level rises.

Vacancy

Roles are **born vacant**. A unit's head role is created with the unit, before anyone is appointed, and the platform is built to operate that way. `vacancy_reason` is a closed set of three values, and they are not interchangeable:

value	means
<code>structural_placeholder</code>	the platform minted it — the mandatory head role of a new unit, or a coordinator

value	means
<code>pending_appointment</code>	a real seat awaiting its first occupant
<code>vacated</code>	it was filled and the occupant was removed

The distinction is not bookkeeping. A structural placeholder has no business holding a clearance provisioned in advance; a seat awaiting its first hire legitimately may; and a vacated seat's prior clearance was a real decision made about *that* seat. Assigning a user clears the reason, because a filled seat has no vacancy to explain.

❌ **Vacancy is not a trust predicate, an execution predicate, a reachability predicate, or a knowledge-access predicate.** `is_vacant` means *no human occupies this seat*, and nothing else.

Everything that follows from that is worth stating positively, because the intuition runs the other way:

- **Every organisational role is a role agent, and the agent is active because the ROLE exists** — not because someone is sitting in it. A freshly-built organisation is entirely vacant and entirely operational.
- **Autonomy is set by posture, composed with the constraint envelope and the unit's ceiling.** Occupancy is not an input to that composition.
- **A superior can task down the tree and reach every role agent beneath.** An empty chair in the middle of the structure does not sever the subtree below it.
- **Accountability is stamped per task at issue time.** An autonomous task on a vacant role is legitimate precisely *because* the issuing human carries the accountability for it.

Vacancy has exactly one consequence. A *supervised* task needs a human in an inbox, and an empty seat has none — so the platform re-homes the human half of the work to the superior. It walks the reporting edge upward, skipping external roles (which are governance-only and never an anchor) and skipping further vacant seats, verifying tenancy at every hop, until it reaches an occupied role. It walks up; it never blocks.

The failure mode that behaviour prevents is a supervised task parked in an inbox nobody will ever open. That is what happens if vacancy is treated as "route it here anyway" — the task is correctly created, correctly assigned, and invisible. The symptom is work that is neither refused nor completed, and which shows as pending indefinitely.

Vacant roles are discoverable: `api:GET /api/v1/organization-roles` filters the population, and `api:GET /api/v1/organization-roles/without-agents` finds the narrower and more interesting case — roles that have no delegate agent at all, which is a provisioning defect rather than an ordinary state.

The bounds that attach to a role

Two independent limits attach to the same node, and designing as though they were one is the most common modelling error on the platform.

bound	answers	created by	becomes effective when
envelope	what may this role do ?	<code>api:POST /api/v1/role-envelopes</code>	it is activated
clearance	what may this role see ?	<code>api:POST /api/v1/role-clearances</code>	it is approved

An envelope is authored by a *defining* role for a *target* role, and composes along the reporting chain by **monotonic tightening** — an intersection, never a union. The structural consequence is worth drawing out: because a target's bounds are always a subset of its definer's, **you cannot express an escalation by adding an envelope**. Widening is not something the model can say. If a delegate genuinely needs to exceed its bounds, the answer is a human decision through the emergency path, not a second envelope.

A clearance carries its own vetting lifecycle, and the approval requirements rise with the level requested — from a single approval at the lowest vetted level, through justification, to multiple approvals and formal vetting at the top. The requester and the approver must be different people, and that distinctness is enforced rather than assumed. `api:POST /api/v1/role-clearances/{id}/approve` and `/api/v1/role-clearances/{id}/reject` are the decision surface.

🚫 **Creating is not granting, in both cases, and the failure is identical.** An envelope defaults to draft and enforces nothing until `api:POST /api/v1/role-envelopes/{id}/activate`. A clearance lands pending and grants nothing until it is approved. A provisioning script that creates fifty of something and walks away has produced an organisation that lists as fully bounded in every view and enforces nothing. Nothing errors. `api:GET /api/v1/governance/envelope-coverage` and `api:GET /api/v1/governance/envelope-hydration-status` are the reads that tell you the difference between created and in force.

The role, once

question	answer
Is a role a person?	No — a position . The occupant is an attribute.
What does <code>job_description</code> do?	It becomes the primary content of the role agent's system prompt .
Are responsibilities summarised into the prompt?	No — carried in full .
Does <code>authority_level</code> gate knowledge access?	No . Clearance does.
Does <code>authority_level</code> constrain reporting?	No — not depth, not direction, not peers.
What does it gate?	The granting ceiling, agent subtype, provisioned posture, share-policy floor, emergency and kill-switch tiers.
Are new roles occupied?	No — born vacant , and fully operational that way.

question	answer
What does vacancy change?	Exactly one thing: a supervised task routes to the superior.
Where does clearance live?	On a separate clearance object, not on the role.
Where does posture live?	On the agent; the role's authority level sets what it is provisioned at.
When does an envelope start enforcing?	On activation , not on creation.

Next: 10.5 — Trust chains and delegation

10.5 — Trust chains and delegation

A trust chain is the record of where an agent's authority came from. The property it exists to guarantee is EATP's: **every action an agent takes traces back, through an unbroken and tamper-evident chain, to a human who authorised it.** No agent holds authority intrinsically. It holds delegated authority, and the chain is the delegation.

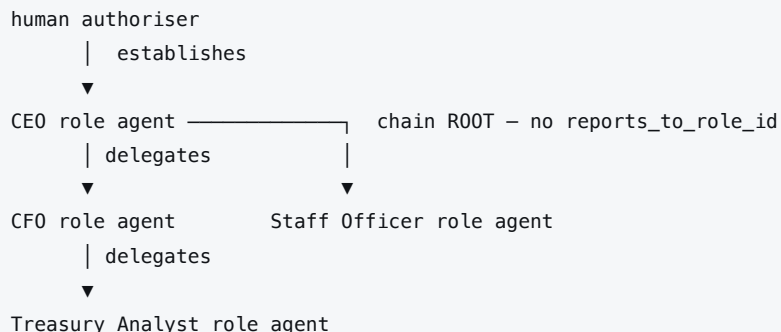
This chapter is the chain as a structural object — what it is derived from, what it carries, how it is generated from the organisation, how it is verified, and what happens when it is withdrawn. 06.4 covers the decision the chain participates in at request time; this chapter covers where the chain came from and what shape the graph of them has.

The single idea to carry out: chains are derived from the reporting relation and nothing else, so the reporting edges you set in 10.4 are the delegation graph — you do not author it separately, and you cannot correct it separately.

Derived from reporting, never from containment

The generation rule is one sentence: **for each role that has a reporting parent, delegate from the parent role's agent to this role's agent.** Roles with no reporting parent are chain roots and are left as roots — that is correct, not a gap, and the platform does not fill it in from containment.

THE DELEGATION GRAPH IS THE REPORTING GRAPH



Each arrow is one TrustChain row: trustor_agent_id -> trustee_agent_id.
Each arrow exists because a reporting edge exists. Containment contributes NOTHING to this graph – a role contained in one unit and reporting into another delegates along the REPORTING line.

Read it as: containment says where a role lives; reporting says whose authority it acts under. The chain follows the second. A role contained in Engineering but reporting to the General Counsel receives its delegation from the General Counsel's agent, and that is the intended behaviour of a deliberately drawn matrix edge.

Two details of the generation are worth knowing because they are visible in the chains you read back.

The constraint envelope attached to a delegation comes from the CHILD role, not the parent. The delegation carries the bounds of the position being delegated to. This is consistent with monotonic tightening — the child's envelope is already a subset of the parent's, so attaching the child's is

attaching the narrower one.

The delegation's scope is derived from the child's authority band. A child at authority 3 or above receives a `full` delegation; at 1 or 2 it receives a `partial` one. The chain also records a maximum cascade depth derived from the child's band, which bounds how far authority may be re-delegated beneath it.

Generation is a call you can run: `api:POST /api/v1/organization-builder/generate-trust-chains` derives the graph for the organisation. `api:POST /api/v1/trust/establish` creates a chain from an authority, and `api:POST /api/v1/trust/delegate` creates a single delegation directly when you need one outside the derived set.

What a chain carries

`sdk:aegis_sdk.TrustChain` is the typed shape. The fields divide into the parties, the lifecycle, the bounds, and the tamper evidence.

field	what it holds
the parties	
<code>trustor_agent_id</code>	the delegator — the agent granting authority
<code>trustee_agent_id</code>	the delegatee — the agent receiving it
<code>human_origin_data</code>	the human the lineage terminates at, and how they authenticated
<code>organization_id</code>	the tenant
<code>organization_unit_id</code>	the unit context the delegation sits in
lifecycle	
<code>status</code>	<code>active</code> · <code>completed</code> · <code>expired</code> · <code>revoked</code> · <code>suspended</code>
<code>expires_at</code>	when the delegation lapses
<code>revoked_at</code> / <code>revoked_by</code> / <code>revocation_reason</code>	the withdrawal record
scope and bounds	
<code>delegation_type</code>	<code>full</code> · <code>partial</code> · <code>conditional</code> — scope, never origin
<code>constraint_envelope_id</code>	the envelope bounding what the trustee may do
<code>constraint_envelope_data</code>	the computed envelope carried with the chain
<code>derived_from_org_structure</code>	whether this chain was generated from the reporting tree
<code>source_bridge_id</code>	set when the delegation originated at a bridge , not a reporting edge
tamper evidence	

field	what it holds
<code>genesis_signature</code>	the signature over the chain's immutable content
<code>integrity_hash</code>	the hash binding that content and signature together
<code>previous_chain_hash</code>	links this chain to its predecessor
<code>signing_key_id</code>	which key signed it

⚠ **trustor** and **trustee**, not **delegator** and **delegate**. The two vocabularies are both natural and only one is the field name. Code written against `delegator_agent_id` reads nothing and fails silently against a `None`.

🚫 **delegation_type** describes SCOPE, never ORIGIN. `full`, `partial` and `conditional` say *how much* authority was delegated. They do not say *where the delegation came from*. A bridge-originated delegation is marked by `source_bridge_id` being set, and it still carries an ordinary scope value. Writing `"bridge"` into `delegation_type` conflates the two axes and makes the scope of that delegation unreadable — the value that told you how much authority crossed is now telling you which mechanism crossed it, and nothing is recording the first.

Expiry is bounded, and absence is not the safe default

A delegation carries an explicit expiry, and the default window is **365 days**. Chains derived from the organisation are minted with that bound written into them.

🚫 **An absent expiry means unbounded, not immediate.** This is the one place in the trust model where the intuitive reading is backwards. A chain with no expiry is not a chain that has already lapsed; it is a chain the expiry check never evaluates, which is to say a delegation valid forever. **Omission is the permissive default here**, so a delegation minted by your own code with the expiry left unset authorises execution indefinitely and looks, in every listing, exactly like one that was bounded.

Note also that `expired` is a status that describes a chain, not an event that fires on a clock. Expiry is adjudicated from the timestamp when a chain is evaluated — the row does not transition to `expired` by itself at the moment the window closes. A chain listing as `active` past its expiry is not a defect and is not usable; read the expiry, not just the status.

Verifying a chain

Four reads answer four different questions.

call	answers
<code>api:GET /api/v1/trust/chains/{id}</code>	what does this chain say?
<code>api:GET /api/v1/trust/chains/{id}/delegation-path</code>	what is the full lineage back to the human?

call	answers
<code>api:GET /api/v1/trust/chains/{id}/summary</code>	the condensed view for display
<code>api:POST /api/v1/trust/verify</code>	is the cryptography intact — signature and integrity hash?

`api:GET /api/v1/trust/agents/{id}/trust-context` is the aggregate for an agent: its chain, its delegation path, its position, how long until expiry, and any warnings. It is the right single call when you are deciding whether an agent is currently fit to act, because a chain that exists is not the same as a chain that is usable.

Verification is two independent checks and both must hold. The signature proves the chain's immutable content was signed by the recorded key. The integrity hash proves that content and that signature bind together, chained to the previous chain's hash. Only immutable fields are signed — the parties, the genesis data, the human origin, the envelope, the expiry, the scope — deliberately excluding status and timestamps, so that revoking a chain does not invalidate the signature attesting to how it was created. A revoked chain is still a provably authentic record of a delegation that was made and then withdrawn, which is exactly what an audit needs.

The signing root is a **trust authority**: `api:GET /api/v1/trust/authorities` lists them, `api:POST /api/v1/trust/authorities` creates one, and `api:GET /api/v1/trust/authorities/{id}/agents` shows what hangs off it. The authority is the organisational anchor the lineage terminates at above the human.

```
# Before relying on an agent's authority, read the context – not just the chain.
context = await client.trust.get_agent_trust_context(agent_id=analyst_agent_id)

if context["has_warnings"]:
    # expires_in_days, a suspended ancestor, a posture below what the task needs
    for warning in context["warnings"]:
        log.warning("trust context", agent=analyst_agent_id, detail=warning)
```

Revocation, and why it cascades

Withdrawing authority from one agent has to withdraw it from everything that derived authority from that agent, or the guarantee the chain exists to provide is gone. Revocation therefore comes in a graded set of operations, and choosing between them is a real decision.

operation	scope
<code>api:POST /api/v1/trust/revoke</code>	an agent and everything downstream of it
<code>api:POST /api/v1/trust/revoke/{id}/cascade</code>	the same, named explicitly
<code>api:POST /api/v1/trust/revoke/by-human/{id}</code>	everything tracing to one person
<code>api:POST /api/v1/trust/revoke-delegation</code>	one delegation edge — the only non-cascading removal

🚫 **Every revocation route cascades. There is no non-cascading revoke.** The two general routes behave identically, so reaching for the first in the belief that it touches one chain removes the whole subtree beneath it. Only `api:POST /api/v1/trust/revoke-delegation` is narrow, and it removes an edge rather than an agent's standing. The symptom of the mistake is a revocation that "worked" and a department that stopped acting an hour later.

Preview before you cascade. `api:GET /api/v1/trust/revoke/{id}/impact` returns the agents that would be affected, how many, and whether any of them currently hold active workloads. It is a read, it is cheap, and a cascade is not reversible by re-running it — a reactivation is a different operation with a different record.

The human-origin cascade deserves its own mention because it is the operation that makes the EATP guarantee operationally real. When a person leaves, or their access is withdrawn, **every delegation anywhere in the organisation that traces to them is revoked in one action**. That is only expressible because every chain records its human origin; in a model where agents held intrinsic authority, the same question would have no answer.

Cascades are durable and resumable. A cascade runs as a recorded job with its targets written down before any step executes and its completions appended as it goes, so an interrupted cascade is resumable rather than restartable. `api:GET /api/v1/trust/revoke/jobs/incomplete` lists jobs that did not finish; `api:POST /api/v1/trust/revoke/jobs/{id}/resume` continues one. Per-agent steps are idempotent, so resuming over already-revoked agents is a no-op.

🚫 **Check for incomplete revocation jobs after any cascade over a large subtree.** An interrupted cascade leaves part of the tree revoked and part of it still holding authority — and the part still holding it looks entirely normal, because a chain that was never reached is an ordinary active chain. The symptom is an agent that continues working after you believed its authority was withdrawn, which reads as a revocation bug rather than as an unfinished job. The incomplete-jobs list is the read that distinguishes them.

The reversible brake is a posture, not a chain state. Where revocation is terminal, a leave of absence, an investigation or a temporary stand-down needs something you can lift afterwards — and that is a *downward* posture override (`api:PUT /api/v1/agents/{id}/trust-posture`), which applies immediately rather than waiting on an approval. 09.5 carries the posture surface in full.

🚫 **There is no operation that suspends a trust chain.** `suspended` appears in the status list above because the platform reaches it on its own, when a cascade revokes a bridge-sourced chain — it is a state the chain can be found in, never a button you press. Reaching for a suspend call because the status exists is the trap: the state is real, the operation is not, and what looks like the reversible option is a lifecycle outcome you cannot invoke. Use the downward posture override instead.

The audit trail

Chains are one half of the record; the audit entries are the other. `sdk:aegis_sdk.TrustAuditEntry` is the shape.

call	answers
<code>api:GET /api/v1/trust/audit</code>	what has happened across the trust surface
<code>api:GET /api/v1/trust/audit/by-human/{id}</code>	everything attributable to one person
<code>api:GET /api/v1/trust/audit/trace/{id}</code>	the full trace for one action

The by-human read is the counterpart of the by-human revocation, and the pair is the practical shape of the accountability claim: for any person, you can answer both *what was done under their authority* and *withdraw all of it*. Those two questions being answerable from the platform is what makes the governance provable rather than asserted.

Six ways a chain graph goes wrong

Each of these produces a structure that is internally consistent and wrong, which is why they are worth having as a checklist rather than as intuitions.

defect	what it looks like	the read that finds it
unattached head roles	subtrees that are separate authority islands	<code>api:GET /api/v1/organization-roles/{id}/reporting-chain</code> returning one entry
delegation minted without an expiry	authority valid forever, indistinguishable from bounded	read <code>expires_at</code> , not <code>status</code>
<code>delegation_type</code> used to record origin	scope unreadable; bridge chains miscounted	<code>source_bridge_id</code> is the origin marker
interrupted cascade	part of a subtree still authorised	<code>api:GET /api/v1/trust/revoke/jobs/incomplete</code>
containment walked for authority	a plausible answer about the wrong relation	compare against <code>/api/v1/trust/{id}/reporting-chain</code>
revocation used for a temporary stand-down	an irreversible record of a reversible intent	the withdrawal record; there is no undo — a posture override was the tool

Trust chains, once

question	answer
What is a chain derived from?	The reporting relation. Containment contributes nothing.
Who are the parties?	<code>trustor_agent_id</code> (delegator) → <code>trustee_agent_id</code> (delegatee).
Whose envelope does a delegation carry?	The child role's.

question	answer
What does <code>delegation_type</code> mean?	Scope — <code>full</code> · <code>partial</code> · <code>conditional</code> . Never origin.
How is a bridge-originated chain marked?	<code>source_bridge_id</code> , not the type field.
What is the default expiry?	365 days, written explicitly at mint.
What does an absent expiry mean?	Unbounded — not immediate.
What does verification check?	Signature and integrity hash; both must hold.
Why isn't <code>status</code> signed?	So revoking a chain does not invalidate the record of how it was made.
What precedes a cascade?	The impact preview.
What makes a person's authority withdrawable?	Every chain records its human origin.
Which operation is reversible?	A downward posture override . Revocation is terminal and cascades.

Next: 10.6 — Role agents

10.6 — Role agents

Every organisational role is a role agent. Not *may have* one, not *gets one when someone is hired* — the agent exists and is active because the **role** exists. This chapter is that agent: when it comes into being, what it is provisioned with, how much autonomy it holds and who decides that, and what actually stands between it and an action.

The framing to correct first, because it shapes everything after it: a role agent is not a robot assigned to help an employee. **It is the position, acting.** The CFO's role agent acts as the CFO — inside the CFO's envelope, at the CFO's clearance, under a delegation from the CEO's agent, and with the CFO position's job description as its instructions. Whether a human currently occupies that position is a separate question, and it is not this one.

The single idea to carry out: the agent's authority comes from the position, its instructions come from the position, and its autonomy comes from posture — so provisioning an agent is not a step after describing the organisation, it is a consequence of it.

When an agent comes into being

A role's agent is created with the role. `auto_generate_agent` defaults on, and a role created through `api:POST /api/v1/organization-roles` arrives with its agent provisioned and bound. The same happens for the head role the platform mints when you create a unit — so a freshly-created department has a head role, that head role has an agent, and the agent is active, all before anyone is appointed.

Two things modify that.

External roles get no operational agent. A role with `is_external` set — a board member, an advisor, outside counsel — has `auto_generate_agent` forced off. External roles participate in governance: they approve, observe and audit. They do not execute work, and the platform enforces that rather than relying on you to remember it. **The failure mode that prevents is a board seat that can execute** — which, in every listing, looks exactly like an ordinary senior role.

Bulk provisioning has its own call. `api:POST /api/v1/organization-builder/generate-agents` provisions agents across a structure, which is the path for an organisation imported or compiled in one pass rather than built role by role.

🚫 **The agent is created ACTIVE, and a vacant role's agent is active too.** Vacancy is not an agent-lifecycle predicate (10.4). A role agent that waited for a hire would mean a freshly-built organisation — which is entirely vacant by construction — was entirely unable to act, which is the opposite of the intended behaviour.

⚠️ **active is a lifecycle state, not a grant.** An active agent is one that exists and is eligible to be reached; it is not one that may do anything it likes. Everything it may actually do is decided by the envelope, the posture, the clearance and the chain, every time it acts. Reading `status: active` as "this agent is permitted" is the single most common overreading of this field.

The binding

A role holds at most one delegate agent, which is why the link operations take the role rather than the agent as their subject: `api:POST /api/v1/organization-roles/{id}/link-agent` and the bodyless `api:POST /api/v1/organization-roles/{id}/unlink-agent`. The role carries the binding as `shadow_agent_id`.

Linking does not widen anything. The agent acts inside the envelope the role already carries, so the link is a *naming* of who acts as this position — not a grant of anything. That is worth internalising because it inverts the usual instinct: you do not give an agent permissions, you attach it to a position that already has bounds.

`api:GET /api/v1/organization-roles/without-agents` is the read that finds roles with no agent bound. On a well-provisioned organisation the result should be the external roles and nothing else; anything else on that list is a provisioning defect, and it presents as a position that exists, addresses correctly, has an envelope, and never responds.

read	answers
<code>api:GET /api/v1/delegate-agents</code>	every delegate agent in the organisation
<code>api:GET /api/v1/delegate-agents/for-unit/{id}</code>	the agents acting for roles in one unit
<code>api:GET /api/v1/delegate-agents/{id}/trust-chain</code>	the delegation this agent is acting under
<code>api:GET /api/v1/delegate-agents/{id}/health</code>	whether it is currently fit to act
<code>api:GET /api/v1/delegate-agents/{id}/activity</code>	what it has been doing
<code>api:GET /api/v1/delegate-agents/dashboard-summary</code>	the population at a glance

`api:POST /api/v1/delegate-agents/{id}/deactivate` and `/api/v1/delegate-agents/{id}/activate` are the lifecycle controls, and `api:PATCH /api/v1/delegate-agents/{id}` amends one.

What the agent is provisioned with

Four things are derived from the role at provisioning time. Three of them are derived from `authority_level`, which is why that field matters far more than its "informational" description suggests.

FROM THE ROLE, TO THE AGENT

title + authority band	→ IDENTITY	"You are the {title} agent, operating {authority band}."
job_description	→ INSTRUCTIONS	the bulk of the system prompt
responsibilities_json	→ (in full)	
approval_authority_json	→ what it may approve	
reports_to_role_id	→ where it escalates	
authority_level	→ SUBTYPE	1-3 specialist · 4-5 manager
	→ POSTURE	the autonomy it starts at
	→ CEILINGS	emergency + blast-radius tiers
constraint_template_id	→ ENVELOPE	what it may do
the role's clearance	→ VISIBILITY	what it may see
the reporting edge	→ TRUST CHAIN	whose authority it acts under

Read it as: nothing here is configured on the agent. Every one of these is read from the position. An agent whose behaviour is wrong is almost always a role whose fields are wrong, and editing the agent is treating the symptom.

10.4 covers the prompt assembly in detail — the short version is that `job_description` is the primary content and `responsibilities_json` is carried in whole.

Type, subtype and status

Three enums describe an agent and they answer genuinely different questions.

Type (`sdk:aegis_sdk.AgentType`) is what the agent *is* from a lifecycle standpoint, and the vocabulary is wider than the create surface — which matters, because you will see values in listings you cannot create:

creatable through the API	minted by the platform itself
<code>chat</code> , <code>task</code> , <code>pipeline</code> , <code>custom</code>	<code>shadow</code> , <code>pseudo</code> , <code>tool</code> , <code>esa</code>

Presence in the right-hand column is not something you provision; those agents come into existence because some *other* object did. A tool agent is created by registering a tool. A role's delegate agent is created by the role. **Finding an agent in a listing you never made is normal**, and it does not mean someone provisioned it behind your back.

Subtype (`sdk:aegis_sdk.AgentSubtype`) is behavioural — the part the agent plays. Role agents are provisioned as `specialist` at authority levels 1–3 and `manager` at 4–5; the wider enum also carries `esa`, `pseudo` and `governance`.

Status (`sdk:aegis_sdk.AgentStatus`) is the lifecycle: `draft`, `active`, `archived`, `deprecated`, `suspended`, `revoked`. Treat `suspended` and `revoked` as reachable rather than exotic — an agent is transitioned to `suspended` by the platform when something goes wrong, which is precisely the moment your code is running and reading it.

Posture — the autonomy dial

Posture is how much of its delegated authority an agent may exercise unaided. The five values are CARE-aligned and lowercase:

posture	what it means for the agent	provisioned at authority
<code>pseudo</code>	acts only as a proxy for a human	1
<code>supervised</code>	acts, with a human in the loop	2
<code>shared_planning</code>	plans jointly, executes agreed work	3
<code>continuous_insight</code>	acts, with continuous human visibility	4
<code>delegated</code>	acts within its envelope, unaided	5

The mapping from authority to posture is one-to-one, and it fails closed. Anything outside 1–5 resolves to `pseudo`, the most restrictive value. Because there is no posture field on the role itself, this mapping is how a provisioning decision about autonomy gets expressed: the authority band you set is the posture the agent starts at.

Posture is then managed on the agent, through its own surface:

call	purpose
<code>api:GET /api/v1/agents/{id}/trust-posture</code>	what posture is this agent at?
<code>api:PUT /api/v1/agents/{id}/trust-posture</code>	set it
<code>api:GET /api/v1/agents/{id}/trust-posture/evaluate</code>	what does the platform assess it should be?
<code>api:GET /api/v1/agents/{id}/trust-posture/evidence</code>	on what evidence
<code>api:GET /api/v1/agents/{id}/trust-posture/history</code>	how it has moved
<code>api:GET /api/v1/agents/{id}/trust-posture/upgrade-eligibility</code>	may it move up yet?
<code>api:POST /api/v1/agents/{id}/trust-posture/request-upgrade</code>	ask
<code>api:POST /api/v1/agents/{id}/trust-posture/approve / /reject</code>	a human decides
<code>api:GET /api/v1/posture/pending-approvals</code>	everything currently awaiting a decision

`sdk:agis_sdk.TrustPostureModule` wraps these and `sdk:agis_sdk.TrustPosture` is the typed value. **Posture progression is a human decision with evidence behind it** — the evaluate and evidence reads exist so the decision is made against a record rather than a feeling, and the upgrade path is a request plus an approval rather than a write.

⚠ **Raising `authority_level` to raise an agent's autonomy is the wrong lever.** Authority level sets what the agent was *provisioned* at; the posture surface is what moves it afterwards. Editing the band to move autonomy also moves the granting ceiling, the subtype, the emergency-approval tier and the kill-switch scope — four unrelated changes to make one, and every one of them permanent until someone notices.

The ceiling cascades, and the effective value is a minimum

An agent's posture is not the whole story, because the unit it sits in carries a ceiling and that ceiling cascades down the containment tree.

```
EFFECTIVE POSTURE = min( the agent's posture,
                        the unit's EFFECTIVE ceiling,
                        the posture selected for this task )

where the unit's effective ceiling is itself
      min( this unit's declared ceiling, its parent's effective ceiling )
```

Read it as: **a child unit can never exceed its parent's ceiling, whatever its own value reads.** Setting a permissive ceiling on a leaf beneath a restrictive branch changes nothing, and nothing tells you that.

`api:GET /api/v1/organization-units/{id}/posture-ceiling` returns both the declared value and the effective one. **Read the effective one.** The declared value is half the picture and it is the half that looks authoritative.

🚫 The setter, `api:PUT /api/v1/organization-units/{id}/posture-ceiling`, has an asymmetry worth carrying into your code: **omitting the argument leaves the ceiling unchanged, while passing an explicit null clears it — and a cleared ceiling widens what the unit may do.** Those two are one keyword apart at the call site and opposite in effect. The symptom of getting it wrong is agents in a subtree operating at a higher autonomy than anyone granted, with no event marking when it changed.

What actually stands between an agent and an action

An active agent with a valid chain is still checked, every time. Six conditions hold, and the list is worth having explicitly because "the agent is active" covers none of them.

condition	what it means
the organisation is active	the tenant itself is not suspended
a trust chain exists	the agent holds a delegation at all
that chain is usable	active, unexpired, not revoked or suspended
the role resolves, same-tenant	the position the agent acts as is real and belongs to this tenant
the posture permits it	the effective posture is not one that denies this action
the budget is not exhausted	the financial dimension of the envelope still has room

And over all of it, the **constraint envelope** bounds what the action may be at all — across the five constraint dimensions: financial, operational, temporal, data access and communication.

`api:GET /api/v1/delegate-agents/{id}/health` is the aggregate read, and `api:GET /api/v1/trust/agents/{id}/trust-context` carries the chain-side half with its warnings (10.5).

```
# "Is this agent fit to act?" is not a status read.
health = await client.delegate_agents.get_health(agent_id=analyst_agent_id)
context = await client.trust.get_agent_trust_context(agent_id=analyst_agent_id)

fit = (
    health["status"] == "healthy"
    and not context["has_warnings"]
)
```

Five ways a role agent disappoints you

Each looks like a different problem and each is the same class: something read from the position was wrong or absent.

symptom	actual cause
the agent responds generically, like any other role	<code>job_description</code> and <code>responsibilities_json</code> were left empty
the agent refuses work it clearly should do	the envelope was created but never activated
the agent sees less than expected	the clearance was created but never approved
the agent is more autonomous than intended in a subtree	a unit ceiling was cleared rather than left unchanged
the position exists but never responds at all	no agent bound — <code>api:GET /api/v1/organization-roles/without-agents</code>

None of these produces an error. That is the shape of the whole chapter: the role agent is derived from the position, so a position described incompletely yields an agent that behaves incompletely, correctly, and without complaint.

Role agents, once

question	answer
Which roles have agents?	All of them, except <code>is_external</code> roles.
When is the agent created?	With the role — and it is created active .
Does a vacant role have an active agent?	Yes. Vacancy is not an agent predicate.
How many agents per role?	At most one.
Does linking an agent grant anything?	No — it names who acts as the position.

question	answer
Where do its instructions come from?	The role's <code>job_description</code> and <code>responsibilities_json</code> .
What sets its starting posture?	<code>authority_level</code> , one-to-one, failing closed to <code>pseudo</code> .
How is posture changed afterwards?	The posture surface — request, evidence, human approval.
What does the unit contribute?	A ceiling that cascades; effective posture is a minimum .
Does <code>status: active</code> mean permitted?	No. It means eligible to be reached.

Next: 10.7 — Re-orgs, bridges and workspaces

10.7 — Re-orgs, bridges and workspaces

The six chapters before this one describe a structure at rest. Real organisations are not at rest: they restructure, they need people to work across the boundaries they just drew, and they run temporary efforts that do not fit the chart at all. This chapter is the three mechanisms for that — moving the structure, crossing it, and working beside it — plus the tenant boundary that contains all of them.

The through-line is worth stating up front, because it is what makes these three one chapter rather than three appendices: **none of them weakens the structure**. A move recomputes governance rather than detaching it. A bridge is an explicit, authorised, bounded crossing — not a hole. A workspace groups people without granting them anything. Every mechanism here is additive and every one of them composes by intersection.

The single idea to carry out: when two bounds meet, the most restrictive wins — on every one of these paths, without exception.

Moving a unit is a governance event

`api:POST /api/v1/organization-units/{id}/move` re-parents a unit and everything beneath it. It is not a chart edit, and the response's cascade summary is the honest measure of what it touched.

WHAT A MOVE CASCADES, AND WHICH PARTS ARE MANDATORY

- | | | |
|---|--|--|
| 1 | snapshot | the old parent, level and path, and every descendant |
| 2 | re-parent | the unit and its subtree |
| 3 | ADDRESSES
[MANDATORY] | recomputed for the ORGANISATION — not patched in place, because sequential numbering means a move can renumber nodes outside the subtree. Failure rolls the move back. |
| 4 | POSTURE
CEILINGS | ceilings recomputed down the tree
[MANDATORY] — failure rolls the move back. |
| 5 | KNOWLEDGE
SHARE POLICIES
[MANDATORY] | every share policy touching the moved subtree is re-checked for ancestry. Any that no longer holds is SUSPENDED. If the re-check itself fails, ALL policies touching the subtree are suspended pending review — and if even that fails, the move is rolled back. |
| 6 | bridges | role-anchored bridges survive; their denormalised unit references are re-derived |
| 7 | trust chains | chains whose structural premise changed are revoked and replacements created — scoped to the moved subtree, so a concurrent change elsewhere is not swept up |
| 8 | audit | the move is recorded |

Read it as: steps 3, 4 and 5 are load-bearing enough that failing them undoes the move. Step 5 is the strictest, and the reasoning is explicit — **a stale share policy grants knowledge access from a unit the role is no longer part of**, which is a live exposure rather than an inconsistency. When the platform cannot determine which policies still hold, it suspends all of them rather than guessing.

🚫 **Read the cascade summary before treating a move as tidying up the chart.** Addresses change, and every address beneath the moved node changes with them (10.3). Anything you cached by address is now stale. Trust chains under the moved subtree are torn down and rebuilt, which means the delegation an agent held a moment ago has a new identity. **Nothing about the chart looks different afterwards, which is exactly why the summary is the thing to read.**

Re-parenting a *role* — changing `reports_to_role_id` — is the other half of restructuring, and it carries its own check: the new reporting edge is validated for envelope tightening before it is accepted. A move that would place a role under a parent whose envelope does not contain the role's own is refused, because accepting it would express a widening, and widening is not something the model can say (10.4).

Bridges — crossing the structure, deliberately

A bridge is a governed edge between two parts of the organisation that do not have a reporting relationship. It is how Legal advises Engineering, how an incident incident-response effort reaches across three departments, how a committee spanning four units does its work.

🚫 **A bridge is not a reporting edge and never becomes one.** Trust chains minted by a bridge carry `source_bridge_id` and are marked as bridge-originated; they are not part of the reporting-derived delegation graph (10.5). A bridge lets authority *cross*; it does not restructure who reports to whom, and it does not survive as a relationship when the bridge is revoked.

Three kinds exist, and they are three separate constructs rather than one with a duration field.

kind	shape	duration	approval	mints trust chains
standing	role ↔ role, two parties	permanent, with optional review and expiry	two distinct humans, each currently occupying their anchor role	yes
scoped	N units, tied to a workspace and an objective	bounded window; may auto-expire on objective completion	approved as a unit	no
ad-hoc	unit → unit, directional	minutes, from activation	the target unit approves the initiator's request	no

Standing bridges anchor on roles, not units — and that is what makes them survive a re-org. A bridge between the Head of Legal and the Head of Platform is a relationship between two positions; moving either position's unit re-derives the bridge's bookkeeping and leaves the bridge itself intact. A bridge anchored on containers would break on every restructure.

Standing bridges are created with `api:POST /api/v1/standing-bridges`, authorised with `api:POST /api/v1/standing-bridges/{id}/authorize`, and brought into force with `api:POST /api/v1/standing-bridges/{id}/activate`. `api:POST /api/v1/standing-bridges/{id}/check-interaction` asks whether a specific interaction would be permitted, and `/api/v1/standing-bridges/{id}/review`, `/api/v1/standing-bridges/{id}/suspend` and `/api/v1/standing-bridges/{id}/revoke` are the lifecycle. `api:GET /api/v1/standing-bridges/for-unit/{id}` lists what touches a unit and `api:GET /api/v1/standing-bridges/overdue` finds the ones past review. `sdk:aegis_sdk.modules.StandingBridgesModule` is the typed surface.

 **Dual authorisation means two different people, each of whom currently holds the anchor role they are signing for.** All three conditions are checked at activation: both signatures present, the two signatories distinct, and each still occupying the position they signed as. A signature from someone who has since moved on does not carry the bridge, and the platform re-derives occupancy rather than trusting the stored record. **The failure mode that closes is a cross-unit grant carrying one person's authority under two names.**

Scoped bridges are the construct for a bounded, multi-party effort: `api:POST /api/v1/scoped-bridges`, with `/api/v1/scoped-bridges/{id}/approve`, `/api/v1/scoped-bridges/{id}/extend`, `/api/v1/scoped-bridges/{id}/renew`, `/api/v1/scoped-bridges/{id}/complete`, `/api/v1/scoped-bridges/{id}/expire` and `/api/v1/scoped-bridges/{id}/cancel` as the lifecycle, `/api/v1/scoped-bridges/{id}/add-unit` to bring a unit in, and `api:GET /api/v1/scoped-bridges/for-workspace/{id}` to find the ones attached to a workspace. `sdk:aegis_sdk.modules.ScopedBridgesModule` wraps them.

Ad-hoc bridges are the urgent path — a request from one unit to another that the target approves and activates, bounded to a duration in minutes. `api:GET /api/v1/ad-hoc-bridges/pending-for/{id}` is the inbox, `api:POST /api/v1/ad-hoc-bridges/{id}/approve`, `/api/v1/ad-hoc-bridges/{id}/reject` and `/api/v1/ad-hoc-bridges/{id}/activate` are the decisions, and `sdk:aegis_sdk.modules.AdHocBridgesModule` is the typed path.

What is checked when something crosses

An interaction over a bridge passes an ordered set of checks and stops at the first refusal, with the refusal naming the check that produced it:

#	check	asks
1	bridge status	is this bridge in force at all?
2	direction	does the bridge permit traffic this way?
3	interaction type	is this interaction on the allowed list and off the prohibited one?
4	dual authorisation	for a standing bridge requiring it, is it properly authorised?
5	participant role	for a scoped bridge, is the actor a participant in a permitted capacity?
6	path access	is the material being reached within the bridge's shared paths?

And over the crossing itself, the composition rule:

```
EFFECTIVE POSTURE AT A CROSSING
= min( the source's posture,
       the target's posture,
       the bridge's own ceiling )
```

That minimum is what prevents trust laundering. Without it, an agent at a restrictive posture could route work through a bridge into a permissive part of the organisation and come back with an answer it was never entitled to compute. The crossing takes the tightest of the three, so a bridge can never be a route to more autonomy than either side already held.

A standing bridge's `knowledge_sharing_level` is `restricted`, `partial` or `full`, and **only `partial` and `full` grant anything**. A bridge at `restricted` is a communication channel, not a knowledge grant — and a bridge that is not active grants nothing whatever its level says.

Knowledge share policies — downward, and never transitive

Units are knowledge boundaries. Material crosses one by an explicit policy, never by adjacency and never by proximity on the chart. `api:POST /api/v1/knowledge-share-policies` creates one, with `/id/suspend` and `/id/reactivate` as the controls and `api:GET /api/v1/knowledge-share-policies/due-for-review` for the ones needing attention.

Three properties govern them, and each blocks a different wrong assumption.

Downward only. The source unit must be an **ancestor** of the target in the containment tree. A policy sharing sideways or upward is refused at creation. Knowledge flows down the tree by grant; it does not flow up or across by policy.

Never transitive. If A shares with B and B shares with C, **C still cannot see A's material**. Every hop is its own policy. Designs that assume otherwise promise a capability that does not exist — in the safe direction, which is why they survive review and surface as "why can't they see it?" months later.

The authority floor is on the requester. A policy's `min_authority_level` bounds who may *receive* under it. It does not bound who may author it, and reading it the other way produces an authoring model the platform does not implement.

Policies can carry conditions — a time window, a project scope, a requirement for an active session — so a grant can be narrower than "this unit, these paths".

⚠ **A policy that explicitly denies is not the same as no policy at all, and the difference decides whether a bridge can help.** When an access evaluation finds a matching policy that refuses — wrong path, wrong classification, wrong type, authority below the floor, a condition unmet — that is an explicit denial and a bridge does not override it. When no policy matches, a bridge may legitimately supply the access. `api:POST /api/v1/governance/explain-access` is the read that tells you which of the two you are looking at, and it is worth reaching for immediately: a clearance failure, a classification failure and a missing policy all present identically from outside.

`sdk:aegis_sdk.KnowledgeModule` covers the item lifecycle and
`sdk:aegis_sdk.modules.KnowledgeGovernModule` the review and share surface.

Workspaces — the orthogonal axis

A workspace is where work happens, and it is deliberately **not** part of the organisational tree. It has no unit and no role. Its membership is by **user**, so a workspace can assemble people from four departments without altering any reporting line or containment edge — which is the whole point of it.

`api:POST /api/v1/workspaces` creates one, `api:POST /api/v1/workspaces/{id}/members` adds a person, `api:POST /api/v1/workspaces/{id}/work-units` attaches work, and `api:POST /api/v1/workspaces/{id}/documents` attaches material. `api:POST /api/v1/workspaces/{id}/archive` and `/api/v1/workspaces/{id}/restore` are the lifecycle. `sdk:aegis_sdk.modules.WorkspacesModule` is the typed surface.

Governance in a workspace is **classification-based**, and the two classification fields are different quantities that are easy to conflate:

field	is a	meaning
<code>classification</code>	write default	what should material created here be marked?
<code>contains_classification</code>	read-side ceiling	the high-water mark of everything currently attached to it

`contains_classification` rises as material is attached and **does not fall when material is detached** — lowering it is a deliberate, recorded act by a named actor with a reason, not a side effect. It is the admission test: a person may join only if their effective clearance meets or exceeds it.

🚫 **That test is a floor, not an intersection — and the thing it prevents is declassification by invite.**

The joiner's clearance is checked against the workspace's mark; the mark does not move down to accommodate the joiner. If it did, adding one under-cleared member would silently reduce the sensitivity the workspace claims to hold, and every subsequent admission would be judged against the reduced value.

Workspace classification runs `public`, `restricted`, `confidential` and stops there — the two highest levels require compartment-level need-to-know, which is a property of knowledge items rather than of workspaces. Material above `confidential` is governed where compartments exist, not here.

Scoped bridges are the link between the two axes: a scoped bridge carries a `workspace_id`, which is how unit participation is brought into a workspace under governance rather than by membership alone.

The flat Team is membership; a unit is a boundary

One more construct is easy to reach for and usually wrong. `api:POST /api/v1/teams` and `api:POST /api/v1/teams/{id}/members` create a working group: a set of people, with no classification of its own and no knowledge-boundary semantics. `sdk:aegis_sdk.TeamsModule` wraps it.

If you want a boundary, you want a unit. Flat Teams are the right construct for "these people work together". They are the wrong construct for "these people and nobody else may see this", and reaching for one there produces an access model that is more permissive than you believe it is — with nothing in

any listing to distinguish the two intentions.

Note the vocabulary collision this creates with the `T` in the D/T/R grammar: a **team unit** (`unit_type: "team"`) is a container in the tree with a head role, a classification default and a posture ceiling. A flat **Team** created through the teams surface is a membership list beside the tree. Two constructs, one word, and only the first is addressable.

The tenant boundary contains all of it

Everything in this part is scoped to one organisation. `api:POST /api/v1/organizations` creates one, `api:GET /api/v1/organizations/{id}` reads it, and `api:GET /api/v1/auth/me/organizations` lists the ones your principal can see — which is how you discover you are holding credentials for more than one. `sdk:aegis_sdk.OrganizationsModule` is the typed write path; `sdk:aegis_sdk.OrgStandupModule` carries the list reads.

The tenant boundary is enforced by answering as not-found, not by refusing. A unit, role, chain, agent, bridge or workspace belonging to another organisation comes back `404`, never `403` — because a `403` would confirm the object exists. Every lookup verifies tenancy, including lookups by primary key and lookups served from cache.

Plan for one organisation per customer, and treat a second as a migration you have not decided to make yet. The scoping is total: nothing crosses it, and no bridge, policy or workspace spans it. Anything you build across two organisations is something you are maintaining outside the platform's model, with none of the guarantees the rest of this part describes.

The three mechanisms, once

mechanism	changes the structure?	grants access?	survives a re-org?
unit move	yes — and cascades	no	it is the re-org
standing bridge	no	yes, at <code>partial</code> or <code>full</code> , when active	yes — anchored on roles
scoped bridge	no	within its window and paths	within its window
ad-hoc bridge	no	briefly, on approval	it expires in minutes
knowledge share policy	no	downward, non-transitively	revalidated , suspended if broken
workspace	no	no — membership only	unaffected; it is orthogonal
Team (flat entity)	no	no — membership only	unaffected

And the rule they all obey: **when two bounds meet, the most restrictive wins**. Envelopes intersect down the reporting chain. Posture takes the minimum at a bridge crossing. Unit ceilings cascade as a minimum. Workspace admission is a floor the joiner must clear. There is no path through this model

along which a bound gets wider, and if you find yourself designing one, the structure is the thing to change.

Next: Part 11 — The deployment pack

Part 11 — The deployment pack

Audience: the operator who runs an Aegis deployment. Not the integrator attaching to one — that is part 07 — but the person who provisions the substrate, sets the configuration, applies the release, watches it, and answers for it at three in the morning. You hold the cluster credentials. You decide the maintenance window. You are the one the auditor asks.

Part 07 describes the SHAPE of a deployment: what the components are, who owns what, where the boundaries fall. **This part is the working pack** — the substrate to provision, the configuration keys to set, the runtime and its node catalogue, the stores and their migrations, the release and rollback operation, the day-to-day watch, and the checks that prove the whole thing is correct.

The two parts are deliberately disjoint. Where this part needs a shape, it links to part 07 rather than restating it. **Read part 07 first if you have not; read this one with a terminal open.**

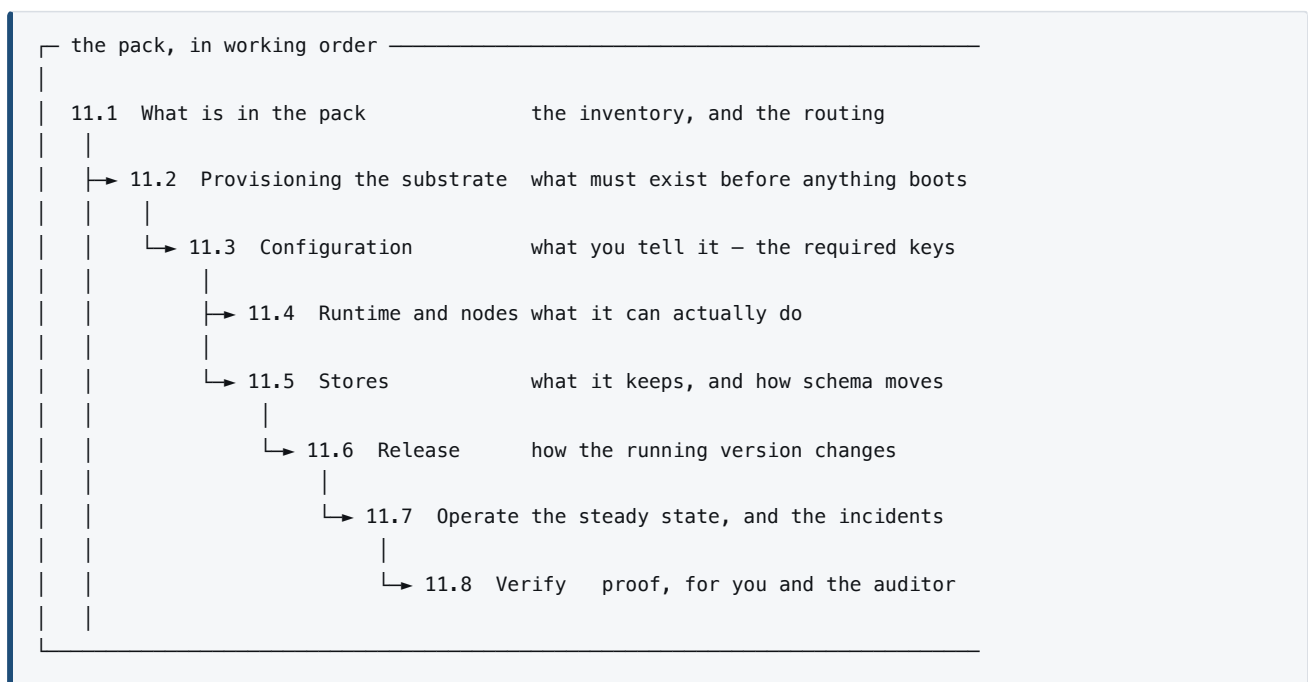
The chapters

#	Chapter	You leave able to
11.1	What is in the pack	Name every artefact you were given, say what each is for, and route a question to the right chapter
11.2	Provisioning the substrate	Stand up the cluster, namespace, quotas, storage, ingress and certificates a deployment needs before it boots
11.3	Configuration reference	Set every required key, know which are secrets, and recognise what a missing one looks like at runtime
11.4	The runtime and its nodes	Name the runtime package, reach every one of its node types, and derive the catalogue against your own install
11.5	Data stores and migrations	Size and tune the database and cache, apply schema changes in order, and take a backup you have restored

#	Chapter	You leave able to
11.6	Releasing and rolling back	Build and stamp an image, run the deploy operation with its preflights, and reverse it
11.7	Operating day to day	Read health, watch the right numbers, respect the capacity ceilings, and work the incident playbooks
11.8	Verifying the deployment	Prove the running deployment matches the repository, and that a partner can reach it

How the eight chapters relate

They are one deployment, described in the order you actually meet it: what you were given, what you build it on, what you tell it, what it runs, what it stores, how you change it, how you watch it, and how you prove it.



Read it as: each arrow is a **precondition**, not a call path. You cannot usefully set configuration before the namespace exists, and you cannot verify a release you have not applied. Four things follow from the shape:

- **11.1 is the map.** Every later chapter is one row of its inventory expanded.
- **11.3 is the chapter you will return to most.** A deployment that will not boot, or boots and refuses everything, is a configuration question far more often than it is anything else.
- **11.4 stands slightly apart.** It is the capability surface rather than the infrastructure, and it is what you hand a builder who asks "what can this actually do?"

- **11.8 assumes all of them.** A verification is only meaningful once you know what was supposed to be there.

Where this part sits relative to the others

Part 02 is how work runs through a governed organisation. Part 04 is the API surface an integration calls. Part 06 is the platform's architecture. Part 07 is the deployment's shape and the ownership line.

This part is the deployment's **operation**. Part 07 tells you a managed database holds the state and that backups are the operator's responsibility; this part tells you how to size it, how to migrate it, and how to prove the restore works.

The practical test is the question you are asking. *"What does a rollout do to in-flight requests?"* is part 07 — it is the shape. *"Which configuration key turns on asymmetric JWT, and what happens if I set only half the pair?"* is 11.3. *"Which node type do I use to call a webhook from a workflow?"* is 11.4.

Every cluster name, registry, namespace, storage account and hostname in this part is a placeholder — `<cluster-name>`, `<registry>`, `<app-namespace>`, `<your-deployment>` — because those belong to your deployment, not to the pack.

Next: 11.1 — What is in the pack

11.1 — What is in the pack

The deployment pack is everything you need to stand an Aegis deployment up, run it, change it, and prove it — as opposed to everything you need to *understand* one, which is part 07. It is a small set of artefacts with a strict division of labour, and most operator confusion comes from reaching for the wrong one: a manifest change where a configuration key was wanted, a rebuild where a re-reference would have done.

This chapter is the inventory and the routing. It does not tell you how to use any single artefact — each of the seven chapters that follow does exactly that for one row of the table below.

The one idea to carry out: the pack separates what you DECLARE from what you BUILD from what you RUN, and those three never touch the same file.

The three layers, and what lives in each

Layer	Artefacts	Changes when
Declare	Kubernetes manifests, overlays, quotas, network policy, ingress	The deployment's shape changes — replicas, limits, routes
Configure	A config map of non-secret settings, a secret set, a licence file	An environment-specific value changes — a URL, a key, a mode
Build	Two Dockerfiles, their pins, the build stamp, the provenance record	The code changes
Run	The deploy operation, its preflights, the drift verifier	You are releasing, rolling back, or checking

The split matters because the three have different blast radii and different reversal costs. A manifest change affects every environment that inherits the base. A configuration change affects one. A rebuild produces different bytes and must be re-verified. **Reaching for a rebuild when a configuration value was wrong is the most common wasted hour in this pack**, and the symptom is a release that ships identical code with a new digest, changes nothing, and leaves you no closer to the cause.

The inventory

```
| DECLARE – Kubernetes, inherited by every environment  
|   base/      namespace · backend · web · postgres · redis  
|               ingress · network-policy · resource-quota · monitoring  
|               backup CronJob · configmap · secrets  
|   overlays/  per-environment patches; the base is never edited  
|  
| CONFIGURE – values, not structure  
|   config map  non-secret settings, readable by anyone in the cluster  
|   secrets     credentials and keys, mounted rather than baked  
|   licence     a signed file, mounted from a secret, never in an image  
|  
| BUILD – two images, pinned  
|   api image   the backend: API on one port, agent listener on another  
|   web image   nginx serving a built single-page app, unprivileged  
|  
| RUN – operations, each with its own gate  
|   deploy      manual dispatch, takes a ref, refuses on preflight  
|   drift check compares the live cluster against the repository
```

Read it as: the four boxes are **ownership boundaries**, not directories. A change that crosses one is a change with a wider blast radius than it looks, and the crossing is where review should concentrate.

The four workloads, and the fifth thing

A running deployment is four pod-producing workloads in one namespace, plus a scheduled job. Their shapes are the subject of 11.2 and 11.5; the inventory is here so you can recognise what you are looking at in a pod list.

Workload	Kind	Replicas	What it is
backend	Deployment	3	Two listeners in one container — the API and an agent listener
web	Deployment	2	nginx serving a built SPA, unprivileged, read-only root
postgres	StatefulSet	1	In-cluster relational store with its own persistent volume
redis	Deployment	1	Cache and ephemeral state, append-only persistence
postgres-backup	CronJob	—	Hourly dump and upload, two containers, never overlapping runs

🚫 **The backend runs TWO listeners in ONE container, and that is load-bearing.** The API and the agent listener share a process tree. The entrypoint is PID 1, arms a termination forwarder **before** the first spawn, and exits the container when **either** listener exits. Get that ordering wrong and a shutdown signal reaches the wrapper but not the children: the pod lingers, the rollout stalls at its grace period, and the logs show a clean shutdown that never completed.

What is NOT in the pack, and where it is instead

Being precise about the absences saves more time than any of the inclusions.

Not here	Where it is
The platform source	Not distributed. You deploy images, not code.
The runtime's source	Not distributed. It arrives as a published package — 11.4
Your organisation's data	The database, which you operate but do not author
The governance model	Configured through the API, not through manifests — part 02
The integration's own credentials	Issued per organisation at the API, not provisioned in the cluster
Cloud account setup	Your cloud provider's problem, upstream of everything here

The row that catches people is the second. The runtime is a published package with a version constraint, and there is no repository of it to clone, patch or build. When a node behaves unexpectedly the remedy is a version change and a workflow change, never a source change — and a plan that assumes otherwise stalls at the point where someone goes looking for a file that was never shipped.

Routing a question to a chapter

You are asking	Chapter
"What has to exist before I can apply anything?"	11.2
"Which keys are required, and which are secrets?"	11.3
"What node types can a workflow use?"	11.4
"How big should the database be, and how do I migrate?"	11.5
"How do I ship this, and how do I take it back?"	11.6
"It is up — what do I watch?"	11.7
"How do I prove it is right?"	11.8
"Who owns this bit, me or the platform?"	07.1

The one call to make first

Before anything else, confirm you are talking to the deployment you think you are talking to, as the principal you expect. `api:GET /api/v1/auth/me` answers both at once, and a deployment pointed at the wrong estate is indistinguishable from a correct one until you ask.

```
# Orientation: which deployment, and as whom
curl -sS -H "Authorization: Bearer $AEGIS_TOKEN" \
  "$AEGIS_BASE_URL/api/v1/auth/me"          # the principal your credential resolves to
```

From the SDK the same check is one call, and `sdk:aegis_sdk.ClientConfig` is where the base URL and credential are held:

```
client = AgenticOSClient(ClientConfig(base_url=base_url, api_key=api_key))
me = await client.auth.get_me()
```

Run it before you debug anything else. A correct client pointed at a stale environment produces working calls, plausible data, and conclusions that do not apply to the deployment you are actually changing — and nothing in the response says so.

Next: 11.2 — Provisioning the substrate

11.2 — Provisioning the substrate

Everything in the rest of this part assumes a cluster that already has a namespace, quotas, storage classes, an ingress controller and a working certificate. This chapter is how those come to exist, in the order they have to exist in, and what each one is actually for.

07.2 describes what a namespace does and does not separate — read it for the boundary model. This chapter does not restate it; it tells you what to create.

The one idea to carry out: provisioning is ordered, and every dependency is silent when unmet. A missing storage class does not error at apply time — the claim simply stays pending, and a pod that never schedules looks identical to a pod that is slow to start.

The order, and why it is an order

1. cluster	node pools: one for platform components, one or more for workloads
2. namespace	the unit everything below is scoped to
3. storage class	must exist BEFORE any claim is made against it
4. resource quota	and a limit range, so unbounded pods get bounds
5. ingress controller	the thing that will terminate TLS and own the hostname
6. certificate	issued for the hostname, before the ingress references it
7. secrets + config	what the workloads read at start — chapter 11.3
8. workloads	only now does anything boot

Read it as: each step is a **precondition** for the one below, and each unmet precondition fails **quietly**. Steps 3 and 6 are the two that reliably catch people out, because a pending volume claim and an unresolved certificate both present as "the deployment did not come up" with nothing in the application's own logs.

The cluster and its node pools

Two pool kinds, at minimum:

- **A system pool** for the cluster's own components — ingress, controllers, the machinery that makes the cluster work. Tainted, so only components that explicitly tolerate the taint land there.
- **One or more workload pools** for the application workloads and any ephemeral job execution.

The separation exists so a workload that consumes its pool's capacity cannot take the cluster's own control components down with it. **When the split is absent the failure presents as the platform being broken rather than as one workload being greedy** — ingress stops answering, and the first thing anyone looks at is the application.

Size the workload pool against the resource quota below, not against the request sums: a quota that exceeds the pool's real capacity admits pods the scheduler then cannot place, and pending-for-capacity is a different problem with a different fix from pending-for-quota.

The namespace and its quota

One application namespace holds all four workloads and the backup job. Create it first; everything else is scoped to it.

The quota is a ceiling on the namespace's total consumption, and the limit range supplies bounds for containers that declare none:

Object	What it bounds	Reference shape
ResourceQuota	The namespace's total	Requests around 20 CPU / 40Gi; limits around 40 / 80Gi; a pod ceiling around 50
LimitRange	Any single container that declares none	A default request and limit per container, plus a maximum

Treat those numbers as the reference starting point and size them against your own workload profile.

The limit range is the half people skip, and the symptom is memorable: one container with no declared limit expands until it takes the node's memory, and every unrelated pod on that node is evicted — so the incident looks like a node failure rather than like one unbounded container.

△ **A quota and a limit are different mechanisms with different symptoms.** Hitting the quota refuses NEW workloads while existing ones keep running. Hitting a memory limit restarts ONE container. Hitting a CPU limit throttles it without restarting — slower responses, no restart, nothing in the logs. That third one is routinely mistaken for an application performance problem and profiled for days.

Storage classes and persistent volumes

Two claims are made in the reference shape, both inside the namespace:

Claim	Reference size	Access	Holds
postgres	50Gi	single	The relational store's data directory
redis	10Gi	single	Append-only persistence for the cache

Both leave the storage class **unset by default**, which means the cluster's default class is used. Set it explicitly for anything you care about performance on — the database in particular wants an SSD-backed class, and the default on many clusters is not one.

Sizing the database claim is a one-way door on some platforms. Expanding a volume is supported by many storage classes and by no means all of them, so a claim provisioned at the reference 50Gi on a non-expandable class becomes a migration rather than a resize. Check `allowVolumeExpansion` on the class you choose before you make the claim, not after the disk fills.

```
# Orientation: does the class you are about to use support expansion?
kubectl get storageclass                # the classes available, and which is default
kubectl get storageclass <class> -o jsonpath='{.allowVolumeExpansion}{"\n"}'
```

The database image must carry the vector extension

The knowledge store uses a **vector column** for similarity retrieval, so **the deployment requires a Postgres image with the vector extension available**, and you select that image when you provision the data store. A stock Postgres image does not carry it.

Two provisioning consequences, both decided before first boot:

- **Pick a vector-capable image**, not a stock one. The extension must be present in the image for the database to enable it.
- **The extension is enabled in the database**, once, at provisioning. It is a property of the database instance, not of the application.

Confirm it positively, as a provisioning step rather than a later diagnosis:

```
# Orientation: is the extension available to install, and is it installed?
kubectl -n <app-namespace> exec statefulset/<postgres> -- \
  psql -U "$PGUSER" -d "$PGDATABASE" -c "SELECT name FROM pg_available_extensions WHERE name='vector';"
kubectl -n <app-namespace> exec statefulset/<postgres> -- \
  psql -U "$PGUSER" -d "$PGDATABASE" -c "SELECT extname FROM pg_extension WHERE extname='vector';"
```

The first query answers *can this image provide it*; the second answers *is it enabled here*. **An empty first result is an image choice to correct, and an empty second is a one-line enablement** — two different remedies, which is why both queries are run rather than one.

11.5 covers what the extension costs you in storage and index maintenance once the corpus grows.

Ingress and certificates

The ingress is the deployment's single public door: it terminates TLS, owns the hostname, and routes to the web and backend services. 07.4 owns the boundary model; the provisioning facts are these.

- **The controller comes first.** An ingress resource with no controller watching it is accepted by the API server and does nothing. There is no error — the resource simply never acquires an address.
- **The certificate must exist before the ingress references it.** An ingress referencing an absent TLS secret serves the controller's default certificate, so clients get a name-mismatch error while the deployment reports healthy.

- **The hostname is yours.** It is a placeholder everywhere in this pack (`<your-deployment>`), and it appears in more than one place — the ingress, the allowed-origins configuration, and the front-end's own base URL. Setting it in one place and not the others produces a deployment that loads and then fails every request from the browser on a cross-origin check.

```
# Orientation: has the ingress actually acquired an address?
kubectl -n <app-namespace> get ingress          # ADDRESS empty means no controller took it
kubectl -n <app-namespace> get secret <tls-secret> # absent means the default cert is being served
```

Network policy — provision the cluster with an engine

The namespace ships a default-deny posture with explicit allowances. **Provision the cluster WITH a network-policy engine**, because a Kubernetes cluster only enforces NetworkPolicy when one is enabled — and admission is not enforcement.

This is a property of Kubernetes rather than of any one deployment, and it is the reason the requirement is a provisioning step rather than a review item:

	Cluster WITH an engine	Cluster WITHOUT one
The API server accepts policy objects	yes	yes
<code>kubectl get networkpolicy</code> lists them	yes	yes
A manifest review passes	yes	yes
Traffic is actually restricted	yes	no

Read the table down the right-hand column: **every readable signal is identical, and only the last row differs**. So the engine is selected when the cluster is created, or added to a running cluster with the provider's own update command — and the enforcement check in 11.8 is what proves it, rather than any amount of reading the objects.

The policy invariants — design rules the pack holds

Five properties every policy in the set satisfies. Hold them when you add one:

- **Every `podSelector` matches a real workload.** A selector matching nothing is a policy with no subject.
- **Every ingress port is a real `containerPort`.** A port nothing listens on admits nothing and protects nothing.
- **Egress-restricted policies permit DNS on both UDP and TCP 53.** Name resolution falls back to TCP for large responses, and a UDP-only allowance produces intermittent resolution failures under exactly the conditions that make them hardest to reproduce.
- **No bare wildcard egress peer.** An allowance wide enough to reach anything is the absence of an egress policy wearing one.

- **Ingress and egress are reciprocal.** If A may egress to B, B admits ingress from A. A one-sided pair is denied by the other half, and the symptom is a connection that times out rather than one that is refused.

🚫 **The backup job needs a policy allowance of its own.** It is a CronJob pod with its own labels, and a policy written around the four long-running workloads will not match it. The symptom is an hourly job that connects to nothing, fails, and — because failed job history is kept deliberately short in a cluster's default settings — leaves you a gap in the backup record rather than an alarm.

Pod security

The reference posture across every workload: non-root, no privilege escalation, all capabilities dropped. The web image re-adds exactly one — the capability to bind a privileged port — and runs with a **read-only root filesystem**, which requires writable scratch mounts for the cache, runtime and temporary directories.

Adding a read-only root to a container that was not built for it is the failure to expect, and it does not present as a permissions error. The process starts, tries to write its first temporary file, and dies in a way the application's own logging never reaches — so the pod crash-loops with an empty log.

Provisioning checklist

Run through this before applying any workload.

- ☐ The cluster has a system pool and at least one workload pool, and the system pool is tainted so tenant workloads cannot land on it.
- ☐ The namespace exists, and the quota and limit range are applied to it.
- ☐ The storage class you intend is named explicitly, and you have checked whether it allows volume expansion.
- ☐ The Postgres image carries the **vector extension**, and the extension is enabled in the database.
- ☐ An ingress controller is running and healthy.
- ☐ A certificate for your hostname is issued and its secret is present in the namespace.
- ☐ The cluster was provisioned **with a network-policy engine**, and enforcement is confirmed per 11.8 rather than by reading the policy objects.
- ☐ The backup job's labels are covered by a policy allowance.
- ☐ Secrets and configuration are present — 11.3 is the list, and no workload will start without them.

Next: 11.3 — Configuration reference

11.3 — Configuration reference

This is the chapter you will come back to. A deployment that will not boot, or boots and refuses everything, or boots and serves a blank page, is a configuration question far more often than it is a code, network or cluster question — and the three symptoms above map to three different missing keys.

Configuration arrives in two surfaces. **Non-secret settings** live in a config map: readable by anyone who can read the namespace, versioned with the manifests, safe in a diff. **Secrets** live in a secret set: mounted into the container at start, never baked into an image, never committed. The split is not cosmetic — anything in the first surface is visible to every workload in the namespace and to anyone reading a manifest.

The one idea to carry out: every key below has a failure mode, and almost none of them is an error message with the key's name in it.

Non-secret settings

These are the config-map keys. Values shown are the reference defaults; the ones marked **set per environment** have no sensible default and must be supplied.

Key	Reference default	What it actually does
ENVIRONMENT	production	Names the environment. Read by logging and by behaviour that differs outside production.
DEBUG	false	Verbose error surfaces. Never true outside a throwaway environment — it widens what an error response reveals.
LOG_LEVEL	info	Emitted log verbosity. <code>debug</code> on a busy deployment is a cost decision, not just a noise one.
JWT_ALGORITHM	HS256	Symmetric by default; RS256 switches to the asymmetric key pair below.
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	15	Access-token lifetime. Shorter means more refreshes, not less access.
JWT_REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh-token lifetime. This is how long a stolen refresh token is useful.
POSTGRES_DB	kaizen_studio	Database name. Must match what the connection string in the secret points at.
POSTGRES_PORT	5432	Database port.

Key	Reference default	What it actually does
REDIS_PORT	6379	Cache port.
AEGIS_DB_USE_APP_ROLE	false	Turns on connecting as a restricted application role rather than the owner.
AEGIS_DB_APP_ROLE	application_role	The role name used when the switch above is on. Inert while it is off.
AEGIS_ARTIFACT_STORAGE_PATH	/var/lib/aegis/artifacts	Where artifacts are written. Must be a writable mount, not the image's root filesystem.
CORS_ORIGINS	set per environment	Origins the browser is allowed to call from. Must include your front-end's exact scheme, host and port.
FRONTEND_URL	set per environment	The front-end's own address, used where the backend needs to construct a link back.
AZURE_TENANT_ID	common	Directory tenant for single sign-on. <code>common</code> accepts any directory; name yours to restrict it.
AEGIS_ALLOW_SELF_SERVE_ORG_ADMIN	false	Whether a new organisation may self-provision an administrator.
AEGIS_BOOTSTRAP_ADMIN_EMAILS	(empty)	Addresses granted administrator on first boot. Empty means nobody is bootstrapped.
AEGIS_GENESIS_SUPER_ADMIN_ENABLED	false	The first-boot super-administrator path. Off after the deployment is established.
AEGIS_ENTITLEMENT_ENFORCEMENT_MODE	off	Whether licence entitlements are enforced, observed, or ignored.

The three keys that produce the three classic symptoms

Symptom	The key
Front-end loads, every API call fails in the browser	<code>CORS_ORIGINS</code> does not exactly match the front-end origin
Deployment healthy, nobody can log in	<code>AEGIS_BOOTSTRAP_ADMIN_EMAILS</code> empty on a fresh database
Everything works, then stops after fifteen minutes	Refresh path misconfigured; the access token simply expired

`CORS_ORIGINS` is the one that wastes the most time, because the failure is entirely browser-side. The backend logs a successful request. The front-end shows a network error with no status. Nothing in either log names the origin check, and the scheme and port are part of the match — `https://host`

and `https://host:443` are different origins to a browser.

Secrets

Mounted from the secret set, never baked into an image, never committed. Every one of these is required unless the row says otherwise.

Secret key	Environment name	Purpose
<code>database-url</code>	<code>DATABASE_URL</code>	The connection string the backend uses.
<code>app-database-url</code>	<code>AEGIS_APP_DATABASE_URL</code>	The restricted-role connection string. Needed when the app-role switch is on.
<code>postgres-user</code>	—	The database role the in-cluster Postgres is created with.
<code>postgres-password</code>	—	Its password. Also consumed by the backup job.
<code>postgres-db</code>	—	The database created at first start.
<code>redis-url</code>	<code>REDIS_URL</code>	Cache connection string, including credentials.
<code>redis-password</code>	—	The cache's own password.
<code>jwt-secret</code>	<code>JWT_SECRET_KEY</code>	Symmetric signing key. Used when the algorithm is <code>HS256</code> .
—	<code>JWT_SECRET_KEY_PREVIOUS</code>	The outgoing symmetric key during a rotation. Optional.
—	<code>JWT_PRIVATE_KEY</code> / <code>JWT_PUBLIC_KEY</code>	The asymmetric pair. Required when the algorithm is <code>RS256</code> .
—	<code>JWT_PUBLIC_KEY_PREVIOUS</code>	The outgoing public key during an asymmetric rotation. Optional.
<code>encryption-key</code>	<code>AEGIS_ENCRYPTION_KEY</code>	Application-level field encryption.
<code>encryption-master-key</code>	<code>ENCRYPTION_MASTER_KEY</code>	The master key the above derives from.
<code>credential-encryption-key</code>	<code>CREDENTIAL_ENCRYPTION_KEY</code>	Protects stored third-party credentials.
<code>cursor-encryption-key</code>	<code>CURSOR_ENCRYPTION_KEY</code>	Protects opaque pagination cursors against tampering.
<code>hash-pepper</code>	<code>HASH_PEPPER</code>	Pepper for password hashing.
<code>pii-hash-salt</code>	<code>AEGIS_PII_HASH_SALT</code>	Salt for pseudonymising personal data in derived records.

Secret key	Environment name	Purpose
azure-client-id / -secret	AZURE_CLIENT_ID / AZURE_CLIENT_SECRET	Workload credentials for cloud resources.
azure-sso-client-id / -secret	AZURE_SSO_CLIENT_ID / AZURE_SSO_CLIENT_SECRET	Single sign-on application registration.
license-json	via LICENSE_FILE_PATH	The signed licence, mounted as a file. Never baked into an image.

Generate the real values before the first apply

The secret template ships placeholders, and you generate the real values with the provided generator before first apply. The generator produces every key in the table above with the right shape and writes the secret in one step:

```
# The one command that populates every generated key
bash <deployment-pack>/scripts/generate-secrets.sh
```

It requires `openssl`. Run it once, before the first apply.

The recipes, if you are generating a single key by hand. They are not interchangeable — three keys need a Fernet-shaped value and the rest are plain hex, and a hex string handed to a Fernet consumer is rejected at first use.

Key	Recipe
AEGIS_ENCRYPTION_KEY , CREDENTIAL_ENCRYPTION_KEY , CURSOR_ENCRYPTION_KEY	Fernet — 32 url-safe base64 bytes, a 44-character key
ENCRYPTION_MASTER_KEY , HASH_PEPPER , AEGIS_PII_HASH_SALT	<code>openssl rand -hex 32</code>
JWT_SECRET_KEY	<code>openssl rand -hex 32</code> — at least 32 characters
postgres-password , redis-password	<code>openssl rand -hex 32</code>
postgres-user	A name plus a random suffix
The single sign-on client secret	Supplied by your identity provider, not generated

 A Fernet key is not `openssl rand -hex 32`. The consumer requires 32 url-safe base64-encoded bytes and rejects anything else by length. Generate it as base64 and translate `+/` to `-_`, or let the generator do it — a hex value in one of those three slots fails at first use, not at apply.

Two one-way doors — decide them at provisioning

Two keys cannot be changed later, so they are provisioning decisions rather than operational ones.

- **The PII hash salt is set at genesis or never.** It pseudonymises personal data in derived records, and there is no re-encryption path — changing it later leaves every existing derived record unmatchable against every new one. Set it when you provision, and back it up with the deployment.
- **The master key is generated, never left at its template value.** It is the root the other derived keys hang from. Generate it at provisioning and treat it as the deployment's most valuable secret.

Both of these are decided once. **Deciding them by not deciding them is the failure mode** — the placeholder is a valid string, so a deployment that never ran the generator starts, serves, and stores data under a value that is neither secret nor yours.

The rotation keys are pairs, and half a pair is worse than none

`JWT_SECRET_KEY_PREVIOUS` and `JWT_PUBLIC_KEY_PREVIOUS` exist so a signing key can be rotated without invalidating every live session: the new key signs, the previous key still verifies, and the overlap is drained before the old key is removed.

🚫 **Removing the previous key before its tokens have expired logs everyone out at once**, and the symptom is a support incident rather than an error: the deployment is healthy, the logs are clean, and every user is simultaneously asked to sign in again. Drain for at least the refresh-token lifetime — the reference `7` days — before dropping the outgoing key.

The encryption keys are not interchangeable, and losing one is not recoverable

Four distinct keys protect four distinct things. **They are not a group you can rotate together casually**, and a value encrypted under a key you no longer have is gone — there is no recovery path, because that is the property the key exists to provide.

Key	What becomes unreadable if you lose it
<code>ENCRYPTION_MASTER_KEY</code>	Everything derived from it
<code>AEGIS_ENCRYPTION_KEY</code>	Encrypted application fields
<code>CREDENTIAL_ENCRYPTION_KEY</code>	Every stored third-party credential
<code>CURSOR_ENCRYPTION_KEY</code>	Nothing durable — cursors are ephemeral

The last row is the only safe one to rotate without ceremony: an invalidated cursor means a client's next page request fails and the client starts over.

The licence is mounted, never baked

The licence is a signed file. The **public key that verifies it** is baked into the image as a trust anchor; the **licence itself** is not, and is mounted from a secret at `LICENSE_FILE_PATH`.

That split is what lets one image serve every deployment. **Baking a licence into an image ties that image to one deployment silently** — it still runs anywhere, and it reports the wrong entitlements in every environment it was not built for, with no error at any point.

Confirming configuration actually landed

A config map or secret is read at container **start**. Editing one does not change a running pod, and this is the single most common false conclusion in this chapter: the value is correct in the cluster and stale in the process.

```
# Orientation: what the cluster holds, and whether the pods have picked it up
kubectl -n <app-namespace> get configmap <name> -o yaml    # what is declared
kubectl -n <app-namespace> get secret <name> -o jsonpath='{.data}' | tr ',' '\n' | cut -d'"' -f2
#   ^ key NAMES only – never decode values into a terminal or a transcript
kubectl -n <app-namespace> rollout restart deployment/<backend>  # what makes it take effect
```

From the API, two operations report the configuration as the deployment understands it, which is the reading that matters:

```
org = await client.settings.get_organization()
posture = await client.compliance.get_dashboard()
```

`api:GET /api/v1/settings/organization` is the organisation-level view, and `api:GET /api/v1/compliance/dashboard` reports the governance posture the deployment is actually enforcing rather than the one the manifests declare.

⚠ **Never decode a secret's value into a terminal, a log or a transcript.** Read the key NAMES to confirm presence. A decoded value lives in scrollback, in shell history, and in whatever captured the session — and rotating a key because it was pasted into a chat is an avoidable afternoon.

Configuration checklist

- ☐ `CORS_ORIGINS` and `FRONTEND_URL` both name your exact front-end origin, scheme and port included.
- ☐ `DEBUG` is `false` and `ENVIRONMENT` names the right environment.
- ☐ The generator has been run, so no key still holds its template placeholder. Confirm by key name, never by decoding a value.
- ☐ The three Fernet-shaped keys are Fernet-shaped, not hex.
- ☐ The PII hash salt and the master key were generated at provisioning — both are one-way doors.
- ☐ Every secret in the table above is present. Confirm by key name.
- ☐ If `JWT_ALGORITHM` is `RS256`, both halves of the key pair are set.
- ☐ If `AEGIS_DB_USE_APP_ROLE` is `true`, `AEGIS_APP_DATABASE_URL` is set and the role exists in the database.
- ☐ The licence is mounted from a secret, and no image contains one.
- ☐ `AEGIS_BOOTSTRAP_ADMIN_EMAILS` names someone, on a fresh database.
- ☐ The four encryption keys are backed up somewhere that survives the cluster, and you know which of them is unrecoverable.

Next: 11.4 — The runtime and its nodes

11.4 — The runtime and its nodes

Aegis executes work as **workflows**, and a workflow is a graph of **nodes**. Every node is a typed unit the runtime knows how to run: call an endpoint, query a database, invoke a model, branch, merge, encrypt, alert. The set of node types the runtime recognises is the deployment's capability surface — it is the honest answer to "what can this actually do?", and it is the reach the rest of this part exists to keep running.

The runtime arrives as a **published package**, not as source. There is nothing to clone and nothing to build: you install a version, the version determines the catalogue, and the catalogue is introspectable from the package itself. That last property is the important one, because it means you never have to trust a list — including this one.

The one idea to carry out: the node catalogue is DERIVED from your own install, and the derivation is two lines.

The runtime package and its version constraint

Fact	Value
Package	kailash-enterprise
Version constraint	<code>>=4.47.0,<4.48.0</code>
Import name	kailash
Distribution shape	A single compiled extension, <code>abi3</code> from Python 3.10
Interpreters	Wheels for CPython 3.10 through 3.14
Typing	Ships <code>py.typed</code> with stub files, so a type checker sees it

Pin to a minor range rather than a floating latest. **A runtime version change is a capability change**, and it is the one dependency in this pack whose upgrade can alter what an existing workflow does rather than merely what it can do — which is why the upper bound is part of the pin and not decoration.

The package carries sub-namespaces for the surfaces built on it. Fifteen import with the base install — `dataflow`, `nexus`, `kaizen`, `mcp`, `a2a`, `pact`, `enterprise`, `events`, `infra`, `delegate`, `trust_plane`, `nodes`, `workflow`, `runtime` and `distill`. The node catalogue below is reached from the top level.

One does not: `ml` requires the `ml` extra. Install it only if you need that surface:

```
pip install 'kailash-enterprise[ml]'
```

Without the extra, importing `kailash.ml` raises rather than resolving, and the error names the install command — so treat it as a dependency you choose, not as a namespace that is missing.

Derive the catalogue yourself — the method

Everything in this chapter came from this, and you should re-run it against your own install rather than reading the table below as standing fact:

```
# The total, and the full list, from the installed distribution
python -c "from kailash import NodeRegistry; t=sorted(NodeRegistry().list_types()); print(len(t));
print('\n'.join(t))"
```

It needs no database, no configuration and no network. `NodeRegistry()` constructs a registry populated with every built-in type; `list_types()` returns their names exactly as you write them in a workflow.

```
# Does this install recognise a specific type? (the check to run before you rely on one)
python -c "from kailash import NodeRegistry; print(NodeRegistry().has_type('HTTPRequestNode'))"
```

The registry exposes exactly three public methods — `pkg:kailash.NodeRegistry.list_types`, `pkg:kailash.NodeRegistry.has_type` and `pkg:kailash.NodeRegistry.register_callback`. There is no metadata accessor: you can enumerate node-type **strings** from the package, and you cannot introspect descriptions, categories or parameter schemas from it.

That is why the catalogue below is worth having — **it supplies what the registry cannot**. The counts and the names are yours to re-derive; the grouping and the descriptions are the part the package does not carry.

Derived on version 4.47: 138 built-in node types. That number is the denominator for everything below, and the categories that follow sum to exactly it. If your install reports a different total, your install is the authority.

The catalogue, by category

Twenty categories, disjoint, summing to 138.

Category	Count
Core workflow and control flow	10
Code execution	4
Data transform	12
Files and documents	7
HTTP and API	7
Databases and SQL	4
Vector and retrieval	9
AI and multimodal	8
Cache	3

Category	Count
Redis	3
Kafka and streaming	6
Authentication and identity	9
Authorization and governance	7
Security and data protection	13
Monitoring and diagnostics	13
Alerts	6
Distributed transactions	5
Edge and distributed placement	6
Platform and infrastructure	4
MCP and tool integration	2
Total	138

Core workflow and control flow

Type	What it is for
<code>ConditionalNode</code>	Branch on a predicate
<code>SwitchNode</code>	Multi-way branch on a value
<code>MergeNode</code>	Join several branches back into one
<code>LoopNode</code>	Iterate over a collection
<code>ParallelNode</code>	Fan out independent work
<code>WaitNode</code>	Pause for a duration or a condition
<code>RetryNode</code>	Re-attempt a failing step under a policy
<code>ErrorHandlerNode</code>	Catch and route a failure
<code>NoOpNode</code>	Do nothing — a placeholder or a join point
<code>BatchProcessorNode</code>	Process a collection in bounded batches

Code execution

Type	What it is for
<code>EmbeddedPythonNode</code>	Run inline Python
<code>EmbeddedJSNode</code>	Run inline JavaScript
<code>SubprocessNode</code>	Run an external process

Type	What it is for
CodeValidationNode	Check code before it executes

Data transform

Type	What it is for
DataMapperNode	Reshape a record between two schemas
JSONTransformNode	Transform JSON structurally
TextTransformNode	Transform free text
StringOperationsNode	String manipulation primitives
MathOperationsNode	Arithmetic and numeric primitives
ArrayOperationsNode	Collection manipulation primitives
FormatConverterNode	Convert between serialisation formats
FilterNode	Drop records failing a predicate
SchemaValidatorNode	Reject records that do not match a schema
TextSplitterNode	Chunk text for embedding or processing
XMLParserNode	Parse XML into structured data
CSVProcessorNode	Read and write delimited data

Files and documents

Type	What it is for
FileReaderNode	Read a file
FileWriterNode	Write a file
ExcelReaderNode	Read a spreadsheet
PDFReaderNode	Extract content from a PDF
DocumentLoaderNode	Load a document into the processing pipeline
DocumentStoreNode	Persist a document
DirectoryReaderNode	Enumerate a directory

HTTP and API

Type	What it is for
HTTPRequestNode	Make one HTTP request
HTTPBatchNode	Make many, with concurrency control

Type	What it is for
GraphQLNode	Issue a GraphQL query or mutation
RateLimitedAPINode	Call an API under a rate-limit policy
WebhookNode	Receive or emit a webhook
WebSocketNode	Hold a websocket connection
WebScrapingNode	Fetch and extract from a web page

Databases and SQL

Type	What it is for
SQLQueryNode	Execute a query
SQLTransactionNode	Group statements into one transaction
DatabaseConnectionNode	Establish and hold a connection
GraphDatabaseNode	Query a graph store

Vector and retrieval

Type	What it is for
VectorSearchNode	Similarity search
VectorQueryNode	Query a vector collection
VectorUpsertNode	Insert or update vectors
VectorDeleteNode	Remove vectors
EmbeddingNode	Produce embeddings
EmbeddingStoreNode	Persist embeddings
RerankNode	Re-order candidates by relevance
RAGPipelineNode	Retrieve-then-generate, as one step
ContextAssemblerNode	Assemble retrieved context for a model

AI and multimodal

Type	What it is for
LLMNode	Call a language model
SummarizationNode	Summarise text
SentimentNode	Score sentiment
ClassificationNode	Classify into labels

Type	What it is for
VisionNode	Interpret an image
AudioNode	Process audio
TextToSpeechNode	Synthesise speech
ImageGenerationNode	Generate an image

Cache

Type	What it is for
CacheGetNode	Read from cache
CacheSetNode	Write to cache
CacheInvalidateNode	Evict an entry or a pattern

Redis

Type	What it is for
RedisCommandNode	Issue a Redis command
RedisPubSubNode	Publish or subscribe
RedisStreamNode	Read or write a Redis stream

Kafka and streaming

Type	What it is for
KafkaProducerNode	Produce to a topic
KafkaConsumerNode	Consume from a topic
KafkaStreamNode	Stream-process a topic
StreamProducerNode	Produce to a generic stream
StreamConsumerNode	Consume from a generic stream
StreamTransformNode	Transform a stream in flight

⚠ **TokenGenerationNode** resolves to the authentication implementation. Two distinct implementations carry that name and the registry is first-wins, so the auth one is what a workflow gets. Read it as the auth node wherever it appears, and name the category you mean when you write about it.

Authentication and identity

Type	What it is for
JWTAuthNode	Issue or verify a JWT
OAuth2Node	Drive an OAuth2 exchange
APIKeyAuthNode	Authenticate by API key
SSONode	Single sign-on integration
MFANode	Second-factor challenge
AuthFlowNode	Compose a multi-step authentication flow
SessionManagerNode	Create, read and end sessions
TokenGenerationNode	Mint a token
DirectoryIntegrationNode	Read from a directory service

Authorization and governance

Type	What it is for
ABACPermissionEvaluatorNode	Evaluate attribute-based access
UserManagementNode	Create and modify users
TenantAssignmentNode	Place a principal in a tenant
ComplianceCheckNode	Assert a compliance control
GDPRComplianceNode	Data-subject rights operations
DataRetentionPolicyNode	Apply a retention rule
DataLineageNode	Record where data came from

Security and data protection

Type	What it is for
EncryptionNode	Encrypt or decrypt
HashingNode	Hash a value
CertificateNode	Handle a certificate
SignatureVerificationNode	Verify a signature
CredentialValidatorNode	Validate a credential's shape and state
SecretRotationNode	Rotate a secret
DataMaskingNode	Mask sensitive fields
InputSanitizationNode	Sanitise untrusted input

Type	What it is for
<code>OutputEncodingNode</code>	Encode output for a sink
<code>ThreatDetectionNode</code>	Detect a known threat pattern
<code>SecurityEventNode</code>	Emit a security event
<code>BehaviorAnalysisNode</code>	Score behaviour against a baseline
<code>EnterpriseAuditLoggerNode</code>	Write an audit record

Monitoring and diagnostics

Type	What it is for
<code>HealthCheckNode</code>	Probe a dependency
<code>MetricsCollectorNode</code>	Gather metrics
<code>LogNode</code>	Emit a log line
<code>LogProcessorNode</code>	Process logs in a pipeline
<code>PerformanceAnomalyNode</code>	Detect a performance outlier
<code>PerformanceBenchmarkNode</code>	Measure a benchmark
<code>DeadlockDetectorNode</code>	Detect a lock cycle
<code>RaceConditionDetectorNode</code>	Detect a concurrency hazard
<code>TransactionMonitorNode</code>	Watch transaction behaviour
<code>ConnectionDashboardNode</code>	Report connection-pool state
<code>ResourceAnalyzerNode</code>	Analyse resource consumption
<code>ResourceOptimizerNode</code>	Recommend a resource change
<code>ResourceScalerNode</code>	Apply a scaling decision

Alerts

Type	What it is for
<code>AlertNode</code>	Emit a generic alert
<code>EmailAlertNode</code>	Alert by email
<code>SlackAlertNode</code>	Alert to Slack
<code>TeamsAlertNode</code>	Alert to Teams
<code>DiscordAlertNode</code>	Alert to Discord
<code>PagerDutyAlertNode</code>	Page an on-call responder

Distributed transactions

Type	What it is for
<code>DistributedTransactionManagerNode</code>	Choose and drive a distributed-transaction strategy
<code>TwoPhaseCommitCoordinatorNode</code>	Two-phase commit, for strong consistency
<code>SagaCoordinatorNode</code>	Saga orchestration, for availability
<code>SagaStepNode</code>	One saga step and its compensation
<code>TransactionContextNode</code>	Carry transaction context across steps

Edge and distributed placement

Type	What it is for
<code>EdgeDataNode</code>	Read or write at an edge location
<code>EdgeStateMachineNode</code>	Run a state machine at the edge
<code>EdgeCoordinationNode</code>	Coordinate across edge locations
<code>EdgeMigrationNode</code>	Move work between locations
<code>EdgeMonitoringNode</code>	Observe an edge location
<code>EdgeWarmingNode</code>	Pre-warm an edge location

Platform and infrastructure

Type	What it is for
<code>KubernetesNode</code>	Interact with a cluster
<code>DockerNode</code>	Interact with a container runtime
<code>CloudNode</code>	Call a cloud provider surface
<code>PlatformNode</code>	Platform-level operations

MCP and tool integration

Type	What it is for
<code>MCPServiceDiscoveryNode</code>	Discover available tool servers
<code>EnterpriseMCPExecutorNode</code>	Execute a tool through a governed path

Model-generated nodes — eleven per registered model

The 138 above are built in. **Registering a data model generates eleven more, for that model alone**, so the live registry is larger than the built-in catalogue and grows with your schema.

The naming rule is `<Verb><ModelName>` — verb first, model name exactly as registered. Derived by registering one model named `Invoice` against a clean registry and diffing the type list:

Verb	Generated type	Operation
Create	CreateInvoice	Insert one row
Read	ReadInvoice	Fetch one row by key
Update	UpdateInvoice	Modify one row
Delete	DeleteInvoice	Remove one row
List	ListInvoice	Fetch many, filtered and paged
Count	CountInvoice	Count matching rows
Upsert	UpsertInvoice	Insert or update one row
BulkCreate	BulkCreateInvoice	Insert many
BulkUpdate	BulkUpdateInvoice	Modify many
BulkDelete	BulkDeleteInvoice	Remove many
BulkUpsert	BulkUpsertInvoice	Insert or update many

Derive it for your own models the same way — register, then diff:

```
# Before-and-after around your own registration; the difference is the generated set
python -c "from kailash import NodeRegistry; print(len(NodeRegistry().list_types()))"
```

 **The verb comes FIRST, and there is no `Node` suffix.** A workflow naming `InvoiceReadNode` or `ReadInvoiceNode` fails at build time with an unknown-node-type error, not at run time — which is the good outcome, but the message names the type you asked for rather than the type you meant, so it reads as a missing capability rather than a spelling.

Registration is all-or-nothing, and pre-checked. The eleven names are validated against the registry before any of them installs, so a collision with an existing type leaves the registry **byte-unchanged** rather than half-wired.

That is the property to design around: a model whose name collides gives you a clean refusal and a registry you can still reason about, instead of a partial set where six verbs work and five do not. **A half-registered model would be the expensive failure** — the workflows naming the installed verbs would run, and the ones naming the rest would fail at build, from what looks like the same change.

The execution contract

Four objects, in this order. Verified against the installed distribution.

Step	Object	Call
1	<code>pkg:kailash.NodeRegistry</code>	<code>NodeRegistry()</code> — populated with every built-in type
2	<code>WorkflowBuilder</code>	<code>add_node(type_name, node_id, config)</code> , then <code>connect(src, src_out, tgt, tgt_in)</code>
3	<code>build</code>	<code>builder.build(registry)</code> → a <code>Workflow</code>
4	<code>pkg:kailash.Runtime</code>	<code>Runtime(registry, config)</code> , then <code>execute(workflow, inputs)</code>

`execute` returns a dict with three keys — `results`, `run_id` and `metadata` — and `execute_async` is the awaitable form of the same call. `RuntimeConfig` carries the tuning: `debug`, `max_concurrent_nodes`, `enable_cycles`, `node_timeout`, `workflow_timeout` and `conditional_execution`.

All four resolve at the **top level** of `kailash`, and that import line is the whole published entry point:

```
from kailash import WorkflowBuilder, NodeRegistry, Runtime, RuntimeConfig
```

🚫 **There is no registry factory and no runtime factory in the package.** You construct `NodeRegistry()` and `Runtime(...)` directly, or wrap them in a factory of your own. An import of a helper that is not in the line above raises `ImportError` at import time — which is the good outcome, but it names the symbol you asked for rather than the one you needed, so it reads as a missing feature rather than as the wrong entry point.

```
registry = NodeRegistry()
builder = WorkflowBuilder()
builder.add_node("HTTPRequestNode", "fetch", {"url": endpoint, "method": "GET"})
builder.add_node("SchemaValidatorNode", "check", {"schema": invoice_schema})
builder.connect("fetch", "body", "check", "record")
workflow = builder.build(registry)

runtime = Runtime(registry, RuntimeConfig(max_concurrent_nodes=4))
outcome = await runtime.execute_async(workflow, inputs={"tenant": tenant_id})
```

`build()` is where an unknown node type is caught, and it is the earliest point at which a type in a type name becomes visible. A workflow assembled but never built carries the error silently — so build eagerly, at assembly time, rather than at the point of first execution.

Custom types are registered before any Runtime shares the registry

`register_callback` adds a type of your own. **Register every custom type BEFORE constructing a Runtime against that registry.**

🚫 **Registering late fails at USE, not at registration.** The registration call itself succeeds whenever you make it, and a Runtime that already holds the registry does not see the addition — so every workflow naming your custom type fails with `unknown node type`, and the registration that would explain it reported success. Register at start-up, in one place, before anything is constructed against the registry.

Registered is not runnable — each category has its own preconditions

`list_types()` reports that a node type is **registered**. It does not report that the node can run: most categories reach something outside the process, and that dependency is configured separately.

Category	Needs configured before the node runs
AI/multimodal, vector and retrieval	A model provider credential and endpoint
Kafka and streaming	A broker address and topic permissions
Redis, Cache	A cache URL and credential
Databases and SQL, Vector	A connection string and a reachable database
HTTP/API, alerts	A reachable destination, and egress permitted
Platform and infrastructure	Credentials for the platform being addressed
Auth, authorization, security	The relevant keys — 11.3
Control flow, transform, code	Nothing beyond the workflow itself

The last row is the only one that is self-contained, and it is why a workflow of pure control flow and transforms runs in an environment where everything else fails. A green run over that subset is not evidence the rest of the catalogue is reachable.

What a deployment change does to the catalogue

Change	Effect on the catalogue
Runtime version upgrade	Types may be added; behaviour of existing types may change
Registering a new data model	Eleven new types appear, for that model
Renaming a data model	Eleven types disappear and eleven appear — every workflow naming the old set breaks at build
Configuration change	None. The catalogue is a property of the package and the registered models

The rename row is the one to plan around. It is a build-time break across every workflow that referenced the model, it surfaces all at once, and the remedy is a coordinated edit rather than a compatibility shim — the generated types are derived from the name and there is no alias.

Next: 11.5 — Data stores and migrations

11.5 — Data stores and migrations

Two stores, and they fail in opposite directions. The relational database holds everything that must survive — the organisation, its roles and envelopes, its governance record — and losing it is unrecoverable. The cache holds nothing durable, and losing it costs latency. Almost every operational decision about these two follows from which side of that line a thing is on.

07.5 describes what lives where and why. This chapter is how you size them, tune them, move schema through them, and take a backup you have actually restored.

The one idea to carry out: a backup nobody has restored is a hypothesis, and the only instrument that settles it is a restore that produces a usable database.

The database

A single-replica StatefulSet with its own persistent volume in the reference shape, or a managed instance outside the cluster. Either way the properties that matter to you are the same.

Property	Reference
Kind	StatefulSet, 1 replica
Volume claim	50Gi, single-writer
Storage class	Unset by default — name an SSD-backed class explicitly
Vector extension	Required. Embeddings live here, not in a separate service

The image requirement — a vector-capable Postgres

Because embeddings live in this database, **the image you run must carry the vector extension**, and you choose that image when you provision the data store. 11.2 has the selection step and the two confirmation queries.

It is an **image** decision rather than a configuration one, and that is why it belongs at provisioning: a stock Postgres image cannot be made vector-capable by a setting, so meeting the requirement after the store is populated turns a one-line image choice into a migration.

Sizing, and the growth that is not a leak

Three things drive the size, and only one of them is what people plan for.

Driver	Growth shape
Organisational records	Grows with your organisation; roughly bounded
Audit and governance	Monotonic. Retained on purpose, never pruned

Driver	Growth shape
Embeddings	Grows with the knowledge corpus, and indexes with it

Storage only grows, and that is the intended behaviour. A capacity plan built on a steady state is wrong in the only direction that matters, and the symptom arrives as a full volume rather than as a slow degradation: writes begin failing, the application surfaces them as generic errors, and the database is the last thing anyone checks because it was healthy an hour ago.

The vector extension adds a second cost most plans miss: **index maintenance is the most expensive routine database operation here**, and it scales with the corpus. Adding knowledge is not free at the storage layer, and it is not free at the CPU layer either.

Connections are the ceiling nobody sizes for

Every backend replica holds a pool. Replicas times pool size is the demand; the database's maximum connections is the supply, and the supply is usually much smaller than people assume.

More replicas is not unconditionally more throughput, and past a point it is less. The symptom is distinctive and easily misread: requests begin timing out while the database's CPU is idle, because the work is queued waiting for a connection rather than waiting for the database. Scaling the deployment up makes it worse.

```
# Orientation: the supply, and what is currently drawn against it
kubectl -n <app-namespace> exec statefulset/<postgres> -- \
  psql -U "$PGUSER" -d "$PGDATABASE" -c "SHOW max_connections;"
kubectl -n <app-namespace> exec statefulset/<postgres> -- \
  psql -U "$PGUSER" -d "$PGDATABASE" -c "SELECT count(*) FROM pg_stat_activity;"
```

Raise the database's ceiling **before** raising the replica count, not after. Reversing the order produces an outage during the scale-up.

The cache

A single-replica Deployment with append-only persistence and a `Recreate` update strategy — deliberately, because two replicas backed by one volume is a corruption, not a redundancy.

Property	Reference
Kind	Deployment, 1 replica
Strategy	<code>Recreate</code> — the old pod exits before the new starts
Persistence	Append-only
Volume claim	10Gi

`Recreate` means a cache restart is a brief outage of the cache, not a rolling handover. That is correct and it is worth knowing before you watch it happen: during the gap, reads fall through to the database and latency rises sharply, then recovers as the cache refills.

⚠ **Nothing authoritative belongs in the cache, and the way that rule gets broken is by accident.** A rate-limit counter, a lock, or a session that is not also persisted turns the cache from an optimisation into a dependency — and the failure only shows up the first time the cache restarts, which may be months after the change that caused it.

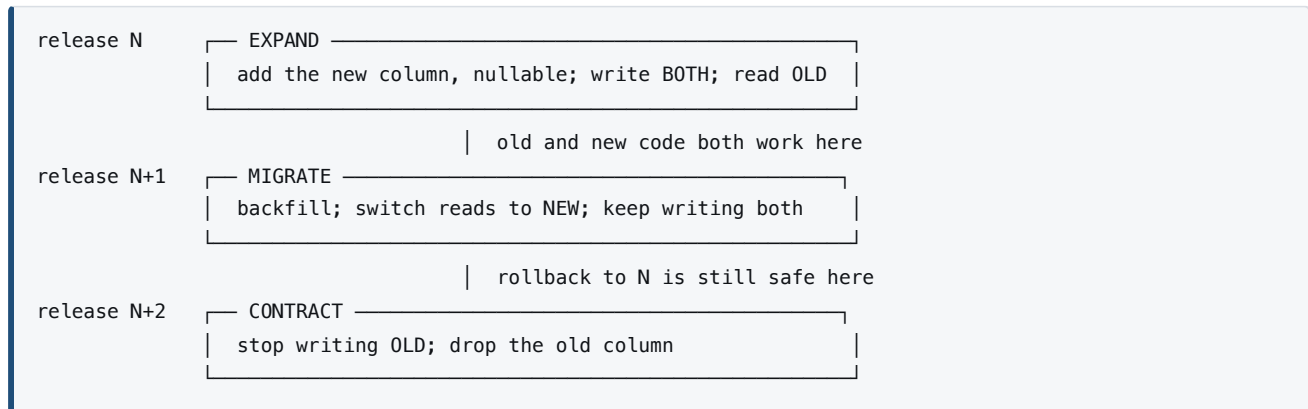
Schema migrations

Migrations are applied **as part of the release**, before the new version begins serving. Three properties govern how you plan them.

- **Ordered.** Migrations apply in sequence and the sequence is not negotiable. Applying them out of order, or applying one twice, is what a migration ledger exists to prevent.
- **Forward-only in practice.** A down-migration that drops a column drops the data in it. Plan reversals as new forward migrations.
- **They gate the rollout.** The rollout uses `maxUnavailable: 0`, so a failed migration halts the change with the previous version still serving — which is the right outcome, and it means a stuck rollout is often a migration question.

The expand-migrate-contract sequence

A schema change that both old and new code must tolerate cannot be one step.



Read it as: three releases, and **the rollback window closes at the third**. Between N and N+2 you can go back; after CONTRACT you cannot, because the column the previous version reads no longer exists.

Collapsing this into one release is the migration failure to expect, and it does not fail at migration time. The migration succeeds. The rollout proceeds. The old replicas — still serving during the surge — begin erroring against a column that has gone, and the incident reads as a code fault in a version that worked ten minutes ago.

Backup and restore

The reference shape backs up hourly on a CronJob, with two containers: one takes the dump, one uploads it.

Property	Reference
Schedule	Hourly, offset from the top of the hour
Concurrency	<code>Forbid</code> — a slow dump must not stack
Successful-run history	1
Failed-run history	3 — failures are kept visible, successes are not
Service-account token	Not mounted — the job has no cluster API identity
Upload credential	Its own secret, separate from the database's

The history asymmetry is deliberate and worth understanding: keeping failures and discarding successes means `kubectl get jobs` shows you problems rather than noise. **A job list that looks empty is the healthy state**, which is the opposite of most people's instinct and causes the occasional false alarm.



concurrencyPolicy: Forbid means a dump that overruns its hour **SKIPS** the next run rather than **stacking**. That is the safe behaviour, and it produces a silent gap in the backup record — the job list shows no failure, because no run was attempted. A database that has grown past its backup window degrades the backup frequency without ever alarming.

The restore drill

A configured backup and a working backup are different facts, and only one of them is observable.

- ☐ A dump exists, from the last hour, with a size in the range you expect.
- ☐ It has been **restored into a real database**, not merely downloaded.
- ☐ The restored database serves a read the application recognises.
- ☐ You know how long the restore took, because that number is your recovery time and a number nobody has measured is not one.
- ☐ The restore landed somewhere you could validate **before** it replaced anything.

From the API, the backup surface is visible and drivable — which is the cheapest way to confirm the record from outside the cluster:

```
backups = await client.settings.list_backups()
created = await client.settings.create_backup()
```

`api:GET /api/v1/settings/backups` lists what exists, `api:POST /api/v1/settings/backups` takes one on demand, and `api:POST /api/v1/settings/backups/restore/{id}` drives a restore.

Store checklist

- ☐ The database's storage class is named explicitly and is SSD-backed.
- ☐ You have checked whether that class allows volume expansion.

- ☐ `max_connections` exceeds replicas × pool size, with headroom.
 - ☐ The cache holds nothing authoritative — confirm, do not assume.
 - ☐ Migrations follow expand-migrate-contract wherever a rollback window matters.
 - ☐ The hourly dump has completed within the last hour, and its size is sane.
 - ☐ A restore has been **performed** within recent memory, and you know how long it took.
-

Next: 11.6 — Releasing and rolling back

11.6 — Releasing and rolling back

A release here is four things in a fixed order: build two images from one commit, stamp them so the running system can say what it is, apply them behind preflights that refuse rather than proceed, and keep the previous digest so the reversal is a re-reference rather than a rebuild.

07.3 makes the case for digests over tags and owns that argument. This chapter is the operation: what the build hardens, what the stamp is for, what the deploy refuses, and what a rollback actually does.

The one idea to carry out: the deploy operation is manual, takes a commit, and every preflight in it exists because something once passed without it.

Two images, built from one commit

Image	Contents	Runs as
<code><registry>/<api></code>	The backend — API listener plus agent listener	Non-root, capabilities dropped
<code><registry>/<web></code>	nginx serving a built single-page app	Non-root, read-only root, one capability re-added for binding

Both pin their base image to an exact patch version. Neither installs a toolchain that survives into the runtime layer.

What the build hardens, and why each step is there

- **Toolchain tarballs are verified against digests pinned in the Dockerfile itself**, with length validators on the hex. A pinned digest that is silently truncated verifies nothing while looking rigorous, so the length check is the part that makes the pin real.
- **Vendored dependency floors are re-pinned with independent digests**, and the build **fails** rather than continuing when one does not match. An `else` branch that warns and proceeds is the same as no check at all.
- **The front-end module tree is deleted from the runtime image** after the build, while the guards that produced it remain. The artefact ships; the machinery that made it does not.
- **The build toolchain and its installers are purged and asserted gone**. The assertion matters more than the purge: a removal that silently fails leaves a compiler in a production image and nothing says so.
- **The licence signature's public key is baked in as a trust anchor**. The licence itself never is — it is mounted from a secret (11.3).

The lockfile is what makes the image deterministic — not the ranges

This is the most useful sentence in the build section, so it is stated on its own: **the dependency ranges are not what pins the image. The lockfile is.**

Of the 33 runtime dependencies, **7 carry an upper bound and the other 26 are deliberately unbounded**. A reader who audits the ranges and concludes the build is reproducible has audited the wrong artefact — the ranges express compatibility intent, and the lockfile expresses what actually installs.

The install path is what enforces it:

1. `uv export --frozen` resolves **from the lockfile**, never from the range set, and emits a hash-pinned requirements file.
2. The build **asserts that export actually contains the runtime pin**, and fails fatally if it does not.
3. `pip install --no-deps --require-hashes` installs exactly that, refusing any unpinned or unhashed line.
4. The resolver tool is then **removed, and its absence verified**.

To change a pinned version, edit the constraint AND re-resolve the lockfile. Editing the manifest alone fails the build — the frozen export refuses to re-resolve, so the assertion in step 2 catches the mismatch rather than silently installing the old version. That refusal is the design: a manifest edit that appeared to succeed while installing something else is the outcome `--frozen` exists to prevent.

The production interpreter is the image's pin

The image pins **Python 3.12.9 exactly**. The project manifest declares only a `>=3.12` floor, and a CI workflow pins its own interpreter for tooling.

🚫 **Do not infer the production interpreter from a CI workflow.** The three numbers serve three different purposes — the floor says what the code tolerates, the CI pin says what a checking job runs on, and **only the image pin says what production executes**. Reading the CI pin as production's is a conclusion nothing in the logs will contradict, because both interpreters run the same code successfully right up until one of them does not.

🚫 **The single most consequential build mistake is baking the licence.** The image still runs everywhere. It simply reports the wrong entitlements in every deployment it was not built for, with no error at any point — so the first symptom is a capability behaving differently in two environments that are supposedly running the same version.

Stamping — one tag, both images, asserted back

One value — the commit — drives the build argument and both image tags. Three rules make the stamp trustworthy:

1. **The commit argument is REQUIRED, with no silent default.** A default would produce a build that succeeds and is unattributable.
2. **A dirty tree or a shallow clone is REFUSED.** Both produce a stamp that names a commit whose content is not what was built.

3. **Each image generates its stamp INSIDE the build, after the source copy, and asserts it back.** Generating it outside lets a host file shadow it; the assertion is what catches a stamp that did not take.

The result is two surfaces you can read at runtime: a **version operation on the backend**, and a **static version file the web image serves**. Both should name the same commit, and 11.8 is where that gets checked.

Two images stamped with different commits is the failure this design exists to catch, and without the stamp it is invisible: the deployment works, the front end and the back end are from different releases, and the mismatch presents as an intermittent contract error weeks later.

Provenance — what it proves, and what it does not

A signed build-provenance record binds the image digest to the build stamp. It is emitted on **release events only** — never on a proposal — and it **fails closed** without its signing key rather than emitting an unsigned record.

⚠ **The attested image and a later rebuild agree on INPUTS, not on bytes.** A digest difference between a rebuild and the attestation is EXPECTED, and it is not evidence of tampering. Use an attested digest to verify **that** artifact; never to predict what another build of the same commit will produce. Reading a legitimate digest difference as a supply-chain incident is the misreading to expect, and it is expensive because it is alarming.

A vulnerability scan runs daily and on changes to dependency manifests, at high and critical severity. It is **advisory** — it reports, it does not block — so treat its output as a queue to work rather than as a gate that has already been satisfied.

The deploy operation

Manually dispatched. There is no push, merge or schedule that triggers it, and that separation is deliberate: merging is a code decision and deploying is an operational one.

Input	Meaning
<code>ref</code>	The commit to deploy. A full or abbreviated SHA, not a branch name
<code>confirm</code>	The target namespace, typed out
<code>deliver</code>	The commit or commits this deploy MUST land, checked against the ref AND the running cluster

The preflights, and what each one catches

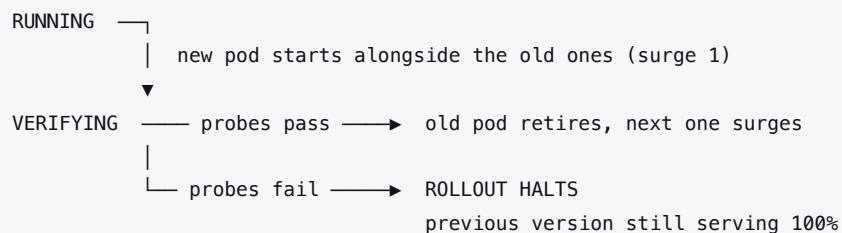
Preflight	Refuses when
Confirmation gate	The typed namespace does not match the target. Without it, any string deploys
Ref identity fence	<code>HEAD</code> does not resolve to the requested ref — verified by effect, not by trust
Delivery check	The ref does not contain the commits the deploy claims to deliver
Tooling and credentials	A required tool is missing, or no cluster context is selected, each named individually
Network policy enforcement	Rendered policy would not actually be enforced

The delivery check is the one that saves you from the most embarrassing failure: deploying a ref that builds, applies and comes up healthy, while not containing the fix everyone believes has just shipped. Nothing about a healthy rollout distinguishes that case, which is why the check compares against the running cluster as well as against the ref.

```
# Orientation: what the cluster is running right now, before you change it
kubectl -n <app-namespace> get deploy -o wide           # the image references in force
kubectl -n <app-namespace> rollout status deploy/<backend> --timeout=5m
```

The rollout, and why it cannot half-finish

Both Deployments roll with `maxSurge: 1` and `maxUnavailable: 0`.



Read it as: `maxUnavailable: 0` is what makes a bad release a non-event. A pod that will not boot halts the rollout with the previous version serving every request. Nothing is lost, and the deployment sits in a mixed state until someone acts.

The consequence to internalise: a **halted rollout is not an outage, and treating it as one causes the outage**. The instinct under pressure is to force the new version through by scaling down the old; that trades a safe halt for a real interruption. Read the failing pod's logs first.

Probes are single slots, not lists

Each container has **one** liveness probe, **one** readiness probe and **one** startup probe. Each is a single object, not a list.

➖ Adding a second `LivenessProbe:` key does not add a probe — it REPLACES the first. The manifest is valid, the apply succeeds, and the check you thought you had added has silently removed the check you already had. There is no warning at any layer. If you need two conditions, they go inside one probe's target, not into two keys.

Reference shape for the backend: liveness after 30s, every 10s, 3 failures to act; readiness after 10s, every 5s; startup after 10s, every 5s, 12 failures — a 60-second budget for a slow start.

The startup probe is what stops a slow boot being read as a failed one. Without it, a database migration that takes 40 seconds trips liveness, the container is killed, and the pod crash-loops through a migration that would have succeeded.

Explicit lifespan handling is load-bearing

The permissive default swallows a startup exception and leaves a pod **Ready while every request fails**. That is the worst available outcome: the rollout completes, the previous version retires, and the deployment reports healthy while serving nothing but errors. Explicit lifespan handling turns that into a failed start, which `maxUnavailable: 0` then converts into a halted rollout.

The worker health-check timeout is raised for a related reason. The default kills a worker whose interpreter is busy across a model round-trip — a normal, long, entirely healthy operation. **A hard kill runs no handler**, so a dispatched task row is left in progress forever, and the symptom is a queue that accumulates work nobody is processing and nothing has marked failed.

Rolling back

Reversal is re-referencing the previous digest. The image still exists, it is the exact bytes that ran before, and the operation is minutes with no build.

Step	Action
1	Identify the previous digest. Do not reconstruct it from a tag
2	Confirm the schema is compatible — 11.5
3	Dispatch the deploy operation with the previous commit as <code>ref</code>
4	Watch the rollout; <code>maxUnavailable: 0</code> protects you if the reversal is bad too
5	Verify — 11.8

Step 2 is the step that is skipped, and it is the one that makes a rollback dangerous rather than routine. If the release you are reversing included a CONTRACT migration, the previous version reads a column that no longer exists: the rollback applies cleanly, the pods come up, and every request touching that table fails. The expand-migrate-contract sequence exists precisely so this window is known rather than discovered.

A rollback you have never rehearsed is a hypothesis. Knowing the previous digest exists is not the same as having deployed it.

Next: 11.7 — Operating day to day

11.7 — Operating day to day

Most of running a deployment is not releasing it. It is knowing which numbers mean something, which ceilings you are approaching, and what to do in the first five minutes of an incident — and the hardest part is that the deployment's own health signals are narrower than they look.

This chapter is the steady state and the playbooks. Capacity limits that follow from the architecture rather than from your usage live in 07.7; what follows here is what you act on.

The one idea to carry out: a green health check means a process is answering, and nothing more — every other question needs a different instrument.

Health and readiness, and what each actually asserts

Three probes, three different questions, and conflating them is how a broken deployment stays in a rotation.

Probe	Asks	Failing it causes
Startup	Has it finished booting yet?	The other probes stay suspended
Liveness	Is the process still alive?	The container is restarted
Readiness	Can it serve traffic <i>right now</i> ?	The pod is removed from the service , not restarted

All three target the same health path in the reference shape. That is fine, and it has one consequence worth holding: **a dependency failure that the health path does not consult is invisible to all three.** The pod stays Ready, the service keeps routing to it, and every request that needs that dependency fails.

At the API level there are governance-specific health operations, and they answer a narrower question than the pod probes:

```
trust = await client.trust.get_health()
compliance = await client.compliance.get_health()
```

`api:GET /api/v1/trust/health` and `api:GET /api/v1/compliance/health` report that the governance services respond. **They do not report that governance is configured as you intended, that your envelopes are the ones you think, or that any control is enforced.** Treating a green there as evidence for those is how a misconfiguration survives a monitoring system that was watching the wrong thing.

What to watch

Six things, in the order they will hurt you.

Watch	Where	Acts when
Database volume headroom	Cluster metrics	Below ~25% — it only grows
Connection-pool saturation	<code>api:GET /api/v1/pools/{id}/metrics</code>	Waits appear while CPU is idle
Error rate	<code>api:GET /api/v1/metrics/errors</code>	Any sustained rise, especially post-release
Execution throughput	<code>api:GET /api/v1/metrics/executions</code>	Drops without a load change
Backup recency	<code>api:GET /api/v1/settings/backups</code>	Most recent is older than an hour
Spend	<code>api:GET /api/v1/analytics/costs</code>	Any step change — model calls are the driver

Two aggregate views are worth a dashboard:

```
summary = await client.metrics.get_summary()
overview = await client.analytics.get_overview()
```

`api:GET /api/v1/metrics/summary` is the operational roll-up and `api:GET /api/v1/analytics/overview` the work-level one; `api:GET /api/v1/metrics/timeseries` is where you go once you know which number moved. For service-level attainment, `api:GET /api/v1/analytics/sla` is the reporting surface.

In the console: the same figures appear on the metrics and analytics dashboards, and `api:GET /api/v1/metrics/dashboard` is the operation behind the operational one.

Logs and records — two kinds, not interchangeable

	Audit evidence	Operational telemetry
Answers	Who did what, and was it permitted	Is it healthy, and is it fast
Produced by	The governance layer, deliberately	The runtime, as a by-product
Retention	Long, set by policy	Short, set by cost
Completeness	Complete for its scope	Sampled, dropped, rotated
If you lose it	You cannot evidence your governance	You lose situational awareness

Evidence is a product artefact; telemetry is an operational one. A retention obligation is satisfied by the first column, never by the second — and the mistake is easy to make because both are visible and one is far easier to query.

```
entries = await client.audit.list_logs()
export = await client.audit.export()
```

`api:GET /api/v1/audit/logs` reads the governance record and `api:GET /api/v1/audit/export` extracts it. `api:GET /api/v1/settings/audit-log` is the settings-change record specifically — who changed the deployment's own configuration, which is the log you will want during an incident whose cause turns out to be a change.

Container logs are the telemetry half, and the backend's two listeners share one stream:

```
# Orientation: recent backend output, both listeners interleaved
kubectl -n <app-namespace> logs deploy/<backend> --since=15m --tail=200
kubectl -n <app-namespace> logs deploy/<backend> --previous      # the container that just died
```

`--previous` is the flag people forget under pressure. After a crash-loop restart the current container's log is empty and reassuring; the one that explains the crash belongs to the container that has already gone.

Capacity — what scales and what does not

Scales with demand	Does not scale
Backend and web replicas	Database write throughput — it scales up , not out
Node pool size	Vector index build cost
Cache capacity	Retention — evidence grows and is kept
Job execution concurrency	Storage growth, which is monotonic

Three ceilings are structural, and worth designing around rather than arguing with.

- **Database write throughput** is bounded by the instance size, and changing it is a planned operation rather than an automatic one.
- **Vector index maintenance** scales with the knowledge corpus and is the most expensive routine database operation.
- **Connection counts** are bounded by the database. Every replica consumes some, so **more replicas is not unconditionally more throughput, and past a point it is less** — see 11.5 for the arithmetic and the order of operations.

Incident playbooks

Five shapes, with the distinguishing signal first because that is what you have at minute one.

Pods are Ready and every request fails

Distinguishing signal: health green, error rate at or near 100%, logs show requests arriving.

The permissive-lifespan failure: a startup exception was swallowed and the pod reports Ready while nothing works. Check the backend's start-up log lines for an exception that did not prevent the process continuing. **Roll back rather than restart** — a restart reproduces the same start.

A rollout has stopped part-way

Distinguishing signal: old and new pods both present, `rollout status` hanging, previous version still serving.

This is `maxUnavailable: 0` working. **It is not an outage.** Read the new pod's logs — most often a migration, a missing configuration key, or a probe budget too short for a slow start. Do not scale the old version down to force the new one through; that converts a safe halt into a real interruption.

Latency rose, nothing changed

Distinguishing signal: CPU idle, error rate flat, response times up.

Two candidates in order. **CPU throttling** — a container at its CPU limit is slowed rather than killed and logs nothing, which is why this is routinely profiled as an application problem for days. **Connection-pool waits** — the work is queued for a connection rather than for the database. Check `api:GET /api/v1/pools/{id}/metrics` before profiling anything.

The backup record has a gap, with no failed job

Distinguishing signal: job list clean, most recent dump older than an hour.

`concurrencyPolicy: Forbid` skipped a run because the previous dump was still going. No run was attempted, so nothing failed and nothing alarmed. The database has outgrown its backup window — fix the window or the dump, not the alert.

A capability works in one environment and not another

Distinguishing signal: identical image reference, different behaviour.

Three candidates, in cost order: a configuration key differing between the two environments (11.3); a licence baked into an image rather than mounted (11.6); or a runtime version difference changing the node catalogue (11.4). Compare configuration first — it is the cheapest to check and the most often the answer.

The first five minutes

```
# 1. Is it up, and which version?
kubectl -n <app-namespace> get pods -o wide
kubectl -n <app-namespace> get deploy -o wide           # image references in force

# 2. What died, and what did it say before it died?
kubectl -n <app-namespace> logs deploy/<backend> --previous --tail=100

# 3. Is this a change? (the settings-change record, not the container log)
```

Then, from the API, in this order: `api:GET /api/v1/auth/me` to confirm you are looking at the deployment you think you are; `api:GET /api/v1/metrics/errors` for shape and onset; `api:GET /api/v1/settings/audit-log` for a recent configuration change.

Step one of step three is the one that resolves incidents fastest, because a large share of them are a change somebody made and nobody correlated.

Next: 11.8 — Verifying the deployment

11.8 — Verifying the deployment

A deployment that is up is not the same as a deployment that is correct, and the gap between them is where most of the expensive surprises live. This chapter is how you close it: prove the cluster matches the repository, prove the running version is the one you meant, prove a partner can actually reach what you built, and prove the documentation you ship resolves.

Each check below answers one question and **only** that question. The temptation — and it is the failure mode this whole chapter exists to block — is to run the cheapest one and let it stand for the others.

The one idea to carry out: name what each check would say if the answer were NO, before you cite it as evidence that the answer is yes.

The four checks, and what each is blind to

Check	Answers	Does not answer
Drift	Does the live cluster match the repository?	Whether the repository is right
Policy enforcement	Is NetworkPolicy actually enforced?	Whether the policies are right
Version surfaces	Is the running code the commit you deployed?	Whether that commit behaves
Reachability	Can a partner call what you built?	Whether the answers are correct
Anchor check	Does every claim in the shipped handbook resolve?	Whether the prose is true

Read the right-hand column as the reason you run all five. **A green drift check on a wrong manifest is a correctly-applied mistake**, and it is indistinguishable from a correctly-applied intent by every instrument in this chapter — so each check's answer is worth exactly the question it was asked.

Drift verification

Drift is the live cluster diverging from the repository — a hand-applied fix, a scaled replica count nobody committed, an image reference edited in place during an incident. It accumulates silently and surfaces at the next deploy, when something that had been working is overwritten by the declared state.

The verifier runs on a daily schedule and on demand, and it has **three** outcomes, not two:

Outcome	Meaning
MATCHES	Live state equals the repository
DRIFTED	A named difference exists
UNDETERMINED	The check could not reach the cluster, or could not resolve a value

 **UNDETERMINED is a failure, not a pass.** It means the check did not look — and "could not look" and "looked and found nothing" produce the same silence from anything that treats the outcome as a boolean. The scheduled run fails on DRIFTED **and** on UNDETERMINED, deliberately. A verifier that reports clean when it could not reach the cluster is worse than no verifier, because it retires the question.

The check runs against the live cluster on its own schedule, never on the pull-request path. That placement is deliberate: **production staleness is not a property of the change under review**, and a gate that reds unrelated proposals is muted within a week.

```
# Orientation: the difference between declared and live, per workload
kubectl -n <app-namespace> get deploy -o wide           # image references in force
kubectl -n <app-namespace> get deploy/<backend> -o jsonpath='{.spec.replicas}{"\n"}'
```

Policy enforcement — prove the engine is live

11.2 requires the cluster to be provisioned with a network-policy engine. This is the check that proves it, and it is the only one that can: every other signal — the policy objects, the listing, the manifest review — reads identically with and without an engine.

The deploy operation runs this as a preflight and **refuses to apply policies to a cluster that has no engine to enforce them**, so a deploy that reaches the apply step has already answered this. Run it independently when you want the answer without a deploy:

```
# Orientation: is an enforcing engine present on this cluster?
kubectl get nodes -o jsonpath='{.items[0].metadata.labels}' | tr ',' '\n' | grep -i policy
kubectl -n kube-system get pods           # the engine runs here when one is installed
```

Three outcomes, and the third is not a pass:

Outcome	Meaning
ENFORCING	An engine is present; the policies bind
NOT ENFORCING	Policies are admitted and inert — fix the cluster
INDETERMINATE	The check could not read the cluster

An engine can also be installed on a **running** cluster with the provider's update command, so NOT ENFORCING is a remediation rather than a rebuild.

Version surfaces

Two surfaces report what is running: a **version operation on the backend**, and a **static version file the web image serves**. Both derive from the stamp generated inside each build (11.6).

Three states, and the third is the one worth looking for:

Reading	Means
Both name the commit you deployed	Correct
Either names something else	The deploy did not land what you believe it landed
They name DIFFERENT commits from each other	A split release — front end and back end from different builds

The **split** is the **dangerous one** because nothing about it is unhealthy. Both pods are Ready, both surfaces answer, the deployment reports green — and the contract between the two halves is whatever the older one assumed. It surfaces later as an intermittent field mismatch that reads like a code defect.

Check them together, never one alone.

Reachability — can a partner actually call this?

An operator's verification and an integrator's experience are different measurements, and the operator's is the one that passes while the partner is blocked. The cheapest partner-shaped probe is the SDK itself, pointed at the deployment from outside the cluster.

```
client = AgenticOSClient(ClientConfig(base_url=base_url, api_key=api_key))
me = await client.auth.get_me()
orgs = await client.auth.list_my_organizations()
```

`api:GET /api/v1/auth/me` confirms the deployment and the principal; `api:GET /api/v1/auth/me/organizations` confirms the organisations that credential resolves to — **and no others**, which is the half people skip. `sdk:aegis_sdk.__version__` reports the client's own version, which is what you quote when asking whether a partner's client is compatible.

Four probes, in increasing depth:

Probe	Establishes
<code>api:GET /api/v1/auth/me</code>	The edge answers, and the credential resolves
<code>api:GET /api/v1/auth/me/organizations</code>	The credential's scope is what you intended
<code>api:GET /api/v1/knowledge/search</code>	The database and its vector extension are both working
One operation you expect to be REFUSED	Governance is enforcing, not merely present

 **The fourth is the only one that tests the product's actual job.** A permission model tested only in the permissive direction is untested — every call succeeding is exactly what a deployment with governance switched off also produces. Attempt something your credential should not be allowed to do, and confirm the refusal.

Governance integrity

Two operations verify the governance record itself rather than the services that write it:

```
integrity = await client.compliance.get_chain_integrity()
verified = await client.compliance.verify_audit()
```

`api:GET /api/v1/compliance/chain/integrity` checks the audit chain's internal consistency; `api:GET /api/v1/compliance/audit/verify` verifies the entries. Together they are what an auditor is shown, and they answer a question no health check touches: **not "is the service up" but "is the record intact"**.

`api:GET /api/v1/compliance/dashboard` is the posture roll-up, and `api:GET /api/v1/trust/metrics` reports the trust surface's own numbers.

The handbook anchor check

The handbook you ship makes claims about operations and symbols. The anchor check resolves every one of them against the derived operation set and the installed package, and it has exactly two outcomes — `OK`, or `FAIL` with a count.

```
# Capture, then read. Never pipe into a filter and read the filter's status
python -m aegis_sdk.handbook.check > /tmp/anchors.txt 2>&1; EXIT=$?
cat /tmp/anchors.txt; echo "EXIT=$EXIT"
```

Capturing before reading is not fussiness. A pipeline's exit status is its last stage's, so `... | tail` reports `tail`'s success and a failing check reads as a pass. That trap costs a release rather than a minute, because the green is banked and nobody re-runs it.

The verification sequence

Run these in order. Each one's failure makes the next one's result meaningless.

- ☐ **Drift** reports MATCHES — and UNDETERMINED was treated as a failure.
- ☐ **Policy enforcement** reports ENFORCING, from the cluster rather than from the policy objects.
- ☐ **Both version surfaces** name the commit you deployed, and name the same one as each other.
- ☐ **Reachability**: the edge answers, the principal is right, the organisation list is exactly what you expect.
- ☐ **A refusal** was observed for an operation that should be refused.
- ☐ **Governance integrity** and audit verification both pass.
- ☐ **The backup record** shows a dump within the last hour, and a restore has been performed within recent memory — 11.5.
- ☐ **The anchor check** exits `OK`, read from a captured file.

What to establish before you depend on a deployment

Three things this chapter cannot measure, which you settle by asking rather than by probing — and each has a right answer and a recognisable wrong one.

Question	A good answer names
When was a restore last performed , and how long did it take?	A date and a duration
What is the retention policy for audit evidence?	A period, and who can delete within it
Who is on call, and by what route are they reached?	A rota and an escalation path, in writing

One item that is easy to skip and should not be: confirm the person answering is able to answer. "I will find out" is a fine answer. A confident answer about a deployment the respondent has not inspected is not — and this part has named enough silent failure modes that the difference between the two is worth the extra question.

This is the last chapter of part 11, and the end of the handbook. For what the platform can do once it is running, go to Part 08 — The capability catalogue; for the governance model your deployment now enforces, go to Part 09 — The governance architecture.